目录

第 1 章 Linux 快速入门.....1

1.1 为什么要学习 Linux.....1

1.2 Linux 操作系统简介.....3

1.3 Linux 操作系统优点.....4

1.4 Linux 操作系统发行版.....4

1.5 32 位与 64 位操作系统的区别.....7

1.6 Linux 内核命名规则.....8

第 2 章 Linux 发展及系统安装.....10

2.1 Linux 发展前景及就业形势.....10

2.2 Windows 操作系统简介.....11

2.3 硬盘分区简介.....13

2.4 Linux 安装环境准备.....15

2.5 Linux 系统安装图解.....25

2.6 菜鸟学好 Linux 大绝招.....34

2.7 本章小结.....36

2.8 同步作业.....36

第 3 章 CentOS 系统管理.....37

3.1 操作系统启动概念.....37

3.1.1 BIOS 37
3.1.2 MBR 38
3.1.3 GPT 39
3.1.4 GRUB 40
3.2 Linux 操作系统启动流程 41
3.3 CentOS6 与 CentOS7 区别 46
3.4 TCP/IP 协议概述 49
3.5 IP 地址及网络常识 50
3.5.1 IP 地址分类 51
3.5.2 子网掩码 54
3.5.3 网关地址 55
3.5.4 MAC 地址 55
3.6 Linux 系统配置 IP 56
3.7 Linux 系统配置 DNS 58
3.8 Linux 网卡名称命名 59
3.9 CentOS7 密码重置 61
3.10 远程管理 Linux 服务器 64
3.11 Linux 系统目录功能 67
第 4 章 Linux 必备命令 69

4.1 Linux 命令集 69
4.2 cd 命令详解 71
4.3 ls 命令详解 71
4.4 pwd 命令详解 73
4.5 mkdir 命令详解 73
4.6 **rm 命令详解** 74
4.7 **cp 命令详解** 75
4.8 **mv 命令详解** 77
4.9 **touch 命令详解** 78
4.10 **cat 命令详解** 79
4.11 **head 命令详解** 81
4.12 **tail 命令详解** 82
4.13 **echo 命令详解** 83
4.14 **df 命令详解** 85
4.15 mount 命令详解 86
4.16 本章小结 93
4.17 同步作业 94
第 5 章 Linux 用户及权限管理 94
5.1 Linux 用户及组 95

5.2 Linux 用户管理.....96
5.3 Linux 组管理.....98
5.4 Linux 用户及组案例.....100
5.5 Linux 权限管理.....103
5.6 Chown 属主及属组.....105
5.7 Chmod 用户及组权限.....105
5.8 Chmod 二进制权限.....106
5.9 Linux 特殊权限及掩码.....108
5.10 本章小结.....110
5.11 同步作业.....110
第 6 章 Linux 软件包企业实战.....111
6.1 RPM 软件包管理.....112
6.2 Tar 软件包管理.....115
6.2.1 Tar 命令参数详解.....115
6.2.2 TAR 企业案例演示.....117
6.2.3 TAR 实现 Linux 操作系统备份.....118
6.2.4 Shell+TAR 实现增量备份.....121
6.3 ZIP 软件包管理.....125
6.4 源码包软件安装.....128

6.5 YUM 软件包管理 129
6.5.1 YUM 工作原理 130
6.5.2 YUM 企业案例演练 131
6.6 YUM 优先级配置实战 135
6.7 基于 ISO 镜像构建 YUM 本地源 137
6.8 基于 HTTP 构建 YUM 网络源 140
6.9 YUM 源端软件包扩展 142
6.10 同步外网 YUM 源 144
6.11 本章小结 146
6.12 同步作业 146
第 7 章 Linux 磁盘管理 147
7.1 计算机硬盘简介 147
7.2 硬盘 Block 及 Inode 详解 149
7.3 硬链接介绍 151
7.4 软链接介绍 152
7.5 Linux 下磁盘实战操作命令 153
7.6 基于 GPT 格式磁盘分区 158
7.7 MOUNT 命令工具 161
7.7.1 Mount 命令参数详解 162

7.7.2 企业常用 Mount 案例 163
7.8 Linux 硬盘故障修复 164
7.9 本章小结 167
7.10 同步作业 167
第 8 章 Linux 文件服务器企业实战 168
8.1 进程与线程概念及区别 169
8.2 Vsftpd 服务器企业实战 171
8.2.1 FTP 传输模式 171
8.2.2 Vsftpd 服务器简介 173
8.2.3 Vsftpd 服务器安装配置 173
8.2.4 Vsftpd 匿名用户配置 176
8.2.5 Vsftpd 系统用户配置 178
8.2.6 Vsftpd 虚拟用户配置 181
第 9 章 Apache WEB 服务器企业实战 186
9.1 Apache WEB 服务器入门简介 187
9.2 Prefork MPM 工作原理 188
9.3 Worker MPM 工作原理 189
9.4 Apache WEB 服务器安装 190
9.5 Apache 虚拟主机企业应用 193

9.6 Apache 常用目录学习 197
9.7 Apache 配置文件详解 198
9.8 Apache Rewrite 规则实战 201
第 10 章 MYSQL 服务器企业实战 206
10.1 MYSQL 数据库入门简介 207
10.2 Mysql 数据库安装方式 210
10.3 Mysql 数据库必备命令操作 215
10.4 Mysql 数据库字符集设置 215
10.5 Mysql 数据库密码管理 215
10.6 MySQL 数据库索引案例 215
10.7 MySQL 数据库集群实战 215
10.8 MySQL 主从复制实战 215
10.9 MySQL 主从同步排错思路 215
第 11 章 LAMP 架构企业实战 215
11.1 LAMP 企业架构简介 215
11.2 Apache 与 PHP 工作原理 215
11.3 LAMP 企业安装配置 215
11.4 LAMP 企业架构拓展实战 215
11.5 LAMP+Redis 企业实战 215

11.5.1 Redis 入门简介 215
11.5.2 LAMP+Redis 工作机制 215
11.5.3 LAMP+Redis 操作案例 215
11.6 Redis 配置文件详解 215
11.7 Redis 常用配置 215
11.8 Redis 集群主从实战 215
11.9 Redis 数据备份与恢复 215
11.9.1 Redis 主从复制备份 215
11.10 LAMP 企业架构读写分离 215
第 12 章 CentOS7 实战 Kickstart 批量 系统部署 215
12.1 Kickstart 使用背景介绍 215
12.2 Kickstart 企业实战配置 215
12.3 TFTP+PXE 配置 215
12.4 配置 TFTPBOOT 引导案例 215
12.5 HTTPD+KICKSTART 配置 215
12.6 DHCP 服务配置演练 215
12.7 开启新虚拟机，BIOS 以网卡启动 215
12.8 Kickstart 企业生产环境扩展 215
第 13 章 Zabbix 分布式监控企业实战 215

13.1 Zabbix 监控系统入门简介.....215
13.2 Zabbix 监控组件及流程.....215
13.3 Zabbix 监控平台概念.....215
13.4 Zabbix 监控平台部署.....215
13.5 Zabbix 配置文件优化实战.....215
13.6 Zabbix 自动发现及注册.....215
13.7 Zabbix 监控邮件报警实战.....215
13.8 Zabbix 监控 MYSQL 主从实战.....215
13.9 Zabbix 日常问题汇总.....215
13.10 Zabbix 触发命令及脚本.....215
13.11 Zabbix 分布式监控实战.....215
13.12 Zabbix 监控微信报警实战.....215
第 14 章 Nginx WEB 服务器企业实战.....215
14.1 Nginx WEB 入门简介.....215
14.2 Nginx 工作原理.....215
14.3 Nginx 安装配置.....215
14.4 Nginx 管理及升级.....215
14.5 Nginx 虚拟主机实战.....215
14.6 Nginx Location 深入剖析.....215

14.7 企业实战 Nginx 动静分离架构 215
14.8 企业实战 LNMP 高性能服务器 215
14.9 Nginx Rewrite 规则详解一 215
14.10 Nginx 日志切割案例 215
14.11 Nginx 防盗链配置案例配置 215
第 15 章 大数据备份企业实战 215
15.1 企业级数据库备份实战 215
15.2 数据库备份方法及策略 215
15.3 Xtrabackup 企业实战 215
15.4 Percona-xtrabackup 备份实战 215
15.5 Innobackupex 增量备份 215
15.6 Mysql 增量备份恢复 215
第 16 章 Shell 企业编程基础 215
16.1 SHELL 编程入门简介 215
16.2 SHELL 脚本及 Hello World 215
16.3 Shell 编程之变量详解 215
16.4 Shell 编程之系统变量 215
16.5 Shell 编程之环境变量 215
16.6 Shell 编程之用户变量 215

16.7 If 条件语句实战 215
16.8 MySQL 数据库备份脚本 215
16.9 LAMP 一键自动化安装脚本 215
16.10 For 循环语句实战 215
16.11 Shell 编程函数实战 215
16.12 Shell 编程四剑客之 Find 215
16.13 Shell 编程四剑客之 GREP 215
16.14 Shell 数组编程 215
第 17 章 Shell 编程高级企业实战 215
17.1 Shell 编程实战系统备份脚本 215
17.2 Shell 编程实战收集服务器信息 215
17.3 Shell 编程实战拒绝恶意 IP 登录 215
17.4 Shell 编程实战 LAMP 一键安装 215
17.5 Shell 编程实战 MYSQL 主从复制 215
17.6 Shell 编程实战修改 IP 及主机名 215
17.7 Shell 编程实战 Zabbix 安装配置 215
17.8 Shell 编程实战 Nginx 虚拟主机 215
17.9 Shell 编程实战 Nginx、Tomcat 脚本 215
17.10 Shell 编程实战 Docker 管理脚本 215

17.11 Shell 编程实战 Bind 管理脚本 215
第 18 章 黑客攻防 Linux 服务器企业实战 215
18.1 基于二进制方式安装 DenyHosts 215
18.2 DenyHosts 配置目录详解 215
18.3 DenyHosts 配置实战 215
18.4 启动 DenyHosts 服务 215
18.5 删除被 DenyHosts 禁止 IP 215
18.6 配置 Denyhosts 发送报警邮件 215
18.7 基于 SHELL 全自动脚本实现防黑客攻击 215
第 19 章 企业邮件入门简介 215
19.1 Sendmail 安装配置 215
19.2 Dovecot 服务配置 215
19.3 Sendmail 别名配置 215
19.4 测试邮件收发 215
19.5 配置 Openwebmail 215
19.6 Postfix 入门简介 215
19.7 Postfix 服务安装 215
19.8 Postfix 服务器配置 215
19.9 Foxmail 本地邮箱配置 215

19.10 Postfixadmin 配置.....215
19.11 RoundCube GUI WEB 配置.....215
19.12 Postfix 虚拟用户配置.....215
19.13 Postfix+Extmail 配置实战.....215
19.14 Postfix+Extman 配置实战.....215
19.15 MailGraph_ext 安装配置： 215
19.16 基于 Postfix+Extman 虚拟用户注册.....215
19.17 基于 Extman 自动注册并登陆.....215
第 20 章 自动化运维发展前景.....215
20.1 传统运维方式简介.....215
20.2 自动化运维简介.....215
20.3 运维自动化的具体内容.....215
20.4 建立高效的 IT 自动化运维管理 215
20.5 IT 自动化运维工具 215
20.6 IT 自动化运维体系 215
第 21 章 Puppet 自动运维企业实战.....215
21.1 Puppet 入门简介.....215
21.2 Puppet 工作原理.....215
21.3 Puppet 安装配置.....215

21.4 Puppet 企业案例演示.....215
21.5 Puppet 常见资源及模块.....215
21.6 Puppet file 资源案例.....215
21.7 Puppet package 资源案例.....215
21.8 Puppet service 资源案例.....215
21.9 Puppet exec 资源案例.....215
21.10 Puppet cron 资源案例.....215
21.11 Puppet 日常管理与配置.....215
21.11.1 Puppet 自动认证.....215
21.11.2 Puppet 客户端自动同步.....215
21.11.3 Puppet 服务端主动推送.....215
21.12 Puppet 批量部署案例.....215
21.12.1 Puppet 批量修改静态 IP 案例.....215
21.12.2 Puppet 批量配置 NTP 同步服务器.....215
21.12.3 Puppet 自动部署及同步网站.....215
第 22 章 企业级 Docker 虚拟化平台实战.....215
22.1 虚拟化技术概述及简介.....215
22.2 互联网虚拟化技术种类.....215
22.3 KVM 虚拟化概念.....215

22.4 ESXI 虚拟化技术概念 215
22.5 XEN 虚拟化技术概念 215
22.6 Docker 虚拟化技术概念 215
22.7 虚拟化技术实现方式 215
22.8 Docker 虚拟化特点 215
22.9 为什么使用 Docker 215
22.10 Docker 引擎架构 215
22.11 Docker 镜像&容器&仓库 215
22.12 CentOS6.x（6.5+）Linux Docker 平台实战 215
22.13 CentOS7.x（7.0+）Linux Docker 平台实战 215
22.14 Ubuntu（16.04+）Linux Docker 平台实战 215
22.15 Docker 仓库源更新实战 215
22.16 Docker 典型命令演练 215
22.16.1 docker search 命令实战 215
22.16.2 docker pull 命令实战 215
22.16.3 docker images 命令实战 215
22.16.4 docker run 命令实战 215
22.16.5 docker ps 命令实战 215
22.16.6 docker inspect 命令实战 215

22.16.7 docker exec 命令实战 215
22.16.8 docker stop|start 命令实战 215
22.16.9 docker kill 命令实战 215
22.16.10 docker rm 命令实战 215
22.16.11 docker rmi 命令实战 215
22.17 CentOS6.x Docker 桥接网络实战 215
22.18 CentOS7.x Docker 桥接网络实战 215
22.19 基于 Docker 构建 JAVA 服务器实战 215
22.20 基于 Docker WEB 管理 Docker 容器 215
22.21 Docker Tomcat 性能优化 215
22.22 Docker 生产环境资源监控方案&监控实战 215
22.22.1 docker stats 监控工具 215
22.22.2 CAdvisor 监控工具 215
22.22.3 CAdvisor 部署配置 215
22.22.4 构建 Cadvisor+InfluxDB+Grafana 平台 215
22.23 Docker 构建 Nginx 服务器实战 215
22.24 Docker 构建 LNAMP 架构实战 215
22.25 Docker 自动化部署二 215

老男孩 1 对 1 职业规划咨询+100G 运维入门到精通全套资料 加老师 V：Linux9090

Linux 运维学习手册

第1章 Linux 快速入门

Linux 是一套免费使用和自由传播的类 UNIX 操作系统，是一个基于 POSIX 移植操作系统接口（Portable Operating System Interface of UNIX，POSIX）和 UNIX 的多用户、多任务、支持多线程和多 CPU 的操作系统。

目前被广泛使用于企业服务器、WEB 网站平台、大数据、虚拟化、Android、超级计算机等领域，未来 Linux 将应用各行各业，例如云计算、物联网、人工智能等。

本章向读者介绍 Linux 发展简介、Linux 发行版特点、32 位及 64 位 CPU 特性及 Linux 内核命名规则。

1.1 为什么要学习 Linux

我们为什么要学习 Linux？我们目前的处境是什么？我们想达到什么样的目标？在谈到这三个问题时，相信每个人都有自己的答案，我们来自不同的家庭，各种经历也都不一样，但最终的目标都是希望通过学习技术，提升自己的专业技术。真正做一个对社

会有贡献的人。

想想我们刚步入学堂的那一刻起，心里就狠狠下决心，以后不管做什么，都要有一番出息，可是 20 年、30 年过去了，我们收获了什么，得到了什么，到底是在追求什么？方向又在哪里呢？

在生活中各种挫折、感情、生活以及很多零碎的事情，让我们很难静下心来学习，当我们某天突然惊醒，年少已不在。所以今天就下定决心，现在就要学习，去行动，去改变。

人生最可怕的是在自以为舒适的地方待得太久，等到外界改变来的时候，已经晚了，我们不能像温水煮青蛙一样，待在温水里，没有觉察到周围事物的变化，最终被社会所淘汰,如图 1-1 所示。



图 1-1 温水煮青蛙

1.2 Linux 操作系统简介

Linux 操作系统是基于 UNIX 以网络为核心的设计思想，是一个性能稳定的多用户网络操作系统，Linux 能运行各种工具软件、应用程序及网络协议，它支持安装在 32 位和 64 位 CPU 硬件上。

通常的讲，Linux 这个词本身只表示 Linux 内核，但是人们已经习惯用 Linux 来形容整个基于 Linux 内核的操作系统，并且是一种使用 GNU 通用公共许可证(GNU General Public License，GPL）工程各种工具和数据库的操作系统。

GNU 是“GNU is Not Unix”，UNIX 是一种广泛使用的商业操作系统，由于 GNU 将要实现以 UNIX 系统的接口标准，因此 GNU 计划可以分别开发不同的操作系统部件，并且采用了部分当时已经可自由使用的软件。

为了保证 GNU 软件可以自由地“使用、复制、修改和发布”，所有的 GNU 软件都在一份禁止其他人添加任何限制的情况下授权所有权利给任何人的协议条款里，我们把这个条款称之为 GNU 通用公共许可证（GNU General Public License，GPL）。

1991 年的 10 月 5 日，Linux 创始人 Linus Torvalds 在 comp.os.minix 新闻组上发布消息，正式向外宣布 Linux 内核的诞生，1994 年 3 月 Linux 1.0 发布，代码量 17 万行，当时是完全按照自由免费的协议发布，随后正式采用 GPL 协议，目前 GPL 协议版本包括：GPLv1、GPLv2、GPLv3 以及未来的 GPLv4、GPLv5 等。

1.3 Linux 操作系统优点

随着 IT 产业的不断发展，Linux 操作系统应用领域越来越广泛，尤其是近年来 Linux 在服务器领域飞速的发展，主要得益于 Linux 操作系统具备的如下优点：

- 开源、免费；
- 系统迭代更新；
- 系统性能稳定；
- 安全性高；
- 多任务，多用户；
- 耗资源少；
- 内核小；
- 应用领域广泛；
- 使用及入门容易。

1.4 Linux 操作系统发行版

学习 Linux 操作系统，需要选择不同的发行版本，Linux 操作系统是一个大类别，Linux 操作系统主流发行版本包括：Red Hat Linux、CentOS、Ubuntu、SUSE Linux、Fedora Linux 等，具体发行版本区别如下：

1. Red Hat Linux

Red Hat Linux 是最早的 Linux 发行版本之一，同时也是最著名的 Linux 版本，Red

Hat Linux 已经创造了自己的品牌，也是读者经常听到的“红帽操作系统”。Red Hat 1994 年创立，目前公司全世界有 3000 多人，一直致力于开放的源代码体系，向用户提供一套完整的服务，这使得它特别适合在公共网络中使用。这个版本的 Linux 也使用最新的内核，还拥有大多数人都需要使用的主体软件包。

Red Hat Linux 发行版操作系统的安装过程非常简单，图形安装过程提供简易设置服务器的全部信息，磁盘分区过程可以自动完成，还可以通过图形界面（Graphical User Interface，GUI）完成安装，即使对于 Linux 新手来说这些都非常简单。后期如果想批量安装 Red Hat Linux 系统，可以通过批量的工具来实现快速安装。

2. CentOS

社区企业版操作系统（Community Enterprise Operating System，CentOS）是 Linux 发行版之一，它是来自于 Red Hat Enterprise Linux 依照开放源代码所编译而成。由于出自同样的源代码，因此有些要求高度稳定性的服务器以 CentOS 替代商业版的 Red Hat Enterprise Linux 使用。

CentOS 于 Red Hat Linux 不同之处在于 CentOS 并不包含封闭的源代码软件，可以开源免费使用，得到运维人员、企业、程序员的青睐，CentOS 发行版操作系统是目前企业使用最多的系统之一，2016 年 12 月 12 日，CentOS7 基于 Red Hat Enterprise Linux 的 CentOS Linux 7 (1611) 系统正式对外发布。

3. Ubuntu

Ubuntu 是一个以桌面应用为主的 Linux 操作系统，其名称来自非洲南部祖鲁语或

豪萨语的“ubuntu”一词（译为吾帮托或乌班图），意思是“人性”、“我的存在是因为大家的存在”，是非洲传统的一种价值观。

Ubuntu 基于 Debian 发行版和 GNOME 桌面环境， Ubuntu 发行版操作系统的目标在于为一般用户提供一个最新的、同时稳定的以开放自由软件构建而成的操作系统，目前 Ubuntu 具有庞大的社区力量，用户可以方便地从社区获得帮助。

4. SUSE Linux

SUSE(发音 /ˈsuːsə/) ,SUSE Linux 出自德国 ,SuSE Linux AG 公司发行维护的 Linux 发行版，是属于此公司的注册商标 2003 年 11 月 4 日，Novell 表示将会对 SUSE 提出收购。收购的工作于 2004 年 1 月完成。

Novell 也向大家保证 SUSE 的开发工作仍会继续下去，Novell 更把公司内全线电脑的系统换成 SUSE LINUX ,并同时表示将会把 SUSE 特有而优秀的系统管理程序 - YaST2 以 GPL 授权释出。

5. Fedora Linux

Fedora 是一个知名的 Linux 发行版，是一款由全球社区爱好者构建的面向日常应用的快速、稳定、强大的操作系统。它允许任何人自由地使用、修改和重发布，无论现在还是将来。它由一个强大的社群开发，这个社群的成员以自己的不懈努力，提供并维护自由、开放源码的软件和开放的标准。

Fedora 约每六个月会发布新版本 ,美国当地时间 2015 年 11 月 3 日 ,北京时间 2015 年 11 月 4 日 ,Fedora Project 宣布 Fedora 23 正式对外发布 ,2022 年 6 月发布 Fedora

26 版本。

1.5 32 位与 64 位操作系统的区别

学习 Linux 操作系统之前，需要理解计算机基本的常识，计算机内部对数据的传输和储存都是使用二进制，二进制是计算技术中广泛采用的一种数制，而 Bit（比特）则表示二进制位，二进制数是用 0 和 1 两个数码来表示的数。基数为 2，进位规则是“逢二进一”，0 或者 1 分别表示一个 Bit 二进制位。

Bit 位是计算机最小单位，而字节是计算机中数据处理的基本单位，转换单位为：1Byte=8Bit，4Byte=32Bit。

随着计算机技术的发展，尤其是中央处理器（Central Processing Unit，CPU）技术的变革，CPU 的位数指的是通用寄存器（General-Purpose Registers， GPRs）的数据宽度，也就是处理器一次可以处理的数据量多少。

目前主流 CPU 处理器分为 32 位 CPU 处理器和 64 位 CPU 处理器，32 位 CPU 处理器可以一次性处理 4 个字节的数据量。而 64 位处理器一次性处理 8 个字节的数据量（1Byte=8bit），64 位 CPU 处理器对计算机处理器在 RAM 里（随机存取储存器）处理信息的效率比 32 位 CPU 做了很多优化，效率更高。

X86_32 位操作系统和 X86_64 操作系统也是基于 CPU 位数的支持，具体区别如下：

- 32 位操作系统表示 32 位 CPU 对内存寻址的能力；
- 64 位操作系统表示 64 位 CPU 对内存寻址的能力；

- 32 位的操作系统安装在 32 位 CPU 处理器和 64 位 CPU 处理器上；
- 64 位操作系统只能安装 64 位 CPU 处理器上；
- 32 位操作系统对内存寻址不能超过 4GB；
- 64 位操作系统对内存寻址可以超过 4GB，企业服务器更多安装 64 位操作系统，支持更多内存资源的利用；
- 64 位操作系统是为高性能处理需求设计，数据处理、图片处理、实时计算等领域需求；
- 32 位操作系统是为普通用户设计，普通办公、上网冲浪等需求。

1.6 Linux 内核命名规则

Linux 内核是 Linux 操作系统的核心，一个完整的 Linux 发行版包括进程管理、内存管理、文件系统、系统管理、网络操作等部分。

Linux 内核官网可以下载 Linux 内核版本、现行版本，历史版本，可以了解版本与版本之间的特性。

Linux 内核版本命名在不同的时期有其不同的命名规范，其中在 2.X 版本中，X 如果为奇数表示开发版、X 如果为偶数表示稳定版，从 2.6.X 以及 3.X，内核版本命名就没有严格的约定规范。

从 Linux 内核 1994 年发布 1.0 发布到目前主流 2.6、3.X 版本，4.X 属于开发调试阶段，查看 Linux 操作系统内核如图 1-2 所示：

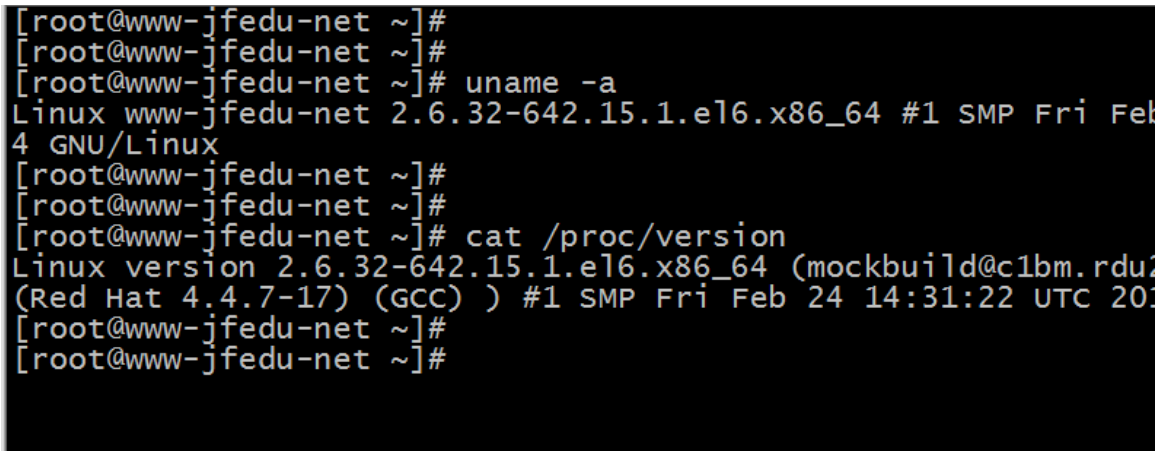


图 1-2 操作系统内核

Linux 内核命名格式为 “R.X.Y-Z”，其中 R、X、Y、Z 命名意义如下：

- 数字 R 表示内核版本号，版本号只有在代码和内核有重大改变的时候才会改变，到目前为止有 4 个大版本更新。
- 数字 X 表示内核主版本号，主版本号根据传统的奇偶系统版本编号来分配，奇数为开发版，偶数为稳定版。
- 数字 Y 表示内核次版本号 ,次版本号是无论在内核增加安全补丁、修复 Bug、实现新的特性或者驱动时都会改变。
- 数字 Z 表示内核小版本号，小版本号会随着内核功能的修改、Bug 修复而发生变化。

官网内核版本如图 1-3 所示，Mainline 表示主线开发版本，Stable 表示稳定版本，稳定版本主要由 mainline 测试通过而发布。Longterm 表示长期支持版，会持续更新及 Bug 修复，如果长期版本被标记为 EOL（End of Life），则表示不再提供更新。



图 1-3 官网内核版本

第2章 Linux 发展及系统安装

互联网飞速发展，用户对网站体验要求也越来越高，目前主流 WEB 网站后端承载系统均为 Linux 操作系统，Android 手机也是基于 Linux 内核而研发，企业大数据、云存储、虚拟化等先进技术也均是基于 Linux 操作系统为载体，满足企业的高速发展。

本章向读者介绍 Linux 发展前景、Windows 与 Linux 操作系统区别、硬盘分区介绍、CentOS 7 Linux 操作系统安装图解及菜鸟学好 Linux 必备大绝招。

2.1 Linux 发展前景及就业形势

权威部门统计，未来几年内我国软件行业的从业机会十分庞大，中国每年对 IT 软件人才的需求将达到 200 万人左右。而对于 Linux 专业人才的就业前景，更是广阔；据悉

在未来 5-10 年内 Linux 专业人才的需求将达到 150 万，尤其是有 Linux 行业经验的，资深的 Linux 工程师非常缺乏，薪资也非常诱人，平均月薪 15000-25000，甚至更高，Linux 行业薪资如图 2-1 所示：

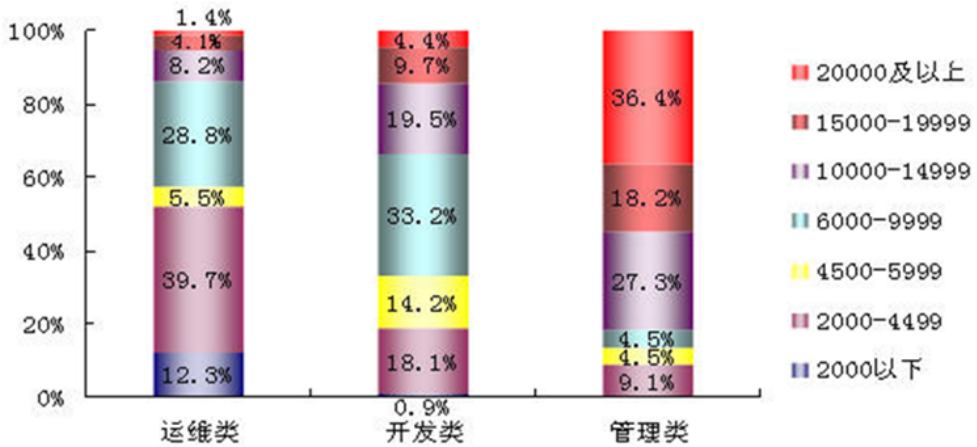


图 2-1 Linux 行业薪资

2.2 Windows 操作系统简介

为什么要学习 Windows 操作系统呢，了解 Windows 系统结构，可以让我们快速学习 Linux 操作系统，通过对比学习的方法，我们可以更快的学会 Linux。

计算机硬件组成包括:CPU、内存、网卡、硬盘、DVD 光驱、电源、主板、显示器、鼠标、键盘等设备，计算机硬件是不能直接被人使用的，需要在其上安装各种操作系统，安装完操作系统，并安装驱动程序，方可进行操作、办公、上网冲浪等。

计算机的硬件组成：

- CPU，相当于人的大脑，中央处理器；
- 内存，存储设备，临时存储，CPU 所需要数据，从内存中读取，内存读写速度很快；

- 硬盘，持久化设备，内存空间小，费用高，大量的数据存在硬盘，硬盘读写速度比内存慢；(SSD、SAS、SATA)；

驱动程序主要指的是设备驱动程序(Device Driver)，是一种可以使计算机系统和设备通信的特殊程序，相当于硬件的接口，操作系统只有通过这个接口，才能控制硬件设备，进行资源调度。

Windows 操作系统主要以窗口形式对用户展示，操作系统须安装在硬盘上，安装系统之前需对硬盘进行分区并格式化，默认 Windows 操作系统安装在 C 盘分区，D 盘分区用于存放数据文件。

通俗的讲，安装操作系统时，需要对磁盘进行格式化，格式化需要指定格式化的类型，告诉操作系统如何去管理磁盘空间，文件如何存放，如何查找及调用。操作系统不知道怎么存放文件以及文件结构，文件系统概念就诞生了。

文件系统是操作系统用于明确磁盘或分区上文件的方法和数据存储结构，文件系统由三部分组成：与文件管理有关软件、被管理文件以及实施文件管理所需数据结构。

Windows 操作系统，文件系统类型一般有 FAT、FAT16、FAT32、NTFS 等，不同的文件系统类型，有不同的特性，例如 NTFS 文件系统类型支持文件及文件夹安全设置，而 FAT32 文件系统类型不支持，NTFS 支持单文件最多为单个磁盘分区的容量大小 2T，而 FAT32 单个最大文件不能超过 4GB。

Windows 操作系统从设计层面来讲，主要用来管理电脑硬件与软件资源的程序，大致包括五个方面的管理功能:进程与处理机管理、作业管理、存储管理、设备管理、文件

管理。Windwos 操作系统从个人使用角度来讲，主要用于个人电脑办公、软件安装、上网冲浪、游戏、数据分析、数据存储等功能。

2.3 硬盘分区简介

学习 Windows、Linux 操作系统，必然要了解硬盘设备，硬盘是电脑主要的存储媒介之一，硬盘要能够安装系统或者存放数据，必须进行分区和格式化，Windows 系统常见分区有三种：主磁盘分区、扩展磁盘分区、逻辑磁盘分区。

一块硬盘设备，主分区至少有 1 个，最多 4 个，扩展分区可以为 0，最多 1 个，且主分区+扩展分区总数不能超过 4 个，逻辑分区可以有若干个。在 Windows 下激活的主分区是硬盘的启动分区，他是独立的，也是硬盘的第一个分区，通常就是我们所说的 C 盘系统分区。

扩展分区是不能直接用的，他是以逻辑分区的方式来使用的，扩展分区可分成若干逻辑分区。他们的关系是包含的关系，所有的逻辑分区都是扩展分区的一部分。

在 Windows 系统安装时，硬盘驱动器是通过磁盘 0,磁盘 1 来显示,其中磁盘 0 表示第一块硬盘,磁盘 1 表示第二块硬盘,然后在第一块硬盘磁盘 0 上进行分区，最多不能超过 4 个主分区,分区为 C、D、E、F。

硬盘接口是硬盘与主机系统间的连接部件，作用是在硬盘缓存和主机内存之间传输数据。不同的硬盘接口决定着硬盘与计算机之间的连接速度，在整个系统中，硬盘接口的优劣直接影响着程序运行快慢和系统性能好坏，常见的硬盘接口类型为：IDE

（Integrated Drive Electronics）、SATA（Serial Advanced Technology Attachment）、SCSI（Small Computer System Interface）、SAS（Serial Attached SCSI）和光纤通道等。

IDE 接口硬盘多用于家用，部分也应用于传统服务器，SCSI、SAS 接口的硬盘则主要应用于服务器市场，而光纤通道用于高端服务器上，SATA 主要用于个人家庭办公电脑及低端服务器。

在 Linux 操作系统中，读者可以看到硬盘驱动器的第一块 IDE 硬盘接口的硬盘设备为 hda，或者 SATA 硬盘接口的硬盘设备为 sda，主分区编号为 hda1-4 或者 sda1-4，逻辑分区从 5 开始。如果有第二块硬盘，主分区编号为 hdb1-4 或者 sdb1-4。

不管是 Windows 还是 Linux 操作系统，硬盘的总容量=主分区的容量+扩展分区的容量，而扩展分区的容量=各个逻辑分区的容量之和。主分区也可成为“引导分区”，会被操作系统和主板认定为这个硬盘的第一个分区，所以 C 盘永远都是排在所有磁盘分区的第一的位置上。

MBR（Master Boot Record）和 GPT（GUID Partition Table）是在磁盘上存储分区信息的两种不同方式。这些分区信息包含了分区从哪里开始的信息，这样操作系统才知道哪个扇区是属于哪个分区的，以及哪个分区是可以启动操作系统的。

在磁盘上创建分区时，必须选择 MBR 或者 GPT，默认是 MBR，也可以通过其他方式修改为 GPT 方式。MBR 分区的硬盘最多支持 4 个主分区，如果想支持更多主分区，可以考虑使用 GPT 格式分区。

2.4 Linux 安装环境准备

要学好 Linux 这门技术，首先需安装 Linux 操作系统，Linux 操作系统安装是每个初学者的门槛。而安装 Linux 操作系统，最大的困惑莫过于给操作系统进行磁盘分区。

虽然目前各种发行版本的 Linux 已经提供了友好的图形交互界面，但很多初学者还是感觉无从下手,这其中原因主要是不清楚 Linux 的分区规定。

Linux 系统安装中规定，同样每块硬盘设备最多只能分 4 个主分区（其中包含扩展分区）构成，任何一个扩展分区都要占用一个主分区号码，也就是在一个硬盘中，主分区和扩展分区一共最多是 4 个。

为了让读者能将本书所有 Linux 技术应用于企业，本书案例以企业里主流 Linux 操作系统 CentOS 为蓝本，目前主流 CentOS 发行版本为 CentOS7.2。

读者在安装 CentOS 操作系统时，如果没有多余的计算机裸机设备，可以基于 Windows 主机上安装 Vmware workstation 工具，该工具的用途可以在 Windows 主机上创建多个计算机裸机设备资源，包括：CPU、内存、硬盘、网卡、DVD 光驱、USB 接口、声卡，创建的多个计算机裸机设备共享 Windows 主机的所有资源。

读者在安装 CentOS 操作系统时，如果有多余的计算机裸机设备或者企业服务器，可以将 CentOS 系统直接安装在多余的设备上，安装之前需要下载 CentOS7.2 操作系统镜像文件(International Organization for Standardization,ISO 9660 标准)，通过刻录工具，将 ISO 镜像文件刻录至 DVD 光盘或者 U 盘里，通过 DVD 或者 U 盘启动然后安装系统。

如下为在 Windwos 主机上安装 VMware workstation 虚拟机软件，虚拟机软件的用途是可以在真实机上模拟一个新的计算机完整的资源设备，进而可以在计算机裸设备上安装 CentOS7.2 操作系统步骤：

（1） 安装环境准备

- VMware workstation 10.0
- CentOS 7.6 x86_64

（2） Vmware Workstation 10.0 下载

http://download3.vmware.com/software/wkst/file/VMware-workstation-full-10.0.1-1379776.exe

（3） CentOS7.2 操作系统 ISO 镜像下载

http://124.205.69.165/files/706900000291EB25/mirror.math.princeton.edu/pub/CentOS/7/isos/x86_64/CentOS-7-x86_64-DVD-1511.iso

（4） 将 VMware workstation 10.0 和 CentOS7 ISO 镜像文件下载至 Windows 系统，双击 VMware-workstation-full-10.0.1-1379776.exe，根据提示完成安装，会在 Windows 桌面显示 VMware Workstation 图标，如图 2-2 所示：

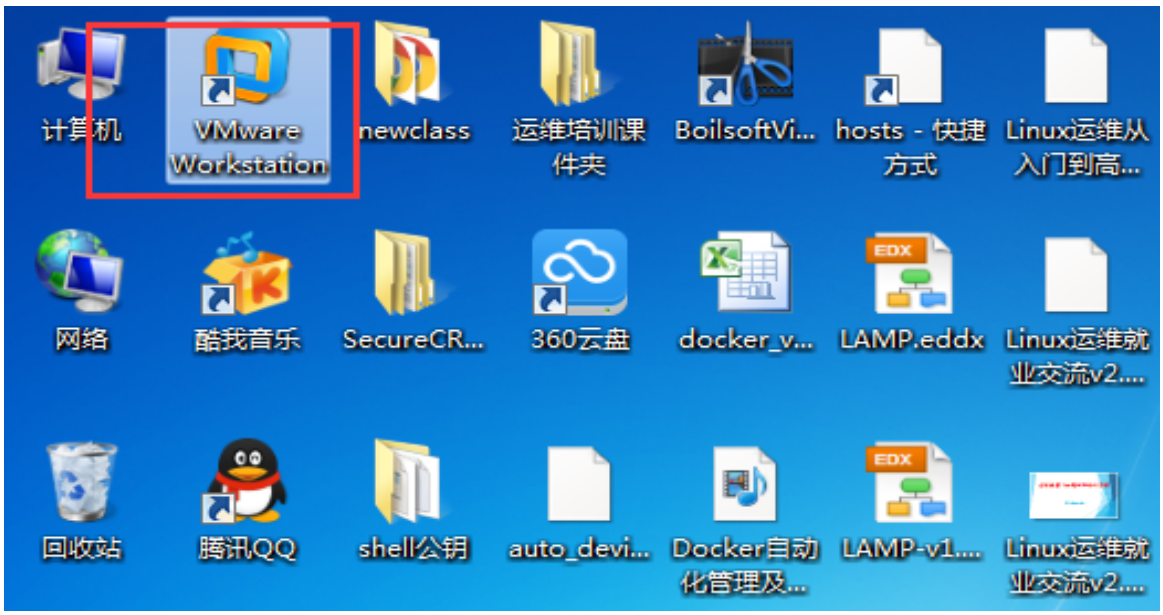


图 2-2 VMware Workstation 图标

（5） 双击桌面 VMware Workstation 图标打开虚拟机软件，单击“创建新的虚拟机”，如图 2-3 所示：



图 2-3 VMware Workstation 创建虚拟机

（6） 新建虚拟机向导，选择自定义（高级）(C）选项，如图 2-4 所示：

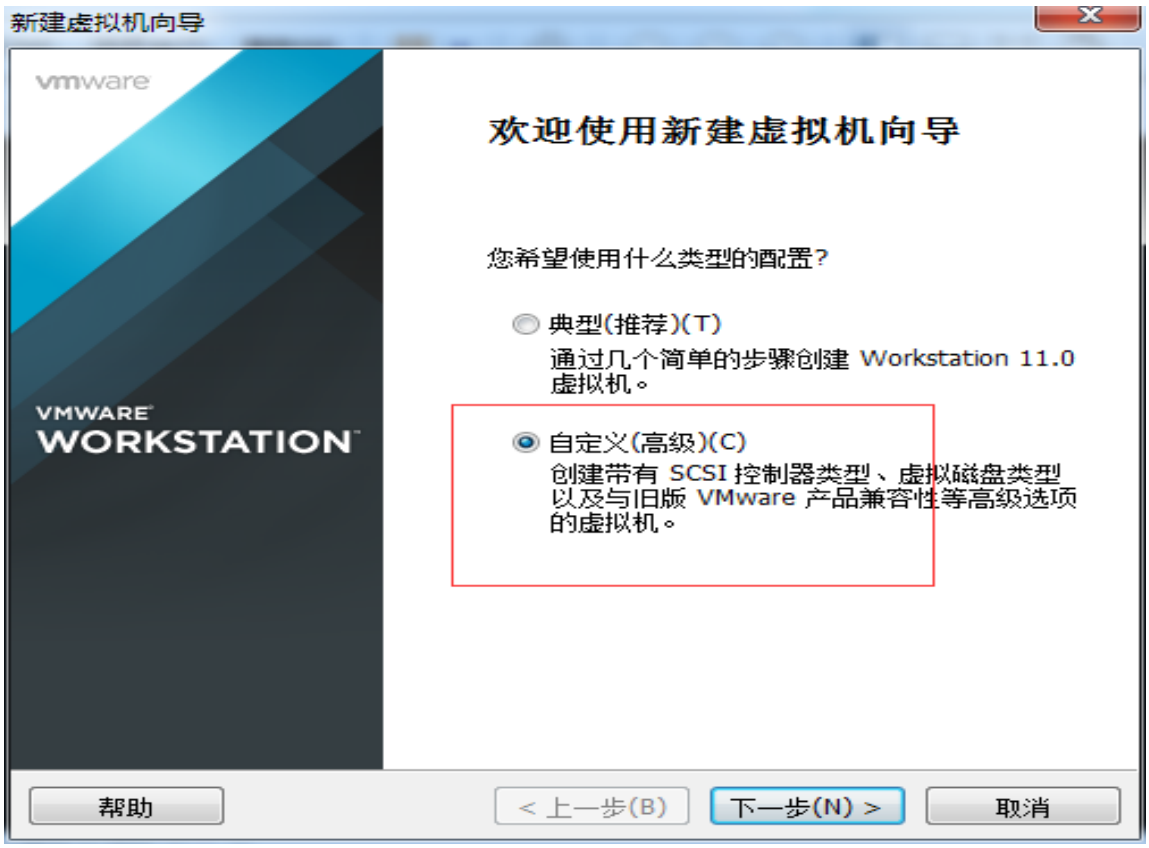


图 2-4 创建虚拟机向导

（7） 安装客户机操作系统，选择“稍后安装操作系统（S）”，如图 2-5 所示:

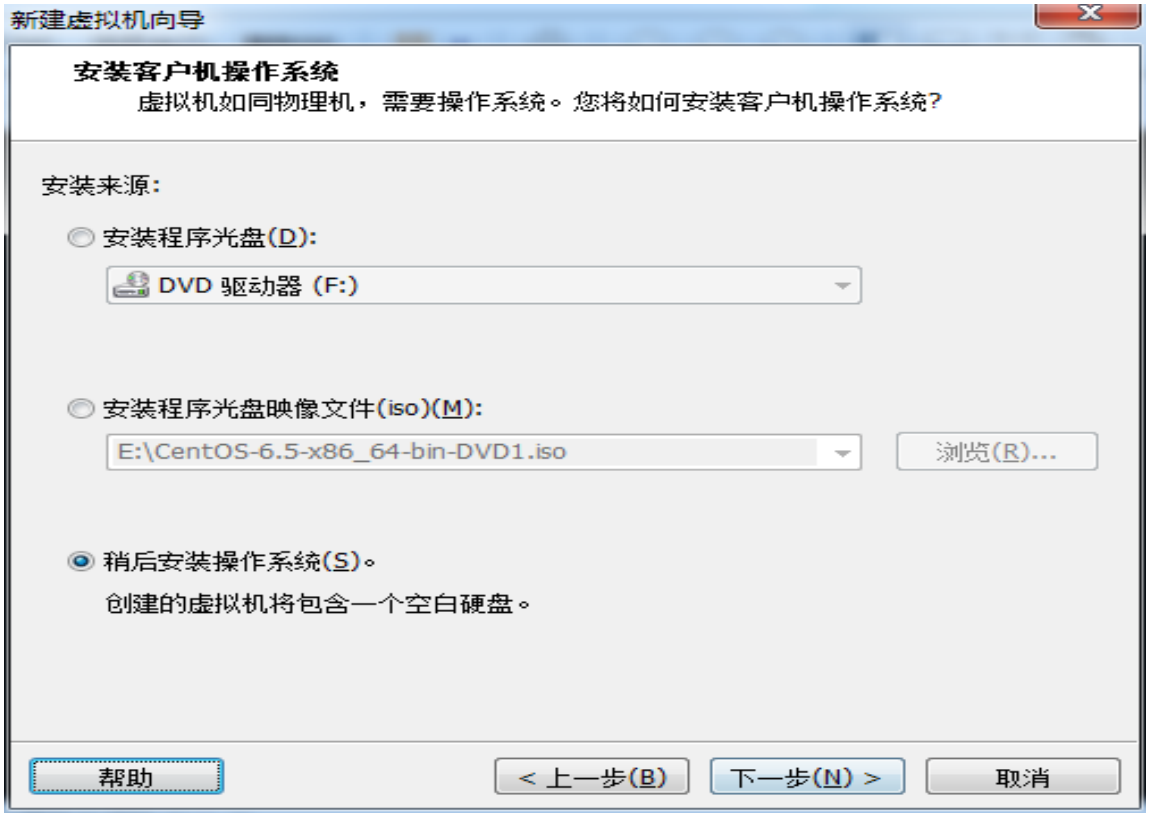


图 2-5 安装客户机操作系统

（8） 选择客户机操作系统，由于我们即将安装 CentOS7.2 操作系统，所以需要勾选“Linux（L）”，同时版本（V）选择“CentOS64 位”，如图 2-6 所示：

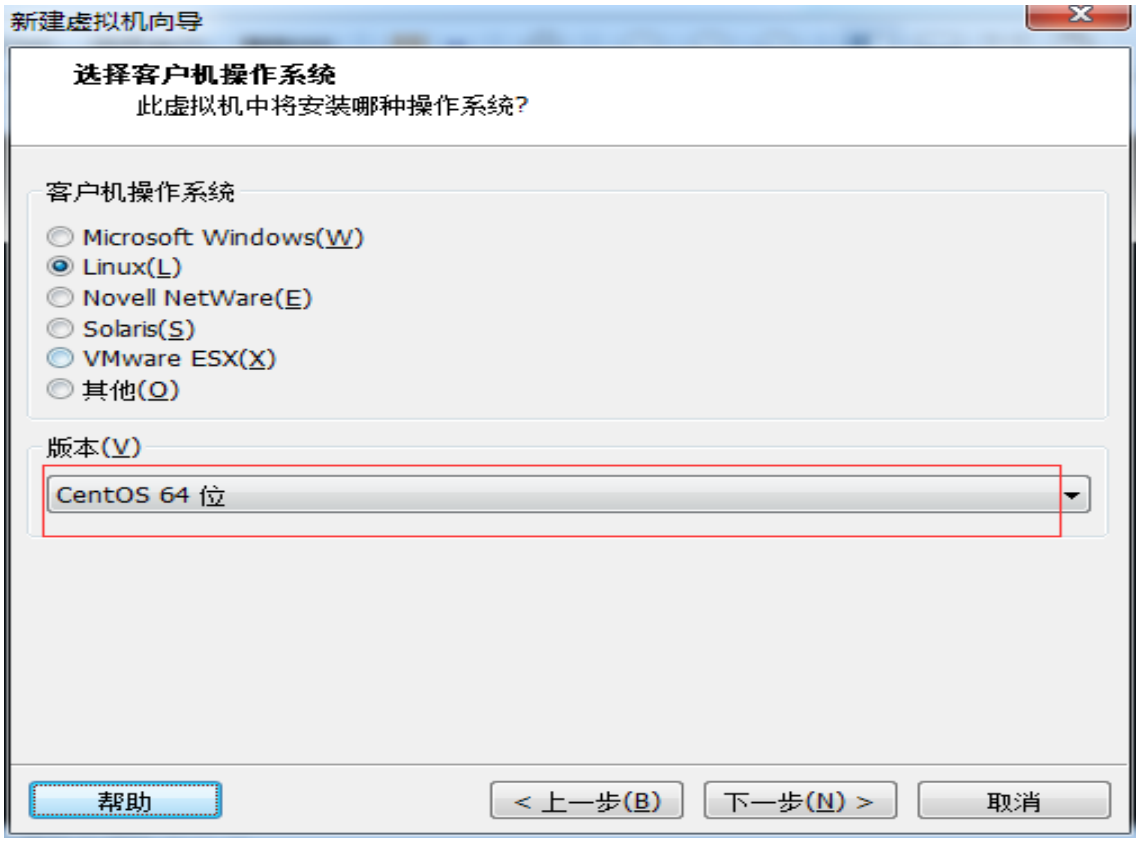


图 2-6 操作系统版本

（9） 虚拟机内存设置，默认为 1024MB，如图 2-7 所示：

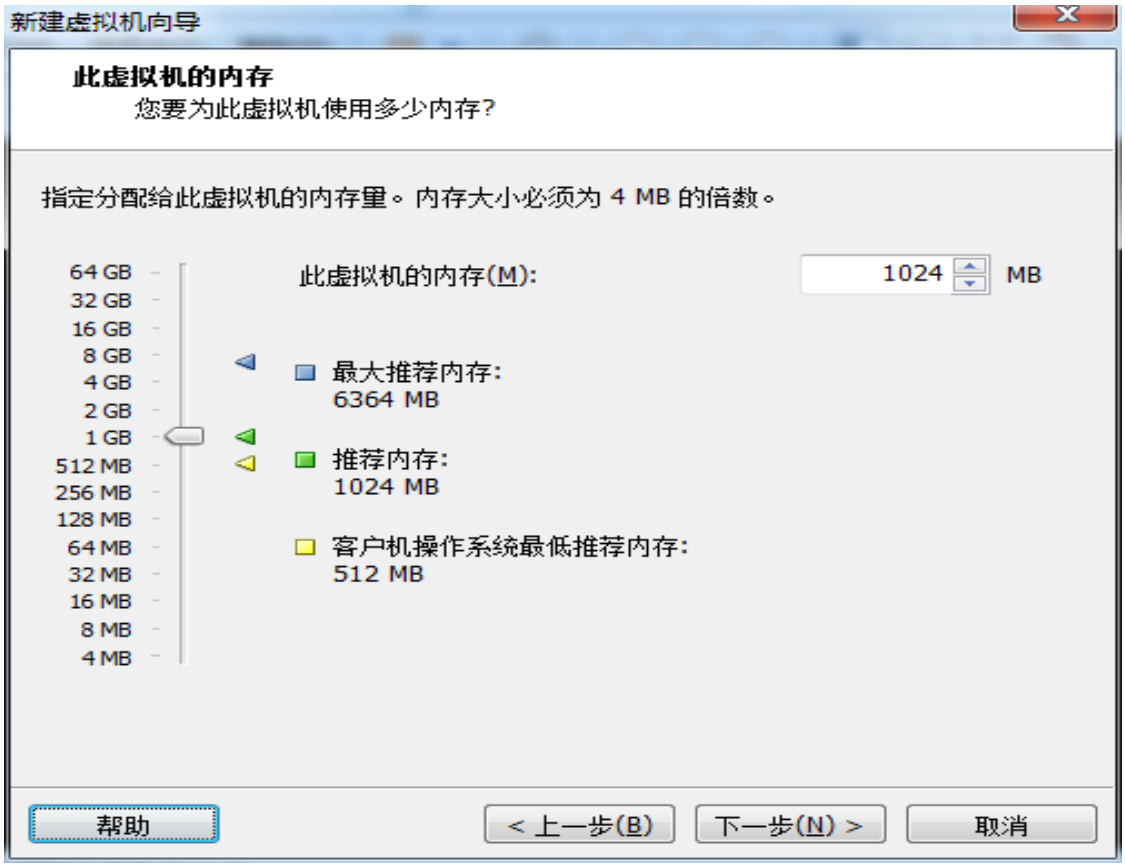


图 2-7 虚拟机内存分配

（10） 选择虚拟机网络类型，此处选择-网络连接为-“使用桥接模式”，如图 2-8 所示：

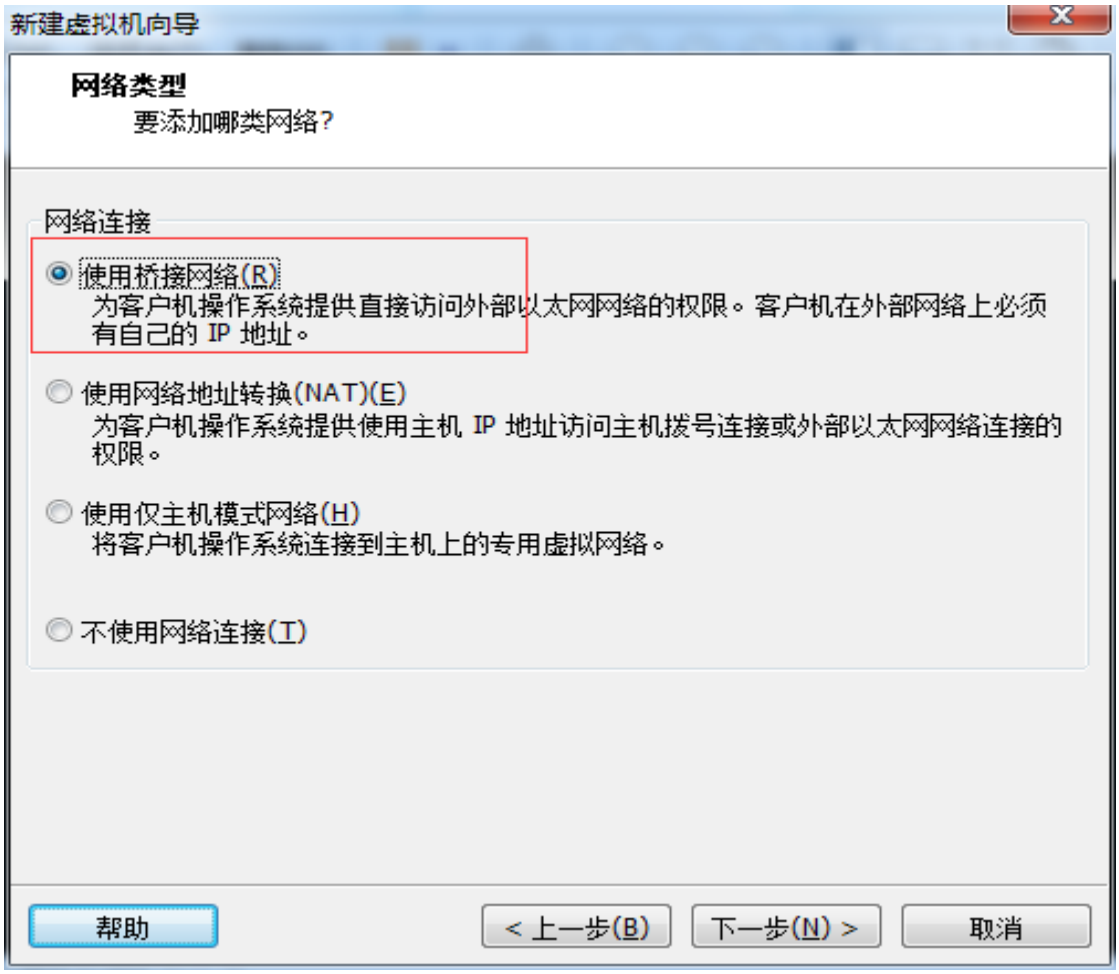


图 2-8 虚拟机网络类型

（11） 指定磁盘容量，设置虚拟机硬盘大小为 40GB，将虚拟磁盘拆分成多个文件，如下图 2-9 所示：

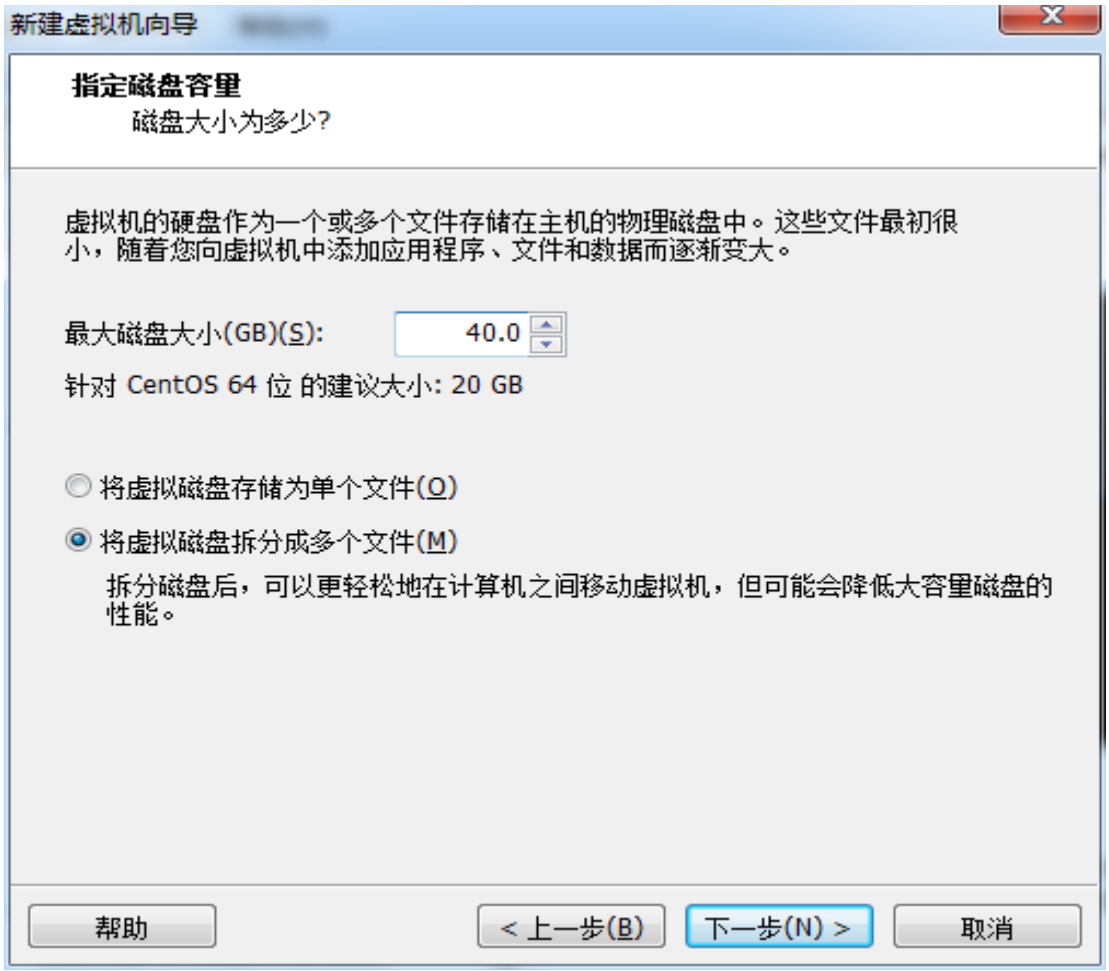


图 2-9 设置虚拟机磁盘容量

（12） 虚拟机硬件资源创建完成，设备详情里面包括计算机常用设备，例如内存、处理器、硬盘、CD/DVD、网络适配器等，如图 2-10 所示：



图 2-10 虚拟机裸机设备

（13） 将 CentOS7.2 ISO 系统镜像文件添加至虚拟机 CD/DVD 中，双击虚拟机“CD/DVD（IDE）自动检测”选项，选择 CentOS-7-x86_64-DVD-1511.iso 镜像文件，如图 2-11 所示：

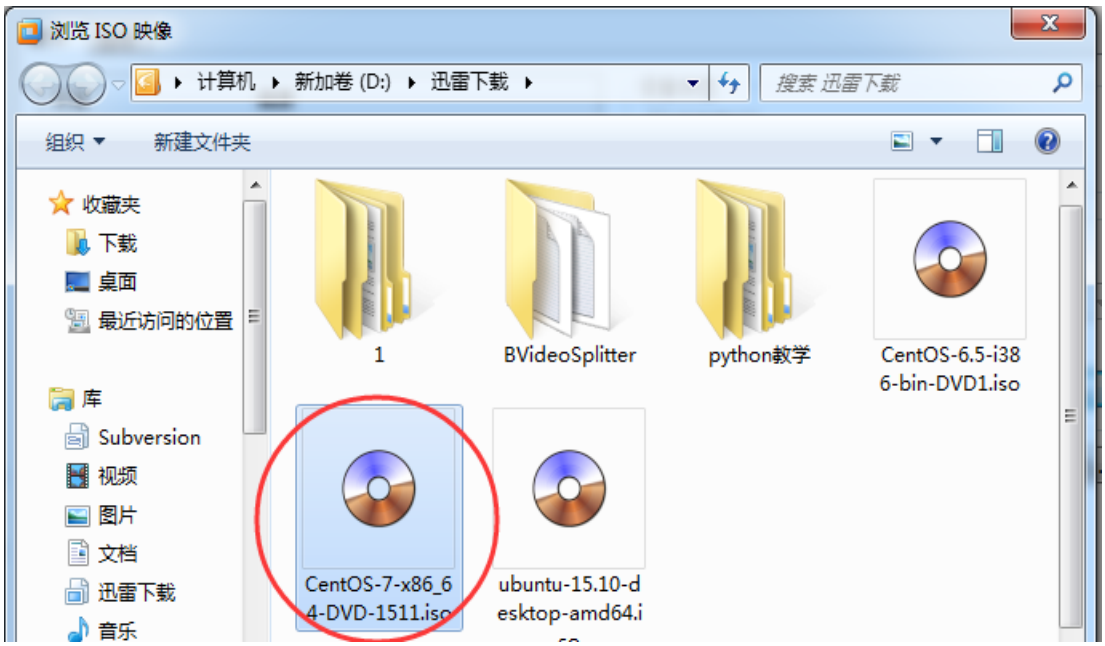


图 2-11 选择系统安装镜像

2.5 Linux 系统安装图解

如果直接在硬件设备上安装 CentOS 系统，不需要安装虚拟机等步骤，直接将 U 盘或者光盘插入 DVD 光驱即可，打开电源设备。

（1） 如图 2-12 所示，光标选择 Install CentOS 7，直接按 Enter 键进行系统安装。

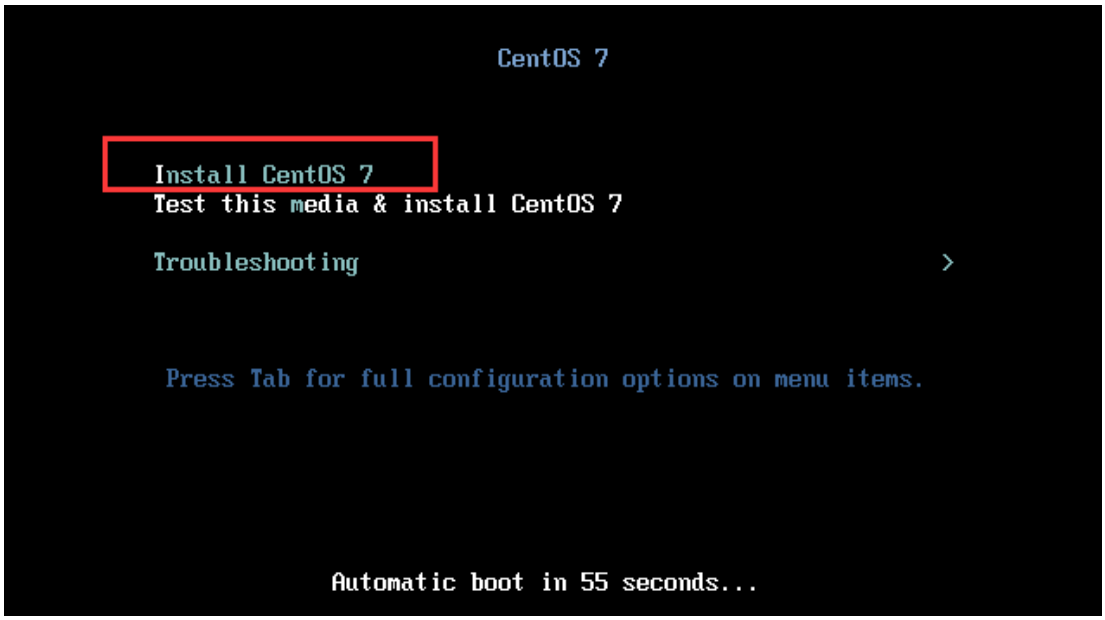


图 2-12 选择安装菜单

（2） 继续按 Enter 键启动安装进程，进入光盘检测，按 Esc 键跳过检测，如图 2-13 所示：

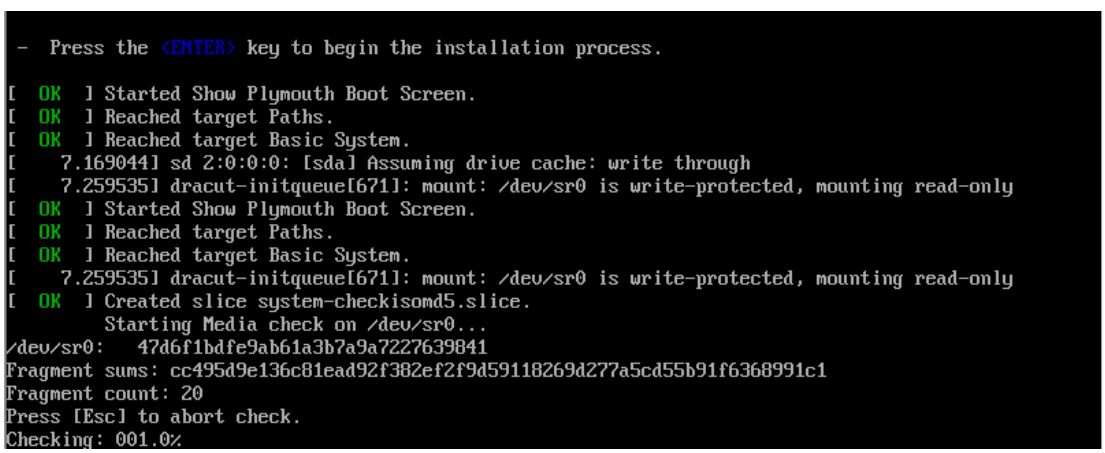


图 2-13 跳过 ISO 镜像检测

（3） CentOS7 欢迎界面，选择安装过程中界面显示的语言，初学者可以选择“简体中文”或者默认 English，如图 2-14 所示：

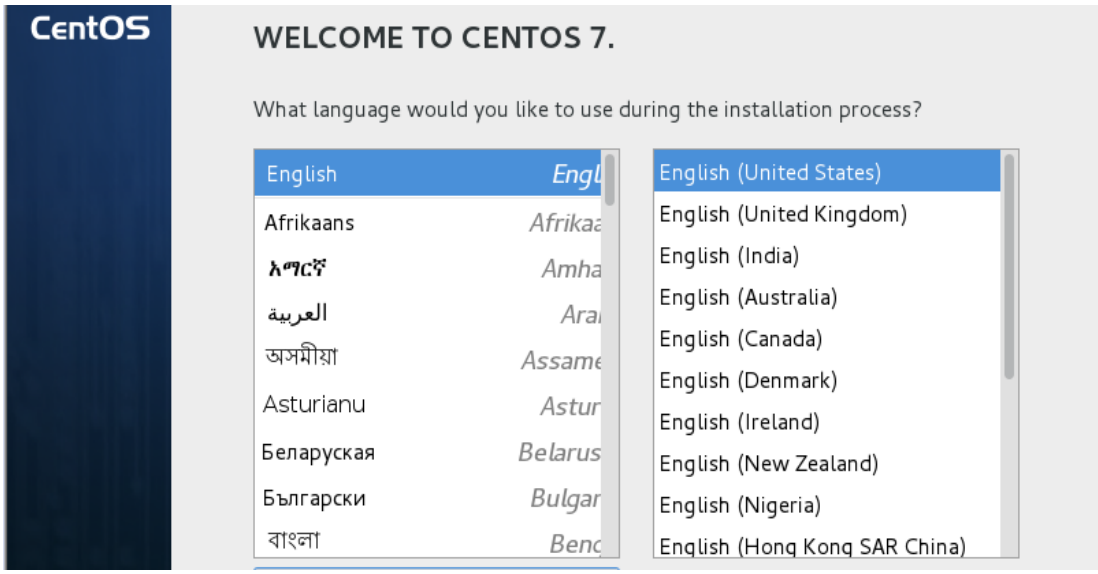


图 2-14 选择安装过程语言

（4） CentOS7 Installation Summary 安装总览界面，如图 2-15 所示：

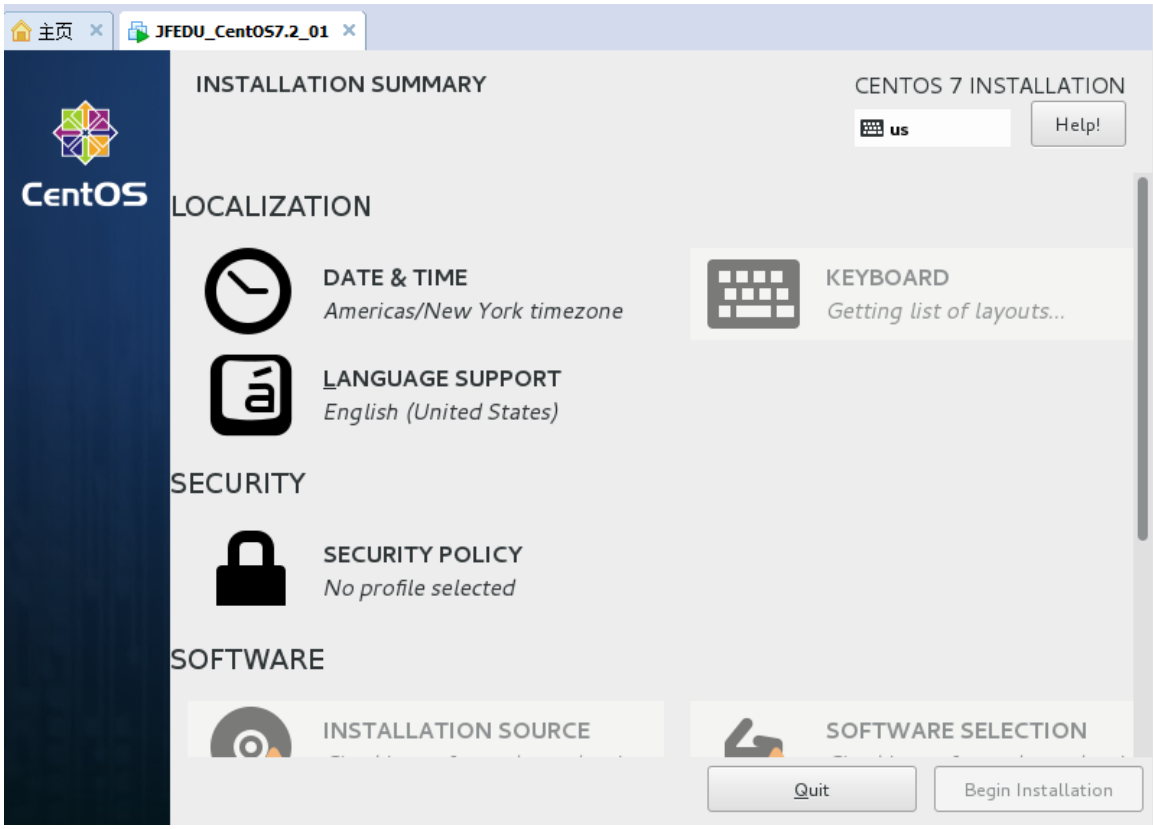


图 2-15 Installation Summary 界面

（5） 选择“I will configure partioning.”，单击 Done，如图 2-16 所示：

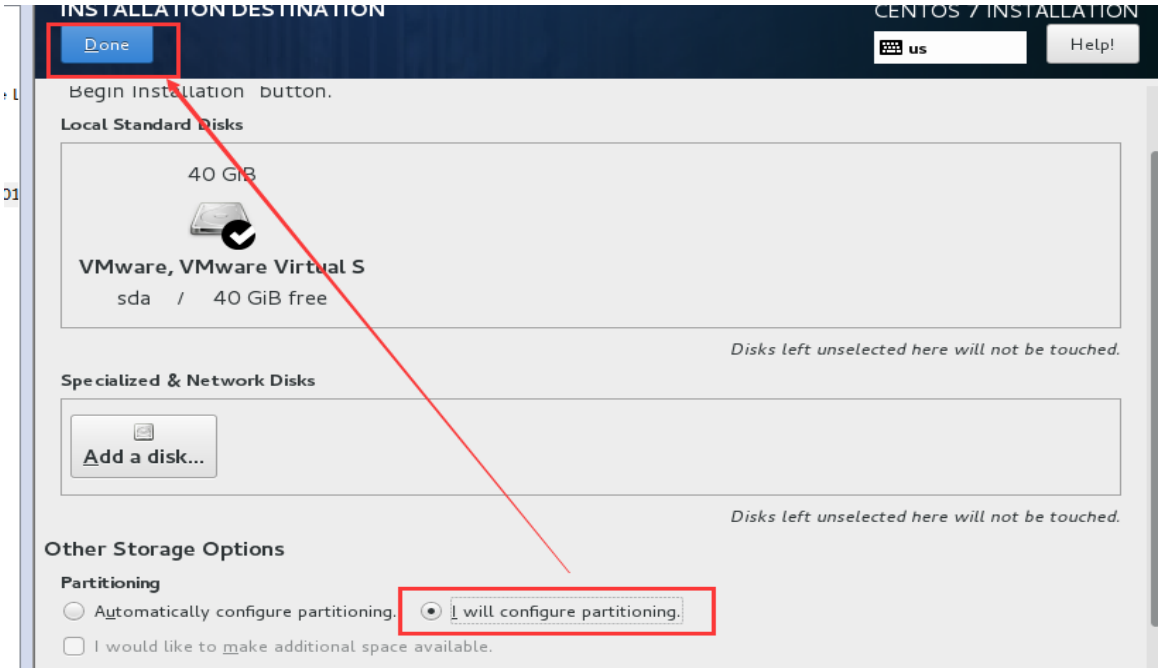


图 2-16 磁盘分区方式选择

（6） 下拉框选择“Standard Partition”，选择+号，创建分区，如图 2-17 所示：

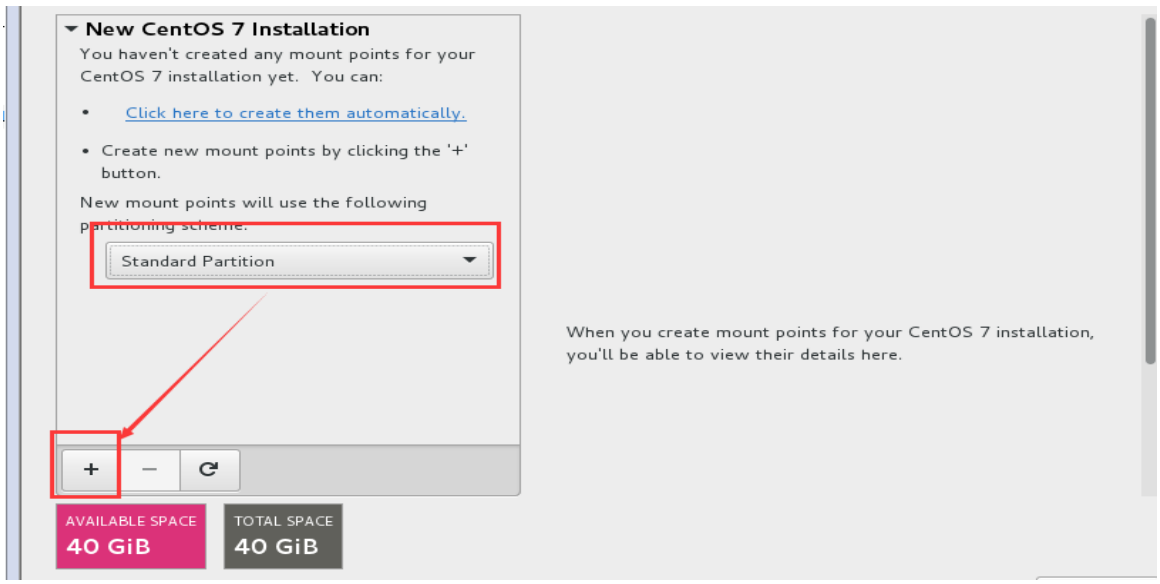


图 2-17 磁盘分区类型选择

（7） Linux 操作系统分区与 Windows 操作系统分区 C 盘、D 盘有很大区别，Liunx 操作系统是采用树形的文件系统管理方式，所有的文件存储以/（根）开始，如图 2-18 所示。

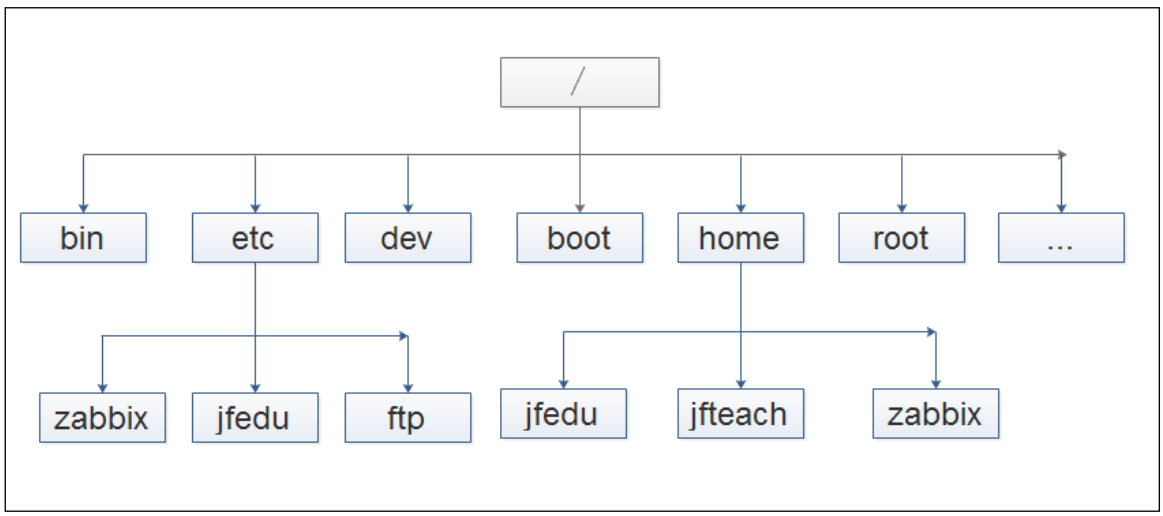


图 2-18 Linux 文件系统目录结构

Linux 是以文件的方式存储，例如/dev/sda 代表整块硬盘，/dev/sda1 表示硬盘第一分区，/dev/sda2 表示硬盘第二分区，为了能将目录和硬盘分区关联，所以 Linux 采用挂载点的方式来关联磁盘分区，/boot 目录、/根目录、/data 目录跟磁盘管理后，称之为分区，每个分区功能如下：

- /boot 分区用于存放 Linux 内核及系统启动过程所需文件；
- Swap 分区又称为交换分区，类似 Windows 系统的虚拟内存，物理内存不够用时，以供程序使用 Swap 内存；物理内存 32G，虚拟内存 512MB！阿里云交换分区设置 0；

- /分区用于系统安装核心分区及所有文件存放的根系统；
- /data 分区为自定义分区，企业服务器中用于作应用数据存放。

如图 2-19 所示，创建/boot 分区并挂载，分区大小为 200MB：

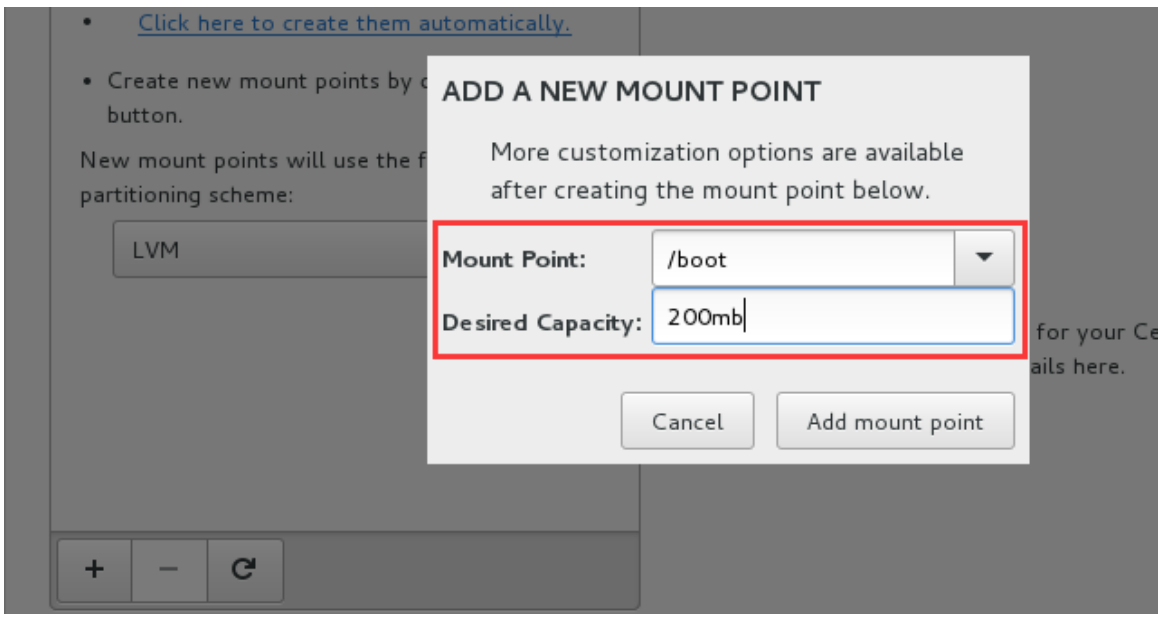


图 2-19 创建/boot 分区

单击“Add mount point”即可，磁盘分区默认文件系统类型为 XFS，根据如上方法，依次创建 swap 分区，大小为 2048MB，创建/分区，大小为剩余所有空间，最终如图 2-20 所示：

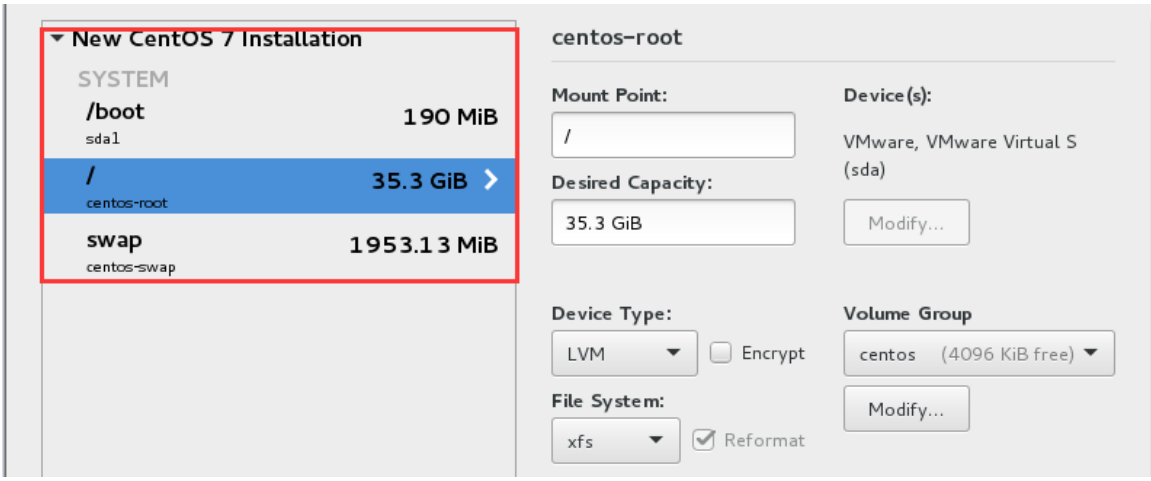


图 2-20 磁盘完整分区

（8） 选择 SOFTWARE SELECTION，设置为 Minimal Install 最小化安装，如果后期需要开发包、开发库等软件，可以在系统安装完后，根据需求安装即可，如图 2-21 所示:

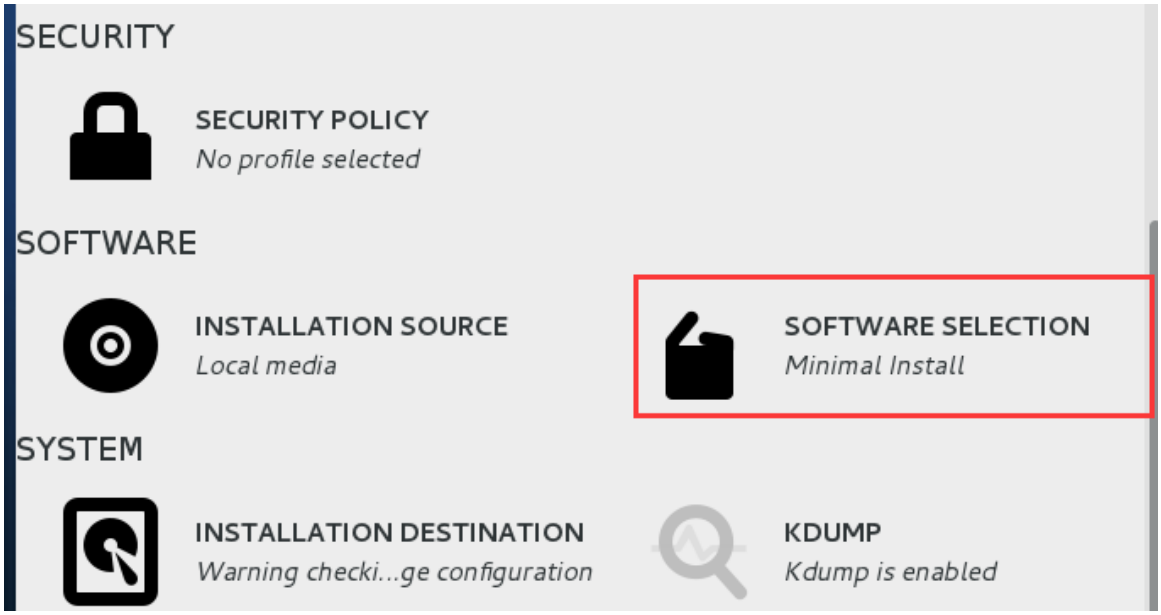


图 2-21 选择安装的软件

（9） 操作系统时区选择，选择 Asia-Shanghai，关闭 Network Time，如图 2-22 所

示：

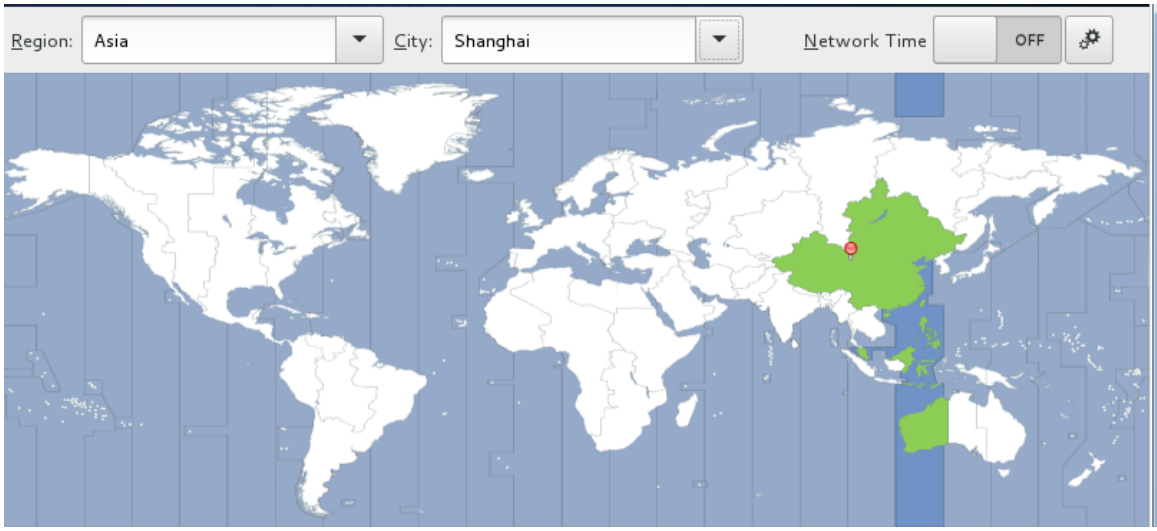


图 2-22 操作系统时区选择

（10） 以上配置完毕后，单击“Begin Installation”，单击“Root PASSWORD”设置 Root 用户密码，如图 2-23 所示，如果需要新增普通用户，可以单击“USER CREATEION”进行创建即可。

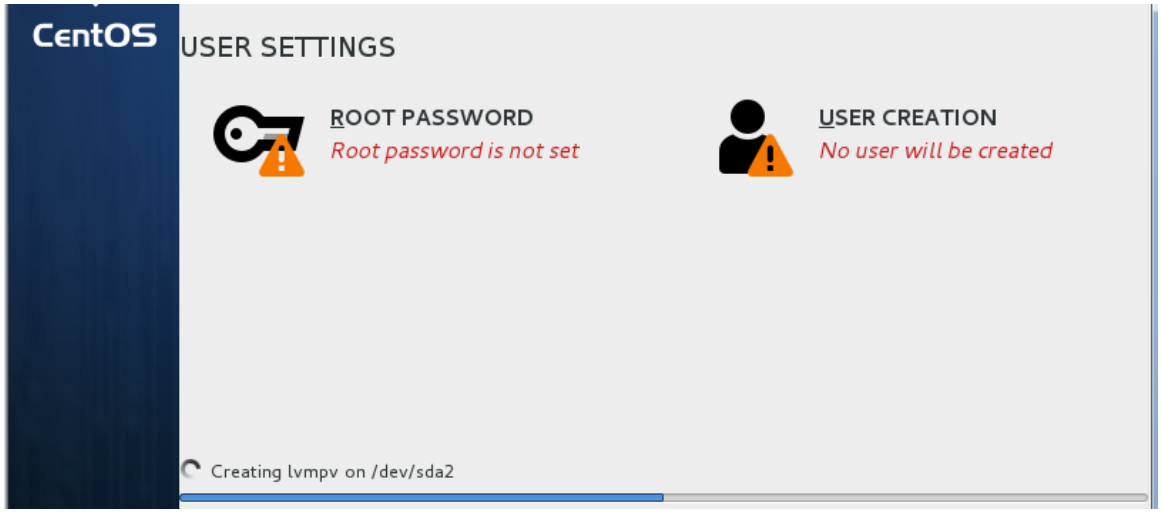


图 2-23 设置 ROOT 用户密码

（11） 安装进程完毕，单击“Reboot”重启系统，如图 2-24 所示：

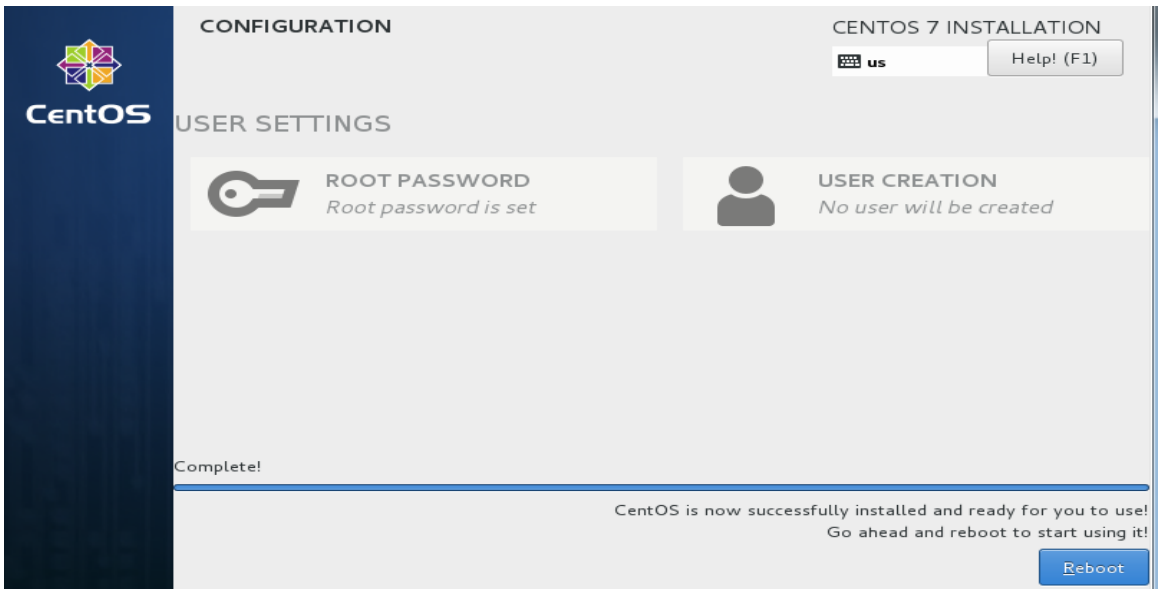


图 2-24 系统安装完毕

（12） 重启 CentOS 7 Linux 操作系统，进入 Login 登录界面，“localhost login:”处输入 root，按 Enter 键，然后“Password：”处输入系统安装时设定的密码，输入密码时不会提示，密码输入完按 Enter 键，即可登录 CentOS 7 Linux 操作系统，默认登录的终端称为 Shell 终端，所有的后续操作指令均在 Shell 终端上执行，默认显示字符提示[root@localhost ~]#，其中#代表当前是 root 用户登录，如果是$表示当前为普通用户。如图 2-25 所示：

```
CentOS Linux 7 (Core)
Kernel 3.10.0-327.el7.x86_64 on an x86_64

localhost login: root
Password:
Last login: Thu Aug 18 12:25:10 on tty1
[root@localhost ~]#
[root@localhost ~]# cat /etc/redhat-release
CentOS Linux release 7.2.1511 (Core)
[root@localhost ~]# uname -a
Linux localhost.localdomain 3.10.0-327.el7.x86_64 #1 SMP Thu Nov 19 22:10:57 UTC 2015 x86_64 x
[root@localhost ~]# pwd
/root
[root@localhost ~]# ls
anaconda-ks.cfg
```

图 2-25 Login 登录系统界面

2.6 菜鸟学好 Linux 大绝招

系统安装是初学者的门槛，系统安装完毕后，很多初学者不知道该如何学习，不知道如何快速进阶，下面作者总结了菜鸟学好 Linux 技能的大绝招：

- 初学者完成 Linux 系统分区及安装之后，需熟练掌握 Linux 系统管理必备命令，命令包括：cd、ls、pwd、clear、chmod、chown、chattr、useradd、userdel、groupadd、vi、vim、cat、more、less、mv、cp、rm、rmdir、touch、ifconfig、ip addr、ping、route、echo、wc、expr、bc、ln、head、tail、who、hostname、top、df、du、netstat、ss、kill、alias、man、tar、zip、unzip、jar、fdisk、free、uptime、lsof、lsmod、lsattr、dd、date、crontab、ps、find、awk、sed、grep、sort、uniq 等，每个命令至少练习 30 遍，逐步掌握每个命令的用法及应用场景；

- 初学者进阶之路，需熟练构建 Linux 下常见服务（DHCP、SAMBA、DNS、Apache、MySQL、Nginx、Zabbix、Squid、Varnish、LVS、Keepalived、ELK、MQ、Zookeeper、Docker、Openstack、Hbase、Mongodb、Redis 等，遇到问题先思考，没有头绪可以借助百度、Google 搜索引擎，问题解决后，将解决问题的步骤总结并形成文档；
- 理解操作系统的每个命令，每个服务的用途，为什么要配置这个服务，为什么需要调整该参数，只有带着目标去学习才能更快的成长，才能让你去发掘更多新知识；
- 熟练搭建 Linux 系统上各种服务之后，需要理解每个服务的完整配置和优化，可以拓展思维。例如 LAMP 所有服务放在一台机器上，能否分开放在多台服务器以平衡压力呢，该如何去构建和部署呢？一台物理机构建 Docker 虚拟化，如果是 100 台、1000 台如何去实施呢，会遇到哪些问题呢；
- Shell 是 Linux 最经典的命令解释器，Shell 脚本可以实现自动化运维，平时多练习 Shell 脚本编程，每个 Shell 脚本多练习几遍，从中吸取关键的参数、语法，不断的练习，不断的提高；
- 建立个人学习博客，把平时工作、学习中的知识都记录到博客，一方面可以供别人参考，另一方面可以提高自己文档编写及总结的能力；
- 学习 Linux 技术是一个长期的过程，一定要坚持，遇到各种错误、问题可以借助百度、Google 搜索引擎，如果解决不了，可以请教同学、朋友及你的老

师；

- 通过以上步骤的学习方法，不断进步，如果想达到高级、资深大牛级别，还需要进一步深入学习 WEB 集群架构、网站负载均衡、网站架构优化、自动化运维、运维开发、虚拟化等知识；
- 多练习才是硬道理，实践出真知。

2.7 本章小结

通过对本章内容的学习，读者对 Linux 系统有了一个初步的理解，了解 Linux 行业的发展前景，学会了如何在企业中或者虚拟机中安装 Linux 系统。

对 32 位、64 位 CPU 处理器以及对 Linux 内核版本命名也有进一步的认识，同时掌握了学习 Linux 的大绝招。

2.8 同步作业

1. 企业中服务器品牌 DELL R730，其硬盘总量为 300G，现需安装 CentOS 7 Linux 操作系统，请问如何进行分区？
2. GNU 与 GPL 的区别是什么？
3. 企业一台 Linux 服务器，查看该 Linux 内核显示：3.10.0-327.36.3.el7.x86_64，请分别说出点号分割的每个数字及字母的含义？
4. CentOS Linux 至今发布了多少个系统版本？
5. 如果 Linux 系统采用光盘安装，如何将 ISO 镜像文件刻录成光盘，请写出具体实现

流程？

第3章 CentOS 系统管理

Linux 系统安装完毕，需要对 Linux 系统进行管理和维护，让 Linux 服务器能真正应用于企业中。

本章向读者介绍 Linux 系统引导原理、启动流程、系统目录、权限、命令及 CentOS7 和 CentOS6 在系统管理、命令方面有什么区别，让我们一起来遨游在 Linux 的海洋里。

3.1 操作系统启动概念

不管是 Windows 还是 Linux 操作系统，底层设备一般均为物理硬件，操作系统启动之前会对硬件进行检测，然后硬盘引导启动操作系统，如下为操作系统启动相关的各个概念：

3.1.1 BIOS

基本输入输出系统（Basic Input Output System，BIOS）是一组固化到计算机主板上的只读内存镜像（Read Only Memory image，ROM）芯片上的程序，它保存着计算机最重要的基本输入输出的程序、系统设置信息、开机后自检程序和系统自启动程序。主要功能是为计算机提供最底层的、最直接的硬件设置和控制。

3.1.2 MBR

全新硬盘在使用之前必须进行分区格式化，硬盘分区初始化的格式主要由两种，分别是：MBR 格式和 GPT 格式。

如果使用 MBR 格式，操作系统将创建主引导记录扇区（Main Boot Record，MBR），MBR 位于整块硬盘的 0 磁道 0 柱面 1 扇区，主要功能是操作系统对磁盘进行读写时，判断分区的合法性以及分区引导信息的定位。

主引导扇区总共为 512 字节，MBR 只占用了其中的 446 个字节，另外的 64 个字节为硬盘分区表（Disk Partition Table，DPT），最后两个字节“55，AA”是分区的结束标志。

在 MBR 硬盘中，硬盘分区信息直接存储于主引导记录（MBR）中，同时主引导记录还存储着系统的引导程序，如图 3-1 所示：

0000-0088	Master Boot Record 主引导程序	主引导 程序
0089-01BD	出错信息数据区	数据区
01BE-01CD	分区项 1（16 字节）	分区表
01CE-01DD	分区项 2（16 字节）	
01DE-01ED	分区项 3（16 字节）	
01EE-01FD	分区项 4（16 字节）	
01FE	55	结束标志
01FF	AA	

图 3-1 MBR 分区表内容

MBR 是计算机启动最先执行的硬盘上的程序，只有 512 字节大小，所以不能载入操作系统的核心，只能先载入一个可以载入计算机核心的程序，我们称之为引导程序。

因为 MBR 分区标准决定了 MBR 只支持在 2TB 以下的硬盘，对于后面的多余空间只能浪费。为了支持能使用大于 2T 硬盘空间，微软和英特尔公司在可扩展固件接口(Extensible Firmware Interface，EFI)方案中开发了全局唯一的标识符（Globally unique identifier，GUID），进而全面支持大于 2T 硬盘空间在企业中使用。

3.1.3 GPT

全局唯一的标识符（Globally unique identifier，GUID），正逐渐取代 MBR 成为新标准。它和统一的可扩展固件接口 (Unified Extensible Firmware Interface,UEFI)相辅相成。

UEFI 用于取代老旧的 BIOS，而 GPT 则取代老旧的 MBR。之所以称为“GUID 分区表”，是因为驱动器上的每个分区都有一个全局唯一的标识符。

在 GPT 硬盘中，分区表的位置信息储存在 GPT 头中。出于兼容性考虑，第一个扇区同样有一个与 MBR 类似的标记，叫做受保护的主引导记录（Protected Main Boot Record，PMBR）。

PMBR 的作用是当使用不支持 GPT 的分区工具时，整个硬盘将显示为一个受保护的分区，以防止分区表及硬盘数据遭到破坏，而其中存储的内容和 MBR 一样，之后才是 GPT 头。

GPT 优点支持 2T 以上磁盘，如果使用 Fdisk 分区，最大只能建立 2TB 大小的分区，

创建大于 2TB 的分区，需使用 parted，同时必须使用 64 位操作系统，Mac、Linux 系统都能支持 GPT 分区格式，Windows 7/8 64bit、Windows Server 2008 64bit 支持 GPT。图 3-2 所示，为 GPT 硬盘分区表内容：

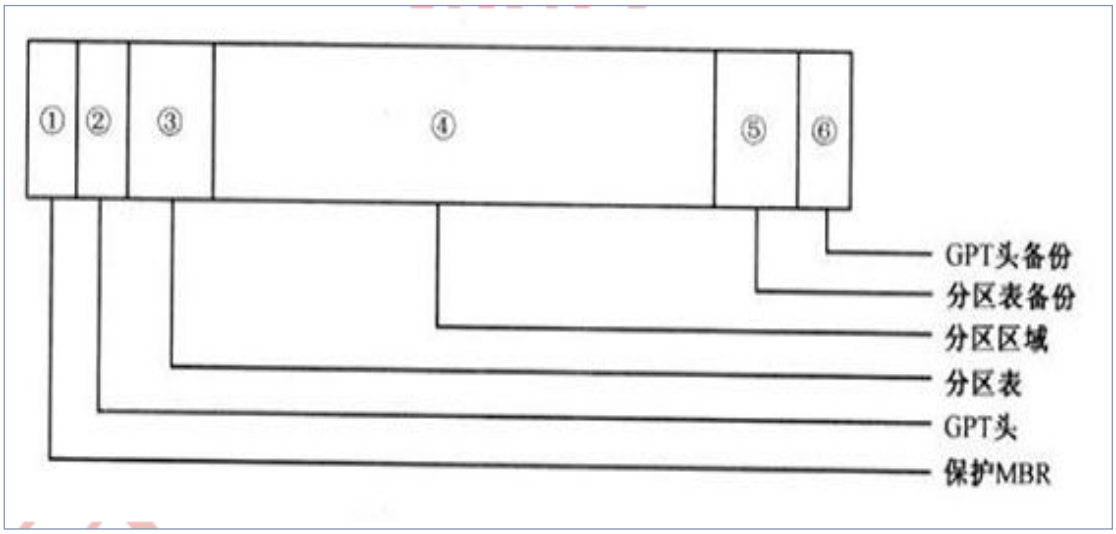


图 3-2 GPT 分区表内容

3.1.4 GRUB

GNU 项目的多操作系统启动程序（GRand Unified Bootloader，GRUB），可以支持多操作系统的引导，它允许用户可以在计算机内同时拥有多个操作系统，并在计算机启动时选择希望运行的操作系统。

GRUB 可用于选择操作系统分区上的不同内核，也可用于向这些内核传递启动参数。它是一个多重操作系统启动管理器。用来引导不同系统，如 Windows，Linux。Linux 常见的引导程序包括：LILO、GRUB、GRUB2，CentOS 7 Linux 默认使用 GRUB2 引导程序，引导系统启动。如图 3-3 所示为 GRUB 加载引导流程：

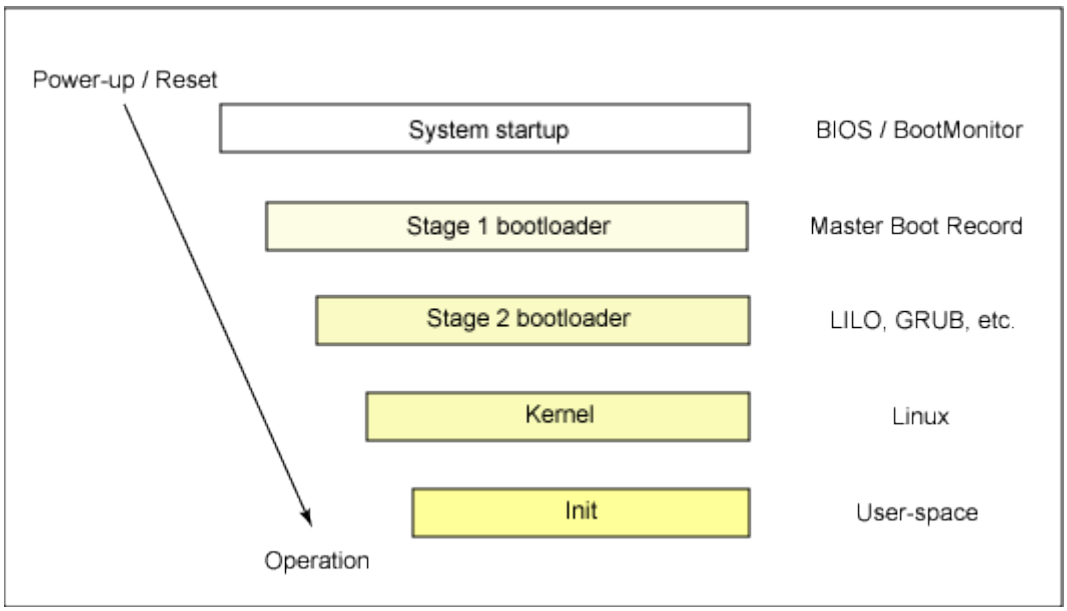


图 3-3 GRUB 引导流程

GRUB2 是基于 GRUB 开发成更加安全强大的多系统引导程序，最新 Linux 发行版都是使用 GRUB2 作为引导程序。同时 GRUB2 采用了模块化设计，使得 GRUB2 核心更加精炼，使用更加灵活，同时也就不需要像 GRUB 分为 stage1,stage1.5,stage2 三个阶段。

3.2 Linux 操作系统启动流程

初学者对 Linux 操作系统启动流程的理解 ,能有助于后期在企业中更好的维护 Linux 服务器，能快速定位系统问题，进而解决问题。Linux 操作系统启动流程如图 3-4 所示：

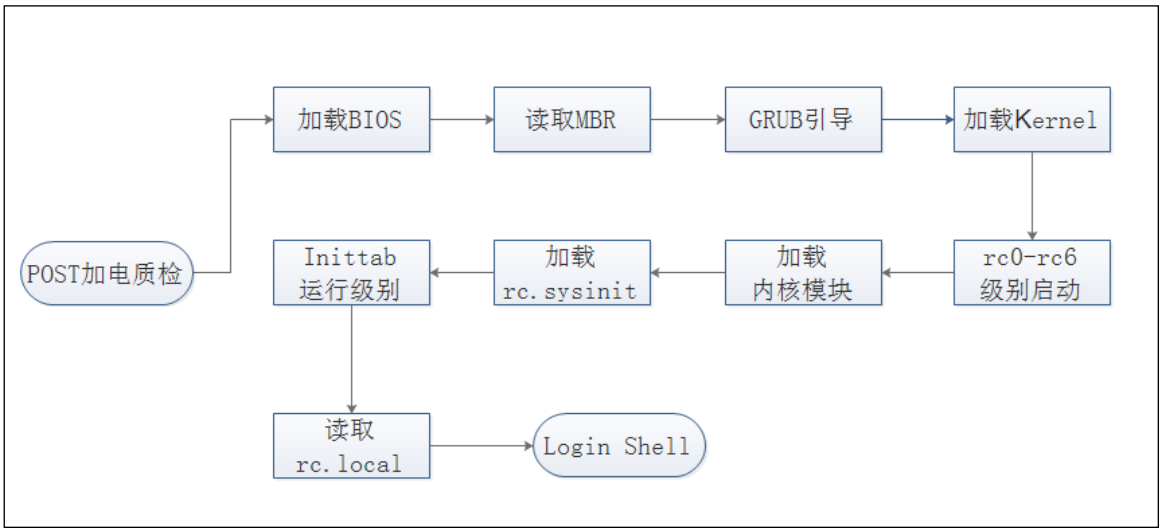


图 3-4 系统启动流程

（1） 加载 BIOS

计算机电源加电质检，首先加载基本输入输出系统（Basic Input Output System，BIOS），BIOS 中包含硬件 CPU、内存、硬盘等相关信息，包含设备启动顺序信息、硬盘信息、内存信息、时钟信息、即插即用（Plug-and-Play，PNP）特性等。加载完 BIOS 信息，计算机将根据顺序进行启动。

（2） 读取 MBR

读取完 BIOS 信息，计算机将会查找 BIOS 所指定的硬盘 MBR 引导扇区，将其内容复制到 0x7c00 地址所在的物理内存中。被复制到物理内存的内容是 Boot Loader，然后进行引导。

（3） GRUB 引导

GRUB 启动引导器是计算机启动过程中运行的第一个软件程序，当计算机读取内存中的 GRUB 配置信息后，会根据其配置信息来启动硬盘中不同的操作系统。

（4） 加载 Kernel

计算机读取内存映像，并进行解压缩操作，屏幕一般会输出“Uncompressing Linux”的提示，当解压缩内核完成后，屏幕输出“OK, booting the kernel”。系统将解压后的内核放置在内存之中，并调用 start_kernel()函数来启动一系列的初始化函数并初始化各种设备，完成 Linux 核心环境的建立。

（5） 设定 Inittab 运行等级

内核加载完毕，会启动 Linux 操作系统第一个守护进程 init，然后通过该进程读取 /etc/inittab 文件，/etc/inittab 文件的作用是设定 Linux 的运行等级，Linux 常见运行级别如下：

- 0：关机模式；
- 1：单用户模式；
- 2：无网络支持的多用户模式；
- 3：字符界面多用户模式；
- 4：保留，未使用模式；
- 5：图像界面多用户模式；

- 6：重新引导系统，重启模式。

（6） 加载 rc.sysinit

读取完运行级别，Linux 系统执行的第一个用户层文件/etc/rc.d/rc.sysinit，该文件功能包括：设定 PATH 运行变量、设定网络配置、启动 swap 分区、设定/proc、系统函数、配置 Selinux 等。

（7） 加载内核模块

读取/etc/modules.conf 文件及/etc/modules.d 目录下的文件来加载系统内核模块。该模块文件，可以后期添加或者修改及删除。

（8） 启动运行级别程序

根据之前读取的运行级别，操作系统会运行 rc0.d 到 rc6.d 中的相应的脚本程序，来完成相应的初始化工作和启动相应的服务。其中以 S 开头表示系统即将启动的程序，如果以 K 开头，则代表停止该服务。S 和 K 后紧跟的数字为启动顺序编号。如图 3-5 所示：

```
[root@www-jfedu-net etc]# ls rc
rc          rc1.d/      rc3.d/      rc5.d/      rc.d/       rc.sysinit
rc0.d/      rc2.d/      rc4.d/      rc6.d/      rc.local
[root@www-jfedu-net etc]# ls rc5.d/
K01smartd               K16php-fpm              K75ntpdate              K99lvm2-mon
K05atd                  K35dovecot              K75quota_nld            K99rngd
K05trqauthd             K36mysqld               K80redis                S01sysstat
K10flow-capture         K50kdump                K85mdmonitor            S10network
K10psacct               K50netconsole           K85messagebus           S11portrese
K10saslauthd            K50snmpd                K87irqbalance           S12rsyslog
K10zabbix_server        K50snmptrapd            K87restorecond          S25cups
K15collectd             K60munge                K88auditd               S26udev-pos
K15htcacheclean         K61nfs-rdma             K89rdisc                S30nscd
K15httpd                K74acpid                K92ip6tables            S50aegis
K15svnserve             K74haldaemon            K92iptables             S50cloud-in
K16abrt-ccpp            K75blk-availability     K95rdma                 S51cloud-in
K16abrtd                K75netfs                K99cpuspeed             S52cloud-in
[root@www-jfedu-net etc]# ls rc5.d/
```

图 3-5 运行级别服务

（9） 读取 rc.local 文件

操作系统启动完相应服务之后，会读取执行/etc/rc.d/rc.local 文件，可以将需要开机启动的任务加入到该文件末尾，系统会逐行去执行并启动相应命令，如图 3-6 所示：

```
[root@www-jfedu-net ~]# cat /etc/rc.d/rc.local
#!/bin/sh
#
# This script will be executed *after* all the other init scripts.
# You can put your own initialization stuff in here if you don't
# want to do the full Sys V style init stuff.
touch /var/lock/subsys/local
modprobe nf_conntrack
svnserve -d -r /data/svn/ --listen-port=8801
###############
/sbin/swapon /data/swapfile
/etc/init.d/httpd restart
/etc/init.d/mysqld restart
/usr/local/nginx/sbin/nginx
[root@www-jfedu-net ~]#
```

图 3-6 开机运行加载文件

（10） 执行/bin/login 程序

执行/bin/login 程序，启动到系统登录界面，操作系统等待用户输入用户名和密码，即可登录到 Shell 终端，如图 3-7 所示，输入用户名、密码即可登录 Linux 操作系统，至此 Linux 操作系统完整流程启动完毕。

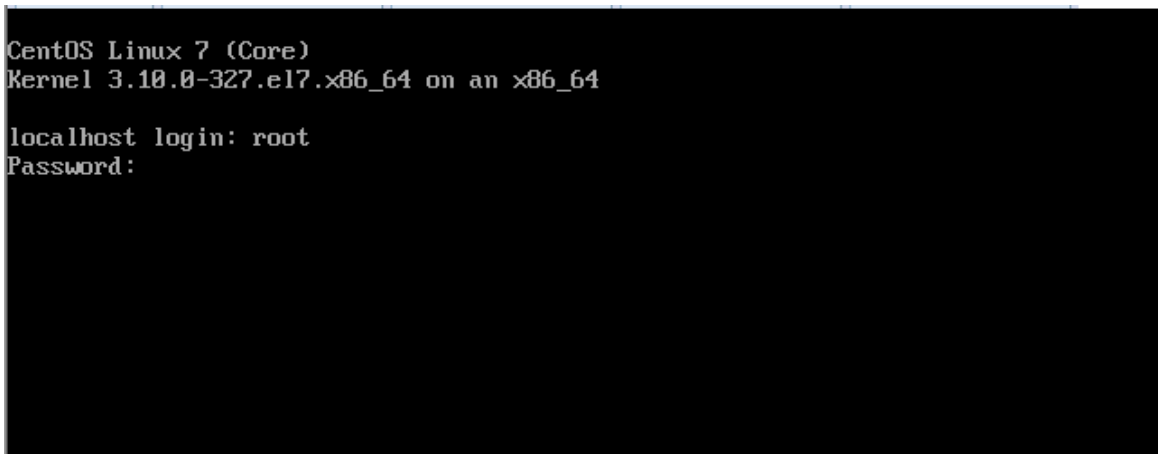


图 3-7 系统登录界面

3.3 CentOS6 与 CentOS7 区别

CentOS6 默认采用 Sysvinit 风格，Sysvinit 就是 system V 风格的 init 系统，Sysvinit 用术语 runlevel 来定义"预订的运行模式"。Sysvinit 检查 '/etc/inittab' 文件中是否含有'initdefault' 项，该选项指定 init 的默认运行模式。Sysvinit 使用脚本，文件命名规则和软链接来实现不同的 Runlevel,串行启动各个进程及服务。

CentOS7 默认采用 Systemd 风格，Systemd 是 Linux 系统中最新的初始化系统（init），它主要的设计目标是克服 Sysvinit 固有的缺点，提高系统的启动速度。

Systemd 和 Ubuntu 的 Upstart 是竞争对手，预计会取代 UpStart。Systemd 的目标是尽可能启动更少的进程，尽可能将更多进程并行启动。如图 3-8 所示为 CentOS6

与 CentOS7 操作系统的区别：

A	B	C	D
编号	系统功能	CentOS6	CentOS7
1	init系统	sysvinit	systemd
2	桌面系统	GNOME 2.x	GNOME 3.x/GNOME Shell
3	文件系统	ext4	xfs
4	内核版本	2.6.x	3.10.x
5	启动加载器	GRUB Legacy (+efibootmgr)	GRUB2
6	防火墙	iptables	firewalld
7	数据库	mysql	MariaDB
8	文件目录	/bin, /sbin, /lib, and /lib64在/根下	/bin, /sbin, /lib, and /lib64在/usr下
9	主机名	/etc/sysconfig/network	/etc/hostname
10	时间同步	ntp, ntpq -p	chrony,chronyc sources

11	修改时间	#vi /etc/sysconfig/clock
		ZONE="Asia/Tokyo"
		UTC=fales
		#ln -s
		/usr/share/zoneinfo/Asia/Tokyo
		/etc/localtime
			#timedatectl set-timezone Asia/Tokyo
			#timedatectl status

12	区域及字符设置	/etc/sysconfig/i18n	/etc/locale.conf localectl set-locale LANG=zh_CN.utf8 localectl status
13	启动停止服务	#service service_name start #service service_name stop #service sshd restart/status/reload	#systemctl start service_name #systemctl stop service_name #systemctl restart/status/reload sshd
14	自动启动	chkconfig service_name on/off	#systemctl enable service_name #systemctl disable service_name
15	服务列表	chkconfig --list	#systemctl list-unit-files #systemctl --type service
16	Kill服务	kill -9 <PID>	systemctl kill --signal=9 sshd
17	网络及端口信息	netstat	ss
18	IP信息	ifconfig	ip address show
19	路由信息	route -n	ip route show
20	关闭停止系统	shutdown -h now	systemctl poweroff
21	单用户模式	init S	systemctl rescue
22	运行模式	vim /etc/inittab id:3:initdefault:	systemctl set-default graphical.target systemctl set-default multi-user.target

图 3-8 CentOS6 与 CentOS7 操作系统区别

Linux 操作系统文件系统类型主要由 EXT3、EXT4、XFS 等,其中 CentOS6 普遍采用 EXT3 和 EXT4 文件系统格式，而 CentOS7 默认采用 XFS 格式。如下为 EXT3、EXT4、

XFS 区别：

- EXT4 是第四代扩展文件系统（Fourth EXtended filesystem，EXT4）是 Linux 系统下的日志文件系统，是 EXT3 文件系统的后继版本；
- EXT3 类型文件系统支持最大 16TB 文件系统和最大 2TB 文件；
- EXT4 分别支持 1EB（1EB=1024PB，1PB=1024TB）的文件系统，以及 16TB 的单个文件；
- EXT3 只支持 32,000 个子目录，而 EXT4 支持无限数量的子目录；
- EXT4 磁盘结构的 inode 个数支持 40 亿，而且 EXT4 的单个文件大小支持到 16T(4K block size) ；
- XFS 是一个 64 位文件系统，最大支持 8EB 减 1 字节的单个文件系统，实际部署时取决于宿主操作系统的最大块限制，常用语 64 位操作系统，发挥更好的性能；
- XFS 一种高性能的日志文件系统，最早于 1993 年，由 Silicon Graphics 为他们的 IRIX 操作系统而开发，是 IRIX 5.3 版的默认文件系统；
- XFS 于 2000 年 5 月，Silicon Graphics 以 GPL 发布这套系统的源代码，之后被移植到 Linux 内核上，XFS 特别擅长处理大文件，同时提供平滑的数据传输。

3.4 TCP/IP 协议概述

要学好 Linux，对网络协议也要有充分的了解和掌握，例如传输控制协议/因特网互联协议（Transmission Control Protocol/Internet Protocol，TCP/IP），TCP/IP 名为网络通讯协议，是 Internet 最基本的协议、Internet 国际互联网络的基础，由网络层的 IP 协议和传输层的 TCP 协议组成。

TCP/IP 定义了电子设备如何连入因特网，以及数据如何在它们之间传输的标准。协议采用了 4 层的层级结构，每一层都呼叫它的下一层所提供的协议来完成自己的需求。

TCP 负责发现传输的问题，一有问题就发出信号，要求重新传输，直到所有数据安全正确地传输到目的地，而 IP 是给因特网的每台联网设备规定一个地址。

基于 TCP/IP 的参考模型将协议分成四个层次，它们分别是网络接口层、网际互连层（IP 层）、传输层（TCP 层）和应用层。如图 3-9 为 TCP/IP 跟 OSI 参考模型层次的对比：

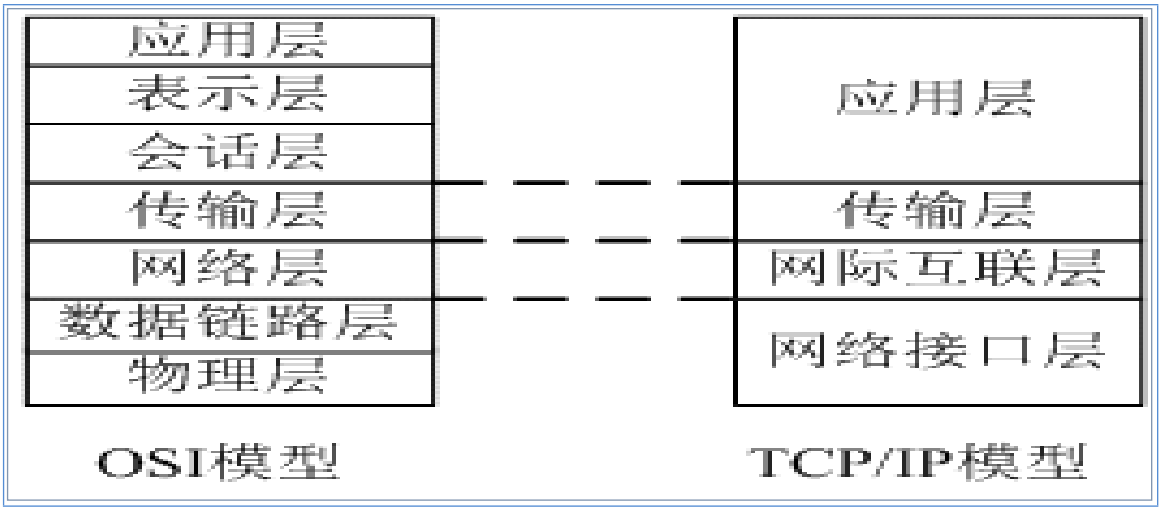


图 3-9 ISO7 层模型与 TCP/IP 四层对比

OSI 模型与 TCP/IP 模型协议功能实现对照表，如图 3-10 所示：

OSI 7层	每层功能	TCP/IP协议簇
应用层	文件传输，电子邮件，文件服务，虚拟终端	TFTP、HTTP、SNMP、FTP、SMTP、DNS、Telnet
表示层	数据格式化，代码转换，数据加密	没有协议
会话层	解除或建立与别的接点的联系	没有协议
传输层	提供端对端的接口	TCP、UDP
网络层	为数据包选择路由	IP、ICMP、OSPF、BGP、IGMP 、ARP、RARP
数据链路层	传输有地址的帧以及错误检测功能	SLIP、PPP、MTU
物理层	以二进制数据形式在物理媒体上传输数据	ISO2110、IEEE802、IEEE802.2

图 3-10 ISO7 层模型与 TCP/IP 层次功能对比

3.5 IP 地址及网络常识

互联网协议地址（Internet Protocol Address，IP），IP 地址是 IP 协议提供的一种统一的地址格式，它为互联网上的每一个网络和每一台主机分配一个逻辑地址，以此来屏蔽物理地址的差异。IP 地址被用来给 Internet 上的每个通信设备的一个编号，每台联网的 PC 上都需要有 IP 地址，这样才能正常通信。

IP 地址是一个 32 位的二进制数 ,通常被分割为 4 个“8 位二进制数”（即 4 个字节）。IP 地址通常用“点分十进制”表示成（a.b.c.d）的形式，其中，a,b,c,d 都是 0~255 之

间的十进制整数。

常见的 IP 地址，分为 IPv4 与 IPv6 两大类。IP 地址编址方案将 IP 地址空间划分为 A、B、C、D、E 五类，其中 A、B、C 是基本类，D、E 类作为多播和保留使用。

IPV4 有 4 段数字，每一段最大不超过 255。由于互联网的蓬勃发展，IP 位址的需求量愈来愈大，使得 IP 位址的发放愈趋严格，各项资料显示全球 IPv4 位址在 2011 年已经全部分发完毕。

地址空间的不足必将妨碍互联网的进一步发展。为了扩大地址空间，拟通过 IPv6 重新定义地址空间。IPv6 采用 128 位地址长度。在 IPv6 的设计过程中除了一劳永逸地解决了地址短缺问题以外，IPV6 的诞生可以给全球每一粒沙子配置一个 IP 地址，还考虑了在 IPv4 中解决不好的其它问题，如图 3-11 所示：

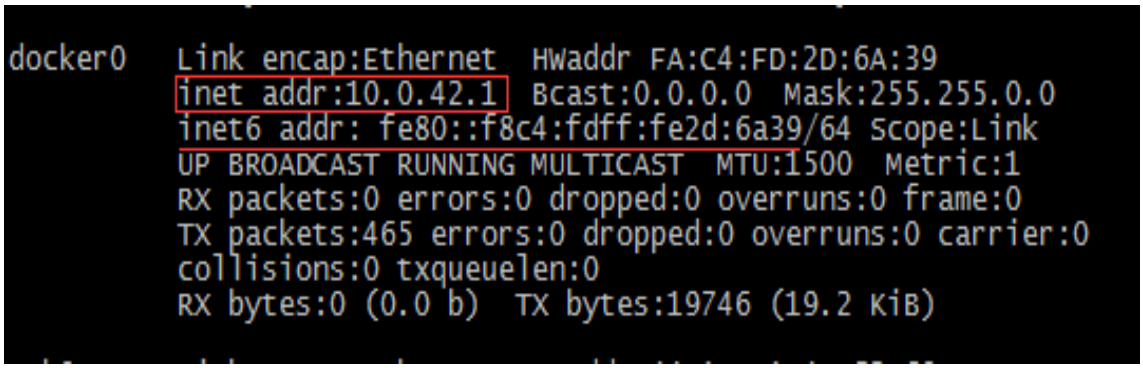


图 3-11 IPV4 与 IPV6 地址

3.5.1 IP 地址分类

IPV4 地址编址方案有 A、B、C、D、E 五类，其中 A、B、C 是基本类，D、E 类作为多播和保留使用，如下为分类详解：

1. A 类 IP 地址

一个 A 类 IP 地址是指，在 IP 地址的四段号码中，第一段号码为网络号码，剩下的三段号码为本地计算机的号码。如果用二进制表示 IP 地址的话，A 类 IP 地址就由 1 字节的网络地址和 3 字节主机地址组成，网络地址的最高位必须是“0”。A 类 IP 地址中网络的标识长度为 8 位，主机标识的长度为 24 位，A 类网络地址数量较少，有 126 个网络，每个网络可以容纳主机数达 1600 万台。

A 类 IP 地址 地址范围 1.0.0.0 到 127.255.255.255 （二进制表示为：00000001 00000000 00000000 00000000 - 01111110 11111111 11111111 11111111），最后一个为广播地址，A 类 IP 地址的子网掩码为 255.0.0.0，每个网络支持的最大主机数为 256 的 3 次方-2=16777214 台。

2. B 类 IP 地址

一个 B 类 IP 地址是指在 IP 地址的四段号码中，前两段号码为网络号码。如果用二进制表示 IP 地址的话，B 类 IP 地址就由 2 字节的网络地址和 2 字节主机地址组成，网络地址的最高位必须是“10”。

B 类 IP 地址中网络的标识长度为 16 位，主机标识的长度为 16 位，B 类网络地址适用于中等规模的网络，有 16384 个网络，每个网络所能容纳的计算机数为 6 万多台。

B 类 IP 地址地址范围 128.0.0.0-191.255.255.255（二进制表示为：10000000 00000000 00000000 00000000----10111111 11111111 11111111 11111111）。

最后一个是广播地址，B 类 IP 地址的子网掩码为 255.255.0.0，每个网络支持的最大主机数为 256 的 2 次方-2=65534 台。

3. C 类 IP 地址

一个 C 类 IP 地址是指在 IP 地址的四段号码中，前三段号码为网络号码，剩下的一段号码为本地计算机的号码。如果用二进制表示 IP 地址的话，C 类 IP 地址就由 3 字节的网络地址和 1 字节主机地址组成，网络地址的最高位必须是“110”。C 类 IP 地址中网络的标识长度为 24 位，主机标识的长度为 8 位，C 类网络地址数量较多，有 209 万余个网络。适用于小规模的局域网络，每个网络最多只能包含 254 台计算机。

C 类 IP 地址范围 192.0.0.0-223.255.255.255[3] （二进制表示为：11000000 00000000 00000000 - 11011111 11111111 11111111 11111111）。C 类 IP 地址的子网掩码为 255.255.255.0，每个网络支持的最大主机数为 256-2=254 台。

4. D 类 IP 地址

D 类 IP 地址又称之为多播地址(Multicast Address)，即组播地址。在以太网中，多播地址命名了一组应该在这个网络中应用接收到一个分组的站点。多播地址的最高位必须是“1110”，范围从 224.0.0.0 到 239.255.255.255。

5. 特殊的地址

每一个字节都为 0 的地址(“0.0.0.0”)表示当前主机，IP 地址中的每一个字节都为 1 的 IP 地址（“255．255．255．255”）是当前子网的广播地址，IP 地址中凡是以“11110”开头的 E 类 IP 地址都保留用于将来和实验使用。

IP 地址中不能以十进制“127”作为开头，而以数字 127．0．0．1 到 127．255．255．255 段的 IP 地址称为回环地址，用于回路测试，如：127.0.0.1 可以

代表本机 IP 地址，网络 ID 的第一个 8 位组也不能全置为“0”，全“0”表示本地网络。

3.5.2 子网掩码

子网掩码(Subnet Mask)又名网络掩码、地址掩码，它是一种用来指明一个 IP 地址的哪些位标识的是主机所在的子网，以及哪些位标识的是主机的位掩码。

通常的讲，子网掩码不能单独存在，它必须结合 IP 地址一起使用。子网掩码只有一个作用，就是将某个 IP 地址划分成网络地址和主机地址两部分。

子网掩码是一个 32 位地址，用于屏蔽 IP 地址的一部分以区别网络标识和主机标识，并说明该 IP 地址是在局域网上，还是在远程网上。

对于 A 类地址，默认的子网掩码是 255.0.0.0，而对于 B 类地址来说默认的子网掩码是 255.255.0.0；对于 C 类地址来说默认的子网掩码是 255.255.255.0。

互联网是由各种小型网络构成的，每个网络上都有许多主机，这样便构成了一个有层次的结构。IP 地址在设计时就考虑到地址分配的层次特点，将每个 IP 地址都分割成网络号和主机号两部分，以便于 IP 地址的寻址操作。

子网掩码的设定必须遵循一定的规则。与二进制 IP 地址相同，子网掩码由 1 和 0 组成，且 1 和 0 分别连续。子网掩码的长度也是 32 位，左边是网络位，用二进制数字“1”表示，1 的数目等于网络位的长度；右边是主机位，用二进制数字“0”表示，0 的数目等于主机位的长度。

3.5.3 网关地址

网关（Gateway）是一个网络连接到另一个网络的“关口”， 网关实质上是一个网

络通向其他网络的 IP 地址。主要用于不同网络传输数据。

例如我们电脑设备上网，如果是接入到同一个交换机，在交换机内部传输数据是不需要经过网关的，但是如果两台设备不在一个交换机网络，则需要在本机配置网关，内网服务器的数据通过网关，网关把数据转发到其他的网络的网关，直至找到对方的主机网络，然后返回数据。

3.5.4 MAC 地址

媒体访问控制（Media Access Control 或者 Medium Access Control，MAC），也即是物理地址、硬件地址，用来定义网络设备的位置。

在 OSI 模型中，第三层网络层负责 IP 地址，第二层数据链路层则负责 MAC 地址。因此一个主机会有一个 MAC 地址，而每个网络位置会有一个专属于它的 IP 地址。

IP 地址工作在 OSI 参考模型的第三层网络层。两者之间分工明确，默契合作，完成通信过程。IP 地址专注于网络层，将数据包从一个网络转发到另外一个网络；而 MAC 地址则专注于数据链路层，将一个数据帧从一个节点传送到相同链路的另一个节点。

IP 地址和 MAC 地址一般是成对出现。如果一台计算机要和网络中另一外计算机通信，那么这两台设备必须配置 IP 地址和 MAC 地址，而 MAC 地址是网卡出厂时设定的，这样配置的 IP 地址就和 MAC 地址形成了一种对应关系。

在数据通信时，IP 地址负责表示计算机的网络层地址，网络层设备（如路由器）根据 IP 地址来进行操作；MAC 地址负责表示计算机的数据链路层地址，数据链路层设备，根据 MAC 地址来进行操作。IP 和 MAC 地址这种映射关系是通过地址解析协议(Address

Resolution Protocol，ARP）来实现的。

3.6 Linux 系统配置 IP

Linux 操作系统安装完毕，那接下来如何让 Linux 操作系统能上外网呢？如下为 Linux 服务器配置 IP 的方法。

Linux 服务器网卡默认配置文件在/etc/sysconfig/network-scripts/下 ,命名的名称一般为:ifcfg-eth0 ifcfg-eth1 ，eth0 表示第一块网卡，eth1 表示第二块网卡，依次类推，例如 DELL R720 标配有 4 块千兆网卡，在系统显示的名称依次为：eth0、eth1、eth2、eth3。

修改服务器网卡 IP 地址命令为 vi /etc/sysconfig/network-scripts/ifcfg-eth0（注 CentOS7 网卡名 ifcfg-eno16777736）。vi 命令打开网卡配置文件，默认为 DHCP 方式，配置如下：

```
DEVICE=eth0

BOOTPROTO=dhcp

HWADDR=00:0c:29:52:c7:4e

ONBOOT=yes

TYPE=Ethernet
```

vi 命令打开网卡配置文件，修改 BOOTPROTO 为 DHCP 方式，同时添加 IPADDR、NETMASK、GATEWAY 信息如下：

```
DEVICE=eth0

BOOTPROTO=static

HWADDR=00:0c:29:52:c7:4e

ONBOOT=yes

TYPE=Ethernet

IPADDR=192.168.1.103

NETMASK=255.255.255.0

GATEWAY=192.168.1.1
```

服务器网卡配置文件，详细参数如下：

```
DEVICE=eth0     #物理设备名

ONBOOT=yes     # [yes|no]（重启网卡是否激活网卡设备）

BOOTPROTO=static #[none|static|bootp|dhcp]（不使用协议|静态分配|BOOTP 协

议|DHCP 协议）

TYPE=Ethernet                #网卡类型

IPADDR=192.168.1.103       #IP 地址

NETMASK=255.255.255.0    #子网掩码

GATEWAY=192.168.1.1       #网关地址
```

服务器网卡配置完毕后，重启网卡服务：/etc/init.d/network restart 即可。

然后查看 ip 地址，命令为：ifconfig 或者 ip addr show 查看当前服务器所有网卡的 IP

地址。

CentOS 7 Linux 中，如果没有 ifconfig 命令，可以用 ip addr list/show 查看，也可以安装 ifconfig 命令，需安装软件包 net-tools，命令如图 3-12 所示：

```
yum install net-tools -y
```

```
[root@localhost ~]# ifconfig
-bash: ifconfig: command not found
[root@localhost ~]# yum install net-tools -y
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
 * base: mirrors.btte.net
 * extras: mirror.bit.edu.cn
 * updates: mirrors.tuna.tsinghua.edu.cn
Resolving Dependencies
--> Running transaction check
---> Package net-tools.x86_64 0:2.0-0.17.20131004git.el7 will be installed
--> Finished Dependency Resolution

Dependencies Resolved
```

图 3-12 YUM 安装 net-tools 工具

3.7 Linux 系统配置 DNS

如上网卡 IP 地址配置完毕，如果服务器需上外网，还需配置域名解析地址（Domain Name System，DNS），DNS 主要用于将请求的域名转换为 IP 地址，DNS 地址配置方法如下：

修改 vi /etc/resolv.conf 文件，在文件中加入如下两条:

```
nameserver 202.106.0.20

nameserver 8.8.8.8
```

如上分别表示主 DNS 于备 DNS，DNS 配置完毕后，无需重启网络服务,DNS 是立即生效。

可以 ping -c 6 www.baidu.com 查看返回结果，如果有 IP 返回，则表示服务器 DNS 配置正确，如图 3-13 所示：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ping -c 6 www.baidu.com
PING www.a.shifen.com (103.235.46.39) 56(84) bytes of data.
64 bytes from 103.235.46.39: icmp_seq=1 ttl=47 time=150 ms
64 bytes from 103.235.46.39: icmp_seq=2 ttl=47 time=146 ms
64 bytes from 103.235.46.39: icmp_seq=3 ttl=47 time=145 ms
64 bytes from 103.235.46.39: icmp_seq=4 ttl=47 time=163 ms
64 bytes from 103.235.46.39: icmp_seq=5 ttl=47 time=158 ms
64 bytes from 103.235.46.39: icmp_seq=6 ttl=47 time=165 ms

--- www.a.shifen.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5171ms
rtt min/avg/max/mdev = 145.050/154.883/165.090/7.825 ms
[root@www-jfedu-net ~]#
```

图 3-13 ping 命令返回值

3.8 Linux 网卡名称命名

CentOS7 服务器，默认网卡名为 ifcfg-eno16777736，如果我们想改成 ifcfg-eth0，使用如下步骤即可：

（1） 编辑/etc/sysconfig/grub 文件，命令为 vi /etc/sysconfig/grub，在倒数第二行 quiet 后加入如下代码，并如图 3-14 所示：

```
net.ifnames=0 biosdevname=0
```

```
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR="$(sed 's, release .*$,,g' /etc/system-release)"
GRUB_DEFAULT=saved
GRUB_DISABLE_SUBMENU=true
GRUB_TERMINAL_OUTPUT="console"
GRUB_CMDLINE_LINUX="crashkernel=auto rhgb quiet net.ifnames=0 biosdevname=0"
GRUB_DISABLE_RECOVERY="true"
~
~
~
```

图 3-14 网卡配置 ifnames 设置

（2） 执行命令 grub2-mkconfig -o /boot/grub2/grub.cfg ,生成新的 grub.cfg 文件，如图 3-15 所示：

```
grub2-mkconfig -o /boot/grub2/grub.cfg
```

```
[root@localhost ~]#
[root@localhost ~]# grub2-mkconfig -o /boot/grub2/grub.cfg
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-3.10.0-327.el7.x86_64
Found initrd image: /boot/initramfs-3.10.0-327.el7.x86_64.img
Found linux image: /boot/vmlinuz-0-rescue-6fa6fd8dd8874932be8afaae5b91800f
Found initrd image: /boot/initramfs-0-rescue-6fa6fd8dd8874932be8afaae5b91800f
done
[root@localhost ~]#
```

图 3-15 生成新的 grub.cnf 文件

（3） 重命名网卡名称，执行命令 mv ifcfg-eno16777736 ifcfg-eth0，修改 ifcfg-eth0 文件中 DEVICE= eno16777736 为 DEVICE= eth0，如图 3-16 所示：

```
[root@localhost network-scripts]# ls
ifcfg-eth0   ifdown-isdn      ifup          ifup-plip      ifup-tunn
ifcfg-lo     ifdown-post      ifup-aliases  ifup-plusb     ifup-wire
ifdown       ifdown-ppp       ifup-bnep     ifup-post      init.ipv6
ifdown-bnep  ifdown-routes    ifup-eth      ifup-ppp       network-f
ifdown-eth   ifdown-sit       ifup-ib       ifup-routes    network-f
ifdown-ib    ifdown-Team      ifup-ippp     ifup-sit
ifdown-ippp  ifdown-TeamPort  ifup-ipv6     ifup-Team
ifdown-ipv6  ifdown-tunnel    ifup-isdn     ifup-TeamPort
[root@localhost network-scripts]# mv ifcfg-eno16777736 ifcfg-eth0
```

图 3-16 重命名网卡名称

（4） 重启服务器，并验证网卡名称是否为 eth0，Reboot 完后，如图 3-17 所示：

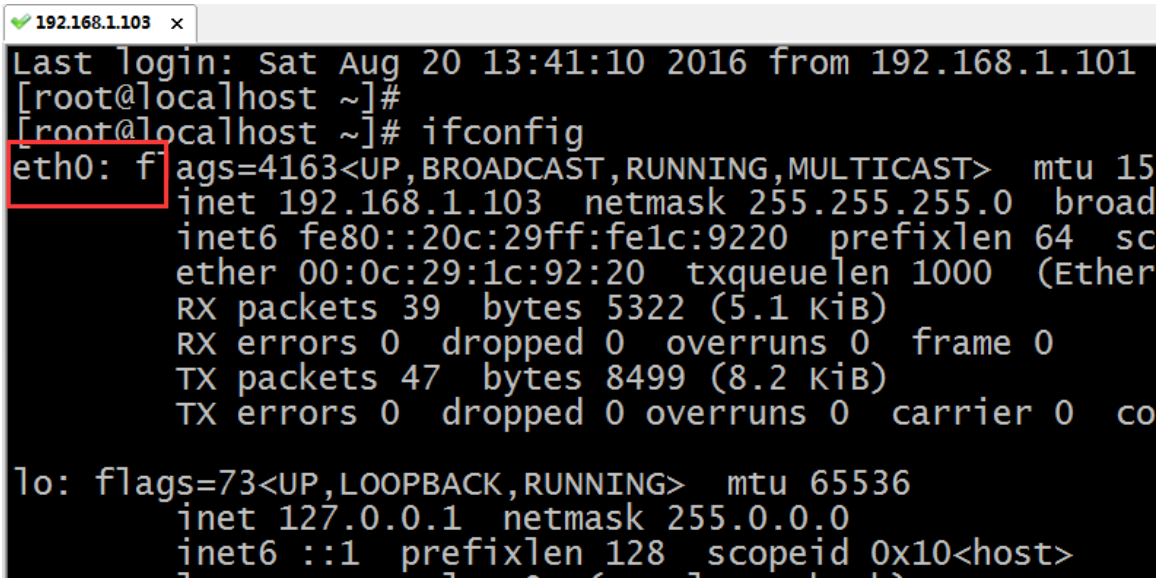


图 3-17 验证网卡设备名称

3.9 CentOS7 密码重置

修改 CentOS7 ROOT 密码非常简单，只需登录系统，执行命令 passwd 回车即可，但是如果忘记 ROOT，无法登录系统，该如何去重置 ROOT 用户的密码呢？如下为重置 ROOT 用户的密码的方法：

（1） Reboot 重启系统，系统启动进入欢迎界面，加载内核步骤时，按 e，然后选中“CentOS Linux （3.10.0-327.e17.x86_64）7 （Core）”，如图 3-18 所示：

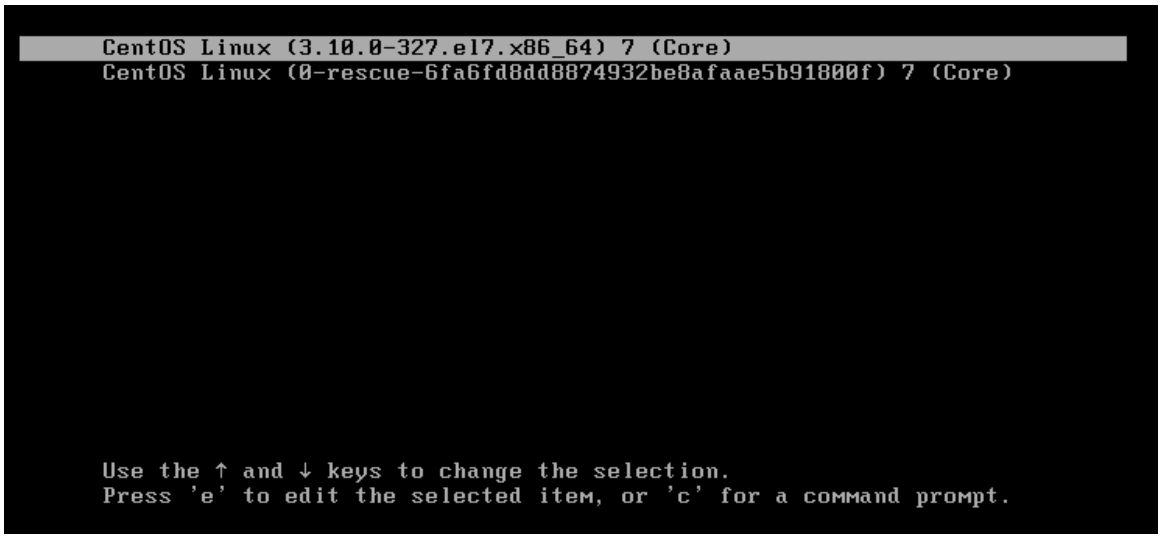


图 3-18 内核菜单选择界面

（2） 继续按 e 进入编辑模式，找到 ro crashkernel=auto xxx 项，将 ro 改成 rw init=/sysroot/bin/sh，如图 3-19 所示：

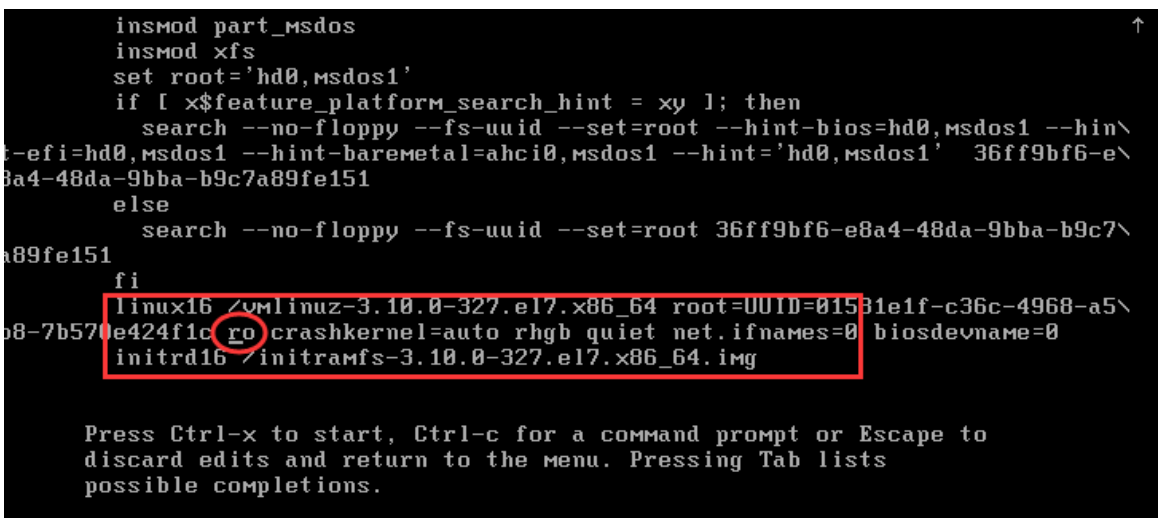


图 3-19 内核编辑界面

（3） 修改为后如图 3-20 所示：

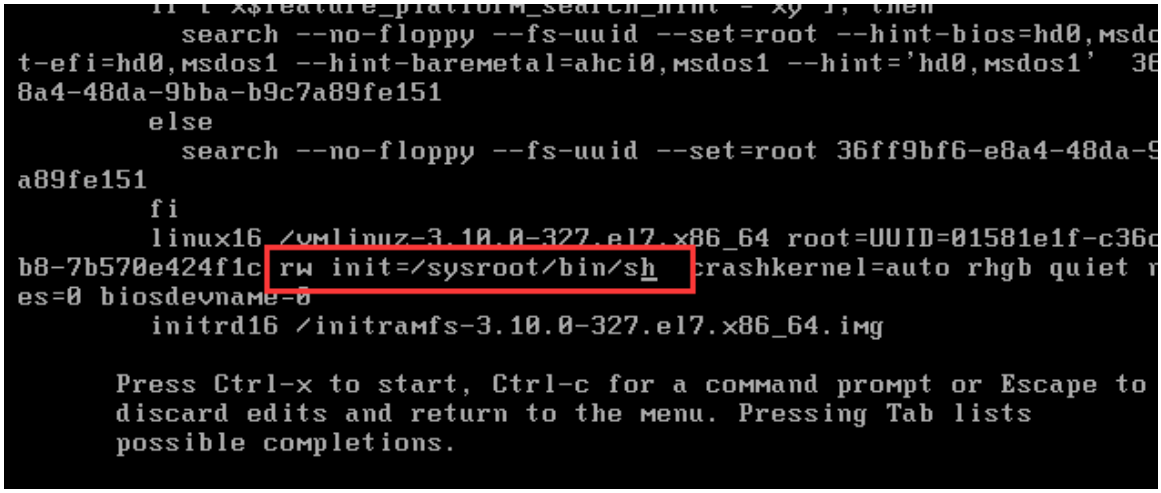


图 3-20 内核编辑界面

（4） 按 ctrl+x 按钮进入单用户模式，如图 3-21 所示：

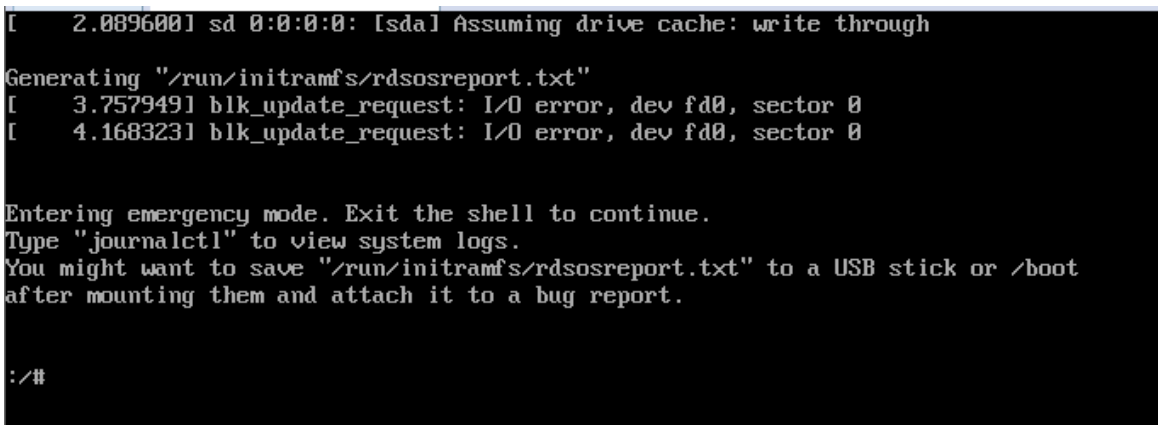


图 3-21 进入系统单用户模式

（5） 执行命令 chroot /sysroot 访问系统 ,并使用 passwd 修改 root 密码 ,如图 3-22 所示：

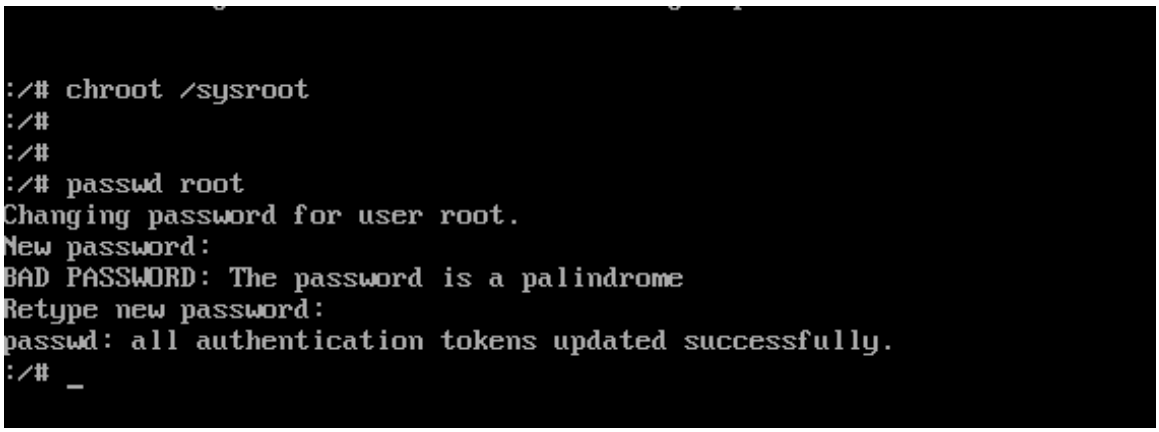


图 3-22 修改 ROOT 用户密码

（6） 更新系统信息，touch /.autorelabel，执行命令 touch /.autorelabel，在/目录下创建一个.autorelabel 文件，如果该文件存在，系统在重启时就会对整个文件系统进行 relabeling 重新标记，可以理解为对文件进行底层权限的控制和标记，如果 seLinux 属于 disabled 关闭状态则不需要执行这条命令，如图 3-23 所示：

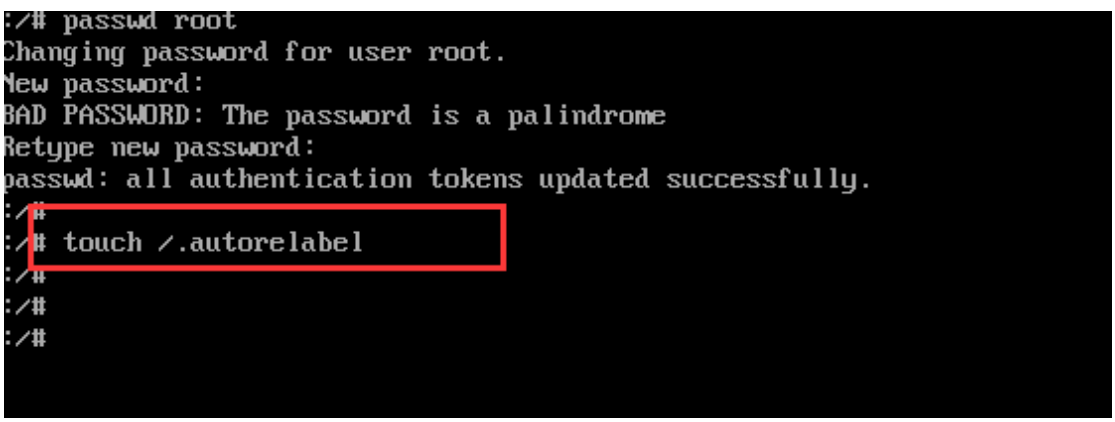


图 3-23 创建 autorelabel 文件

3.10 远程管理 Linux 服务器

系统安装完毕后，可以通过远程工具来连接到 Linux 服务器，远程连接服务器管理

的好处在于可以跨地区管理服务器，例如读者在北京，想管理的服务器在上海某 IDC 机房，通过远程管理后，不需要到 IDC 机房现场去操作，直接通过远程工具即可管理，与在现场的管理是一模一样。

远程管理 Linux 服务器要满足如下三个步骤：

（1） 服务器配置 IP 地址，如果服务器在公网，需配置公网 IP，如果服务器在内部局域网，可以直接配置内部私有 IP 即可；

（2） 服务器安装 SSHD 软件服务并启动该服务，几乎所有的 Linux 服务器系统安装完毕均会自动安装并启动 SSHD 服务，SSHD 服务监听 22 端口，关于 SSHD 服务、OpenSSH 及 SSH 协议后面章节会讲解；

（3） 在服务器中防火墙服务需要允许 22 端口对外开放 ,初学者可以临时关闭防火墙，CentOS6 Linux 关闭防火墙的命令 : service iptables stop，而 CentOS7 Linux 关闭防火墙的命令：systemctl stop firewalld.service。

常见的 Linux 远程管理工具包括：SecureCRT、Xshell、Putty、Xmanger 等工具。目前主流的远程管理 Linux 服务器工具为 SecureCRT ,官网 https://www.vandyke.com 下载并安装 SecureCRT，打开工具，点击左上角 quick connect 快速连接，弹出界面如图 3-24 所示，连接配置具体步骤如下：

- 协议（P）: 选择 SSH2
- 主机名（H）: 输入 Linux 服务器 IP 地址
- 端口（o）: 22

- 防火墙（F）：None
- 用户名（U）：root

单击下方的“连接”，会提示输入密码，输入 root 用户对应密码即可。

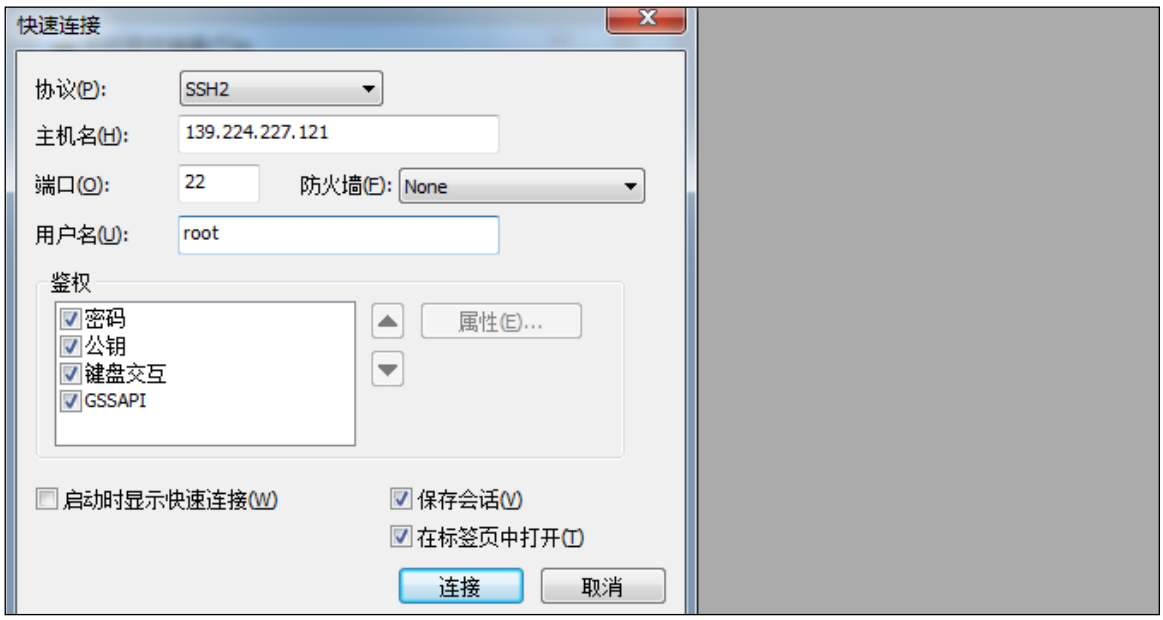


图 3-24 SecureCRT 远程 Linux 服务器

通过 SecureCRT 远程连接 Linux 服务器之后，会发现如图 3-25 所示界面，与服务器本地操作界面一样，在命令行可以执行命令，操作结果与在服务器现场操作是一样。

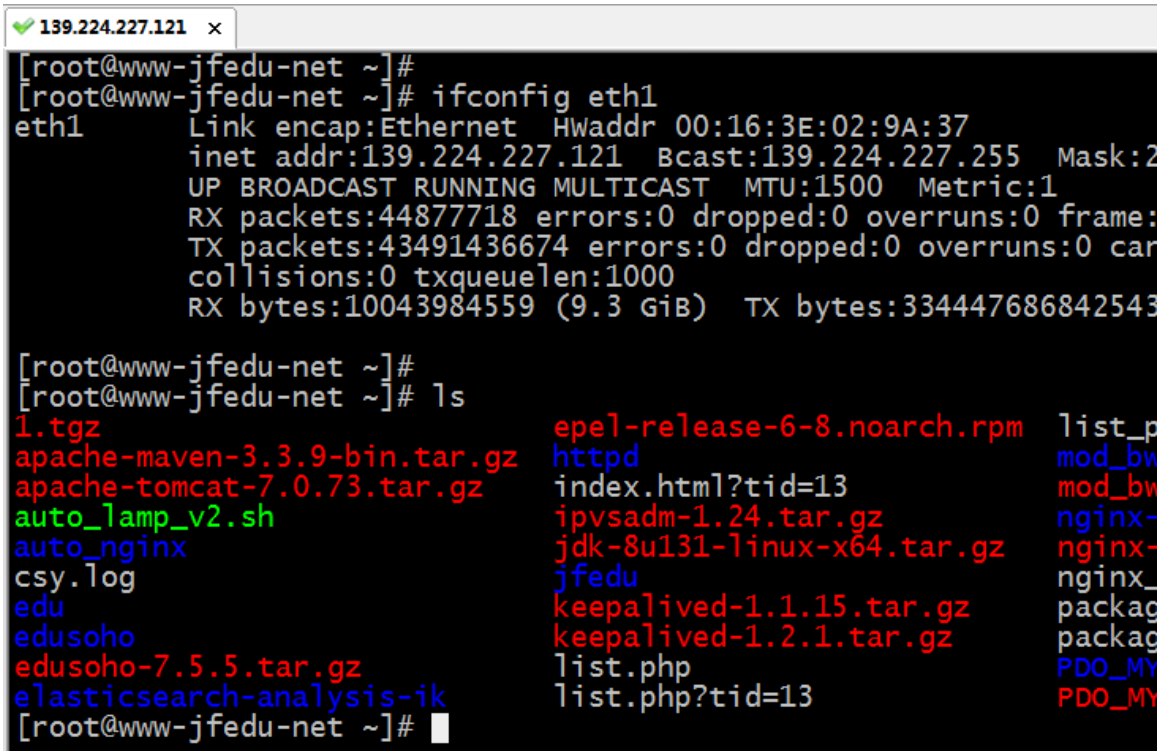


图 3-25 SecureCRT 远程 Linux 服务器

3.11 Linux 系统目录功能

通过以上知识的学习,读者已经能够独立安装并配置 Linux 服务器 IP 并远程连接 ,为了进一步学习 Linux，需熟练掌握 Linux 系统各个目录的功能。

Linux 主要树结构目录包括:/、/root、/home、/usr、/bin、/tmp、/sbin、/proc、/boot 等，如图 3-26 所示，为典型的 Linux 目录结构如下：

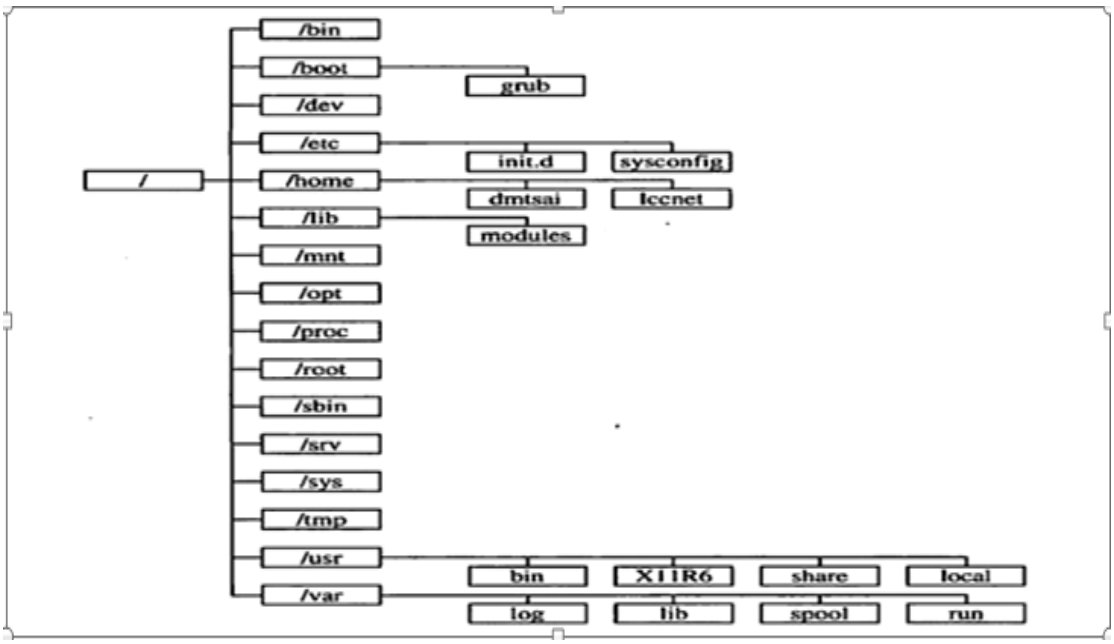


图 3-26 Linux 目录树形结构

Linux 系统中常见目录功能如下：

- / 根目录；
- /bin 存放必要的命令；
- /boot 存放内核以及启动所需的文件；
- /dev 存放硬件设备文件；
- /etc 存放系统配置文件；
- /home 普通用户的宿主目录，用户数据存放在其主目录中；
- /lib|lib64 存放必要的运行库；
- /mnt 存放临时的映射文件系统，通常用来挂载使用；
- /proc 存放存储进程和系统信息；

- /root 超级用户的主目录；
- /sbin 存放系统管理程序；
- /tmp 存放临时文件；
- /usr 存放应用程序，命令程序文件、程序库、手册和其它文档；
- /var 系统默认日志存放目录。

第4章 Linux 必备命令

Linux 系统启动默认为字符界面，一般不会启动图形界面，所以对命令行的熟练程度能更加方便、高效的管理 Linux 系统。

本章向读者介绍 Linux 系统必备命令各项参数及功能场景，Linux 常见命令包括：cd、ls、pwd、mkdir、rm、cp、mv、touch、cat、head、tail、chmod、vim 等。

4.1 Linux 命令集

初学者完成 Linux 系统安装以后，学习 Linux 操作系统必备的指令，基于 Linux 指令管理 Linux 操作系统，必备 Linux 指令有哪些？

■ 基础命令相关一：

Cd、ls、pwd、help、man、if、for、while、case、select、read、test、ansible、iptables、firewall-cmd、salt、mv、cut、uniq、sort、wc、source、sestatus、setenforce；

- 基础命令相关二：

Date、ntpdate、crontab、rsync、ssh、scp、nohup、sh、bash、hostname、hostnamectl、source、ulimit、export、env、set、at、dir、db_load、diff、dmsetup、declare；

- 用户权限相关：

Useradd、userdel、usermod、groupadd、groupmod、groupdel、Chmod、chown、chgrp、umask、chattr、lsattr、id、who、whoami、last、su、sudo、w、chpasswd、chroot；

- 文件管理相关：

Touch、mkdir、rm、rmdi、vi、vim、cat、head、tail、less、more、find、sed、grep、awk、echo、ln、stat、file；

- 软件资源管理：

Rpm、yum、tar、unzip、zip、gzip、wget、curl、rz、sz、jar、apt-get、bzip2、service、systemctl、make、cmake、chkconfig；

- 系统资源管理：

Fdisk、mount、umount、mkfs.ext4、fsck.ext4、parted、lvm、dd、du、df、top、iftop、free、w、uptime、iostat、vmstat、iotop、ps、netstat、lsof、ss、sar；

- 网络管理相关：

Ping、ifconfig、ip addr、ifup、ifdown、nmcli、route、nslookup、traceroute、

dig、tcpdump、nmap、brctl、ethtool、setup、arp、ab、iperf；

■ Linux 系统开关机：

Init、reboot、shutdown、halt、poweroff、runlevel、login、logout、exit；

4.2 cd 命令详解

cd 命令主要用于目录切换，例如：cd /home 切换至/home 目录，cd /root 表示切换至/root 目录 ；cd ../切换至上一级目录；cd ./切换至当前目录。

其中.和..可以理解为相对路径 ,例如 cd ./test 表示以当前目录为参考 ,表示相对于当前，而 cd /home/test 表示完整的路径，理解为绝对路径），如图 4-1 所示：

```
[root@www-jfedu-net tmp]# cd
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# cd /home/
[root@www-jfedu-net home]#
[root@www-jfedu-net home]# cd /root/
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# cd /
[root@www-jfedu-net /]#
[root@www-jfedu-net /]# cd /usr/local/sbin/
[root@www-jfedu-net sbin]#
[root@www-jfedu-net sbin]# cd /root/
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# cd /opt/
[root@www-jfedu-net opt]#
[root@www-jfedu-net opt]#
```

图 4-1 Linux cd 命令操作

4.3 ls 命令详解

ls 命令主要用于浏览目录下的文件或者文件夹，使用方法参考：ls ./ 查看当前目

录所有的文件和目录，ls -a 查看所有的文件，包括隐藏文件,以.开头的文件，常用参数详解如下:

-a, --all	不隐藏任何以. 开始的项目；
-A, --almost-all	列出除. 及.. 以外的任何项目；
--author	与-l 同时使用时列出每个文件的作者；
-b, --escape	以八进制溢出序列表示不可打印的字符；
--block-size=大小	块以指定大小的字节为单位；
-B, --ignore-backups	不列出任何以"~"字符结束的项目；
-d, --directory	当遇到目录时列出目录本身而非目录内的文件；
-D, --dired	产生适合 Emacs 的 dired 模式使用的结果；
-f	不进行排序，-aU 选项生效，-lst 选项失效；
-i, --inode	显示每个文件的 inode 号；
-I, --ignore=PATTERN	不显示任何符合指定 shell PATTERN 的项目；
-k	即--block-size=1K；
-l	使用较长格式列出信息；
-n, --numeric-uid-gid	类似 -l，但列出 UID 及 GID 号；
-N, --literal	输出未经处理的项目名称 (如不特别处理控制字符) ；
-r, --reverse	排序时保留顺序；
-R, --recursive	递归显示子目录；

-s, --size	以块数形式显示每个文件分配的尺寸；
-S	根据文件大小排序；
-t	根据修改时间排序；
-u	同-lt 一起使用：按照访问时间排序并显示； 同-l 一起使用：显示访问时间并按文件名排序； 其他：按照访问时间排序；
-U	不进行排序；按照目录顺序列出项目；
-v	在文本中进行数字(版本)的自然排序。

4.4 pwd 命令详解

pwd 命令主要用于显示或者查看当前所在的目录路径，如图 4-2 所示：

```
[root@www-jfedu-net ~]# cd
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# pwd
/root
[root@www-jfedu-net ~]# cd /home/
[root@www-jfedu-net home]#
[root@www-jfedu-net home]# pwd
/home
[root@www-jfedu-net home]#
[root@www-jfedu-net home]# cd /tmp/
[root@www-jfedu-net tmp]#
[root@www-jfedu-net tmp]# pwd
/tmp
[root@www-jfedu-net tmp]#
[root@www-jfedu-net tmp]#
```

图 4-2 pwd 命令查看当前目录

4.5 mkdir 命令详解

mkdir 命令主要用于创建目录，用法 mkdir dirname，命令后接目录的名称，常用参数详解如下:

用法：mkdir [选项]... 目录；若指定目录不存在则创建目录；

长选项必须使用的参数对于短选项时也是必需使用的；

-m, --mode=模式 设置权限模式(类似 chmod)，而不是 rwxrwxrwx 减 umask；

-p, --parents 需要时创建目标目录的上层目录，但即使这些目录已存在也不当作错误处理；

-v, --verbose 每次创建新目录都显示信息；

-Z, --context=CTX 将每个创建的目录的 SELinux 安全环境设置为 CTX；

--help 显示此帮助信息并退出；

--version 显示版本信息并退出。

4.6 **rm** 命令**详解**

rm 命令主要用于删除文件或者目录，用法 rm –rf test.txt (-r 表示递归，-f 表示强制)，常用参数详解如下:

用法：rm [选项]... 文件...删除 (unlink) 文件。

-f, --force 强制删除。忽略不存在的文件，不提示确认；

-i 在删除前需要确认；

-I 在删除超过三个文件或者递归删除前要求确认。此选项比-i 提示内容更少，但同样可以阻止大多数错误发生；

-r, -R, --recursive 递归删除目录及其内容；

-v, --verbose 详细显示进行的步骤；

--help 显示此帮助信息并退出；

--version 显示版本信息并退出；

默认时，rm 不会删除目录，使用--recursive(-r 或-R)选项可删除每个给定的目录，以及其下所有的内容；

要删除第一个字符为"-"的文件（例如"-foo"），请使用以下方法之一：

rm -- -foo

rm ./-foo

4.7 cp 命令详解

cp 命令主要用于拷贝文件，用法,cp old.txt /tmp/new.txt ，常用来备份，如果拷贝目录需要加-r 参数，常用参数详解如下:

用法：cp [选项]... [-T] 源文件 目标文件

或：cp [选项]... 源文件... 目录

或：cp [选项]... -t 目录 源文件...

将源文件复制至目标文件，或将多个源文件复制至目标目录。

长选项必须使用的参数对于短选项时也是必需使用的。

-a, --archive 等于-dR --preserve=all；

--backup[=CONTROL 为每个已存在的目标文件创建备份；

-b 类似--backup 但不接受参数；

--copy-contents 在递归处理是复制特殊文件内容；

-d 等于--no-dereference --preserve=links；

-f, --force 如果目标文件无法打开则将其移除并重试(当 -n 选项；

存在时则不需再选此项)；

-i, --interactive 覆盖前询问(使前面的 -n 选项失效)；

-H 跟随源文件中的命令行符号链接；

-l, --link 链接文件而不复制；

-L, --dereference 总是跟随符号链接；

-n, --no-clobber 不要覆盖已存在的文件(使前面的 -i 选项失效)；

-P, --no-dereference 不跟随源文件中的符号链接；

-p 等于--preserve=模式,所有权,时间戳；

--preserve[=属性列表 保持指定的属性(默认：模式,所有权,时间戳)，

如果；

可能保持附加属性：环境、链接、xattr 等；

-c same as --preserve=context；

--sno-preserve=属性列表 不保留指定的文件属性；

--parents 复制前在目标目录创建来源文件路径中的所有目录；

-R, -r, --recursive 递归复制目录及其子目录内的所有内容。

4.8 mv 命令详解

mv 命令主要用于重命名或者移动文件或者目录，用法, mv old.txt new.txt，常用参数详解如下:

用法：mv [选项]... [-T] 源文件 目标文件；

或：mv [选项]... 源文件... 目录；

或：mv [选项]... -t 目录 源文件；

将源文件重命名为目标文件，或将源文件移动至指定目录。长选项必须使用的参数对于短选项时也是必需使用的。

--backup[=CONTROL] 为每个已存在的目标文件创建备份；

-b 类似--backup 但不接受参数；

-f, --force 覆盖前不询问；

-i, --interactive	覆盖前询问；
-n, --no-clobber	不覆盖已存在文件，如果您指定了-i、-f、-n 中的多个，仅最后一个生效；
--strip-trailing-slashes	去掉每个源文件参数尾部的斜线；
-S, --suffix=SUFFIX	替换常用的备份文件后缀；
-t, --target-directory=DIRECTORY	将所有参数指定的源文件或目录；移动至 指定目录；
-T, --no-target-directory	将目标文件视作普通文件处理；
-u, --update	只在源文件文件比目标文件新，或目标文件；不存在时才进行移动；
-v, --verbose	详细显示进行的步骤；
--help	显示此帮助信息并退出；
--version	显示版本信息并退出。

4.9 touch 命令详解

touch 命令主要用于创建普通文件，用法为 touch test.txt，如果文件存在，则表示修改当前文件时间，常用参数详解如下:

用法：touch [选项]... 文件...

将每个文件的访问时间和修改时间改为当前时间；

不存在的文件将会被创建为空文件，除非使用-c 或-h 选项；

如果文件名为"-"则特殊处理，更改与标准输出相关的文件的访问时间；

长选项必须使用的参数对于短选项时也是必需使用的；

-a	只更改访问时间；
-c, --no-create	不创建任何文件；
-d, --date=字符串	使用指定字符串表示时间而非当前时间；
-f	(忽略)；
-h, --no-dereference	会影响符号链接本身，而非符号链接所指示的目的地；
	(当系统支持更改符号链接的所有者时，此选项才有用)；
-m	只更改修改时间；
-r, --reference=文件	使用指定文件的时间属性而非当前时间；
-t STAMP	使用[[CC]YY]MMDDhhmm[.ss] 格式的时间而非当前时间；
--time=WORD	使用 WORD 指定的时间：access、atime、use 都等于-a；
	选项的效果，而 modify、mtime 等于-m 选项的效果；
--help	显示此帮助信息并退出；
--version	显示版本信息并退出。

4.10 cat 命令详解

cat 命令主要用于查看文件内容，用法 cat test.txt 可以查看 test.txt 内容，常用参数详解如下:

```
用法：cat [选项]... [文件]...
将[文件]或标准输入组合输出到标准输出。
-A, --show-all                  等于-vET；
-b, --number-nonblank           对非空输出行编号；
-e                              等于-vE；
-E, --show-ends                 在每行结束处显示"$"；
-n, --number                    对输出的所有行编号；
-s, --squeeze-blank             不输出多行空行；
-t                              与-vT 等价；
-T, --show-tabs                 将跳格字符显示为^I；
-u                              (被忽略)；
-v, --show-nonprinting          使用^ 和 M- 引用，除了 LFD 和 TAB 之外；
--help                          显示此帮助信息并退出；
--version                       显示版本信息并退出。
```

cat 还有一种用法，cat ...EOF...EOF，表示追加内容至/tmp/test.txt 文件中，如下:

```
cat >>/tmp/test.txt<<EOF
My Name is JFEDU.NET
I am From Bei jing.
EOF
```

cat test.txt |more 分页显示 text 内容，|符号是管道符，用于把|前的输出作为后面命令的输入。More 命令常用于分页查看某文件或者内容。

4.11 head 命令详解

head 命令主要用于查看文件内容，通常查看文件前 10 行，head -10 /var/log/messages 可以查看该文件前 10 行的内容，常用参数详解如下:

```
用法：head [选项]... [文件]...
将每个指定文件的头 10 行显示到标准输出。
如果指定了多于一个文件，在每一段输出前会给出文件名作为文件头。
如果不指定文件，或者文件为"-"，则从标准输入读取数据，长选项必须使用的参数对于
短选项时也是必需使用的；
-q, --quiet, --silent                    不显示包含给定文件名的文件头；
-v, --verbose                            总是显示包含给定文件名的文件头；
--help                                   显示此帮助信息并退出；
--version                                显示版本信息并退出；
```

-c, --bytes=[-]K 显示每个文件的前 K 字节内容，如果附加"-"参数，则除了每个文件的最后 K 字节数据外显示剩余全部内容；

-n, --lines=[-]K 显示每个文件的前 K 行内容，如果附加"-"参数，则除了每个文件的最后 K 行外显示剩余全部内容。

4.12 tail 命令详解

tail 命令主要用于查看文件内容，通常查看末尾 10 行，tail –fn 100 /var/log/messages 可以实时查看该文件末尾 100 行的内容，常用参数详解如下:

用法：tail [选项]... [文件]...

显示每个指定文件的最后 10 行到标准输出。

若指定了多于一个文件，程序会在每段输出的开始添加相应文件名作为头。

如果不指定文件或文件为"-" ，则从标准输入读取数据。

长选项必须使用的参数对于短选项时也是必需使用的。

-n, --lines=K 输出的总行数，默认为 10 行；

-q, --quiet, --silent 不输出给出文件名的头；

--help 显示此帮助信息并退出；

--version 显示版本信息并退出；

-f, --follow[={name|descriptor}] 即时输出文件变化后追加的数据；

-f, --follow 等于--follow=descriptor

-F 即--follow=name –retry

-c, --bytes=K 输出最后 K 字节；另外，使用-c +K 从每个文件的第 K 字节输出。

4.13 echo 命令详解

echo 命令主要用于打印字符或者回显，例如输入 echo ok，会显示 ok， echo ok > test.txt 则会把 ok 字符覆盖 test.txt 内容。>表示覆盖，原内容被覆盖，>>表示追加，原内容不变。

例如 echo ok >> test.txt,表示向 test.txt 文件追加 OK 字符，不覆盖原文件里的内容，常用参数详解如下:

使用-e 扩展参数选项时，与如下参数一起使用，有不同含义，例如：

\a 发出警告声

\b 删除前一个字符

\c 最后不加上换行符号；

\f 换行但光标仍旧停留在原来的位置；

\n 换行且光标移至行首；

\r 光标移至行首，但不换行；

\t 插入 tab； \v 与\f 相同；

\\ 插入\字符；

```
\033[30m 黑色字 \033[0m

\033[31m 红色字 \033[0m

\033[32m 绿色字 \033[0m

\033[33m 黄色字 \033[0m

\033[34m 蓝色字 \033[0m

\033[35m 紫色字 \033[0m

\033[36m 天蓝字 \033[0m

\033[37m 白色字 \033[0m

\033[40;37m 黑底白字 \033[0m

\033[41;37m 红底白字 \033[0m

\033[42;37m 绿底白字 \033[0m

\033[43;37m 黄底白字 \033[0m

\033[44;37m 蓝底白字 \033[0m

\033[45;37m 紫底白字 \033[0m

\033[46;37m 天蓝底白字 \033[0m

\033[47;30m 白底黑字 \033[0m
```

echo 颜色打印扩展，auto_lamp_v2.sh 内容如下：

```
echo -e "\033[36mPlease Select Install Menu follow:\033[0m"

echo -e "\033[32m1)Install Apache Server\033[1m"
```

```
echo "2)Install MySQL Server"

echo "3)Install PHP Server"

echo "4)Configuration index.php and start LAMP server"

echo -e "\033[31mUsage: { /bin/sh $0 1|2|3|4|help}\033[0m"
```

执行结果如图 4-3 所示：

```
[root@www-jfedu-net ~]# sh auto_lamp_v2.sh
------------------------------

Please Select Install Menu follow:
1)Install Apache Server
2)Install MySQL Server
3)Install PHP Server
4)Configuration index.php and start LAMP server
Usage: { /bin/sh auto_lamp_v2.sh 1|2|3|4|help}
[root@www-jfedu-net ~]#
```

图 4-3 echo –e 颜色打印

4.14 df 命令详解

df 命令常用于磁盘分区查询，常用命令 df –h，查看磁盘分区信息，常用参数详解如下:

```
用法：df [选项]... [文件]...

显示每个文件所在的文件系统的信息，默认是显示所有文件系统。

长选项必须使用的参数对于短选项时也是必需使用的。

-a, --all                    显示所有文件系统的使用情况，包括虚拟文件系统；
```

-B, --block-size=SIZE	使用字节大小块；
-h, --human-readable	以人们可读的形式显示大小；
-H, --si	同-h，但是强制使用 1000 而不是 1024；
-i, --inodes	显示 inode 信息而非块使用量；
-k	即--block-size=1K；
-l, --local	只显示本机的文件系统；
--no-sync	取得使用量数据前不进行同步动作(默认)；
-P, --portability	使用 POSIX 兼容的输出格式；
--sync	取得使用量数据前先进行同步动作；
-t, --type=类型	只显示指定文件系统为指定类型的信息；
-T, --print-type	显示文件系统类型；
-x, --exclude-type=类型	只显示文件系统不是指定类型信息；
--help	显示帮助信息并退出；
--version	显示版本信息并退出。

4.15 mount 命令详解

挂接命令(mount)

首先，介绍一下挂接(mount)命令的使用方法，mount 命令参数非常多，这里主要讲一下今天我们要用到的。

命令格式：

mount [-t vfstype] [-o options] device dir

其中：

1.-t vfstype 指定文件系统的类型，通常不必指定。mount 会自动选择正确的类型。常用类型有：

光盘或光盘镜像：iso9660

DOS fat16 文件系统：msdos

Windows 9x fat32 文件系统：vfat

Windows NT ntfs 文件系统：ntfs

Mount Windows 文件网络共享：smbfs

UNIX(LINUX) 文件网络共享：nfs

2.-o options 主要用来描述设备或档案的挂接方式。常用的参数有：

loop：用来把一个文件当成硬盘分区挂接上系统

ro：采用只读方式挂接设备

rw：采用读写方式挂接设备

iocharset：指定访问文件系统所用字符集

3.device 要挂接(mount)的设备。

4.dir 设备在系统上的挂接点(mount point)。

挂接光盘镜像文件

1、从光盘制作光盘镜像文件。将光盘放入光驱，执行下面的命令。

#cp /dev/cdrom /home/sunky/mydisk.iso 或

#dd if=/dev/cdrom of=/home/sunky/mydisk.iso

注：执行上面的任何一条命令都可将当前光驱里的光盘制作成光盘镜像文件 /home/sunky/mydisk.iso

2、将文件和目录制作成光盘镜像文件，执行下面的命令。

#mkisofs -r -J -V mydisk -o /home/sunky/mydisk.iso /home/sunky/ mydir

注：这条命令将/home/sunky/mydir 目录下所有的目录和文件制作成光盘镜像文件 /home/sunky/mydisk.iso，光盘卷标为：mydisk

3、光盘镜像文件的挂接(mount)

#mkdir /mnt/vcdrom

注：建立一个目录用来作挂接点(mount point)

#mount -o loop -t iso9660 /home/sunky/mydisk.iso /mnt/vcdrom

注：使用/mnt/vcdrom 就可以访问盘镜像文件 mydisk.iso 里的所有文件了。

挂接移动硬盘

对 linux 系统而言，USB 接口的移动硬盘是当作 SCSI 设备对待的。插入移动硬盘之前，应先用 fdisk –l 或 more /proc/partitions 查看系统的硬盘和硬盘分区情况。

[root at pldyrouter /]# fdisk -l

接好移动硬盘后，再用 fdisk –l 或 more /proc/partitions 查看系统的硬盘和硬盘分区

情况.应该可以发现多了一个 SCSI 硬盘/dev/sdc 和它的两个磁盘分区/dev /sdc1?、/dev/sdc2,其中/dev/sdc5 是/dev/sdc2 分区的逻辑分区。我们可以使用下面的命令挂接/dev/sdc1 和 /dev/sdc5。

#mkdir -p /mnt/usbhd1

#mkdir -p /mnt/usbhd2

注：建立目录用来作挂接点(mount point)

#mount -t ntfs /dev/sdc1 /mnt/usbhd1

#mount -t vfat /dev/sdc5 /mnt/usbhd2

注 : 对 ntfs 格式的磁盘分区应使用-t ntfs 参数，对 fat32 格式的磁盘分区应使用-t vfat 参数。若汉字文件名显示为乱码或不显示，可以使用下面的命令格式。

#mount -t ntfs -o iocharset=cp936 /dev/sdc1 /mnt/usbhd1

#mount -t vfat -o iocharset=cp936 /dev/sdc5 /mnt/usbhd2

linux 系统下使用 fdisk 分区命令和 mkfs 文件系统创建命令可以将移动硬盘的分区制作成 linux 系统所特有的 ext2、ext3 格式。这样，在 linux 下使用就更方便了。使用下面的命令直接挂接即可。

#mount /dev/sdc1 /mnt/usbhd1

挂接 U 盘

和 USB 接口的移动硬盘一样对 linux 系统而言 U 盘也是当作 SCSI 设备对待的。使用方法和移动硬盘完全一样。插入 U 盘之前，应先用 fdisk –l 或 more /proc/partitions 查

看系统的硬盘和硬盘分区情况。

[root at pldyrouter root]# fdisk -l

插入 U 盘后 ,再用 fdisk –l 或 more /proc/partitions 查看系统的硬盘和硬盘分区情况。

[root at pldyrouter root]# fdisk -l

系统多了一个 SCSI 硬盘/dev/sdd 和一个磁盘分区/dev/sdd1,/dev/sdd1 就是我们要挂接的 U 盘。

#mkdir -p /mnt/usb

注：建立一个目录用来作挂接点(mount point)

#mount -t vfat /dev/sdd1 /mnt/usb

注:现在可以通过/mnt/usb 来访问 U 盘了，若汉字文件名显示为乱码或不显示，可以使用下面的命令。

#mount -t vfat -o iocharset=cp936 /dev/sdd1 /mnt/usb

挂接 Windows 文件共享

Windows 网络共享的核心是 SMB/CIFS ,在 linux 下要挂接(mount)windows 的磁盘共享，就必须安装和使用 samba 软件包。现在流行的 linux 发行版绝大多数已经包含了 samba 软件包，如果安装 linux 系统时未安装 samba 请首先安装 samba。当然也可以到 http://www.samba.org 网站下载.....新的版本是 3.0.10 版。

当 windows 系统共享设置好以后，就可以在 linux 客户端挂接(mount)了，具体操作如下：

mkdir –p /mnt/samba

注：建立一个目录用来作挂接点(mount point)

mount -t smbfs -o username=administrator,password=pldy123 //10.140.133.23/c$ /mnt/samba

注:administrator 和 pldy123 是 ip 地址为 10.140.133.23 windows 计算机的一个用户名和密码，c$是这台计算机的一个磁盘共享

如此就可以在 linux 系统上通过/mnt/samba 来访问 windows 系统磁盘上的文件了。以上操作在 redhat as server 3、redflag server 4.1、suse server 9 以及 windows NT 4.0、windows 2000、windows xp、windows 2003 环境下测试通过。

挂接 UNIX 系统 NFS 文件共享

类似于 windows 的网络共享，UNIX(Linux)系统也有自己的网络共享，那就是 NFS(网络文件系统)，下面我们就以 SUN Solaris2.8 和 REDHAT as server 3 为例简单介绍一下在 linux 下如何 mount nfs 网络共享。

在 linux 客户端挂接(mount)NFS 磁盘共享之前，必须先配置好 NFS 服务端。

1、Solaris 系统 NFS 服务端配置方法如下：

(1)修改 /etc/dfs/dfstab，增加共享目录

share -F nfs -o rw /export/home/sunky

(2)启动 nfs 服务

/etc/init.d/nfs.server start

(3)NFS 服务启动以后，也可以使用下面的命令增加新的共享

share /export/home/sunky1

share /export/home/sunky2

注：/export/home/sunky 和/export/home/sunky1 是准备共享的目录

2、linux 系统 NFS 服务端配置方法如下：

(1)修改 /etc/exports,增加共享目录

/export/home/sunky 10.140.133.23(rw)

/export/home/sunky1 *(rw)

/export/home/sunky2 linux-client(rw)

注：/export/home/ 目录下的 sunky、sunky1、sunky2 是准备共享的目录，10.140.133.23、*、 linux-client 是被允许挂接此共享 linux 客户机的 IP 地址或主机名。如果要使用主机名 linux-client 必须在服务端主机 /etc/hosts 文件里增加 linux-client 主机 ip 定义。格式如下：

10.140.133.23 linux-client

(2)启动与停止 NFS 服务

/etc/rc.d/init.d/portmap start (在 REDHAT 中 PORTMAP 是默认启动的)

/etc/rc.d/init.d/nfs start 启动 NFS 服务

/etc/rc.d/init.d/nfs stop 停止 NFS 服务

注：若修改/etc/export 文件增加新的共享，应先停止 NFS 服务，再启动 NFS 服务方能

使新增加的共享起作用。使用命令 exportfs -rv 也可以达到同样的效果。

3、linux 客户端挂接(mount)其他 linux 系统或 UNIX 系统的 NFS 共享

mkdir –p /mnt/nfs

注：建立一个目录用来作挂接点(mount point)

#mount -t nfs -o rw 10.140.133.9:/export/home/sunky /mnt/nfs

注：这里我们假设 10.140.133.9 是 NFS 服务端的主机 IP 地址，当然这里也可以使用主机名，但必须在本机/etc/hosts 文件里增加服务端 ip 定义。/export/home/sunky 为服务端共享的目录。

如此就可以在 linux 客户端通过/mnt/nfs 来访问其它 linux 系统或 UNIX 系统以 NFS 方式共享出来的文件了。以上操作在 redhat as server 3、redflag server4.1、suse server 9 以及 Solaris 7、Solaris 8、Solaris 9 for x86&sparc 环境下测试通过

linux 加载光驱

（1）使用光驱前，要先 mount 一下：# mount /dev/cdrom /mnt/cdrom，然后您就可以进入/mnt/cdrom 目录下读取光盘内容了；

（2）当您想退出光盘时，须使用 umout 命令，否则光驱就会一直处于死锁状态：# umount /mnt/cdrom。

4.16 本章小结

通过对本章内容的学习，读者对 Linux 操作系统引导有了进一步的理解，能够快速

解决 Linux 启动过程中的故障，同时学习了 CentOS6 与 CentOS7 系统的区别，理解了 TCP/IP 协议及 IP 地址相关基础内容。

学会了 Linux 初学必备的 16 个 Linux 命令，能使用命令熟练的操作 Linux 系统，通过对 VIM 编辑器的深入学习，能够熟练编辑、修改系统中任意的文本及配置文件。对 Linux 系统的认识及操作有了更进一步的飞跃。

4.17 同步作业

1. 修改密码的命令默认为 passwd，需要按 Enter 键两次，如何一条命令快速修改密码呢?
2. 企业服务器，某天发现系统访问很慢，需要查看系统内核日志，请写出查看系统内核日志的命令；
3. 如果在 Linux 系统/tmp/目录，快速创建 1000 个目录，目录名为：jfedu1、jfedu2、jfedu 依次类推，不断增加；
4. Httpd.conf 配置文件中存在很多以#号开头的行，请使用 vim 相关指令删除#开头的行；

第5章 Linux 用户及权限管理

Linux 是一个多用户的操作系统，引入用户，可以更加方便管理 Linux 服务器，系统默认需要以一个用户的身份登入，而且在系统上启动进程也需要以一个用户身份去运行，

用户可以限制某些进程对特定资源的权限控制。

本章向读者介绍 Linux 系统如何管理创建、删除、修改用户角色、用户权限配置、组权限配置及特殊权限深入剖析。

5.1 Linux 用户及组

Linux 操作系统对多用户的管理，是非常繁琐的，所以用组的概念来管理用户就变得简单，每个用户可以在一个独立的组，每个组也可以有零个用户或者多个用户。

Linux 系统用户是根据用户 ID 来识别的，从默认 ID 编号从 0 开始，但是为了和老式系统兼容，用户 ID 限制在 60000 以下，Linux 用户分总共分为三种，分别如下：

- root 用户 （ID 0）
- 系统用户 （ID 1-499）
- 普通用户 （ID 500 以上）

Linux 系统中的每个文件或者文件夹，都有一个所属用户及所属组，使用 id 命令可以显示当前用户的信息，使用 passwd 命令可以修改当前用户密码。Linux 操作系统用户的特点如下：

- 每个用户拥有一个 UserID，操作系统实际读取的是 UID，而非用户名；
- 每个用户属于一个主组，属于一个或多个附属组，一个用户最多有 31 个附属组；
- 每个组拥有一个 GroupID；
- 每个进程以一个用户身份运行，该用户可对进程拥有资源控制权限；

- 每个可登陆用户拥有一个指定的 Shell 环境。

5.2 Linux 用户管理

Linux 用户在操作系统可以进行日常管理和维护，涉及到的相关配置文件如下：

- /etc/passwd 保存用户信息
- /etc/shdaow 保存用户密码（以加密形式保存）
- /etc/group 保存组信息
- /etc/login.defs 用户属性限制,密码过期时间,密码最大长度等限制
- /etc/default/useradd 显示或更改默认的 useradd 配置文件

如需创建新用户，可以使用命令 useradd，执行命令 useradd jfedu1 即可创建 jfedu1 用户，同时会创建一个同名的组 jfedu1，默认该用户属于 jfedu1 主组。

Useradd jfedu1 命令默认创建用户 jfedu1，会根据如下步骤进行操作：

- 读取/etc/default/useradd，根据配置文件执行创建操作；
- 在/etc/passwd 文件中添加用户信息；
- 如使用 passwd 命令创建密码，密码会被加密保存在/etc/shdaow 中；
- 为 jfedu1 创建家目录：/home/jfedu1；
- 将/etc/skel 中的.bash 开头的文件复制至/home/jfedu1 家目录；
- 创建与用户名相同的 jfedu1 组，jfedu1 用户默认属于 jfeud1 同名组；
- Jfedu1 组信息保存在/etc/group 配置文件中。

在使用 useradd 命令创建用户时，可以支持如下参数：

用法：useradd [选项] 登录

useradd -D

useradd -D [选项]

选项：

-b, --base-dir BASE_DIR 指定新账户的家目录；

-c, --comment COMMENT 新账户的 GECOS 字段；

-d, --home-dir HOME_DIR 新账户的主目录；

-D, --defaults 显示或更改默认的 useradd 配置；

-e, --expiredate EXPIRE_DATE 新账户的过期日期；

-f, --inactive INACTIVE 新账户的密码不活动期；

-g, --gid GROUP 新账户主组的名称或 ID；

-G, --groups GROUPS 新账户的附加组列表；

-h, --help 显示此帮助信息并推出；

-k, --skel SKEL_DIR 使用此目录作为骨架目录；

-K, --key KEY=VALUE 不使用 /etc/login.defs 中的默认值；

-l, --no-log-init 不要将此用户添加到最近登录和登录失败数据库；

-m, --create-home 创建用户的主目录；

-M, --no-create-home	不创建用户的主目录；
-N, --no-user-group	不创建同名的组；
-o, --non-unique	允许使用重复的 UID 创建用户；
-p, --password PASSWORD	加密后的新账户密码；
-r, --system	创建一个系统账户；
-R, --root CHROOT_DIR	chroot 到的目录；
-s, --shell SHELL	新账户的登录 shell；
-u, --uid UID	新账户的用户 ID；
-U, --user-group	创建与用户同名的组；
-Z, --selinux-user SEUSER	为 SELinux 用户映射使用指定 SEUSER。

Useradd 案例演示：

（1） 新建 jfedu 用户，并加入到 jfedu1，jfedu2 附属组；

```
useradd -G jfedu1,jfedu2 jfedu
```

（2） 新建 jfedu3 用户，并指定新的家目录，同时指定其登陆的 SHELL；

```
useradd jfedu3  -d  /tmp/  -s  /bin/bash
```

5.3 Linux 组管理

所有的 Linux 或者 Windows 系统都有组的概念，通过组可以更加方便的管理用户，组的概念应用于各行行业，例如企业会使用部门、职能或地理区域的分类方式来管理成

员，映射在 Linux 系统，同样可以创建用户，并用组的概念对其管理。

Linux 组有如下特点：

- 每个组有一个组 ID；
- 组信息保存在/etc/group 中；
- 每个用户至少拥有一个主组，同时还可以拥有 31 个附属组。

通过命令 groupadd、groupdel、groupmod 来对组进行管理，详细参数使用如下：

groupadd 用法	
-f, --force	如果组已经存在则成功退出； 并且如果 GID 已经存在则取消 –g；
-g, --gid GID	为新组使用 GID；
-h, --help	显示此帮助信息并推出；
-K, --key KEY=VALUE	不使用 /etc/login.defs 中的默认值；
-o, --non-unique	允许创建有重复 GID 的组；
-p, --password PASSWORD	为新组使用此加密过的密码；
-r, --system	创建一个系统账户；
groupmod 用法	
-g, --gid GID	将组 ID 改为 GID；
-h, --help	显示此帮助信息并推出；
-n, --new-name NEW_GROUP	改名为 NEW_GROUP；

-o, --non-unique	允许使用重复的 GID；
-p, --password PASSWORD	将密码更改为(加密过的) PASSWORD；
groupdel 用法	
groupdel jfedu	删除 jfedu 组；

Groupadd 案例演示：

（1） groupadd 创建 jingfeng 组

```
groupadd jingfeng
```

（2） groupadd 创建 jingfeng 组，并指定 GID 为 1000；

```
groupadd -g 1000 jingfeng
```

（3） groupadd 创建一个 system 组，名为 jingfeng 组

```
groupadd  -r  jingfeng
```

Groupmod 案例演示：

（4） groupmod 修改组名称，将 jingfeng 组名，改成 jingfeng1；

```
groupmod -n jingfeng1 jingfeng
```

（5） groupmod 修改组 GID 号，将原 jingfeng1 组 gid 改成 gid 1000；

```
groupmod –g 1000 jingfeng1
```

5.4 Linux 用户及组案例

Useradd 主要用于新建用户,而用户新建完毕，可以使用 usermod 来修改用户及组

的属性，如下为 usermod 详细参数：

用法：usermod [选项] 登录

选项：

-c, --comment 注释　　GECOS 字段的新值；

-d, --home HOME_DIR　　用户的新主目录；

-e, --expiredate EXPIRE_DATE　　设定帐户过期的日期为 EXPIRE_DATE；

-f, --inactive INACTIVE　　过期 INACTIVE 天数后，设定密码为失效状态；

-g, --gid GROUP　　强制使用 GROUP 为新主组；

-G, --groups GROUPS　　新的附加组列表 GROUPS；

-a, --append GROUP　　将用户追加至上边 -G 中提到的附加组中，并不从其它组中删除此用户；

-h, --help　　显示此帮助信息并推出；

-l, --login LOGIN　　新的登录名称；

-L, --lock　　锁定用户帐号；

-m, --move-home　　将家目录内容移至新位置（仅于 -d 一起使用）；

-o, --non-unique　　允许使用重复的(非唯一的) UID；

-p, --password PASSWORD　　将加密过的密码（PASSWORD）设为新密码；

-R, --root CHROOT_DIR	chroot 到的目录；
-s, --shell SHELL	该用户帐号的新登录 shell 环境；
-u, --uid UID	用户帐号的新 UID；
-U, --unlock	解锁用户帐号；
-Z, --selinux-user SEUSER	用户账户的新 SELinux 用户映射。

Usermod 案例演示：

（1） 将 jfedu 用户属组修改为 jfedu1，jfedu2 附属组；

```
usermod -G jfedu1,jfedu2 jfedu
```

（2） 将 jfedu 用户加入到 jfedu3，jfedu4 附属组，-a 为添加新组，原组保留；

```
usermod –a -G jfedu3,jfedu4 jfedu
```

（3） 修改 jfedu 用户，并指定新的家目录，同时指定其登陆的 SHELL；

```
usermod -d /tmp/ -s /bin/sh jfedu
```

（4） 将 jfedu 用户名修改为 jfedu1；

```
usermod -l jfedu1 jfedu
```

（5） 锁定 jfedu1 用户及解锁 jfedu1 用户方法；

```
usermod –L jfedu1

usermod -U jfedu1
```

Userdel 案例演示：

使用 userdel 可以删除指定用户及其用户的邮箱目录或者 Selinux 映射环境：

- userdel jfedu1 保留用户的家目录；
- userdel -r jfedu1 删除用户及用户家目录，用户 login 系统无法删除；
- userdel -rf jfedu1 强制删除用户及该用户家目录，不论是否 login 系统。

5.5 Linux 权限管理

Linux 权限是操作系统用来限制对资源访问的机制，权限一般分为读、写、执行。系统中每个文件都拥有特定的权限、所属用户及所属组，通过这样的机制来限制哪些用户或用户组可以对特定文件进行相应的操作。

Linux 每个进程都是以某个用户身份运行，进程的权限与该用户的权限一样，用户的权限越大，则进程拥有的权限就越大。

Lnux 中有的文件及文件夹都有至少权限三种权限，常见的权限如表 5-1 所示:

权限	对文件的影响	对目录的影响
r（读取）	可读取文件内容	可列出目录内容
w（写入）	可修改文件内容	可在目录中创建删除内容
x（执行）	可作为命令执行	可访问目录内容
目录必须拥有 x 权限，否则无法查看其内容		

表 5-1 Linux 文件及文件及权限

Linux 权限授权，默认是授权给三种角色，分别是 user、group、other，Linux 权限与用户之间的关联如下：

- U 代表 User，G 代表 Group，O 代表 Other；
- 每个文件的权限基于 UGO 进行设置；
- 权限三位一组（rwx），同时需授权给三种角色，UGO；
- 每个文件拥有一个所属用户和所属组，对应 UGO，不属于该文件所属用户或所属组使用 O 来表示；

在 Linux 系统中，可以通过 ls –l 查看 jfedu.net 目录的详细属性，如图 5-1 所示：

drwxrwxr-x	2	jfedu1 jfedu1	4096	Dec 10 01:36	jfedu.net



图 5-1 Linux jfedu.net 目录详细属性

jfedu.net 目录属性参数详解如下：

- d 表示目录，同一位置如果为-则表示普通文件；
- rwxrwxr-x 表示三种角色的权限，每三位为一种角色，依次为 u，g，o 权限，如上则表示 user 的权限为 rwx，group 的权限为 rwx，other 的权限为 r-x；

- 2 表示文件夹的链接数量，可理解为该目录下子目录的数量；
- 从左到右，第一个 jfedu1 表示该用户名，第二个 jfedu1 则为组名，其他人角色默认不显示；
- 4096 表示该文件夹占据的字节数；
- Dec 10 01:36 表示文件创建或者修改的时间；
- Jfedu.net 为目录的名，或者文件名。

5.6 Chown 属主及属组

修改某个用户、组对文件夹的属主及属组，用命令 chown 实现，案例演示如下：

（1） 修改 jfedu.net 文件夹所属的用户为 root，其中-R 参数表示递归处理所有的文件及子目录。

```
chown -R  root  jfedu.net
```

（2） 修改 jfedu.net 文件夹所属的组为 root。

```
chown  -R  :root  jfedu.net 或者 chgrp  –R  root  jfedu.net
```

（3） 修改 jfedu.net 文件夹所属的用户为 root，组也为 root。

```
chown  -R  root:root  jfedu.net
```

5.7 Chmod 用户及组权限

修改某个用户、组对文件夹的权限，用命令 chmod 实现，其中以代指 ugo 、-、= 代表加入、删除和等于对应权限，具体案例如下：

（1） 授予用户对 jfedu.net 目录拥有 rwx 权限

```
chmod  –R  u+rwx  jfedu.net
```

（2） 授予组对 jfedu.net 目录拥有 rwx 权限

```
chmod  –R  g+rwx  jfedu.net
```

（3） 授予用户、组、其他人对 jfedu.net 目录拥有 rwx 权限

```
chmod  –R  u+rwx,g+rwx,o+rwx  jfedu.net
```

（4） 撤销用户对 jfedu.net 目录拥有 w 权限

```
chmod  –R  u-w  jfedu.net
```

（5） 撤销用户、组、其他人对 jfedu.net 目录拥有 x 权限

```
chmod  –R  u-x,g-x,o-x  jfedu.net
```

（6） 授予用户、组、其他人对 jfedu.net 目录只有 rx 权限

```
chmod  –R  u=rx,g=rx,o=rx  jfedu.net
```

5.8 Chmod 二进制权限

Linux 权限默认使用 rwx 来表示 ,为了更简化在系统中对权限进行配置和修改 ,Linux 权限引入二进制表示方法，如下代码：

```
Linux 权限可以将 rwx 用二进制来表示，其中有权限用 1 表示，没有权限用 0 表示；

Linux 权限用二进制显示如下：

rwx=111
```

r-x=101

rw-=110

r--=100

依次类推，转化为十进制，对应十进制结果显示如下：

rwx=111=4+2+1=7

r-x=101=4+0+1=5

rw-=110=4+4+0=6

r--=100=4+0+0=4

得出结论，用 r=4,w=2,x=1 来表示权限。

使用二进制方式来修改权限案例演示如下，其中默认 jfedu.net 目录权限为 755：

（1） 授予用户对 jfedu.net 目录拥有 rwx 权限

```
chmod  –R  755  jfedu.net
```

（2） 授予组对 jfedu.net 目录拥有 rwx 权限

```
chmod  –R  775  jfedu.net
```

（3） 授予用户、组、其他人对 jfedu.net 目录拥有 rwx 权限

```
chmod  –R  777  jfedu.net
```

（4） 撤销用户对 jfedu.net 目录拥有 w 权限

```
chmod  –R  555  jfedu.net
```

（5） 撤销用户、组、其他人对 jfedu.net 目录拥有 x 权限

```
chmod  –R  644  jfedu.net
```

（6） 授予用户、组、其他人对 jfedu.net 目录只有 rx 权限

```
chmod  –R  555  jfedu.net
```

5.9 Linux 特殊权限及掩码

Linux 权限除了常见的 rwx 权限之外，还有很多特殊的权限，细心的读者会发现，为什么 Linux 目录默认权限 755，而文件默认权限为 644 呢，这是因为 Linux 权限掩码 umask 导致。

每个 Linux 终端都拥有一个 umask 属性，umask 熟悉可以用来确定新建文件、目录的默认权限，默认系统权限掩码为 022。在系统中每创建一个文件或者目录，文件默认权限是 666，而目录权限则为 777，权限对外开放比较大，所以设置了权限掩码之后，默认的文件和目录权限减去 umask 值才是真实的文件和目录的权限。

- 对应目录权限为：777-022=755；
- 对应文件权限为：666-022=644；
- 执行 umask 命令可以查看当前默认的掩码，umask -S 023 可以设置默认的权限掩码。

在 Linux 权限中，除了普通权限外，还有如下表 5-2 所示，三个特殊权限：

权限	对文件的影响	对目录的影响
Suid	以文件的所属用户身份执行，	无

	而非执行文件的用户	
sgid	以文件所属组身份去执行	在该目录中创建任意新文件的所属组与该目录的所属组相同
sticky	无	对目录拥有写入权限的用户仅可以删除其拥有的文件，无法删除其他用户所拥有的文件

表 5-2 Linux 三种特殊权限

Linux 中设置特殊权限方法如下：

- 设置 suid： chmod u+s jfedu.net
- 设置 sgid： chmod g+s jfedu.net
- 设置 sticky： chmod o+t jfedu.net

特殊权限与设置普通权限一样，可以使用数字方式表示：

- SUID = 4
- SGID = 2
- Sticky = 1

可以通过 chmod 4755 jfedu.net 对该目录授予特殊权限为 s 的权限 ,Linux 系统中 s 权限的应用常见包括：su、passwd、sudo，如图 5-2 所示：

```
[root@www-jfedu-net ~]# ll /bin/su
-rwsr-xr-x 1 root root 34904 5月  11 2016 /bin/su
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll /usr/bin/pa
package-cleanup          pango-querymodules-64    paste
package-stash-conflicts  pango-view               patch
pal2rgb                  passwd                   patchperl
[root@www-jfedu-net ~]# ll /usr/bin/passwd
-rwsr-xr-x 1 root root 30768 11月 24 2015 /usr/bin/passwd
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll /usr/bin/sudo
sudo        sudoedit    sudoreplay
[root@www-jfedu-net ~]# ll /usr/bin/sudo
---s--x--x 1 root root 123832 12月  7 08:36 /usr/bin/sudo
You have new mail in /var/spool/mail/root
[root@www-jfedu-net ~]#
```

图 5-2 Linux 特殊权限 s 应用

5.10 本章小结

通过对本章内容的学习，读者可以了解 Linux 用户和组的系统知识，同时账号 Linux 用户和组在系统中各种案例操作。读者可以熟练新建用户、删除用户、修改用户属性、添加组、修改组以及删除组。

5.11 同步作业

1. 某互联网公司职能及员工信息表，如表 5-3 所示，请在 Linux 系统中创建相关员工，并把员工加入到部门。

部门	职能
讲师部(teacher)	jfwu,jfcai
市场部(market)	jfxin,jfqi

管理部(manage)	jfedu,jfteach
运维部(operater)	jfhao,jfyang

表 5-3 Linux 用户和组管理

2. 批量创建 1-100 个用户，用户名以 jfedu 开头，后面紧跟 1,2,3，例如 jfedu1,jfedu2,jfedu3。
3. 使用 useradd 创建用户并通过-p 参数指定密码，设定完密码需通过系统能正常验证并登陆。
4. 小王公司服务器，使用 Root 用户通过 SecureCRT 远程登陆后，如图 5-3 所示，发现登录终端变成 bash-4.1#，是什么原因导致？以及如何修复为正常的登录 SHELL 环境，请写出答案。



图 5-3 SecureCRT 登录 Linux 系统界面

第6章 Linux 软件包企业实战

通过前几章的学习，读者掌握了 Linux 系统基本命令、用户及权限等知识，Linux 整

个体系的关键不在于系统本身，而是在于可以基于 Linux 系统去安装和配置企业中相关的软件、数据及应用程序，所以对软件的维护是运维工程师的重中之重。

本章向读者介绍 Linux 系统软件的安装、卸载、配置、维护以及如何构建企业本地 YUM 光盘源及 HTTP 本地源。

6.1 RPM 软件包管理

Linux 软件包管理大致可分为二进制包、源码包，使用的工具也各不相同。Linux 常见软件包分为两种，分别是源代码包（Source Code）、二进制包（Binary Code），源代码包是没有经过编译的包，需要经过 GCC、C++编译器环境编译才能运行，二进制包无需编译，可以直接安装使用。

通常而言，可以通过后缀简单区别源码包和二进制包，例如.tar.gz、.zip、.rar 结尾的包通常称之为源码包，以.rpm 结尾的软件包称之为二进制包。真正区分是否为源码还是二进制还得基于代码里面的文件来判断，例如包含.h、.c、.cpp、.cc 等结尾的源码文件，称之为源码包，而代码代码里面存在 bin 可执行文件，称之为二进制包。

CentOS 操作系统中有一款默认软件管理的工具，红帽包管理工具（Red Hat Package Manager，RPM）。

使用 RPM 工具可以对软件包实现快速安装、管理及维护。RPM 管理工具适用的操作系统包括：CentOS，RedHat，Fedora，SUSE 等，RPM 工具常用于管理.rpm 后缀结尾的软件包。

RPM 软件包命令规则详解如下：

RPM 包命名格式为：

name-version.rpm

name-version-noarch.rpm

name-version-arch.src.rpm

如下软件包格式：

epel-release-6-8.noarch.rpm

perl-Pod-Plainer-1.03-1.el6.noarch.rpm

yasm-1.2.0-4.el7.x86_64.rpm

RPM 包格式解析如下：

- name 软件名称，例如 yasm、perl-pod-Plainer；
- version 版本号，1.2.0 通用格式：“主版本号.次版本号.修正号”；4 表示是发布版本号，该 RPM 包是第几次编译生成的；
- arch 适用的硬件平台，RPM 支持的平台有：i386、i586、i686、x86_64、sparc、alpha 等。
- .rpm 后缀包表示编译好的二进制包，可用 rpm 命令直接安装；
- .src.rpm 源代码包，源码编译生成.rpm 格式的 RPM 包方可使用；
- el* 软件包发行版本，el6 表示该软件包适用于 RHEL 6.x/CentOS 6.x；

- devel： 开发包；
- noarch： 软件包可以在任何平台上安装。

RPM 工具命令详解如下：

RPM 选项 PACKAGE_NAME	
-a, --all	查询所有已安装软件包；
-q，--query	表示询问用户，输出信息；
-l, --list	打印软件包的列表；
-f, --file	FILE 查询包含 FILE 的软件包；
-i, --info	显示软件包信息，包括名称，版本，描述；
-v, --verbose	打印输出详细信息；
-U, --upgrade	升级 RPM 软件包；
-h,--hash	软件安装，可以打印安装进度条；
--last	列出软件包时，以安装时间排序，最新的在上面；
-e, --erase	卸载 rpm 软件包
--force	表示强制，强制安装或者卸载；
--nodeps	RPM 包不依赖
-l, --list	列出软件包中的文件；
--provides	列出软件包提供的特性；
-R, --requires	列出软件包依赖的其他软件包；

--scripts	列出软件包自定义的小程序。

RPM 企业案例演示：

rpm -q	httpd	检查 httpd 包是否安装；
rpm -ql	httpd	查看软件安装的路径；
rpm -qi	httpd	查看软件安装的版本信息；
rpm -e	httpd	卸载 httpd 软件；
rpm -e --nodeps	httpd	强制卸载 httpd；
rpm -qa|grep	httpd	检查 httpd 相关的软件包是否安装。
rpm -ivh	httpd-2.4.10-el7.x86_64.rpm	安装 httpd 软件包；
rpm -Uvh	httpd-2.4.10-el7.x86_64.rpm	升级 httpd 软件；
rpm -ivh --nodeps	httpd-2.4.10-el7.x86_64.rpm	不依赖其他软件包；

6.2 Tar 软件包管理

Linux 操作系统除了使用 RPM 管理工具对软件包管理之外，还可以通过 tar、zip、jar 等工具进行源码包的管理。

6.2.1 Tar 命令参数详解

-A, --catenate, --concatenate	将存档与已有的存档合并
-c, --create	建立新的存档

-d, --diff, --compare	比较存档与当前文件的不同之处
--delete	从存档中删除
-r, --append	附加到存档结尾
-t, --list	列出存档中文件的目录
-u, --update	仅将较新的文件附加到存档中
-x, --extract, --get	解压文件
-j, --bzip2, --bunzip2	有 bz2 属性的软件包；
-z, --gzip, --ungzip	有 gz 属性的软件包；
-b, --block-size N	指定块大小为 Nx512 字节（缺省时 N=20）；
-B, --read-full-blocks	读取时重组块；
-C, --directory DIR	指定新的目录；
--checkpoint	读取存档时显示目录名；
-f, --file [HOSTNAME:]F	指定存档或设备，后接文件名称；
--force-local	强制使用本地存档，即使存在克隆；
-G, --incremental	建立老 GNU 格式的备份；
-g, --listed-incremental	建立新 GNU 格式的备份；
-h, --dereference	不转储动态链接，转储动态链接指向的文件；
-i, --ignore-zeros	忽略存档中的 0 字节块（通常意味着文件结束）；
--ignore-failed-read	在不可读文件中作 0 标记后再退出；

-k, --keep-old-files	保存现有文件；从存档中展开时不进行覆盖；
-K, --starting-file F	从存档文件 F 开始；
-l, --one-file-system	在本地文件系统中创建存档；
-L, --tape-length N	在写入 N*1024 个字节后暂停，等待更换磁盘；
-m, --modification-time	当从一个档案中恢复文件时 ,不使用新的时间标签;
-M, --multi-volume	建立多卷存档,以便在几个磁盘中存放；
-O, --to-stdout	将文件展开到标准输出；
-P, --absolute-paths	不要从文件名中去除 '/'；
-v, --verbose	详细显示处理的文件；
--version	显示 tar 程序的版本号；
--exclude	FILE 不把指定文件包含在内；
-X, --exclude-from FILE	从指定文件中读入不想包含的文件的列表。

6.2.2 TAR 企业案例演示

tar	-cvf	jfedu.tar.gz	jfedu	打包 jfedu 文件或者目录，打包后名称 jfedu.tar.gz；
tar	-tf	jfedu.tar.gz		查看 jfedu.tar.gz 包中内容；
tar	-rf	jfedu.tar.gz	jfedu.txt	将 jfedu.txt 文件追加到 jfedu.tar.gz 中
tar	-xvf	jfedu.tar.gz		解压 jfedu.tar.gz 程序包；

tar -czvf jfedu.tar.gz jfedu 使用 gzip 格式打包并压缩 jfedu 目录；
tar -cjvf jfedu.tar.bz2 jfedu 使用 bzip2 格式打包并压缩 jfedu 目录；
tar -czf jfedu.tar.gz * -X list.txt 使用 gzip 格式打包并压当前目录所有文件，排除 list.txt 中记录的文件；
tar -czf jfedu.tar.gz * --exclude=zabbix-3.2.4.tar.gz --exclude=nginx-1.12.0.tar.gz 使用 gzip 格式打包并压当前目录所有文件及目录 ,排除 zabbix-3.2.4.tar.gz 和 nginx-1.12.0.tar.gz 软件包。

6.2.3 TAR 实现 Linux 操作系统备份

Tar 命令工具除了用于日常打包、解压源码包或者压缩包之外，最大的亮点是还可以用于 Linux 操作系统文件及目录的备份，使用 tar -g 可以基于 GNU 格式的增量备份，备份原理是基于检查目录或者文件的 atime、mtime、ctime 属性是否被修改。文件及目录时间属性详解如下：

- 文件被访问的时间(Access time,atime)；
- 文件内容被改变的时间（Modified time，mtime)；
- 文件写入、权限更改的时间（Change time，ctime)。

总结，更改文件内容 mtime 和 ctime 都会改变，但 ctime 可以在 mtime 未发生变化时被更改，例如修改文件权限，文件 mtime 时间不变，而 ctime 时间改变。TAR 增

量备份案例演示步骤如下：

（1） /root 目录创建 jingfeng 文件夹，同时在 jingfeng 文件夹中，新建 jf1.txt，jf2.txt 文件，如图 6-1 所示：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# mkdir jingfeng
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# cd jingfeng/
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# pwd
/root/jingfeng
[root@www-jfedu-net jingfeng]# ls
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# touch jf1.txt jf2.txt
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# ll
total 0
-rw-r--r-- 1 root root 0 May  3 17:29 jf1.txt
-rw-r--r-- 1 root root 0 May  3 17:29 jf2.txt
[root@www-jfedu-net jingfeng]#
```

图 6-1 创建 jingfeng 目录及文件

（2） 使用 tar 命令第一次完整备份 jingfeng 文件夹中的内容，-g 指定快照 snapshot 文件，第一次没有该文件则会自动创建，如图 6-2 所示：

```
cd /root/jingfeng/
tar -g /data/backup/snapshot -czvf /data/backup/2022jingfeng.tar.gz
```

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# cd /root/jingfeng/
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# ls
jf1.txt  jf2.txt
[root@www-jfedu-net jingfeng]# tar  -g  /data/backup/snapshot  -czvf  /data/backu
g.tar.gz  *
jf1.txt
jf2.txt
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# ll /data/backup/
total 8
-rw-r--r-- 1 root root 126 May  3 17:37 2017jingfeng.tar.gz
-rw-r--r-- 1 root root  36 May  3 17:37 snapshot
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# cat /data/backup/snapshot
GNU tar-1.23-2
14938042225483817 99[root@www-jfedu-net jingfeng]#
```

图 6-2 tar 备份 jingfeng 目录中文件

（3） 使用 tar 命令第一次完整备份 jingfeng 文件夹中之后，会生成快照文件：/data/backup/snapshot，后期增量备份会以 snapshot 文件为参考，在 jingfeng 文件夹中再创建 jf3.txt jf4.txt 文件，然后通过 tar 命令增量备份 jingfeng 目录所有内容，如图 6-3 所示：

```
cd   /root/jingfeng/

touch jf3.txt jf4.txt

tar -g /data/backup/snapshot -czvf /data/backup/2022jingfeng_add1.tar.gz *
```

```
[root@www-jfedu-net jingfeng]# touch jf3.txt jf4.txt
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# ll
total 0
-rw-r--r-- 1 root root 0 May  3 17:29 jf1.txt
-rw-r--r-- 1 root root 0 May  3 17:29 jf2.txt
-rw-r--r-- 1 root root 0 May  3 17:42 jf3.txt
-rw-r--r-- 1 root root 0 May  3 17:42 jf4.txt
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# tar -g /data/backup/snapshot -czvf /data/backup/201
7jingfeng_add1.tar.gz *
jf3.txt
jf4.txt
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# tar -tf /data/backup/2017jingfeng_add1.tar.gz
jf3.txt
jf4.txt
```

图 6-3 tar 增量备份 jingfeng 目录中文件

如上图 6-3 所示，增量备份时，需-g 指定第一次完整备份的快照 snapshot 文件，同时增量打包的文件名不能跟第一次备份后的文件名重复，通过 tar –tf 可以查看打包后的文件内容。

6.2.4 Shell+TAR 实现增量备份

企业中日常备份的数据包括/boot、/etc、/root、/data 目录等，备份的策略参考：每周 1-6 执行增量备份 ,每周日执行全备份。同时在企业中备份操作系统数据均使用 Shell 脚本完成 ,此处 auto_backup_system.sh 脚本供参考 ,后面章节会系统讲解 Shell 脚本，脚本内容如下：

```
#!/bin/bash
#Automatic Backup Linux System Files
#By Author www.jfedu.net
#Define Variables
SOURCE_DIR=(
    $*
)
TARGET_DIR=/data/backup/
YEAR=`date +%Y`
MONTH=`date +%m`
```

```
DAY=`date +%d`

WEEK=`date +%u`

FILES=system_backup.tgz

CODE=$?

if

    [ -z $SOURCE_DIR ] ; then

    echo -e "Please Enter a File or Directory You Need to Backup:\n------------

------------------------------\nExample $0 /boot /etc ....."

    exit

fi

#Determine Whether the Target Directory Exists

if

    [ ! -d $TARGET_DIR/$YEAR/$MONTH/$DAY ] ; then

    mkdir -p $TARGET_DIR/$YEAR/$MONTH/$DAY

    echo "This $TARGET_DIR Created Successfully !"

fi

#EXEC Full_Backup Function Command

Full_Backup()

{
```

```
if
    [ "$WEEK" -eq "7" ] ; then
    rm -rf $TARGET_DIR/snapshot
    cd $TARGET_DIR/$YEAR/$MONTH/$DAY  ;tar -g $TARGET_DIR/snapshot -
czvf $FILES `echo ${SOURCE_DIR[@]}`
    [ "$CODE" == "0" ]&&echo -e   "----------------------------------------
\nFull_Backup System Files Backup Successfully !"
fi
}
#Perform incremental BACKUP Function Command
Add_Backup()
{
   cd $TARGET_DIR/$YEAR/$MONTH/$DAY  ;
if
    [ -f $TARGET_DIR/$YEAR/$MONTH/$DAY/$FILES ] ; then
    read -p "$FILES Already Exists, overwrite confirmation yes or no ? : " SURE
    if [ $SURE == "no" -o $SURE == "n" ] ; then
    sleep 1  ; exit 0
    fi
```

```
#Add_Backup Files System

    if

        [ $WEEK -ne "7" ] ; then

        cd      $TARGET_DIR/$YEAR/$MONTH/$DAY      ;      tar      -g
$TARGET_DIR/snapshot -czvf $$_$FILES `echo ${SOURCE_DIR[@]}`

        [ "$CODE" == "0" ]&&echo -e   "---------------------------------------
\nAdd_Backup System Files Backup Successfully !"

   fi

else

   if

       [ $WEEK -ne "7" ] ; then

       cd      $TARGET_DIR/$YEAR/$MONTH/$DAY      ;      tar      -g
$TARGET_DIR/snapshot -czvf $FILES `echo ${SOURCE_DIR[@]}`

       [ "$CODE" == "0" ]&&echo -e   "---------------------------------------
\nAdd_Backup System Files Backup Successfully !"

   fi

fi

}

Full_Backup ; Add_Backup
```

6.3 ZIP 软件包管理

ZIP 也是计算机文件的压缩的算法，原名 Deflate（真空），发明者为菲利普·卡兹（Phil Katz），他于 1989 年 1 月公布了该格式的资料。ZIP 通常使用后缀名“.zip”。

主流的压缩格式包括 tar、rar、zip、war、gzip、bz2、iso 等。从性能上比较，TAR、WAR、RAR 格式较 ZIP 格式压缩率较高，但压缩时间远远高于 ZIP，Zip 命令行工具可以实现对 zip 属性的包进行管理，也可以将文件及文件及打包成 zip 格式。如下为 ZIP 工具打包常见参数详解：

-f	freshen：只更改文件；
-u	update：只更改或新文件；
-d	从压缩文件删除文件；
-m	中的条目移动到 zipfile（删除 OS 文件）；
-r	递归到目录；
-j	junk（不记录）目录名；
-l	将 LF 转换为 CR LF（-11 CR LF 至 LF）；
-1	压缩更快 1-9 压缩更好；
-q	安静操作，不输出执行的过程；
-v	verbose 操作/打印版本信息；
-c	添加一行注释；

-z	添加 zipfile 注释；
-o	读取名称使 zip 文件与最新条目一样旧；
-x	不包括以下名称；
-F	修复 zipfile（-FF 尝试更难）；
-D	不要添加目录条目；
-T	测试 zip 文件完整性；
-X	eXclude eXtra 文件属性；
-e	加密 - 不要压缩这些后缀；
-h2	显示更多的帮助。

ZIP 企业案例演示：

（1） 通过 zip 工具打包 jingfeng 文件夹中所有内容，如图 6-4 所示：

```
zip   -rv   jingfeng.zip /root/jingfeng/
```

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# zip  -rv  jingfeng.zip /root/jingfeng/
updating: root/jingfeng/          (in=0) (out=0) (stored 0%)
updating: root/jingfeng/jf4.txt (in=0) (out=0) (stored 0%)
updating: root/jingfeng/jf1.txt (in=0) (out=0) (stored 0%)
updating: root/jingfeng/jf2.txt (in=0) (out=0) (stored 0%)
updating: root/jingfeng/jf3.txt (in=0) (out=0) (stored 0%)
total bytes=0, compressed=0 -> 0% savings
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll jingfeng.zip
-rw-r--r-- 1 root root 858 May  3 18:16 jingfeng.zip
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# zip -T jingfeng
test of jingfeng.zip OK
[root@www-jfedu-net ~]#
```

图 6-4 zip 对 jingfeng 目录打包备份

（2） 通过 zip 工具打包 jingfeng 文件夹中所有内容，排除部分文件，如图 6-5 所示：

```
zip  -rv  jingfeng.zip  *  -x  jf1.txt

zip  -rv  jingfeng.zip  *  -x  jf2.txt -x jf3.txt
```

```
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# zip -rv jingfeng.zip * -x jf1.txt
excluding jf1.txt
file matches zip file -- skipping
updating: jf2.txt       (in=0) (out=0) (stored 0%)
updating: jf3.txt       (in=0) (out=0) (stored 0%)
total bytes=0, compressed=0 -> 0% savings
[root@www-jfedu-net jingfeng]#
[root@www-jfedu-net jingfeng]# zip -rv jingfeng.zip * -x jf2.txt -x jf3.txt
excluding jf2.txt
excluding jf3.txt
file matches zip file -- skipping
updating: jf1.txt       (in=0) (out=0) (stored 0%)
total bytes=0, compressed=0 -> 0% savings
[root@www-jfedu-net jingfeng]#
```

图 6-5 zip 对 jingfeng 目录打包备份，排除部分文件

（3） 通过 zip 工具删除 jingfeng.zip 中 jf3.txt 文件，如图 6-6 所示

```
zip jingfeng.zip -d jf3.txt
```

（4） 通过 unzip 工具解压 jingfeng.zip 文件夹中所有内容，如图 6-6 所示：

```
unzip   jingfeng.zip

unzip   jingfeng.zip   -d   /data/backup/  可以-d 指定解压后的目录
```

```
[root@www-jfedu-net ~]# ll jingfeng.zip
-rw-r--r-- 1 root root 858 May  3 18:16 jingfeng.zip
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# unzip jingfeng.zip
Archive:  jingfeng.zip
   creating: root/jingfeng/
 extracting: root/jingfeng/jf4.txt
 extracting: root/jingfeng/jf1.txt
 extracting: root/jingfeng/jf2.txt
 extracting: root/jingfeng/jf3.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# unzip jingfeng.zip -d /data/backup/
Archive:  jingfeng.zip
   creating: /data/backup/root/jingfeng/
 extracting: /data/backup/root/jingfeng/jf4.txt
 extracting: /data/backup/root/jingfeng/jf1.txt
 extracting: /data/backup/root/jingfeng/jf2.txt
 extracting: /data/backup/root/jingfeng/jf3.txt
```

图 6-6 unzip 对 jingfeng 目录解压

6.4 源码包软件安装

通常使用 RPM 工具管理.rpm 结尾的二进制包，而标准的.zip、tar 结尾的源代码包则不能使用 RPM 工具去安装、卸载及升级，源码包安装有三个步骤，如下：

- ./configure　预编译，主要用于检测系统基准环境库是否满足，生成 MakeFile 文件；
- make　编译，基于第一步生成的 makefile 文件，进行源代码的编译；
- make install　安装，编译完毕之后，将相关的可运行文件安装至系统中；

使用 make 编译时，Linux 操作系统必须有 GCC 编译器，用于编译源码。

源码包安装通常需要./configure、make、make install 三个步骤，某些特殊源码可以只有三步中的其中一个步骤，或者两个步骤。

以 CentOS 7 Linux 系统为基准，在其上安装 Nginx 源码包，企业中源码安装的详

细步骤如下：

（1） Nginx.org 官网下载 Nginx-1.13.0.tar.gz 包

```
wget http://nginx.org/download/nginx-1.13.0.tar.gz
```

（2） Nginx 源码包解压

```
tar  -xvf nginx-1.13.0.tar.gz
```

（3） 源码 Configure 预编译，需进入解压后的目录执行./configure 指令，分号 “;” 表示连接多个命令。

```
cd nginx-1.13.0；./configure
```

（4） make 编译

```
make
```

（5） make install 安装

```
make  install
```

通过以上五个步骤，源码包软件安装成功，源码包在编译及安装时，可能会遇到各种错误，需要把错误解决之后，然后再进行下一步安装即可，后面章节会重点针对企业使用的软件进行案例演练。

6.5 YUM 软件包管理

前端软件包管理器（Yellow Updater Modified，YUM）适用于 CentOS、Fedora、RedHat 及 SUSE 中的 Shell 命令行，主要用于管理 RPM 包，于 RPM 工具使

用范围类似，YUM 工具能够从指定的服务器自动下载 RPM 包并且安装，还可以自动处理依赖性关系。

使用 RPM 工具管理和安装软件时，会发现 rpm 包有依赖，需要逐个手动下载安装，而 YUM 工具的最大便利就是可以自动安装所有依赖的软件包 ,从而提升效率 ,节省时间。

6.5.1 YUM 工作原理

学习 YUM，一定要理解 YUM 工作原理，YUM 正常运行，需要依赖两个部分，一是 YUM 源端，二是 YUM 客户端，也即用户使用端。

YUM 客户端安装的所有 RPM 包都是来自 YUM 服务端 ,YUM 源端通过 HTTP 或者 FTP 服务器发布。而 YUM 客户端能够从 YUM 源端下载依赖的 RPM 包是由于在 YUM 源端生成了 RPM 包的基准信息，包括 RPM 包版本号、配置文件、二进制信息、依赖关系等。

YUM 客户端需要安装软件或者搜索软件 ,会查找/etc/yum.repos.d 下以.repo 结尾文件 ,CentOS Linux 默认的.repo 文件名为 CentOS-Base.repo ,该文件中配置了 YUM 源端的镜像地址，所以每次安装、升级 RPM 包，YUM 客户端均会查找.repo 文件。

YUM 客户端如果配置了 CentOS 官方 repo 源，客户端操作系统必须能联外网，满足网络条件 ,才能下载软件并安装 ,如果没有网络 ,也可以构建光盘源或者内部 YUM 源。在只要 YUM 客户端时，YUM 客户端安装软件，默认会把 YUM 源地址、Header 信息、软件包、数据库信息、缓存文件存储在/var/cache/yum 中，每次使用 YUM 工具，YUM 优先通过 Cache 查找相关软件包，Cache 中不存在，然后在访问外网 YUM 源。

6.5.2 YUM 企业案例演练

由于 YUM 工具的使用简便、快捷、高效，在企业中得到广泛的使用，得到众多 IT 运维、程序人员的青睐，要能熟练使用 YUM 工具，需要先掌握 YUM 命令行参数的使用，如下为 YUM 命令工具的参数详解及实战步骤：

YUM 命令工具指南，YUM 格式为：

YUM [command] [package] -y|-q 其中的[options]是可选。-y 安装或者卸载出现 YES 时，自动确认 yes；-q 不显示安装的过程。

yum install httpd	安装 httpd 软件包；
yum search	YUM 搜索软件包；
yum list httpd	显示指定程序包安装情况 httpd；
yum list	显示所有已安装及可安装的软件包；
yum remove httpd	删除程序包 httpd；
yum erase httpd	删除程序包 httpd；
yum update	内核升级或者软件更新；
yum update httpd	更新 httpd 软件；
yum check-update	检查可更新的程序；
yum info httpd	显示安装包信息 httpd；
yum provides	列出软件包提供哪些文件；
yum provides "*/rz"	列出 rz 命令由哪个软件包提供；

yum grouplist	查询可以用 groupinstall 安装的组名称；
yum groupinstall "Chinese Support"	安装中文支持；
yum groupremove "Chinese Support"	删除程序组 Chinese Support；
yum deplist httpd	查看程序 httpd 依赖情况；
yum clean packages	清除缓存目录下的软件包；
yum clean headers	清除缓存目录下的 headers；
yum clean all	清除缓存目录下的软件包及旧的 headers。

（1） 基于 CentOS 7 Linux，执行命令 yum install httpd -y，安装 httpd 服务，如图 6-7 所示：

```
[root@www-jfedu-net ~]# yum install httpd -y
Loaded plugins: fastestmirror, langpacks
Repository epel is listed more than once in the configuration
Loading mirror speeds from cached hostfile
Resolving Dependencies
--> Running transaction check
---> Package httpd.x86_64 0:2.4.6-45.el7.centos.4 will be installed
--> Finished Dependency Resolution

Dependencies Resolved

================================================================================
 Package          Arch              Version                          Reposi
================================================================================
Installing:
 httpd            x86_64            2.4.6-45.el7.centos.4            update

Transaction Summary
================================================================================
Install  1 Package
```

图 6-7 YUM 安装 httpd 软件

（2） 执行命令 yum grouplist，检查 groupinstall 的软件组名，如图 6-8 所示：

```
[root@www-jfedu-net ~]# yum grouplist|more
Loaded plugins: fastestmirror, langpacks
Repository epel is listed more than once in the configuration
There is no installed groups file.
Maybe run: yum groups mark convert (see man yum)
Loading mirror speeds from cached hostfile
Available Environment Groups:
   Minimal Install
   Compute Node
   Infrastructure Server
   File and Print Server
   MATE Desktop
   Basic Web Server
   Virtualization Host
   Server with GUI
   GNOME Desktop
   KDE Plasma Workspaces
   Development and Creative Workstation
```

图 6-8 YUM Grouplist 显示组安装名称

（3） 执行命令 yum groupinstall "GNOME Desktop" -y，安装 Linux 图像界面，如图 6-9 所示:

```
[root@www-jfedu-net ~]# yum groupinstall "GNOME Desktop" -y
Loaded plugins: fastestmirror, langpacks
Repository epel is listed more than once in the configuration
There is no installed groups file.
Maybe run: yum groups mark convert (see man yum)
Loading mirror speeds from cached hostfile
Resolving Dependencies
--> Running transaction check
---> Package ModemManager.x86_64 0:1.6.0-2.el7 will be installed
--> Processing Dependency: libqmi-utils for package: ModemManager-1.6.0-2.
--> Processing Dependency: libmbim-utils for package: ModemManager-1.6.0-2
--> Processing Dependency: libqmi-glib.so.5()(64bit) for package: ModemMan
_64
--> Processing Dependency: libmbim-glib.so.4()(64bit) for package: ModemMa
6_64
---> Package NetworkManager-adsl.x86_64 1:1.4.0-19.el7_3 will be installed
```

图 6-9 GNOME Desktop 图像界面安装

（4） 执行命令 yum install httpd php php-devel php-mysql mariadb mariadb-server -y，安装中小企业 LAMP 架构环境，如图 6-10 所示:

```
[root@www-jfedu-net ~]# yum install httpd php php-devel php-mysql mariadb m
Loaded plugins: fastestmirror, langpacks
Repository epel is listed more than once in the configuration
Loading mirror speeds from cached hostfile
Package httpd-2.4.6-45.el7.centos.4.x86_64 already installed and latest ver
Package php-5.4.16-42.el7.x86_64 already installed and latest version
Package php-devel-5.4.16-42.el7.x86_64 already installed and latest version
Package php-mysql-5.4.16-42.el7.x86_64 already installed and latest version
Package 1:mariadb-5.5.52-1.el7.x86_64 already installed and latest version
Resolving Dependencies
--> Running transaction check
---> Package mariadb-server.x86_64 1:5.5.52-1.el7 will be installed
--> Finished Dependency Resolution

Dependencies Resolved
```

图 6-10 LAMP 中小企业架构安装

（5） 执行命令 yum remove ntpdate -y，卸载 ntpdate 软件包，如图 6-11 所示:

```
[root@www-jfedu-net ~]# yum  remove  ntpdate -y
Loaded plugins: fastestmirror, langpacks
Repository epel is listed more than once in the configuration
Resolving Dependencies
--> Running transaction check
---> Package ntpdate.x86_64 0:4.2.6p5-25.el7.centos.2 will be erased
--> Processing Dependency: ntpdate = 4.2.6p5-25.el7.centos.2 for package:
entos.2.x86_64
--> Running transaction check
---> Package ntp.x86_64 0:4.2.6p5-25.el7.centos.2 will be erased
--> Finished Dependency Resolution

Dependencies Resolved

================================================================================
 Package          Arch           Version                                  Repo
```

图 6-11 卸载 NTPDATE 软件

（6） 执行命令 yum provides rz 或者 yum provides "*/rz"，查找 rz 命令的提供者，如图 6-12 所示：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# yum provides rz
Loaded plugins: fastestmirror, langpacks
Repository epel is listed more than once in the configuration
Loading mirror speeds from cached hostfile
lrzsz-0.12.20-36.el7.x86_64 : The lrz and lsz modem communications program
Repo        : os
Matched from:
Filename    : /usr/bin/rz


[root@www-jfedu-net ~]#
```

图 6-12 查找 RZ 命令的提供者

（7） 执行命令 yum update -y，升级 Linux 所有可更新的软件包或 Linux 内核升级，

如图 6-13 所示：

```
[root@www-jfedu-net ~]# yum update -y
Loaded plugins: fastestmirror, langpacks
Repository epel is listed more than once in the configuration
Loading mirror speeds from cached hostfile
Resolving Dependencies
--> Running transaction check
---> Package NetworkManager.x86_64 1:1.0.6-27.el7 will be updated
---> Package NetworkManager.x86_64 1:1.4.0-19.el7_3 will be an update
---> Package NetworkManager-libnm.x86_64 1:1.0.6-27.el7 will be updated
---> Package NetworkManager-libnm.x86_64 1:1.4.0-19.el7_3 will be an updat
---> Package NetworkManager-team.x86_64 1:1.0.6-27.el7 will be updated
---> Package NetworkManager-team.x86_64 1:1.4.0-19.el7_3 will be an update
---> Package NetworkManager-tui.x86_64 1:1.0.6-27.el7 will be updated
---> Package NetworkManager-tui.x86_64 1:1.4.0-19.el7_3 will be an update
---> Package abrt-libs.x86_64 0:2.1.11-36.el7.centos will be updated
---> Package abrt-libs.x86_64 0:2.1.11-45.el7.centos will be an update
```

图 6-13 软件包升级或内核升级

6.6 YUM 优先级配置实战

基于 YUM 安装软件时,通常会配置多个 Repo 源，而 Fastest mirror 插件是为拥有多个镜像的软件库配置文件而设计的。它会连接到每一个镜像，计算连接所需的时间，然后将镜像按快到慢排序供 YUM 应用。

默认 CentOS Linux 系统，Fastestmirror 插件是开启的，所以安装软件会从最快的

镜像源安装，但是由于 Repo 源很多，而在这些源中都存在某些软件包,但有些软件有重复,甚至冲突,能否可以优先从一些 Repo 源中去查找，如果找不到，再去其他源中找呢？

可以使用 YUM 优先级插件解决该问题，YUM 提供的插件 yum-plugin-priorities，直接 YUM 安装即可，命令如下：

yum install -y yum-plugin-priorities

```
Installed size: 28 k
Downloading packages:
yum-plugin-priorities-1.1.31-46.el7_5.noarch.rpm
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : yum-plugin-priorities-1.1.31-46.el7_5.noarch
  Verifying  : yum-plugin-priorities-1.1.31-46.el7_5.noarch

Installed:
  yum-plugin-priorities.noarch 0:1.1.31-46.el7_5

Complete!
```

修改 YUM 源优先级配置文件，设置为 Enabled，开启优先级插件，1 为开启，0 为禁止;

vim /etc/yum/pluginconf.d/priorities.conf

enabled = 1

```
[root@www-jfedu-net ~]# vim /etc/yum
yum/          yum.conf      yum.repos.d/
[root@www-jfedu-net ~]# vim /etc/yum/
fssnap.d/             protected.d/          version-groups.c
pluginconf.d/         vars/
[root@www-jfedu-net ~]# vim /etc/yum/pluginconf.d/
fastestmirror.conf  langpacks.conf      priorities.conf
[root@www-jfedu-net ~]# vim /etc/yum/pluginconf.d/prioriti
[main]
enabled = 1
~
~
```

vim 修改/etc/yum.repos./xx.repo 文件，在 base 段中加入如下指令：(优先级为 1 表示优先被查找，越大其反而被后续查找)

priority=1

```
# remarked out baseurl= line instead.
#
#
[base-163-com]
name=CentOS-$releasever - Base - 163.com
#mirrorlist=http://mirrorlist.centos.org/?release=$releas
baseurl=http://mirrors.163.com/centos/$releasever/os/$base
gpgcheck=1
priority=1
gpgkey=http://mirrors.163.com/centos/RPM-GPG-KEY-CentOS-7

#released updates
[updates]
```

基于 YUM 安装 ntpdate 软件，测试已经优先从 163 源中查找；

```
check
_64 0:4.2.6p5-28.el7.centos will be installed
Resolution

================================================================================
            Version                                    Repository
================================================================================

4           4.2.6p5-28.el7.centos                      base-163-com

================================================================================
```

6.7 基于 ISO 镜像构建 YUM 本地源

通常而言，YUM 客户端使用前提是必须联外网，YUM 安装软件时，检查 repo 配置文件查找相应的 YUM 源仓库，企业 IDC 机房很多服务器为了安全起见，是禁止服务器上外网的，所以不能使用默认的官方 YUM 源仓库。

构建本地 YUM 光盘源，其原理是通过查找光盘中的软件包，实现 YUM 安装，配置

步骤如下：

（1） 将 CentOS-7-x86_64-DVD-1511.iso 镜像加载至虚拟机 CD/DVD 或者放入服务器 CD/DVD 光驱中，并将镜像文件挂载至服务器/mnt 目录，如图 6-14 所示，挂载命令：

```
mount        /dev/cdrom      /mnt/
```

```
[root@www-jfedu-net ~]# cd
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# mount      /dev/cdrom    /mnt/
mount: /dev/sr0 is write-protected, mounting read-only
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# cd /mnt/
[root@www-jfedu-net mnt]#
[root@www-jfedu-net mnt]# ls
CentOS_BuildTag  EULA  images    LiveOS    repodata              RPM-GPG-
EFI              GPL   isolinux  Packages  RPM-GPG-KEY-CentOS-7  TRANS.TB
[root@www-jfedu-net mnt]#
[root@www-jfedu-net mnt]#
```

图 6-14 CentOS ISO 镜像文件挂载

（2） 备份/etc/yum.repos.d/CentOS-Base.repo 文件为 CentOS-Base.repo.bak，同时在/etc/yum.repos.d 目录下创建 media.repo 文件，并写入如下内容：

```
[yum]

name=CentOS7

baseurl=file:///mnt

enabled=1

gpgcheck=1
```

```
gpgkey=file:///mnt/RPM-GPG-KEY-CentOS-7
```

Media.repo 配置文件详解：

```
name=CentOS7                                    YUM 源显示名称；
baseurl=file:///mnt                             ISO 镜像挂载目录；
gpgcheck=1                                      是否检查 GPG-KEY；
enabled=1                                       是否启用 YUM 源；
gpgkey=file:///mnt/RPM-GPG-KEY-CentOS-7         指定载目录下的 GPG-KEY 文件
验证。
```

（3） 运行命令 yum clean all 清空 YUM Cache，执行 yum install screen –y 安装 screen 软件如图 6-15 所示：

```
[root@www-jfedu-net ~]# yum clean all
Loaded plugins: fastestmirror
Cleaning repos: yum
Cleaning up everything
Cleaning up list of fastest mirrors
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# yum install screen -y
Loaded plugins: fastestmirror
yum
(1/2): yum/group_gz
(2/2): yum/primary_db
Determining fastest mirrors
Resolving Dependencies
--> Running transaction check
---> Package screen.x86_64 0:4.1.0-0.21.20120314git3c2946.el7 will be insta
--> Finished Dependency Resolution

Dependencies Resolved
```

图 6-15 YUM 安装 Screen 软件

（4） 至此 YUM 光盘源构建完毕，在使用 YUM 源时，会遇到部分软件无法安装，原因是因为光盘中软件包不完整导致，同时光盘源只能本机使用，其他局域网服务器

无法使用。

6.8 基于 HTTP 构建 YUM 网络源

YUM 光盘源默认只能本机使用，局域网其他服务器无法使用 YUM 光盘源，如果想使用的话，需要在每台服务器上构建 YUM 本地源，该方案在企业中不可取，所以需要构建 HTTP 局域网 YUM 源解决，可以通过 CreateRepo 创建本地 YUM 源端，repo 即为 Repository。

构建 HTTP 局域网 YUM 源方法及步骤如下：

（1） 挂载光盘镜像文件至/mnt

```
mount      /dev/cdrom      /mnt/
```

（2） 拷贝/mnt/Packages 目录下所有软件包至/var/www/html/centos/

```
mkdir   -p   /var/www/html/centos/

cp      -R    /mnt/Packages/*   /var/www/html/centos/
```

（3） 使用 Createrepo 创建本地源，执行如下命令会在 Centos 目录生成 repodata 目录，目录内容如图 6-16 所示：

```
yum   install   createrepo*   -y

cd /var/www/html

createrepo   centos/
```

```
[root@www-jfedu-net html]# pwd
/var/www/html
[root@www-jfedu-net html]#
[root@www-jfedu-net html]# cd centos/
[root@www-jfedu-net centos]#
[root@www-jfedu-net centos]#
[root@www-jfedu-net centos]# cd repodata/
[root@www-jfedu-net repodata]#
[root@www-jfedu-net repodata]# ls
40bac61f2a462557e757c2183511f57d07fba2c0dd63f99b48f0b466b7f2b8d2-other.xml.gz
4cdf96c7618bdbb2dfa543c29496faf76d940f0c04f316c8a3476426c65c81cf-primary.sqlite.bz2
9710c85f1049b4c60c74ae5fd51d3e98e4ecd50a43ab53ff64169Ofb164a6d63-other.sqlite.bz2
cfa741341d5d270d5b42d6220e2908d053c39a2d8346986bf48cee360e6f7ce8-filelists.xml.gz
d216d0fba4167a2d086c80ac8c708670a7e45ee3295f27ac2702784fc56623b4-primary.xml.gz
d863fcc08a4e8d47382001c3f22693ed77e03815a76cedf34d8256d4c12f6f0d-filelists.sqlite.bz2
repomd.xml
```

图 6-16 Createrepo 生成 repodata 目录

（4） 利用 HTTP 发布 YUM 本地源

本地 YUM 源通过 CreateRepo 搭建完毕，需要借助 HTTP WEB 服务器发布/var/www/html/centos/中所有软件，YUM 或者 RPM 安装 HTTP WEB 服务器，并启动 httpd 服务。

yum install httpd httpd-devel -y	安装 HTTP WEB 服务；
useradd apache -g apache	创建 apache 用户和组；
systemctl restart httpd.service	重启 HTTPD 服务；
setenforce 0	临时关闭 SeLinux 应用级安全策略；
systemctl stop firewalld.service	停止防火墙；
ps -ef |grep httpd	查看 HTTPD 进程是否启动。

（5） 在 YUM 客户端，创建/etc/yum.repos.d/http.repo 文件，写入如下内容：

```
[base]
name="CentOS7 HTTP YUM"
baseurl=http://192.168.1.115/centos/
```

```
gpgcheck=0

enabled=1

[updates]

name="CentOS7 HTTP YUM"

baseurl=http://192.168.1.115/centos

gpgcheck=0

enabled=1
```

（6） 至此在 YUM 客户端上执行如下命令，如图 6-17 所示：

yum clean all	清空 YUM Cache；
yum install ntpdate -y	安装 NTPDATE 软件。

```
[root@www-jfedu-net ~]# yum clean all
Loaded plugins: fastestmirror
Cleaning repos: base updates
Cleaning up everything
Cleaning up list of fastest mirrors
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# yum install ntpdate -y
Loaded plugins: fastestmirror
base
updates
(1/2): base/primary_db
(2/2): updates/primary_db
Determining fastest mirrors
Resolving Dependencies
--> Running transaction check
```

图 6-17 HTTP YUM 源客户端验证

6.9 YUM 源端软件包扩展

默认使用 ISO 镜像文件中的软件包构建的 HTTP YUM 源，会发现缺少很多软件包，

如果服务器需要挂载移动硬盘，Mount 挂载移动硬盘需要 ntfs-3g 软件包支持，而本地光盘镜像中没有该软件包，此时需要往 YUM 源端添加 ntfs-3g 软件包，添加方法如下：

（1） 切换至/var/www/html/centos 目录，官网下载 NTFS-3G 软件包。

```
cd /var/www/html/centos/
wget http://dl.fedoraproject.org/pub/epel/7/x86_64/n/ntfs-3g-2016.2.22-3.el7.x86_64.rpm
http://dl.fedoraproject.org/pub/epel/7/x86_64/n/ntfs-3g-devel-2016.2.22-3.el7.x86_64.rpm
```

（2） Createrepo 命令更新软件包，同理，如需新增其他软件包，同样把软件下载至本地，然后通过 createrepo 更新即可，如图 6-18 所示：

```
createrepo --update centos/
```

```
[root@www-jfedu-net html]#
[root@www-jfedu-net html]# ls centos/ntfs-3g-*
centos/ntfs-3g-2016.2.22-3.el7.x86_64.rpm  centos/ntfs-3g-devel-2016.
[root@www-jfedu-net html]#
[root@www-jfedu-net html]#
[root@www-jfedu-net html]# createrepo --update centos/
Spawning worker 0 with 2 pkgs
Workers Finished
Saving Primary metadata
Saving file lists metadata
Saving other metadata
Generating sqlite DBs
Sqlite DBs complete
```

图 6-18 CreateRepo update 更新软件包

（3） 客户端 YUM 验证，安装 NTFS-3G 软件包，如图 6-19 所示：

```
[root@www-jfedu-net ~]# yum clean all
Loaded plugins: fastestmirror
Cleaning repos: base updates
Cleaning up everything
Cleaning up list of fastest mirrors
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# yum install ntfs-3g -y
Loaded plugins: fastestmirror
base
updates
(1/2): base/primary_db
(2/2): updates/primary_db
Determining fastest mirrors
Resolving Dependencies
--> Running transaction check
---> Package ntfs-3g.x86_64 2:2016.2.22-3.el7 will be installed
--> Finished Dependency Resolution

Dependencies Resolved
```

图 6-19　YUM INSTALL NTFS-3G 软件包

6.10 同步外网 YUM 源

在企业实际应用场景中，仅仅靠光盘里面的 RPM 软件包是不能满足需要，我们可以把外网的 YUM 源中的所有软件包同步至本地，可以完善本地 YUM 源的软件包数量及完整性。

获取外网 YUM 源软件常见方法包括 Rsync、Wget、Reposync，三种同步方法的区别 Rsync 方式需要外网 YUM 源支持 RSYNC 协议，Wget 可以直接获取，而 Reposync 可以同步几乎所有的 YUM 源，下面以 Reporsync 为案例，同步外网 YUM 源软件至本地，步骤如下：

（1）　下载 CentOS7 REPO 文件至/etc/yum.repos.d/，并安装 reposync 命令工具：

```
wget  http://mirrors.163.com/.help/CentOS7-Base-163.repo

mv CentOS7-Base-163.repo /etc/yum.repos.d/centos.repo
```

```
yum    clean all

yum install yum-utils createrepo –y

yum repolist
```

（2） 通过 reposync 命令工具获取外网 YUM 源所有软件包，-r 指定 repolist id，默认不加-r 表示获取外网所有 YUM 软件包，-p 参数表示指定下载软件的路径，如图 6-20（a）、图 6-20（b）所示：

```
reposync   -r   base       -p       /var/www/html/centos/

reposync   -r   updates    -p       /var/www/html/centos/
```

```
[root@www-jfedu-net ~]# reposync  -r  base      -p      /var/www/html/centos/
No Presto metadata available for base
(1/9299): ORBit2-devel-2.14.19-13.el7.x86_64.rpm
(2/9299): ORBit2-2.14.19-13.el7.i686.rpm
(3/9299): OpenEXR-devel-1.7.1-7.el7.i686.rpm
(4/9299): OpenEXR-devel-1.7.1-7.el7.x86_64.rpm
(5/9299): OpenEXR-1.7.1-7.el7.x86_64.rpm
(6/9299): OpenEXR-libs-1.7.1-7.el7.x86_64.rpm
(7/9299): OpenIPMI-2.0.19-15.el7.x86_64.rpm
(8/9299): OpenEXR-libs-1.7.1-7.el7.i686.rpm
(9/9299): OpenIPMI-devel-2.0.19-15.el7.x86_64.rpm
(10/9299): OpenIPMI-devel-2.0.19-15.el7.i686.rpm
(11/9299): OpenIPMI-libs-2.0.19-15.el7.x86_64.rpm
```

图 6-20（a） Reposync 获取外网 YUM 源软件包

```
[root@www-jfedu-net ~]# reposync  -r  updates     -p      /var/www/html/centos/
(1/1562): 389-ds-base-libs-1.3.5.10-15.el7_3.x86_64.rpm
(2/1562): 389-ds-base-snmp-1.3.5.10-12.el7_3.x86_64.rpm
(3/1562): 389-ds-base-libs-1.3.5.10-20.el7_3.x86_64.rpm
(4/1562): 389-ds-base-snmp-1.3.5.10-15.el7_3.x86_64.rpm
(5/1562): 389-ds-base-snmp-1.3.5.10-18.el7_3.x86_64.rpm
(6/1562): NetworkManager-1.4.0-13.el7_3.x86_64.rpm
(7/1562): 389-ds-base-snmp-1.3.5.10-20.el7_3.x86_64.rpm
(8/1562): NetworkManager-1.4.0-17.el7_3.x86_64.rpm
(9/1562): NetworkManager-1.4.0-19.el7_3.x86_64.rpm
(10/1562): NetworkManager-1.4.0-14.el7_3.x86_64.rpm
(11/1562): NetworkManager-adsl-1.4.0-14.el7_3.x86_64.rpm
(12/1562): NetworkManager-adsl-1.4.0-13.el7_3.x86_64.rpm
(13/1562): NetworkManager-adsl-1.4.0-17.el7_3.x86_64.rpm
(14/1562): NetworkManager-adsl-1.4.0-19.e 0% [                                ]
```

图 6-20（b） Reposync 获取外网 YUM 源软件包

（3） 通过 reposync 工具下载完所有的软件包之后，需要执行 createrepo 更新本地 YUM 仓库：

```
createrepo   /var/www/html/centos/
```

6.11 本章小结

通过对本章内容的学习，读者掌握了 Linux 安装不同包的工具及命令，使用 RPM 及 YUM 管理.RPM 结尾的二进制包，基于 configure、make、make install 实现源码包安装，并能够对软件进行安装、卸载及维护。

能够独立构建企业光盘源、HTTP 网络 YUM 源，实现无外网网络使用 YUM 安装各种软件包及工具，同时能随时添加新的软件包至本地 Yum 源中。

6.12 同步作业

1. RPM 及 YUM 管理工具的区别是什么？
2. 企业中安装软件，何时选择 YUM 安装或者源码编译安装？
3. 将 Linux 系统中 PHP5.3 版本升级至 PHP5.5 版本，升级方法有几种，分别写出升级步骤？
4. 使用源码编译安装 httpd-2.4.25.tar.bz2，写出安装的流程及注意事项。
5. 如何将 CentOS 7 Linux 字符界面升级为图形界面，并设置系统启动默认为图形界面？

第7章 Linux 磁盘管理

Linux 系统一切以文件的方式存储于硬盘，应用程序数据需要时刻读写硬盘，所以企业生产环境中对硬盘的操作变得尤为重要，对硬盘的维护和管理也是每个运维工程师必备工作之一。

本章向读者介绍硬盘简介、硬盘数据存储方式、如何在企业生产服务器添加硬盘、对硬盘进行分区、初始化以及对硬盘进行故障修复等。

7.1 计算机硬盘简介

硬盘是计算机主要存储媒介之一，由一个或者多个铝制或者玻璃制的碟片组成，碟片外覆盖有铁磁性材料，硬盘内部由磁道、柱面、扇区、磁头等部件组成，如图 7-1 所示：

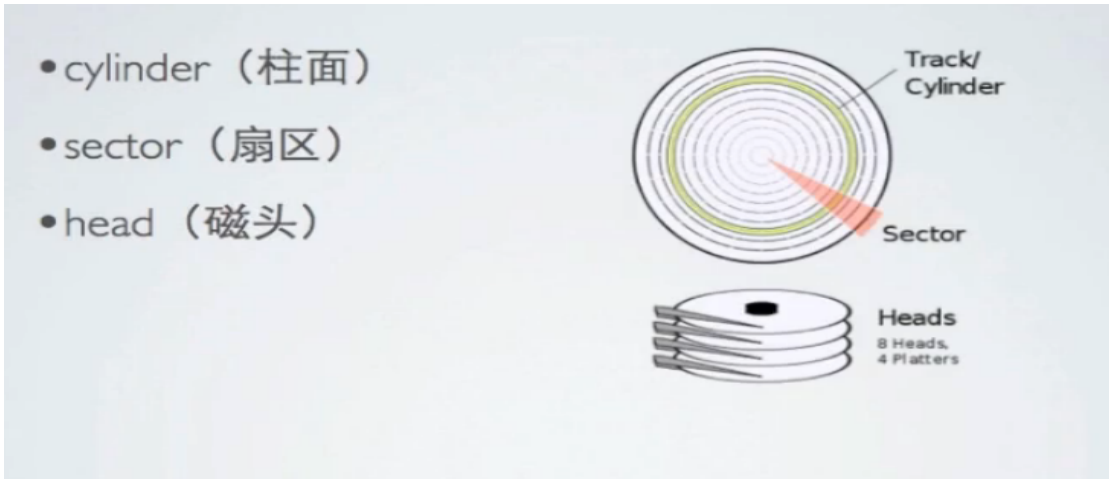


图 7-1 硬盘内部结构组成

Linux 系统中硬件设备相关配置文件存放在/dev/下，常见硬盘命名：/dev/hda、

/dev/sda、/dev/sdb、/dev/sdc、/dev/vda。不同硬盘接口，在系统中识别的设备名称不一样。

IDE 硬盘接口在 Linux 中设备名为/dev/hda，SAS、SCSI、SATA 硬盘接口在 Linux 中设备名为 sda，高效云盘硬盘接口会识别为/dev/vda 等。

文件储存在硬盘上，硬盘的最小存储单位叫做 Sector（扇区），每个 Sector 储存 512 字节。操作系统在读取硬盘的时候，不会逐个 Sector 的去读取，这样效率非常低，为了提升读取效率，操作系统会一次性连续读取多个 Sector，即一次性读取多个 Sector 称为一个 Block（块）。

由多个 Sector 组成的 Block 是文件存取的最小单位。Block 的大小常见的有 1KB、2KB、4KB，Block 在 Linux 中常设置为 4KB，即连续八个 Sector 组成一个 Block。

/boot 分区 Block 一般为 1KB，而/data/分区或者/分区的 Block 为 4K。可以通过如下三种方法查看 Linux 分区的 Block 大小：

```
dumpe2fs /dev/sda1 |grep "Block size"

tune2fs -l /dev/sda1 |grep "Block size"

stat /boot/|grep "IO Block"
```

例如创建一个普通文件，文件大小为 10Bytes，而默认设置 Block 为 4K，如果有 1 万个小文件，由于每个 Block 只能存放一个文件，如果文件的大小比 Block 大，会申请更多的 Block，相反如果文件的大小比默认 Block 小，仍会占用一个 Block，这样剩余的空间会被浪费掉。

- 1 万个文件理论只占用空间大小：10000x10=100000Bytes=97.65625MBytes；
- 1 万个文件真实占用空间大小：

  10000x4096Bytes=40960000Bytes=40000MBytes=40GB。
- 根据企业实际需求，此时可以将 Block 设置为 1K，从而节省更多的空间。

7.2 硬盘 Block 及 Inode 详解

通常而言，操作系统对于文件数据的存放包括两个部分：文件内容、权限及文件属性。操作系统文件存放是基于文件系统，文件系统会将文件的实际内容存储到 Block 中，而将权限与属性等信息存放至 Inode 中。

在硬盘分区中，还有一个超级区块（SuperBlock），SuperBlock 会记录整个文件系统的整体信息，包括 Inode、Block 总量、使用大小、剩余大小等信息，每个 inode 与 block 都有编号对应，方便 Linux 系统快速定位查找文件。

- Superblock：记录文件系统的整体信息，包括 inode 与 block 的总量、使用大小、剩余大小， 以及文件系统的格式与相关信息等；
- Inode：记录文件的属性，权限，同时会记录该文件的数据所在的 block 编号；
- Block：存储文件的内容 如果文件超过默认 Block 大小 ,会自动占用多个 Block。

因为每个 inode 与 block 都有编号，而每个文件都会占用一个 inode ，inode 内则有文件数据放置的 block 号码。如果能够找到文件的 inode，就可以找到该文件所放置数据的 block 号码，从而读取该文件内容。

操作系统进行格式化分区时，操作系统自动将硬盘分成两个区域。一个是数据 Block 区，用于存放文件数据；另一个是 Inode Table 区，用于存放 inode 包含的元信息。

每个 inode 节点的大小，可以在格式化时指定 默认为 128Bytes 或 256Bytes ，/boot 分区 Inode 默认为 128Bytes，其他分区默认为 256Bytes，查看 Linux 系统 Inode 方法如下：

```
dumpe2fs     /dev/sda1 |grep " Inode size "

tune2fs    -l    /dev/sda1 |grep " Inode size"

stat    /boot/|grep "Inode"
```

格式化磁盘时，可以指定默认 Inode 和 Block 的大小，-b 指定默认 Block 值，-I 指定默认 Inode 值，如图 7-2 所示，命令如下：

```
mkfs.ext4 -b 4096 -I 256 /dev/sdb
```

```
[root@www-jfedu-net ~]# mkfs.ext4 -b 4096 -I 256 /dev/sdb
mke2fs 1.42.9 (28-Dec-2013)
/dev/sdb is entire device, not just one partition!
Proceed anyway? (y,n) y
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
2621440 inodes, 10485760 blocks
524288 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=2157969408
320 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
```

图 7-2 格式化硬盘指定 Inode 和 Block

7.3 硬链接介绍

一般情况下，文件名和 inode 编号是一一对应的关系，每个 inode 号码对应一个文件名。但 UNIX/Linux 系统多个文件名也可以指向同一个 inode 号码。这意味着可以用不同的文件名访问同样的内容，对文件内容进行修改，会影响到所有文件名。但删除一个文件名，不影响另一个文件名的访问。这种情况就被称为硬链接（hard link）。

创建硬链接的命令为：ln　jf1.txt　jf2.txt，其中 jf1.txt 为源文件，jf2.txt 为目标文件。如上命令源文件与目标文件的 inode 号码相同，都指向同一个 inode。inode 信息中有一项叫做"链接数"，记录指向该 inode 的文件名总数，这时会增加 1，变成 2，如图 7-3 所示：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# touch jf1.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll jf1.txt
-rw-r--r-- 1 root root 0 May  5 17:19 jf1.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ln jf1.txt jf2.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll jf1.txt
-rw-r--r-- 2 root root 0 May  5 17:19 jf1.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll jf2.txt
-rw-r--r-- 2 root root 0 May  5 17:19 jf2.txt
[root@www-jfedu-net ~]#
```

图 7-3 jf1.txt jf2.txt 硬链接 inode 值变化

同样删除一个 jf2.txt 文件，就会使得 jf1.txt inode 节点中的"链接数"减 1。如果该 inode 值减到 0，表明没有文件名指向这个 inode，系统就会回收这个 inode 号码，以及其所对应 block 区域，如图 7-4 所示：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll jf1.txt jf2.txt
-rw-r--r-- 2 root root 0 May  5 17:19 jf1.txt
-rw-r--r-- 2 root root 0 May  5 17:19 jf2.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# rm -rf jf2.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ll jf1.txt
-rw-r--r-- 1 root root 0 May  5 17:19 jf1.txt
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
```

图 7-4 删除 jf2.txt 硬链接 inode 值变化

实用小技巧：硬链接不能跨分区链接，硬链接只能对文件生效，对目录无效，也即是目录不能创建硬链接。硬链接源文件与目标文件共用一个 inode 值，从某种意义上来，节省 inode 空间。不管是单独删除源文件还是删除目标文件，文件内容始终存在。同时链接后的文件不占用系统多余的空间。

7.4 软链接介绍

除了硬链接以外，还有一种链接-软链接。文件 jf1.txt 和文件 jf2.txt 的 inode 号码虽然不一样，但是文件 jf2.txt 的内容是文件 jf1.txt 的路径。读取文件 jf2.txt 时，系统会自动将访问者导向文件 jf1.txt。

无论打开哪一个文件，最终读取的都是文件 jf1.txt。这时，文件 jf2.txt 就称为文件 jf1.txt 的"软链接"（soft link）或者"符号链接（symbolic link）。

文件 jf2.txt 依赖于文件 jf1.txt 而存在，如果删除了文件 jf1.txt，打开文件 jf2.txt 就会报错："No such file or directory"。

软链接与硬链接最大的不同是文件 jf2.txt 指向文件 jf1.txt 的文件名，而不是文件

jf1.txt 的 inode 号码，因此文件 jf1.txt 的 inode 链接数不会发生变化，如图 7-5 所示：

```
[root@localhost ~]# ls -li jf1.txt
790403 -rw-r--r-- 1 root root 0 May  5 17:50 jf1.txt
[root@localhost ~]#
[root@localhost ~]# ln -s jf1.txt jf2.txt
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# ll -li jf1.txt jf2.txt
790403 -rw-r--r-- 1 root root 0 May  5 17:50 jf1.txt
795230 lrwxrwxrwx 1 root root 7 May  5 17:50 jf2.txt -> jf1.txt
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# rm -rf jf1.txt
[root@localhost ~]#
[root@localhost ~]# ll -li jf2.txt
795230 lrwxrwxrwx 1 root root 7 May  5 17:50 jf2.txt -> jf1.txt
[root@localhost ~]#
```

图 7-5 删除 jf1.txt 源文件链接数不变

实用小技巧：软链接可以跨分区链接，软链接支持目录同时也支持文件的链接。软链接源文件与目标文件 Inode 不相同，从某种意义上来，会消耗省 inode 空间。不管是删除源文件还是重启系统，该软链接还存在，但是文件内容会丢失，一旦新建源同名文件名，软链接文件恢复正常。

7.5 Linux 下磁盘实战操作命令

企业真实场景由于硬盘常年大量读写，经常会出现坏盘，需要更换硬盘。或者由于磁盘空间不足，需添加新硬盘，新添加的硬盘需要经过格式化、分区才能被 Linux 系统所使用，虚拟机 CentOS 7 Linux 模拟 DELL R730 真实服务器添加一块新硬盘，不需要关机，直接插入用硬盘即可，一般硬盘均支持热插拔功能。企业中添加新硬盘的操作流程如下：

（1） 检测 Linux 系统识别的硬盘设备，新添加硬盘被识别为/dev/sdb，如果有多块

硬盘，会依次识别成/dev/sdc、/dev/sdd 等设备名称，如图 7-6 所示：

```
fdisk  -l
```

```
[root@www-jfedu-net ~]# fdisk -l

Disk /dev/sda: 42.9 GB, 42949672960 bytes, 83886080 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x00077233

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1   *        2048      411647      204800   83  Linux
/dev/sda2          411648     1460223      524288   82  Linux swap / Solar
/dev/sda3         1460224    78217215    38378496   83  Linux

Disk /dev/sdb: 42.9 GB, 42949672960 bytes, 83886080 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

图 7-6 Fdisk 查看 Linux 系统硬盘设备

（2） 基于新硬盘/dev/sdb 设备，创建磁盘分区/dev/sdb1，如图 7-7 所示：

```
fdisk /dev/sdb
```

```
[root@www-jfedu-net ~]# fdisk /dev/sdb
Welcome to fdisk (util-linux 2.23.2).

Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table
Building a new DOS disklabel with disk identifier 0x3e5bdbfe.

Command (m for help): m
Command action
   a   toggle a bootable flag
   b   edit bsd disklabel
   c   toggle the dos compatibility flag
   d   delete a partition
   g   create a new empty GPT partition table
   G   create an IRIX (SGI) partition table
   l   list known partition types
   m   print this menu
```

图 7-7 Fdisk /dev/sdb 分区

（3） fdisk 分区命令参数如下，常用参数包括 m、n、p、e、d、w。

b	编辑 bsd disklabel；
c	切换 dos 兼容性标志；
d	删除一个分区；
g	创建一个新的空 GPT 分区表；
G	创建一个 IRIX（SGI）分区表；
l	列出已知的分区类型；
m	打印帮助菜单；
n	添加一个新分区；
o	创建一个新空 DOS 分区表；
p	打印分区表信息；
q	退出而不保存更改；
s	创建一个新的空的 Sun 磁盘标签；
t	更改分区的系统 ID；
u	更改显示/输入单位；
v	验证分区表；
w	将分区表写入磁盘并退出；
x	额外功能。

（4） 创建/dev/sdb1 分区方法，fdisk /dev/sdb，然后按 n-p-1-Enter 键- +20G-

Enter 键-w，最后执行 fdisk –l|tail -10，如图 7-8（a）、图 7-8（b）所示：

```
[root@www-jfedu-net ~]# fdisk /dev/sdb
Welcome to fdisk (util-linux 2.23.2).

Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table
Building a new DOS disklabel with disk identifier 0x3cf5bf9c.

Command (m for help): n
Partition type:
   p   primary (0 primary, 0 extended, 4 free)
   e   extended
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-83886079, default 2048):
Using default value 2048
Last sector, +sectors or +size{K,M,G} (2048-83886079, default 83886079): +20G
Partition 1 of type Linux and of size 20 GiB is set

Command (m for help): w
```

图 7-8（a） Fdisk /dev/sdb 创建/dev/sdb1 分区

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# fdisk -l|tail -10

Disk /dev/sdb: 42.9 GB, 42949672960 bytes, 83886080 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x3cf5bf9c

   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048    41945087    20971520   83  Linux
[root@www-jfedu-net ~]#
```

图 7-8（b） Fdisk –l 查看/dev/sdb1 分区

（5） mkfs.ext4 /dev/sdb1 格式化磁盘分区，如图 7-9 所示：

```
[root@www-jfedu-net ~]# mkfs.ext4 /dev/sdb1
mke2fs 1.42.9 (28-Dec-2013)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
1310720 inodes, 5242880 blocks
262144 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=2153775104
160 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 265
        4096000
```

图 7-9　mkfs.ext4 格式化磁盘分区

（6）　/dev/sdb1 分区格式化，使用 mount 命令挂载到/data/目录，如图 7-10 所示：

mkdir -p /data/	创建/data/数据目录
mount /dev/sdb1 /data	挂载/dev/sdb1 分区至/data/目录
df -h	查看磁盘分区详情
echo "mount /dev/sdb1 /data" >>/etc/rc.local	将挂载分区命令加入/etc/rc.local 开机启动

```
[root@www-jfedu-net ~]# mkdir -p /data/
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# mount  /dev/sdb1   /data
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3        37G  5.7G   31G  16% /
devtmpfs        484M     0  484M   0% /dev
tmpfs           493M     0  493M   0% /dev/shm
tmpfs           493M   13M  480M   3% /run
tmpfs           493M     0  493M   0% /sys/fs/cgroup
/dev/sda1       197M  103M   94M  53% /boot
tmpfs            99M     0   99M   0% /run/user/0
/dev/sdb1        20G   45M   19G   1% /data
[root@www-jfedu-net ~]# echo "mount  /dev/sdb1   /data" >>/etc/rc.local
```

图 7-10　MOUNT 挂载/dev/sdb1 磁盘分区

（7） 自动挂载分区除了可以加入到/etc/rc.local 开机启动之外，还可以加入到/etc/fstab 文件中，如图 7-11 所示：

/dev/sdb1 /data/ ext4 defaults 0 0
mount -o rw,remount / 重新挂载/系统，检测/etc/fstab 是否有误。

```
[root@www-jfedu-net ~]# vim /etc/fstab
# /etc/fstab
# Created by anaconda on Sat Aug 20 13:10:05 2016
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
#
UUID=01581e1f-c36c-4968-a5b8-7b570e424f1c /                       xfs     defaults
UUID=36ff9bf6-e8a4-48da-9bba-b9c7a89fe151 /boot                   xfs     defaults
UUID=fae320d7-1e4c-487d-87ea-8a79e1d0885a swap                    swap    defaults
/dev/sdb1                                 /data/                  ext4    defaults
~
```

图 7-11 /dev/sdb1 磁盘分区加入/etc/fstab 文件

7.6 基于 GPT 格式磁盘分区

MBR 分区标准决定了 MBR 只支持在 2TB 以下的硬盘，为了支持能使用大于 2T 硬盘空间，需使用 GPT 格式进行分区。创建大于 2TB 的分区，需使用 parted 工具。

在企业真实环境中，通常一台服务器有多块硬盘，整个硬盘容量为 10T，需要基于 GTP 格式对 10T 硬盘进行分区，操作步骤如下：

parted -s /dev/sdb mklabel gpt 设置分区类型为 gpt 格式；
mkfs.ext3 /dev/sdb 基于 Ext3 文件系统类型格式化；

mount	/dev/sdb	/data/	挂载/dev/sdb 设备至/data/目录。

（1） 如图 7-12 所示，假设/dev/sdb 为 10T 硬盘，使用 GPT 格式来格式化磁盘：

```
Partition 2 does not end on cylinder boundary.
/dev/sda3               536        6528    48126976

Disk /dev/sdb: 53.7 GB, 53687091200 bytes
255 heads, 63 sectors/track, 6527 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Sector size (logical/physical): 512 bytes / 512 b
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000

[root@localhost ~]# fdisk -l
```

图 7-12 假设/dev/sdb 为 10T 设备

（2） 执行命令：parted -s /dev/sdb mklabel gpt，如图 7-13 所示：

```
[root@localhost ~]# parted -s /dev/sdb  mklabel gpt
[root@localhost ~]#
[root@localhost ~]# fdisk -l |tail

WARNING: GPT (GUID Partition Table) detected on '/dev/sdb'! The
 Use GNU Parted.


Disk /dev/sdb: 53.7 GB, 53687091200 bytes
255 heads, 63 sectors/track, 6527 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000

   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1               1        6528    52428799+  ee  GPT
[root@localhost ~]#
```

图 7-13 设置/dev/sdb 为 GPT 格式磁盘

（3） 基于 mkfs.ext3 /dev/sdb 格式化磁盘，如图 7-14 所示：

```
[root@localhost ~]# mkfs.ext3 /dev/sdb
mke2fs 1.41.12 (17-May-2010)
/dev/sdb is entire device, not just one partition!
无论如何也要继续? (y,n) y
文件系统标签=
操作系统:Linux
块大小=4096 (log=2)
分块大小=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
3276800 inodes, 13107200 blocks
655360 blocks (5.00%) reserved for the super user
第一个数据块=0
Maximum filesystem blocks=4294967296
```

图 7-14　格式/dev/sdb 磁盘

parted 命令行也可以进行分区，如图 7-15（a）、7-15（b）、7-15（c）所示：

parted→select /dev/sdb→mklabel gpt→mkpart primary 0　-1→print

mkfs.ext3　/dev/sdb1

mount　　/dev/sdb1　/data/

```
[root@localhost ~]# parted
GNU Parted 2.1
使用 /dev/sda
Welcome to GNU Parted! Type 'help' to view a list of c
(parted) select /dev/sdb                  选择/dev/sdb硬盘
使用 /dev/sdb
(parted)
(parted) mklabel gpt                      格式类型为GPT
警告: The existing disk label on /dev/sdb will be dest
lost. Do you want to continue?
是/Yes/否/No? yes
(parted)
(parted) mkpart primary 0 -1              将整块硬盘分为一个分区
警告: The resulting partition is not properly aligned
忽略/Ignore/放弃/Cancel?
忽略/Ignore/放弃/Cancel? ignore
(parted)
(parted) print                            打印我们刚分区的磁盘信息
Model: VMware, VMware Virtual S (scsi)
Disk /dev/sdb: 53.7GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
```

图 7-15（a）parted 工具执行 GPT 格式分区

```
[root@localhost ~]# mkfs.ext3  /dev/sdb1
mke2fs 1.41.12 (17-May-2010)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
1310720 inodes, 5242631 blocks
262131 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=4294967296
160 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632
```

图 7-15（b）parted 工具执行 GPT 格式分区

```
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# mount      /dev/sdb1     /data/
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3        30G  4.8G   23G  18% /
tmpfs           242M     0  242M   0% /dev/shm
/dev/sda1       194M   44M  141M  24% /boot
/dev/sdb1        20G  173M   19G   1% /data
[root@localhost ~]#
[root@localhost ~]#
```

图 7-15（c） parted 工具执行 GPT 格式分区

7.7 MOUNT 命令工具

Mount 命令工具主要用于将设备或者分区挂载至 Linux 系统目录下，Linux 系统在分区时，也是基于 mount 机制将/dev/sda 分区挂载至系统目录，将设备与目录挂载之后，Linux 操作系统方可进行文件的存储。

7.7.1 Mount 命令参数详解

如下为企业中 Mount 命令常用参数详解：

mount [-Vh]

mount -a [-fFnrsvw] [-t vfstype]

mount [-fnrsvw] [-o options [,...]] device | dir

mount [-fnrsvw] [-t vfstype] [-o options] device dir

-V： 显示 mount 工具版本号；

-l： 显示已加载的文件系统列表；

-h： 显示帮助信息并退出；

-v： 输出指令执行的详细信息；

-n： 加载没有写入文件/etc/mtab 中的文件系统；

-r： 将文件系统加载为只读模式；

-a： 加载文件/etc/fstab 中配置的所有文件系统；

-o: 指定 mount 挂载扩展参数，常见扩展指令：rw、remount、loop 等，其中-o 相关指令如下：

-o atime： 系统会在每次读取文档时更新文档时间；

-o noatime： 系统会在每次读取文档时不更新文档时间；

-o defaults: 使用预设的选项 rw,suid,dev,exec,auto,nouser 等；

-o exec	允许执行档被执行；
-o user、-o nouser：	使用者可以执行 mount/umount 的动作；
-o remount：	将已挂载的系统分区重新以其他再次模式挂载；
-o ro：	只读模式挂载；
-o rw：	可读可写模式挂载；
-o loop	使用loop模式 ,把文件当成设备挂载至系统目录。
-t：	指定mount挂载设备类型 ,常见类型nfs、ntfs-3g、vfat、iso9660 等，其中-t 相关指令如下：
iso9660	光盘或光盘镜像；
msdos	Fat16 文件系统；
vfat	Fat32 文件系统；
ntfs	NTFS 文件系统；
ntfs-3g	识别移动硬盘格式；
smbfs	挂载 Windows 文件网络共享；
nfs	Unix/Linux 文件网络共享。

7.7.2 企业常用 Mount 案例

Mount 常用案例演示如下：

mount	/dev/sdb1	/data	挂载 /dev/sdb1 分区至

/data/目录

mount /dev/cdrom /mnt 挂载 Cdrom 光盘至/mnt 目录；

mount -t ntfs-3g /dev/sdc /data1 挂载/dev/sdc 移动硬盘至/data1 目录；

mount -o remount,rw / 重新以读写模式挂载/系统；

mount -t iso9660 -o loop centos7.iso /mnt 将 centos7.iso 镜像文件挂载至/mnt 目录；

mount -t fat32 /dev/sdd1 /mnt 将 U 盘/dev/sdd1 挂载至/mnt/目录；

mount -t nfs 192.168.1.11:/data/ /mnt 将 远 程 192.168.1.11:/data 目录挂载至本地/mnt 目录。

7.8 Linux 硬盘故障修复

企业服务器运维中，经常会发现操作系统的分区变成只读文件系统，错误提示信息为

“Read-only file system”，出现只读文件系统，会导致只能读取，而无法写入新文件、新数据等操作。

造成该问题的原因包括：磁盘老旧长期大量的读写、文件系统文件被破坏、磁盘碎片文件、异常断电、读写中断等等。

以企业 CentOS 7 Linux 为案例，来修复文件系统，步骤如下：

（1） 远程备份本地其他重要数据，出现只读文件系统，需先备份其他重要数据，基于 rsync|scp 远程备份，其中/data 为源目录，/data/backup/2022/为目标备份目录。

```
rsync   -av    /data/          root@192.168.111.188:/data/backup/2022/
```

（2） 可以重新挂载/系统，挂载命令如下，测试文件系统是否可以写入文件。

```
mount   -o    remount,rw    /
```

（3） 如果重新挂载/系统无法解决问题，则需重启服务器，以 CD/DVD 光盘引导进入 Linux Rescue 修复模式，如图 7-16（a）、7-16（b）所示，光标选择“Troubleshooting”,按 Enter 键 然后选择“Rescue a CentOS system”,按 Enter 键。

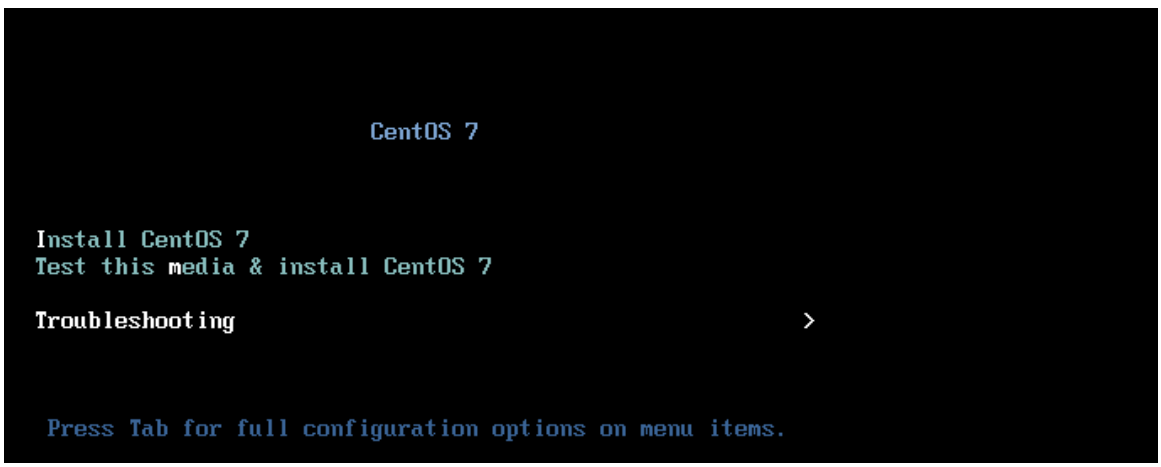


图 7-16（a） 光盘引导进入修复模式

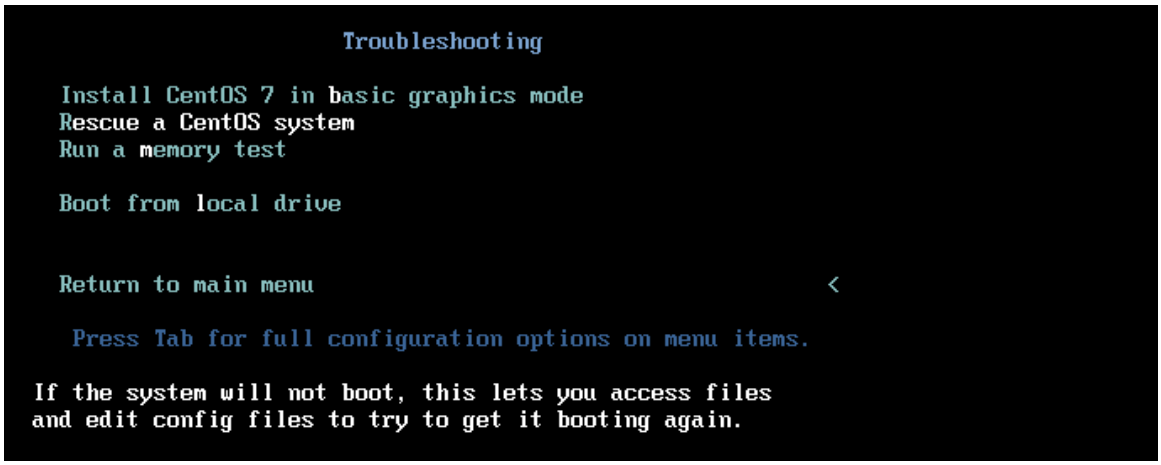


图 7-16（b） 光盘引导进入修复模式

（4） 选择“1）Continue”继续操作，如图 7-17 所示：

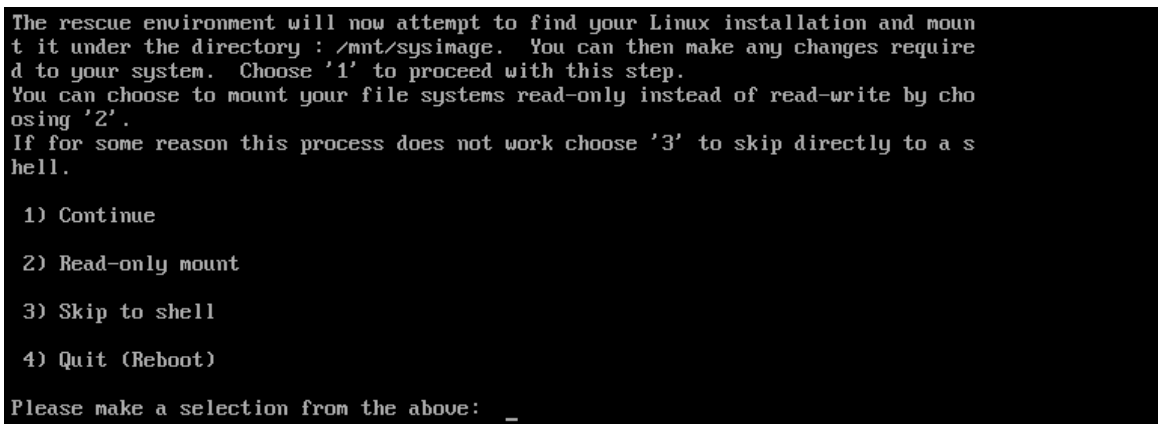


图 7-17 选择 Continue 继续进入系统

（5） 登录修复模式，执行如下命令，df –h 显示原来的文件系统，如图 7-18 所示：

```
chroot /mnt/sysimage

df -h
```

```
sh-4.2# chroot   /mnt/sysimage/



bash-4.2#
bash-4.2#
bash-4.2#
bash-4.2# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3        37G  5.7G   31G  16% /
devtmpfs        474M     0  474M   0% /dev
tmpfs           493M     0  493M   0% /dev/shm
tmpfs           493M   13M  481M   3% /run
/dev/sda1       197M  103M   94M  53% /boot
/dev/sdb1        20G   45M   19G   1% /data
bash-4.2#
```

图 7-18 切换原分区目录

（6） 对有异常的分区进行检测并修复，根据文件系统类型，执行相应的命令如下：

```
umount      /dev/sda3

fsck.ext4   /dev/sda3      –y
```

（7） 修复完成之后，重启系统即可

```
reboot
```

7.9 本章小结

通过对本章内容的学习，读者掌握了 Linux 硬盘内部结构、Block 及 Inode 特性，能够对企业硬盘进行分区、格式化等操作，满足企业的日常需求。

基于 mount 工具，能对硬盘、各类文件系统进行挂载操作，同时对只读文件系统能快速修复并投入使用。

7.10 同步作业

1. 软链接与硬链接的区别是什么？

2. 有一块 4T 的移动硬盘，如何将数据拷贝至服务器/data/目录，服务器空间为 10T，请写出详细步骤？
3. 运维部小刘发现公司 IDC 机房一台 DELL R730 服务器，/data/目录不可写，而/boot 目录可读可写，请问原因是什么导致，如何修复/data/目录？
4. 公司一台 DELL R730 服务器/data/images 目录存放了大量的小文件，运维人员向该目录写入 1M 测试文件，提示磁盘空间不足，而通过 df –h 显示剩余可用空间为 500G，请问是什么原因导致，如何解决该问题？
5. 机房一台 DELL R730 服务器，由于业务需求，临时重启，重启完，20 分钟后还无法登陆系统，检查控制台输出，一直卡在 MySQL 服务启动项，请问如何快速解决让系统正常启动，请写出解决步骤。

第8章 Linux 文件服务器企业实战

运维和管理企业 Linux 服务器，除了要熟练 Linux 系统本身的维护和管理之外，最重要的是熟练甚至精通基于 Linux 系统安装配置各种应用软件，对软件进行调优以及软件在使用中遇到各类问题，能够快速定位并解决问题。

本章向读者介绍进程、线程、企业 Vsftpd 服务器实战、匿名用户访问、系统用户访问及虚拟用户实战等。

8.1 进程与线程概念及区别

Linux 系统各种软件和服务，存在于系统，必然会占用系统各种资源，系统资源是如何分配及调度的呢，本节将给读者展示系统进程、资源及调度相关的内容。

进程（Process）是计算机中的软件程序关于某数据集合上的一次运行活动，是系统进行资源分配和调度的基本单位，是操作系统结构的基础。

在早期面向进程设计的计算机结构中，进程是程序的基本执行实体，在当代面向线程设计的计算机结构中，进程是线程的容器。软件程序是对指令、数据及其组织形式的描述，而进程是程序的实体，通常而言，把运行在系统中的软件程序称之为进程。

除了进程，读者通常会听到线程的概念，线程也被称为轻量级进程(Lightweight Process，LWP)，是程序执行流的最小单元。一个标准的线程由线程 ID，当前指令指针(PC)，寄存器集合和堆栈组成。

线程是进程中的一个实体，是被系统独立调度和分派的基本单位，线程自己不拥有操作系统资源，但是该线程可与同属进程的其它线程共享该进程所拥有的全部资源。

程序、进程、线程三者区别如下：

- 程序：程序并不能单独执行，是静止的，只有将程序加载到内存中，系统为其分配资源后才能够执行；
- 进程：程序对一个数据集的动态执行过程，一个进程包含一个或者更多的线程，一个线程同时只能被一个进程所拥有，进程是分配资源的基本单位。进程拥有独立的内存单元，而多个线程共享内存，从而提高了应用程序的运行效率。

- 线程：线程是进程内的基本调度单位，线程的划分尺度小于进程，并发性更高，线程本身不拥有系统资源，但是该线程可与同属进程的其它线程共享该进程所拥有的全部资源。每一个独立的线程，都有一个程序运行的入口、顺序执行序列、和程序的出口。

如图 8-1 所示，程序、进程、线程三者的关系拓扑图：

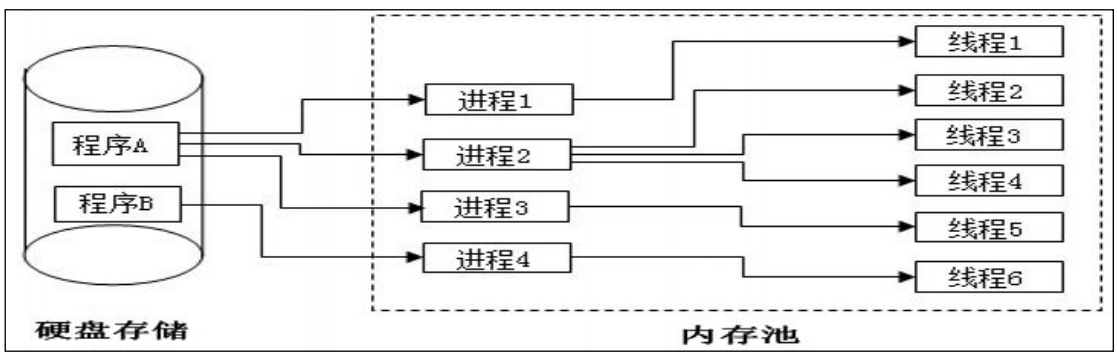


图 8-1　程序、进程、线程关系图

如上图 8-1 所示，多进程、多线程的区别如下：

- 多进程，每个进程互相独立，不影响主程序的稳定性，某个子进程崩溃对其他进程没有影响，通过增加 CPU 可以扩充软件的性能，可以减少线程加锁/解锁的影响，极大提高性能。缺点是多进程逻辑控制复杂，需要和主程序交互，需要跨进程边界，进程之间上下文切换比线程之间上下文切换代价大。
- 多线程，无需跨进程，程序逻辑和控制方式简单，所有线程共享该进程的内存和变量等。缺点是每个线程与主程序共用地址空间，线程之间的同步和加锁控制比较麻烦，一个线程的崩溃会影响到整个进程或者程序的稳定性。

8.2 Vsftpd 服务器企业实战

文件传输协议（File Transfer Protocol，FTP），基于该协议 FTP 客户端与服务端可以实现共享文件、上传文件、下载文件。 FTP 基于 TCP 协议生成一个虚拟的连接，主要用于控制 FTP 连接信息，同时再生成一个单独的 TCP 连接用于 FTP 数据传输。用户可以通过客户端向 FTP 服务器端上传、下载、删除文件，FTP 服务器端可以同时提供给多人共享使用。

FTP 服务是 Client/Server（简称 C/S）模式，基于 FTP 协议实现 FTP 文件对外共享及传输的软件称之为 FTP 服务器源端，客户端程序基于 FTP 协议，则称之为 FTP 客户端，FTP 客户端可以向 FTP 服务器上传、下载文件。

8.2.1 FTP 传输模式

FTP 基于 C/S 模式，FTP 客户端与服务器端有两种传输模式，分别是 FTP 主动模式、FTP 被动模式，主被动模式均是以 FTP 服务器端为参照。主被动模式如图 8-2（a）、8-2（b）所示，主被动模式详细区别如下：

- FTP 主动模式：客户端从一个任意的端口 N（N>1024）连接到 FTP 服务器的 port 21 命令端口，客户端开始监听端口 N+1，并发送 FTP 命令“port N+1”到 FTP 服务器，FTP 服务器以数据端口（20）连接到客户端指定的数据端口（N+1）。
- FTP 被动模式：客户端从一个任意的端口 N（N>1024）连接到 FTP 服务器的

port 21 命令端口，客户端开始监听端口 N+1，客户端提交 PASV 命令，服务器会开启一个任意的端口（P >1024），并发送 PORT P 命令给客户端。客户端发起从本地端口 N+1 到服务器的端口 P 的连接用来传送数据。

在企业实际环境中，如果 FTP 客户端与 FTP 服务端均开放防火墙，FTP 需以主动模式工作，这样只需要在 FTP 服务器端防火墙规则中，开放 20、21 端口即可。关于防火墙配置后面章节会讲解。

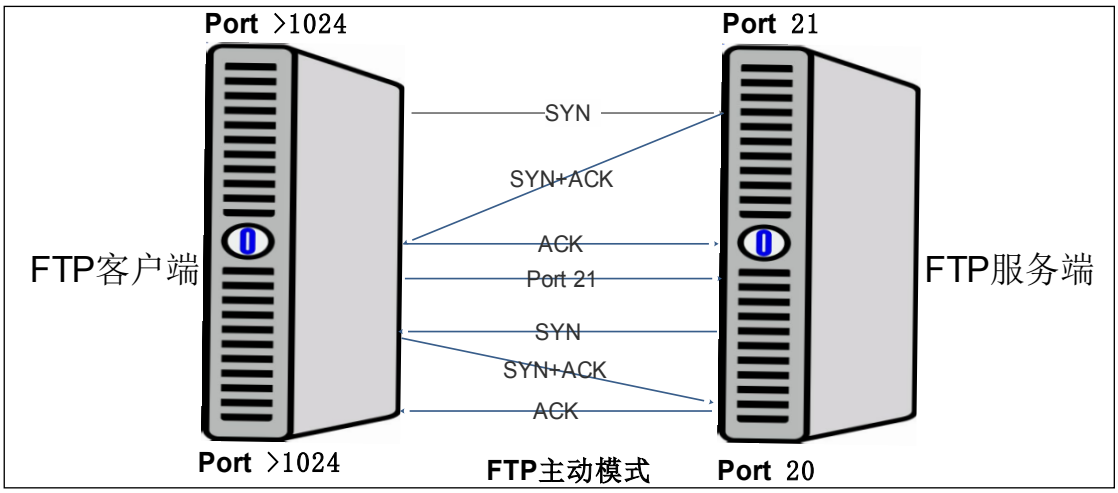


图 8-2（a） FTP 主动模式

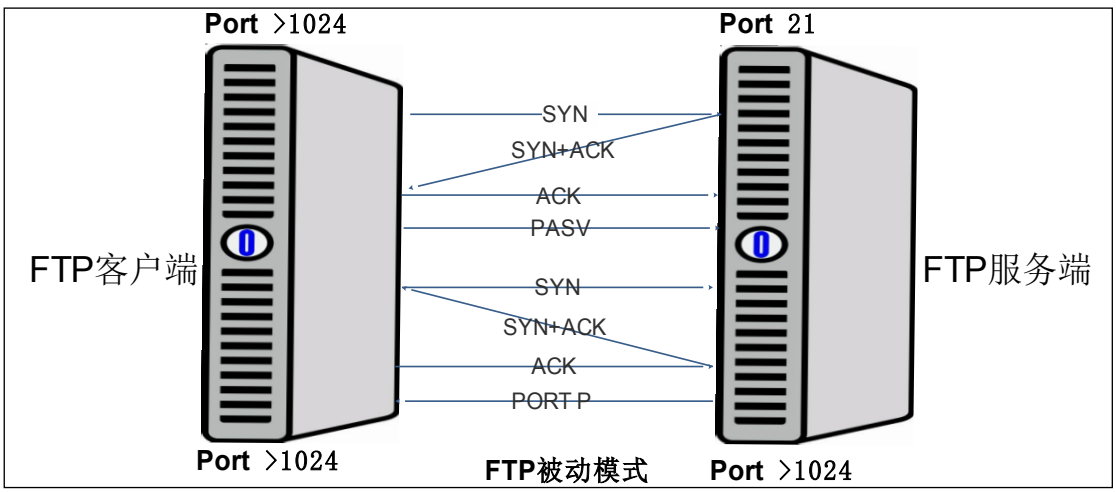


图 8-2（b） FTP 被动模式

8.2.2 Vsftpd 服务器简介

目前主流的FTP服务器端软件包括：Vsftpd、ProFTPD、PureFTPd、Wuftpd、Server-U FTP、 FileZilla Server 等软件，其中 Unix/Linux 使用较为广泛的 FTP 服务器端软件为 Vsftpd 。

非常安全的 FTP 服务进程（ Very Secure FTP daemon ，Vsftpd ），Vsftpd 在 Unix/Linux 发行版中最主流的 FTP 服务器程序，优点小巧轻快，安全易用、稳定高效、满足企业跨部门、多用户的使用（1000 用户）等。

Vsftpd 基于 GPL 开源协议发布，在中小企业中得到广泛的应用，Vsftpd 可以快速上手，基于 Vsftpd 虚拟用户方式，访问验证更加安全。Vsftpd 还可以基于 MYSQL 数据库做安全验证，多重安全防护。

8.2.3 Vsftpd 服务器安装配置

Vsftpd 服务器端安装有两种方法，一是基于 YUM 方式安装，而是基于源码编译安装，最终实现效果完全一致，本文采用 YUM 安装 Vsftpd，步骤如下：

（1） 在命令行执行如下命令，如图 8-3 所示：

```
yum    install    vsftpd*    -y
```

```
[root@www-jfedu-net ~]# yum install vsftpd* -y
已加载插件：fastestmirror, product-id, search-disabled-repos, subscrip
This system is not registered with Subscription Management. You can us
base
extras
updates
updates/7/x86_64/primary_db
Loading mirror speeds from cached hostfile
 * base: mirrors.btte.net
 * extras: centos.ustc.edu.cn
 * updates: ftp.jaist.ac.jp
正在解决依赖关系
--> 正在检查事务
---> 软件包 vsftpd.x86_64.0.3.0.2-11.el7_2 将被 升级
---> 软件包 vsftpd.x86_64.0.3.0.2-21.el7 将被 更新
---> 软件包 vsftpd-sysvinit.x86_64.0.3.0.2-21.el7 将被 安装
```

图 8-3 YUM 安装 Vsftpd 服务端

（2） 打印 vsftpd 安装后的配置文件路径、启动 Vsftpd 服务及查看进程是否启动，

如图 8-4 所示：

```
rpm -ql vsftpd|more

systemctl restart vsftpd.service

ps -ef |grep vsftpd
```

```
[root@www-jfedu-net ~]# rpm -ql vsftpd|more
/etc/logrotate.d/vsftpd
/etc/pam.d/vsftpd
/etc/vsftpd
/etc/vsftpd/ftpusers
/etc/vsftpd/user_list
/etc/vsftpd/vsftpd.conf
/etc/vsftpd/vsftpd_conf_migrate.sh
/usr/lib/systemd/system-generators/vsftpd-generator
/usr/lib/systemd/system/vsftpd.service
/usr/lib/systemd/system/vsftpd.target
/usr/lib/systemd/system/vsftpd@.service
/usr/sbin/vsftpd
/usr/share/doc/vsftpd-3.0.2
/usr/share/doc/vsftpd-3.0.2/AUDIT
/usr/share/doc/vsftpd-3.0.2/BENCHMARKS
```

图 8-4 打印 Vsftpd 软件安装后路径

（3） Vsftpd.conf 默认配置文件详解如下：

anonymous_enable=YES 开启匿名用户访问；

local_enable=YES 启用本地系统用户访问；

write_enable=YES 本地系统用户写入权限；

local_umask=022 本地用户创建文件及目录默认权限掩码；

dirmessage_enable=YES 打印目录显示信息，通常用于用户第一次访问目录时，信息提示；

xferlog_enable=YES 启用上传/下载日志记录；

connect_from_port_20=YES FTP 使用 20 端口进行数据传输；

xferlog_std_format=YES 日志文件将根据 xferlog 的标准格式写入；

listen=NO Vsftpd 不以独立的服务启动，通过 Xinetd 服务管理，建议改成 YES；

listen_ipv6=YES 启用 IPV6 监听；

pam_service_name=vsftpd 登录 FTP 服务器 ,依据/etc/pam.d/vsftpd 中内容进行认证；

userlist_enable=YES vsftpd.user_list 和 ftpusers 配置文件里用户禁止访问 FTP；

tcp_wrappers=YES 设置 vsftpd 与 tcp wrapper 结合进行主机的访问控制，Vsftpd 服务器检查/etc/hosts.allow 和/etc/hosts.deny 中的设置，来决定请求连接的主机，是否允许访问该 FTP 服务器。

（4） 启动 Vsftpd 服务后，通过 Windows 客户端资源管理器访问 Vsftp 服务器端，如图 8-5 所示：

```
ftp://192.168.111.131/
```

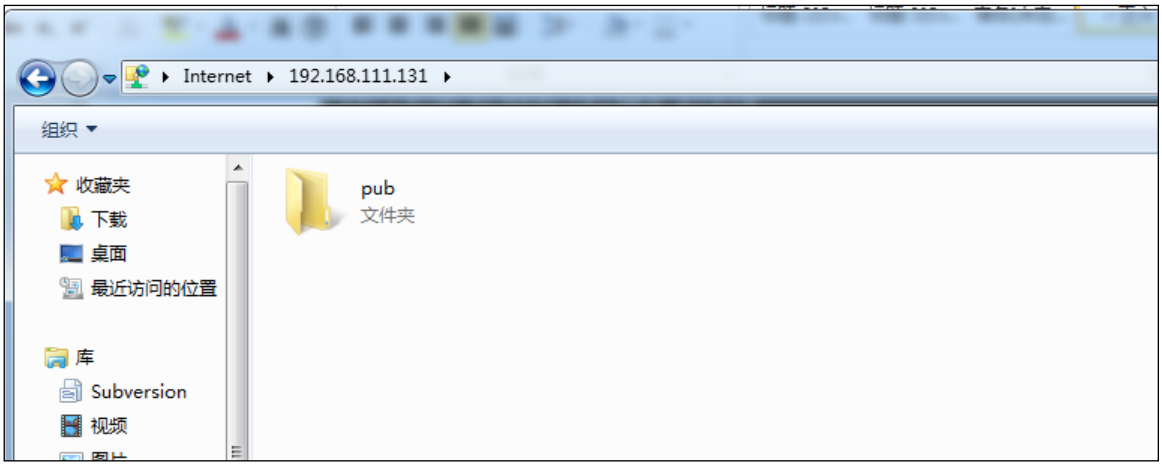


图 8-5 匿名用户访问 FTP 默认目录

FTP 主被动模式，默认为主动模式，设置为被动模式使用端口方法如下：

```
pasv_enable=YES

pasv_min_port=60000

pasv_max_port=60100
```

8.2.4 Vsftpd 匿名用户配置

Vsftpd 默认以匿名用户访问，匿名用户默认访问的 FTP 服务器端路径为：/var/ftp/pub，匿名用户只有查看权限，无法创建、删除、修改。如需关闭 FTP 匿名用户访问，需修改配置文件/etc/vsftpd/vsftpd.conf，将 anonymous_enable=YES 修改为 anonymous_enable=NO，重启 Vsftpd 服务即可。

如果允许匿名用户能够上传、下载、删除文件，需在/etc/vsftpd/vsftpd.conf 配置文件中加入如下代码：

```
anon_upload_enable=YES                  允许匿名用户上传文件；
anon_mkdir_write_enable=YES             允许匿名用户创建目录；
anon_other_write_enable=YES             允许匿名用户其他写入权限。
```

匿名用户完整 vsftpd.conf 配置文件代码如下：

```
anonymous_enable=YES
local_enable=YES
write_enable=YES
local_umask=022
anon_upload_enable=YES
anon_mkdir_write_enable=YES
anon_other_write_enable=YES
dirmessage_enable=YES
xferlog_enable=YES
connect_from_port_20=YES
xferlog_std_format=YES
listen=NO
listen_ipv6=YES
```

```
pam_service_name=vsftpd

userlist_enable=YES

tcp_wrappers=YES
```

由于默认 Vsftpd 匿名用户有两种：anonymous、ftp，所以匿名用户如果需要上传文件、删除及修改等权限，需要 ftp 用户对/var/ftp/pub 目录有写入权限，使用如下 chown 和 chmod 任意一种即可，设置命令如下：

```
chown   -R   ftp        pub/

chmod              o+w        pub/
```

如上 Vsftpd.conf 配置文件配置完毕，同时权限设置完，重启 vsftpd 服务即可，通过 Windows 客户端访问，能够上传文件、删除文件、创建目录等操作，如图 8-6 所示：

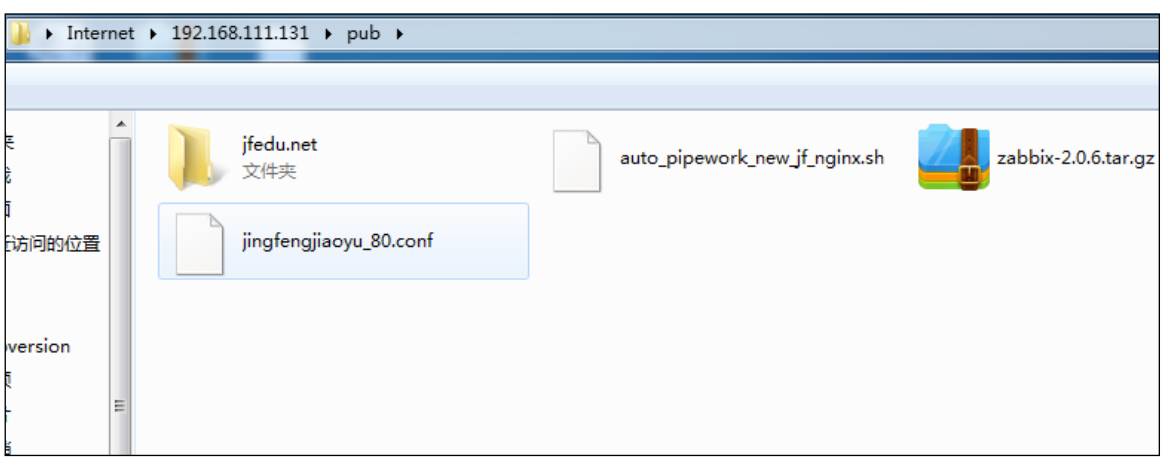


图 8-6 匿名用户访问上传文件

8.2.5 Vsftpd 系统用户配置

Vsftpd 匿名用户设置完毕，匿名用户，任何人都可以查看 FTP 服务器端的文件、目

录，甚至可以修改、删除，此方案如适合存放私密文件在 FTP 服务器端，如何保证文件或者目录专属拥有者呢，Vsftpd 系统用户可以实现该需求。

实现 Vsftpd 系统用户方式验证，只需在 Linux 系统中创建多个用户即可，创建用户使用 useradd，同时给用户设置密码，即可通过用户和密码登录 FTP，进行文件上传、下载、删除等操作。Vsftpd 系统用户实现方法步骤如下：

（1） Linux 系统中创建系统用户 jfedu1、jfedu2，分别设置密码为 123456：

```
useradd    jfedu1
useradd    jfedu2
echo 123456|passwd --stdin   jfedu1
echo 123456|passwd --stdin   jfedu2
```

（2） 修改 vsftpd.conf 配置文件代码如下：

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022
dirmessage_enable=YES
xferlog_enable=YES
connect_from_port_20=YES
xferlog_std_format=YES
```

```
listen=NO

listen_ipv6=YES

pam_service_name=vsftpd

userlist_enable=YES

tcp_wrappers=YES
```

（3） 通过 Windows 资源客户端验证，使用 jfedu1、jfedu2 用户登录 FTP 服务器，即可上传文件、删除文件、下载文件，jfedu1、jfedu2 系统用户上传文件的家目录在/home/jfedu1、/home/jfedu2 下，如图 8-7（a）、8-7（b）所示：

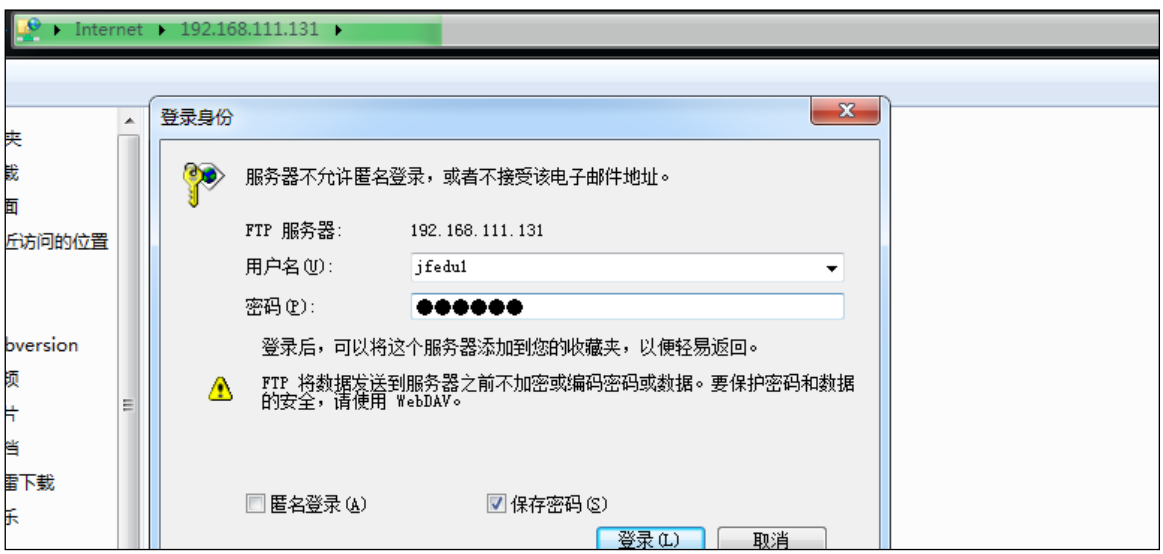


图 8-7（a） jfedu1 用户登录 FTP 服务器

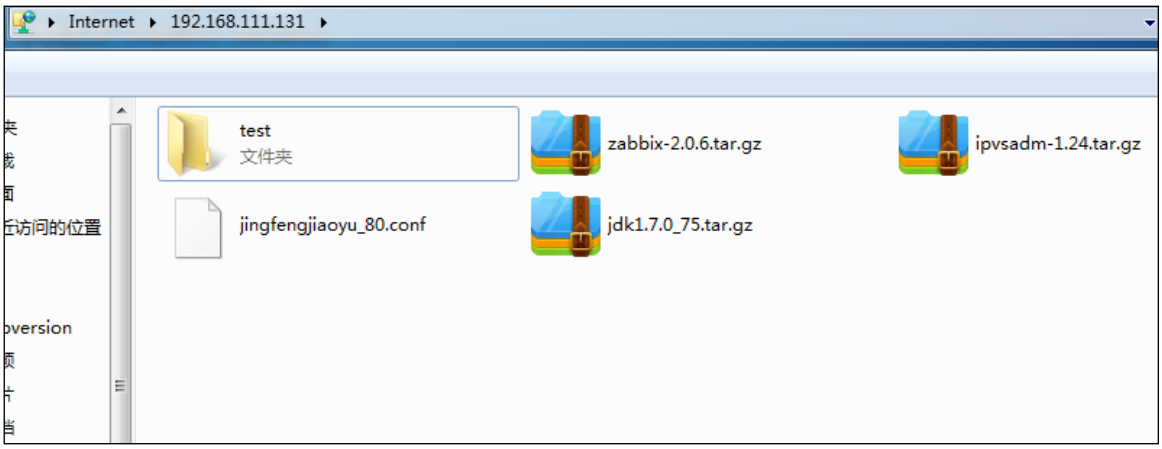


图 8-7（b） jfedu1 登录 FTP 服务器上传文件

8.2.6 Vsftpd 虚拟用户配置

如果基于 Vsftpd 系统用户访问 FTP 服务器，系统用户越多越不利于管理，而且不利于系统安全管理，鉴于此，为了能更加的安全使用 VSFTPD，需使用 Vsftpd 虚拟用户方式。

Vsftpd 虚拟用户原理：虚拟用户就是没有实际的真实系统用户，而是通过映射到其中一个真实用户以及设置相应的权限来实现访问验证，虚拟用户不能登录 Linux 系统，从而让系统更加的安全可靠。

Vsftpd 虚拟用户企业案例配置步骤如下：

（1） 安装 Vsftpd 虚拟用户需用到的软件及认证模块：

```
yum install pam* libdb-utils libdb* --skip-broken -y
```

（2） 创建虚拟用户临时文件/etc/vsftpd/ftpusers.txt，新建虚拟用户和密码，其中 jfedu001、jfedu002 为虚拟用户名，123456 为密码，如果有多个用户，依次格式填写即可：

```
jfedu001

123456

jfedu002

123456
```

（3） 生成 Vsftpd 虚拟用户数据库认证文件，设置权限 700：

```
db_load -T -t hash -f /etc/vsftpd/ftpusers.txt

/etc/vsftpd/vsftpd_login.db

chmod 700 /etc/vsftpd/vsftpd_login.db
```

（4） 配置 PAM 认证文件，/etc/pam.d/vsftpd 行首加入如下两行：

```
auth required pam_userdb.so db=/etc/vsftpd/vsftpd_login

account required pam_userdb.so db=/etc/vsftpd/vsftpd_login
```

（5） 所有 Vsftpd 虚拟用户需要映射到一个系统用户，该系统用户不需要密码，也不需要登录，主要用于虚拟用户映射使用，创建命令如下：

```
useradd -s /sbin/nologin ftpuser
```

（6） 完整 vsftpd.conf 配置文件代码如下：

```
#global config Vsftpd 2022

anonymous_enable=YES

local_enable=YES

write_enable=YES
```

```
local_umask=022

dirmessage_enable=YES

xferlog_enable=YES

connect_from_port_20=YES

xferlog_std_format=YES

listen=NO

listen_ipv6=YES

userlist_enable=YES

tcp_wrappers=YES

#config virtual user FTP

pam_service_name=vsftpd

guest_enable=YES

guest_username=ftpuser

user_config_dir=/etc/vsftpd/vsftpd_user_conf

virtual_use_local_privs=YES
```

如上 Vsftpd 虚拟用户配置文件参数详解：

```
#config virtual user FTP

pam_service_name=vsftpd                    虚拟用户启用 pam 认证；

guest_enable=YES                           启用虚拟用户；
```

```
guest_username=ftpuser                          映射虚拟用户至系统用户 ftpuser；
user_config_dir=/etc/vsftpd/vsftpd_user_conf 设置虚拟用户配置文件所在的目录；
virtual_use_local_privs=YES                     虚拟用户使用与本地用户相同的权
限。
```

（7） 至此，所有虚拟用户共同基于/home/ftpuser 主目录实现文件上传与下载，可以在/etc/vsftpd/vsftpd_user_conf 目录创建虚拟用户各自的配置文件，创建虚拟用户配置文件主目录:

```
mkdir   -p      /etc/vsftpd/vsftpd_user_conf/
```

（8） 如下分别为虚拟用户 jfedu001、jfedu002 用户创建配置文件：

vim /etc/vsftpd/vsftpd_user_conf/jfedu001，同时创建私有的虚拟目录，代码如下：

```
local_root=/home/ftpuser/jfedu001
write_enable=YES
anon_world_readable_only=YES
anon_upload_enable=YES
anon_mkdir_write_enable=YES
anon_other_write_enable=YES
```

vim /etc/vsftpd/vsftpd_user_conf/jfedu002，同时创建私有的虚拟目录，代码如下：

```
local_root=/home/ftpuser/jfedu002
write_enable=YES
anon_world_readable_only=YES
anon_upload_enable=YES
anon_mkdir_write_enable=YES
anon_other_write_enable=YES
```

虚拟用户配置文件内容详解：

```
local_root=/home/ftpuser/jfedu002        jfedu002 虚拟用户配置文件路径；
write_enable=YES                         允许登陆用户有写权限；
anon_world_readable_only=YES             允许匿名用户下载，然后读取文件；
anon_upload_enable=YES                   允许匿名用户上传文件权限，只有在
write_enable=YES 时该参数才生效；
anon_mkdir_write_enable=YES              允许匿名用户创建目录，只有在
write_enable=YES 时该参数才生效；
anon_other_write_enable=YES              允许匿名用户其他权限，例如删除、重命名
等。
```

（9） 创建虚拟用户各自虚拟目录：

```
mkdir -p /home/ftpuser/{jfedu001,jfedu002} ; chown -R ftpuser:ftpuser
/home/ftpuser
```

重启 Vsftpd 服务，通过 Windows 客户端资源管理器登录 Vsftpd 服务端，测试结果如图 8-8（a）、8-8（b）所示：

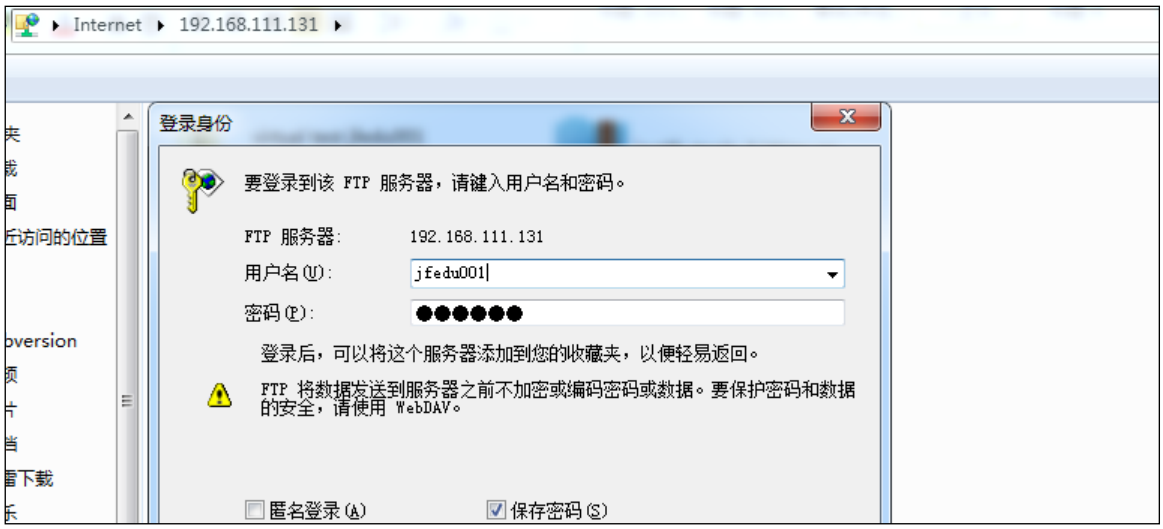


图 8-8（a） jfedu001 虚拟用户登录 FTP 服务器

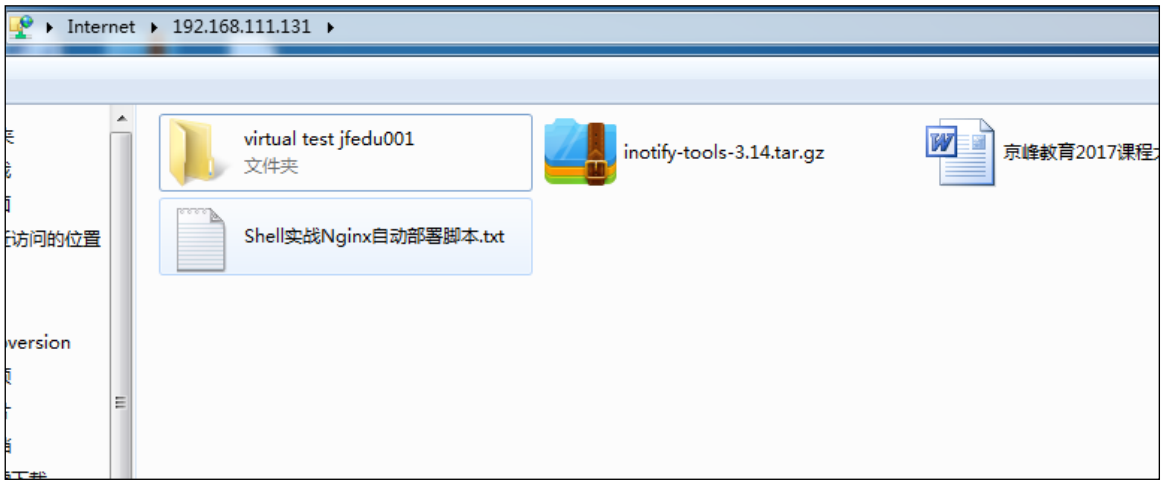


图 8-8（b） jfedu001 虚拟用户上传下载文件

第9章 Apache WEB 服务器企业实战

万维网（WORLD WIDE WEB，WWW)服务器，也称之为 WEB 服务器，主要功能是提供网上信息浏览服务。WWW 是 Internet 的多媒体信息查询工具，是 Internet 上飞快发展的服务，也是目前用的最广泛的服务。正是因为有了 WWW 软件，才使得近年来 Internet 迅速发展。

目前主流的 WEB 服务器软件包括：Apache、Nginx、Lighttpd、IIS、Resin、Tomcat、WebLogic、Jetty 等。

本章向读者介绍 Apache WEB 服务器发展历史、Apache 工作模式深入剖析、Apache 虚拟主机、配置文件详解及 Apache Rewrite 企业实战等。

9.1 Apache WEB 服务器入门简介

Apache HTTP Server 是 Apache 软件基金会的一个开源的网页服务器，是世界使用排名第一的 Web 服务器软件，可以运行在几乎所有广泛使用的计算机平台上，由于其跨平台和安全性被广泛使用，是目前最流行的 Web 服务器端软件之一。

Apache 服务器是一个多模块化的服务器，经过多次修改，成为目前世界使用排名第一的 Web 服务器软件。Apache 取自“A Patchy Server”的读音，即充满补丁的服务器，因为 Apache 基于 GPL 发布，大量开发者不断为 Apache 贡献新的代码、功能、新的特性、修改原来的缺陷。

Apache 服务器的特点是使用简单、速度快、性能稳定，可以做负载均衡及代理服务器来使用。

9.2 Prefork MPM 工作原理

每辆汽车都有发动机引擎，不同的引擎，对车子运行效率也不一样，同样 Apache 也有类似工作引擎或者处理请求的模块，亦可称之为多路处理模块（Multi-Processing Modules，MPM），Apache WEB 服务器有三种处理模块：Prefork MPM、Worker MPM、Event MPM。

在企业中最常用的处理模块为 Prefork MPM 和 Worker MPM，Event MPM 不支持 HTTPS 方式，官网也给出

“This MPM is experimental, so it may or may not work as expected”提示，所以很少被使用。

默认 Apache 处理模块为 Prefork MPM 方式，Prefork 采用的预派生子进程方式，Prefork 用单独的子进程来处理不同的请求，进程之间是彼此独立的，所以比较稳定。

Prefork 的工作原理：控制进程 Master 在最初建立“StartServers”个进程后，为了满足 MinSpareServers 设置的最小空闲进程 ,所以需创建第一个空闲进程 ,等待一秒钟，继续创建两个，再等待一秒钟，继续创建四个，依次按照递增指数级创建进程数，最多每秒同时创建 32 个空闲进程，直到满足至少有 MinSpareServers 设置的值为止。

Apache 的预派生模式（Prefork），基于预派生模式，不必在请求到来时再产生新的进程，从而减小了系统开销以增加性能，不过由于 Prefork MPM 引擎是基于多进程方式提供对外服务，每个进程占内存也相对较高。

9.3 Worker MPM 工作原理

相对于 Prefork MPM,Worker 方式是 2.0 版中全新的支持多线程和多进程混合模型的 MPM，由于使用线程来处理，所以可以处理海量的 HTTP 请求，而系统资源的开销要小于基于 Prefork 多进程的方式。Worker 也是基于多进程，但每个进程又生成多个线程，这样可以保证多线程可以获得进程的稳定性。

Worker MPM 工作原理：控制进程 Master 在最初建立“StartServers”个进程，每个进程会创建 ThreadsPerChild 设置的线程数，多个线程共享该进程内存空间，同时每个线程独立地处理用户的 HTTP 请求。为了不在请求到来时再生成线程，Worker MPM 也可以设置最大最小空闲线程。

Worker MPM 模式下同时处理的请求总数=进程总数 xThreadsPerChild，也即等于 MaxClients。如果服务器负载很高，当前进程数不满足需求，Master 控制进程会 fork 新的进程，最大进程数不能超过 ServerLimit 数，如果需调整的 StartServers 进程数，需同时调整 ServerLimit 值。

Prefork MPM 与 Worker MPM 引擎区别小结如下：

- Prefork MPM 模式：使用多个进程，每个进程只有一个线程，每个进程在某个确定的时间只能维持一个连接，稳定，内存开销较高；
- Worker MPM 模式：使用多个进程，每个子进程包含多个线程，每个线程在某个确定的时间只能维持一个连接，内存占用量比较小，适合大并发、高流量

的 WEB 服务器。Worker MPM 缺点是一个线程崩溃，整个进程就会连同其任何线程一起挂掉。

9.4 Apache WEB 服务器安装

从 Apache 官方分站点下载目前稳定版本 httpd-2.2.32 版本，目前最新版本为 2.4 版本，下载地址如下：

```
http://mirrors.hust.edu.cn/apache/httpd/httpd-2.2.32.tar.bz2
```

Apache WEB 服务器安装步骤如下：

```
tar    -xjvf    httpd-2.2.32.tar.bz2

tar 工具解压 httpd 包；

cd httpd-2.2.32/

进入解压后目录；

yum install apr apr-devel apr-util apr-util-devel -y

安装 APR 相关优化模块；

./configure --prefix=/usr/local/apache2/ --enable-rewrite --enable-so --

enable-mpms-shared=all --with-mpm=event
```

预编译 Apache，启用 rewrite 规则、启用动态加载库、开启 Apache 三种工作引擎,

如果开启模块支持，需要添加配置：

#LoadModule mpm_prefork_module modules/mod_mpm_prefork.so

#LoadModule mpm_event_module modules/mod_mpm_event.so

#LoadModule mpm_worker_module modules/mod_mpm_worker.so

make

编译

make install

安装

Apache2.2.32 安装完毕，如图 10-1 所示：

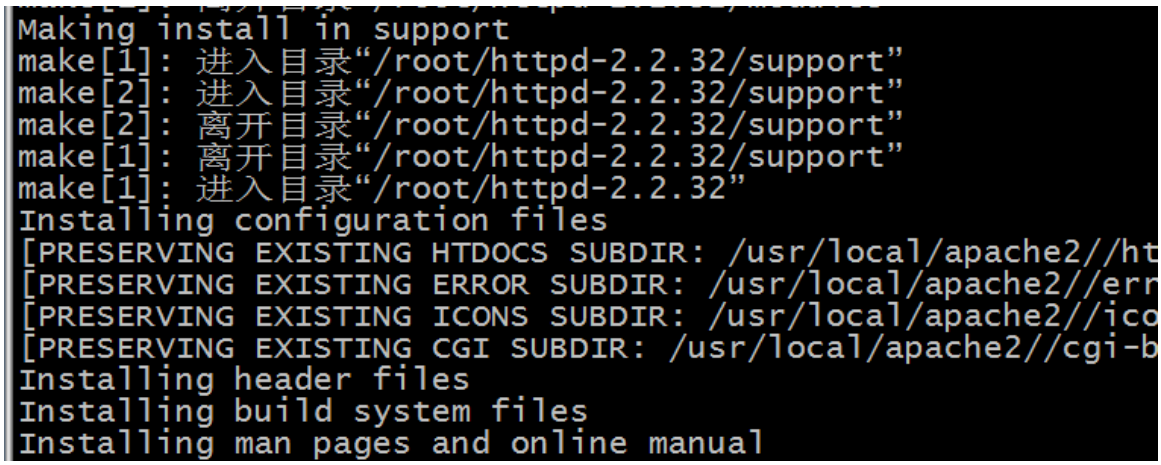


图 10-1 Apache2.2.32 安装图解

启动 Apache 服务，临时关闭 selinux、firewalld 防火墙：

```
/usr/local/apache2/bin/apachectl start

setenforce 0

systemctl    stop     firewalld.service
```

查看 Apache 服务进程，通过客户端浏览器访问 http://192.168.111.131/，如图 10-2（a）、10-2（b）所示：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# /usr/local/apache2/bin/apachectl start
httpd: Could not reliably determine the server's fully qualified domai
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ps -ef |grep httpd
root     14399     1  0 05:05 ?        00:00:00 /usr/local/apache2//bi
daemon   14400 14399  0 05:05 ?        00:00:00 /usr/local/apache2//bi
daemon   14401 14399  0 05:05 ?        00:00:00 /usr/local/apache2//bi
daemon   14402 14399  0 05:05 ?        00:00:00 /usr/local/apache2//bi
daemon   14403 14399  0 05:05 ?        00:00:00 /usr/local/apache2//bi
daemon   14404 14399  0 05:05 ?        00:00:00 /usr/local/apache2//bi
root     14406  2116  0 05:05 pts/0    00:00:00 grep --color=auto http
[root@www-jfedu-net ~]#
```

图 10-2（a） Apache 启动及查看进程

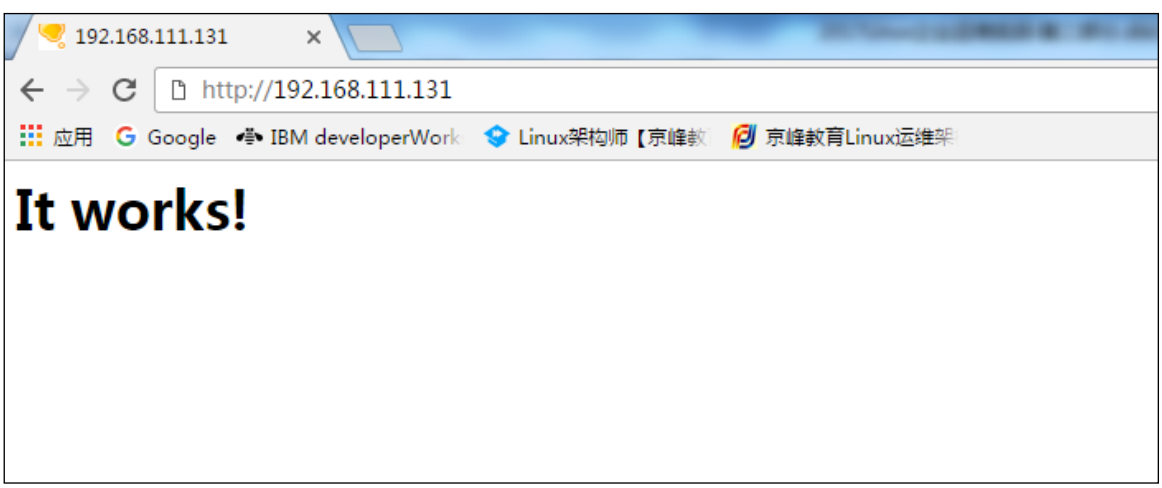


图 10-2（b） 浏览器访问 Apache WEB 服务器

9.5 Apache 虚拟主机企业应用

企业真实环境中，一台 WEB 服务器发布单个网站会非常浪费资源，所以一台 WEB 服务器上会发布多个网站，少则 3-5 个，多则 2-30 个网站。

在一台服务器上发布多网站，也称之为部署多个虚拟主机，WEB 虚拟主机配置方法有三种：

- 基于单 IP 多个 Socket 端口；
- 基于多 IP 地址一个端口；
- 基于单 IP 一个端口不同域名。

其中基于同一端口不同域名的方式在企业中得到广泛的使用和应用，如下为基于一个端口不同域名，在一台 Apache WEB 服务器上部署多个网站，步骤如下：

（1） 创建虚拟主机配置文件 httpd-vhosts.conf，该文件默认已存在，只需去掉 httpd.conf 配置文件中#号即可，如图 10-3 所示：

```
# User home directories
#Include conf/extra/httpd-userdir.conf

# Real-time info on requests and configuration
#Include conf/extra/httpd-info.conf

# Virtual hosts
Include conf/extra/httpd-vhosts.conf

# Local access to the Apache HTTP Server Manual
#Include conf/extra/httpd-manual.conf

# Distributed authoring and versioning (WebDAV)
#Include conf/extra/httpd-dav.conf
```

图 10-3 httpd.conf 配置文件开启虚拟主机

（2） 配置文件/usr/local/apache2/conf/extra/httpd-vhosts.conf 中代码设置为如下：

```
NameVirtualHost *:80

<VirtualHost *:80>

    ServerAdmin support@jfedu.net

    DocumentRoot "/usr/local/apache2/htdocs/jf1"

    ServerName www.jf1.com

    ErrorLog "logs/www.jf1.com_error_log"

    CustomLog "logs/www.jf1.com_access_log" common

</VirtualHost>

<VirtualHost *:80>

    ServerAdmin support@jfedu.net

    DocumentRoot "/usr/local/apache2/htdocs/jf2"

    ServerName www.jf2.com

    ErrorLog "logs/www.jf2.com_error_log"

    CustomLog "logs/www.jf2.com_access_log" common

</VirtualHost>
```

Httpd-vhosts.conf 参数详解：

```
NameVirtualHost *:80                                    开启虚拟主机，并且监
听本地所有网卡接口的 80 端口；
<VirtualHost *:80>                                      虚拟主机配置起始；
    ServerAdmin support@jfedu.net                       管理员邮箱；
    DocumentRoot "/usr/local/apache2/htdocs/jf1"        该虚拟主机发布目录；
    ServerName www.jf1.com                              虚拟主机完整域名；
    ErrorLog "logs/www.jf1.com_error_log"               错误日志路径及文件
名；
    CustomLog "logs/www.jf1.com_access_log" common      访问日志路径及文件
名；
</VirtualHost>                                          虚拟主机配置结束。
```

（3） 创建 www.jf1.com 及 www.jf2.com 发布目录，重启 apache 服务，并分别创建 index.html 页面：

```
mkdir  -p  /usr/local/apache2/htdocs/{jf1,jf2}/
/usr/local/apache2/bin/apachectl restart
echo                    "<h1>              www.jf1.com
Pages</h1>" >/usr/local/apache2/htdocs/jf1/index.html
echo                    "<h1>              www.jf2.com
Pages</h1>" >/usr/local/apache2/htdocs/jf2/index.html
```

（4） Windows 客户端设置 Hosts 映射，将 www.jf1.com、www.jf2.com 与 192.168.111.131 IP 进行映射绑定，映射的目的将域名跟 IP 进行绑定，在浏览器可以输入域名，不需要输入 IP 地址，绑定方法是在“C:\Windows\System32\drivers\etc”文件夹中，使用记事本编辑 hosts 文件，加入如下代码，如图 10-4 所示：

```
192.168.111.131    www.jf1.com

192.168.111.131    www.jf2.com
```

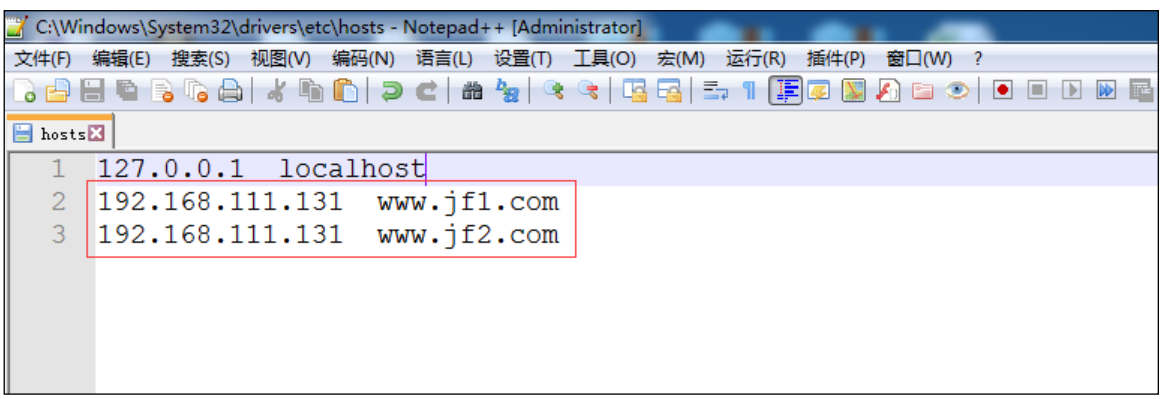


图 10-4 Windows 主机 Hosts 配置

（5） 浏览器访问 www.jf1.com、www.jf2.com 如图 10-5（a）、10-5（b）所示，至此 Apache 基于多域名虚拟主机配置完毕，如果还需添加虚拟主机，直接拷贝其中一个虚拟主机配置、修改 WEB 发布目录即可：



图 10-5（a） www.jf1.com 网站返回内容

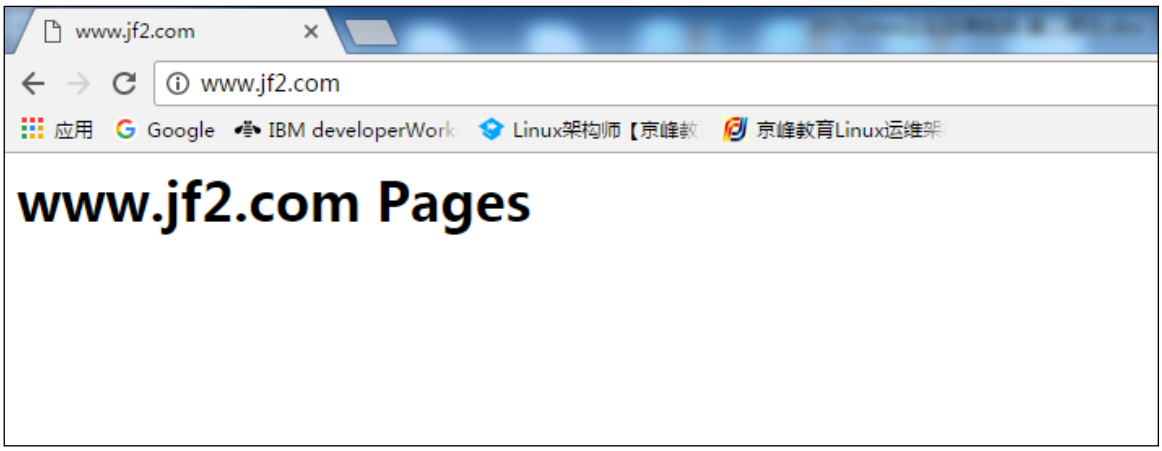


图 10-5（b） www.jf2.com 网站返回内容

9.6 Apache 常用目录学习

Apache 可以基于源码安装、YUM 安装，不同的安装方法，所属的路径特不同，如下为 Apache 常用路径的功能用途：

/usr/lib64/httpd/modules/	Apache 模块存放路径；
/var/www/html/	YUM 安装 Apache 网站发布目录；
/var/www/error/	服务器设置错误信息，浏览器显示；

var/www/icons/	Apache 小图标文件存放目录；
var/www/cgi-bin/	可执行的 CGI 程序存放目录。
/var/log/httpd/	Apache 日志目录；
/usr/sbin/apachectl	Apache 启动脚本；
/usr/sbin/httpd	Apache 二进制执行文件；
/usr/bin/htpasswd	设置 Apache 目录密码访问；
/usr/local/apache2/bin	Apache 命令目录；
/usr/local/apache2/build	Apache 构建编译目录；
/usr/local/apache2/htdocs/	源码安装 Apache 网站发布目录；
/usr/local/apache2/cgi-bin	可执行的 CGI 程序存放目录；
/usr/local/apache2/include	Apache 引用配置文件目录；
/usr/local/apache2/logs	Apache 日志目录；
/usr/local/apache2/man	Apache 帮助文档目录；
/usr/local/apache2/manual	Apache 手册；
/usr/local/apache2/modules	Apache 模块路径。

9.7 Apache 配置文件详解

Apache 的配置文件是 Apache WEB 难点，读者需要掌握配置文件中每个参数的含义，才能理解并在日常运维中去解决 Apache 遇到的故障，如下为 Apache 配置文件详

解：

ServerTokens OS	显示服务器的版本和操作系统内核版本；
ServerRoot "/usr/local/apache2/"	Apache 主配置目录；
PidFile run/httpd.pid	PidFile 进程文件；
Timeout 60	不论接收或发送，当持续连接等待超过 60 秒则该次连接就中断；
KeepAlive Off	关闭持续性的连接；
MaxKeepAliveRequests 100	当 KeepAlive 设置为 On 的时候，该数值可以决定此次连接能够传输的最大传输数量；
KeepAliveTimeout 65	当 KeepAlive 设置为 On 的时候，该连接在最后一次传输后等待延迟的秒数；
<IfModule prefork.c>	Prefork MPM 引擎配置段；
StartServers 8	默认启动 Apache 工作进程数；
MinSpareServers 5	最小空闲进程数；
MaxSpareServers 20	最大空闲进程数；
ServerLimit 4096	Apache 服务器最多进程数；
MaxClients 4096	每秒支持的最大客户端并发；
MaxRequestsPerChild 4000	每个进程能处理的最大请求数；
</IfModule>	

<IfModule worker.c>　　Worker MPM 引擎配置段；

StartServers　　8　　默认启动 Apache 工作进程数；

MaxClients　　4000　　每秒支持的最大客户端并发；

MinSpareThreads　　25　　最小空闲线程数；

MaxSpareThreads　　75　　最小空闲线程数；

ThreadsPerChild　　75　　每个进程启动的线程数；

MaxRequestsPerChild　0　　每个进程能处理的最大请求数，0 表示无限制；

</IfModule>

LoadModule　mod_version.so　　静态加载 apache 相关模块；

ServerAdmin support@jfedu.net　　管理员邮箱，网站异常，错误信息会发生至该邮箱；

DocumentRoot　"/usr/local/apache2/htdocs/"　Apache 网站默认发布目录；

<Directory "/data/webapps/www1">　　设置/data/webapps/www1 目录权限；

　　AllowOverride All

　　Options -Indexes FollowSymLinks

　　Order allow,deny

　　Allow from all

</Directory>

AllowOverride　　设置为 None 时，目录中.htaccess 文件将被

完全忽略，当指令设置为 All 时，.htaccess 文件生效； Options -Indexes FollowSymLinks 禁止浏览目录，去掉”-“,表示浏览目录，常用于下载站点； Order allow,deny 默认情况下禁止所有客户机访问； Order deny,allow 默认情况下允许所有客户机访问； Allow from all 允许所有客户机访问。

9.8 Apache Rewrite 规则实战

Rewirte 规则也称为规则重写，主要功能是实现浏览器访问 HTTP URL 的跳转，其正则表达式是基于 Perl 语言。通常而言，几乎所有的 WEB 服务器均可以支持 URL 重写。Rewrite URL 规则重写的用途：

- 对搜索引擎优化（Search Engine Optimization，SEO）友好，利于搜索引擎抓取网站页面；
- 隐藏网站 URL 真实地址，浏览器显示更加美观；
- 网站变更升级，可以基于 Rewrite 临时重定向到其他页面。

Apache WEB 服务器如需要使用 Rewrite 功能，须添加 Rewrite 模块，基于源码安装是指定参数“--enable-rewrite”，还有一种方法可以动态添加模块，以 DSO 模式安装 Apache,利用模块源码和 Apache apxs 工具完成 Rewrite 模块的添加。

使用 Apache Rewrite，除了安装 Rewrite 模块之外，还需在 httpd.conf 中的全局

配置段或者虚拟主机配置段设置如下指令来开启 Rewrite 功能：

```
RewriteEngine on
```

Apache Rewrite 规则使用中有三个概念需要理解，分别是：Rewrite 结尾标识符、Rewrite 规则常用表达式、Apache Rewrite 变量，如下为三个概念的详解：

（1） Apache Rewrite 结尾标识符，用于 Rewrite 规则末尾，表示规则的执行属性。

R[=code](force redirect)	强制外部重定向；
G(force URL to be gone)	强制 URL 为 GONE，返回 410HTTP 状态码；
P(force proxy)	强制使用代理转发；
L(last rule)	匹配当前规则为最后一条匹配规则，停止匹配后续规则；
N(next round)	重新从第一条规则开始匹配；
C(chained with next rule)	与下一条规则关联；
T=MIME-type(force MIME type)	强制 MIME 类型；
NC(no case)	不区分大小写。

（2） Apache Rewrite 规则常用表达式，主要用于匹配参数、字符串及过滤设置。

.	匹配任何单字符；
[word]	匹配字符串:word；
[^word]	不匹配字符串:word；
jfedu|jfteach	可选择的字符串:jfedu|jfteach；

?	匹配 0 到 1 个字符；
*	匹配 0 到多个字符；
+	匹配 1 到多个字符；
^	字符串开始标志；
$	字符串结束标志；
\n	转义符标志。

（3） Apache Rewrite 变量，常用于匹配 HTTP 请求头信息、浏览器主机名、URL 等。

HTTP headers:HTTP_USER_AGENT, HTTP_REFERER, HTTP_COOKIE, HTTP_HOST, HTTP_ACCEPT；

connection & request: REMOTE_ADDR, QUERY_STRING；

server internals: DOCUMENT_ROOT, SERVER_PORT, SERVER_PROTOCOL；

system stuff: TIME_YEAR, TIME_MON, TIME_DAY。

详解如下:

HTTP_USER_AGENT	用户使用的代理，例如浏览器；
HTTP_REFERER	告知服务器，从哪个页面来访问的；
HTTP_COOKIE	客户端缓存，主要用于存储用户名和密码等信息；
HTTP_HOST	匹配服务器 ServerName 域名；
HTTP_ACCEPT	客户端的浏览器支持的 MIME 类型；

REMOTE_ADDR	客户端的 IP 地址
QUERY_STRING	URL 中访问的字符串；
DOCUMENT_ROOT	服务器发布目录；
SERVER_PORT	服务器端口；
SERVER_PROTOCOL	服务器端协议；
TIME_YEAR	年；
TIME_MON	月；
TIME_DAY	日；

（4） Rewrite 规则实战案例，以下配置均配置在 httpd.conf 或者 vhosts.conf 中，企业中常用的 Rewrite 案例如下：

1. 将 jfedu.net 跳转至 www.jfedu.net。

```
RewriteEngine on                                    启用 rewrite 引擎；
RewriteCond %{HTTP_HOST} ^jfedu.net [NC] 匹配以 jfedu.net 开头的域名,NC 忽略大小写；
RewriteRule ^/(.*)$ http://www.jfedu.net/$1 [L] (.*)表示任意字符串 ,$1 表示引用(.*)的中任意内容。
```

2. 将 www.jf1.com www.jf2.com jfedu.net 跳转至 www.jfedu.net，OR 含义表示或者。

```
RewriteEngine on
```

```
RewriteCond   %{HTTP_HOST}    www.jf1.com          [NC,OR]
RewriteCond   %{HTTP_HOST}    www.jf2.com          [NC,OR]
RewriteCond   %{HTTP_HOST}    ^jfedu.net           [NC]
RewriteRule ^/(.*)$   http://www.jfedu.net/$1      [L]
```

3. 访问 www.jfedu.net 首页，跳转至 www.jfedu.net/newindex/，R=301 表示永久重定向。

```
RewriteEngine on
RewriteRule ^/$   http://www.jfedu.net/newindex/   [L,R=301]
```

4. 访问/newindex/plus/view.php?aid=71 跳转至 http://www.jfedu.net/linux/。

```
RewriteEngine on
RewriteCond     %{QUERY_STRING}     ^tid=(.+)$       [NC]
RewriteRule     ^/forum\.php$          /jfedu/thread-new-%1.html? [R=301,L]
```

5. 访问 www.jfedu.net 首页 ,内容访问 www.jfedu.net/newindex/ ,但是浏览器 URL 地址不改变。

```
RewriteEngine on
RewriteCond   %{HTTP_HOST}    ^www.jfedu.net          [NC]
RewriteRule   ^/$                  /newindex/                [L]
```

6. 访问/forum.php?tid=107258 跳转至/jfedu/thread-new-107258.html。

```
RewriteEngine on
```

```
RewriteCond     %{QUERY_STRING}     ^tid=(.+)$       [NC]

RewriteRule     ^/forum\.php$          /jfedu/thread-new-%1.html? [R=301,L]
```

7. 访问/xxx/123456 跳转至/xxx?id=123456。

```
RewriteEngine on

rewriteRule    ^/(.+)/(\d+)$    /$1?id=$2    [L,R=301]
```

8. 判断是否使用移动端访问网站，移动端访问跳转至 m.jfedu.net。

```
RewriteEngine on

RewriteCond %{HTTP_USER_AGENT} ^iPhone                    [NC,OR]

RewriteCond %{HTTP_USER_AGENT} ^Android                        [NC,OR]

RewriteCond %{HTTP_USER_AGENT} ^WAP                       [NC]

RewriteRule ^/$                 http://m.jfedu.net/index.html        [L,R=301]

RewriteRule ^/(.*)/$            http://m.jfedu.net/$1                [L,R=301]
```

9. 访问/10690/jfedu/123 跳转至/index.php?tid/10690/items=123，[0-9]表示任意一个数字，+表示多个，(.+)表示任何多个字符。

```
RewriteEngine on

RewriteRule ^/([0-9]+)/jfedu/(.+)$ /index.php?tid/$1/items=$2 [L,R=301]
```

第10章 MYSQL 服务器企业实战

MySQL 是一个关系型数据库管理系统，由瑞典 MySQL AB 公司开发，目前属于 Oracle 旗下公司。MySQL 最流行的关系型数据库管理系统，在 WEB 应用方面 MySQL 是最好的 RDBMS (Relational Database Management System，关系数据库管理系统) 应用软件之一。

本章向读者介绍关系型数据库特点、MySQL 数据库引擎特点、数据库安装配置、SQL 案例操作、数据库索引、慢查询、MySQL 数据库集群实战等。

10.1 MYSQL 数据库入门简介

MySQL 是一种关联数据库管理系统，关联数据库将数据保存在不同的表中，而不是将所有数据放在一个大仓库内，这样就增加了速度并提高了灵活性。MySQL 所使用的 SQL 语言是用于访问数据库的最常用标准化语言。

MySQL 数据库主要用于存储各类信息数据，例如：员工姓名、身份证 ID、商城订单及金额、销售业绩及报告，学生考试成绩、网站帖子、论坛用户信息、系统报表等等。

MySQL 软件采用了双授权政策，它分为社区版和商业版，由于其体积小、速度快、总体拥有成本低，尤其是开放源码这一特点，一般中小型网站的开发都选择 MySQL 作为网站数据库。由于其社区版的性能卓越，搭配 PHP 和 Apache 可组成良好的开发环境。

关系数据库管理系统(Relational Database Management System，RDBMS)，是将数据组织为相关的行和列的系统，而管理关系数据库的计算机软件就是关系数据库管理系统，常用的关系型数据库软件有 MYSQL、Mariadb、Oracle、SQL Server、

PostgreSQL、DB2 等。)

RDBMS 数据库的特点如下：

- 数据以表格的形式出现；
- 每行记录数据的真实内容；
- 每列记录数据真实内容的数据域；
- 无数的行和列组成一张表；
- 若干的表组成一个数据库；

目前主流架构 LAMP(Linux+Apache+MySQL+PHP)，MySQL 更是得到各位 IT 运维、DBA 的青睐，虽然 MySQL 数据库已被 Orcacle 公司收购，不过好消息是原来 MySQL 创始人已独立出来自己重新开发了 MariaDB 数据库，开源免费，目前越来越多的人开始尝试使用。MariaDB 数据库兼容 MySQL 数据库所有的功能和相关参数。

MySQL 数据库运行在服务器前，需要选择启动的引擎，好比一辆轿车，性能好的发动机会提升轿车的性能，从而启动、运行更加的高效。同样 MYSQL 也有类似发动机引擎，这里称之为 MYSQL 引擎。

MYSQL 引擎包括：ISAM、MyISAM、InnoDB、MEMORY、CSV、BLACKHOLE、ARCHIVE、PERFORMANCE_SCHEMA、Berkeley、Merge、Federated、Cluster/NDB 等，其中 MyISAM、InnoDB 使用最为广泛，如下为 Myisam BDB Memory InnoDB Archive 引擎功能的对比：

引擎特性	Myisam	BDB	Memory	InnoDB	Archive

批量插入的速度	高	高	高	中	非常高
集群索引	不支持	不支持	不支持	支持	不支持
数据缓存	不支持	不支持	支持	支持	不支持
索引缓存	支持	不支持	支持	支持	不支持
数据可压缩	支持	不支持	不支持	不支持	支持
硬盘空间使用	低	低	NULL	高	非常低
内存使用	低	低	中等	高	低
外键支持	不支持	不支持	不支持	支持	不支持
存储限制	没有	没有	有	64TB	没有
事务安全	不支持	支持	不支持	支持	不支持
锁机制	表锁	页锁	表锁	行锁	行锁
B 树索引	支持	支持	支持	支持	不支持
哈希索引	不支持	不支持	支持	支持	不支持
全文索引	支持	不支持	不支持	不支持	不支持

性能总结：MyISAM MySQL 5.0 之前的默认数据库引擎，最为常用。拥有较高的插入，查询速度，但不支持事务；

InnoDB 事务型数据库的首选引擎，支持 ACID 事务，ACID 包括：原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）、持久性（Durability），一个支持事务（Transaction）的数据库，必需要具有这四种特性，否则在执行事务过程

无法保证数据的正确性。

MYSQL5.5 之后默认引擎为 InnoDB，Innodb 支持行级锁定，支持事物、外键等功能。

BDB 源自 Berkeley DB，事务型数据库的另一种选择，支持 Commit 和 Rollback 等其他事务特性；

Memory 所有数据置于内存的存储引擎，拥有极高的插入，更新和查询效率。但是会占用和数据量成正比的内存空间。并且其内容会在 MySQL 重新启动时丢失；

Mysql 常用的两大引擎有 MyISAM 和 innoDB，那他们有什么明显的区别呢，什么场合使用什么引擎呢？

MyISAM 类型的数据库表强调的是性能，其执行数度比 InnoDB 类型更快，但不提供事务支持，不支持外键，如果执行大量的 SELECT(查询)操作，MyISAM 是更好的选择，支持表锁。

InnoDB 提供事务支持事务、外部键、行级锁等高级数据库功能，执行大量的 INSERT 或 UPDATE，出于性能方面的考虑，可以考虑使用 InnoDB 引擎。

10.2 Mysql 数据库安装方式

MySQL 数据库安装方法有两种，一种是 yum/rpm 通过 YUM 源在线安装，另外一种是通过源码软件编译安装。

（1） YUM 方式安装 MYSQL 方法，执行命令:

```
yum install mysql-server mysql-devel mysql-libs -y CentOS6.x YUM 安装
yum install mariadb-server mariadb mariadb-libs -y CentOS7.x YUM 安装
```

（2） 源码安装 MYSQL5.5.20 方法，通过 cmake、make、make install 三个步骤实现。

```
wget http://down1.chinaunix.net/distfiles/mysql-5.5.20.tar.gz
yum -y install gcc-c++ ncurses-devel cmake make perl gcc autoconf automake zlib libxml2 libxml2-devel libgcrypt libtool bison
tar -xzf mysql-5.5.20.tar.gz
cd mysql-5.5.20
cmake . -DCMAKE_INSTALL_PREFIX=/usr/local/mysql56/ \
-DMYSQL_UNIX_ADDR=/tmp/mysql.sock \
-DMYSQL_DATADIR=/data/mysql/ \
-DSYSCONFDIR=/etc \
-DMYSQL_USER=mysql \
-DMYSQL_TCP_PORT=3306 \
-DWITH_XTRADB_STORAGE_ENGINE=1 \
-DWITH_INNOBASE_STORAGE_ENGINE=1 \
```

```
-DWITH_PARTITION_STORAGE_ENGINE=1 \

-DWITH_BLACKHOLE_STORAGE_ENGINE=1 \

-DWITH_MYISAM_STORAGE_ENGINE=1 \

-DWITH_READLINE=1 \

-DENABLED_LOCAL_INFILE=1 \

-DWITH_EXTRA_CHARSETS=1 \

-DDEFAULT_CHARSET=utf8 \

-DDEFAULT_COLLATION=utf8_general_ci \

-DEXTRA_CHARSETS=all \

-DWITH_BIG_TABLES=1 \

-DWITH_DEBUG=0

make

make install
```

（3） 源码安装 MYSQL5.7.20 方法，通过 cmake、make、make install 三个步骤实现。

```
wget

http://nchc.dl.sourceforge.net/project/boost/boost/1.59.0/boost_1_59_0.tar.gz

tar zxvf    boost_1_59_0.tar.gz

mv boost_1_59_0 /usr/local/boost
```

```
yum -y install gcc-c++ ncurses-devel cmake make perl gcc autoconf automake

zlib libxml2 libxml2-devel libgcrypt libtool bison

cmake  .  -DCMAKE_INSTALL_PREFIX=/usr/local/mysql5/ \

-DMYSQL_UNIX_ADDR=/tmp/mysql.sock \

-DMYSQL_DATADIR=/data/mysql/ \

-DSYSCONFDIR=/etc \

-DMYSQL_USER=mysql \

-DMYSQL_TCP_PORT=3306 \

-DWITH_XTRADB_STORAGE_ENGINE=1 \

-DWITH_INNOBASE_STORAGE_ENGINE=1 \

-DWITH_PARTITION_STORAGE_ENGINE=1 \

-DWITH_BLACKHOLE_STORAGE_ENGINE=1 \

-DWITH_MYISAM_STORAGE_ENGINE=1 \

-DWITH_READLINE=1 \

-DENABLED_LOCAL_INFILE=1 \

-DWITH_EXTRA_CHARSETS=1 \

-DDEFAULT_CHARSET=utf8 \

-DDEFAULT_COLLATION=utf8_general_ci \

-DEXTRA_CHARSETS=all \
```

```
-DWITH_BIG_TABLES=1 \

-DWITH_DEBUG=0 \

-DDOWNLOAD_BOOST=1 \

-DWITH_BOOST=/usr/local/boost

make

make install

/usr/local/mysql5/bin/mysqld --initialize --user=mysql --

basedir=/usr/local/mysql5 --datadir=/data/mysql
```

（4） MYSQL 源码安装参数详解

cmake . -DCMAKE_INSTALL_PREFIX=/usr/local/mysql55	Cmake 预编译；-
DMYSQL_UNIX_ADDR=/tmp/mysql.sock	MYSQL Socket 通信文件
位置；	
-DMYSQL_DATADIR=/data/mysql	MYSQL数据存放路径;
-DSYSCONFDIR=/etc	配置文件路径；
-DMYSQL_USER=mysql	MYSQL 运行用户；
-DMYSQL_TCP_PORT=3306	MYSQL 监听端口；
-DWITH_XTRADB_STORAGE_ENGINE=1	开启 xtradb 引擎支持；
-DWITH_INNOBASE_STORAGE_ENGINE=1	开启 innodb 引擎支持；

-DWITH_PARTITION_STORAGE_ENGINE=1	开启 partition 引擎支持；
-DWITH_BLACKHOLE_STORAGE_ENGINE=1	开启 blackhole 引擎支持；
-DWITH_MYISAM_STORAGE_ENGINE=1	开启 MyISAM 引擎支持；
-DWITH_READLINE=1	启用快捷键功能；
-DENABLED_LOCAL_INFILE=1	允许从本地导入数据；
-DWITH_EXTRA_CHARSETS=1	支持额外的字符集；
-DDEFAULT_CHARSET=utf8	默认字符集 UTF-8；
-DDEFAULT_COLLATION=utf8_general_ci	检验字符；
-DEXTRA_CHARSETS=all	安装所有扩展字符集；
-DWITH_BIG_TABLES=1	将临时表存储在磁盘上；
-DWITH_DEBUG=0	禁止调试模式支持；
make	编译；
make install	安装。

（5） 将源码安装的 Mysql 数据库服务设置为系统服务，可以使用 chkconfig 管理，并启动 MYSQL 数据库，如图 11-1 所示：

```
cd /usr/local/mysql55/

\cp support-files/my-large.cnf /etc/my.cnf

\cp support-files/mysql.server /etc/init.d/mysqld

chkconfig --add mysqld
```

```
chkconfig --level 35 mysqld on

mkdir -p   /data/mysql

useradd   mysql

/usr/local/mysql55/scripts/mysql_install_db --user=mysql                    --

datadir=/data/mysql/ --basedir=/usr/local/mysql55/

ln   -s   /usr/local/mysql55/bin/* /usr/bin/

service   mysqld   restart
```

```
Please report any problems with the /usr/local/mysql55//scripts/mysqlb

[root@localhost mysql55]# service  mysqld  restart
 ERROR! MySQL server PID file could not be found!
Starting MySQL... SUCCESS!
[root@localhost mysql55]# clear
[root@localhost mysql55]#
[root@localhost mysql55]# ps -ef |grep mysql
root      2286     1  0 11:47 pts/0    00:00:00 /bin/sh /usr/local/mys
le=/data/mysql/localhost.pid
mysql     2561  2286  4 11:47 pts/0    00:00:00 /usr/local/mysql55/bin
a/mysql --plugin-dir=/usr/local/mysql55/lib/plugin --user=mysql --log-
sql/localhost.pid --socket=/tmp/mysql.sock --port=3306
root      2585  1234  0 11:48 pts/0    00:00:00 grep mysql
[root@localhost mysql55]#
```

图 11-1 查看 MYSQL 启动进程

（6） 不设置为系统服务，也可以用源码启动方式。

```
cd /usr/local/mysql55

mkdir –p   /data/mysql

useradd   mysql

/usr/local/mysql55/scripts/mysql_install_db --user=mysql                    --

datadir=/data/mysql/ --basedir=/usr/local/mysql55/
```

```
ln -s /usr/local/mysql55/bin/* /usr/bin/

/usr/local/mysql55/bin/mysqld_safe --user=mysql &
```

10.3 Mysql 数据库必备命令操作

MySQL 数据库安装完毕之后，对 MYSQL 数据库中各种指令的操作变得尤为重要，熟练掌握 MYSQL 必备命令是 SA、DBA 必备工作之一，如下为 MYSQL 数据库中操作必备命令，所有操作指令均在 MYSQL 命令行中操作，不能在 Linux Shell 解释器上直接运行：

直接在 Shell 终端执行命令：mysql 或者/usr/local/mysql55/bin/mysql，按 Enter 键，进入 MYSQL 命令行界面如图 11-2 所示：

```
[root@www-jfedu-net ~]# mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1
Server version: 5.5.20-log Source distribution

Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reser

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input stat

mysql>
```

图 11-2 MYSQL 命令行界面

MYSQL 命令行常用命令如下，操作结果如图 11-3（a）、11-3（b）、11-3（c）所示：

show databases；	查看所有的数据库；

create database jfedu；　　创建名为 jfedu 数据库；

use jfedu；　　进入 jfedu 数据库；

show tables；　　查看数据库里有多少张表；

create table t1 (id varchar(20),name varchar(20))；创建名为 t1 表，并创建两个字段，id、name，varchar 表示设置数据长度，用字符来定义长度单位，其中 1 汉字=2 字符=2Bytes；

insert into t1 values ("1","jfedu")；　　向表中插入数据；

select * from t1；　　查看 t1 表数据内容；

Select * from t1 where id=1 and age =’ jfedu’；　　id、age 多个条件查询；

desc t1；　　查看 t1 表字段内容；

alter table t1 modify column name varchar(20)；　　修改 name 字段的长度；

update t1 set name='jfedu.net' where id=1；　　修改 name 字段的内容；

flush　privileges；　　刷新权限；

delete from t1　；　　清空表内容；

drop　table t1　；　　删除表；

drop database jfedu；　　删除 jfedu 数据库；

show variables like '%char%'；　　查看数据库字符集；

show engines；　　查看 MySQL 存储引擎；

show variables like '%storage_engine%'；　　查看 MySQL 默认的存储引擎；

alter table t1 engine=innodb；	修改 MySQL t1 表存储引擎；

```
[root@www-jfedu-net ~]# mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 5
Server version: 5.5.20-log Source distribution

Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| test               |
+--------------------+
4 rows in set (0.00 sec)

mysql> create database jfedu;
Query OK, 1 row affected (0.00 sec)
```

图 11-3（a） MYSQL 命令操作

```
mysql> use jfedu;
Database changed
mysql> show tables;
Empty set (0.00 sec)

mysql> create table t1 (id varchar(20),name varchar(20));
Query OK, 0 rows affected (0.01 sec)

mysql> insert into t1 values ("1","jfedu");
Query OK, 1 row affected (0.00 sec)

mysql> select * from t1;
+------+-------+
| id   | name  |
+------+-------+
| 1    | jfedu |
+------+-------+
1 row in set (0.00 sec)
```

图 11-3（b） MYSQL 命令操作

```
mysql> Select * from t1 where id=1 and name='jfedu';
+------+-------+
| id   | name  |
+------+-------+
| 1    | jfedu |
+------+-------+
1 row in set (0.00 sec)

mysql> desc t1;
+-------+-------------+------+-----+---------+-------+
| Field | Type        | Null | Key | Default | Extra |
+-------+-------------+------+-----+---------+-------+
| id    | varchar(20) | YES  |     | NULL    |       |
| name  | varchar(20) | YES  |     | NULL    |       |
+-------+-------------+------+-----+---------+-------+
2 rows in set (0.00 sec)

mysql> alter table t1 modify column name varchar(20);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> update t1 set name='jfedu.net' where id=1;
Query OK, 1 row affected (0.01 sec)
```

图 11-3（c） MYSQL 命令操作

10.4 Mysql 数据库字符集设置

计算机中储存的信息都是用二进制数方式来表示的，读者每天看到屏幕显示的英文、汉字等字符是二进制数转换之后的结果。通俗的说，将汉字按照某种字符集编码存储在计算机中，称为"编码"。将存储在计算机中的二进制数解析显示出来，称为"解码"，在解码过程中，如果使用了错误的解码规则，会导致显示乱码。

MYSQL 数据库在存储数据时，默认编码 latin1，存储中文字符时，在显示或者 WEB 调用时会显示为乱码，为解决该乱码问题，需修改 Mysql 默认字符集为 UTF-8，有两种方法：

（1） 编辑 vim /etc/my.cnf 配置文件，在相应段中加入相应的参数字符集修改完毕，重启 MySQL 服务即可。

[client]字段里加入： default-character-set=utf8

```
[mysqld]字段里加入： character-set-server=utf8

[mysql]字段里加入 ： default-character-set=utf8
```

（2） MYSQL 命令行中运行如下指令，如图 11-4 所示：

```
show variables like '%char%' ;

SET character_set_client = utf8 ;

SET character_set_results = utf8 ;

SET character_set_connection = utf8 ;
```

```
mysql> SET character_set_client = utf8;
Query OK, 0 rows affected (0.00 sec)

mysql> SET character_set_results = utf8;
Query OK, 0 rows affected (0.00 sec)

mysql> SET character_set_connection = utf8;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like '%char%';
+--------------------------+------------------------------------+
| Variable_name            | Value                              |
+--------------------------+------------------------------------+
| character_set_client     | utf8                               |
| character_set_connection | utf8                               |
| character_set_database   | utf8                               |
| character_set_filesystem | binary                             |
| character_set_results    | utf8                               |
| character_set_server     | utf8                               |
| character_set_system     | utf8                               |
| character_sets_dir       | /usr/local/mysql55/share/charsets/ |
+--------------------------+------------------------------------+
8 rows in set (0.01 sec)
```

图 11-4 设置 MYSQL 数据库字符集

10.5 Mysql 数据库密码管理

MYSQL 数据库在使用过程中为了加强安全防范 ,需要设置密码访问 ,如何设置密码、及密码忘记如何破解呢，如下为设置密码授权、密码修改及密码破解的方法:

（1） MYSQL 创建用户及授权

```
grant all on jfedu.* to test@localhost identified by 'pas';
grant select,insert,update,delete on *.* to test@"%" identified by 'pas';
grant all on jfedu.* to test@'192.168.111.118' identified by 'pas';
```

如上三条 grant 语句授权参数详解如下：

授权 localhost 主机通过 test 用户和 pas 密码访问本地的 jfedu 库的所有权限；

授权所有主机通过 test 用户和 pas 密码访问本地的 jfedu 库的查询、插入、更新、删除权限；

授权 192.168.111.118 主机通过 test 用户和 pas 密码访问本地的 jfedu 库的所有权限;

（2） MYSQL 密码破解方法

在使用 MYSQL 数据库中，偶尔会出现密码忘记，或者被其他人员修改掉数据库权限的，如果需要紧急修改密码，如何破解 MYSQL 密码呢，首先停止 MYSQL 数据库服务，以跳过权限方式启动，命令如下：

```
/etc/init.d/mysqld stop
/usr/bin/mysqld_safe --user=mysql --skip-grant-tables &
```

MYSQL 跳过权限方式启动后，在 Shell 终端执行 mysql 命令并按 Enter 键，进入 mysql 命令行，如图 11-5 所示：

```
[root@localhost ~]# /usr/bin/mysqld_safe --user=mysql --skip-grant-tables &
[2] 2103
[root@localhost ~]# 141126 08:40:58 mysqld_safe Logging to '/var/log/mysqld.log'.
141126 08:40:58 mysqld_safe Starting mysqld daemon with databases from /var/lib/mysql

[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1
Server version: 5.1.73 Source distribution

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

图 11-5 跳过权限启动并登陆 MYSQL

由于 MYSQL 用户及密码认证信息存放在 mysql 库中的 user 表，需进入 mysql 库，更新相应的密码字段即可 ,例如将 MYSQL 中 root 用户的密码均改为 123456 ,如图 11-6 所示：

```
use mysql

update user set password=password( ‘123456’ ) where user=’ root’ ;
```

```
mysql> use mysql
Database changed
mysql> update user set password=password('123456') where user='root';
Query OK, 4 rows affected (0.00 sec)
Rows matched: 4  Changed: 4  Warnings: 0

mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

图 11-6 MYSQL 密码破解方法

MYSQL root 密码修改完，需停止以 Mysql 跳过权限表的启动进程，再以正常方式启动 MYSQL，再次以新的密码登陆即可进入 Mysql 数据库，如图 11-7 所示：

```
[root@localhost ~]# /etc/init.d/mysqld start
Starting MySQL.. SUCCESS!
[root@localhost ~]#
[root@localhost ~]# mysql -uroot -p123456
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1
Server version: 5.5.20 Source distribution

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

图 11-7 MYSQL 正常方式启动

10.6 MySQL 数据库索引案例

MySQL 索引可以用来快速地寻找某些具有特定值的记录，所有 MySQL 索引都以 B-树的形式保存。例如 MYSQL 没有索引，执行 select 时 MySQL 必须从第一个记录开始扫描整个表的所有记录，直至找到符合要求的记录。如果表中数据有上亿条数据，查询一条数据花费的时间会非常长，索引的目的就类似电子书的目录及页码的对应关系。

如果在需搜索条件的列上创建了索引，MySQL 无需扫描全表记录即可快速得到相应的记录行。如果该表有 1000000 条记录，通过索引查找记录至少要比全表顺序扫描快至少 100 倍，这就是索引在企业环境中带来的执行速度的提升。

MYSQL 数据库常见索引类型包括：普通索引（normal）、唯一索引（unique）、全文索引（full text）、主键索引（primary key）、组合索引等，如下为每个索引的应用场景及区别：

普通索引：　　normal，使用最广泛；

唯一索引： unique，不允许重复的索引，允许有空值；

全文索引: full text，只能用于 MyISAM 表，FULLTEXT 主要用于大量的内容检索；

主键索引： primary key 又称为特殊的唯一索引，不允许有空值；

组合索引： 为提高 mysql 效率可建立组合索引；

MYSQL 数据库表创建各个索引命令，以 t1 表为案例，操作如下：

主键索引 ALTER TABLE t1 ADD PRIMARY KEY (`column`) ;

唯一索引 ALTER TABLE t1 ADD UNIQUE (`column`) ;

普通索引 ALTER TABLE t1 ADD INDEX index_name (`column') ;

全文索引 ALTER TABLE t1 ADD FULLTEXT (`column`) ;

组合索引 ALTER TABLE t1 ADD INDEX index_name (`column1`, `column2`, `column3`) ;

如图 11-8 所示，为 t1 表的 id 字段创建主键索引，查看索引是否被创建，然后插入相同的 id，提示报错：

```
mysql> ALTER TABLE t1 ADD PRIMARY KEY ( `id` )
    -> ;
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql>
mysql> show index from t1;
+-------+------------+----------+--------------+-------------+-----------+---
-+--------+------+------------+---------+---------------+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | C
 | Packed | Null | Index_type | Comment | Index_comment |
+-------+------------+----------+--------------+-------------+-----------+---
-+--------+------+------------+---------+---------------+
| t1    |          0 | PRIMARY  |            1 | id          | A         |
 | NULL   |      | BTREE      |         |               |
+-------+------------+----------+--------------+-------------+-----------+---
-+--------+------+------------+---------+---------------+
1 row in set (0.00 sec)

mysql>
mysql> insert into t1 values ("3","jfedu");
ERROR 1062 (23000): Duplicate entry '3' for key 'PRIMARY'
```

图 11-8 MYSQL 主键索引案例演示

MYSQL 数据库表删除各个索引命令，以 t1 表为案例，操作如下：

```
DROP  INDEX  index_name  ON  t1 ;

ALTER TABLE t1 DROP INDEX  index_name ;

ALTER TABLE t1 DROP PRIMARY KEY ;
```

MYSQL 数据库查看表索引：

```
show index from t1 ;

show keys from t1 ;
```

MYSQL 数据库索引的缺点：

- MYSQL 数据库索引虽然能够提高数据库查询速度，但同时会降低更新、删除、插入表的速度，例如如对表进行 INSERT、UPDATE、DELETE 时，update 表 MySQL 不仅要保存数据，还需保存更新索引；
- 建立索引会占用磁盘空间，大表上创建了多种组合索引，索引文件的会占用大量的空间。

10.7 MySQL 数据库集群实战

随着访问量的不断增加，单台 MySQL 数据库服务器压力不断增加，需要对 MYSQL 进行优化和架构改造，MYQSL 优化如果不能明显改善压力情况，可以使用高可用、主从复制、读写分离来、拆分库、拆分表来进行优化。

MYSQL 主从复制集群在中小企业、大型企业中被广泛使用，MYSQL 主从复制的目的是实现数据库冗余备份，将 Master 数据库数据定时同步至 Slave 库中，一旦 Master 数据库宕机，可以将 WEB 应用数据库配置快速切换至 Slave 数据库，确保 WEB 应用较高的可用率，如图 11-12 所示，为 MYSQL 主从复制结构图：

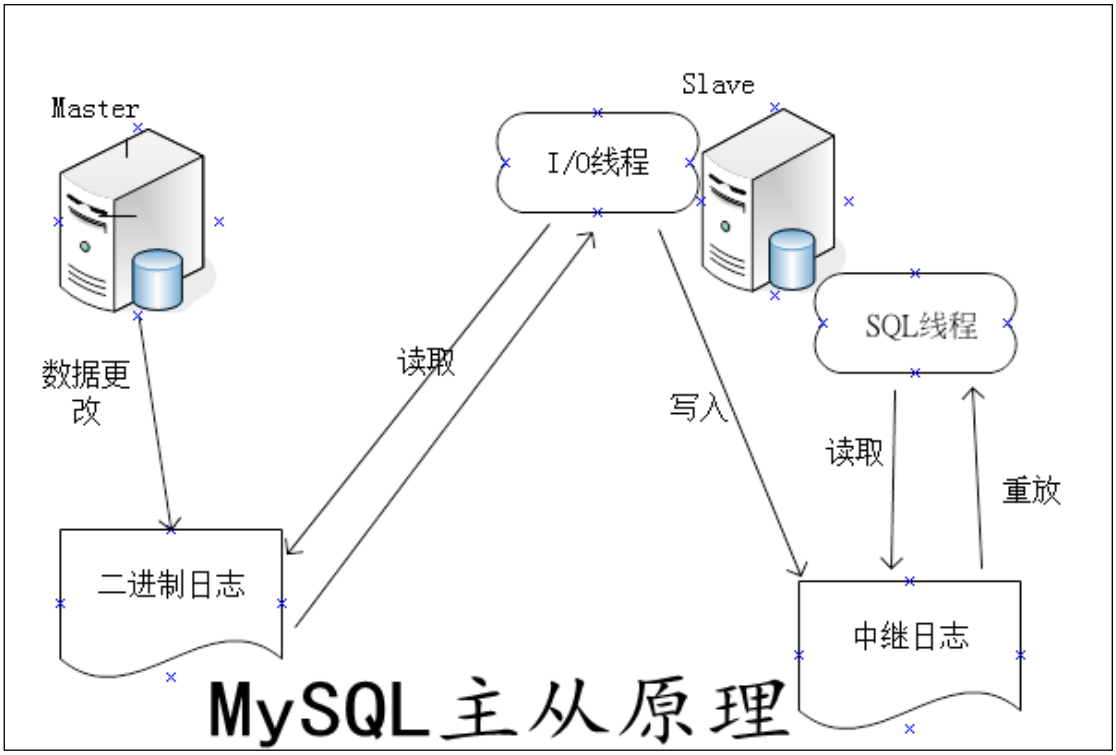


图 11-12 MYSQL 主从原理架构图

Mysql 主从复制集群至少需要 2 台数据库服务器，其中一台为 Master 库，另外一台为 Slave 库，MYSQL 主从数据同步是一个异步复制的过程，要实现复制首先需要在 master 上开启 bin-log 日志功能，bin-log 日志用于记录在 Master 库中执行的增、删、修改、更新操作的 sql 语句，整个过程需要开启 3 个线程，分别是 Master 开启 IO 线程，Slave 开启 IO 线程和 SQL 线程，具体主从同步原理详解如下：

- Slave 上执行 slave start ,Slave IO 线程会通过在 Master 创建的授权用户连接上至 Master，并请求 master 从指定的文件和位置之后发送 bin-log 日志内容；
- Master 接收到来自 slave IO 线程的请求后 ,master IO 线程根据 slave 发送的指定 bin-log 日志 position 点之后的内容，然后返回给 slave 的 IO 线程。

- 返回的信息中除了 bin-log 日志内容外，还有 master 最新的 binlog 文件名以及在 binlog 中的下一个指定更新 position 点；
- Slave IO 线程接收到信息后 ,将接收到的日志内容依次添加到 Slave 端的 relay-log 文件的最末端，并将读取到的 Master 端的 bin-log 的文件名和 position 点记录到 master.info 文件中 ,以便在下一次读取的时候能告知 master 从响应的 bin-log 文件名及最后一个 position 点开始发起请求；
- Slave Sql 线程检测到 relay-log 中内容有更新，会立刻解析 relay-log 的内容成在 Master 真实执行时候的那些可执行的 SQL 语句 ,将解析的 SQL 语句并在 Slave 里执行，执行成功后，Master 库与 Slave 库保持数据一致。

10.8 MySQL 主从复制实战

MYSQL 主从复制环境构建至少需 2 台服务器，可以配置 1 主多从，多主多从，如下为 1 主 1 从，MYSQL 主从复制架构实战步骤：

（1） 系统环境准备

Master：192.168.111.128

Slave： 192.168.111.129

（2） Master 安装及配置

Master端使用源码安装MySQL-5-5版本软件后 ,在/etc/my.cnf 配置文件[mysqld]段中加入如下代码，然后重启 MYSQL 服务即可。如果在安装时 cp my-large.cnf /etc/my.cnf，则无需添加如下代码：

```
server-id = 1

log-bin = mysql-bin
```

Master 端/etc/my.cnf 完整配置代码如下：

```
[client]

port            = 3306

socket   = /tmp/mysql.sock

[mysqld]

port            = 3306

socket   = /tmp/mysql.sock

skip-external-locking

key_buffer_size = 256M

max_allowed_packet = 1M

table_open_cache = 256

sort_buffer_size = 1M

read_buffer_size = 1M

read_rnd_buffer_size = 4M
```

```
myisam_sort_buffer_size = 64M

thread_cache_size = 8

query_cache_size= 16M

thread_concurrency = 8

log-bin=mysql-bin

binlog_format=mixed

server-id      = 1

[mysqldump]

quick

max_allowed_packet = 16M

[mysql]

no-auto-rehash

[myisamchk]

key_buffer_size = 128M

sort_buffer_size = 128M

read_buffer = 2M

write_buffer = 2M

[mysqlhotcopy]

interactive-timeout
```

Master 数据库服务器命令行中 创建 tongbu 用户及密码并设置权限，执行如下命令，查看 bin-log 文件及 position 点，如图 11-13 所示：

```
grant replication slave on *.* to 'tongbu'@'%' identified by '123456';
show master status;
```

```
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> grant replication slave on *.* to 'tongbu'@'%' identified by '123456';
Query OK, 0 rows affected (0.01 sec)

mysql> show master status;
+------------------+----------+--------------+------------------+
| File             | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+------------------+----------+--------------+------------------+
| mysql-bin.000028 |      257 |              |                  |
+------------------+----------+--------------+------------------+
1 row in set (0.00 sec)
```

图 11-13 MYSQL Master 授权用户

（3） Slave 安装及配置

Slave 端使用源码安装 MySQL-5-5 版本软件后 ,在/etc/my.cnf 配置文件[mysqld]段中加入如下代码，然后重启 MYSQL 服务即可。如果在安装时 cp my-large.cnf /etc/my.cnf，则需修改 server-id，MASTER 与 Slave 端 server-id 不能一样，Slave 端也无需开启 bin-log 功能：

```
server-id = 2
```

Slave 端/etc/my.cnf 完整配置代码如下：

```
[client]
```

```
port            = 3306
socket   = /tmp/mysql.sock
[mysqld]
port            = 3306
socket   = /tmp/mysql.sock
skip-external-locking
key_buffer_size = 256M
max_allowed_packet = 1M
table_open_cache = 256
sort_buffer_size = 1M
read_buffer_size = 1M
read_rnd_buffer_size = 4M
myisam_sort_buffer_size = 64M
thread_cache_size = 8
query_cache_size= 16M
thread_concurrency = 8
server-id      = 2
[mysqldump]
quick
```

```
max_allowed_packet = 16M

[mysql]

no-auto-rehash

[myisamchk]

key_buffer_size = 128M

sort_buffer_size = 128M

read_buffer = 2M

write_buffer = 2M

[mysqlhotcopy]

interactive-timeout
```

Slave 指定 Master IP、用户名、密码、bin-log 文件名（mysql-bin.000028）及 position（257），代码如下：

```
change master to

master_host='192.168.1.115',master_user='tongbu',master_password='123456'

,master_log_file='mysql-bin.000001',master_log_pos=297；
```

在 slave 启动 slave start，并执行 show slave status\G 查看 Mysql 主从状态：

```
slave    start；

show    slave    status\G
```

查看 Slave 端 IO 线程、SQL 线程状态均为 YES，代表 Slave 已正常连接 Master 实

现同步：

```
Slave_IO_Running: Yes

Slave_SQL_Running: Yes
```

执行 Show slave status\G，常见参数含义解析：

Slave_IO_State	I/O 线程连接 Master 状态；
Master_User	用于连接 Master 的用户；
Master_Port	Master 端监听端口；
Connect_Retry	主从连接失败，重试时间间隔；
Master_Log_File	I/O 线程读取的 Master 二进制日志文件的名称。
Read_Master_Log_Pos	I/O 线程已读取的 Master 二进制日志文件的位置；
Relay_Log_File	SQL 线程读取和执行的中继日志文件的名称。
Relay_Log_Pos	SQL 线程已读取和执行的中继日志文件的位置；
Relay_Master_Log_File	SQL 线程执行的 Master 二进制日志文件的名称；
Slave_IO_Running	I/O 线程是否被启动并成功地连接到主服务器上；
Slave_SQL_Running	SQL 线程是否被启动；
Replicate_Do_DB	指定的同步的数据库列表；
Skip_Counter	SQL_SLAVE_SKIP_COUNTER 设置的值；

Seconds_Behind_Master	Slave端SQL线程和I/O线程之间的时间差距，单位为秒，常被用于主从延迟检查方法之一。

在 Master 端创建 mysql_db_test 数据库和 t0 表，如图 11-14 所示，命令如下：

```
create database mysql_ab_test charset=utf8；

show databases；

use mysql_ab_test；

create table t0 (id varchar(20),name varchar(20))；

show tables；
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database mysql_ab_test charset=utf8;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| mysql_ab_test      |
| test               |
+--------------------+
4 rows in set (0.00 sec)

mysql> use mysql_ab_test;
Database changed
mysql>
mysql> create table t0 (id varchar(20),name varchar(30));
Query OK, 0 rows affected (0.00 sec)

mysql> show tables;
+------------------------+
| Tables_in_mysql_ab_test |
+------------------------+
| t0                     |
+------------------------+
1 row in set (0.00 sec)
```

图 11-14 MYSQL master 创建数据库和表

Slave 服务器查看是否有 mysql_ab_test 数据库和 t0 的表，如果存在则代表 Slave

从 Master 复制数据成功，证明 MYSQL 主从配置至此已经配置成功，如图 11-15 所示：

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| mysql_ab_test      |
| test               |
+--------------------+
4 rows in set (0.00 sec)

mysql> use mysql_ab_test;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-------------------------+
| Tables_in_mysql_ab_test |
+-------------------------+
| t0                      |
+-------------------------+
1 row in set (0.00 sec)
```

图 11-15 MYSQL Slave 自动同步数据

在 Master 服务器的 t0 表插入两条数据，在 slave 查看是否已同步，master 上执行如下图 11-16 所示：

```
insert into t0 values（“001”,” wugk1”）;

insert into t0 values（“002”,” wugk2”）;

select * from t0；
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mysql_ab_test;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> insert into t0 values ("001","wugk1");
Query OK, 1 row affected (0.00 sec)

mysql> insert into t0 values ("002","wugk2");
Query OK, 1 row affected (0.00 sec)

mysql> select * from t0;
+------+-------+
| id   | name  |
+------+-------+
| 001  | wugk1 |
| 002  | wugk2 |
+------+-------+
2 rows in set (0.00 sec)

mysql>
```

图 11-16 MYSQL Master insert 数据

Slave端执行查询命令 如图11-17所示 ,表示在Master插入的数据已经同步到Slave端：

```
Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mysql_ab_test;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from t0;
+------+-------+
| id   | name  |
+------+-------+
| 001  | wugk1 |
| 002  | wugk2 |
+------+-------+
2 rows in set (0.00 sec)
```

图 11-17 MYSQL Slave 数据已同步

10.9 MySQL 主从同步排错思路

Mysql 主从同步集群在生成环境使用过程中，如果主从服务器之间网络通信条件差或者数据库数据量非常大，容易导致 MYSQL 主从同步延迟。

MYSQL 主从产生延迟之后，一旦主库宕机，会导致部分数据没有及时同步至从库，重新启动主库，会导致从库与主库同步错误，如何快速恢复主从同步关系呢，如下有两种方法：

（1） 忽略错误后，继续同步:

此种方法适用于主从库数据内容相差不大，或者要求数据可以不完全统一的情况，数据要求不严格的情况。

Master 端执行如下命令，将数据库设置全局读锁，不允许写入新数据：

```
flush  tables  with  read  lock；
```

Slave 端停止 Slave I/O 及 sql 线程，同时将同步错误的 SQL 跳过 1 次，跳过会导致数据不一致，最后启动 start slave，同步状态恢复，命令如下：

```
stop slave；

set  global sql_slave_skip_counter =1；

start slave；
```

（2） 重新做主从同步，完全同步：

此种方法适用于主从库数据内容相差很大，或者要求数据完全统一的情况，数据需完全保持一致。

Master 端执行如下命令，将数据库设置全局读锁，不允许写入新数据：

```
flush  tables  with  read  lock；
```

Master 端基于 mysqldump、xtrabackup 工具进行数据库将完整的数据库备份，也可以用 shell 脚本或 python 脚本实现定时备份，备份成功之后，将完整的数据导入至从库，重新配置主从关系，当 Slave 端的 IO 线程、SQL 线程均为 YES 之后，最后将 Master 端读锁解开即可，解锁命令如下：

```
unlock tables；
```

第11章 LAMP 架构企业实战

Linux 下 LAMP（Linux+Apache+MySQL/MariaDB+Perl/PHP/Python）是一组用来搭建动态网站的开源软件架构，本身是各自独立的软件服务，放在一起使用，拥有了越来越高的兼容度，共同组成了一个强大的 Web 应用程序平台。

本章向读者介绍互联网主流企业架构 LAMP 应用案例、PHP 解释性语言详解、LAMP 组合通信原理、LAMP 企业源码架设、LAMP 拓展及使用 Redis 提升 LAMP 性能优化等。

11.1 LAMP 企业架构简介

随着开源潮流的蓬勃发展，开放源代码的 LAMP 已经与 J2EE 和.Net 商业软件形成三足鼎立之势，并且该软件开发的项目在软件方面的投资成本较低，因此受到整个 IT 界的关注。LAMP 架构受到大多数中小企业的运维、DBA、程序员的青睐，Apache 默认

只能发布静态网页，而 LAMP 组合可以发布静态+PHP 动态页面。

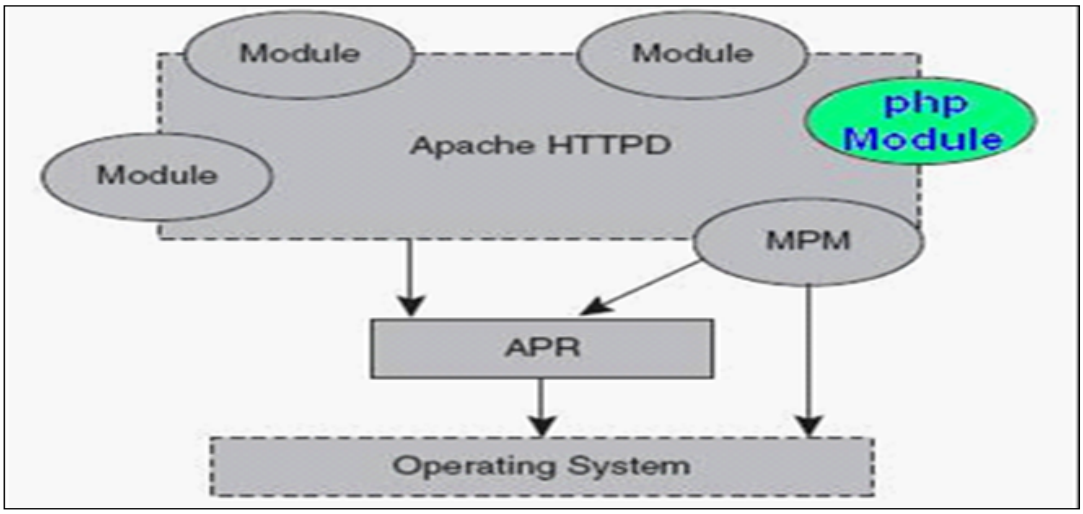
静态页面通常指不与数据库发生交互的页面，是一种基于 w3c 规范的一种网页书写格式，是一种统一协议语言，所以称之为静态网页。静态页面被设计好之后，一般很少去修改，不随着浏览器参数改变而内容改变，需注意的是动态的图片也是属于静态文件。从 SEO 角度来讲，HTML 页面更有利于搜索引擎的爬行和收录。常见的静态页面以.html、.gif、.jpg、.jpeg、.bmp、.png、.ico、.txt、.js、.css 等结尾。

动态页面通常指与数据库发生交互的页面，内容展示丰富，功能非常强大，实用性广。从 SEO 角度来讲，搜索引擎很难全面的爬行和收录动态网页，因为动态网页会随着数据库的更新、参数的变更而发生改变，常见的动态页面以.jsp、.php、.do、.asp、.cgi、.apsx 等结尾。

11.2 Apache 与 PHP 工作原理

LAMP 企业主流架构最重要的三个环节，一是 Apache WEB 服务器，二是 PHP（PHP: Hypertext Preprocessor），三是 MYSQL 数据库。

Apache WEB 服务器主要是基于多模块工作，依赖 PHP SAPI 处理方式中的 PHP_MODULE 去解析 PHP 结尾的文件，如图 12-1 所示：



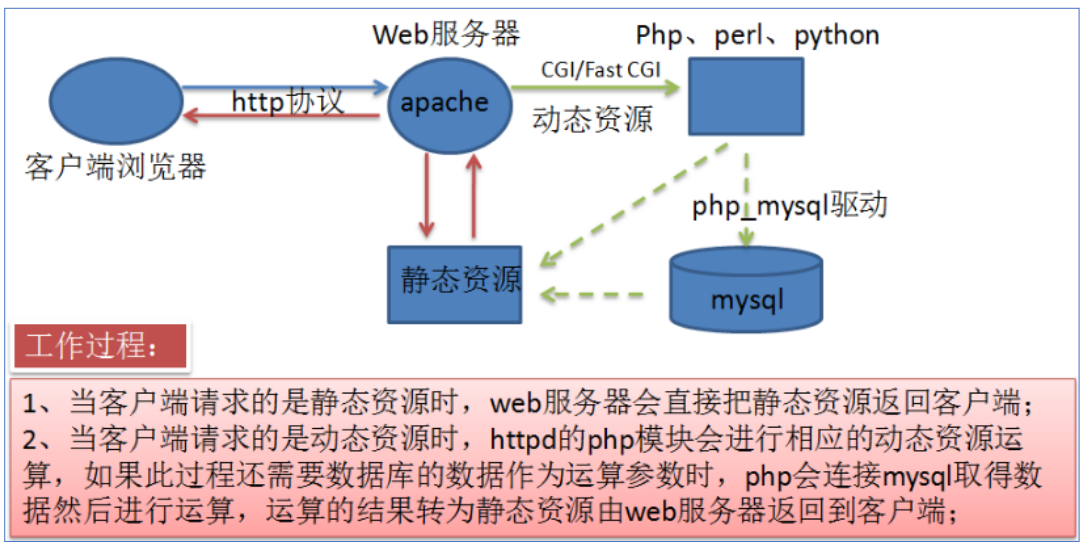


图 12-1 Apache+PHP mod 工作原理

PHP 是一种适用于 web 开发的动态语言，PHP 语言内核基于 C 语言实现包含大量组件的软件框架，是一种功能强大的解释型脚本语言。PHP 底层运行机制如图 12-2 所示：

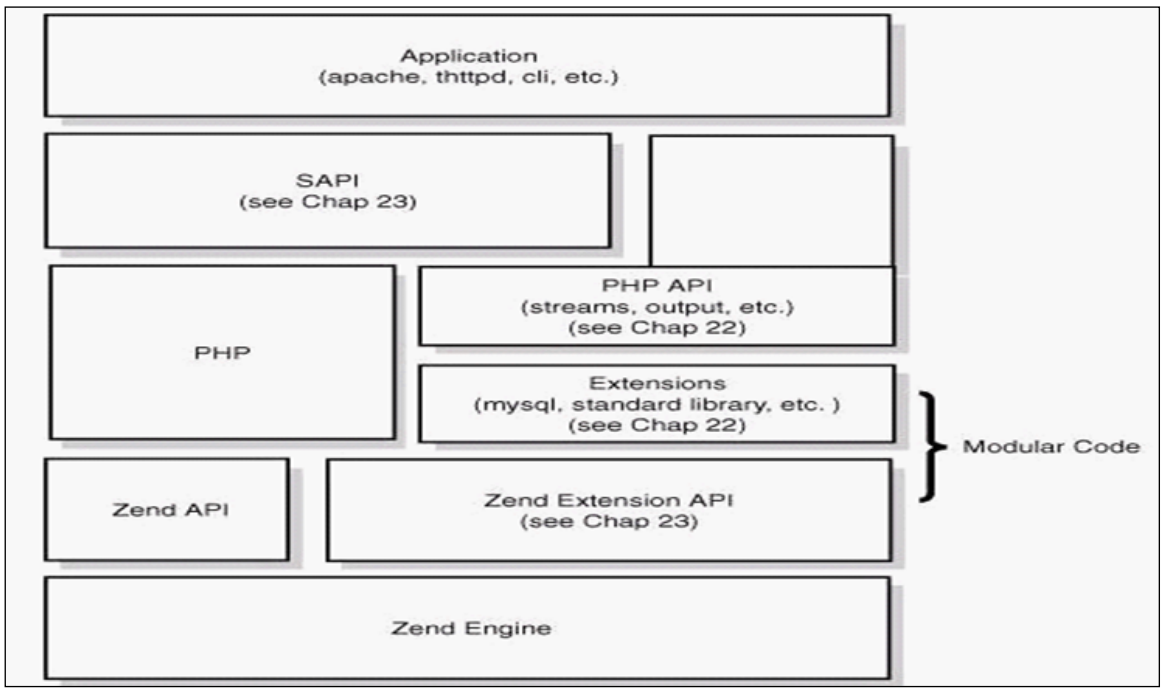


图 12-2 PHP 底层处理机制

PHP 底层工作原理包括 4 个部分：

- Zend 引擎，属于 PHP 内核部分，它负责将 PHP 代码解析为可执行 opcode 的处理并实现相应的处理方法、实现基本的数据结构、内存分配及管理、提供了相应的 api 方法供外部调用，是一切的核心，所有的外围功能均围绕 Zend 实现。
- Extensions，围绕着 Zend 引擎，Extensions 通过组件的方式提供各种基础服务，各种内置函数、标准库等都是通过 Extension 来实现。
- Sapi，服务端应用编程接口（Server Application Programming Interface，Sapi），sapi 通过一系列钩子函数，基于 SAPI 可以让 PHP 与外部进行数据交互。

- 常见的 SAPI 编程接口处理方法包括：apache2handler ：以 apache 作为 webserver，采用 MOD_PHP 模式运行时候的处理方式；cgi：webserver 和 PHP 直接的另一种交互方式，FastCGI 协议；cli：命令行调用的应用模式。
- APP 代码应用，又称之为 PHP 代码程序，基于 sapi 接口生成不同的应用模式，从而被 PHP 引擎解析。

当用户在浏览器地址中输入域名或者域名+PHP 页面，向 HTTP　WEB 服务器 Apache 发起 HTTP 请求，WEB 服务器接受该请求，并根据其后缀判断如果请求的页面是以.php 结尾，WEB 服务器从硬盘或者内存中取出该 PHP 文件，将其发送给 PHP 引擎程序。

PHP 引擎程序将会对 WEB 服务器传送过来的文件进行扫描并根据命令从后台读取、处理数据、并动态地生成相应的 HTML 页面。然后 PHP 引擎程序将生成的 HTML 页面返回给 WEB 服务器，最终 WEB 服务器将 HTML 页面返回给客户端浏览器，浏览器基于 MIME 类型进行解析展示给用户页面。

11.3 LAMP 企业安装配置

构建 LAMP 架构有两种方法，一是使用 YUM 在线安装，另外一种是基于 LAMP 源码编译安装，YUM 在线安装方法如下：

```
yum install httpd httpd-devel mysql mysql-server mysql-devel php php-devel php-mysql -y
```

```
service httpd restart

service mysqld restart
```

YUM 方式安装简单、快捷，但如果需要添加扩展的功能和模块，需使用源码包的方式来编译安装 LAMP。如下为 LAMP 源码编译安装的步骤：

（1） Apache WEB 安装，先安装 apr、apr-utils 库包。

```
yum install apr-devel apr-util-devel -y；

cd  /usr/src  ；

wget http://mirror.bit.edu.cn/apache/httpd/httpd-2.2.31.tar.gz

tar  xzf  httpd-2.2.31.tar.gz

cd httpd-2.2.31

./configure --prefix=/usr/local/apache --enable-so --enable-rewrite

make

make install
```

（2） MYSQL 数据库安装，基于 MYSQL5.5 编译安装，通过 cmake、make、make install 三个步骤实现。

```
wget http://down1.chinaunix.net/distfiles/mysql-5.5.20.tar.gz

yum  install  cmake  make  ncurses-devel ncurses -y

cmake  . -DCMAKE_INSTALL_PREFIX=/usr/local/mysql55 \

-DMYSQL_UNIX_ADDR=/tmp/mysql.sock \
```

```
-DMYSQL_DATADIR=/data/mysql \

-DSYSCONFDIR=/etc \

-DMYSQL_USER=mysql \

-DMYSQL_TCP_PORT=3306 \

-DWITH_XTRADB_STORAGE_ENGINE=1 \

-DWITH_INNOBASE_STORAGE_ENGINE=1 \

-DWITH_PARTITION_STORAGE_ENGINE=1 \

-DWITH_BLACKHOLE_STORAGE_ENGINE=1 \

-DWITH_MYISAM_STORAGE_ENGINE=1 \

-DWITH_READLINE=1 \

-DENABLED_LOCAL_INFILE=1 \

-DWITH_EXTRA_CHARSETS=1 \

-DDEFAULT_CHARSET=utf8 \

-DDEFAULT_COLLATION=utf8_general_ci \

-DEXTRA_CHARSETS=all \

-DWITH_BIG_TABLES=1 \

-DWITH_DEBUG=0

make

make install
```

将源码安装的 Mysql 数据库服务设置为系统服务，可以使用 chkconfig 管理，并启动 MYSQL 数据库：

```
cd /usr/local/mysql55/

\cp support-files/my-large.cnf /etc/my.cnf

\cp support-files/mysql.server /etc/init.d/mysqld

chkconfig --add mysqld

chkconfig --level 35 mysqld on

mkdir  -p  /data/mysql

useradd  mysql

/usr/local/mysql55/scripts/mysql_install_db --user=mysql                    --

datadir=/data/mysql/ --basedir=/usr/local/mysql55/

ln  -s  /usr/local/mysql55/bin/* /usr/bin/

service  mysqld  restart
```

（3） PHP 服务安装，PHP 需与 Apache、MySQL 进行整合，如图 12-3 所示，参数命令如下：

```
cd /usr/src

wget http://mirrors.sohu.com/php/php-5.3.28.tar.bz2

tar jxf  php-5.3.28.tar.bz2

cd php-5.3.28  ;
```

```
./configure --prefix=/usr/local/php5 --with-config-file-path=/usr/local/php5/etc --with-apxs2=/usr/local/apache2/bin/apxs --with-mysql=/usr/local/mysql55/
make
make install
```

```
Installing PHP CLI man page:      /usr/local/php5/man/man1/
Installing build environment:     /usr/local/php5/lib/php/build/
Installing header files:          /usr/local/php5/include/php/
Installing helper programs:       /usr/local/php5/bin/
  program: phpize
  program: php-config
Installing man pages:             /usr/local/php5/man/man1/
  page: phpize.1
  page: php-config.1
Installing PEAR environment:      /usr/local/php5/lib/php/
[PEAR] Archive_Tar    - installed: 1.3.11
[PEAR] Console_Getopt - installed: 1.3.1
warning: pear/PEAR requires package "pear/Structures_Graph" (recommended version 1.0.4)
warning: pear/PEAR requires package "pear/XML_Util" (recommended version 1.2.1)
[PEAR] PEAR           - installed: 1.9.4
Wrote PEAR system config file at: /usr/local/php5/etc/pear.conf
You may want to add: /usr/local/php5/lib/php to your php.ini include_path
[PEAR] Structures_Graph- installed: 1.0.4
[PEAR] XML_Util       - installed: 1.2.1
/usr/src/php-5.3.28/build/shtool install -c ext/phar/phar.phar /usr/local/php5/bin
ln -s -f /usr/local/php5/bin/phar.phar /usr/local/php5/bin/phar
Installing PDO headers:          /usr/local/php5/include/php/ext/pdo/
[root@node2 php-5.3.28]#
```

图 12-3 LAMP 源码编译整合

（4） Apache+PHP 源码整合

为了能让 Apache 发布 PHP 页面，需要将 PHP 安装完成后的 libphp5.so 模块与 Apache 进行整合，vim httpd.conf 编辑配置文件，加入如下代码：

```
LoadModule php5_module modules/libphp5.so
AddType application/x-httpd-php .php
DirectoryIndex index.php index.html index.htm
```

（5） 测试 Apache+PHP 环境

创建 PHP 测试页面，在/usr/local/apache/htdocs 目录下创建 index.php 测试页面，执行如下命令自动创建：

```
cat >/usr/local/apache/htdocs/index.php<<EOF
<?php
phpinfo()；
?>
EOF
```

重新启动 Apache 服务，浏览器输入 Apache WEB 的 IP 访问，如图 12-4 所示，即代表 LAMP 源码环境整合成功。

PHP Version 5.3.28



System	Linux node2 2.6.18-308.el5 #1 SMP Tue Feb 21 20:06:06 EST 2012 x86_64
Build Date	May 26 2014 14:34:36
Configure Command	'./configure' '--prefix=/usr/local/php5' '--with-config-file-path=/usr/local/php/etc' '--with-apxs2=/usr/local/apache/bin/apxs' '--with-mysql=/usr/local/mysql/'
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/php/etc
Loaded Configuration File	(none)
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20090626
PHP Extension	20090626
Zend Extension	220090626

图 12-4 Apache+PHP 测试页面

（6） Discuz PHP 论坛安装

LAMP 源码整合完毕之后，Dicuz 官网下载 Discuz 开源 PHP 软件包，将软件包解压并发布在 Apache Htdocs 发布目录，代码如下：

```
cd /usr/src ;

wget http://download.comsenz.com/DiscuzX/3.1/Discuz_X3.1_SC_UTF8.zip

unzip Discuz_X3.1_SC_UTF8.zip -d /usr/local/apache/htdocs/

cd /usr/local/apache/htdocs/;\mv upload/* .

chmod 757 -R data/ uc_server/ config/ uc_client/
```

通过浏览器访问 Apache WEB IP，如图 12-5 所示，选择“我同意”



图 12-5 Discuz 安装界面一

进入如图 12-6 界面，数据库安装，如果不存在则需要新建数据库并授权。



图 12-6 Discuz 安装界面二

MYSQL 数据库命令行中创建 PHP 连接 MYSQL 的用户及密码，命令如下：

```
create database discuz charset=utf8;

grant all on discuz.* to root@'localhost' identified by "123456";
```

单击下一步，直至安装完成，浏览器自动跳转至如图 12-7 所示界面：



图 12-7 Discuz 安装界面三

11.4 LAMP 企业架构拓展实战

如上 LAMP 服务均安装至单台服务器，随着用户访问量不断的增加，单台服务器压力逐渐增加 ,那如何优化 LAMP 架构 ,如何拆分 LAMP 架构呢 ,怎么把 Apache 和 MySQL 分开放在不同的机器呢。

LAMP 架构拆分的目的在于缓解单台服务器的压力，可以将 PHP、MYSQL 单独安装至多台服务器，本节将实现 LAP+MySQL 的架构，也即是把 MYSQL 单独拆分出去。部署方法有两种：

（1） YUM 安装 LAMP 多机方案

在 Apache WEB 服务器只需只需如下代码：

```
yum install httpd httpd-devel php-devel php php-mysql -y
```

在 MYSQL 数据库服务器只需只需如下代码：

```
yum  install  mysql-server mysql mysql-devel  mysql-libs  -y
```

（2） 源码安装 LAMP 多机方案

源码安装 LAMP 多机方式，Apache WEB 服务与 MYSQL 数据库服务分别部署在不同的服务器即可，PHP 与 Apache 服务部署在一台服务器，PHP 编译参数时加入如下代码进行 LAMP 的整合，mysqlnd 为 PHP 远程连接 MYSQL 数据库服务器的一种方式：

```
./configure --prefix=/usr/local/php5  \
--with-mysql=mysqlnd  --with-mysqli=mysqlnd  --with-pdo-
mysql=mysqlnd  \
--with-apxs2=/usr/local/apache2/bin/apxs
make
make  install
```

11.5 LAMP+Redis 企业实战

LAMP 在企业生产环境中，除了将 MYSQL 单独部署在其他服务器、由于 MYSQL 数据库压力会很大，还会对 MYSQL 实现主从复制及读写分离，同时会对 PHP 网站进行调优，通常 PHP 的优化手段包括：PHP 代码本身优化、PHP 配置文件优化、为 PHP 添加缓存模块，将 PHP 网站数据存入缓存等。

11.5.1 Redis 入门简介

Redis 是一个开源的使用 ANSI C 语言编写、支持网络、可基于内存亦可持久化的

日志型、Key-Value 数据库，并提供多种语言的 API。Redis 是一个 key-value 存储系统。

和 Memcached 缓存类似，Redis 支持存储的 value 类型相对更多，包括 string(字符串)、list(链表)、set(集合)、zset(有序集合)和 hash（哈希类型）。

Redis 是一种高级 key-value 数据库，它跟 memcached 类似，不过 Redis 的数据可以持久化，而且支持的数据类型很丰富，有字符串，链表，集 合和有序集合。支持在服务器端计算集合的并，交和补集(difference)等，还支持多种排序功能。Redis 也被看成是一个数据结构服务器。

Redis 很大程度补偿了 memcached 这类 key/value 存储的不足，在部分场合可以对关系数据库起到很好的补充作用。Redis 提供了 Java，C/C++，C#，PHP，JavaScript，Perl，Object-C，Python，Ruby，Erlang 等客户端，方便易用，得到 IT 人的青睐。

Redis 支持主从同步，数据可以从主服务器向任意数量的从服务器上同步，从服务器可以是关联其他从服务器的主服务器。这使得 Redis 可执行单层树复制，由于完全实现了发布/订阅机制，使得从数据库在任何地方同步树时，可订阅一个频道并接收主服务器完整的消息发布记录，同步对读取操作的可扩展性和数据冗余很有帮助。

目前使用 Redis 的互联网企业有：京东、百度、腾讯、阿里巴巴、新浪、图吧、研修网等等，如表 12-1 所示，为目前主流数据库简单功能对比：

名称	数据库类型	数据存储选项	操作类型	备注

Redis	内存存储，Nosql 数据库	支持字符串、列表、集合、散列表、有序集合	增、删、修改、更新	支持分布式集群、主从同步及高可用、单线程
Memcached	内存缓存数据库，键值对	键值之间的映射	增、删、修改、更新	支持多线程
MySQL	典型关系数据库，RDBMS	数据库由多表主成，每张表包含多行	增、删、修改、更新	支持 ACID 性质
PostgreSQL	典型关系数据库，RDBMS	数据库由多表主成，每张表包含多行	增、删、修改、更新	支持 ACID 性质
MongoDB	硬盘存储，Nosql 数据库	数据库包含多个表	增、删、修改、更新	主从复制，分片，副本集、空间索引

表 12-1 常见数据库功能对比

11.5.2 LAMP+Redis 工作机制

LAMP+Redis 工作机制：用户通过浏览器访问 LAMP 网站，并以用户名和密码登录到网站，默认 Redis 缓存中没有该用户名和密码对应列表，PHP 程序会读取 MYSQL 数据库中的用户名和密码，然后将用户名和密码缓存至 Redis 中，下次用户通过浏览器再次使用同样的用户名和密码登录网站，PHP 无需从数据库中读取该用户和密码信息，而是直接优先从 Redis 缓存中读取并返回，从而减轻 MYSQL 数据库的压力。

Redis 除了可以缓存用户名、密码，还可以换成 PHP 论坛各种数据，例如用户帖子、用户动态等等，如图 12-8 所示：

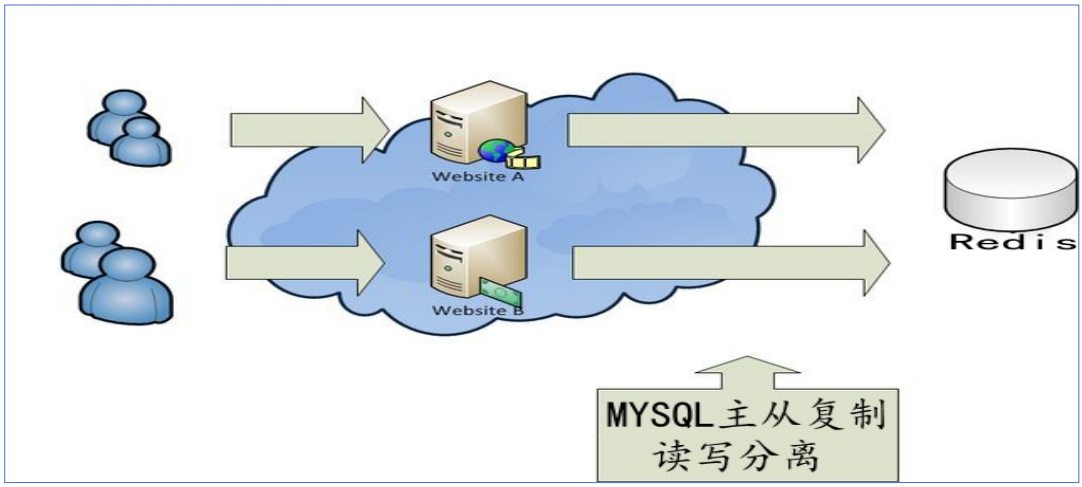


图 12-8 LAMP+Redis 架构流程图

要实现将 LAMP PHP 网站相关数据存入 Redis ,需要一台 Redis 服务器、PHP-redis 连接驱动、PHP 代码连接修改等。

11.5.3 LAMP+Redis 操作案例

LAMP PHP 连接 Redis，首先需安装 Redis 服务器，安装连接驱动，然后修改 PHP 网站配置文件，具体操作步骤如下：

（1） LAMP+Redis 实战环境配置

LAMP 服务器： 192.168.149.128
Redis 主库： 192.168.149.129
Redis 从库： 192.168.149.130

（2） 192.168.149.129 服务器安装部署 Redis 服务，代码如下

```
wget    http://download.redis.io/releases/redis-2.8.13.tar.gz
tar     zxf             redis-2.8.13.tar.gz
cd      redis-2.8.13
make    PREFIX=/usr/local/redis   install
cp      redis.conf      /usr/local/redis/
```

将/usr/local/redis/bin/目录加入至环境变量配置文件/etc/profile 末尾 然后 Shell 终端执行 source /etc/profile 让环境变量生效。

```
export PATH=/usr/local/redis/bin:$PATH
```

Nohup 后台启动及停止 Redis 服务命令：

```
nohup   /usr/local/redis/bin/redis-server   /usr/local/redis/redis.conf   &
/usr/local/redis/bin/redis-cli   -p   6379 shutdown
```

（3） 安装 PHP-Redis 连接驱动

要确保 PHP 能够连接 Redis 缓存服务器，需添加 PHP Redis 扩展程序，也即是添加 PHP 安 ext 扩展模块，添加方法如下:

```
wget   https://github.com/phpredis/phpredis/archive/3.1.2.tar.gz
tar    xzf   3.1.2.tar.gz
/usr/local/php5/bin/phpize
cd     phpredis-3.1.2/
./configure     --with-php-config=/usr/local/php5/bin/php-config     --enable-
```

```
redis

make

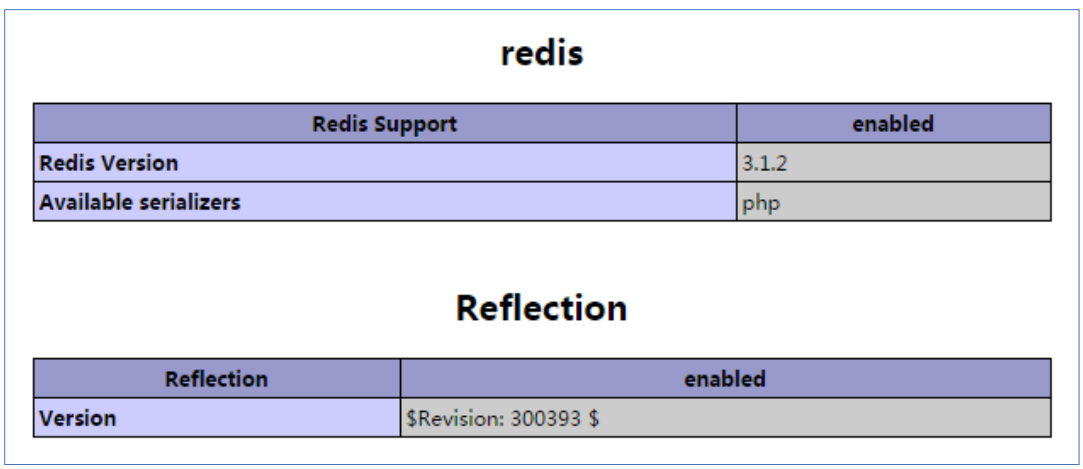
make install
```

修改 vim /usr/local/php5/lib/php.ini 配置文件，添加 redis.so 模块，代码如下：

```
extension_dir = "/usr/local/php5/lib/php/extensions/no-debug-zts-20090626"

extension=redis.so
```

重启 Apache 服务，写入 phpinfo 测试页面，通过浏览器访问，如图 12-9 所示，检查到存在 Redis 模块即可：



redis

Redis Support	enabled
Redis Version	3.1.2
Available serializers	php

Reflection

Reflection	enabled
Version	$Revision: 300393 $

图 12-9 PHP Redis 模块添加

（4） LAMP+Redis 缓存测试

登录 192.168.149.128 WEB 服务器，修改 Discuz PHP 网站发布 /usr/local/apache2/htdcos 目录全局配置文件 config_global.php，查找 CONFIG MEMORY 段，将 redis server 后改为 Redis 主服务器的 IP 192.168.149.129 即可，

如图 12-10 所示：

```
// ------------------------  CONFIG MEMORY  ----------------
$_config['memory']['prefix'] = 'IOKLan_';
$_config['memory']['redis']['server'] = '192.168.149.129';
$_config['memory']['redis']['port'] = 6379;
$_config['memory']['redis']['pconnect'] = 1;
$_config['memory']['redis']['timeout'] = '0';
$_config['memory']['redis']['requirepass'] = '';
$_config['memory']['redis']['serializer'] = 1;

$_config['memory']['memcache']['server'] = '';
$_config['memory']['memcache']['port'] = 11211;
$_config['memory']['memcache']['pconnect'] = 1;
$_config['memory']['memcache']['timeout'] = 1;
```

图 12-10 PHP Redis 配置文件修改

通过浏览器访问 Apache PHP 论坛网站，同时登陆 Redis 服务器，执行命令 redis-cli 进入 Redis 命令行，运行命令 KEYS *，如图 12-11 所示，存在以 IOKLAN 开头的 key，则证明 Redis 成功缓存 LAMP+Discuz 网站信息数据。

```
[root@www-jfedu-net ~]# redis-cli
127.0.0.1:6379>
127.0.0.1:6379> KEYS *
 1) "IOkLan_forum_index_page_1"
 2) "IOkLan_style_default"
 3) "IOkLan_magic"
 4) "IOkLan_usergroups"
 5) "IOkLan_creditrule"
 6) "IOkLan_common_member_field_home_2"
 7) "IOkLan_usergroup_7"
 8) "IOkLan_userstats"
 9) "IOkLan_cronnextrun"
10) "IOkLan_plugin"
11) "IOkLan_common_member_count_1"
12) "IOkLan_onlinelist"
13) "IOkLan_adminmenu"
14) "IOkLan_onlinerecord"
15) "IOkLan_common_member_field_home_1"
```

图 12-11 Redis 缓存 LAMP KEYS 数据

（3） 测试 Redis 缓存是否生效

访问 LAMP+Discuz 网站，创建论坛测试用户 jfedu666，密码 jfedu666，此时用

户数据第一次注册，用户名和密码会写入到 MySQL 数据库表中，同时会写入该数据也会写入到 Redis 缓存，如图 12-12（a）、12-12（b）、12-12（c）所示：

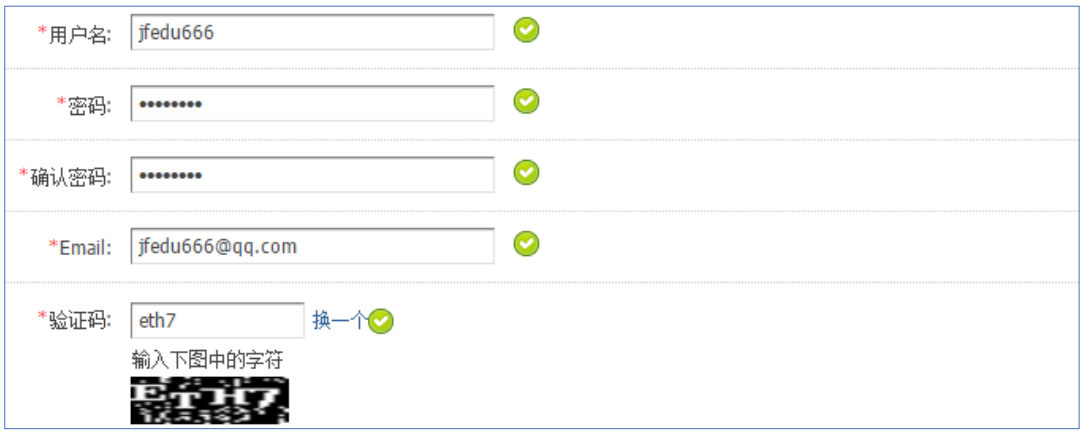


图 12-12（a） 创建论坛用户和密码

```
mysql>
mysql>
mysql> select uid,email,username,password from pre_common_member;
+-----+--------------------+------------+----------------------------------
| uid | email              | username   | password
+-----+--------------------+------------+----------------------------------
|   1 | admin@admin.com    | admin      | b2698f250316a7b6f6b8b8df80bae3
|   2 | jfedu@qq.com       | jfedu      | 15a67a7ada8faf14e164184eb1313a
|   3 | jfedujfedu@163.com | jfedujfedu | c730c9b640b5c63479e34011b979c7
|   5 | jfedu666@qq.com    | jfedu666   | a777c022d7c92fbcb3e5b25e3a8398
+-----+--------------------+------------+----------------------------------
4 rows in set (0.00 sec)
```

图 12-12（b） MySQL 数据库用户查询

```
29) "IOKLan_creditrule"
30) "IOKLan_fields_required"
31) "IOKLan_common_member_5"
32) "IOKLan_userapp"
redis 127.0.0.1:6379> get IOKLan_common_member_5
"a:23:{s:3:\"uid\";s:1:\"5\";s:5:\"email\";s:15:\"jfedu666@qq.com
32:\"a777c022d7c92fbcb3e5b25e3a83983a\";s:6:\"status\";s:1:\"0\";
:\"0\";s:16:\"videophotostatus\";s:1:\"0\";s:7:\"adminid\";s:1:\"
\"0\";s:11:\"extgroupids\";s:0:\"\";s:7:\"regdate\";s:10:\"149705
1:\"0\";s:10:\"timeoffset\";s:4:\"9999\";s:5:\"newpm\";s:1:\"0\";
2:\"allowadmincp\";s:1:\"0\";s:18:\"onlyacceptfriendpm\";s:1:\"0\
"
redis 127.0.0.1:6379>
```

图 12-12（c） Redis 缓存测试案例

将 jfedu666 从 MySQL Discuz 库 pre_common_member 中删除，通过该用户依然可以正常登录 WEB 网站，则证明此时数据读取的是 Redis 缓存服务器，如图 12-13（a）、12-13（b）、12-13（c）所示：

```
mysql>
mysql> delete from pre_common_member where username='jfedu666';
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> select uid,email,username,password from pre_common_member;
+-----+--------------------+------------+----------------------------
| uid | email              | username   | password
+-----+--------------------+------------+----------------------------
|   1 | admin@admin.com    | admin      | b2698f250316a7b6f6b8b8df80
|   2 | jfedu@qq.com       | jfedu      | 15a67a7ada8faf14e164184eb1
|   3 | jfedujfedu@163.com | jfedujfedu | c730c9b640b5c63479e34011b9
+-----+--------------------+------------+----------------------------
3 rows in set (0.00 sec)
```

图 12-13（a） 删除数据库用户和密码



图 12-13（b） 用户名和密码登录 discuz 论坛



图 12-13（c） 用户名和密码登录 discuz 论坛

11.6 Redis 配置文件详解

Redis 是一个内存数据库，附 Redis.conf 常用参数的详解，后面章节会继续深入讲解。

```
# daemonize no   Linux Shell 终端运行 redis，改为 yes 即后台运行 Redis 服务；

daemonize yes

# 当运行多个 redis 服务时，需要指定不同的 pid 文件和端口

pidfile /var/run/redis_6379.pid

# 指定 redis 运行的端口，默认是 6379

port 6379

# 在高并发的环境中，为避免慢客户端的连接问题，需要设置一个高速后台日志

tcp-backlog  511

# 指定 redis 只接收来自于该 IP 地址的请求，如果不进行设置，那么将处理所有请求
```

```
# bind 192.168.1.100 10.0.0.1

# bind 127.0.0.1

#设置客户端连接时的超时时间，单位为秒。当客户端在这段时间内没有发出任何指令，
那么关闭该连接；

timeout 0

# 在 Linux 上，指定值（秒）用于发送 ACKs 的时间。注意关闭连接需要双倍的时间。
默认为 0 。

tcp-keepalive 0

# Redis 总共支持四个日志级别： debug 、 verbose 、 notice 、 warning ，默
认为 verbose

# debug        记录很多信息，用于开发和测试

# varbose      有用的信息，不像 debug 会记录那么多

# notice       普通的 verbose ，常用于生产环境

# warning      只有非常重要或者严重的信息会记录到日志

loglevel notice

#配置 log 文件地址

#默认值为 stdout ，标准输出，若后台模式会输出到 /dev/null 。

logfile /var/log/redis/redis.log

#可用数据库数
```

```
#默认值为 16 ，默认数据库为 0，数据库范围在 0- （ database-1 ）之间

databases 16

#数据写入磁盘快照设置

#保存数据到磁盘，格式如下：

#save <seconds> <changes>

#指出在多长时间内，有多少次更新操作，就将数据同步到数据文件 rdb 。

#相当于条件触发抓取快照，这个可以多个条件配合

#比如默认配置文件中的设置，就设置了三个条件

#save 900 1  900 秒内至少有 1 个 key 被改变

#save 300 10  300 秒内至少有 300 个 key 被改变

#save 60 10000  60 秒内至少有 10000 个 key 被改变

# save 900 1

# save 300 10

# save 60 10000

#  后台存储错误停止写。

stop-writes-on-bgsave-error yes

#  存储至本地数据库时（持久化到 rdb 文件）是否压缩数据，默认为 yes

rdbcompression yes

#  本地持久化数据库文件名，默认值为 dump.rdb
```

```
dbfilename dump.rdb

#   工作目录

#   数据库镜像备份的文件放置的路径。

#   这里的路径跟文件名要分开配置是因为 redis 在进行备份时，先会将当前数据库的

状态写入到一个临时文件中，等备份完成，

#   再把该该临时文件替换为上面所指定的文件，而这里的临时文件和上面所配置的备

份文件都会放在这个指定的路径当中。

# AOF 文件也会存放在这个目录下面

#   注意这里必须制定一个目录而不是文件

dir /var/lib/redis/

################################## 复 制

################################

#   主从复制 . 设置该数据库为其他数据库的从数据库 .

#   设置当本机为 slav 服务时，设置 master 服务的 IP 地址及端口，在 Redis 启动

时，它会自动从 master 进行数据同步

# slaveof <masterip><masterport>

#   当 master 服务设置了密码保护时（ 用 requirepass 制定的密码 ）

# slave 服务连接 master 的密码

# masterauth <master-password>
```

```
#  当从库同主机失去连接或者复制正在进行，从机库有两种运行方式：
# 1)  如果 slave-serve-stale-data 设置为 yes( 默认设置 ) ，从库会继续响应客户端的请求
# 2)  如果 slave-serve-stale-data 是指为 no ，出去 INFO 和 SLAVOF 命令之外的任何请求都会返回一个
#       错误 "SYNC with master in progress"
slave-serve-stale-data yes
#  配置 slave 实例是否接受写。写 slave 对存储短暂数据（在同 master 数据同步后可以很容易地被删除）是有用的，但未配置的情况下，客户端写可能会发送问题。
#  从 Redis2.6 后，默认 slave 为 read-only
slaveread-only yes
#  从库会按照一个时间间隔向主库发送 PINGs. 可以通过 repl-ping-slave-period 设置这个时间间隔，默认是 10 秒
# repl-ping-slave-period 10
# repl-timeout  设置主库批量数据传输时间或者 ping 回复时间间隔，默认值是 60 秒
#  一定要确保 repl-timeout 大于 repl-ping-slave-period
# repl-timeout 60
#  在 slave socket 的 SYNC 后禁用 TCP_NODELAY
```

如果选择“ yes ” ,Redis 将使用一个较小的数字 TCP 数据包和更少的带宽将数据发送到 slave ， 但是这可能导致数据发送到 slave 端会有延迟 ，如果是 Linux kernel 的默认配置，会达到 40 毫秒 .

如果选择 "no" ，则发送数据到 slave 端的延迟会降低，但将使用更多的带宽用于复制 .

repl-disable-tcp-nodelay no

设置复制的后台日志大小。

复制的后台日志越大， slave 断开连接及后来可能执行部分复制花的时间就越长。

后台日志在至少有一个 slave 连接时，仅仅分配一次。

repl-backlog-size 1mb

在 master 不再连接 slave 后，后台日志将被释放。下面的配置定义从最后一个 slave 断开连接后需要释放的时间（秒）。

0 意味着从不释放后台日志

repl-backlog-ttl 3600

如果 master 不能再正常工作，那么会在多个 slave 中，选择优先值最小的一个 slave 提升为 master ，优先值为 0 表示不能提升为 master 。

slave-priority 100

如果少于 N 个 slave 连接，且延迟时间 <=M 秒，则 master 可配置停止接受写操作。

```
#   例如需要至少 3 个 slave 连接，且延迟 <=10 秒的配置：

# min-slaves-to-write 3

# min-slaves-max-lag 10

#   设置 0 为禁用

#    默认 min-slaves-to-write 为 0 （禁用）， min-slaves-max-lag 为 10

################################### 安 全

###################################

#   设置客户端连接后进行任何其他指定前需要使用的密码。

#   警告：因为 redis 速度相当快，所以在一台比较好的服务器下，一个外部的用户可以在一秒钟进行 150K 次的密码尝试，这意味着你需要指定非常非常强大的密码来防止暴力破解

# requirepass jfedu

#   命令重命名 .

#   在一个共享环境下可以重命名相对危险的命令。比如把 CONFIG 重名为一个不容易猜测的字符。

#   举例 ：

# rename-command CONFIG b840fc02d524045429941cc15f59e41cb7be6c52

#   如果想删除一个命令，直接把它重命名为一个空字符 "" 即可，如下：

# rename-command CONFIG ""
```

```
################################## 约束
##################################
#设置同一时间最大客户端连接数，默认无限制，
#Redis 可以同时打开的客户端连接数为 Redis 进程可以打开的最大文件描述符数，
#如果设置 maxclients 0 ，表示不作限制。
#当客户端连接数到达限制时， Redis 会关闭新的连接并向客户端返回 max number
of clients reached 错误信息
# maxclients 10000
# 指定 Redis 最大内存限制， Redis 在启动时会把数据加载到内存中，达到最大内
存后， Redis 会按照清除策略尝试清除已到期的 Key
# 如果 Redis 依照策略清除后无法提供足够空间 ,或者策略设置为“noeviction” ，
则使用更多空间的命令将会报错，例如 SET, LPUSH 等。但仍然可以进行读取操作
# 注意： Redis 新的 vm 机制，会把 Key 存放内存， Value 会存放在 swap 区
# 该选项对 LRU 策略很有用。
# maxmemory 的设置比较适合于把 redis 当作于类似 memcached 的缓存来使用，
而不适合当做一个真实的 DB 。
# 当把 Redis 当做一个真实的数据库使用的时候，内存使用将是一个很大的开销
# maxmemory <bytes>
# 当内存达到最大值的时候 Redis 会选择删除哪些数据？有五种方式可供选择
```

```
# volatile-lru ->  利用 LRU 算法移除设置过过期时间的 key (LRU: 最近使用 Least RecentlyUsed )
# allkeys-lru ->  利用 LRU 算法移除任何 key
# volatile-random ->  移除设置过过期时间的随机 key
# allkeys->random -> remove a randomkey, any key
# volatile-ttl ->  移除即将过期的 key(minor TTL)
# noeviction ->  不移除任何可以，只是返回一个写错误
#  注意：对于上面的策略，如果没有合适的 key 可以移除，当写的时候 Redis 会返回一个错误
#  默认是： volatile-lru
# maxmemory-policy volatile-lru
# LRU 和 minimal TTL 算法都不是精准的算法，但是相对精确的算法（为了节省内存），随意你可以选择样本大小进行检测。
# Redis 默认的灰选择 3 个样本进行检测，你可以通过 maxmemory-samples 进行设置
# maxmemory-samples 3
##############################
AOF################################
#  默认情况下， redis 会在后台异步的把数据库镜像备份到磁盘，但是该备份是非常
```

耗时的，而且备份也不能很频繁，如果发生诸如拉闸限电、拔插头等状况，那么将造成比较大范围的数据丢失。

所以 redis 提供了另外一种更加高效的数据库备份及灾难恢复方式。

开启 append only 模式之后， redis 会把所接收到的每一次写操作请求都追加到 appendonly.aof 文件中，当 redis 重新启动时，会从该文件恢复出之前的状态。

但是这样会造成 appendonly.aof 文件过大，所以 redis 还支持了 BGREWRITEAOF 指令，对 appendonly.aof 进行重新整理。

你可以同时开启 asynchronous dumps 和 AOF

appendonly no

AOF 文件名称 （默认："appendonly.aof"）

appendfilename appendonly.aof

Redis 支持三种同步 AOF 文件的策略：

no: 不进行同步，系统去操作 . Faster.

always: always 表示每次有写操作都进行同步 . Slow, Safest.

everysec: 表示对写操作进行累积，每秒同步一次 . Compromise.

默认是 "everysec" ，按照速度和安全折中这是最好的。

如果想让 Redis 能更高效的运行，你也可以设置为 "no" ，让操作系统决定什么时候去执行

或者相反想让数据更安全你也可以设置为 "always"

```
#  如果不确定就用  "everysec".
# appendfsync always
appendfsync everysec
# appendfsync no
# AOF 策略设置为 always 或者 everysec 时，后台处理进程（后台保存或者 AOF 日志重写）会执行大量的 I/O 操作
#  在某些 Linux 配置中会阻止过长的 fsync() 请求。注意现在没有任何修复，即使 fsync 在另外一个线程进行处理
#  为了减缓这个问题，可以设置下面这个参数 no-appendfsync-on-rewrite
no-appendfsync-on-rewrite no
# AOF  自动重写
#  当 AOF 文件增长到一定大小的时候 Redis 能够调用  BGREWRITEAOF  对日志文件进行重写
#  它是这样工作的： Redis 会记住上次进行些日志后文件的大小（如果从开机以来还没进行过重写，那日子大小在开机的时候确定）
#  基础大小会同现在的大小进行比较。如果现在的大小比基础大小大制定的百分比，重写功能将启动
#  同时需要指定一个最小大小用于 AOF 重写，这个用于阻止即使文件很小但是增长幅度很大也去重写 AOF 文件的情况
```

```
#  设置  percentage 为 0 就关闭这个特性
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
######## # LUA SCRIPTING #########
# 一个 Lua 脚本最长的执行时间为 5000 毫秒（ 5 秒），如果为 0 或负数表示无限执行时间。
lua-time-limit 5000
################################LOW LOG#################################
# Redis Slow Log  记录超过特定执行时间的命令。执行时间不包括 I/O 计算比如连接客户端，返回结果等，只是命令执行时间
#  可以通过两个参数设置 slow log ：一个是告诉 Redis 执行超过多少时间被记录的参数 slowlog-log-slower-than( 微妙 ) ，
#  另一个是 slow log 的长度。当一个新命令被记录的时候最早的命令将被从队列中移除
#  下面的时间以微妙为单位，因此 1000000 代表一秒。
#  注意指定一个负数将关闭慢日志，而设置为 0 将强制每个命令都会记录
slowlog-log-slower-than 10000
#  对日志长度没有限制，只是要注意它会消耗内存
```

```
#  可以通过  SLOWLOG RESET 回收被慢日志消耗的内存

#  推荐使用默认值 128 ，当慢日志超过 128 时，最先进入队列的记录会被踢出

slowlog-max-len 128
```

11.7 Redis 常用配置

Redis 缓存服务器命令行中常用命令如下：

```
Redis  CONFIG 命令格式如下：

redis 127.0.0.1:6379> CONFIG  GET|SET CONFIG_SETTING_NAME

CONFIG  GET * 获取 Redis 服务器所有配置信息；

CONFIG  SET  loglevel  "notice"   设置 Redis 服务器日志级别；

CONFIG  SET  requirepass  "jfedu"

AUTH  jfedu

redis-cli  -h host  -p  port  -a  password 远程连接 redis 数据库；

CLIENT GETNAME              获取连接的名称;

CLIENT SETNAME              设置当前连接的名称;

CLUSTER SLOTS           获取集群节点的映射数组;

COMMAND                     获取 Redis 命令详情数组;

COMMAND COUNT               获取 Redis 命令总数;

COMMAND GETKEYS             获取给定命令的所有键;
```

TIME	返回当前服务器时间;
CONFIG GET parameter	获取指定配置参数的值;
CONFIG SET parameter value	修改 redis 配置参数，无需重启;
CONFIG RESETSTAT	重置 INFO 命令中的某些统计数据;
DBSIZE	返回当前数据库的 key 的数量;
DEBUG OBJECT key	获取 key 的调试信息;
DEBUG SEGFAULT	让 Redis 服务崩溃;
FLUSHALL	删除所有数据库的所有 key;
FLUSHDB	删除当前数据库的所有 key;
ROLE	返回主从实例所属的角色;
SAVE	异步保存数据到硬盘;
SHUTDOWN	异步保存数据到硬盘，并关闭服务器;
SLOWLOG	管理 redis 的慢日志;
SET keys values	设置 key 为 jfedu，值为 123；
DEL jfedu	删除 key 及值；
INFO CPU	查看服务器 CPU 占用信息；
KEYS jfedu	查看是存在 jfedu 的 key；
KEYS *	查看 Redis 所有的 KEY；
CONFIG REWRITE	启动 Redis 时所指定的 redis.conf 配置文件进行改写;

INFO [section]	获取 Redis 服务器的各种信息和统计数值;
SYNC	用于复制功能(replication)的内部命令;
SLAVEOF host port	指定服务器的从属服务器(slave server);
MONITOR	实时打印出 Redis 服务器接收到的命令，调试用;
LASTSAVE	返回最近一次 Redis 成功将数据保存到磁盘上的时间;;
CLIENT PAUSE timeout	指定时间内终止运行来自客户端的命令;
BGREWRITEAOF	异步执行一个 AOF（AppendOnly File） 文件重写操作;
BGSAVE	后台异步保存当前数据库的数据到磁盘。

11.8 Redis 集群主从实战

为了提升 redis 高可用性，我们除了备份 redis dump 数据之外，还需要创建 redis 主从架构，可以利用从将数据库持久化（数据持久化通俗讲就是把数据保存到磁盘上，保证不会因为断电等因素丢失数据。

Redis 需要经常将内存中的数据同步到磁盘来保证持久化。redis 支持两种持久化方式，一种是 Snapshotting（快照）也是默认方式，另一种是 Append-only file（缩写 aof）的方式。）

Redis 主从复制，当用户往 Master 端写入数据时，通过 Redis Sync 机制将数据文件发送至 Slave，Slave 也会执行相同的操作确保数据一致；且实现 Redis 的主从复制

非常简单。同时 slave 上还可以开启二级 slave，三级 slave 从库，跟 MySQL 的主从类似。

Redis 主从配置非常简单，只需要在 Redis 丛库 192.168.149.130 配置中设置如下指令，slaveof 表示指定主库的 IP，192168.149.129 为 master 服务器，6379 为 master 服务器 Redis 端口，配置方法如下：

（1） 192.168.149.129 Redis 主库 redis.conf 配置文件如下：

```
daemonize no

pidfile /var/run/redis.pid

port 6379

tcp-backlog 511

timeout 0

tcp-keepalive 0

loglevel notice

logfile ""

databases 16

save 900 1

save 300 10

save 60 10000

stop-writes-on-bgsave-error yes
```

```
rdbcompression yes

rdbchecksum yes

dbfilename redis.rdb

dir    /data/redis/

slave-serve-stale-data yes

slave-read-only yes

repl-disable-tcp-nodelay no

slave-priority 100

appendonly no

appendfilename "appendonly.aof"

appendfsync everysec

no-appendfsync-on-rewrite no

auto-aof-rewrite-percentage 100

auto-aof-rewrite-min-size 64mb

lua-time-limit 5000

slowlog-log-slower-than 10000

slowlog-max-len 128

latency-monitor-threshold 0

notify-keyspace-events ""
```

```
hash-max-ziplist-entries 512

hash-max-ziplist-value 64

list-max-ziplist-entries 512

list-max-ziplist-value 64

set-max-intset-entries 512

zset-max-ziplist-entries 128

zset-max-ziplist-value 64

hll-sparse-max-bytes 3000

activerehashing yes

client-output-buffer-limit normal 0 0 0

client-output-buffer-limit slave 256mb 64mb 60

client-output-buffer-limit pubsub 32mb 8mb 60

hz 10

aof-rewrite-incremental-fsync yes
```

（2） 192168.149.130 Redis 从库 redis.conf 配置文件如下：

```
daemonize no

pidfile /var/run/redis.pid

port  6379

slaveof  192.168.149.129  6379
```

```
tcp-backlog 511

timeout 0

tcp-keepalive 0

loglevel notice

logfile ""

databases 16

save 900 1

save 300 10

save 60 10000

stop-writes-on-bgsave-error yes

rdbcompression yes

rdbchecksum yes

dbfilename redis.rdb

dir    /data/redis/

slave-serve-stale-data yes

slave-read-only yes

repl-disable-tcp-nodelay no

slave-priority 100

appendonly no
```

```
appendfilename "appendonly.aof"

appendfsync everysec

no-appendfsync-on-rewrite no

auto-aof-rewrite-percentage 100

auto-aof-rewrite-min-size 64mb

lua-time-limit 5000

slowlog-log-slower-than 10000

slowlog-max-len 128

latency-monitor-threshold 0

notify-keyspace-events ""

hash-max-ziplist-entries 512

hash-max-ziplist-value 64

list-max-ziplist-entries 512

list-max-ziplist-value 64

set-max-intset-entries 512

zset-max-ziplist-entries 128

zset-max-ziplist-value 64

hll-sparse-max-bytes 3000

activerehashing yes
```

```
client-output-buffer-limit normal 0 0 0

client-output-buffer-limit slave 256mb 64mb 60

client-output-buffer-limit pubsub 32mb 8mb 60

hz 10

aof-rewrite-incremental-fsync yes
```

（3） 重启 Redis 主库、从库服务，在 Redis 主库创建 key 及 values，登录 Redis 从库查看，如图 12-14（a）、12-14（b）所示:

```
[root@www-jfedu-net-129 ~]# redis-cli
redis 127.0.0.1:6379>
redis 127.0.0.1:6379> set jf1  www.jf1.com
OK
redis 127.0.0.1:6379>
redis 127.0.0.1:6379> set jf2  www.jf2.com
OK
redis 127.0.0.1:6379>
redis 127.0.0.1:6379> set jf3  www.jf3.com
OK
redis 127.0.0.1:6379>
redis 127.0.0.1:6379> get jf1
"www.jf1.com"
redis 127.0.0.1:6379>
redis 127.0.0.1:6379> get jf2
"www.jf2.com"
```

图 12-14（a） Redis 主库创建 key

```
[root@192-168-149-130-jfedu ~]# redis-cli
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379> get jf1
"www.jf1.com"
127.0.0.1:6379>
127.0.0.1:6379> get jf2
"www.jf2.com"
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379> get jf3
"www.jf3.com"
127.0.0.1:6379>
```

图 12-14（b） Redis 从库获取 key 值

11.9 Redis 数据备份与恢复

Redis 所有数据都是保存在内存中，Redis 数据备份可以定期的通过异步方式保存到磁盘上，该方式称为半持久化模式，如果每一次数据变化都写入 aof 文件里面，则称为全持久化模式。同时还可以基于 Redis 主从复制实现 Redis 备份与恢复。

11.9.1 Redis 主从复制备份

通过持久化功能，Redis 保证了即使在服务器重启的情况下也不会损失（或少量损失）数据。但是由于数据是存储在一台服务器上的，如果这台服务器的硬盘出现故障，也会导致数据丢失。

为了避免单点故障，我们希望将数据库复制多个副本以部署在不同的服务器上，即使只有一台服务器出现故障其他服务器依然可以继续提供服务，这就要求当一台服务器上的数据库更新后，可以自动将更新的数据同步到其他服务器上，Redis 提供了复制（replication）功能可以自动实现同步的过程。通过配置文件在 Redis 从数据库中配置文件中加入 slaveof　master-ip　master-port 即可，主数据库无需配置。

Redis 主从复制优点及应用场景， WEB 应用程序可以基于主从同步实现读写分离以提高服务器的负载能力。在常见的场景中，读的频率一般比较大，当单机 Redis 无法应付大量的读请求时，可以通过复制功能建立多个从数据库，主数据库只进行写操作，而从数据库负责读操作，还可以基于 LVS+keepalived+Redis 对 Redis 实现均和高可用。

从数据库持久化持久化通常相对比较耗时，为了提高性能，可以通过复制功能建立一个（或若干个）从数据库，并在从数据库中启用持久化，同时在主数据库禁用持久化。

当从数据库崩溃时重启后主数据库会自动将数据同步过来，所以无需担心数据丢失。而当主数据库崩溃时，需要在从数据库中使用 SLAVEOF NO ONE 命令将从数据库提升成主数据库继续服务，并在原来的主数据库启动后使用 SLAVE　OF 命令将其设置成新的主数据库的从数据库，即可将数据同步回来。

11.10　LAMP 企业架构读写分离

LAMP+Discuz+Redis 缓解了 MYSQL 的部分压力，但是如果访问量非常大，Redis 缓存中第一次没有缓存数据，会导致 MYSQL 数据库压力增大，此时可以基于分库、分表、分布式集群、或者读写分离来分担 MYSQL 数据库的压力，以读写分离为案例，来实现分担 MYSQL 数据库的压力。

MYSQL 读写分离的原理其实就是让 Master 数据库处理事务性增、删除、修改、更新操作（CREATE、INSERT、UPDATE、DELETE），而让 Slave 数据库处理 SELECT 操作，MYSQL 读写分离前提是基于 MYSQL 主从复制，这样可以保证在 Master 上修改数据，Slave 同步之后，WEB 应用可以读取到 Slave 端的数据。

实现 MYSQL 读写分离可以基于第三方插件，也可以通过开发修改代码实现，具体实现的读写分离的常见方式有如下四种：

- MySQL-Proxy 读写分离；
- Amoeba 读写分离；
- Mycat 读写分离；
- **基于程序读写分离(效率很高，实施难度大，开发改代码)**

Amoeba 是以 MySQL 为底层数据存储，并对 WEB、APP 应用提供 MySQL 协议接口的 proxy。它集中地响应 WEB 应用的请求，依据用户事先设置的规则，将 SQL 请求发送到特定的数据库上执行，基于此可以实现负载均衡、读写分离、高可用性等需求。

Amoeba 相当于一个 SQL 请求的路由器，目的是为负载均衡、读写分离、高可用性提供机制，而不是完全实现它们。用户需要结合使用 MySQL 的 Replication 等机制来实现副本同步等功能。

Mysql-Proxy 是 MySQL 官方提供的 mysql 中间件服务，支持无数客户端连接，同时后端可连接若干台 Mysql-Server 服务器，MYSQL-Proxy 自身基于 MySQL 协议，连接 MYSQL-Proxy 的客户端无需修改任何设置， 跟正常连接 MYSQL Server 没有区别，无需修改程序代码。

MySQL Proxy 是 App 应用（客户端）与 MYSQL Server 之间的一个连接代理，MySQL Proxy 负责将 APP 应用的 SQL 请求根据转发规则，转发至相应的后端数据库，基于 lua 脚本，可以实现复杂的连接控制和过滤，从而实现数据读写分离和负载均衡的需求。

Mysql-Proxy 允许用户指定 Lua 脚本对 SQL 请求进行拦截 ,对请求进行分析与修改，

还允许用户指定 Lua 脚本对服务器的返回结果进行修改，加入一些结果集或者去除一些结果集，对 SQL 的请求通常为读请求、写请求，基于 Lua 脚本，可以实现将 SQL 读请求转发至后端 Slave 服务器，将 SQL 写请求转发至后端 Master 服务器。

如图 12-16 所示，为 MYSQL-PROXY 读写分离架构图，通过架构图可以清晰看到 SQL 请求整个流向的过程。

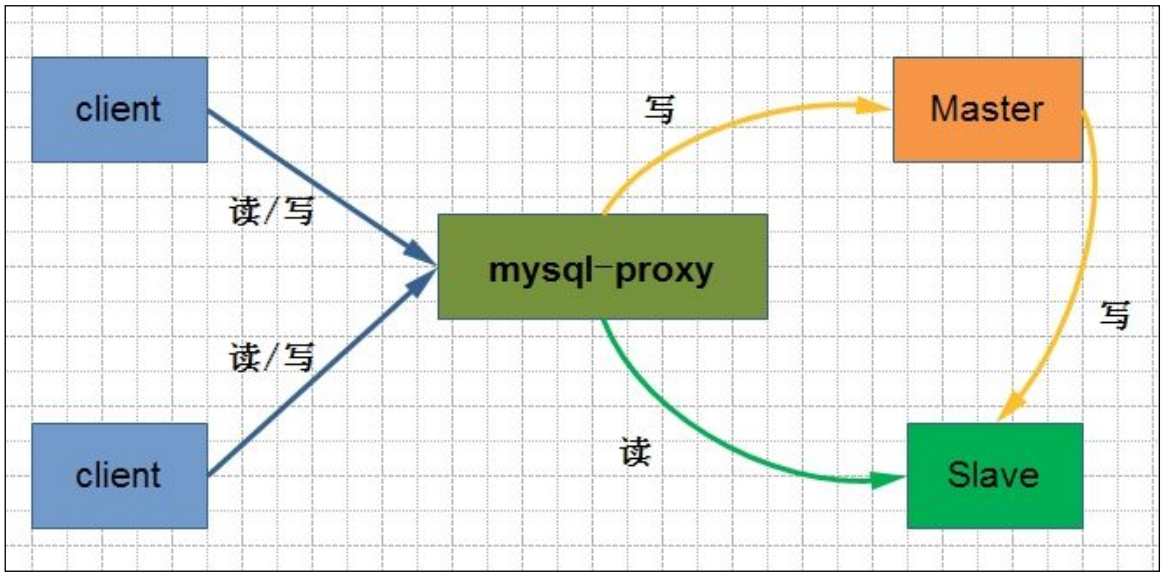


图 12-16 MYSQL-Proxy 读写分离流程

Mysql-Proxy 读写分离架构实战配置，如图 12-17 所示，两台 WEB 通过 MYSQL-Proxy 连接后端 1.14 和 1.15 MYSQL 服务器。

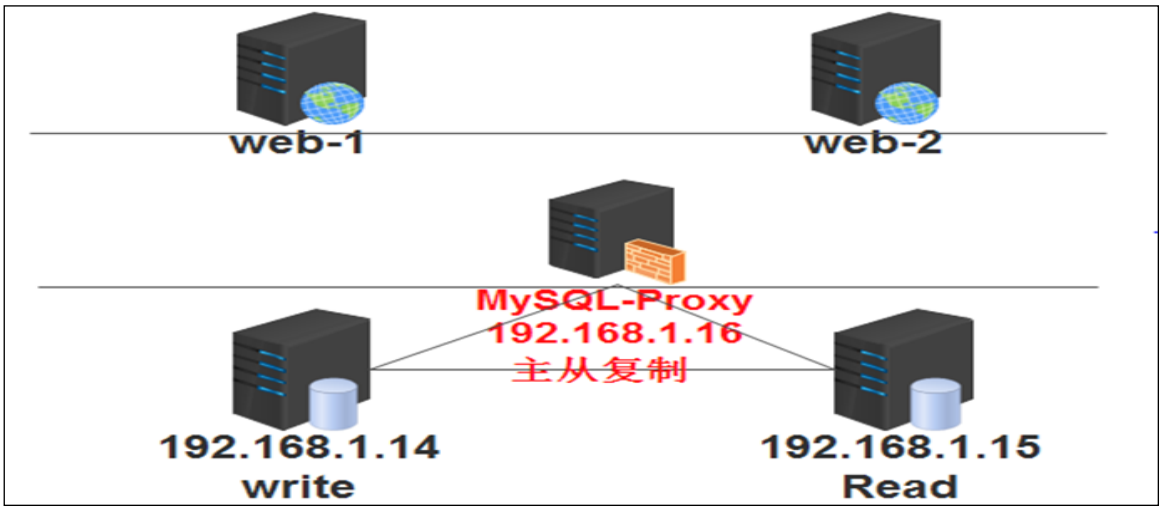


图 12-17 MYSQL-Proxy 实施架构图

构建 Mysql 读写分离架构首先需要将两台 MYSQL 服务器配置为主从复制（前文已存在，此处省略配置），配置完毕后，在 192.168.1.16 服务器上安装 Mysql-Proxy 服务即可，配置步骤如下：

（1） 下载 MYSQL-Proxy 软件版本，解压并重命名至/usr/local/mysql-proxy，命令如下：

```
wget http://ftp.ntu.edu.tw/pub/MySQL/Downloads/MySQL-Proxy/mysql-proxy-0.8.4-linux-el6-x86-64bit.tar.gz
useradd -r mysql-proxy
tar zxvf mysql-proxy-0.8.4-linux-el6-x86-64bit.tar.gz -C /usr/local
mv /usr/local/mysql-proxy-0.8.4-linux-el6-x86-64bit /usr/local/mysql-proxy
```

（2） 环境变量配置文件/etc/profile 中加入如下代码保存退出，然后执行 source /etc/profile 使环境变量配置生效即可：

```
export  PATH=$PATH:/usr/local/mysql-proxy/bin/
```

（3） 启动 MYSQL-Proxy 中间件，命令如下：

```
mysql-proxy --daemon --log-level=debug --user=mysql-proxy --keepalive --log-file=/var/log/mysql-proxy.log --plugins="proxy" --proxy-backend-addresses="192.168.1.162:3306" --proxy-read-only-backend-addresses="192.168.1.163:3306" --proxy-lua-script="/usr/local/mysql-proxy/share/doc/mysql-proxy/rw-splitting.lua" --plugins=admin --admin-username="admin" --admin-password="admin" --admin-lua-script="/usr/local/mysql-proxy/lib/mysql-proxy/lua/admin.lua"
```

（4） Mysql-Proxy 的相关参数详解如下：

```
--help-all                          ：获取全部帮助信息；
--proxy-address=host:port           ：代理服务监听的地址和端口，默认为 4040；
--admin-address=host:port           ：管理模块监听的地址和端口，默认为 4041；
--proxy-backend-addresses=host:port ：后端 mysql 服务器的地址和端口；
--proxy-read-only-backend-addresses=host:port ：后端只读 mysql 服务器的地址和端口；
--proxy-lua-script=file_name        ：完成 mysql 代理功能的 Lua 脚本；
```

--daemon ：以守护进程模式启动 mysql-proxy；

--keepalive ：在 mysql-proxy 崩溃时尝试重启之；

--log-file=/path/to/log_file_name ：日志文件名称；

--log-level=level ：日志级别；

--log-use-syslog ：基于 syslog 记录日志；

--plugins=plugin ：在 mysql-proxy 启动时加载的插件；

--user=user_name ：运行 mysql-proxy 进程的用户；

--defaults-file=/path/to/conf_file_name ：默认使用的配置文件路径，其配置段使用[mysql-proxy]标识；

--proxy-skip-profiling ：禁用 profile；

--pid-file=/path/to/pid_file_name ：进程文件名；

（5） MYSQL-Proxy 启动后，在服务器端查看端口，其中 4040 为 proxy 代理端口用于 WEB 应用连接，4041 位管理端口用于 SA 或者 DBA 管理，如图 12-48 所示：

```
[root@localhost ~]# netstat -ntpl|grep mysql-proxy
tcp        0      0 0.0.0.0:4040            0.0.0.0:*               LISTEN      2520/my
sql-proxy
tcp        0      0 0.0.0.0:4041            0.0.0.0:*               LISTEN      2520/my
sql-proxy
[root@localhost ~]#
```

图 12-48 MYSQL-Proxy 启动端口

（6） 基于 4041 端口 MySQL-Proxy 查看读写分离状态，登录 4041 管理端口，命令如下：

```
mysql   -h192.168.1.16   -uadmin   -p   -P 4041
```

（7） 以 4041 管理口登录，然后执行 select 命令，如图 12-18 所示 state 均为 up 状态，type 类型为 rw、ro，则证明读写分离状态成功。如果状态为 unknown 未知状态，可以 4040 端口登录执行：show databases；命令，直到 state 变成 up 状态为止。

```
select   *   from   backends ;
```

```
mysql> select * from backends;
+-------------+-------------------+-------+------+------+-------------------+
| backend_ndx | address           | state | type | uuid | connected_clients |
+-------------+-------------------+-------+------+------+-------------------+
|           1 | 192.168.1.14:3306 | up    | rw   | NULL |                 4 |
|           2 | 192.168.1.15:3306 | up    | ro   | NULL |                 0 |
+-------------+-------------------+-------+------+------+-------------------+
2 rows in set (0.00 sec)

mysql>
mysql>
```

图 12-18 MYSQL-Proxy 读写分离状态

（8） 读写分离数据测试，以 3306 端口登录到从库，进行数据写入和测试，在从库上创建 jfedu_test 测试库，并写入内容，如图 12-19 所示：

```
mysql> create database jfedu_test;
Query OK, 1 row affected (0.00 sec)

mysql>
mysql>
mysql> use jfedu_test
Database changed
mysql>
mysql> create table t1 (id char(20),name char(20));
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql>
mysql> insert into t1 values (01,'jfedu.net');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t1 values (02,'jfteach.com');
Query OK, 1 row affected (0.00 sec)
```

图 12-19 MYSQL-Proxy 读写分离测试

（9） 读写分离数据测试，以 4040 代理端口登录，执行如下命令，可以查看到数据即证明读写分离成功。

```
mysql -h192.168.1.16 -uroot -p123456 -P4040 -e "select * from jfedu_test.t1；"
```

（10） 登录 Apache WEB 服务器，修改 Discuz PHP 网站发布 /usr/local/apache2/htdcos 目录全局配置文件 config_global.php，查找 dbhost 段，将 192.168.1.16 改成 192.168.1.16:40404，如图 12-20 所示：

```
$_config = array();

// ----------------------------  CONFIG DB  ----------------------------
$_config['db']['1']['dbhost'] = '192.168.1.16:4040';
$_config['db']['1']['dbuser'] = 'root';
$_config['db']['1']['dbpw'] = '123456';
$_config['db']['1']['dbcharset'] = 'utf8';
$_config['db']['1']['pconnect'] = '0';
$_config['db']['1']['dbname'] = 'discuz';
$_config['db']['1']['tablepre'] = 'pre_';
$_config['db']['slave'] = '';
$_config['db']['common']['slave_except_table'] = '';

// ----------------------------  CONFIG MEMORY  ----------------------------
```

图 12-20 MYSQL-Proxy 读写分离测试

第12章 CentOS7 实战 Kickstart 批量系统部署

12.1 Kickstart 使用背景介绍

随着公司业务不断增加，经常需要采购新服务器，并要求安装 Linux 系统，并且要求 Linux 版本要一致，方便以后的维护和管理，每次人工安装 linux 系统会浪费掉更多时间，如果我们有办法能节省一次一次的时间岂不更好呢？

大中型互联网公司一次采购服务器上百台,如果采用人工手动一台一台的

安装,一个人得搞坏 N 张光盘,得多少个加班加点才能完成这项”艰巨”的任务呢,我们可以看到全人工来完成这样的工作太浪费人力了,有没有自动化安装平台呢,通过一台已存在的系统然后克隆或者复制到新的服务器呢。Kickstart 可以毫不费力的完成这项工作。

PXE(preboot execute environment ,预启动执行环境)是由 Intel 公司开发的最新技术，工作于 Client/Server 的网络模式，支持工作站通过网络从远端服务器下载映像，并由此支持通过网络启动操作系统，在启动过程中，终端要求服务器分配 IP 地址，再用 TFTP（trivial file transfer protocol）协议下载一个启动软件包到本机内存中执行。

要使用 Kickstart 安装平台，包括的完整架构为：Kickstart+DHCP+NFS(HTTP)+TFTP+PXE，从架构可以看出，大致需要安装的服务，例如 dhcp、tftp、httpd、kickstart/pxe 等。

12.2 Kickstart 企业实战配置

基于 YUM 安装 DHCP、TFTP、HTTPD 服务，指令如下：

```
yum  install  httpd httpd-devel  tftp-server  xinetd  dhcp*  -y
```

配置 tftp 服务，开启 tftp 服务；

```
cat>/etc/xinetd.d/tftp<<EOF
service tftp
```

```
{
disable = no
socket_type = dgram
protocol = udp
wait = yes
user = root
server = /usr/sbin/in.tftpd
server_args = -u nobody -s /tftpboot
per_source = 11
cps = 100 2
flags = IPv4
}
EOF
```

只需要把 disable = yes 改成 disable = no 即可，基于 sed 命令也可以实现：

```
sed  -i  '/disable/s/yes/no/g'  /etc/xinetd.d/tftp
```

12.3 TFTP+PXE 配置

要实现远程安装系统，需要在 TFTPBOOT 目录指定相关 PXE 内核模块及相关参数，配置步骤如下：

#挂载本地光盘

mount /dev/cdrom /mnt

#安装 syslinux 必备文件

yum install syslinux syslinux-devel -y

#软链接至/根系统下；

```
ln -s /var/lib/tftpboot /
mkdir -p /var/lib/tftpboot/pxelinux.cfg/
\cp /mnt/isolinux/isolinux.cfg /var/lib/tftpboot/pxelinux.cfg/default
\cp /usr/share/syslinux/vesamenu.c32 /var/lib/tftpboot/
\cp /mnt/images/pxeboot/vmlinuz /var/lib/tftpboot/
\cp /mnt/images/pxeboot/initrd.img /var/lib/tftpboot/
\cp /usr/share/syslinux/pxelinux.0 /var/lib/tftpboot/
chmod 644 /var/lib/tftpboot/pxelinux.cfg/default
```

12.4 配置 TFTPBOOT 引导案例

```
cat>/tftpboot/pxelinux.cfg/default<<EOF
default vesamenu.c32
timeout 10
display boot.msg
```

```
menu clear
menu background splash.png
menu title CentOS Linux 7
label linux
  menu label ^Install CentOS Linux 7
  menu default
  kernel vmlinuz
  append  initrd=initrd.img  inst.repo=http://192.168.0.131/centos7
quiet ks=http://192.168.0.131/ks.cfg
label check
  menu label Test this ^media & install CentOS Linux 7
  kernel vmlinuz
  append                                          initrd=initrd.img
inst.stage2=hd:LABEL=CentOS\x207\x20x86_64 rd.live.check quiet
EOF
```

配置文件详解：

192.168.0.131 是 kickstart 服务器，/centos7 是 HTTPD 共享 linux 镜像的目录，即 linux 存放安装文件的路径：

ks.cfg 是 kickstart 主配置文件;

设置 timeout 10 /*超时时间为 10S */;

ksdevice=ens33 代表当我们有多块网卡的时候，要实现自动化需要设置从 ens33 安装。

TFTP 配置完毕，由于是 TFTP 是非独立服务，需要依赖 xinetd 服务来启动，启动命令为：

chkconfig tftp --level 35 on && service xinetd restart

12.5 HTTPD+KICKSTART 配置

远程系统安装，客户端需要下载系统所需的软件包，所以需要使用 NFS 或者 httpd 把镜像文件共享出来。

mkdir -p /var/www/html/centos7/

mount /dev/cdrom /var/www/html/centos7/

#cp /dev/cdrom/* /var/www/html/centos7/ （可选配置）

配置 kickstart，可以使用 system-kickstart 系统软件包来配置，ks.cfg 配置文件内容如下：

```
cat>/var/www/html/ks.cfg<<EOF
install
text
keyboard 'us'
```

```
rootpw www.jfedu.net

timezone Asia/Shanghai

url --url=http://192.168.0.131/centos7

reboot

lang zh_CN

firewall --disabled

network   --bootproto=dhcp --device=ens33

auth   --useshadow   --passalgo=sha512

firstboot --disable

selinux     disabled

bootloader --location=mbr

clearpart --all --initlabel

part /boot --fstype="ext4" --size=300

part / --fstype="ext4" --grow

part swap --fstype="swap" --size=512

%packages

@base

@core

%end
```

```
EOF
```

12.6 DHCP 服务配置演练

DHCP 服务配置文件代码如下：

```
cat>/etc/dhcp/dhcpd.conf<<EOF
ddns-update-style interim;
ignore client-updates;
next-server 192.168.0.131;
filename "pxelinux.0";
allow booting;
allow bootp;
subnet 192.168.0.0 netmask 255.255.255.0 {
#default gateway
option routers                  192.168.0.1;
option subnet-mask          255.255.255.0;
range dynamic-bootp 192.168.0.180 192.168.0.200;
host ns {
hardware ethernet    00:1a:a0:2b:38:81;
fixed-address 192.168.0.101;}
```

```
}

EOF
```

重启各个服务，启动新的客户端验证测试：

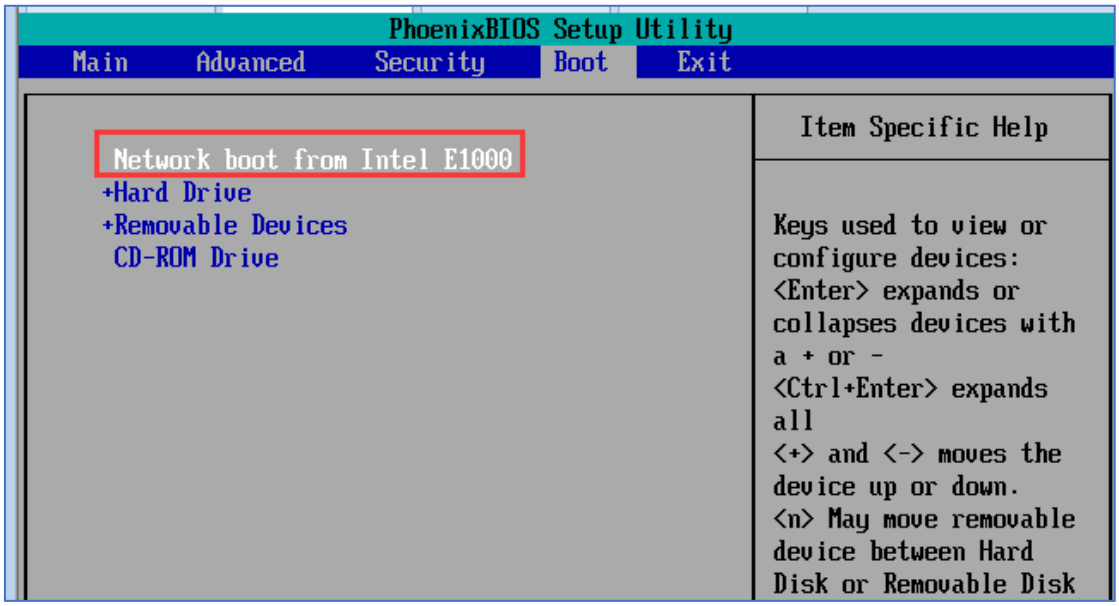
service httpd restart

service dhcpd restart

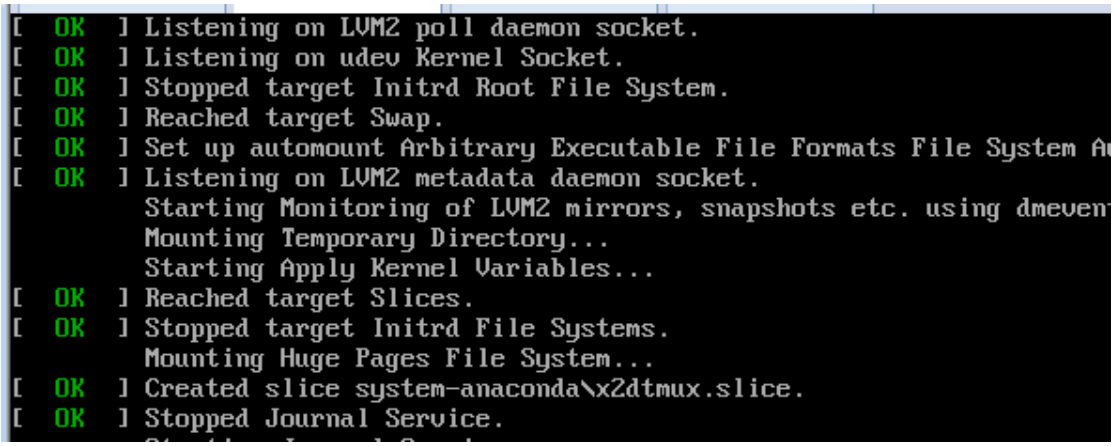
service xinetd restart

```
[root@Jfedu-Server-131 ~]#
[root@Jfedu-Server-131 ~]# service httpd restart
service dhcpd restart
service xinetd restartRedirecting to /bin/systemctl restart  httpd.service
[root@Jfedu-Server-131 ~]# service dhcpd restart
Redirecting to /bin/systemctl restart  dhcpd.service
[root@Jfedu-Server-131 ~]# service xinetd restart
Redirecting to /bin/systemctl restart  xinetd.service
[root@Jfedu-Server-131 ~]#
[root@Jfedu-Server-131 ~]# ps -ef|grep -E "httpd|dhcpd|xinetd"
root      16583     1  0 18:08 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    16586 16583  0 18:08 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    16587 16583  0 18:08 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    16588 16583  0 18:08 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    16589 16583  0 18:08 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
apache    16590 16583  0 18:08 ?        00:00:00 /usr/sbin/httpd -DFOREGROUND
dhcpd     16609     1  0 18:08 ?        00:00:00 /usr/sbin/dhcpd -f -cf /etc/d
 dhcpd -group dhcpd --no-pid
root      16629     1  0 18:08 ?        00:00:00 /usr/sbin/xinetd -stayalive -
etd.pid
root      16631 15348  0 18:09 pts/0    00:00:00 grep --color=auto -E httpd|dh
[root@Jfedu-Server-131 ~]#
```

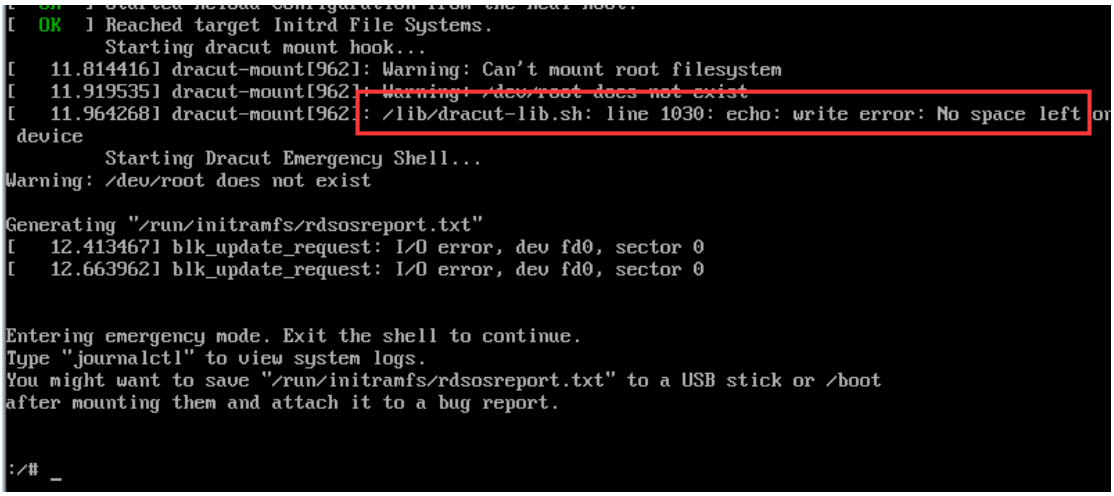
12.7 开启新虚拟机，BIOS 以网卡启动



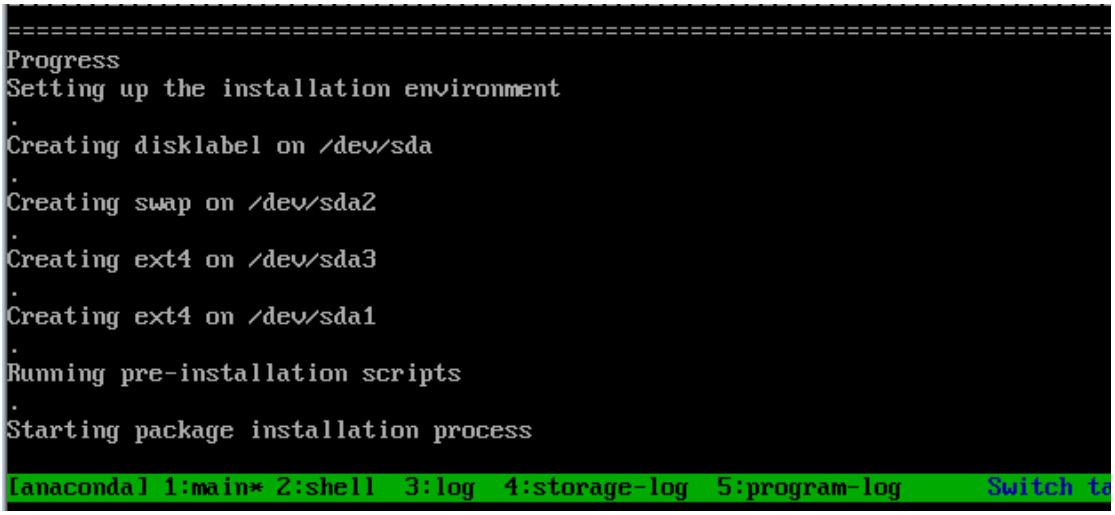




如果安装时报错如下：



需要调整客户端虚拟机的内存设置为 2G+；



12.8 Kickstart 企业生产环境扩展

在真实环境中，通常我们会发现一台服务器好几块硬盘，做完 raid，整个硬盘有等 10T，如果来使用 kickstart 自动安装并分区呢；一般服务器硬盘超过 2T，如何来使用 kickstart 安装配置呢？这里就不能使用 MBR 方式来分区，需要采用 GPT 格式来引导并分区。需要在 ks.cfg 末尾添加如下命令来实现需求：

```
%pre
parted -s /dev/sdb mklabel gpt
%end
```

为了实现 kickstart 安装完系统后，自动初始化系统等等工作，我们可以在系统安装完后，自动执行定制的脚本，需要在 ks.cfg 末尾加入如下配置：

```
%post
```

```
mount   -t   nfs 192.168.0.79:/centos/init     /mnt
cd   /mnt/ ;/bin/sh   auto_init.sh
%end
```

KICKSTART 所有配置就此告一段落，真实环境需要注意，新服务器跟 kickstart 最后独立在一个网络，不要跟办公环境或者服务器机房网络混在一起，如果别的机器以网卡就会把它的系统重装成 Linux 系统。

第13章 Zabbix 分布式监控企业实战

企业服务器对用户提供服务，作为运维工程师最重要的事情就是保证该网站正常稳定的运行，需要实时监控网站、服务器的运行状态，并且有故障及时去处理。

监控网站无需人工时刻去访问 WEB 网站或者登陆服务器去检查，可以借助开源监控软件例如 Zabbix、Cacti、Nagios、Ganglia 等来实现对网站的 7x24 小时的监控，并且做到有故障及时报警通知 SA 解决。

本章向读者介绍企业级分布式监控 Zabbix 入门、Zabbix 监控原理、最新版本 Zabbix 安装实战、Zabbix 批量监控客户端、监控 MYSQL、WEB 关键词及微信报警等。

13.1 Zabbix 监控系统入门简介

Zabbix 是一个基于 WEB 界面的提供分布式系统监控的企业级的开源解决方案，Zabbix 能监视各种网络参数，保证服务器系统的安全稳定的运行，并提供灵活的通知机制以让 SA 快速定位并解决存在的各种问题。Zabbix 分布式监控系统的优点如下：

- 支持自动发现服务器和网络设备；
- 支持底层自动发现；
- 分布式的监控体系和集中式的 WEB 管理；
- 支持主动监控和被动监控模式；
- 服务器端支持多种操作系统：Linux, Solaris, HP-UX, AIX, FreeBSD, OpenBSD, MAC 等；
- Agent 客户端支持多种操作系统：Linux, Solaris, HP-UX, AIX, FreeBSD,Windows 等；
- 基于 SNMP、IPMI 接口方式、Agent 方式；
- 安全的用户认证及权限配置；
- 基于 WEB 的管理方法，支持自由的自定义事件和邮件、短信发送；
- 高水平的业务视图监控资源，支持日志审计，资产管理等功能；
- 支持高水平 API 二次开发、脚本监控、自 Key 定义、自动化运维整合调用。

13.2 Zabbix 监控组件及流程

Zabbix 监控组件如图 13-1 所示，主要由三大组件，分别是 Zabbix server 端、Zabbix Proxy、Agent 客户端，其中 Zabbix Server 端包括：WEB GUI、Database、Zabbix_Server。

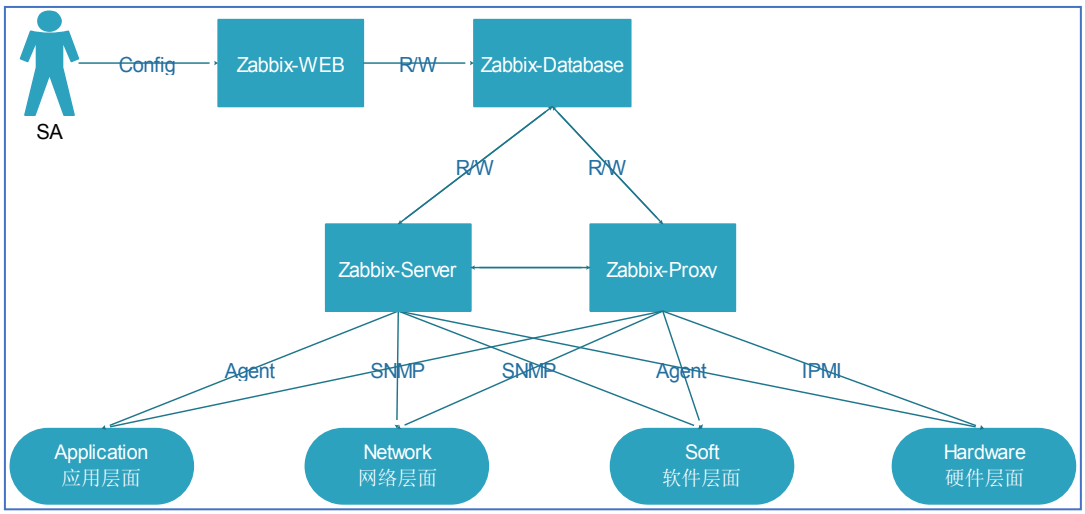


图 13-1 Zabbix 监控组件

Zabbix 监控系统具体监控系统流程如图 13-2 所示：

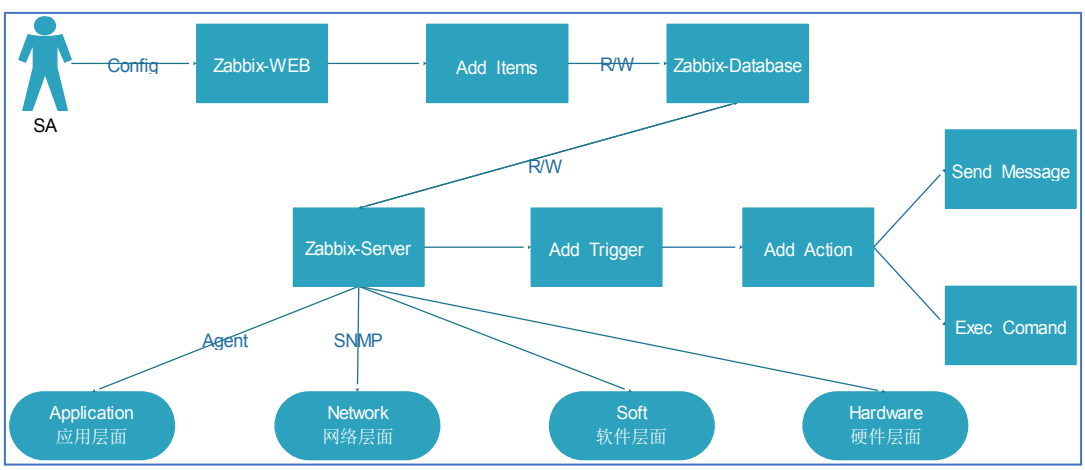


图 13-2 Zabbix 监控流程图

Zabbix 监控完整流程包括：Agentd 安装在被监控的主机上，Agent 负责定期收集客户端本地各项数据，并发送到 Zabbix Server 端，Zabbix Server 收到数据，将数据存储到数据库中，用户基于 Zabbix WEB 可以看到数据在前端展现图像。

当 Zabbix 监控某个具体的项目，该项目会设置一个触发器阀值，当被监控的指标超过该触发器设定的阀值，会进行一些必要的动作，动作包括：邮件、微信报警或者执行命令等操作。如下为 Zabbix 完整监控系统，各个部分负责的工作：

- Zabbix Server：负责接收 agent 发送的报告信息的核心组件，所有配置，统计数据及操作数据均由其组织进行；
- Database Storage：专用于存储所有配置信息，以及存储由 Zabbix 收集到的数据；
- Web interface：Zabbix 的 GUI 接口，通常与 Server 运行在同一台主机上；
- Proxy：常用于分布监控环境中，代理 Server 收集部分被监控端的监控数据并统一发往 Server 端；
- Zabbix Agent：部署在被监控主机上，负责收集本地数据并发往 Server 端或 Proxy 端；

Zabbix 监控部署在系统中，会包含常见的四个程序：zabbix_server 、zabbix_get、zabbix_agentd 、zabbix_proxy、zabbix_sender 等。四个程序启动后分别对应四个进程，如下为每个进程的功能：

- Zabbix_server：Zabbix 服务端守护进程，其中 zabbix_agentd、zabbix_get、zabbix_sender、zabbix_proxy 的数据最终均是提交给 Zabbix_Server；

- Zabbix_Agentd：客户端守护进程，负责收集客户端数据，例如收集 cpu 负载、内存、硬盘使用情况等；
- Zabbix_get：Zabbix 数据获取工具，单独使用的命令，通常在 server 或者 proxy 端执行获取远程客户端信息的命令；
- Zabbix_sender：zabbix 数据发送工具，用于发送数据给 server 或者 proxy，通常用于耗时比较长的检查。很多检查非常耗时间，导致 zabbix 超时。于是我们在脚本执行完毕之后，使用 sender 主动提交数据；
- Zabbix_proxy：zabbix 分布式代理守护进程，分布式监控架构需要部署 Zabbix_Proxy。

13.3 Zabbix 监控平台概念

Zabbix 监控系统包括很多监控概念，掌握 Zabbix 监控概念能对 Zabbix 监控快速的理解，如下为 Zabbix 常用术语及解释。

主机（host）：	被监控的网络设备，可以写 IP 或者 DNS；
主机组（host group）：	主机组用于管理主机，可以批量设置权限；
监控项（item）：	具体监控项，items 值由独立的 keys 进行识别；
触发器（trigger）：	为某个 items 设置触发器，达到触发器会执行 action 动作；
事件（event）：	例如达到某个触发器，称之为一个事件；
动作（action）：	对于特定事件事先定义的处理方法，默认可以发送信息及发送命令；

报警升级（escalation）：发送警报或执行远程命令的自定义方案，如隔 5 分钟发送一次警报，共发送 5 次等。

媒介（media）：发送通知的方式，可以支持 Mail、SMS、Scripts 等；

通知（notification）：通过设置的媒介向用户发送的有关某事件的信息；

远程命令 达到触发器，可以在被监控端执行命令；

模板（template）：可以快速监控被监控端，模块包含：item、trigger、graph、screen、application；

web 场景（web scennario）用于检测 web 站点可用性，监控 HTTP 关键词；

web 前端（frontend）：Zabbix 的 web 接口；

图形（graph） 监控图像；

屏幕（screens） 屏幕显示；

幻灯（slide show） 幻灯显示。

13.4 Zabbix 监控平台部署

（1） Zabbix Server 端和 Zabbix Agent 执行如下代码：

Zabbix 监控平台部署，至少需要安装四个组件，分别是 Zabbix_Server、Zabbix_Web、Databases、Zabbix_Agent，如下为 Zabbix 监控平台安装配置详细步骤：

（2） 系统环境

Server 端：192.168.149.128

Agent 端：192.168.149.129

（3） 下载 zabbix 版本，各个版本之间安装方法相差不大，可以根据实际情况选择安装版本，本文版本为 Zabbix-3.2.6.tar.gz。

```
wget https://jaist.dl.sourceforge.net/project/zabbix/ZABBIX%20Latest%20Stable/3.2.6/zabbix-3.2.6.tar.gz
yum -y install curl curl-devel net-snmp net-snmp-devel perl-DBI
groupadd zabbix
useradd -g zabbix zabbix
usermod -s /sbin/nologin zabbix
```

（4） Zabbix Server 端配置

创建 zabbix 数据库，执行授权命令：

```
create database zabbix charset=utf8；
grant all on zabbix.* to zabbix@localhost identified by '123456'；
flush privileges；
```

解压 zabbix 软件包并将 Zabbix 基础 SQL 文件导入数据至 Zabbix 数据库：

```
tar zxvf zabbix-3.2.6.tar.gz
cd zabbix-3.2.6
mysql -uzabbix -p123456 zabbix <database/mysql/schema.sql
```

```
mysql -uzabbix -p123456 zabbix <database/mysql/images.sql

mysql -uzabbix -p123456 zabbix < database/mysql/data.sql
```

切换至 Zabbix 解压目录，执行如下代码，安装 Zabbix_server：

```
./configure --prefix=/usr/local/zabbix/ --enable-server --with-mysql --enable-

ipv6 --with-net-snmp  --with-libcurl

make

make install

ln -s /usr/local/zabbix/sbin/zabbix_*  /usr/local/sbin/
```

Zabbix server 安装完毕，cd /usr/local/zabbix/etc/目录，如图 13-3 所示：

```
[root@localhost etc]# ls
zabbix_agent.conf    zabbix_agentd.conf    zabbix_server.conf
zabbix_agent.conf.d  zabbix_agentd.conf.d  zabbix_server.conf.d
[root@localhost etc]# ll
total 24
-rw-r--r-- 1 root root 1601 May 19 21:52 zabbix_agent.conf
drwxr-xr-x 2 root root 4096 May 19 21:52 zabbix_agent.conf.d
-rw-r--r-- 1 root root  111 May 20 23:55 zabbix_agentd.conf
drwxr-xr-x 2 root root 4096 May 19 21:52 zabbix_agentd.conf.d
-rw-r--r-- 1 root root   94 May 20 23:55 zabbix_server.conf
drwxr-xr-x 2 root root 4096 May 19 21:53 zabbix_server.conf.d
[root@localhost etc]# pwd
/usr/local/zabbix/etc
[root@localhost etc]#
```

图 13-3 Zabbix 监控流程图

备份 Zabbix server 配置文件，代码如下：

```
cp  zabbix_server.conf  zabbix_server.conf.bak
```

将 zabbix_server.conf 配置文件中代码设置为如下：

```
LogFile=/tmp/zabbix_server.log
```

```
DBHost=localhost

DBName=zabbix

DBUser=zabbix

DBPassword=123456
```

同时 cp zabbix_server 启动脚本至/etc/init.d/目录，启动 zabbix_server,

Zabbix_server 默认监听端口为 10051。

```
cd  zabbix-3.2.6

cp  misc/init.d/tru64/zabbix_server  /etc/init.d/zabbix_server

chmod  o+x  /etc/init.d/zabbix_server
```

配置 Zabbix interface Web 页面，安装 HTTP WEB 服务器，将 Zabbix WEB 代码发布至 Apache 默认发布目录，由于 Zabbix3.2+ PHP 版本需要使用 PHP5.4.0 版本，请将本机 PHP 版本升级至 5.4.0+，PHP5.3 升级至 PHP5.6，代码如下：

```
rpm -Uvh http://repo.webtatic.com/yum/el6/latest.rpm

yum remove php*

yum  install  php56w.x86_64  php56w-cli.x86_64  php56w-common.x86_64

php56w-gd.x86_64  php56w-ldap.x86_64  php56w-mbstring.x86_64  php56w-

mcrypt.x86_64 php56w-mysql.x86_64 php56w-pdo.x86_64  -y

yum  install  httpd  httpd-devel  httpd-tools  -y

cp -a  /root/zabbix-3.2.6/frontends/php/*  /var/www/html/
```

```
sed      -i   '/date.timezone/i date.timezone = PRC'   /etc/php.ini
```

重新启动 Zabbix Server、HTTP、MYSQL 服务，代码如下：

```
/etc/init.d/zabbix_server    restart

/etc/init.d/httpd       restart

/etc/init.d/mysqld      restart
```

（5） Zabbix WEB GUI 安装配置

通过浏览器 Zabbix_WEB 验证 ,通过浏览器访问 http://192.168.149.128/ 如图 13-4 所示：

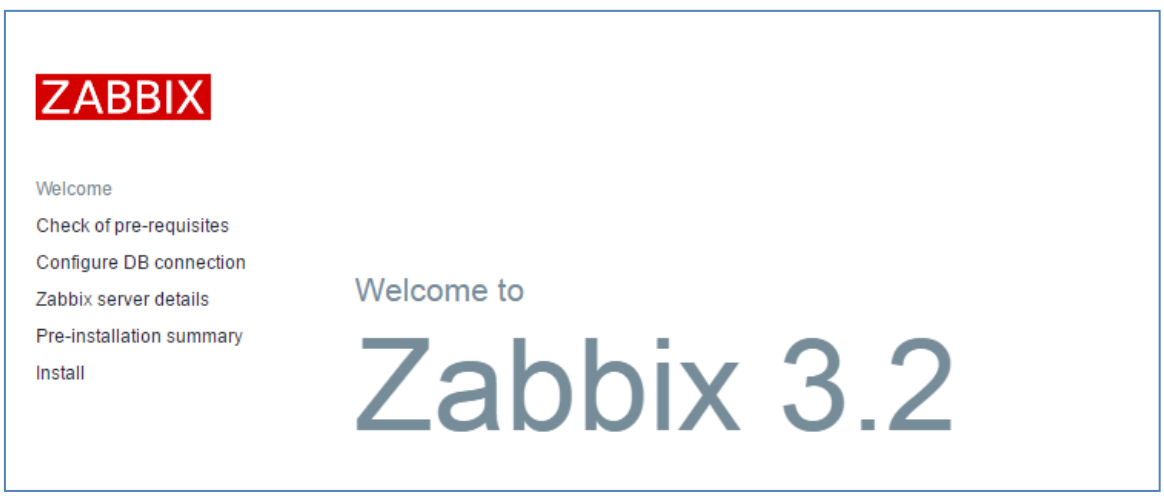


图 13-4 Zabbix WEB 安装界面

单击下一步，出现如图 13-5 所示，如果有错误提示，需要把错误依赖解决完，方可进行下一步操作。

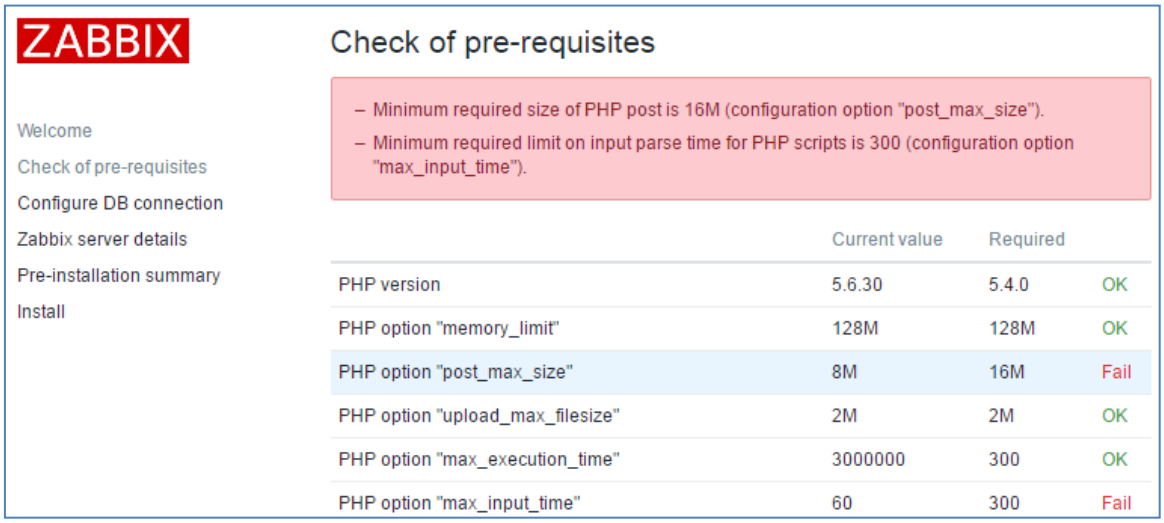


图 13-5 Zabbix WEB 安装错误提示

如上异常错误解决方法代码如下，安装缺失的软包，并修改 php.ini 对应参数的值即可，如图 13-6 所示：

```
yum install php-mbstring php-bcmath php-gd php-xml -y
yum install gd gd-devel -y
sed -i '/post_max_size/s/8/16/g;/max_execution_time/s/30/300/g;/max_input_time/s/60/300/g;s/\;date.timezone.*/date.timezone \= PRC/g;s/\;always_populate_raw_post_data/always_populate_raw_post_data/g' /etc/php.ini
service httpd restart
```

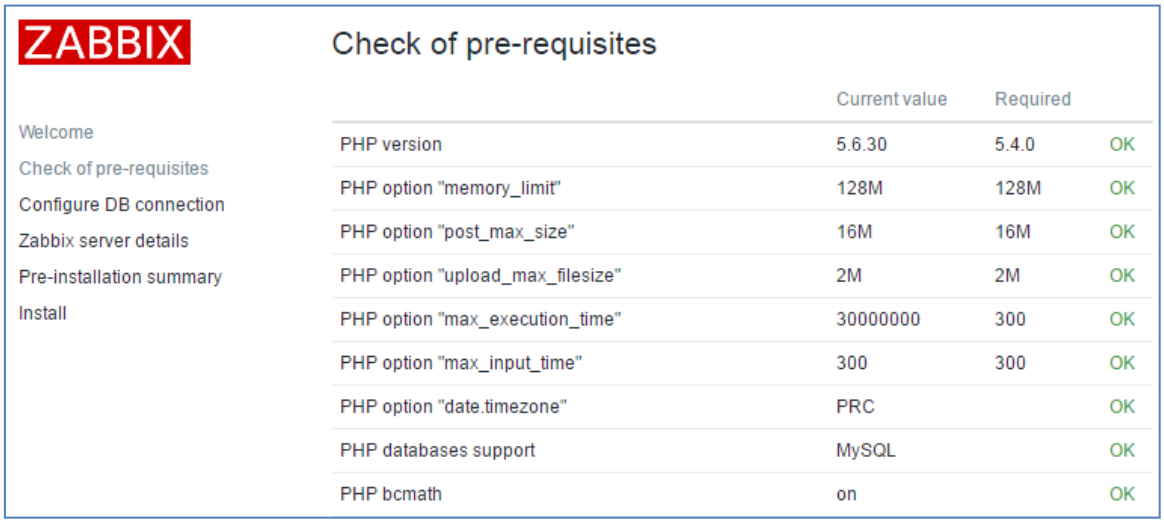


图 13-6 Zabbix WEB 测试安装环境

单击下一步，如图 13-7 所示，配置数据库连接，输入数据库名、用户、密码，单击 Test connection，显示 OK，单击下一步即可。

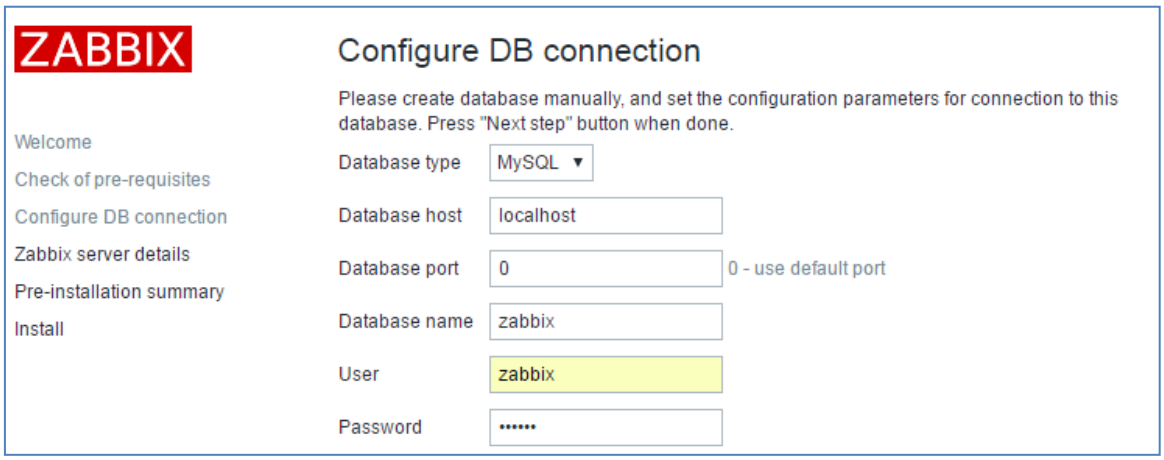


图 13-7 Zabbix WEB 数据库配置

继续单击下一步出现如图 13-8 所示，填写 Zabbix Title 显示，可以为空，可以输入自定义的名称。



图 13-8 Zabbix WEB 详细信息

单击下一步，如图 13-9 所示，需修创建 zabbix.conf.php 文件，执行如下命令，或者单击“Download the configuration file”下载 zabbix.conf.php 文件，并将该文件上传至/var/www/html/conf/，并设置可写权限，刷新 WEB 页面，zabbix.conf.php 内容代码如下，最后单击 Finish 即可：

```
<?php
// Zabbix GUI configuration file.
global $DB；
$DB['TYPE']        = 'MYSQL'；
$DB['SERVER']      = 'localhost'；
$DB['PORT']        = '0'；
$DB['DATABASE'] = 'zabbix'；
$DB['USER']        = 'zabbix'；
$DB['PASSWORD'] = '123456'；
```

```
// Schema name. Used for IBM DB2 and PostgreSQL.

$DB['SCHEMA'] = '' ;

$ZBX_SERVER          = 'localhost' ;

$ZBX_SERVER_PORT = '10051' ;

$ZBX_SERVER_NAME = '京峰教育-分布式监控系统' ;

$IMAGE_FORMAT_DEFAULT = IMAGE_FORMAT_PNG ;
```

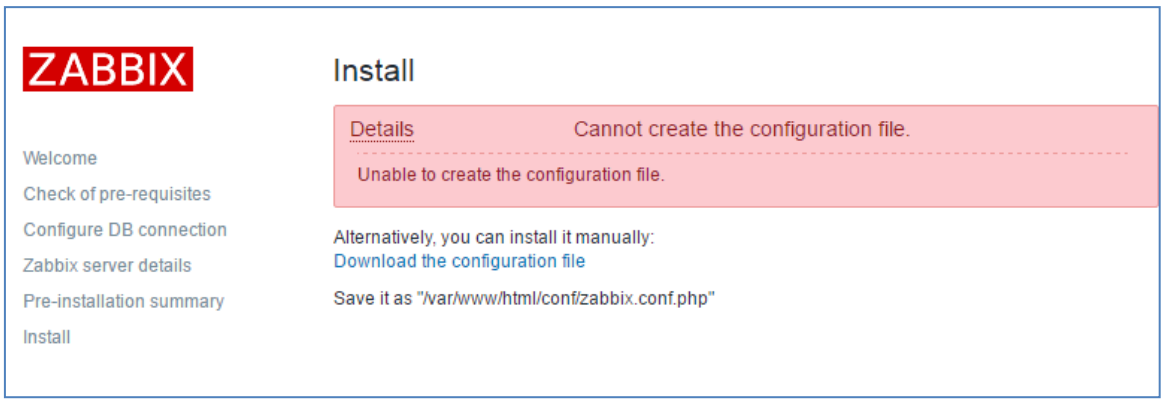


图 13-9 Zabbix WEB 配置文件测试

登录 Zabbix WEB 界面，默认用户名和密码为：admin/zabbix，如图 13-10（a）、13-10（b）所示：



图 13-10（a） Zabbix WEB 登录界面

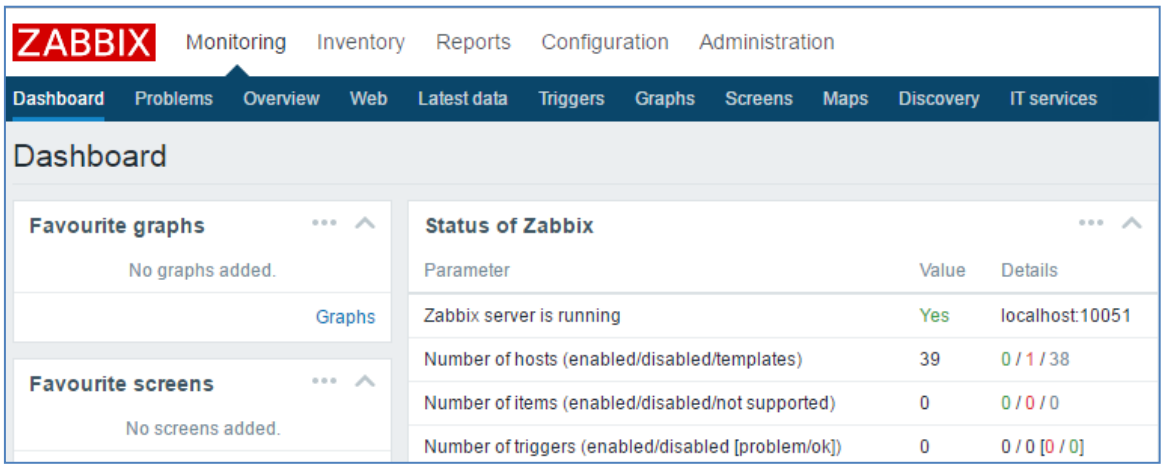


图 13-10（b） Zabbix WEB 后台界面

（6） Agent 客户端安装配置

解压 zabbix-3.2.6.tar.gz 源码文件，切换至解压目录，编译安装 Zabbix，命令如下：

```
./configure  --prefix=/usr/local/zabbix   --enable-agent

make
```

```
make install

ln  -s  /usr/local/zabbix/sbin/zabbix_*  /usr/local/sbin/
```

修改 zabbix_agentd.conf 客户端配置文件，执行如下命令，zabbix_agentd.conf 内容，指定 server IP，同时设置本地 Hostname 为本地 IP 地址或者 DNS 名称：

CPU、内存、负载、网卡、磁盘、IO、应用服务、端口、登录用户、

```
LogFile=/tmp/zabbix_agentd.log

Server=192.168.149.128

ServerActive=192.168.149.128

Hostname = 192.168.149.129
```

同时 cp zabbix_agentd 启动脚本至/etc/init.d/目录，启动 zabbix_agentd 服务即可， Zabbix_agentd 默认监听端口为 10050。

```
cd zabbix-3.2.6

cp misc/init.d/tru64/zabbix_agentd /etc/init.d/zabbix_agentd

chmod o+x /etc/init.d/zabbix_agentd

/etc/init.d/zabbix_agentd  start
```

（7） Zabbix 监控客户端

Zabbix 服务端和客户端安装完毕之后，需通过 Zabbix Server 添加客户端监控，Zabbix WEB 界面添加客户端监控的操作步骤如下，如图 13-11 所示：

```
Zabbix-WEB →configuration →hosts →Create host →Host name 和 Agent
```

interfaces，同时选择添加 templates 模板→选择 Add →勾选 Template OS Linux-选择 Add 提交；

注*此处 Host name 名称与 Agentd.conf 配置文件中 Hostname 保持一致，否则会报错。

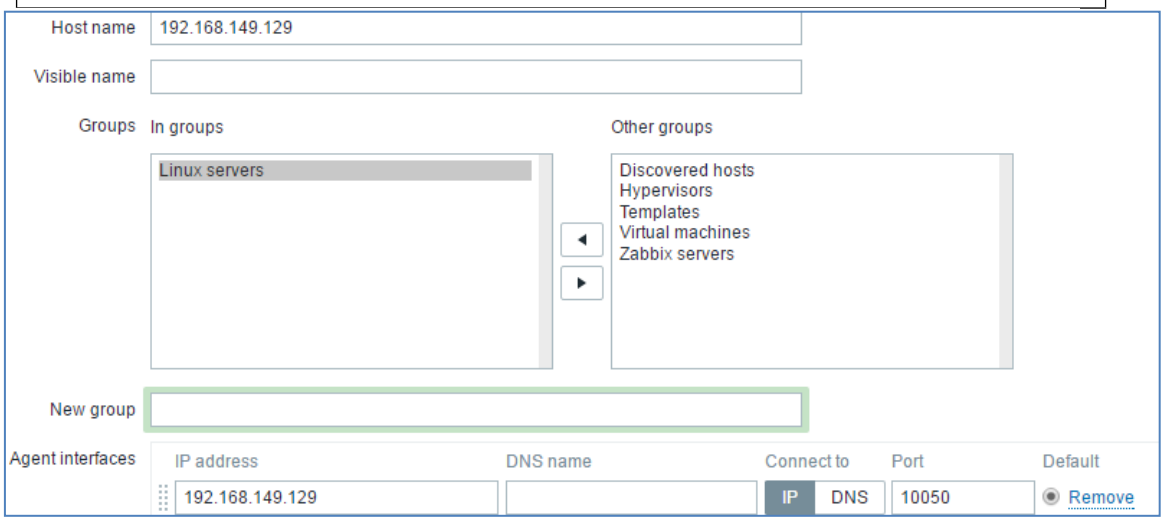


图 13-11 Zabbix 添加客户端监控

将客户端主机链接至“Template OS Linux”，启用模板完成主机默认监控，单击 Add

，继续单击 Update 即可，如图 13-12 所示：

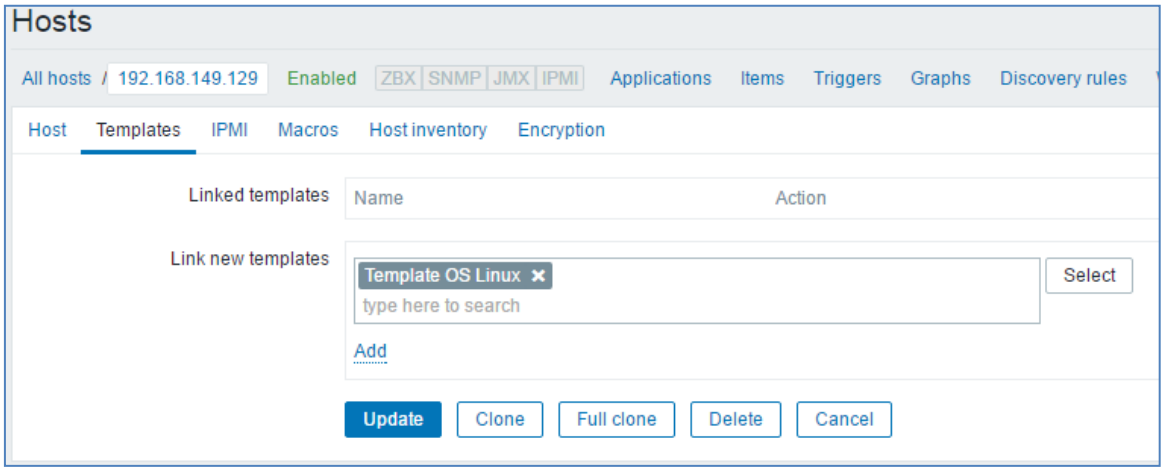


图 13-12 Zabbix 为客户端监控添加模板

单击 Zabbix WEB→Monitoring→Graphs→Group→Host→Graph，监控图像如图 13-13（a）、13-13（b）所示：

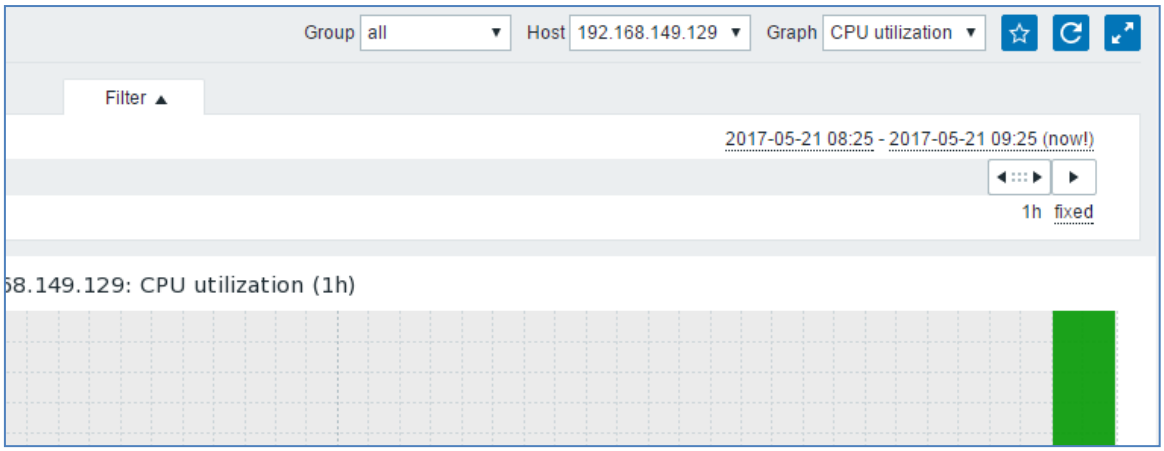


图 13-13（a） Zabbix 客户端监控图像

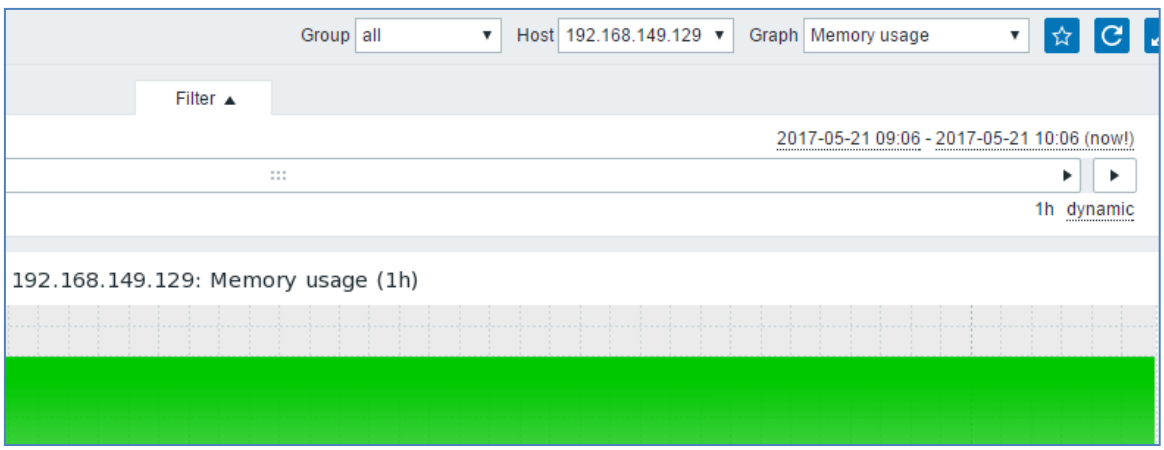


图 13-13（b） Zabbix 客户端监控图像

如果无法监控到客户端，可以在 Zabbix Server 端，执行命令获取 Agent 的 items KEY 值是否有返回，例如 system.uname 为返回客户端的 uname 信息，监测命令如下：

```
/usr/local/zabbix/bin/zabbix_get  -s  192.168.149.130  -k  system.uname
```

13.5 Zabbix 配置文件优化实战

Zabbix 监控系统组件分为 Server、Proxy、Agentd 端，对参数的详细了解，能够更加深入理解 Zabbix 监控功能，及对 Zabbix 进行调优，如下为三个组件常用参数详解：

（1） Zabbix_server.conf 配置文件参数详解：

DBHost	数据库主机地址；
DBName	数据库名称；
DBPassword	数据库密码；
DBPort	数据库端口，默认为 3306；
AlertScriptsPath	告警脚本存放路径；
CacheSize	存储监控数据的缓存；
CacheUpdateFrequency	更新一次缓存时间；
DebugLevel	日志级别；
LogFile	日志文件；
LogFileSize	日志文件大小，超过自动切割；
LogSlowQueries	数据库慢查询记录，单位毫秒；
PidFile	PID 文件；
ProxyConfigFrequency	Proxy 被动模式下，Server 多少秒同步配置文件至

proxy；	
ProxyDataFrequency	被动模式下，Server 间隔多少秒向 proxy 请求历史数据；
StartDiscoverers	发现规则线程数；
Timeout	连接 Agent 超时时间；
TrendCacheSize	历史数据缓存大小；
User	Zabbix 运行的用户；
HistoryCacheSize	历史记录缓存大小；
ListenIP	监听本机的 IP 地址；
ListenPort	监听端口；
LoadModule	模块名称；
LoadModulePath	模块路径。

（2） Zabbix_Proxy.conf 配置文件参数详解：

ProxyMode	Proxy 工作模式，默认为主动模式，主动发送数据至 Server；
Server	指定 Server 端地址；
ServerPort	Server 端 PORT；
Hostname	Proxy 端主机名；
ListenPort	Proxy 端监听端口；

LogFile	Proxy 代理端日志路径；
PidFile	PID 文件的路径；
DBHost	Proxy 端数据库主机名；
DBName	Proxy 端数据库名称；
DBUser	Proxy 端数据库用户；
DBPassword	Proxy 端数据库密码；
DBSocket	Proxy 数据库 SOCKET 路径；
DBPort	Proxy 数据库端口号；
DataSenderFrequency	Proxy 向 Server 发送数据的时间间隔；
StartPollers	Proxy 程池数量；
StartDiscoverers	Proxy 端自动发现主机的线程数量；
CacheSize	内存缓存配置；
StartDBSyncers	同步数据线程数；
HistoryCacheSize	历史数据缓存大小；
LogSlowQueries	慢查询日志记录，单位为毫秒；
Timeout	超时时间。

（3） Zabbix_agentd.conf 配置文件参数详解：

EnableRemoteCommands	运行服务端远程至客户端执行命令或者脚本；
Hostname	客户端主机名；

ListenIP	监听的 IP 地址；
ListenPort	客户端监听端口；
LoadModulePath	模块路径；
LogFile	日志文件路径；
PidFile	PID 文件名；
Server	指定 Server IP 地址；
ServerActive	Zabbix 主动监控 server 的 ip 地址；
StartAgents	Agent 启动进程，如果设置为 0，表示禁用被动监控；
Timeout	超时时间
User	运行 Zabbix 的用户；
UserParameter	用户自定义 key；
BufferSize	缓冲区大小；
DebugLevel	Zabbix 日志级别。

13.6 Zabbix 自动发现及注册

熟练通过 Zabbix 监控平台监控单台客户端之后，企业中有成千上万台服务器，如果手工添加会非常耗时间，造成大量的人力成本的浪费，有没有什么好的自动化添加客户端的方法呢？

Zabbix 自动发现就是为了解决批量监控而设计的功能之一，什么是自动发现呢，简单来说就是 Zabbix Server 端可以基于设定的规则，自动批量的去发现局域网若干服务器，并自动把服务器添加至 Zabbix 监控平台，省去人工手动频繁的添加，节省大量的人力成本。

Zabbix 相对于 Nagios、Cacti 监控来说，如果要想批量监控，Nagios、Cacti 需要手动单个添加设备、分组、项目、图像，也可以使用脚本，但是不能实现自发方式添加。

Zabbix 最大的特点之一就是可以批量自动主机并监控，利用发现(Discovery)模块，实现自动发现主机、自动将主机添加到主机组、自动加载模板、自动创建项目（Items）、自动创建监控图像，操作步骤如下：

（1） Configuration→discovery → Create discovery rule，如图 13-14 所示：

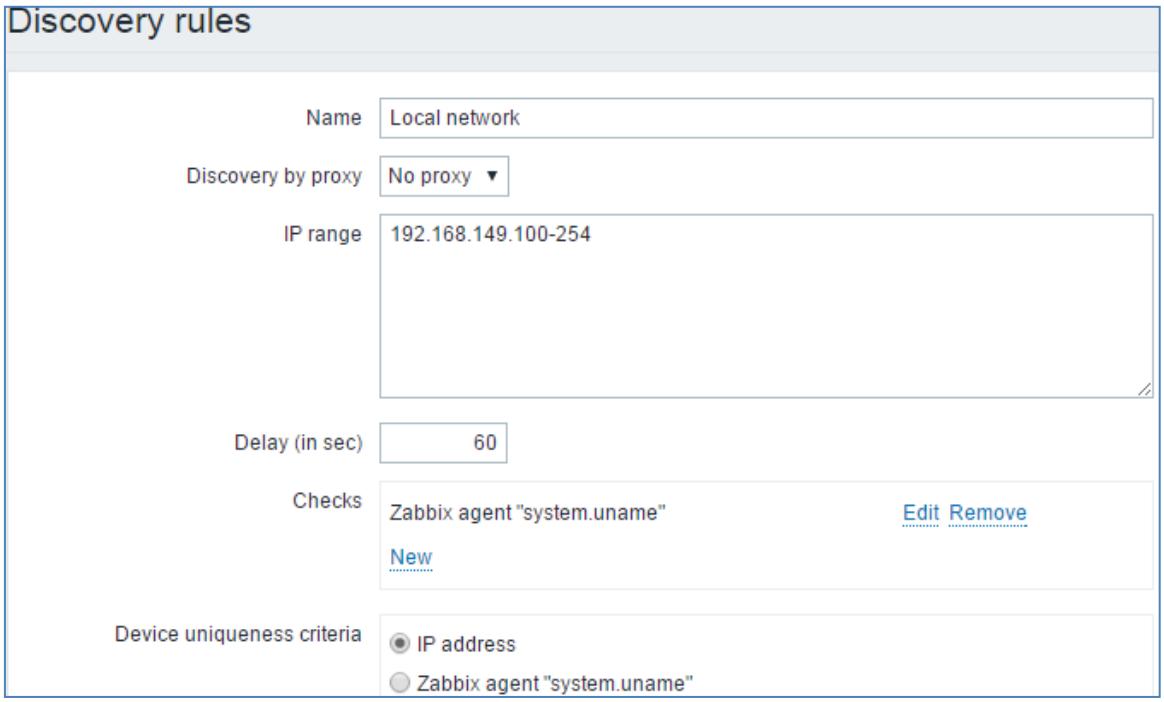


图 13-14 创建客户端发现规则

Name： 规则名称；

Discovery by proxy：通过代理探索；

IP range： zabbix_server 探索区域的 IP 范围；

Delay： 搜索一次的时间间隔；

Checks： 检测方式，如用 ping 方式去发现主机，zabbix_server 需安装 fping，此处使用 Agent 方式发现；

Device uniqueness criteria: 以 IP 地址作为被发现主机的标识。

（2） Zabbix 客户端安装 Agent

由于发现规则里选择 checks 方式为 Agent，所以需在所有被监控的服务器安装 zabbix Agent，安装的方法可以手动安装，也可以使用 Shell 脚本，附 Zabbix 客户端安装脚本，脚本运行方法：sh auto_install_zabbix.sh。

```
#!/bin/bash
#auto install zabbix
#by jfedu.net 2022
#############
ZABBIX_SOFT="zabbix-3.2.6.tar.gz"
INSTALL_DIR="/usr/local/zabbix/"
SERVER_IP="192.168.149.128"
IP=`ifconfig|grep Bcast|awk '{print $2}'|sed 's/addr://g'`
```

```
AGENT_INSTALL(){

yum -y install curl curl-devel net-snmp net-snmp-devel perl-DBI

groupadd zabbix  ; useradd -g zabbix zabbix ; usermod -s /sbin/nologin

zabbix

tar -xzf $ZABBIX_SOFT ; cd `echo $ZABBIX_SOFT|sed 's/.tar.*//g'`

./configure  --prefix=/usr/local/zabbix  --enable-agent&&make install

if [ $? -eq 0 ] ; then

    ln -s /usr/local/zabbix/sbin/zabbix_* /usr/local/sbin/

fi

cd -  ; cd zabbix-3.2.6

cp  misc/init.d/tru64/zabbix_agentd  /etc/init.d/zabbix_agentd  ; chmod o+x

/etc/init.d/zabbix_agentd

#config zabbix agentd

cat >$INSTALL_DIR/etc/zabbix_agentd.conf<<EOF

LogFile=/tmp/zabbix_agentd.log

Server=$SERVER_IP

ServerActive=$SERVER_IP

Hostname = $IP

EOF
```

```
#start zabbix agentd

/etc/init.d/zabbix_agentd restart

/etc/init.d/iptables stop

setenforce 0

}

AGENT_INSTALL
```

（3） 创建发现 Action

Zabbix 发现规则创建完毕，客户端 Agent 安装完后，被发现的 IP 主机不会自动添加至 Zabbix 监控列表，需要添加发现动作，添加方法如下：

```
Configuration→ Actions → Event source(选择 Discovery) → Create action
```

添加规则时，系统默认存在一条发现规则，可以新建规则，也可以编辑默认规则，如图 13-15（a）、13-15（b）、13-15（c）所示，编辑默认发现规则，单击 Operations 设置发现操作，分别设置 Add host、Add to host groups、Link to templates，最后启用规则即可：

老男孩 1 对 1 职业规划咨询+100G 运维入门到精通全套资料 加老师 V：Linux9090

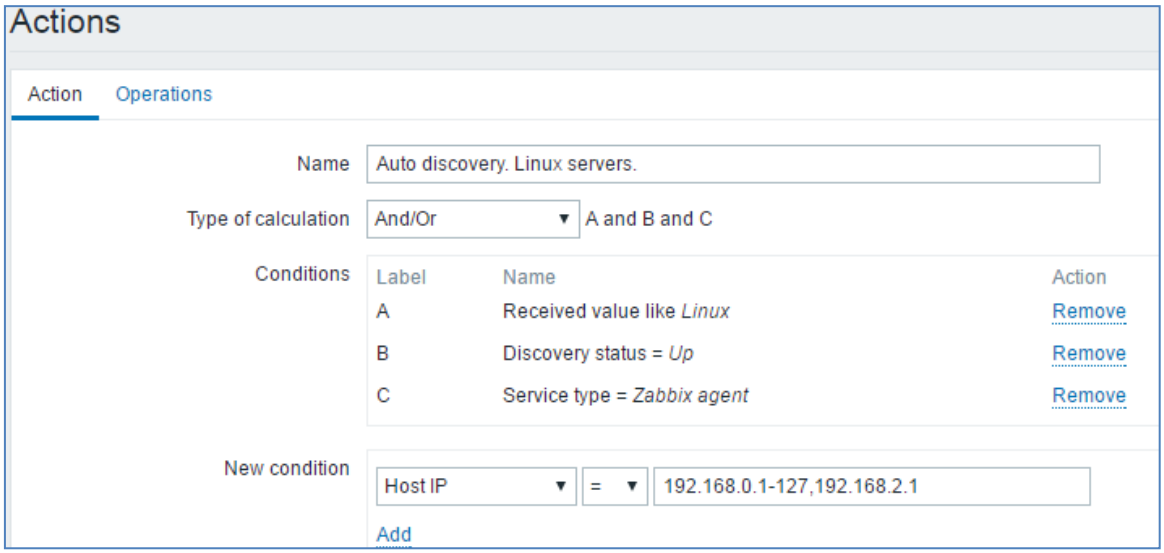


图 13-15（a） 创建客户端发现动作

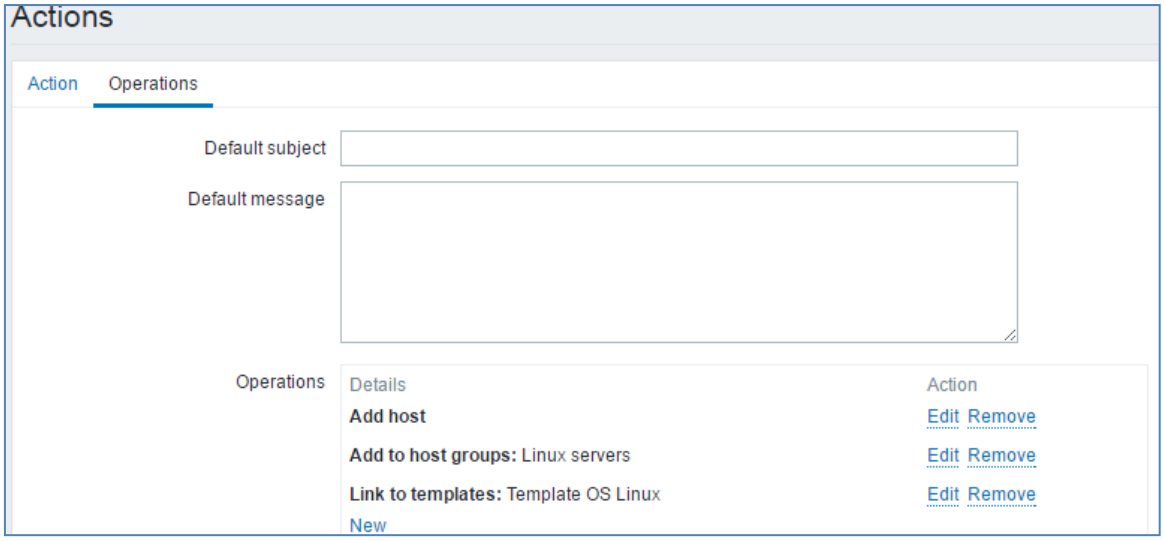


图 13-15（b） 客户端发现自动添加至 Zabbix

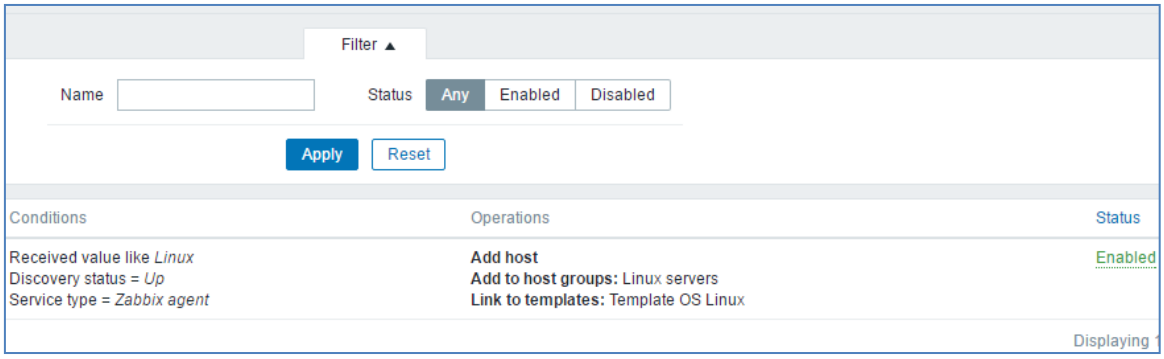


图 13-15（c） 客户端发现自动添加至 Zabbix

Montoring→Discovery，查看通过发现规则找到的服务器 IP 列表，如图 13-16 所示：

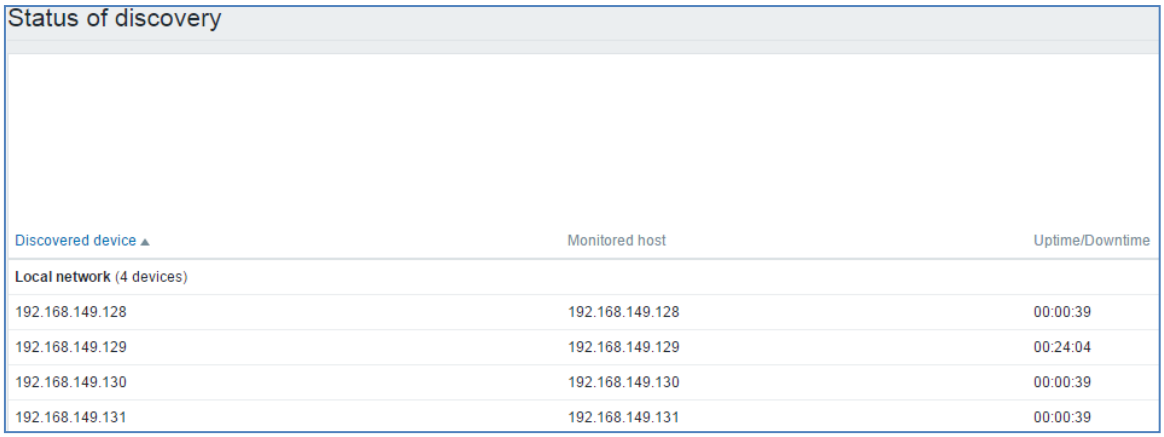


图 13-16 被发现的客户端列表

Configuration→Hosts，查看 4 台主机是否被自动监控至 Zabbix 监控平台，如图 13-17 所示：

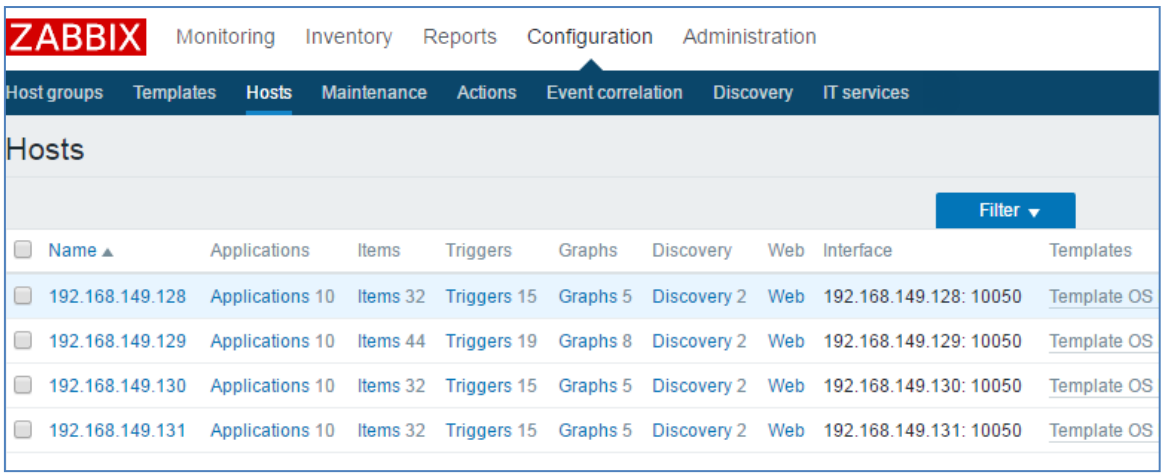


图 13-17 自动发现的主机被添加至 Hosts 列表

Monitoring→Graphs，监控图像查看，如图 13-18（a）、13-18（b）所示，可以选择 Host、Graph 分别查看各种的监控图像：

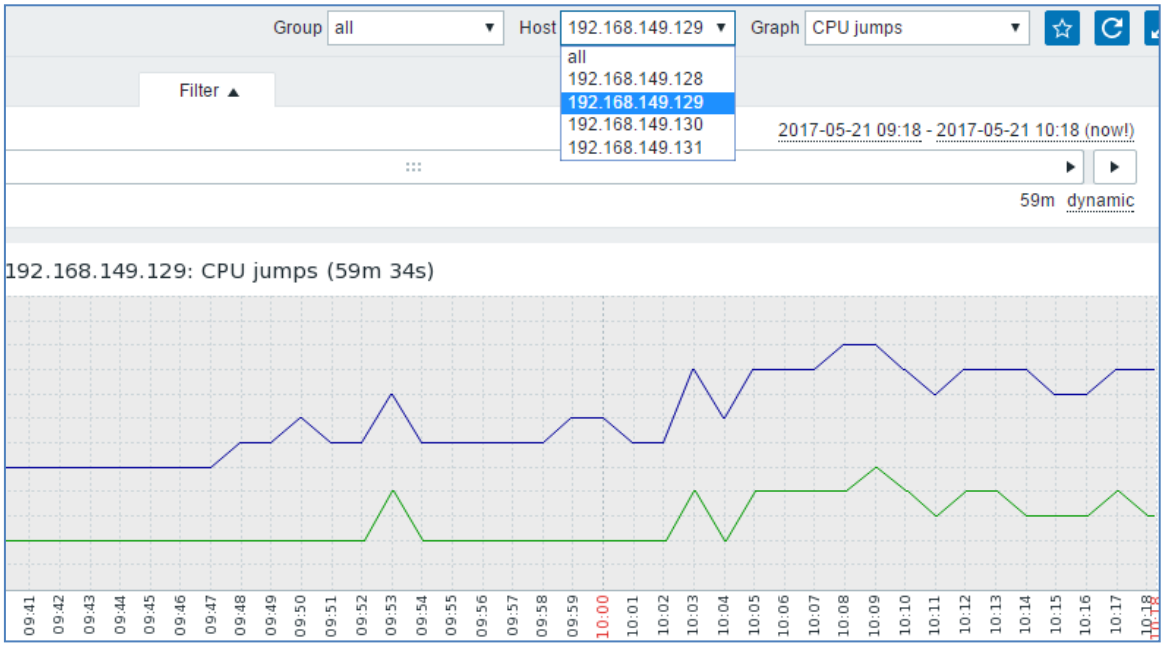


图 13-18（a） 客户端监控图像

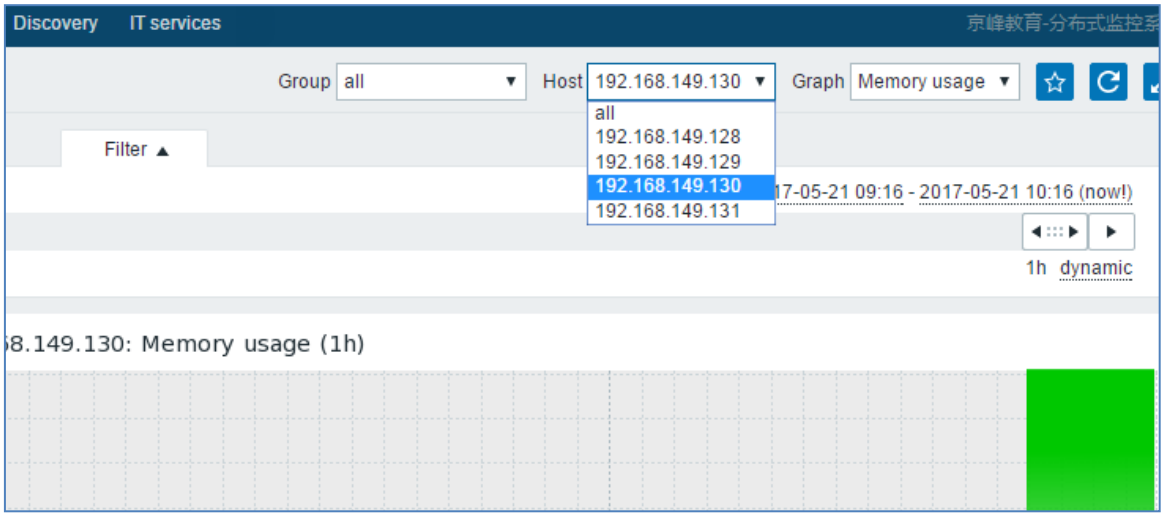


图 13-18（b） 客户端监控图像

13.7 Zabbix 监控邮件报警实战

Zabbix 监控服务端、客户端都已经部署完成，被监控主机已经添加，Zabiix 监控运行正常，通过查看 Zabbix 监控服务器，可以了解服务器的运行状态是否正常，运维人员不会时刻登录 Zabbix 监控平台刷新，查看服务器的状态。

可以在 Zabbix 服务端设置邮件报警，当被监控主机宕机或者达到设定的触发器预设值时，不管任何时候，会自动发送报警邮件、微信信息到指定的人员，运维人员收到信息有利于第一时间解决故障。Zabbix 邮件报警设置步骤如下：

（1） 设置邮件模板及邮件服务器

Administration→Media types→Create media type，填写邮件服务器信息，根据提示设置完毕，如图 13-19（a）、13-19（b）所示：

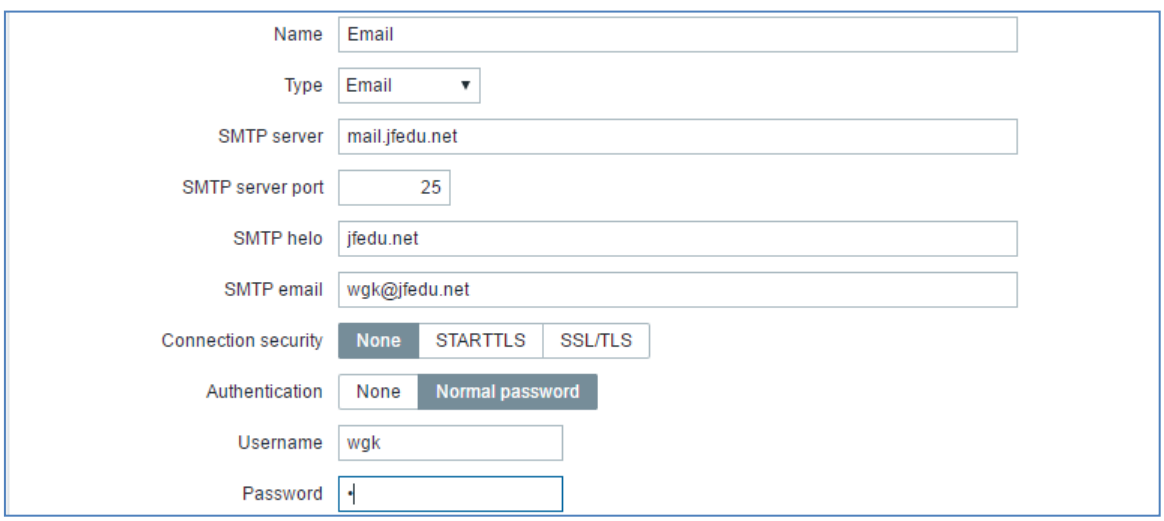


图 13-19（a） Zabbix 邮件报警邮箱设置

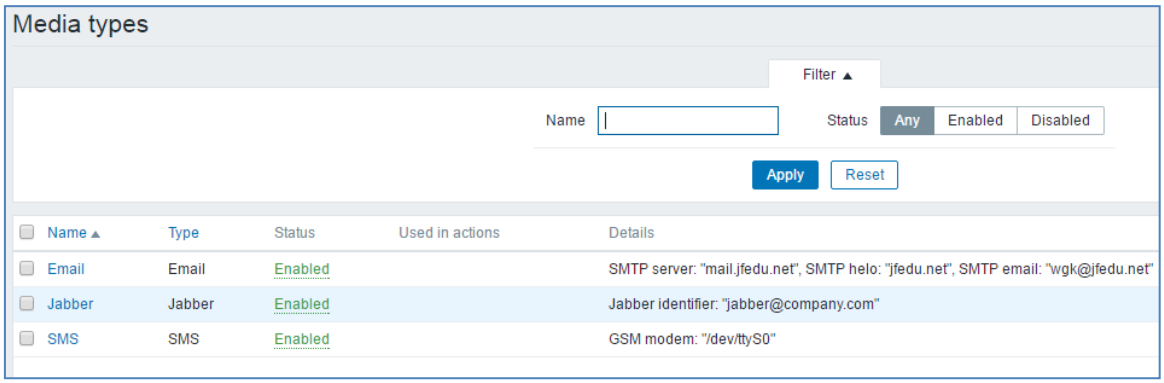


图 13-19（b） Zabbix 邮件报警邮箱设置

（2） 配置接收报警的邮箱

Administration-user-Admin (Zabbix Administrator)-user-admin，选择 Media，单击 Add 添加发送邮件的类型“Email”，同时指定接收邮箱地址：wgkgood@163.com，根据实际需求改成自己的接收人，如图 13-20 所示：

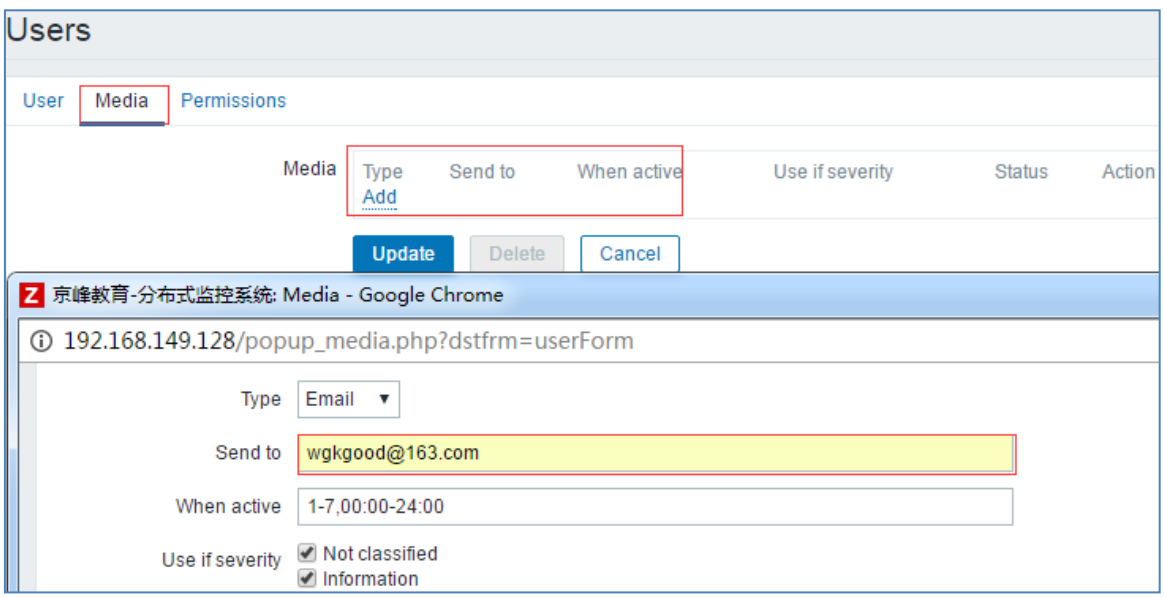


图 13-20 Zabbix 邮件报警添加接收人

（3） 添加报警触发器

Configuration→Actions→Action→ Event source→Triggers-Create Action，如图 13-21（a）、13-21（b）、13-21（c）所示，分别设置 Action、Operations、Recovery operations。

- Action→New condition 选择“Trigger serverity>=Warning”；
- Operations→设置报警间隔为 60s，自定义报警信息，报警信息发送至 administrators 组；
- Recovery operations →自定义恢复信息，恢复信息发送至 administrators 组。

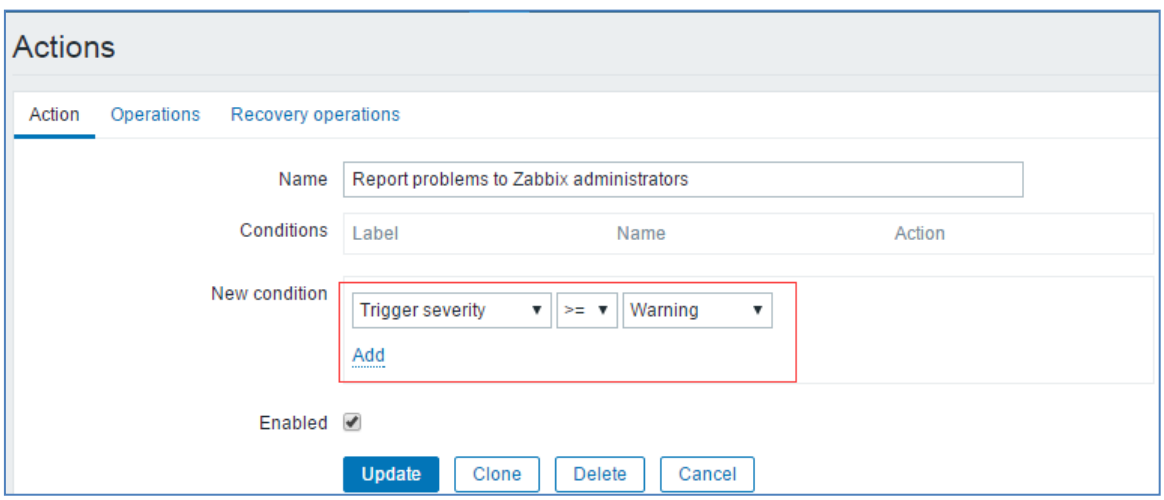


图 13-21（a） 邮件报警 Action 设置

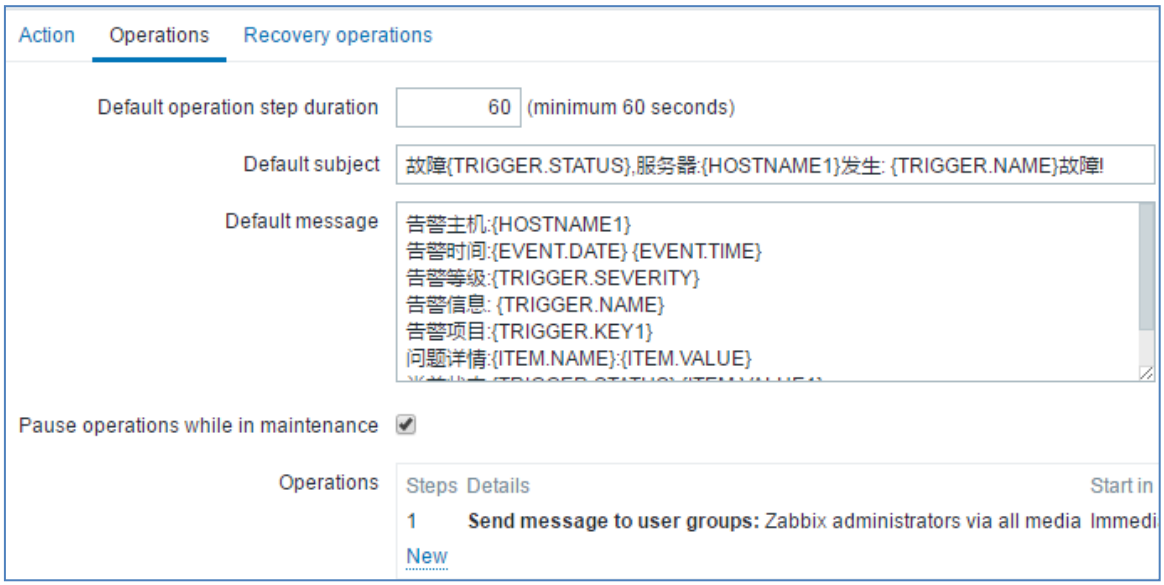


图 13-21（b） 邮件报警 Operations 设置

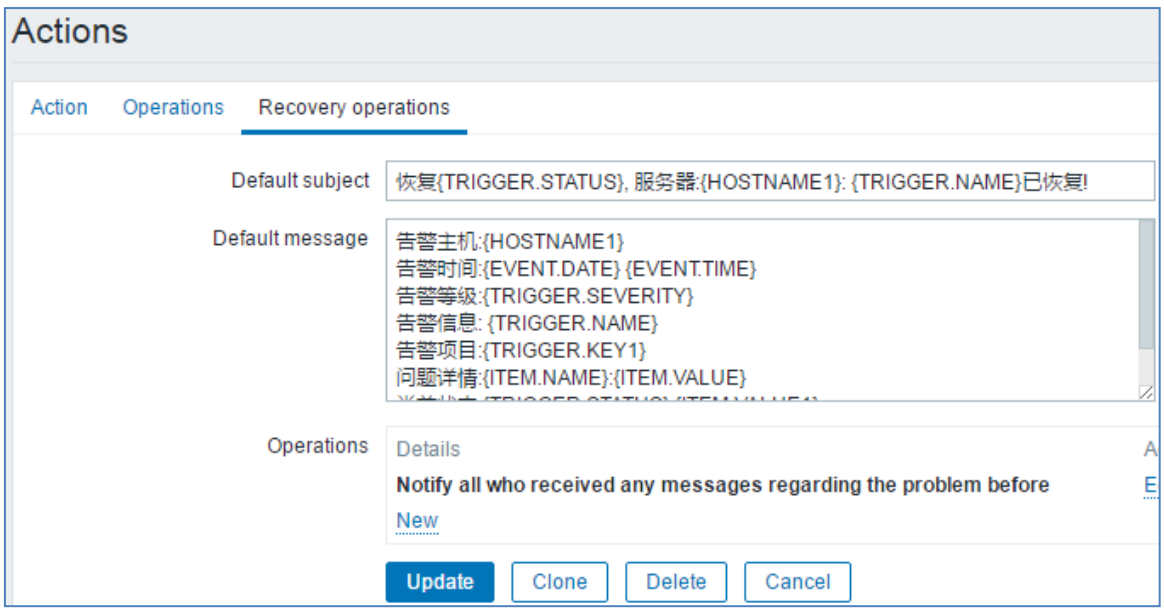


图 13-21（c） 邮件报警 Recovery Operations 设置

报警邮件标题可以使用默认信息，亦可使用如下中文报警内容：

名称：Action-Email

默认标题：故障{TRIGGER.STATUS},服务器:{HOSTNAME1}发生: {TRIGGER.NAME}

故障!

默认信息：

告警主机:{HOSTNAME1}

告警时间:{EVENT.DATE} {EVENT.TIME}

告警等级:{TRIGGER.SEVERITY}

告警信息: {TRIGGER.NAME}

告警项目:{TRIGGER.KEY1}

问题详情:{ITEM.NAME}:{ITEM.VALUE}

当前状态:{TRIGGER.STATUS}:{ITEM.VALUE1}

事件 ID:{EVENT.ID}

恢复邮件标题可以使用默认信息，亦可使用如下中文报警恢复内容：

恢复标题：恢复{TRIGGER.STATUS}，服务器:{HOSTNAME1}: {TRIGGER.NAME}已恢复!

恢复信息：

告警主机:{HOSTNAME1}

告警时间:{EVENT.DATE} {EVENT.TIME}

告警等级:{TRIGGER.SEVERITY}

告警信息: {TRIGGER.NAME}

告警项目:{TRIGGER.KEY1}

问题详情:{ITEM.NAME}:{ITEM.VALUE}

当前状态:{TRIGGER.STATUS}:{ITEM.VALUE1}

事件 ID:{EVENT.ID}

Monitoring→Problems ,检查有问题的 Action 事件 ,单击 Time 下方时间 ,如图 13-22（a）、13-22（b）所示，可以看到邮件是否执行成功或者失败：

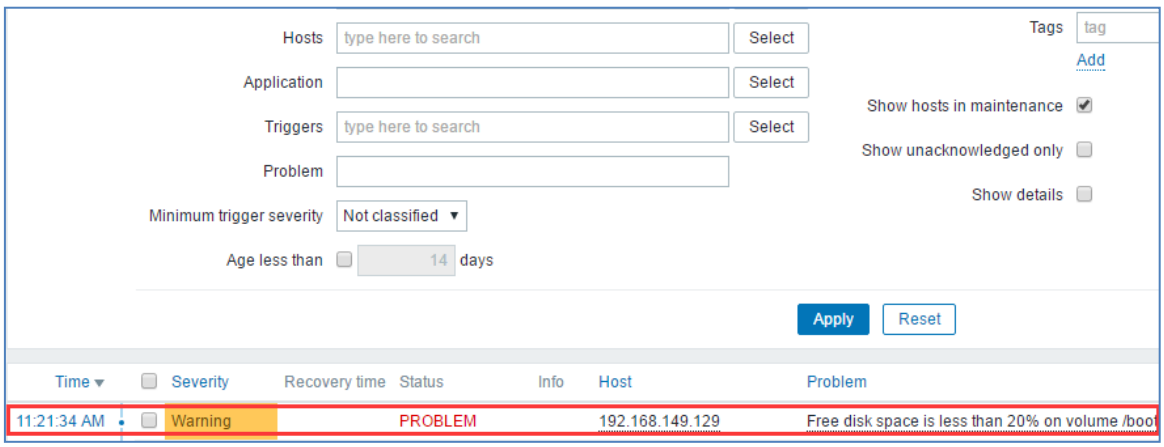


图 13-22（a） Zabbix 查看有问题的事件

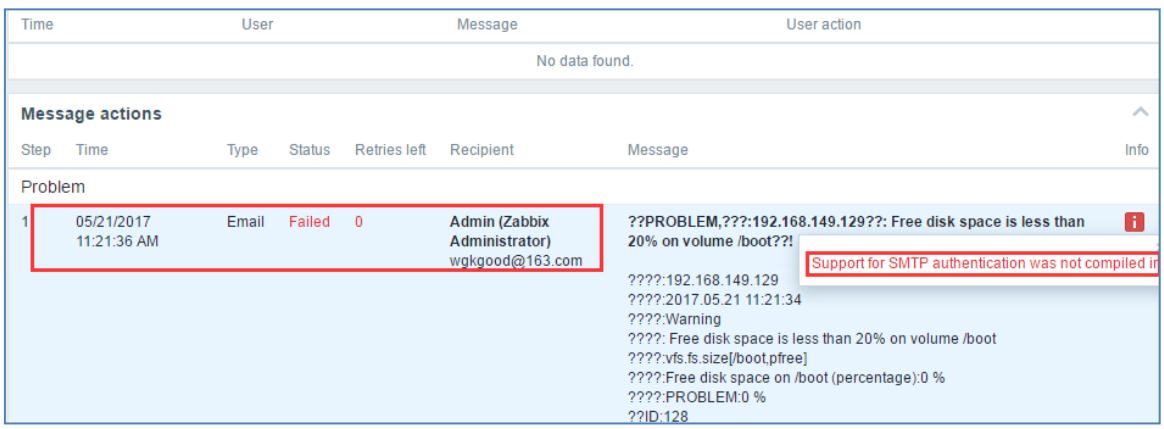


图 13-22（b） Zabbix 有问题的事件执行任务

Zabbix 邮件发送失败，报错 Support for SMTP authentication was not compiled in，原因是由于 Zabbix CURL 版本要求至少是 7.20+版本，升级 CURL，升

级方法：

创建 repo 源，vim /etc/yum.repos.d/city-fan.repo，并且写入如下的语句：

```
cat>/etc/yum.repos.d/city-fan.repo<<EOF

[CityFan] name=City Fan Repo baseurl=http://www.city-fan.org/ftp/contrib/yum-

repo/rhel5/x86_64/ enabled=1 gpgcheck=0

EOF

yum clean all

yum install curl
```

```
作为依赖被安装:
  c-ares.x86_64 0:1.12.0-2.0.cf.rhel5                com
  compat-openldap.x86_64 1:2.3.43-2.el6              lib
  libssh2-devel.x86_64 0:1.8.0-2.0.cf.rhel5

更新完毕:
  curl.x86_64 0:7.53.1-2.0.cf.rhel5

作为依赖被升级:
  libcurl.x86_64 0:7.53.1-2.0.cf.rhel5       libcurl-d
  libssh2.x86_64 0:1.8.0-2.0.cf.rhel5
```

curl -V

```
[root@localhost yum.repos.d]# cd
[root@localhost ~]#
[root@localhost ~]# curl -V
curl 7.53.1 (x86_64-redhat-linux-gnu) libcurl/7.53.
 libssh2/1.8.0
Protocols: dict file ftp ftps gopher http https ima
tp smb smbs smtp smtps telnet tftp
Features: AsynchDNS IPv6 Largefile GSS-API Kerberos
 HTTPS-proxy Metalink
[root@localhost ~]#
```

CURL 升级完毕之后，测试邮件发送，还是报同样的错误，原因是因为需要重新将 Zabbix_Server 服务通过源码编译安装一遍，安装完 zabbx_server，重启服务，乱码问题是由于数据库字符集需改成 UTF-8 格式，同时将数 Zabbix 库导出，然后修改 latin1 为 utf8，再将 SQL 导入，重启 Zabbix 即可，最终如图 13-23（a）、13-23（b）、13-23（c）所示：

Time	Type	Status	Retries left	Recipient	Message
m					
05/21/2017 12:15:03 PM	Email	Sent		Admin (Zabbix Administrator) wgkgood@163.com	**故障PROBLEM,服务器:192.168.149.130发生: Free disk spa than 20% on volume /boot故障!** 告警主机:192.168.149.130 告警时间:2017.05.21 12:15:01 告警等级:Warning 告警信息: Free disk space is less than 20% on volume /boot 告警项目:vfs.fs.size[/boot,pfree] 问题详情:Free disk space on /boot (percentage):0 % 当前状态:PROBLEM:0 % 事件ID:226

图 13-23（a） Zabbix 事情发送邮件进程

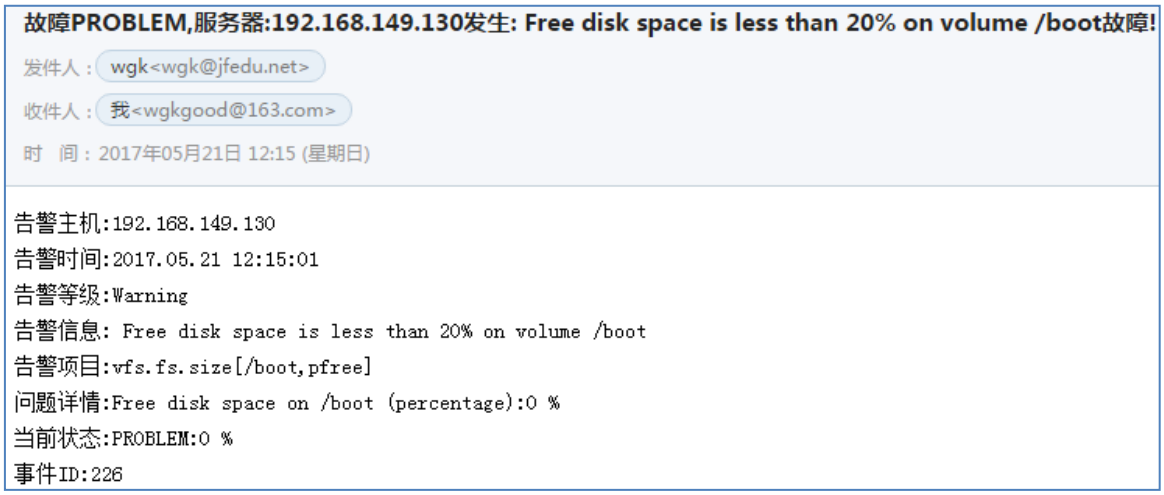


图 13-23（b） Zabbix 监控故障 item 发送报警邮件

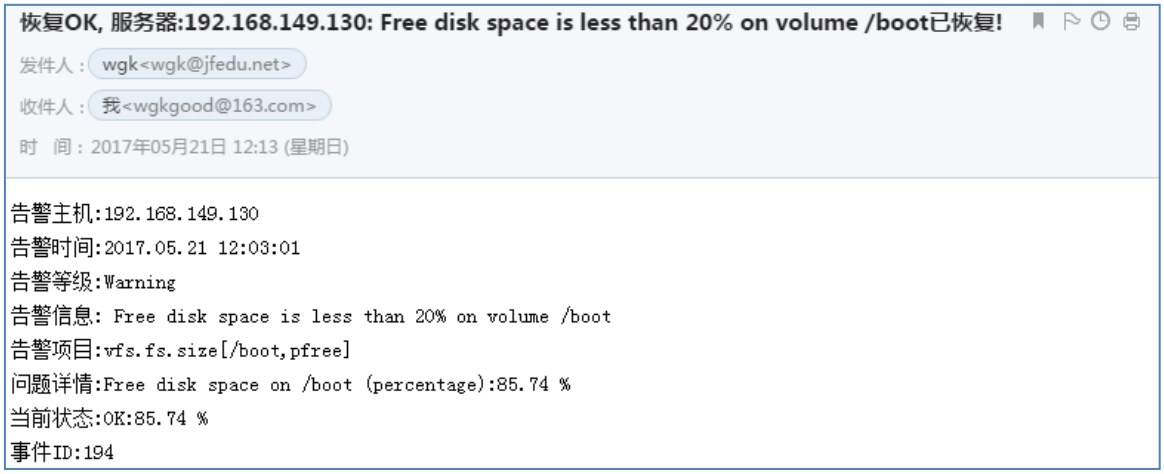


图 13-23（c） Zabbix 监控故障 item 恢复发送邮件

13.8 Zabbix 监控 MYSQL 主从实战

Zabbix 监控除了可以使用 Agent 监控客户端服务器状态、CPU、内存、硬盘、网卡流量，同时 Zabbix 还可以监控 MySQL 主从用、监控 LAMP、Nginx WEB 服务器等等，如下为 Zabbix 监控 MYSQL 主从复制的步骤：

（1） 在 Zabbix Agent 端/data/sh 目录创建 Shell 脚本：mysql_ab_check.sh，写

入如下代码：

```
#!/bin/bash

/usr/local/mysql/bin/mysql -uroot -e 'show slave status\G' |grep -E

"Slave_IO_Running|Slave_SQL_Running"|awk '{print $2}'|grep -c Yes
```

（2） 在客户端 Zabbix_agentd.conf 配置文件中加入如下代码：

```
UserParameter=mysql.replication,sh /data/sh/mysql_ab_check.sh
```

（3） Zabbix 服务器端获取监控数据，如果返回值为 2，则证明从库 I/O、SQL 线程均为 YES，表示主从同步成功:

```
/usr/local/zabbix/bin/zabbix_get -s 192.168.149.129 -k mysql.replication
```

（4） Zabbix WEB 平台 ,在 192.168.149.129 hosts 中创建 item 监控项 ,如图 13-24（a）、13-24（b）所示，单击右上角 create item，Key 填写 Zabbix Agentd 配置文件中的 mysql.replication 即可：

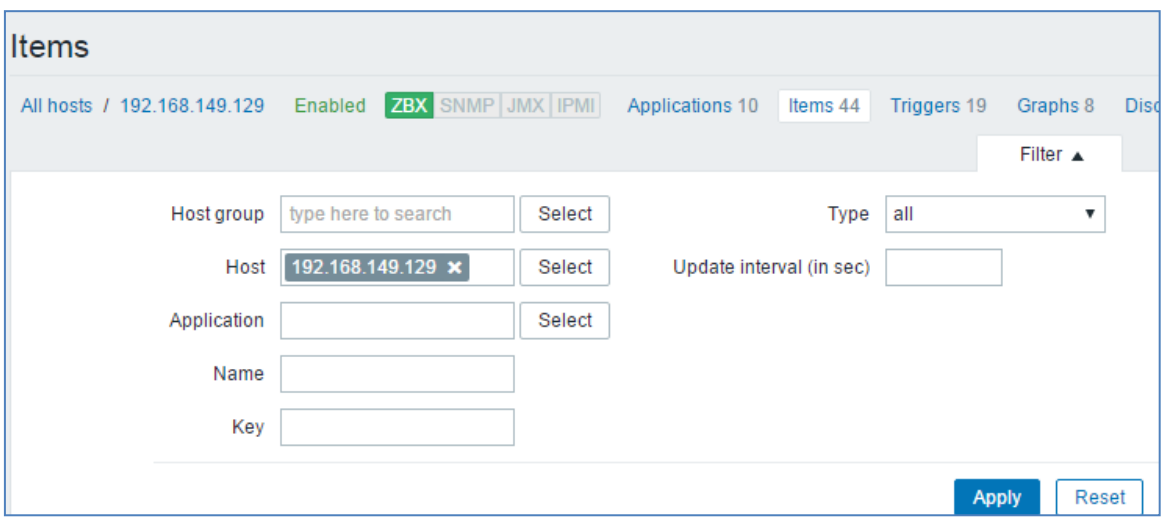


图 13-24（a） Zabbix 添加 MYSQL 主从 item

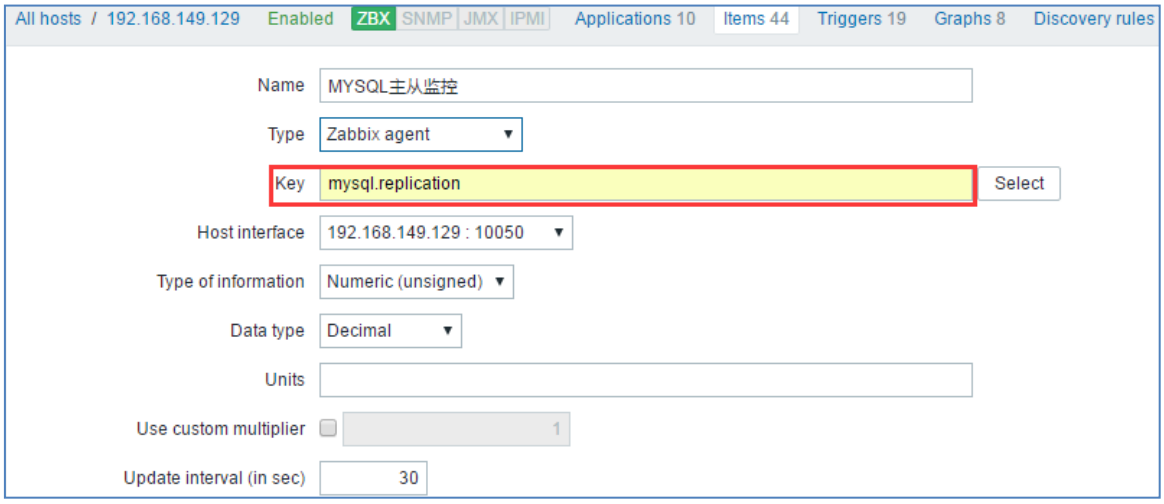


图 13-24（b） Zabbix 添加 MYSQL 主从 item

MYSQL 主从监控项创建 Graph 图像，如图 13-25（a）、13-25（b）所示：

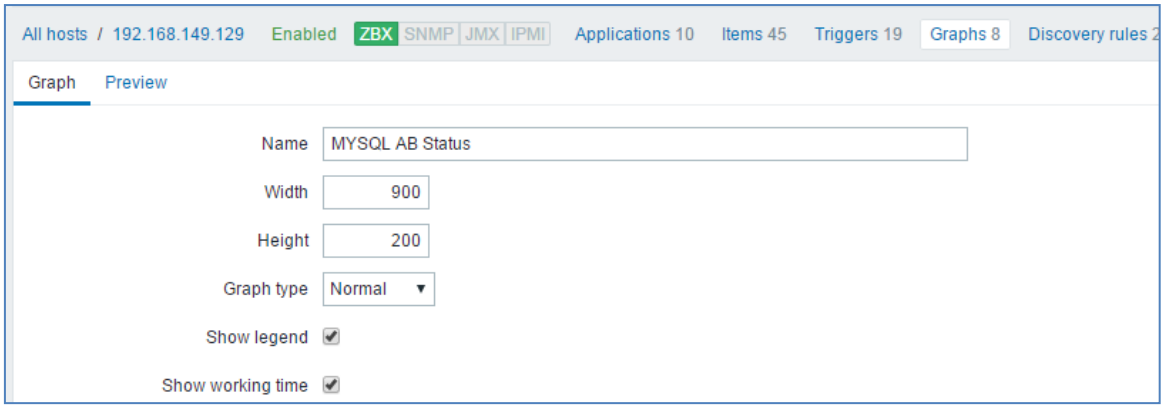


图 13-25（a） 创建 MYSQL 主从监控图像



图 13-25（b） 创建 MYSQL 主从监控图像

MYSQL 主从监控项创建触发器，如图 13-26（a）、13-26（b）所示，MYSQL 主从状态监控，设置触发器条件为 key 值不等于 2 即可，不等于 2 即表示 MYSQL 主从同步状态异常，匹配触发器，执行 Actions：

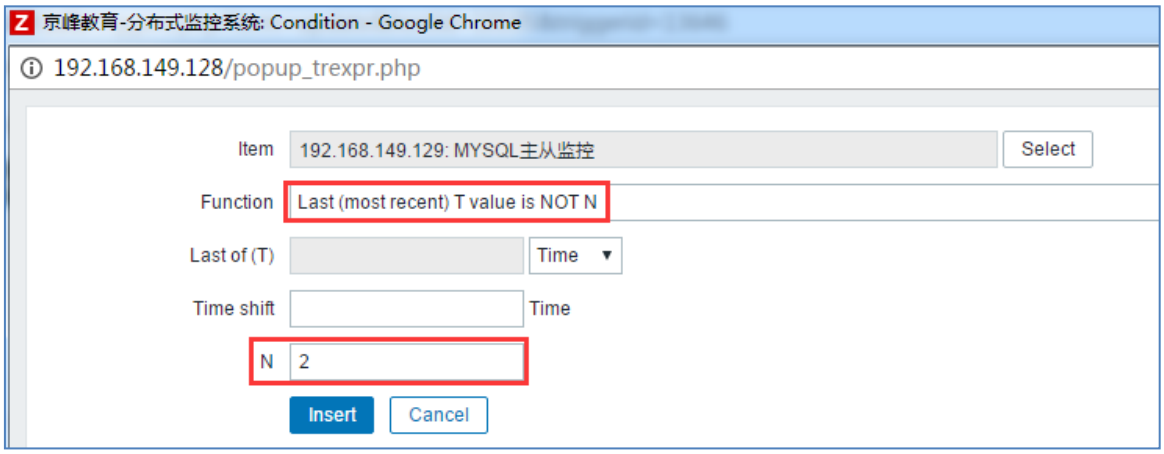


图 13-26（a） 创建 MYSQL 主从监控触发器

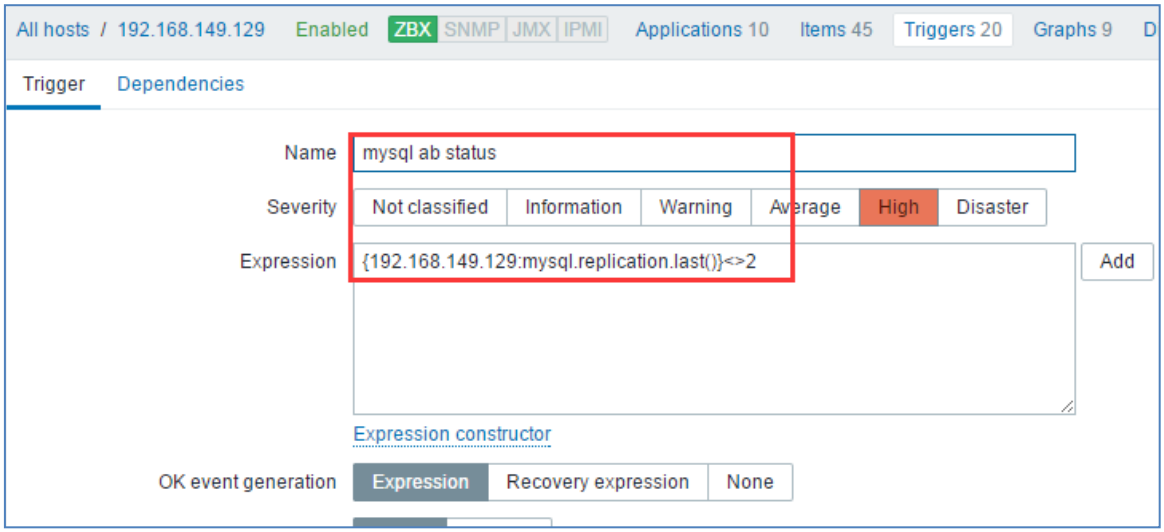


图 13-26（b） 创建 MYSQL 主从监控触发器

如果主从同步状态异常，Key 值不等于 2，会触发邮件报警，报警信息如图 13-27 所示：



图 13-27 MYSQL 主从监控报警邮件

13.9 Zabbix 日常问题汇总

Zabbix 可以设置中文汉化，如果出现乱码解决办法，如果访问 zabbix 出现如下历史记录乱码， WEB 界面乱码，原因是因为数据库导入前不是 UTF-8 字符集，需要修改为 UTF-8 模式，如图 13-28 所示：

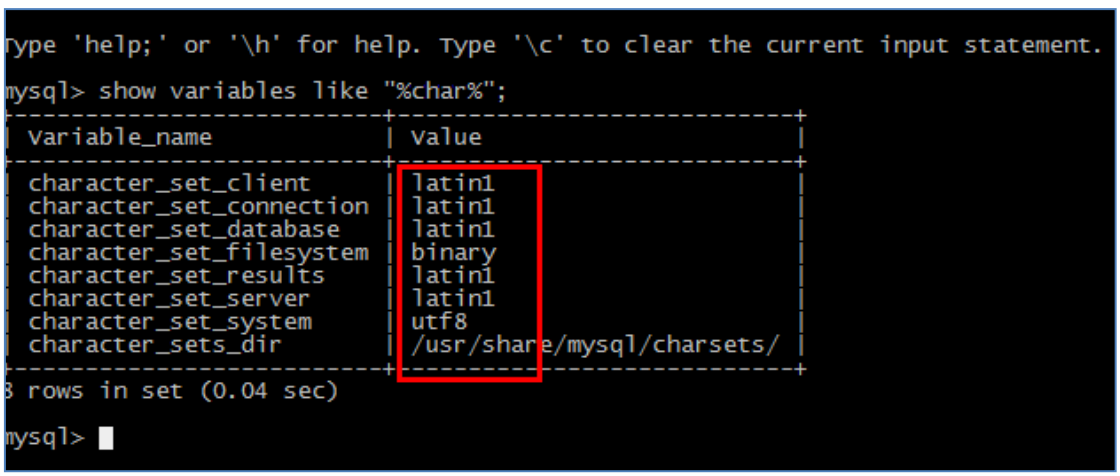


图 13-28 数据库原字符集 latin1

MYSQL 数据库修改字符集方法，vim /etc/my.cnf 在配置段加入如下代码：

```
[mysqld]

character-set-server= utf8

[client]

default-character-set = utf8

[mysql]

default-character-set = utf8
```

备份 zabbix 数据库，并删除原数据库，重新创建，再导入备份的数据库，修改导入的 zabbix.sql 文件里面的 latin1 为 utf8，然后再导入到 zabbix 库，乱码问题解决。

```
sed   -i   's/latin1/utf8/g'   zabbix.sql
```

如果在查看 graphs 监控图像界面的时候时候出现乱码，如图 13-29 所示：

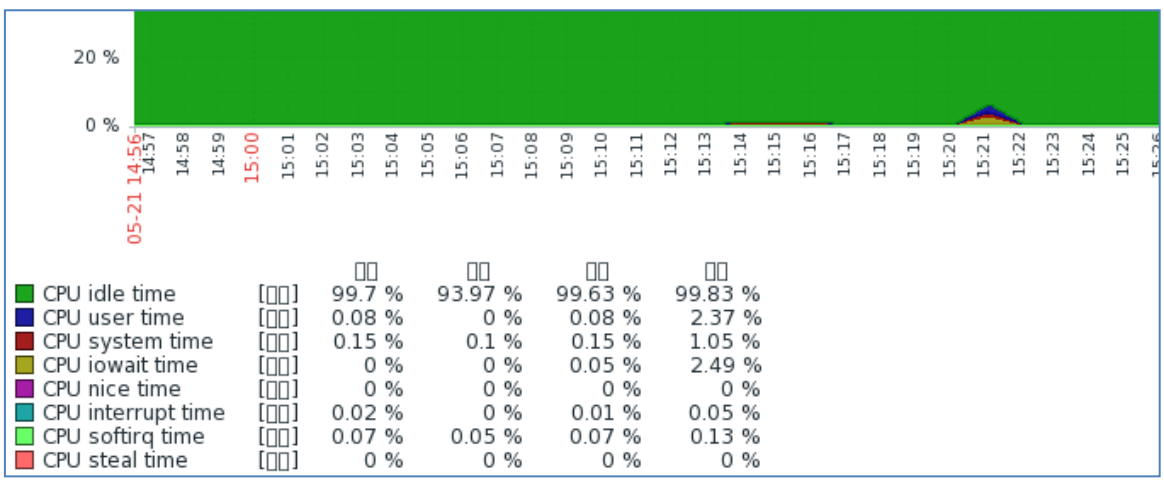


图 13-29 Graphs 图像乱码

从 windows 下控制面板->字体->选择一种中文字库，例如“楷体”，如图 13-30 所示：



图 13-30 上传 Windows 简体中文字体

将字体文件 cp 至 zabbix 服务 dauntfonts 目录下，/var/www/html/zabbix/fonts，并且将 STKAITI.TTF 重命名为 DejaVuSans.ttf，最好刷新 Graph 图像，乱码问题解决，如图 13-31（a）、13-31（b）所示：

```
[root@localhost ~]# cd /var/www/html/fonts/
[root@localhost fonts]#
[root@localhost fonts]#
[root@localhost fonts]# ls
DejaVuSans.ttf
[root@localhost fonts]#
[root@localhost fonts]#
[root@localhost fonts]# rz -y
rz waiting to receive.
 zmodem trl+C d
  100%   12437 KB 12437 KB/s 00:00:01       0 Errors

[root@localhost fonts]# ls
DejaVuSans.ttf  stkaiti.ttf
[root@localhost fonts]# mv stkaiti.ttf DejaVuSans.ttf
mv: overwrite `DejaVuSans.ttf'? y
[root@localhost fonts]#
[root@localhost fonts]# ls
DejaVuSans.ttf
```

图 13-31（a） 上传 Windows 简体中文字体

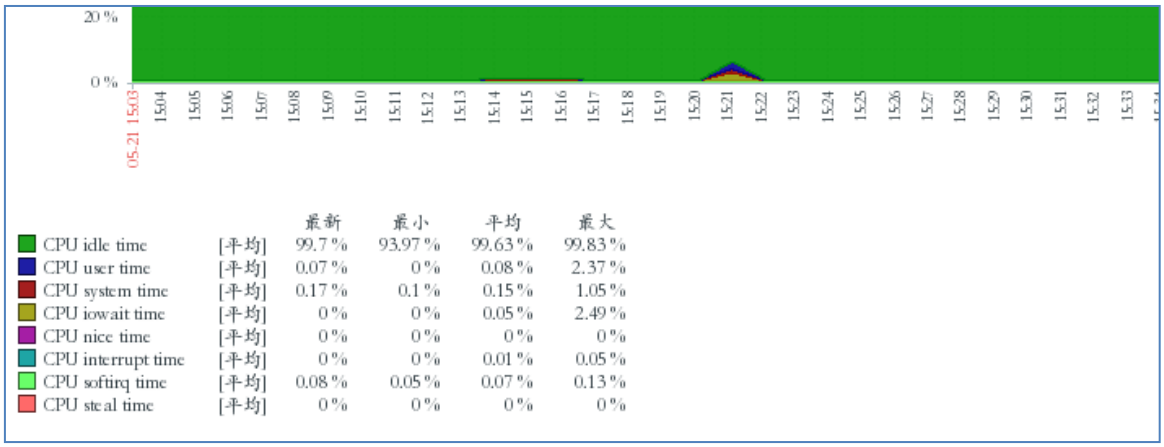


图 13-31（b） Graph 图像乱码问题解决

13.10 Zabbix 触发命令及脚本

Zabbix 监控在对服务或者设备进行监控的时候，如果被监控客户端服务异常，满足触发器，默认可以发送邮件报警、短信报警及微信报警。Zabbix 还可以远程执行命令或者脚本对部分故障实现自动修复。具体可以执行的任务包括：

- 重启应用程序，例如 Apache、Nginx、MySQL、Tomcat 服务等；
- 通过 IPMI 接口重启服务器；
- 删除服务器磁盘空间及数据；
- 执行脚本及资源调度管理；
- 远程命令最大长度为 255 字符；
- 同时支持多个远程命令；
- Zabbix 代理不支持远程命令。

使用 Zabbix 远程执行命令，首先需在 zabbix 客户端配置文件开启对远程命令的支

持，在 zabbix_agentd.conf 行尾加入如下代码，并重启服务，如图 13-32 所示：

```
EnableRemoteCommands = 1
```

```
[root@localhost ~]# cd /usr/local/zabbix/etc/
[root@localhost etc]#
[root@localhost etc]# ls
zabbix_agentd.conf  zabbix_agentd.conf.d
[root@localhost etc]#
[root@localhost etc]# vim zabbix_agentd.conf
LogFile=/tmp/zabbix_agentd.log
Server=192.168.149.128
ServerActive=192.168.149.128
Hostname = 192.168.149.129
UserParameter=mysql.replication,sh /data/sh/mysql_ab_check.sh
EnableRemoteCommands = 1
~
```

图 13-32 客户端配置远程命令支持

创建 Action，Configuration→Actions→Triggers，如图 13-33（a）、13-33（b）所示，类型选择“Remote Command”，steps 表示执行命令 1-3 次， step duration 设置每次命令间隔时间的 5 秒执行一次，执行命令方式选择 Zabbix agent，基于 sudo 执行命令即可：

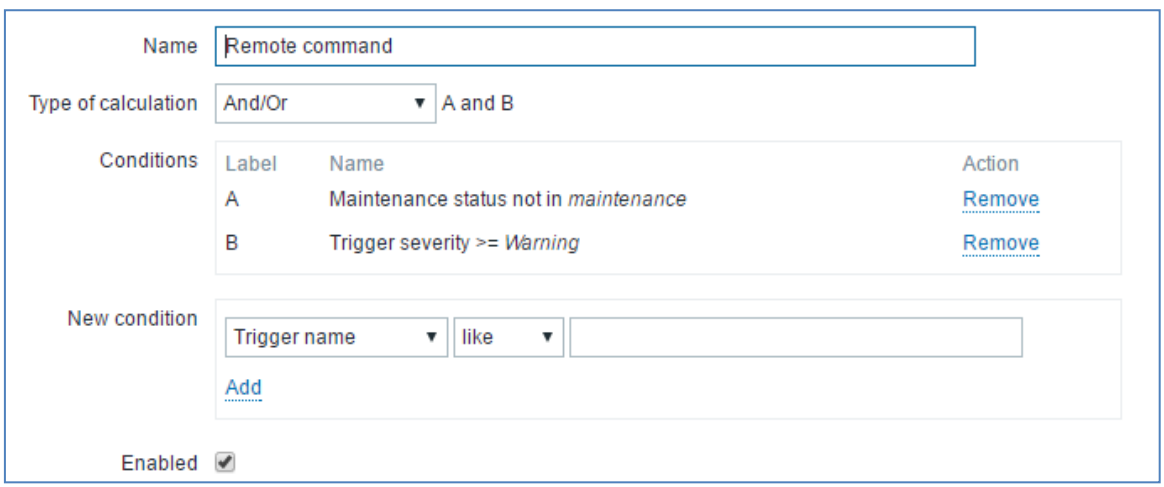


图 13-33（a） 客户端触发器满足条件

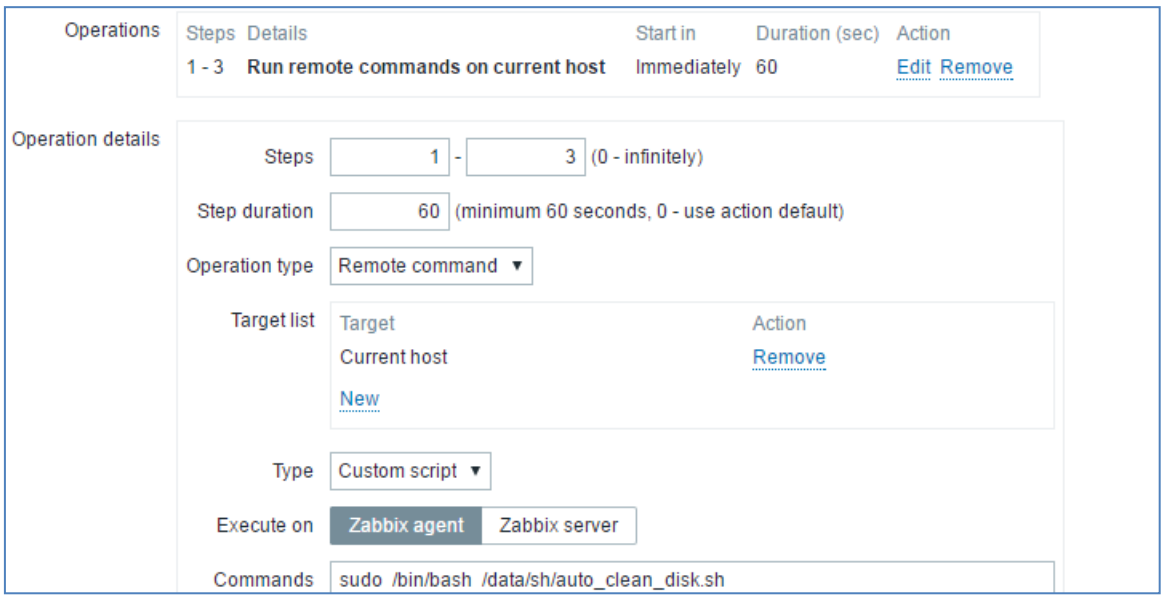


图 13-33（b） Operations 类型选择 Remote Command

Zabbix 客户端 Sudoer 配置文件中添加 zabbix 用户拥有执行权限且无需密码登录:

```
Defaults:zabbix          !requiretty

zabbix   ALL=(ALL)       NOPASSWD: ALL
```

Zabbix 客户端/data/sh/，创建 auto_clean_disk.sh，脚本代码如下：

```
#!/bin/bash

#auto clean disk space

#2022 年 6 月 21 日 10:12:18

#by author jfedu.net

rm -rf /boot/test.img

find /boot/ -name "*.log" -size +100M -exec rm -rf {} \ ;
```

将 192.168.149.129 服务器/boot 目录，临时写满，然后满足触发器，实现远程命

令执行，查看问题事件命令执行结果，如图 13-34（a）、13-34（b）所示：



图 13-34（a） Remote Command 执行成功

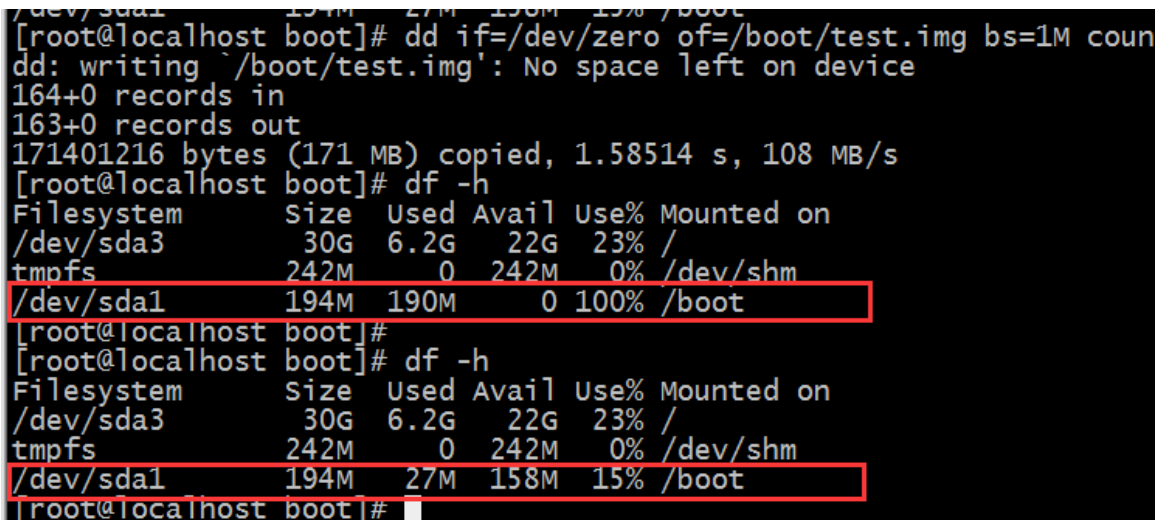


图 13-34（b） Remote Command 执行磁盘清理成功

如果 Zabbix 客户端脚本或者命令没有执行成功，http 服务没有停止,可以在 Zabbix server 端执行如下命令，如图 13-35 所示：

```
/usr/local/zabbix/bin/zabbix_get -s 192.168.149.129 -k "system.run[sudo /etc/init.d/httpd restart]"
```

```
ifconfig eth0
):Ethernet  HWaddr 00:0C:29:3F:F8:29
 192.168.149.128  Bcast:192.168.149.255  Mask:255.255.255.0
:: fe80::20c:29ff:fe3f:f829/64 Scope:Link
AST RUNNING MULTICAST  MTU:1500  Metric:1
:453673 errors:0 dropped:0 overruns:0 frame:0
:538109 errors:0 dropped:0 overruns:0 carrier:0
:0 txqueuelen:1000
68319859 (160.5 MiB)  TX bytes:239519442 (228.4 MiB)

 /usr/local/zabbix/bin/zabbix_get -s 192.168.149.129 -k "system.run[sud
)K  ]
)d: Could not reliably determine the server's fully qualified domain na
```

图 13-35 测试 Remote Command 命令

13.11 Zabbix 分布式监控实战

Zabbix 是一个分布式监控系统，它可以以一个中心点、多个分节点的模式运行，使用 Proxy 能大大的降低 Zabbix Server 的压力，Zabbix Proxy 可以运行在独立的服务器上，如图 13-36 所示：

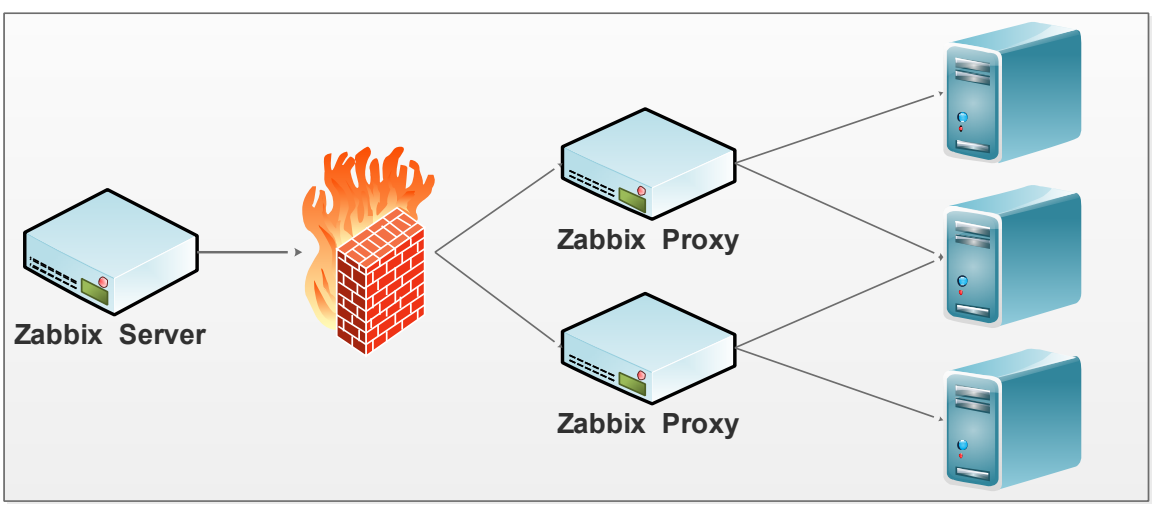


图 13-36 Zabbix Proxy 网络拓扑图

安装 Zabbix Proxy，基于 Zabbix-3.2.6.tar.gz 软件包，同时需要导入 zabbix 基本

框架库，具体实现方法如下：

（1） 下载 Zabbix 软件包，代码如下：

```
wget http://sourceforge.net/projects/zabbix/files/ZABBIX%20Latest%20Stable
/3.2.6/zabbix-3.2.6.tar.gz/download
```

（2） Zabbix Proxy 上执行如下代码：

```
yum -y install curl curl-devel net-snmp net-snmp-devel perl-DBI
groupadd zabbix ；useradd -g zabbix zabbix；usermod -s /sbin/nologin
zabbix
```

（3） Zabbix Proxy 端配置

创建 zabbix 数据库，执行授权命令：

```
create database zabbix_proxy charset=utf8;
grant all on zabbix_proxy.* to zabbix@localhost identified by '123456';
flush privileges;
```

解压 zabbix 软件包并将 Zabbix 基础 SQL 文件导入数据至 Zabbix 数据库：

```
tar zxvf zabbix-3.2.6.tar.gz
cd zabbix-3.2.6
mysql -uzabbix -p123456 zabbix_proxy <database/mysql/schema.sql
mysql -uzabbix -p123456 zabbix_proxy <database/mysql/images.sql
```

切换至 Zabbix 解压目录，执行如下代码，安装 Zabbix_server：

```
./configure --prefix=/usr/local/zabbix/ --enable-proxy --enable-agent --with-

mysql --enable-ipv6 --with-net-snmp --with-libcurl

make

make install

ln -s /usr/local/zabbix/sbin/zabbix_*  /usr/local/sbin/
```

Zabbix Proxy 安装完毕，cd /usr/local/zabbix/etc/目录，如图 13-37 所示：

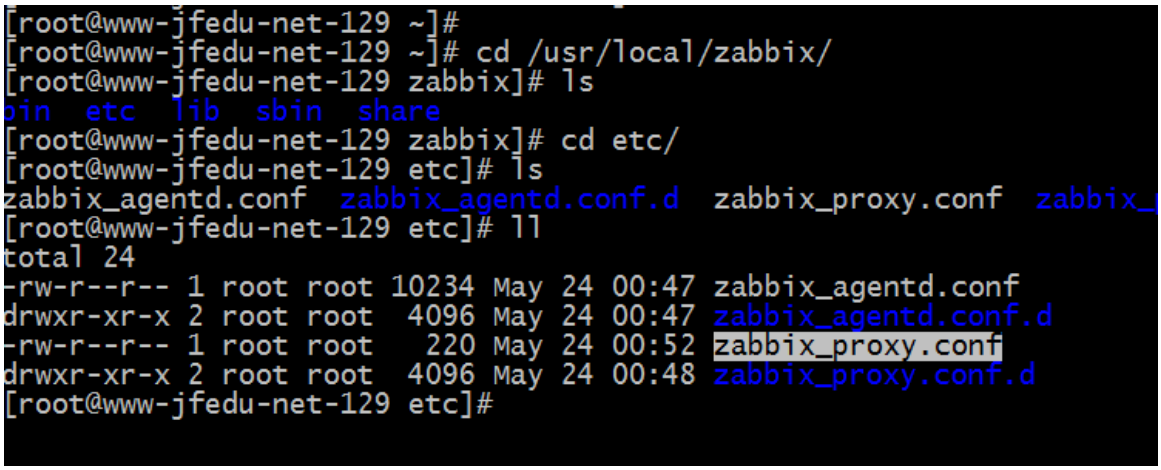


图 13-37 Zabbix Proxy 安装目录

（4） 备份 Zabbix Proxy 配置文件，代码如下：

```
cp  zabbix_proxy.conf  zabbix_proxy.conf.bak
```

（5） 将 zabbix_proxy.conf 配置文件中代码设置为如下：

```
Server=192.168.149.128

Hostname=192.168.149.130

LogFile=/tmp/zabbix_proxy.log
```

```
DBName=zabbix_proxy

DBUser=zabbix

DBPassword=123456

Timeout=4

LogSlowQueries=3000

DataSenderFrequency=30

HistoryCacheSize=128M

CacheSize=128M
```

（6） Zabbix 客户端安装 Agent ,同时配置 Agent 端 Server 设置为 Proxy 服务器的 IP 地址或者主机名，zabbix_agentd.conf 配置文件代码：

```
LogFile=/tmp/zabbix_agentd.log

Server=192.168.149.130

ServerActive=192.168.149.130

Hostname = 192.168.149.131
```

（7） Zabbix Server WEB 端添加 Proxy ,实现集中管理和分布式添加监控 ,如图 13-38（a）、13-38（b）、13-38（c）所示：

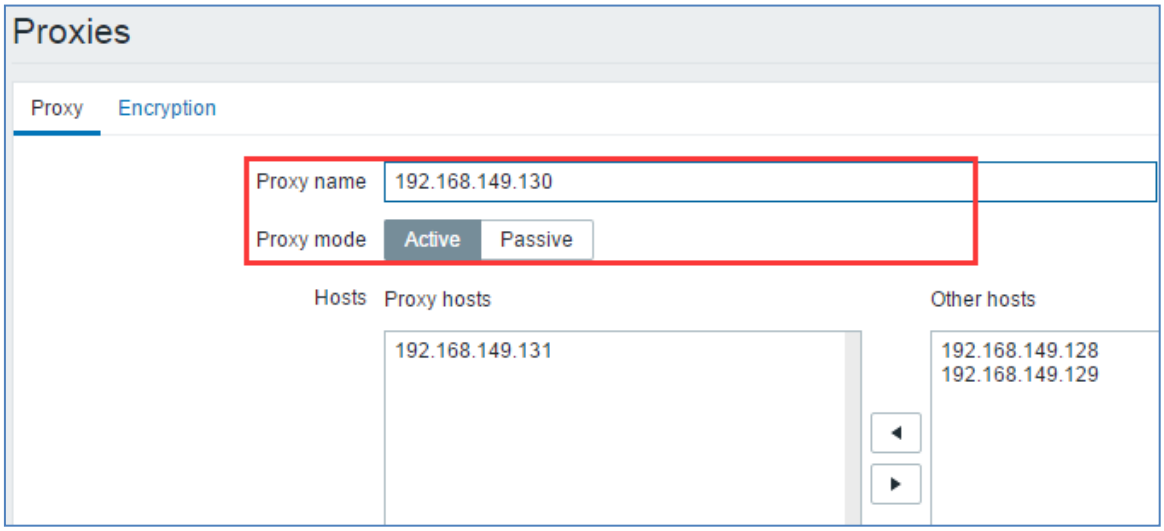


图 13-38（a） Zabbix Proxy WEB 添加

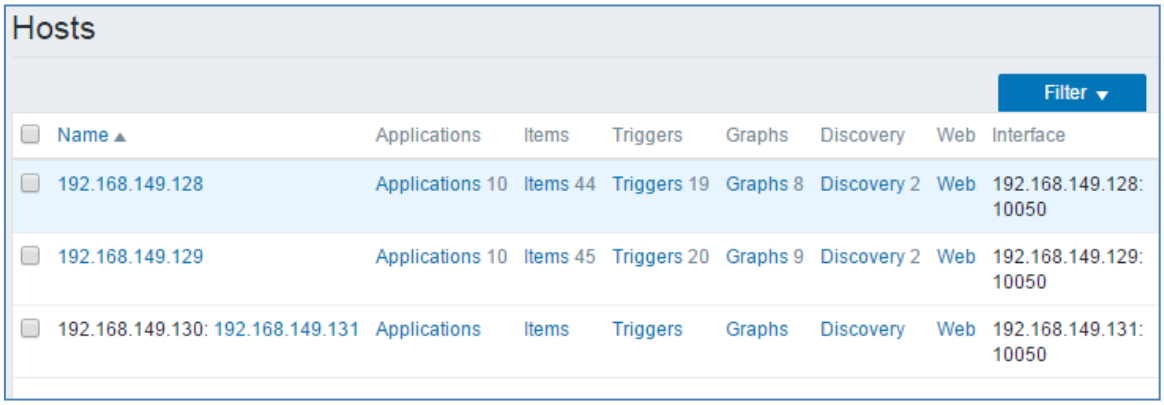


图 13-38（b） Zabbix Proxy 监控客户端

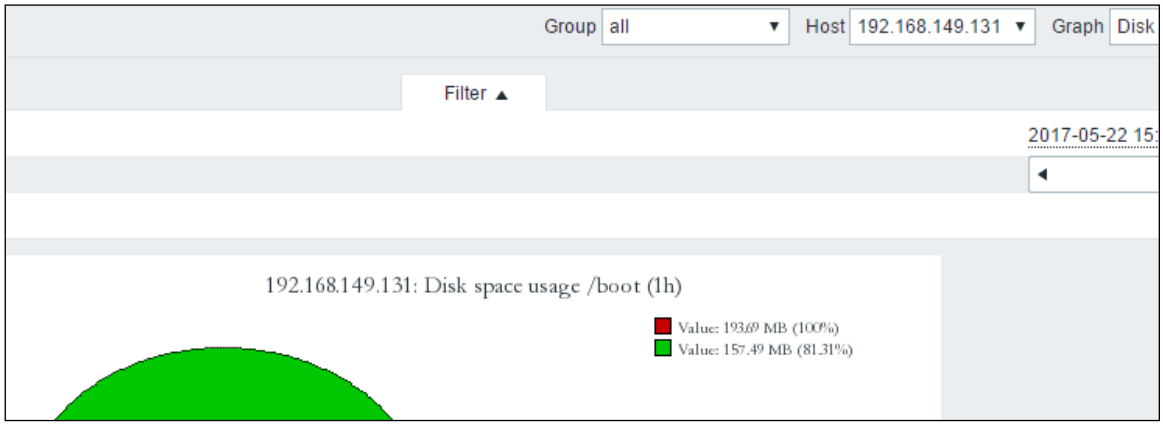


图 13-38（c） Zabbix Proxy 监控客户端图像

13.12 Zabbix 监控微信报警实战

Zabbix除了可以使用邮件报警之外，还可以通过多种方式把告警信息发送到指定人，例如短信报警方式，越来越多的企业开始使用 Zabbix 结合微信作为主要的告警方式，因为每个人每天都在使用微信，这样可以及时有效的把告警信息推送到接收人，方便告警的及时处理。Zabbix 微信报警怎么设置呢，设置的步骤有哪些呢，方法步骤如下：

（1） 微信企业号注册

企业号注册地址：https://qy.weixin.qq.com/填写企业注册信息，等待审核完，并且微信扫描登录企业公众号，如图 13-39（a）、13-39（b）所示：



图 13-39（a） 微信企业公众号注册



图 13-39（b） 微信企业公众号登录

（2） 通讯录添加运维部门及人员

登录新建的企业号，通过提前把企业成员信息添加到组织或者部门，需要填写手机号、微信号或邮箱，通过这样方式让别人扫码关注企业公众号，为了后面企业号推送消息给企业成员，如图 13-40（a）、13-40（b）所示：



图 13-40（a） 微信企业公众号通讯录



图 13-40（b） 微信企业公众号通讯录

（3） 企业应用-创建应用

除了对个人添加微信报警之外,还可以添加不同管理组，接受同一个应用推送的消息，

成员账号，组织部门 ID，应用 Agent ID，CorpID 和 Secret，调用 API 接口需要用到这些信息，如图 13-41（a）、13-41（b）、13-41（c）所示：



图 13-41（a） 微信企业公众号创建应用



图 13-41（b） 微信企业公众号创建应用



图 13-41（c） 微信企业公众号创建应用

（4） 获取企业 CorpID，单击企业公众号首页“我的企业”，即可看到，如图 13-42 所示：



图 13-42 微信企业公众号 CorpID

（5） 微信接口调试，调用微信接口需要一个调用接口的凭证：Access_token 通过 CorpID 和 Secret 可以获得 Access_token，微信企业号接口调试地址：http://qydev.weixin.qq.com/debug，如图 13-43（a）、13-43（b）所示：

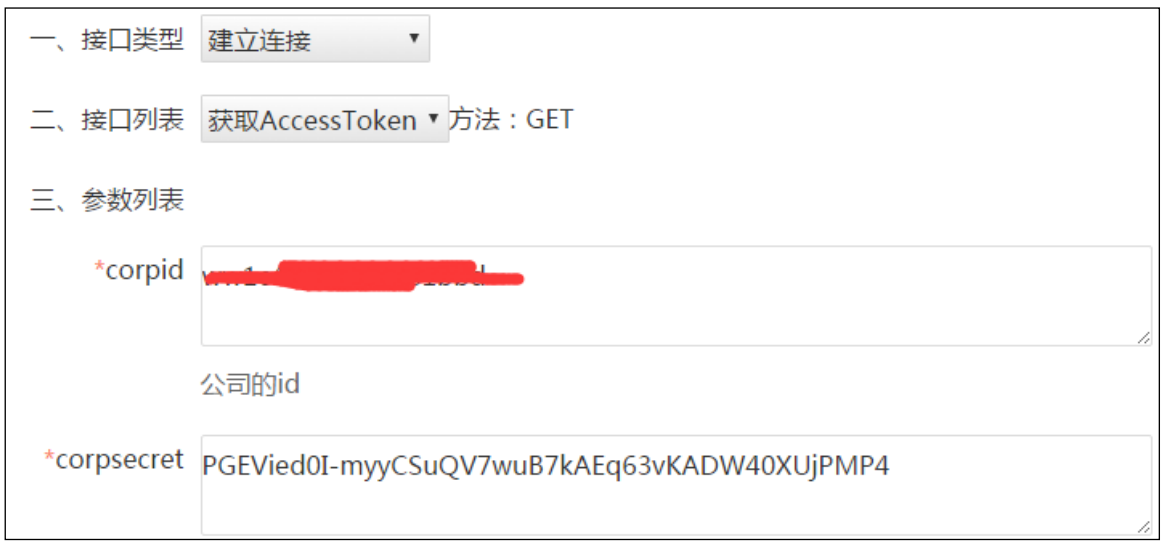


图 13-43（a） 微信企业公众号调试

建立连接：获取AccessToken

请求地址： https://qyapi.weixin.qq.com/cgi-bin/gettoken?corpid=ww1c4b5aad35681bbd&co
myyCSuQV7wuB7kAEq63vKADW40XUjPMP4

返回结果： HTTP/1.0 200 OK
Connection: close
Error-Code: 0
Error-Msg: ok
Content-Type: application/json; charset=UTF-8
Content-Length: 362

{"errcode":0,"errmsg":"ok","access_token":"LrY3hh-aoKO7mFBOAVfUpkcsLQ6-rx2
U4RWMwSIYMnPVJmXTLmX0pATmDqHpTyatQ035H7yOK1psTGynDX5Zy2lkvRVc

图 13-43（b） 微信企业公众号调试

（6） 获取微信报警工具

```
mkdir   -p /usr/local/zabbix/alertscripts

cd   /usr/local/zabbix/alertscripts

wget http://dl.cactifans.org/tools/zabbix_weixin.x86_64.tar.gz

tar   zxvf zabbix_weixin.x86_64.tar.gz

mv   zabbix_weixin/weixin   .

chmod o+x weixin

mv zabbix_weixin/weixincfg.json /etc/

rm -rf zxvf zabbix_weixin.x86_64.tar.gz

rm -rf zabbix_weixin/
```

修改/etc/ weixincfg.json 配置文件中 corpid、secret、agentid，并测试脚本发送信息，如图 13-44（a）、13-44（b）所示：

cd /usr/local/zabbix/alertscripts

./weixin wuguangke 京峰教育报警测试 Zabbix 故障报警

./weixin contact subject body

标准信息格式：

Contact，为你的微信账号，注意不是微信号，不是微信昵称，可以把用户账号设置成微信号或微信昵称，Subject 告警主题，Body 告警详情。

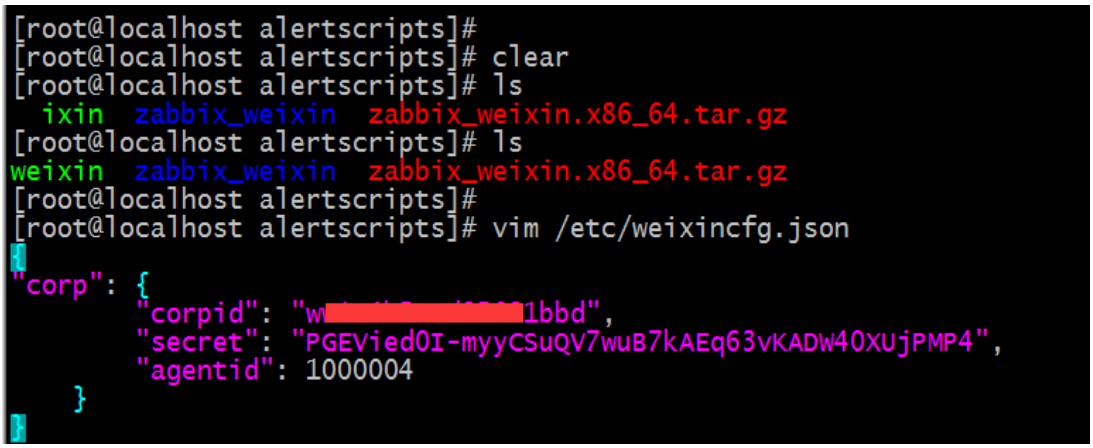


图 13-44（a） Zabbix Server 端微信配置文件

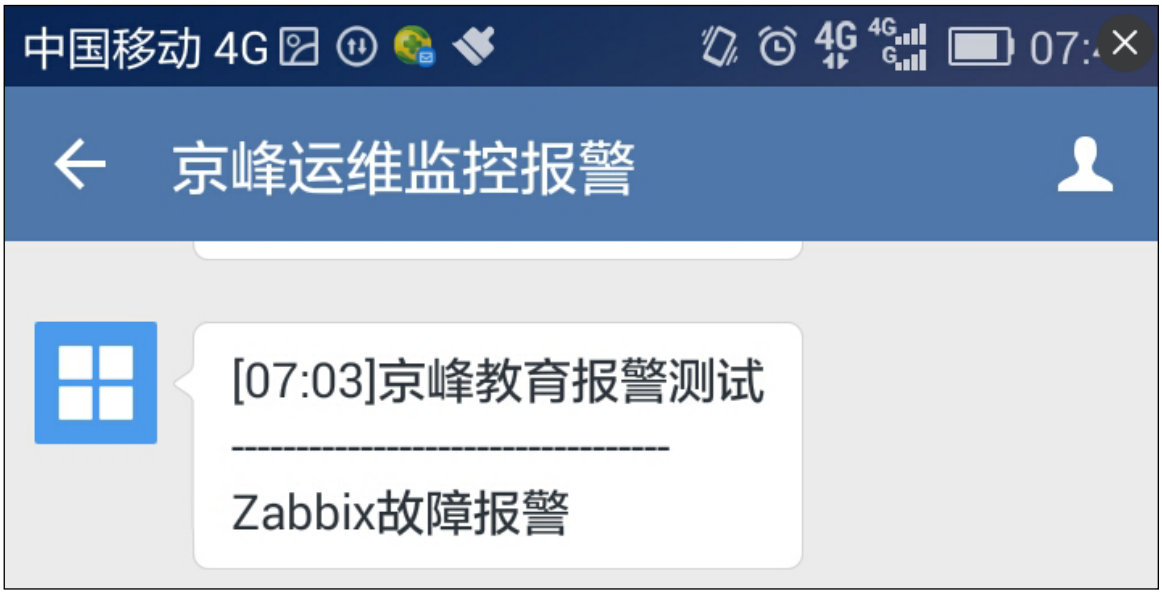


图 13-44（b） Zabbix Server 端微信配置文件

（7） 脚本调用设置

Zabbix_Server 端设置脚本执行路径，编辑 zabbix_server.conf 文件，添加如下内容：

```
AlertScriptsPath=/usr/local/zabbix/alertscripts
```

（8） Zabbix WEB 端配置，设置 Actions 动作，并设置触发微信报警，如图 13-45（a）、13-45（b）、13-45（c）所示：

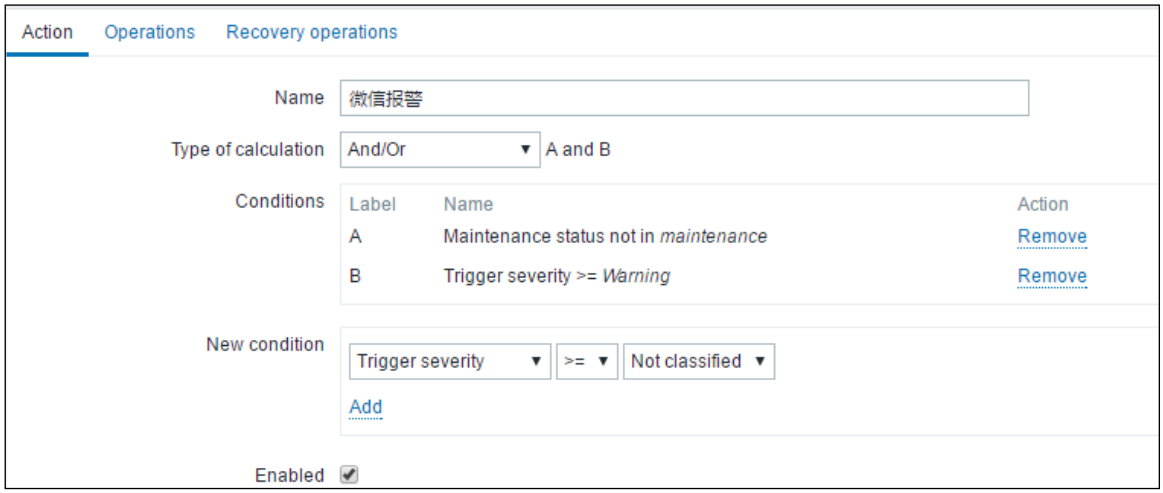


图 13-45（a） Zabbix Server Action 动作配置

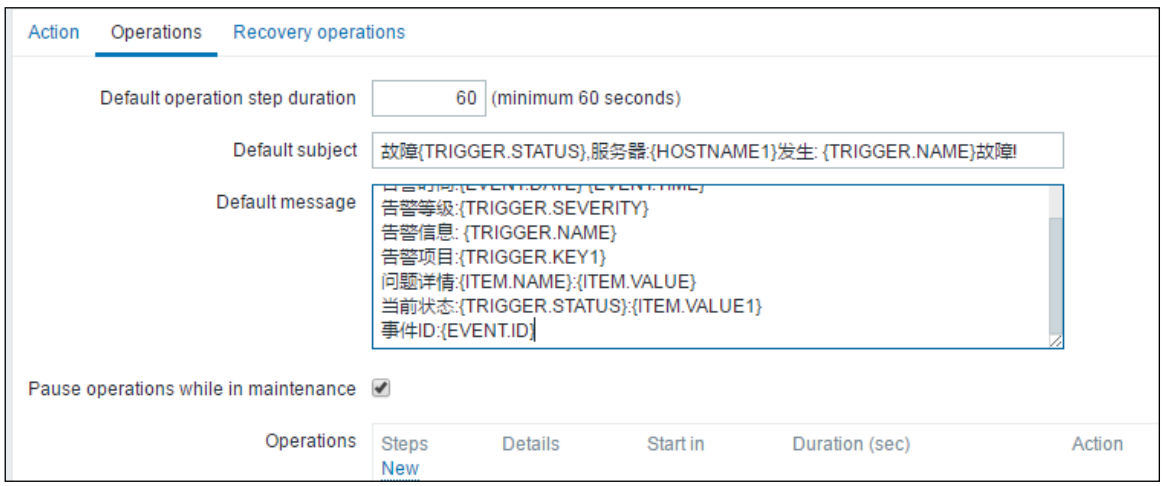


图 13-45（b） Zabbix Server Action 动作配置

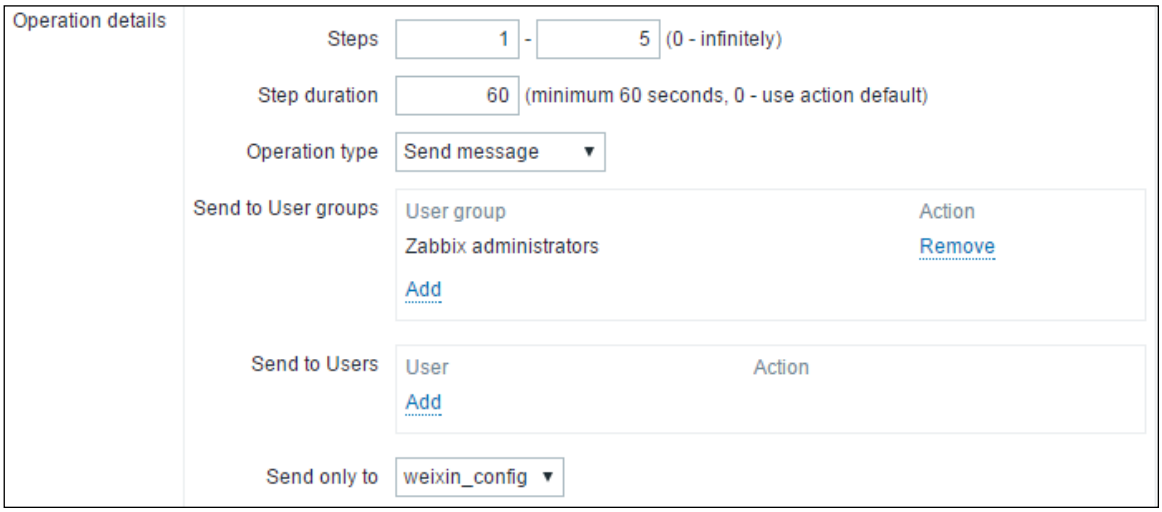


图 13-45（c） Zabbix Server Action 动作配置

（9） 配置 Media Type 微信脚本，Administration→Media Types→Create Media Type 如图 13-46 所示，脚本加入三个参数：{ALERT.SENDTO}、{ALERT.SUBJECT}、{ALERT.MESSAGE}：

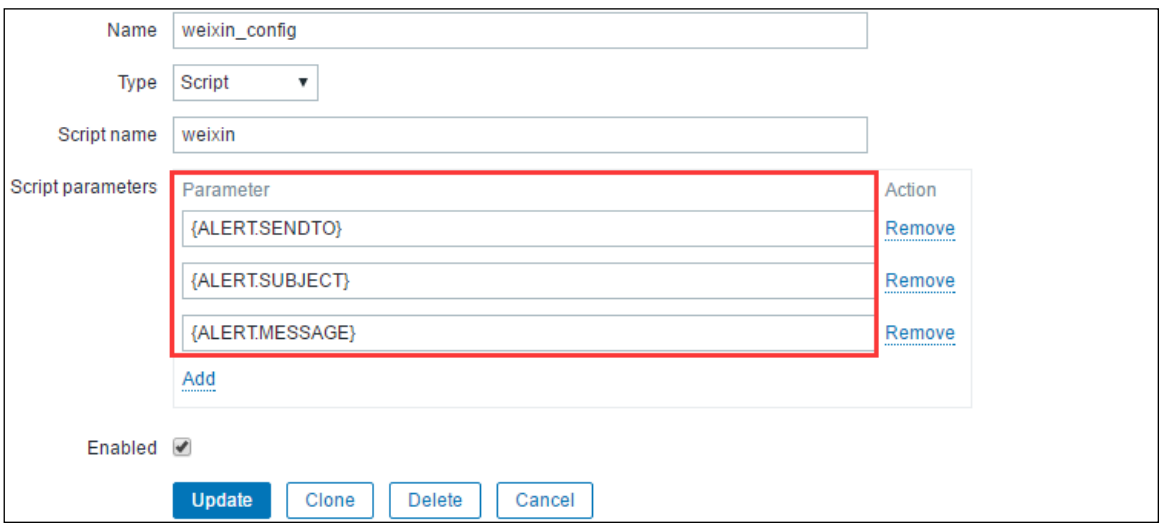


图 13-46 Zabbix Server Media Types 配置

（10） 配置接收微信信息的用户，Administration→Users→Admin→Media 如图 13-47 所示：

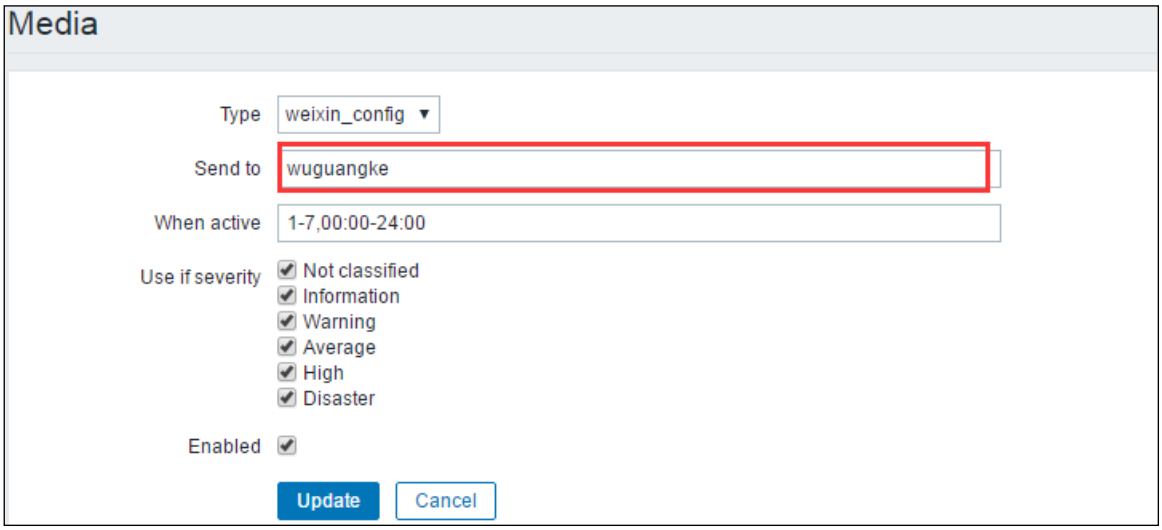


图 13-47 Zabbix Server Users Media

（11） 微信报警信息测试，磁盘容量剩余不足 20%，会触发微信报警，如图 13-48（a）、13-48（b）、13-48（c）所示：

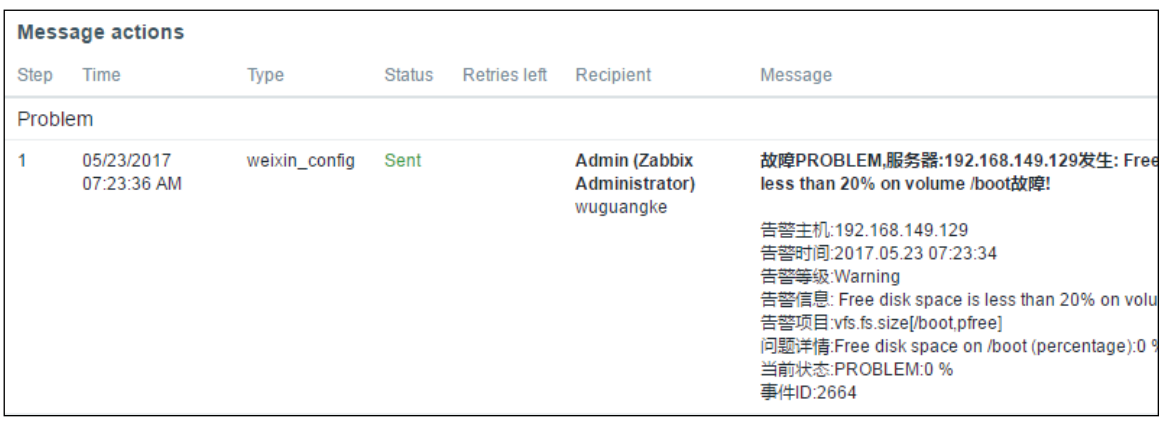


图 13-48（a）Zabbix 微信报警信息



图 13-48（b）Zabbix 微信报警故障信息



图 13-48（c）Zabbix 微信报警恢复信息

第14章 Nginx WEB 服务器企业实战

万维网（WORLD WIDE WEB，WWW)服务器，也称之为 WEB 服务器，主要功能是提供网上信息浏览服务。目前主流的 WEB 服务器软件包括：Apache、Nginx、Lighttpd、IIS、Resin、Tomcat、WebLogic、Jetty。

本章向读者介绍 Nginx 高性能 WEB 服务器、Nginx 工作原理、安装配置及升级、Nginx 配置文件深入剖析、Nginx 虚拟主机、Location 案例演示、Nginx Rewirte 企业案例实战、HTTPS 安全 WEB 服务器及 Nginx 高性能集群实战等。

14.1 Nginx WEB 入门简介

Nginx ("engine x") 是一个高性能 HTTP 和 反向代理 服务器、IMAP、POP3、SMTP 服务器。 Nginx 是由 Igor Sysoev 为俄罗斯访问量第二的 Rambler.ru 站点开发的，第一个公开版本 0.1.0 发布于 2004 年 10 月 4 日。其将源代码以类 BSD 许可证的形式发布，因它的稳定性、丰富的功能集、示例配置文件和低系统资源的消耗而闻名。

由于 Nginx 的高性能、轻量级，目前越来越多的互联网企业开始使用 Nginx WEB 服务器。据 Netcraft 统计，在 2022 年 4 月份，世界上最繁忙的网站中有 28.72 %使用 Nginx 作为其服务器或者代理服务器。

> nginx [engine x] is an HTTP and reverse proxy server, a mail proxy server, and a generic TCP/UDP proxy server, originally written by Igor Sysoev.

For a long time, it has been running on many heavily loaded Russian sites including Yandex, Mail.Ru, VK, and Rambler.

According to Netcraft, nginx served or proxied 28.72% busiest sites in April 2022. Here are some of the success stories: Netflix,Wordpress.com, FastMail.FM.

The sources and documentation are distributed under the 2-clause BSD-like license.

Commercial support is available from Nginx, Inc.

它已经在众多流量很大的俄罗斯网站上使用了很长时间，这些网站包括 Yandex、Mail.Ru、VKontakte，以及 Rambler。目前互联网主流公司京东、360、百度、新浪、腾讯、阿里都在使用 Nginx 作为自己的 WEB 服务器。

Nginx 特点是占有内存少，并发能力强，事实上 Nginx 的并发能力确实在同类型的网页服务器中表现较好。

Nginx 相对于 Apache 优点如下：

- 高并发响应性能非常好，官方 Nginx 处理静态文件并发 5w/s；
- 负载均衡及反向代理性能非常强；
- 系统内存和 CPU 占用率低；
- 可对后端服务进行健康检查；
- 支持 PHP cgi 方式和 FastCGI 方式；

- 可以作为缓存服务器、邮件代理服务器；
- 配置代码简洁且容易上手。

14.2 Nginx 工作原理

Nginx WEB 服务器最主要就是各种模块的工作，模块从结构上分为核心模块、基础模块和第三方模块，其中三类模块分别如下：

- 核心模块：HTTP 模块、EVENT 模块和 MAIL 模块等；
- 基础模块:HTTP Access 模块、HTTP FastCGI 模块、HTTP Proxy 模块和 HTTP Rewrite 模块；
- 第三方模块：HTTP Upstream Request Hash 模块、Notice 模块和 HTTP Access Key 模块、Limit_req 模块等；

Nginx 的模块从功能上分为如下三类。

- Handlers（处理器模块）：此类模块直接处理请求，并进行输出内容和修改 headers 信息等操作，Handlers 处理器模块一般只能有一个；
- Filters （过滤器模块）：此类模块主要对其他处理器模块输出的内容进行修改操作，最后由 Nginx 输出；

- Proxies （代理类模块）：此类模块是 Nginx 的 HTTP Upstream 之类的模块，这些模块主要与后端一些服务比如 FastCGI 等进行交互，实现服务代理和负载均衡等功能。

Nginx 由内核和模块组成，其中内核的设计非常微小和简洁，完成的工作也非常简单，仅仅通过查找配置文件将客户端的请求映射到一个 location block，而 location 是 Nginx 配置中的一个指令，用于访问的 URL 匹配，而在这个 location 中所配置的每个指令将会启动不同的模块去完成相应的工作，如图 14-1 所示：

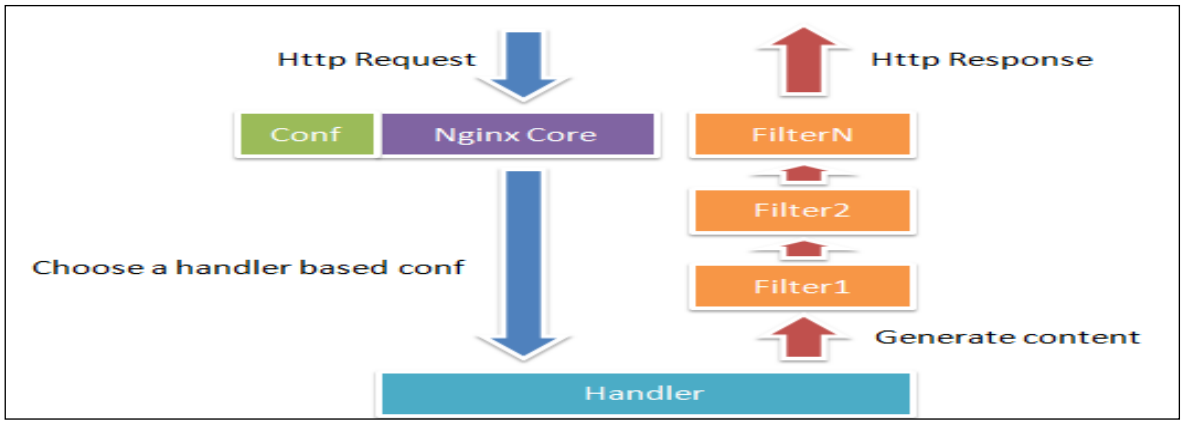


图 14-1 Nginx WEB 工作流程图

Nginx 的高并发得益于其采用了 epoll 模型，与传统的服务器程序架构不同，epoll 是 Linux 内核 2.6 以后才出现的，Nginx 采用 epoll 模型，异步非阻塞，而 apache 采用的是 select 模型：

Select 特点：select 选择句柄的时候，是遍历所有句柄，也就是说句柄有事件响应时，select 需要遍历所有句柄才能获取到哪些句柄有事件通知，因此效率是非常低。

epoll 的特点：epoll 对于句柄事件的选择不是遍历的，是事件响应的，就是句柄上

事件来就马上选择出来，不需要遍历整个句柄链表，因此效率非常高。

Nginx 默认以 80 端口监听在服务器上 ,并且启动一个 master 进程 ,同时有 master 进程生成多个工作进程，当浏览器发起一个 HTTP 连接请求，每个进程都有可能处理这个连接，怎么做到的呢？怎么保证同一时刻一个 HTTP 请求被一个工作进程处理呢。

首先每个 worker 进程都是从 Master 进程 fork 出来，在 Master 进程里面，建立好需要 listen 的 socket（listenfd）之后，会 fork 出多个 worker 进程。

所有 worker 进程的 listenfd 会在新连接到来时变得可读，为保证只有一个进程处理该连接，所有 worker 进程在注册 listenfd 读事件前抢 accept_mutex，抢到互斥锁的那个进程注册 listenfd 读事件，在读事件里调用 accept 接受该连接。

当一个 worker 进程在 accept 这个连接之后，就开始读取请求、解析请求、处理请求，产生数据后，再返回给客户端，最后才断开连接，这样形成一个完整的请求流程。如图 14-2 所示：

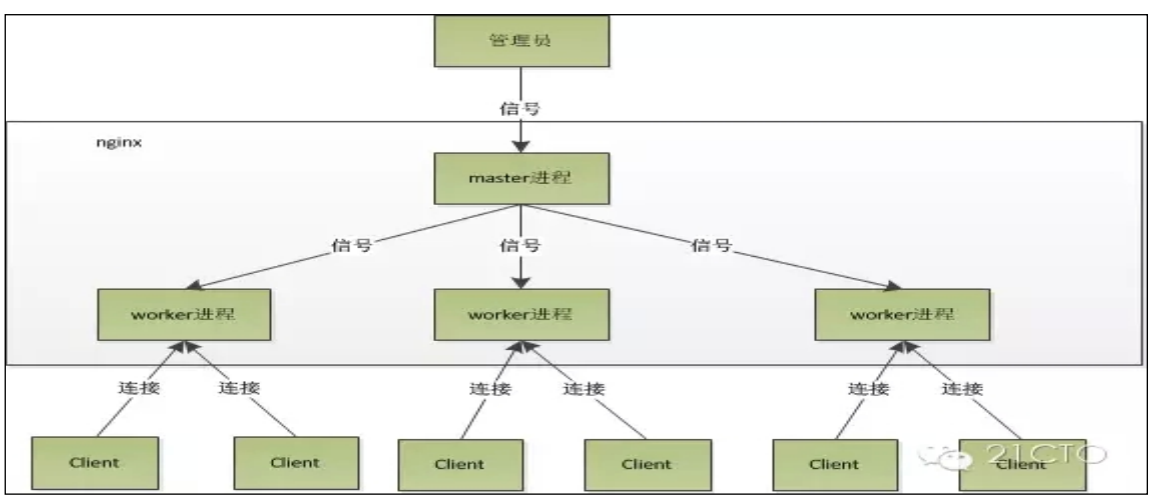


图 14-2 Nginx Worker 进程工作原理

14.3 Nginx 安装配置

Nginx WEB 安装时可以指定很多的模块，默认需要安装 Rewrite 模块，也即是需要系统有 PCRE 库，安装 Pcre 支持 Rewrite 功能。如下为安装 Nginx WEB 服务器方法：源码的路径，而不是编译后的路径，否则会报错。

```
#安装 PCRE 库支持

yum install pcre-devel pcre -y

#下载 Nginx 源码包

cd /usr/src

wget -c http://nginx.org/download/nginx-1.12.0.tar.gz

#解压 Nginx 源码包

tar -xzf nginx-1.12.0.tar.gz

#进入解压目录，然后 sed 修改 Nginx 版本信息为 JWS

cd nginx-1.12.0 ； sed -i -e 's/1.12.0//g' -e 's/nginx\//JWS/g' -e

's/"NGINX"/"JWS"/g' src/core/nginx.h

#预编译 Nginx

useradd www ; ./configure --user=www --group=www --

prefix=/usr/local/nginx --with-http_stub_status_module --with-

http_ssl_module
```

```
#.configure 预编译成功后，执行 make 命令进行编译

make

#make 执行成功后，执行 make install  正式安装

make install

#至此 Nginx WEB 服务器安装完毕。
```

测试 Nginx 服务安装是否正确，同时启动 Nginx WEB 服务，代码命令如下：

```
/usr/local/nginx/sbin/nginx  -t  检查 nginx 配置文件是否正确，返回 OK 即正确。

[root@localhost ~]# /usr/local/nginx/sbin/nginx -t

nginx: the configuration file /usr/local/nginx/conf/nginx.conf syntax is ok

nginx: configuration file /usr/local/nginx/conf/nginx.conf test is successful

[root@localhost ~]#

然后启动 nginx，/usr/local/nginx/sbin/nginx 回车即可。查看进程是否已启动：

[root@localhost ~]# ps -ef |grep nginx

nobody      5381 30285   0 May16 ?          09:04:31 nginx: worker process

root        30285        1   0   2022 ?            00:00:00 nginx: master process

/usr/local/nginx/sbin/nginx

root       32260 32220   0 12:34 pts/0      00:00:00 grep nginx

[root@localhost ~]#
```

通过浏览器访问 Nginx 默认测试页面，如图 14-3 所示：

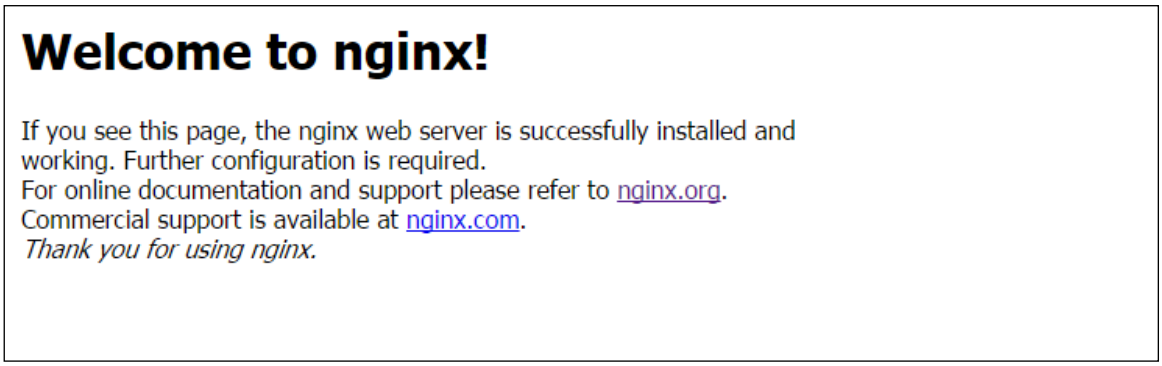


图 14-3 Nginx WEB 浏览器访问

14.4 Nginx 管理及升级

Nginx WEB 服务器安装完毕，可以执行如下命令对其进管理和维护，命令如下：

```
#查看 nginx 进程

ps -ef|grep nginx

#平滑启动 nginx

kill -HUP `cat /var/run/nginx.pid`

或者

nginx -s reload

其中进程文件路径在配置文件 nginx.conf 中可以找到。

平滑启动的意思是在不停止 nginx 的情况下，重启 nginx，重新加载配置文件，启动新

的工作线程，完美停止旧的工作线程。

#完美停止 nginx

kill -QUIT `cat /var/run/nginx.pid`
```

```
#快速停止 nginx

kill -TERM `cat /var/run/nginx.pid`

或者

kill -INT `cat /var/run/nginx.pid`

#完美停止工作进程（主要用于平滑升级）

kill -WINCH `cat /var/run/nginx.pid`

#强制停止 nginx

pkill -9 nginx

#检查对 nginx.conf 文件的修改是否正确

nginx -t -c /etc/nginx/nginx.conf  或者  nginx -t

#停止 nginx 的命令

nginx -s stop 或者 pkill nginx

#查看 nginx 的版本信息

nginx -v

#查看完整的 nginx 的配置信息

nginx -V
```

Nginx WEB 服务器定期更新，如果需要将低版本升级或者将高版本降级，升级或者降级方法如下，分为四个步骤，包括软件下载、预编译、编译、配置，具体方法如下：

```
wget http://www.nginx.org/download/nginx-1.4.2.tar.gz
```

获取旧版本 nginx 的 configure 选项

/usr/local/nginx/sbin/nginx -V

编译新版本的 Nginx

tar -xvf nginx-1.4.2.tar.gz

cd nginx-1.4.2

./configure --prefix=/usr/local/nginx --user=www --group=www --with-http_stub_status_module --with-http_ssl_module

make

备份旧版本的 nginx 可执行文件，复制新版本的 nginx 这行文件

mv /usr/local/nginx/sbin/nginx /usr/local/nginx/sbin/nginx.old

cp objs/nginx /usr/local/nginx/sbin/

测试新版本 nginx 是否正常

/usr/local/nginx/sbin/nginx -t

平滑重启升级 nginx

kill –QUIT `cat /usr/local/nginx/log/nginx.oldbin` ##关闭旧版 nginx

验证 nginx 是否升级成功

/usr/local/nginx/sbin/nginx -V 显示最新编译的版本信息即可。

14.5 Nginx 虚拟主机实战

在真实的企业服务器环境中，为了充分利用服务器的资源，单台 Nginx WEB 服务器同时会配置 N 个网站，也可称之为配置 N 个虚拟域名的主机，即多个域名对应同一个 80 端口。

在 Nginx.conf 中加入 server 代码，Nginx 虚拟主机完整代码如下：

```
worker_processes    1；
events {
      worker_connections    1024；
}
http {
      include           mime.types；
      default_type    application/octet-stream；
      sendfile            on；
      keepalive_timeout    65；
#virtual hosts config 2022/5/18
server {
            listen           80；
            server_name    www.jf1.com；
```

```
        access_log   logs/jf1.access.log ;
        location / {
            root    html/jf1 ;
            index   index.html index.htm ;
        }
}
server {
        listen          80 ;
        server_name   www.jf2.com ;
        access_log   logs/jf2.access.log ;
        location / {
            root    html/jf2 ;
            index   index.html index.htm ;
        }

 }
}
```

创建两个不同的目录 mkdir –p /usr/local/nginx/html/{jf1,jf2} ,然后分别在两个目录创建两个不同的 index.html 网站页面即可。通过 Windows 客户端配置 hosts 绑定 IP

与两个域名的对应关系，在 IE 浏览器访问测试效果，如图 14-4（a）、14-4（b）所示：



图 14-4（a） Nginx 虚拟主机 www.jf1.com



图 14-4（b） Nginx 虚拟主机 www.jf2.com

14.6 Nginx Location 深入剖析

Nginx 由内核和模块组成，其中内核的设计非常微小和简洁，完成的工作也非常简单，仅仅通过查找配置文件将客户端的请求映射到一个 location block，而 location 是 Nginx 配置中的一个指令，用于访问的 URL 匹配，而在这个 location 中所配置的每个指令将会启动不同的模块去完成相应的工作。

默认 Nginx.conf 配置文件中至少存在一个 location /，即表示客户端浏览器请求的

URL 为：域名+/，如果 location /newindex/，则表示客户端浏览器请求的 URL 为：域名+/newindex/。常见 Location 匹配 URL 的方式如下：

=	字面精确匹配；
^~	最大前缀匹配；
/	不带任何前缀：最大前缀匹配；
~	大小写相关的正则匹配；
~*	大小写无关的正则匹配；
@	location 内部重定向的变量。

其中 Location =、^~、/属于普通字符串匹配，Location ~、~*属于正则表达式匹配，Location 优先级与其在 Nginx.conf 配置文件中的先后顺序无关。

Location = 精确匹配会第一个被处理，如果发现精确匹配，Nginx 则停止搜索其他任何 Location 的匹配。

普通字符匹配，正则表达式规则和完整 URL 规则将被优先和查询匹配，^~为最大前缀匹配，如果匹配到该规则，Nginx 则停止搜索其他任何 Location 的匹配，否则 nginx 会继续处理其他 location 指令。

正则匹配"~"和"~*"，如果找到相应的匹配，则 Nginx 停止搜索其他任何 Location 的匹配；当没有正则表达式或者没有正则表达式被匹配的情况下，那么匹配程度最高的逐字匹配指令会被使用。

Location 规则匹配优先级总结如下：

(location =) > (location 完整路径) > (location ^~ 路径) > (location ~|~* 正则顺序) > (location 部分起始路径) > (/)

如下为 Nginx Location 规则案例演示：

```
location    = / {
   [ configuration    L1 ]
   #只会匹配/，优先级比 Location /低。
}
location    = /index.html {
   [ configuration    L2 ]
   #只会匹配/index.html，优先级最高。
}
location    / {
   [ configuration L3 ]
   #匹配任何请求，因为所有请求都是以"/"开始；
   #但是更长字符匹配或者正则表达式匹配会优先匹配，优先级最低。
}
location = /images/ {
   [ configuration L4 ]
   #匹配任何以/images/开始的请求，并停止匹配其它 location；
```

```
}

location ~* \.(html|txt|gif|jpg|jpeg)$ {

   [ configuration L5]

   # 匹配以 html、txt、gif、jpg、jpeg 结尾的 URL 文件请求；

   # 但是所有/images/目录的请求将由 [Configuration L4]处理。

}
```

浏览器发起 HTTP Request URI 案例与 Location 规则案例匹配如下：

- / ->匹配 configuration L3；
- /index.html 匹配 configuration L2；
- /images/匹配 configuration L4；
- /images/logo.png 匹配 configuration L4；
- /img/test.jpg 匹配 configuration L5。

企业生产环境中无需在 Nginx.conf 配置文件中同时添加五种规则匹配 ,如下为企业生产环境 Nginx Location 部分配置代码：

```
location /

{

      root /var/www/html/；

      expires          60d；

}
```

```
location ~ .*\.(gif|jpg|jpeg|bmp|png|ico|txt|js|css)$
{
    root /var/www/html/ ;
    expires      60d ;
}
location ~ .*\.(jsp|php|cgi|do)$
{
    root /var/www/html/ ;
    proxy_pass http://linux_web ;
    proxy_http_version 1.1 ;
    proxy_set_header Connection "" ;
    proxy_set_header Host    $host ;
    proxy_set_header X-Real-IP $remote_addr ;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for ;
}
location =/newindex.html
{
    root /var/www/newwww/ ;
    expires      60d ;
```

```
}
```

14.7 企业实战 Nginx 动静分离架构

Nginx 动静分离简单来说就是把动态跟静态请求分开，不能理解成只是单纯的把动态页面和静态页面物理分离。严格意义上说应该是动态请求跟静态请求分开，可以理解成使用 Nginx 处理静态页面，Tomcat、Resin、PHP、ASP 处理动态页面。

动静分离从目前实现角度来讲大致分为两种，一种是纯粹的把静态文件独立成单独的域名，放在独立的服务器上，也是目前主流推崇的方案；另外一种方法就是动态跟静态文件混合在一起发布，通过 Nginx 来分开。

Nginx 线上 WEB 服务器动静分离及 Nginx.conf 完整配置文件代码如下：

```
user www www；

worker_processes 8；

worker_cpu_affinity 00000001 00000010 00000100 00001000 00010000

00100000 01000000 10000000；

pid /usr/local/nginx/nginx.pid；

worker_rlimit_nofile 102400；

events

{

use epoll；
```

```
worker_connections 102400；
}
http
{
  include          mime.types；
  default_type   application/octet-stream；
  FastCGI_intercept_errors on；
  charset   utf-8；
  server_names_hash_bucket_size 128；
  client_header_buffer_size 4k；
  large_client_header_buffers 4 32k；
  client_max_body_size 300m；
  sendfile on；
  tcp_nopush       on；
  keepalive_timeout 60；
  tcp_nodelay on；
  client_body_buffer_size   512k；
  proxy_connect_timeout      5；
  proxy_read_timeout          60；
```

```
    proxy_send_timeout          5 ;
    proxy_buffer_size           16k ;
    proxy_buffers               4 64k ;
    proxy_busy_buffers_size 128k ;
    proxy_temp_file_write_size 128k ;
    gzip on ;
    gzip_min_length   1k ;
    gzip_buffers      4 16k ;
    gzip_http_version 1.1 ;
    gzip_comp_level 2 ;
    gzip_types        text/plain application/x-javascript text/css application/xml ;
    gzip_vary on ;
log_format   main   '$remote_addr - $remote_user [$time_local] "$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent"   $request_time' ;
upstream   jvm_web1 {
     server             192.168.149.130:8080        weight=1        max_fails=2
fail_timeout=30s ;
     server             192.168.149.130:8081        weight=1        max_fails=2
```

```
fail_timeout=30s；

}

include vhosts.conf；

}
```

如下为 vhosts.conf 配置文件中内容：

```
server

  {

    listen          80；

    server_name www.jf1.com；

    index   index.jsp index.html index.htm；

    root   /data/webapps/www1；

    location /

    {

          proxy_next_upstream http_502 http_504 error timeout invalid_header；

          proxy_set_header   Host   $host；

          proxy_set_header   X-Real-IP   $remote_addr；

          proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for；

          proxy_pass http://jvm_web1；

    }
```

```
    location ~ .*\.(php|jsp|cgi|shtml)?$
    {
            proxy_set_header Host    $host ;
            proxy_set_header X-Real-IP $remote_addr ;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for ;
            proxy_pass http://jvm_web1 ;
    }
    location ~ .*\.(html|htm|gif|jpg|jpeg|bmp|png|ico|txt|js|css)$
    {
          root /data/webapps/www1 ;
          expires          30d ;
    }
          access_log    /data/logs/jvm_web1/access.log main ;
          error_log     /data/logs/jvm_web1/error.log    crit ;
}
```

配置文件代码中：location ~ .*\.(php|jsp|cgi|shtml)表示匹配动态页面请求，然后将请求 proxy_pass 到后端服务器，而 location ~ .*\.(html|htm|gif|jpg|jpeg |ico|txt|js|css)表示匹配静态页面请求本地返回。

检查 Nginx 配置是否正确即可，然后测试动静分离是否成功，在 192.168.149.130 服务器启动 8080、8081 Tomcat 服务或者 LAMP 服务，删除后端 Tomcat 或者 LAMP 服务器上的某个静态文件，测试是否能访问该文件，如果可以访问说明静态资源 Nginx 直接返回了，如果不能访问，则证明动静分离不成功。

14.8 企业实战 LNMP 高性能服务器

公共网关接口”(Common Gateway Interface，CGI)，是 HTTP 服务器与本机或者其它机器上的程序进行通信的一种工具，其程序须运行在网络服务器上。CGI 可以用任何一种语言编写，只要这种语言具有标准输入、输出和环境变量，如 php、perl、tcl 等。

Fast-CGI 是从 CGI 发展改进而来的。传统 CGI 接口方式的主要缺点是性能很差，因为每次 HTTP 服务器遇到动态程序时都需要重新启动脚本解析器来执行解析，然后将结果返回给 HTTP 服务器。这在处理高并发访问时几乎是不可用的。另外传统的 CGI 接口方式安全性也很差，现在已经很少使用。

FastCGI 接口方式采用 C/S 结构，可以将 HTTP 服务器和脚本解析服务器分开，同时在脚本解析服务器上启动一个或者多个脚本解析守护进程。当 HTTP 服务器每次遇到动态程序时，可以将其直接交付给 Fast-CGI 进程来执行，然后将得到的结果返回给浏览器。这种方式可以让 HTTP 服务器专一地处理静态请求或者将动态脚本服务器的结果返回给客户端，这在很大程度上提高了整个应用系统的性能。

FastCGI 是语言无关的、可伸缩架构的 CGI 开放扩展，将 CGI 解释器进程保持在内

存中 ,以此获得较高的性能。FastCGI 是一个协议 ,php-fpm 实现了这个协议 ,php-fpm 的 FastCGI 协议需要有进程池，php-fpm 实现的 FastCGI 进程叫 php-cgi，所以 php-fpm 其实是他自身的 FastCGI 或 php-cgi 进程管理器。

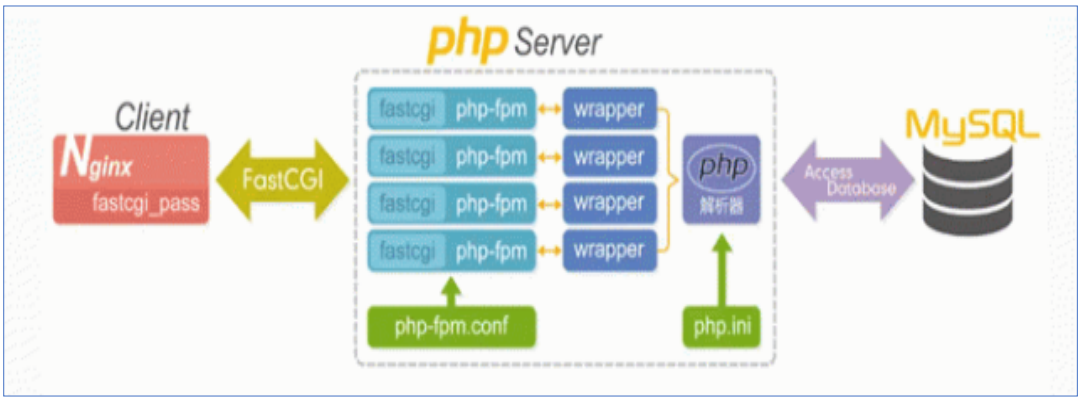
LNMP WEB 架构中，Nginx 为一款高性能 Web 服务器，本身是不能处理 PHP 的，当接收到客户端浏览器发送 HTTP Request 请求时,Nginx 服务器响应并处理 web 请求，静态资源 CSS、图片、视频、TXT 等静态文件请求，Nginx 服务器可以直接处理并回应。

但是 PHP 动态页面请求 Nginx 不能直接处理，Nginx 服务器会将 PHP 网页脚本通过接口传输协议（网关协议）PHP-FCGI（Fast-CGI）传输给 PHP-FPM（进程管理程序),PHP-FPM 不做处理 ,然后 PHP-FPM 调用 PHP 解析器进程 ,PHP 解析器解析 PHP 脚本信息。PHP 解析器进程可以启动多个，可以实现多进行并发执行。

PHP 解释器将解析后的脚本返回到 PHP-FPM，PHP-FPM 再通过 Fast-CGI 的形式将脚本信息传送给 Nginx,Nginx 服务器再通过 Http Response 的形式传送给浏览器,浏览器再进行解析与渲染然后进行呈现。

如图 14-5 所示：





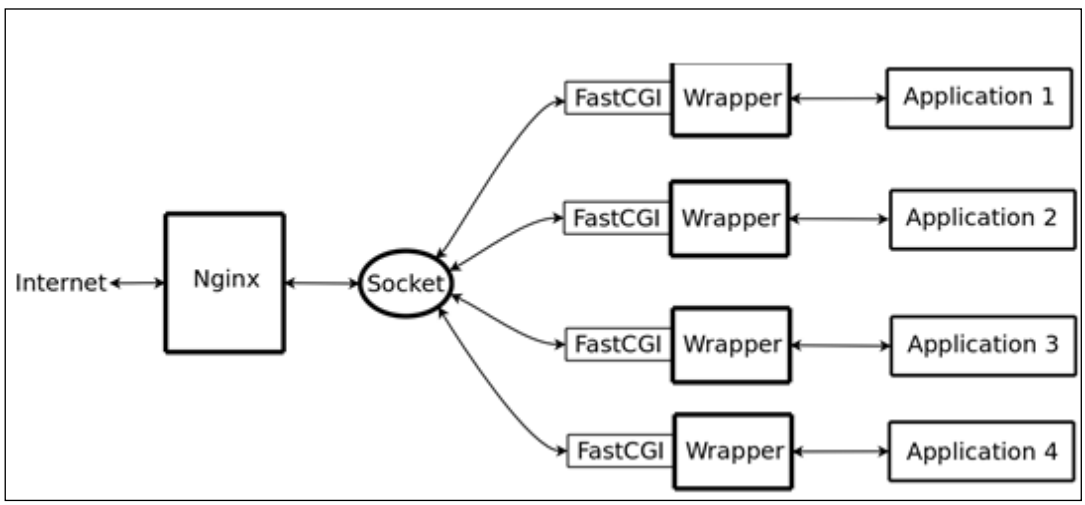


图 14-5 Nginx+FastCGI 通信原理图

CGI、FAST-CGI、PHP-CGI、PHP-FPM 概念总结：

- CGI：Common gateway interface 通用网关接口；
- FastCGI：Fast common gateway interface 快速通用网关接口；
- PHP-FPM：PHP-Fast CGI Process Manager CGI，是 FastCGII 协议的实现并且带进程管理器；

- PHP-CGI 是解析 PHP 代码的程序，属于 PHP 程序解释器，只负责解析请求，不负责进程管理；

- PHP-FPM 是 PHP-CGI 进程管理器，可以有效控制内存和进程、可以平滑重载 PHP 配置。

企业级 LNMP（Nginx+PHP（FastCGI）+MySQL）主流架构配置方法如下，分别安装 Nginx、MYSQL、PHP 服务，步骤如下：

（1） Nginx 安装配置

```
wget -c http://nginx.org/download/nginx-1.12.0.tar.gz
tar -xzf nginx-1.12.0.tar.gz
cd nginx-1.12.0
useradd www ; ./configure --user=www --group=www --prefix=/usr/local/nginx --with-http_stub_status_module --with-http_ssl_module
make
make install
```

（2） MYSQL 安装配置

```
yum install cmake ncurses-devel ncurses –y
wget http://down1.chinaunix.net/distfiles/mysql-5.5.20.tar.gz
```

```
tar    -xzf    mysql-5.5.20.tar.gz

cd    mysql-5-5.20

cmake    . -DCMAKE_INSTALL_PREFIX=/usr/local/mysql55 \

-DMYSQL_UNIX_ADDR=/tmp/mysql.sock \

-DMYSQL_DATADIR=/data/mysql \

-DSYSCONFDIR=/etc \

-DMYSQL_USER=mysql \

-DMYSQL_TCP_PORT=3306 \

-DWITH_XTRADB_STORAGE_ENGINE=1 \

-DWITH_INNOBASE_STORAGE_ENGINE=1 \

-DWITH_PARTITION_STORAGE_ENGINE=1 \

-DWITH_BLACKHOLE_STORAGE_ENGINE=1 \

-DWITH_MYISAM_STORAGE_ENGINE=1 \

-DWITH_READLINE=1 \

-DENABLED_LOCAL_INFILE=1 \

-DWITH_EXTRA_CHARSETS=1 \

-DDEFAULT_CHARSET=utf8 \

-DDEFAULT_COLLATION=utf8_general_ci \

-DEXTRA_CHARSETS=all \
```

```
-DWITH_BIG_TABLES=1 \

-DWITH_DEBUG=0

make

make install
```

（3） PHP 安装配置

```
wget http://museum.php.net/php5/php-5.3.10.tar.gz

yum -y install gd curl curl-devel libjpeg libjpeg-devel libpng libpng-devel freetype freetype-devel libxml2 libxml2-devel

cd php-5.3.10

./configure --prefix=/usr/local/php5 --enable-fpm --enable-debug --with-gd --with-jpeg-dir --with-png-dir --with-freetype-dir --enable-mbstring --with-curl --with-mysql=/usr/local/mysql55/ --with-mysqli=/usr/local/mysql55/bin/mysql_config --with-config-file-path=/usr/local/php5/etc

make

make install

cp php.ini-development /usr/local/php5/etc/php.ini

cp /usr/local/php5/etc/php-fpm.conf.default /usr/local/php5/etc/php-fpm.conf
```

```
/usr/local/php5/sbin/php-fpm

cp sapi/fpm/init.d.php-fpm /etc/init.d/php-fpm
```

（4） Nginx 配置文件配置

```
server {

 include port.conf ;

 server_name www.jfedu.net jfedu.net ;

  location / {

 index index.html index.php ;

 root /usr/local/nginx/html ;

 }

 location   ~ \.php$ {

                root              html ;

                FastCGI_pass     127.0.0.1:9000 ;

                FastCGI_index    index.php ;

                FastCGI_param    SCRIPT_FILENAME    html$FastCGI_script_name ;
```

```
        include        FastCGI_params;

    }

}
```

（5） 测试 LNMP 架构测试，创建 index.php 测试页面，如图 14-6 所示：

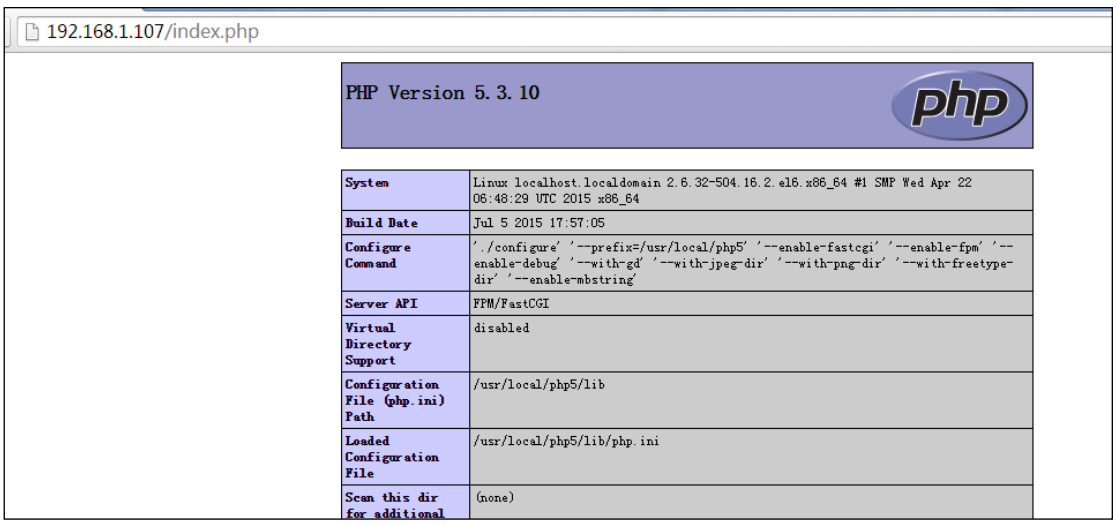


图 14-6 LNMP 企业实战测试页面

14.9 Nginx Rewrite 规则详解一

Rewirte 规则也称为规则重写，主要功能是实现浏览器访问 HTTP URL 的跳转，其正则表达式是基于 Perl 语言。通常而言，几乎所有的 WEB 服务器均可以支持 URL 重写。Rewrite URL 规则重写的用途：

- 对搜索引擎优化（Search Engine Optimization，SEO）友好，利于搜索引擎抓取网站页面；

- 隐藏网站 URL 真实地址，浏览器显示更加美观；
- 网站变更升级，可以基于 Rewrite 临时重定向到其他页面。

Nginx Rewrite 规则使用中有三个概念需要理解，分别是：Rewrite 结尾标识符、Rewrite 规则常用表达式、Nginx Rewrite 变量，如下为三个概念的详解：

（1） Nginx Rewrite 结尾标识符，用于 Rewrite 规则末尾，表示规则的执行属性。

last ：相当于 Apache 里的(L)标记，表示完成 rewrite 匹配；
break：本条规则匹配完成后，终止匹配，不再匹配后面的规则。
redirect：返回 302 临时重定向，浏览器地址会显示跳转后的 URL 地址。
permanent：返回 301 永久重定向，浏览器地址栏会显示跳转后的 URL 地址。
其中 last 和 break 用来实现 URL 重写时，浏览器地址栏 URL 地址不变。

（2） Nginx Rewrite 规则常用表达式，主要用于匹配参数、字符串及过滤设置。

.	匹配任何单字符；
[word]	匹配字符串:word；
[^word]	不匹配字符串:word；
jfedu|jfteach	可选择的字符串:jfedu|jfteach；
?	匹配 0 到 1 个字符；
*	匹配 0 到多个字符；
+	匹配 1 到多个字符；
^	字符串开始标志；

$	字符串结束标志；
\n	转义符标志。

（3） Nginx Rewrite 变量，常用于匹配 HTTP 请求头信息、浏览器主机名、URL 等。

HTTP headers:HTTP_USER_AGENT, HTTP_REFERER, HTTP_COOKIE, HTTP_HOST,

HTTP_ACCEPT；

connection & request: REMOTE_ADDR, QUERY_STRING；

server internals: DOCUMENT_ROOT, SERVER_PORT, SERVER_PROTOCOL；

system stuff: TIME_YEAR, TIME_MON, TIME_DAY。

详解如下:

HTTP_USER_AGENT	用户使用的代理，例如浏览器；
HTTP_REFERER	告知服务器，从哪个页面来访问的；
HTTP_COOKIE	客户端缓存，主要用于存储用户名和密码等信息；
HTTP_HOST	匹配服务器 ServerName 域名；
HTTP_ACCEPT	客户端的浏览器支持的 MIME 类型；
REMOTE_ADDR	客户端的 IP 地址
QUERY_STRING	URL 中访问的字符串；
DOCUMENT_ROOT	服务器发布目录；
SERVER_PORT	服务器端口；
SERVER_PROTOCOL	服务器端协议；

```
TIME_YEAR                    年；

TIME_MON                     月；

TIME_DAY                     日；
```

（4） Nginx Rewrite 以下配置均配置在 nginx.conf 或者 vhosts.conf 中，企业中常用的 Nginx Rewrite 案例如下：

1. 将 jfedu.net 跳转至 www.jfedu.net。

```
if ($host = 'jfedu.net' ) {
        rewrite ^/(.*)$ http://www.jfedu.net/$1    permanent ;
}
```

2. 访问 www.jfedu.net 跳转 www.test.com/new.index.html。

```
rewrite     ^/$    http://www.test.com/index01.html    permanent ;
```

3. 访问/jfedu/test01/跳转至/newindex.html，浏览器地址不变。

```
rewrite     ^/jfedu/test01/$        /newindex.html        last ;
```

4. 多域名跳转到 www.jfedu.net。

```
if ($host !=    'www.jfedu.net'    ) {
rewrite ^/(.*)$    http://www.jfedu.net/$1    permanent ;
}
```

5. 访问文件和目录不存在跳转至 index.php。

```
if ( !-e $request_filename )
```

```
{

rewrite    ^/(.*)$    /index.php    last ;

}
```

6. 目录对换 /xxxx/123456 ====> /xxxx?id=123456。

```
rewrite       ^/(.+)/(\d+)          /$1?id=$2            last ;
```

7. 判断浏览器 User Agent 跳转。

```
if( $http_user_agent  ~ MSIE)

{

rewrite ^(.*)$ /ie/$1 break ;

}
```

8. 禁止访问以.sh,.flv,.mp3 为文件后缀名的文件。

```
location ~ .*\.(sh|flv|mp3)$

{

      return 403 ;

}
```

9. 将移动用户访问跳转至移动端。

```
if ( $http_user_agent ~* "(Android)|(iPhone)|(Mobile)|(WAP)|(UCWEB)" )

{

rewrite ^/$          http://m.jfedu.net/          permanent ;
```

```
}
```

10. 匹配 URL 访问字符串跳转。

```
if ($args ~* tid=13){
  return 404；
}
```

11. 访问/10690/jfedu/123 跳转至/index.php?tid/10690/items=123 ,[0-9]表示任意一个数字，+表示多个，(.+)表示任何多个字符。

```
rewrite     ^/([0-9]+)/jfedu/(.+)$        /index.php?tid/$1/items=$2        last；
```

14.10 Nginx 日志切割案例

Nginx WEB 服务器每天会产生大量的访问日志，而且不会自动地进行切割，如果持续天数访问，将会导致该 access.log 日志文件容量非常大，不便于 SA 查看相关的网站异常日志。

可以基于 Shell 脚本结合 Crontab 计划任务对 Nginx 日志进行自动、快速的切割，其切割的方法使用 mv 命令即可，如图 14-8 所示。

```
#!/bin/bash
#auto mv nginx log shell
#by author jfedu.net
S_LOG=/usr/local/nginx/logs/access.log
```

```
D_LOG=/data/backup/`date +%Y%m%d`

echo -e "\033[32mPlease wait start cut shell scripts...\033[1m"

sleep 2

if [ ! -d $D_LOG ] ; then

        mkdir -p    $D_LOG

fi

mv $S_LOG    $D_LOG

kill    -USR1    `cat /usr/local/nginx/logs/nginx.pid`

echo "----------------------------------------"

echo "The Nginx log Cutting Successfully!"

echo "You can access backup nginx log $D_LOG/access.log files."
```

```
+ S_LOG=/usr/local/nginx/logs/access.log
++ date +%Y%m%d
+ D_LOG=/data/backup/20170525
+ echo -e '\033[32mPlease wait start cut shell scripts...\033[1m'
Please wait start cut shell scripts...
+ sleep 2
+ '[' '!' -d /data/backup/20170525 ']'
+ mv /usr/local/nginx/logs/access.log /data/backup/20170525
++ cat /usr/local/nginx/logs/nginx.pid
+ kill -USR1 48298
+ echo ----------------------------------------
----------------------------------------
+ echo 'The Nginx log Cutting Successfully!'
The Nginx log Cutting Successfully!
+ echo 'You can access backup nginx log /data/backup/20170525/access.log
You can access backup nginx log /data/backup/20170525/access.log files.
```

图 14-8 Nginx 日志切割

将如上脚本内容写入 auto_nginx_log.sh 文件，crontab /var/spool/cron/root 文件中添加如下代码，每天凌晨自动切割日志：

```
0  0  *  *  *  /bin/sh  /data/sh/auto_nginx_log.sh >>/tmp/nginx_cut.log
2>&1
```

14.11 Nginx 防盗链配置案例配置

防盗链的含义是网站内容本身不在自己公司的服务器上，而通过技术手段，直接在调用其他公司的服务器网站数据，而向最终用户提供此内容。一些小网站盗链高访问量网站的音乐、图片、软件的链接，然后放置在自己的网站中，通过这种方法盗取高访问量网站的空间和流量。

网站每天访问量很大，而且占用了很多不必要的带宽，浪费资源，所以必须采取一些限制措施。防盗链其实就是采用服务器端编程技术，通过 URL 过滤、主机名等实现的防止盗链的软件。

例如 http://www.jfedu.net/linux/页面，如果没有配置防盗链，别人就能轻而易举的在其的网站上引用该页面。Nginx 防盗链配置代码如下：

```
server {
        listen          80；
        server_name   jfedu.net www.jfedu.net；
        location / {
            root    html/b；
            index   index.html index.htm；
```

```
        }
        location ~* \.(gif|jpg|png|swf|flv)$ {
                valid_referers none blocked   jfedu.net   *.jfedu.net ;
                root     html/b ;
        if ($invalid_referer) {
                #rewrite ^/ http://www.jfedu.net/403.html ;
                return 403 ;
                }
        }
}
```

Nginx 防盗链参数详解：

valid_referers 表示可用的 referers 设置

none 表示没有 referers，直接通过浏览器或者其他工具访问。

blocked 表示有 referers，但是被代理服务器或者防火墙隐藏；

jfedu.net 表示通过 jfedu.net 访问的 referers；

.jfedu.net 表示通过.jfedu.net 访问的 referers，*表示任意 host 主机。

除了以上方法，按照如下方法设置也可以实现防盗链：

```
location ~* \.(gif|jpg|png|swf|flv)$
if ($host !=’ *.jfedu.net’ ) {
```

```
        return 403；

}
```

防盗链测试，找另外一台测试服务器，基于 Nginx 发布如下 test.html 页面，代码如下，去调用 www.jfedu.net 官网的 test.png 图片，由于 www.jfedu.net 官网设置了防盗链，所以无法访问该图片。

```
<html>

<h1>TEST Nginx PNG</h1>

<img src="http://www.jfedu.net/test.png">

</html>
```

默认没有配置 Nginx 防盗链，网站正常调用 www.jfedu.net 的 logo 图片，访问如图 14-9 所示：

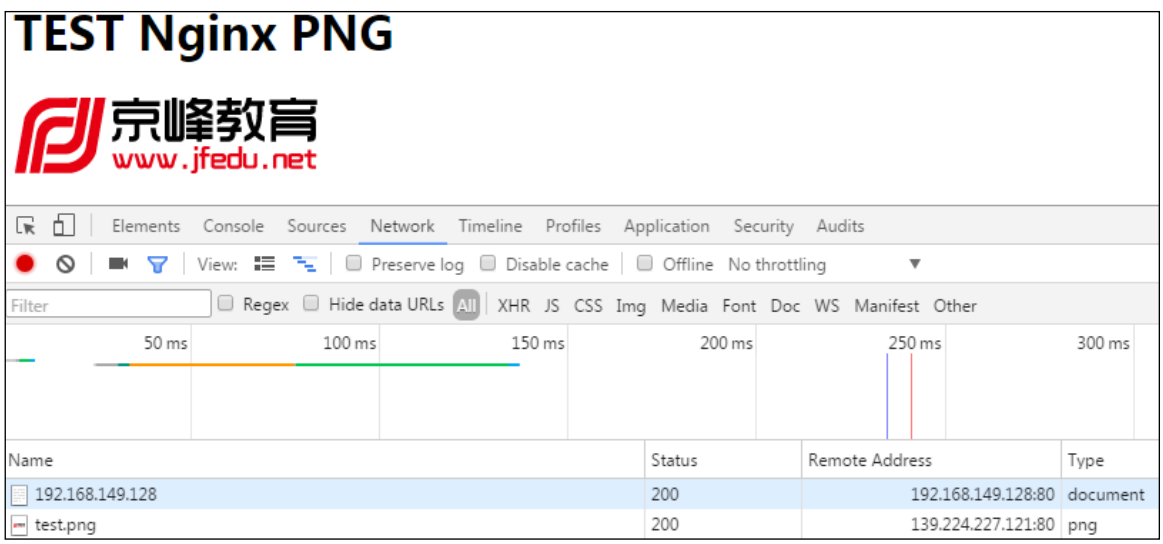


图 14-9 Nginx 无防盗链正常调用图片

配置 Nginx 防盗链 ,网站无法正常调用 www.jfedu.net 的 logo 图片 ,访问如图 14-

10 所示：

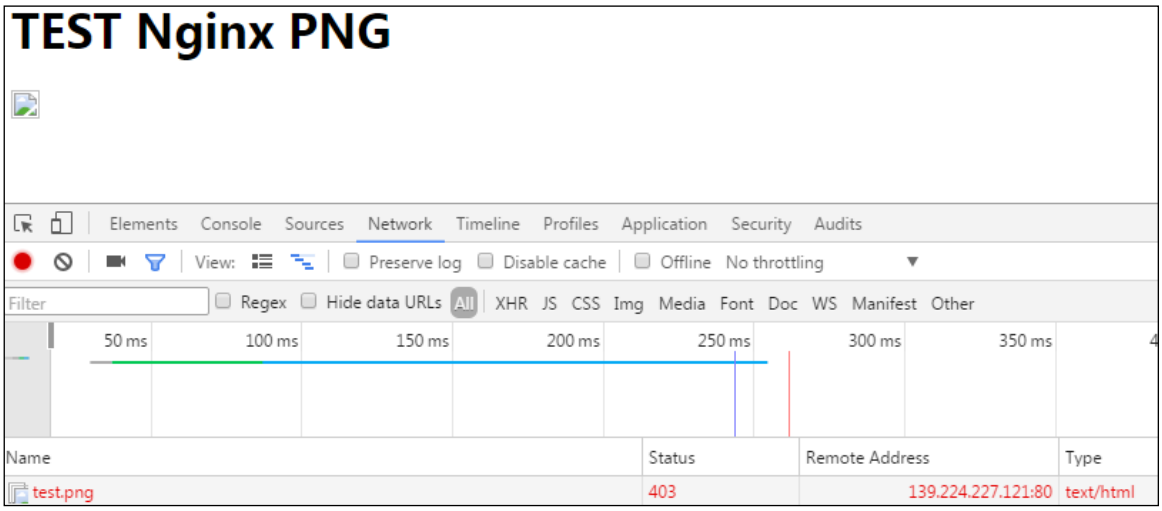


图 14-10 Nginx 防盗链 403 禁止访问

第15章　大数据备份企业实战

随着互联网不断的发展,企业对运维人员的能力要求也越来越高,尤其是要求运维人员能处理各种故障、专研自动化运维技术、云计算机、虚拟化等，满足公司业务的快速发展。

本章向读者介绍数据库备份方法、数量量 2T 及以上级别数据库备份方案、xtrabackup 企业工具案例演示、数据库备份及恢复实战等。

15.1 企业级数据库备份实战

在日常的运维工作中，数据是公司非常重要的资源，尤其是数据库的相关信息，如

果数据丢了，少则损失几千元，高则损失上千万。所以在运维工作中要及时注意网站数据的备份，尤其要及时对数据库进行备份。

企业中如果数据量上 T 级别，维护和管理是非常复杂的，尤其是对数据库进行备份操作。

15.2 数据库备份方法及策略

企业中 MySQL 数据库备份最常用的方法如下：

- 直接 cp 备份
- Sqlhotcopy
- 主从同步复制
- Mysqldump 备份
- Xtrabackup 备份

Mysqldump 和 Xtrabackup 均可以备份 MYSQL 数据，如下为 Mysqldump 工具使用方法：

通常小于 100G 的 MYSQL 数据库可以使用默认 Mysqldump 备份工具进行备份，如果超过 100G 的大数据，由于 Mysqldump 备份方式是采用的逻辑备份，最大的缺陷是备份和恢复速度较慢。

基于 Mysqldump 备份耗时会非常长，而且备份期间会锁表，锁表直接导致数据库只能访问 Select，不能执行 Insert、Update 等操作，进而导致部分 WEB 应用无法写入

新数据。

如果是 Myisam 引擎表，当然也可以执行参数--lock-tables=false 禁用锁表，但是有可能造成数据信息不一致。

如果是支持事务的表，例如 InnoDB 和 BDB，--single-transaction 参数是一个更好的选择，因为它不锁定表。

```
mysqldump -uroot -p123456 --all-databases --opt --single-transaction > 2022all.sql
```

其中--opt 快捷选项，等同于添加--add-drop-tables --add-locking --create-option --disable-keys --extended-insert --lock-tables --quick --set-charset 选项。本选项能让 Mysqldump 很快的导出数据，并且导出的数据能很快导回。该选项默认开启，但可以用--skip-opt 禁用。

如果运行 Mysqldump 没有指定--quick 或 --opt 选项，则会将整个结果集放在内存中。如果导出大数据库的话可能会导致内存溢出而异常退出。

15.3 Xtrabackup 企业实战

Mysql 冷备、Mysqldump、Mysql 热拷贝均不能实现对数据库进行增量备份，在实际环境中增量备份非常的实用，如果数据量小于 100G，存储空间足够，可以每天进行完整备份，如果每天产生的数据量大，需要定制数据备份策略例如：每周日使用完整备份，周一到周六使用增量备份，或者每周六完整备份，周日到周五使用增量备份。

Percona-xtrabackup 是为实现增量备份而生一款主流备份工具，Xtrabackup 有两个主要的工具，分别为：xtrabackup、innobackupex。

Percona XtraBackup 是 Percona 公司开发的一个用于 MySQL 数据库物理热备的备份工具，支持 MySQl、Percona Server 及 MariaDB，开源免费，是目前互联网数据库备份最主流的工具之一。

Xtrabackup 只能备份 InnoDB 和 XtraDB 两种数据引擎的表，而不能备份 MyISAM 数据表，Innobackupex-1.5.1 则封装了 Xtrabackup，是一个封装好的脚本，使用该脚本能同时备份处理 innodb 和 Myisam，但在处理 Myisam 时需要加一个读锁。

XtraBackup 备份原理，Innobackupex 在后台线程不断追踪 InnoDB 的日志文件，然后复制 InnoDB 的数据文件。数据文件复制完成之后，日志的复制线程也会结束。这样就得到了不在同一时间点的数据副本和开始备份以后的事务日志。完成上面的步骤之后，就可以使用 InnoDB 崩溃恢复代码执行事务日志(Redo log)，以达到数据的一致性。其备份优点如下：

- 备份速度快，物理备份更加可靠；
- 备份过程不会打断正在执行的事务，无需锁表；
- 能够基于压缩等功能节约磁盘空间和流量；
- 自动备份校验；
- 还原速度快；
- 可以流传将备份传输到另外一台机器上；

- 节约磁盘空间和网络带宽。

Innobackupex 工具的备份过程原理，如图 16-1 所示：

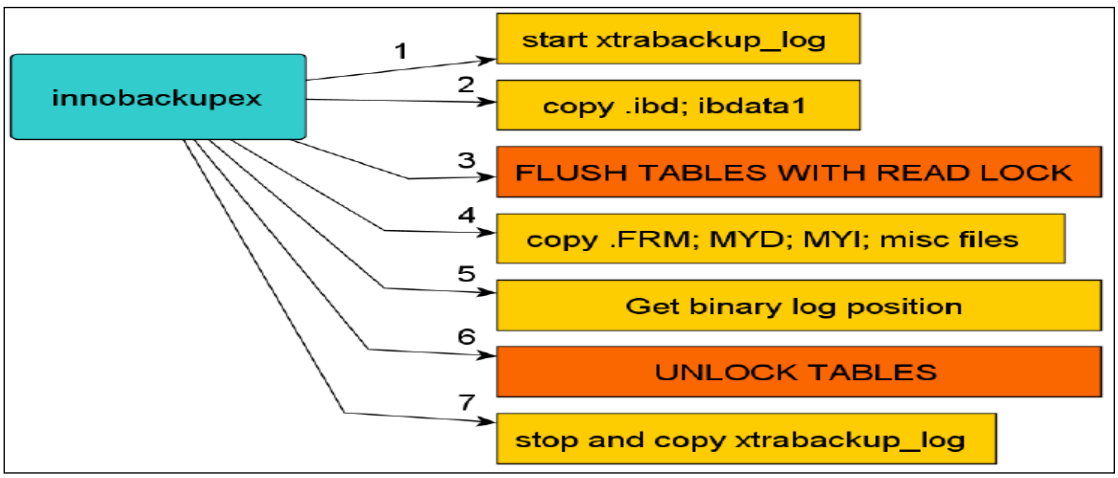


图 16-1 Innobackupex 备份过程

Innobackupex 备份过程中首先启动 Xtrabackup_log 后台检测的进程，实时检测 Mysql redo 的变化，一旦发现 Redo 有新的日志写入，立刻将日志写入到日志文件 Xtrabackup_log 中，并复制 Innodb 的数据文件和系统表空间文件 idbdata1 到备份目录。

Innode 引擎表备份完之后，执行 Flush table with read lock 操作进行 MyIsam 表备份。拷贝.frm .myd .myi 文件，并且在这一时刻获得 binary log 的位置，将表进行解锁 unlock tables，停止 Xtrabackup_log 进程，完整整个数据库的备份。

15.4 Percona-xtrabackup 备份实战

基于 Percona-xtrabackup 备份，需要如下几个步骤：

（1） 官网下载 Percona-Xtrabackup

Percona 官方 wiki 使用帮助：

http://www.percona.com/docs/wiki/percona-xtrabackup:start

wget http://www.percona.com/redir/downloads/XtraBackup/XtraBackup-2.0.0/binary/Linux/x86_64/percona-xtrabackup-2.0.0.tar.gz

（2） Percona-xtrabackup 软件安装方法，cp innobackupex、xtrabackup、xtrabackup_51 xtrabackup_55 工具到/usr/bin 目录下，代码如下：

```
tar zxvf percona-xtrabackup-2.0.0.tar.gz
cp  percona-xtrabackup-2.0.0/bin/innobackupex /usr/bin/innobackupex
cp  percona-xtrabackup-2.0.0/bin/xtrabackup /usr/bin/xtrabackup
cp  percona-xtrabackup-2.0.0/bin/xtrabackup_51 /usr/bin/xtrabackup_51
cp  percona-xtrabackup-2.0.0/bin/xtrabackup_55 /usr/bin/xtrabackup_55
```

Percona-xtrabackup

（3） MYSQL 数据库全备份，如图 16-2（a）、16-2（b）所示：

```
innobackupex  --user=root  --password=123456  /data/backup/mysql/
```

```
[root@localhost ~]#
[root@localhost ~]# ll /data/backup/mysql/
总用量 0
[root@localhost ~]#
[root@localhost ~]# innobackupex --user=root --password=123456 /data/backup/mysql/

InnoDB Backup Utility v1.5.1-xtrabackup; Copyright 2003, 2009 Innobase Oy
and Percona Inc 2009-2012.  All Rights Reserved.

This software is published under
the GNU GENERAL PUBLIC LICENSE Version 2, June 1991.

141220 00:10:18  innobackupex: Starting mysql with options:  --password=xxxxxxxx --user='r
141220 00:10:18  innobackupex: Connected to database with mysql child process (pid=7647)
141220 00:10:24  innobackupex: Connection to database server closed
IMPORTANT: Please check that the backup run completes successfully.
           At the end of a successful backup run innobackupex
           prints "completed OK!".
```

图 16-2（a） Innobackupex 完整备份

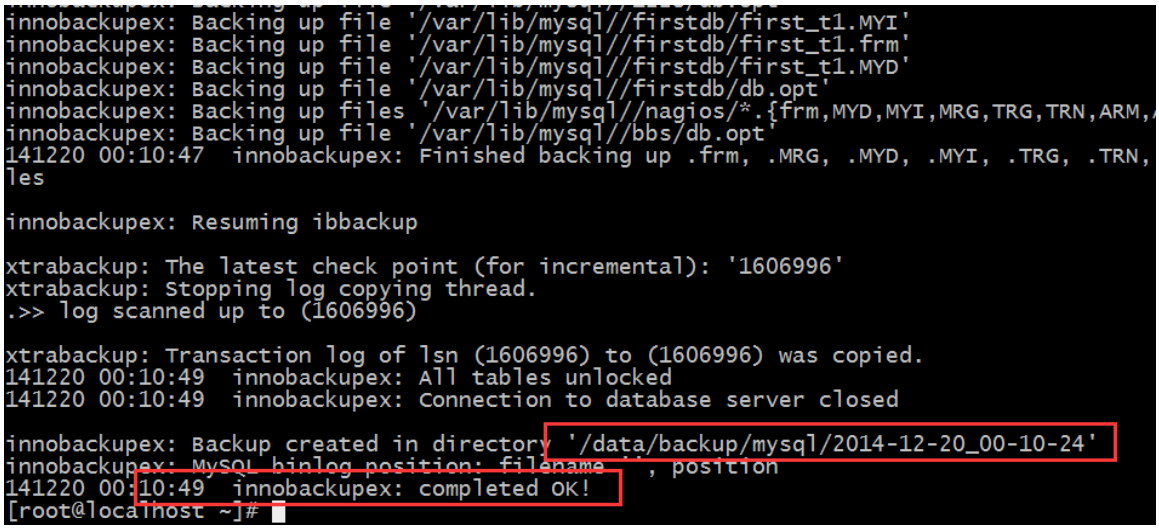


图 16-2（b） Innobackupex 完整备份

（4） Innobackupex 数据库恢复 ,恢复前先保证数据一致性 ,执行如下命令 ,如图 16-3 所示：

```
innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456 --
apply-log  /data/backup/mysql/2014-12-20_00-10-24
```

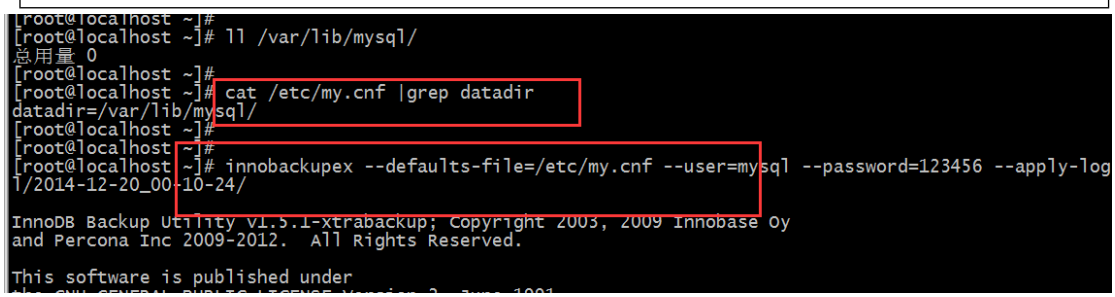


图 16-3 Innobackupex apply-log 恢复

通常数据库备份完成后，数据尚不能直接用于恢复操作，因为备份的数据时是一个过程，在备份过程中，有任务会写入数据，可能会包含尚未提交的事务或已经提交但尚未同步至数据文件中的事务。

因此此时数据文件仍处理不一致状态 ,基于--apply-log 可以通过回滚未提交的事务及同步已经提交的事务至数据文件使数据文件处于一致性状态，方可进行恢复数据。

Apply-log 过程可以在任何机器上运行，没有强制在线上或者备份库上运行，可以把备份复制在闲置的服务器上去运行，以此来降低备份库的压力，必须保证 backup 和 apply-log 所使用的 mysqlbackup 的版本要一致。

（5） 删除原数据目录/var/lib/mysql 数据，使用参数--copy-back 恢复完整数据，授权 mysql 用户给所有的数据库文件，如图 16-4 所示：

```
rm  -rf  /var/lib/mysql/*

innobackupex --defaults-file=/etc/my.cnf --user=mysql --password=123456 --

copy-back /data/backup/mysql/2014-12-20_00-10-24/

chown  –R  mysql:mysql  /var/lib/mysql/
```

```
drwxr-xr-x 2 root root    36864 12月 20 00:23 shareku
drwxr-xr-x 2 root root     4096 12月 20 00:23 test
drwxr-xr-x 2 root root     4096 12月 20 00:23 test01
drwxr-xr-x 2 root root     4096 12月 20 00:23 web02
drwxr-xr-x 2 root root     4096 12月 20 00:23 wordpress
drwxr-xr-x 2 root root     4096 12月 20 00:23 wugk1
-rw-r--r-- 1 root root       77 12月 20 00:23 xtrabackup_checkpoints
[root@localhost mysql]# chown -R mysql:mysql /var/lib/mysql/
[root@localhost mysql]#
[root@localhost mysql]#
[root@localhost mysql]# /usr/local/mysql/bin/mysqld_safe --user=mysql&
[1] 11335
[root@localhost mysql]# 141220 00:24:49 mysqld_safe Logging to '/var/log/my
141220 00:24:49 mysqld_safe Starting mysqld daemon with databases from /var

[root@localhost mysql]#
[root@localhost mysql]# ps -ef |grep mysql
root     11335  2948  0 00:24 pts/5    00:00:00 /bin/sh /usr/local/mysql/bi
mysql    11497 11335  0 00:24 pts/5    00:00:00 /usr/local/mysql/bin/mysqld
/mysql/ --plugin-dir=/usr/local/mysql/lib/plugin --user=mysql --log-error=/
mysqld.pid --socket=/var/lib/mysql/mysql.sock
root     11518  2948  0 00:24 pts/5    00:00:00 grep mysql
[root@localhost mysql]#
```

图 16-4 Innobackupex 数据恢复

查看数据库恢复信息，数据完全恢复，如图 16-5 所示：

```
[root@localhost mysql]# mysql -uroot -p123456
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 5
Server version: 5.5.20 Source distribution

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All ri

Oracle is a registered trademark of Oracle Corporation and/or
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| 122dsfds           |
| 122s               |
| asset              |
| audit              |
```

图 16-5 Innobackupex 数据恢复

15.5 Innobackupex 增量备份

增量备份仅能应用于 InnoDB 或 XtraDB 表，对于 MyISAM 表而言，执行增量备份时其实进行的是完全备份。

（1） 增量备份之前必须执行完全备份，如图 16-6 所示：

```
innobackupex --user=root --password=123456 --databases=wugk01 /data/backup/mysql/
```

```
innobackupex: Backup created in directory '/data/backup/mysql/2014-12-20_
innobackupex: MySQL binlog position: filename '', position
141220 13:02:02  innobackupex: completed OK!
[root@localhost mysql]# ls
2014-12-20_13-01-43
[root@localhost mysql]# clear
[root@localhost mysql]# ls
2014-12-20_13-01-43
[root@localhost mysql]# cd 2014-12-20_13-01-43/
[root@localhost 2014-12-20_13-01-43]# ls
backup-my.cnf  ibdata1  wugk01  xtrabackup_binary  xtrabackup_binlog_info
[root@localhost 2014-12-20_13-01-43]# pwd
/data/backup/mysql/2014-12-20_13-01-43
[root@localhost 2014-12-20_13-01-43]#
```

图 16-6 Innobackupex 完整备份

（2） 执行第一次增量备份：

```
innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456 --databases=wugk01 --incremental /data/backup/mysql/ --incremental-basedir=/data/backup/mysql/2014-12-20_13-01-43/
```

增量备份完后，会在/data/backup/mysql/目录下生成新的备份目录，如图 16-7 所示：

```
xtrabackup: Transaction log of lsn (1613058) to (1613058) was copied.
141220 13:07:49  innobackupex: All tables unlocked
141220 13:07:49  innobackupex: Connection to database server closed

innobackupex: Backup created in directory '/data/backup/mysql/2014-12-20_13-07-31'
innobackupex: MySQL binlog position: filename '', position
141220 13:07:49  innobackupex: completed OK!
[root@localhost mysql]# ls
2014-12-20_13-01-43  2014-12-20_13-07-31
[root@localhost mysql]# du -sh *
19M     2014-12-20_13-01-43
256K    2014-12-20_13-07-31
[root@localhost mysql]#
```

图 16-7 Innobackupex 增量备份

（3） 数据库插入新数据，如图 16-8 所示：

```
Database changed
mysql> insert into test_01 values('003','wuguangke3');
Query OK, 1 row affected (0.02 sec)

mysql> select * from test_01;
+------+------------+
| id   | name       |
+------+------------+
| 001  | wuguangke  |
| 002  | wuguangke2 |
| 003  | wuguangke3 |
+------+------------+
3 rows in set (0.00 sec)
```

图 16-8 数据库 insert into 新数据

（4） 执行第二次增量备份，备份命令如下，如图 16-9（a）、16-9（b）：

```
innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456 --databases=wugk01 --incremental /data/backup/mysql/ --incremental-basedir=/data/backup/mysql/2014-12-20_13-07-31/
```

```
[root@localhost mysql]#
[root@localhost mysql]# innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456
mental /data/backup/mysql/ --incremental-basedir=/data/backup/mysql/2014-12-20_13-07-31/

InnoDB Backup Utility v1.5.1-xtrabackup; Copyright 2003, 2009 Innobase Oy
and Percona Inc 2009-2012.  All Rights Reserved.

This software is published under
the GNU GENERAL PUBLIC LICENSE Version 2, June 1991.

141220 13:11:14  innobackupex: Starting mysql with options:  --defaults-file='/etc/my.cnf' --p
ot' --unbuffered --
141220 13:11:14  innobackupex: Connected to database with mysql child process (pid=4094)
```

图 16-9（a）数据库增量备份

```
141220 13:11:39  innobackupex: completed OK!
[root@localhost mysql]# innobackupex --defaults-file=/etc/my.cnf --user=ro
mental /data/backup/mysql/ --incremental-basedir=/data/backup/mysql/2014-1
[root@localhost mysql]# clear
[root@localhost mysql]# ls
2014-12-20_13-01-43  2014-12-20_13-07-31  2014-12-20_13-11-20
[root@localhost mysql]# du -sh *
19M     2014-12-20_13-01-43
256K    2014-12-20_13-07-31
224K    2014-12-20_13-11-20
[root@localhost mysql]#
```

图 16-9（b）数据库增量备份

15.6 Mysql 增量备份恢复

删除原数据库中表及数据记录信息，如图 16-10 所示：

```
Database changed
mysql> show tables;
+------------------+
| Tables_in_wugk01 |
+------------------+
| test_01          |
+------------------+
1 row in set (0.00 sec)

mysql> drop table test_01;
Query OK, 0 rows affected (0.01 sec)

mysql> show tables;
Empty set (0.00 sec)
```

图 16-10 删除数据库表信息

MYSQL 增量备份数据恢复方法如下步骤：

（1） 基于 Apply-log 确保数据一致性：

```
innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456 --
apply-log --redo-only /data/backup/mysql/2014-12-20_13-01-43/
```

（2） 执行第一次增量数据恢复：

```
innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456 --
apply-log --redo-only /data/backup/mysql/2014-12-20_13-01-43/ --
incremental-dir=/data/backup/mysql/2014-12-20_13-07-31/
```

（3） 执行第二次增量数据恢复：

```
innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456 --
apply-log --redo-only /data/backup/mysql/2014-12-20_13-01-43/ --
incremental-dir=/data/backup/mysql/2014-12-20_13-11-20/
```

（4） 执行完整数据恢复：

```
innobackupex --defaults-file=/etc/my.cnf --user=root --password=123456  --
copy-back /data/backup/mysql/2014-12-20_13-01-43/
```

（5） 测试数据库已完全恢复，如图 16-11 所示：

```
Database changed
mysql> show tables;
+------------------+
| Tables_in_wugk01 |
+------------------+
| test_01          |
+------------------+
1 row in set (0.00 sec)

mysql> select * from test_10;
ERROR 1146 (42S02): Table 'wugk01.test_10' doesn't exist
mysql> select * from test_01;
+------+-------------+
| id   | name        |
+------+-------------+
| 001  | wuguangke   |
| 002  | wuguangke2  |
| 003  | wuguangke3  |
+------+-------------+
3 rows in set (0.00 sec)
```

图 16-11 数据库表信息完整恢复

第16章 Shell 企业编程基础

说到 Shell 编程，很多从事 Linux 运维工作的朋友都不陌生，都对 Shell 有基本的了解，读者可能刚开始接触 Shell 的时候，有各种想法，感觉编程非常困难，SHELL 编程是所有编程语言中最容易上手，最容易学习的编程脚本语言。

本章向读者介绍 Shell 编程入门、Shell 编程变量、If、While、For、Case、Select 基本语句案例演练及 Shell 编程四剑客 Find、Grep、Awk、Sed 深度剖析等。

16.1 SHELL 编程入门简介

曾经有人说过，学习 Linux 不知道 Shell 编程，那就是不懂 Linux，现在细细品味确实是这样。Shell 是操作系统的最外层，Shell 可以合并编程语言以控制进程和文件，以及启动和控制其它程序。

Shell 通过提示您输入，向操作系统解释该输入，然后处理来自操作系统的任何结果输出，简单来说 Shell 就是一个用户跟操作系统之间的一个命令解释器。

Shell 是用户与 Linux 操作系统之间沟通的桥梁，用户可以输入命令执行，又可以利用 Shell 脚本编程去运行，如图 17-1 所示：

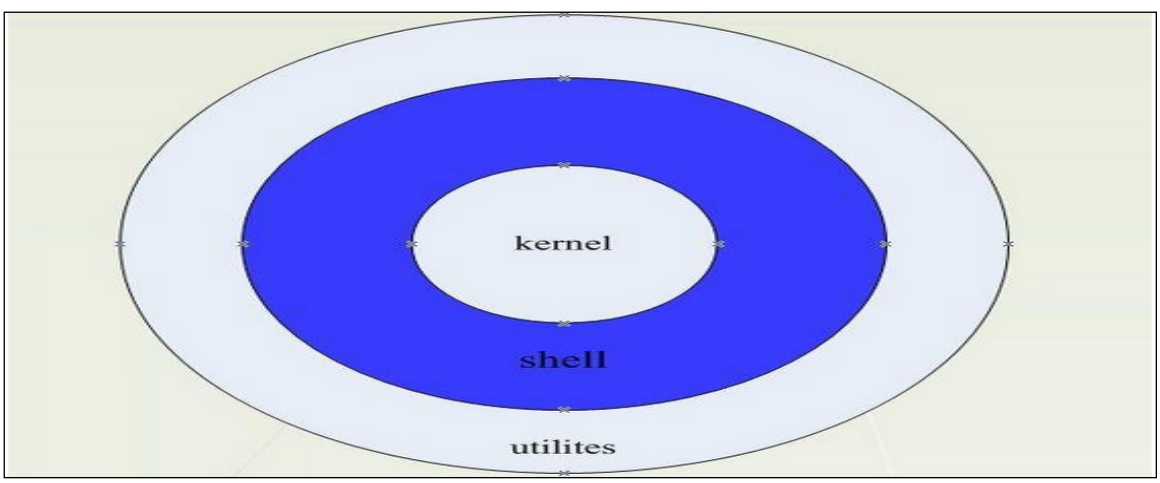
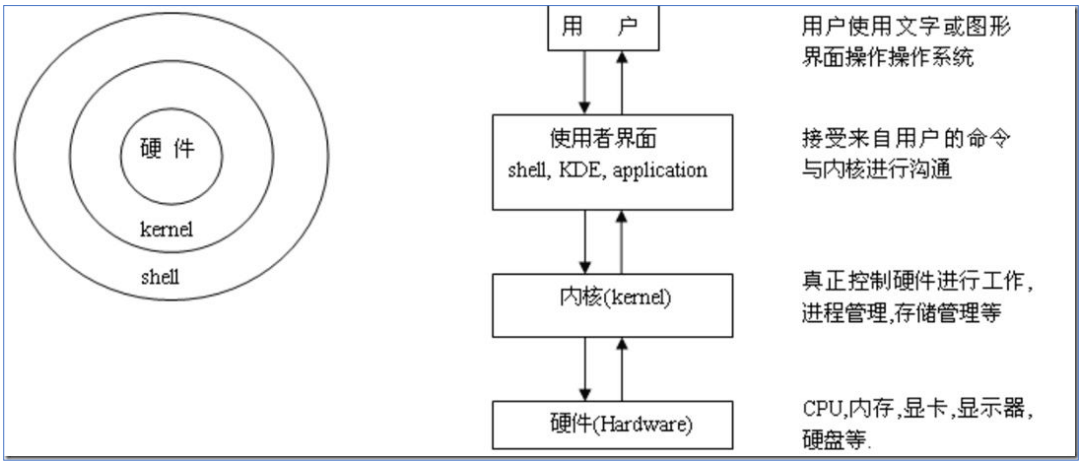


图 17-1 Shell、用户及 Kernel 位置关系



Linux Shell 种类非常多，常见的 SHELL 如下：

- Bourne Shell（/usr/bin/sh 或/bin/sh）
- **Bourne Again Shell（/bin/bash）**
- C Shell（/usr/bin/csh）
- K Shell（/usr/bin/ksh）
- Shell for Root（/sbin/sh）

不同的 Shell 语言的语法有所不同，一般不能交换使用，最常用的 shell 是 Bash，也就是 Bourne Again Shell。Bash 由于易用和免费，在日常工作中被广泛使用，也是大多数 Linux 操作系统默认的 Shell 环境。

Shell、Shell 编程、Shell 脚本、Shell 命令之间都有什么区别呢？简单来说 Shell 是一个整体的概念，Shell 编程与 Shell 脚本统称为 Shell 编程，Shell 命令是 Shell 编程底层具体的语句和实现方法。

16.2 SHELL 脚本及 Hello World

要熟练掌握 Shell 编程语言 ,需要大量的练习 ,初学者可以用 Shell 打印“Hello World”字符，寓意着开始新的启程！

Shell 脚本编程需要如下几个事项：

- Shell 脚本名称命名一般为英文、大写、小写；
- 不能使用特殊符号、空格来命名；
- Shell 脚本后缀以.sh 结尾；
- 不建议 Shell 命名为纯数字，一般以脚本功能命名。
- Shell 脚本内容首行需以#!/bin/bash 开头；
- Shell 脚本中变量名称尽量使用大写字母，字母间不能使用“-”，可以使用“_”；
- Shell 脚本变量名称不能以数字、特殊符号开头。

如下为第一个 Shell 编程脚本，脚本名称为：first_shell.sh，代码内容如下：

```
#!/bin/bash

#This is my First shell

#By author jfedu.net 2022

echo    “Hello   World ”
```

First_shell.sh 脚本内容详解如下：

#!/bin/bash	固定格式，定义该脚本所使用的 Shell 类型；

```
#This is my First shell        #号表示注释，没有任何的意义，SHELL 不会解析它；
#By author jfedu.net 2022  表示脚本创建人，#号表示注解；
echo    “Hello    World !”    Shell 脚本主命令，执行该脚本呈现的内容。
```

Shell 脚本编写完毕，如果运行该脚本，运行用户需要有执行权限，可以使用 chmod o+x first_shell.sh 赋予可执行权限。然后./first_shell.sh 执行即可，还可以直接使用命令执行: /bin/sh first_shell.sh 直接运行脚本，不需要执行权限，最终脚本执行显示效果一样。

初学者学习 Shell 编程，可以将在 Shell 终端运行的各种命令依次写入到脚本内容中，可以把 Shell 脚本当成是 Shell 命令的堆积。

16.3 Shell 编程之变量详解

Shell 是非类型的解释型语言，不像 C++、JAVA 语言编程时需要事先声明变量，Shell 给一个变量赋值，实际上就是定义了变量，在 Linux 支持的所有 shell 中，都可以用赋值符号(=)为变量赋值，Shell 变量为弱类型，定义变量不需要声明类型，但在使用时需要明确变量的类型，可以使用 Declare 指定类型，Declare 常见参数有：

```
+/-    "-"可用来指定变量的属性，"+"为取消变量所设的属性；
-f     仅显示函数；
r      将变量设置为只读；
x      指定的变量会成为环境变量，可供 shell 以外的程序来使用；
```

i　指定类型为数值，字符串或运算式。

Shell 编程中变量分为三种，分别是系统变量、环境变量和用户变量，Shell 变量名在定义时，首个字符必须为字母（a-z，A-Z），不能以数字开头，中间不能有空格，可以使用下划线（_），不能使用（-），也不能使用标点符号等。

例如定义变量 A=jfedu.net，定义这样一个变量，A 为变量名，jfedu.net 是变量的值，变量名有格式规范，变量的值可以随意指定。变量定义完成，如需要引用变量，可以使用$A。

如下脚本 var.sh 脚本内容如下：

```
#!/bin/bash
#By author jfedu.net 2022
A=123
echo     “Printf variables is $A.”
```

执行该 Shell 脚本，结果将会显示：Printf variables is jfedu.net。

16.4 Shell 编程之系统变量

Shell 常见的变量之一系统变量，主要是用于对参数判断和命令返回值判断时使用，系统变量详解如下：

$0	当前脚本的名称；
$n	当前脚本的第 n 个参数,n=1,2,…9；

$*	当前脚本的所有参数(不包括程序本身)；
$#	当前脚本的参数个数(不包括程序本身)；
$?	命令或程序执行完后的状态，返回 0 表示执行成功；
$$	程序本身的 PID 号。

16.5 Shell 编程之环境变量

Shell 常见的变量之二环境变量，主要是在程序运行时需要设置，环境变量详解如下：

PATH	命令所示路径，以冒号为分割；
HOME	打印用户家目录；
SHELL	显示当前 Shell 类型；
USER	打印当前用户名；
ID	打印当前用户 id 信息；
PWD	显示当前所在路径；
TERM	打印当前终端类型；
HOSTNAME	显示当前主机名。

16.6 Shell 编程之用户变量

Shell 常见的变量之三用户变量，用户变量又称为局部变量，主要用在 Shell 脚本内部或者临时局部使用，系统变量详解如下：

A=jfedu.net	自定义变量 A；
N_SOFT=nginx-1.12.0.tar.gz	自定义变量 N_SOFT；
BACK_DIR=/data/backup/	自定义变量 BACK_DIR；
IP1=192.168.1.11	自定义变量 IP1；
IP2=192.168.1.12	自定义变量 IP2。

创建 Echo 打印菜单 Shell 脚本，脚本代码如下：

```
#!/bin/bash

#auto install httpd

#By author jfedu.net 2022

echo -e '\033[32m----------------------------\033[0m'

FILE=httpd-2.2.31.tar.bz2

URL=http://mirrors.cnnic.cn/apache/httpd/

PREFIX=/usr/local/apache2/

echo -e "\033[36mPlease Select Install Menu:\033[0m"

echo

echo "1)官方下载 Httpd 文件包."

echo "2)解压 apache 源码包."

echo "3)编译安装 Httpd 服务器."

echo "4)启动 HTTPD 服务器."
```

```
echo -e '\033[32m----------------------------\033[0m'

sleep 20
```

运行脚本，执行结果如图 17-2 所示：

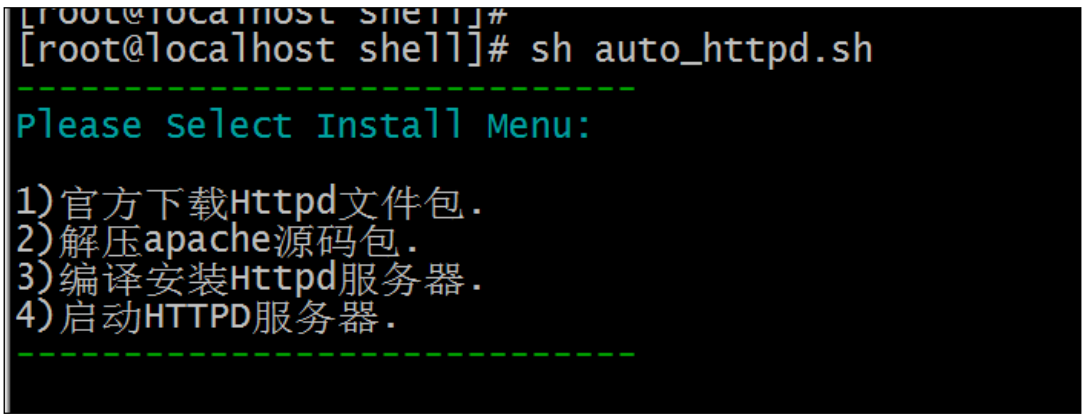


图 17-2 Echo 打印菜单脚本

16.7 If 条件语句实战

Linux Shell 编程中，if、for、while、case 等条件流程控制语句用的非常多，熟练掌握以上流程控制语句及语法的实验，对编写 Shel 脚本有非常大的益处。

If 条件判断语句，通常以 if 开头，fi 结尾。也可加入 else 或者 elif 进行多条件的判断，if 表达式如下：

```
if  (表达式)
    语句 1
else
    语句 2
```

```
fi
```

If 语句 Shell 脚本编程案例如下：

（1） 比较两个整数大小。

```
#!/bin/bash
#By author jfedu.net 2022
NUM=100
if (( $NUM > 4 )) ; then
    echo "The Num $NUM more than 4."
else
    echo "The Num $NUM less than 4."
fi
```

（2） 判断系统目录是否存在。

```
#!/bin/bash
#judge DIR or Files
#By author jfedu.net 2022
if [ ! -d /data/20140515 -a ! -d /tmp/2022/ ] ; then
        mkdir -p /data/20140515
fi
```

If 常见判断逻辑运算符详解：

-f	判断文件是否存在 eg: if [-f filename] ;
-d	判断目录是否存在 eg: if [-d dir] ;
-eq	等于，应用于整型比较 equal ;
-ne	不等于，应用于整型比较 not equal ;
-lt	小于，应用于整型比较 letter ;
-gt	大于，应用于整型比较 greater ;
-le	小于或等于，应用于整型比较 ;
-ge	大于或等于，应用于整型比较 ;
-a	双方都成立（and） 逻辑表达式 –a 逻辑表达式 ;
-o	单方成立（or） 逻辑表达式 –o 逻辑表达式 ;
-z	空字符串 ;
||	单方成立 ;
&&	双方都成立表达式。

（3） if 多个条件测试分数判断。

```
#!/bin/bash
#By author jfedu.net 2022
scores=$1
if  [[ $scores -eq 100 ]] ;  then
   echo "very good!" ;
```

```
elif [[ $scores -gt 85 ]] ;  then
    echo "good!" ;
elif [[ $scores -gt 60 ]] ;  then
    echo "pass!" ;
elif [[ $scores -lt 60 ]] ;  then
    echo "no pass!"
fi
```

16.8 MySQL 数据库备份脚本

MySQL 数据库备份是运维工程师的工作之一，如下为自动备份 Mysql 数据库脚本。

```
#!/bin/bash
#auto backup mysql
#By author jfedu.net 2022
#Define PATH 定义变量
BAK_DIR=/data/backup/mysql/`date +%Y-%m-%d`
MYSQLDB=webapp
MYSQLPW=backup
MYSQLUSR=backup
```

```
#must use root user run scripts 必须使用 root 用户运行，$UID 为系统变量
if
    [ $UID -ne 0 ] ; then
    echo This script must use the root user ! ! !
    sleep 2
    exit 0
fi
#Define DIR and mkdir DIR 判断目录是否存在，不存在则新建
if
    [  !  -d   $BAKDIR ] ; then
    mkdir   -p   $BAKDIR
fi
#Use mysqldump backup Databases
/usr/bin/mysqldump          -u$MYSQLUSR          -p$MYSQLPW          -d
$MYSQLDB >$BAKDIR/webapp_db.sql
echo   "The mysql backup successfully "
```

16.9 LAMP 一键自动化安装脚本

通过前面章节对 if 语句和变量的学习，现基于所学知识，编写一键源码安装 LAMP

脚本，

编写脚本可以养成先分解脚本的各个功能的习惯，有利于快速写出脚本，写出更高效的脚本。

一键源码安装 LAMP 脚本，可以拆分为如下功能：

（1） LAMP 打印菜单：

- 安装 apache WEB 服务器；
- 安装 Mysql DB 服务器；
- 安装 PHP 服务器；
- 整合 LAMP 架构
- 启动 LAMP 服务；

（2） Apache 服务器安装部署：

Apache 官网下载 httpd-2.2.31.tar.gz 版本，解压，进入安装目录，configure、make 、make install。

（3） Mysql 服务器的安装：

Mysql 官网下载 mysql-5.5.20.tar.gz 版本 解压 进入安装目录 ,configure、make 、make install。

（4） PHP 服务器安装：

PHP 官网下载 php-5.3.8.tar.gz 版本，解压，进入安装目录，configure、make 、make install。

（5） LAMP 整合及服务启动：

```
vi   /usr/local/apache2/htdocs/index.php

<?php

phpinfo()；

?>

/usr/local/apache2/bin/apachectl restart

service mysqld   restart
```

一键源码安装 LAMP 脚本，auto_install_lamp.sh 内容如下：

```
#!/bin/bash

#auto install LAMP

#By author jfedu.net 2022

#Httpd define path variable

H_FILES=httpd-2.2.31.tar.bz2

H_FILES_DIR=httpd-2.2.31

H_URL=http://mirrors.cnnic.cn/apache/httpd/httpd-2.2.31.tar.bz2

H_PREFIX=/usr/local/apache2/

#MySQL define path variable

M_FILES=mysql-5.5.20.tar.gz

M_FILES_DIR=mysql-5.5.20
```

```
M_URL=http://down1.chinaunix.net/distfiles/ mysql-5.5.20.tar.gz

M_PREFIX=/usr/local/mysql/

#PHP define path variable

P_FILES=php-5.3.28.tar.bz2

P_FILES_DIR=php-5.3.28

P_URL=http://mirrors.sohu.com/php/

P_PREFIX=/usr/local/php5/

echo -e '\033[32m----------------------------\033[0m'

echo

if [ -z "$1" ] ; then

    echo -e "\033[36mPlease Select Install Menu follow:\033[0m"

    echo -e "\033[32m

1)编译安装 Apache 服务器\033[1m"

    echo "2)编译安装 MySQL 服务器"

    echo "3)编译安装 PHP 服务器"

    echo "4)配置 index.php 并启动 LAMP 服务"

    echo -e "\033[31mUsage: { /bin/sh $0 1|2|3|4|help}\033[0m"

    exit

fi
```

```
if [[ "$1" -eq "help" ]] ; then
        echo -e "\033[36mPlease Select Install Menu follow:\033[0m"
        echo -e "\033[32m1)编译安装 Apache 服务器\033[1m"
        echo "2)编译安装 MySQL 服务器"
        echo "3)编译安装 PHP 服务器"
        echo "4)配置 index.php 并启动 LAMP 服务"
        echo -e "\033[31mUsage: { /bin/sh $0 1|2|3|4|help}\033[0m"
        exit
fi
#Install httpd web server
if [[ "$1" -eq "1" ]] ; then
    wget -c $H_URL/$H_FILES && tar -jxvf $H_FILES && cd $H_FILES_DIR
&&./configure --prefix=$H_PREFIX
    if [ $? -lt 0 ] ; then
        make && make install
    fi
fi
#Install Mysql DB server
if [[ "$1" -eq "2" ]] ; then
```

```
wget -c $M_URL/$M_FILES && tar -xzvf $M_FILES && cd $M_FILES_DIR
&&yum install cmake -y ；cmake . -DCMAKE_INSTALL_PREFIX=$M_PREFIX \
-DMYSQL_UNIX_ADDR=/tmp/mysql.sock \
-DMYSQL_DATADIR=/data/mysql \
-DSYSCONFDIR=/etc \
-DMYSQL_USER=mysql \
-DMYSQL_TCP_PORT=3306 \
-DWITH_XTRADB_STORAGE_ENGINE=1 \
-DWITH_INNOBASE_STORAGE_ENGINE=1 \
-DWITH_PARTITION_STORAGE_ENGINE=1 \
-DWITH_BLACKHOLE_STORAGE_ENGINE=1 \
-DWITH_MYISAM_STORAGE_ENGINE=1 \
-DWITH_READLINE=1 \
-DENABLED_LOCAL_INFILE=1 \
-DWITH_EXTRA_CHARSETS=1 \
-DDEFAULT_CHARSET=utf8 \
-DDEFAULT_COLLATION=utf8_general_ci \
-DEXTRA_CHARSETS=all \
-DWITH_BIG_TABLES=1 \
```

```
-DWITH_DEBUG=0

make && make install

/bin/cp support-files/my-small.cnf    /etc/my.cnf

/bin/cp support-files/mysql.server /etc/init.d/mysqld

chmod +x /etc/init.d/mysqld

chkconfig --add mysqld

chkconfig mysqld on

if [ $? -eq 0 ] ; then

      make && make install

      echo -e "\n\033[32m-----------------------------------------------\033[0m"

                  echo    -e    "\033[32mThe    $M_FILES_DIR    Server    Install
Success !\033[0m"

            else

                  echo  -e  "\033[32mThe $M_FILES_DIR Make or Make install
ERROR,Please Check....."

                  exit 0

            fi

fi

#Install PHP server
```

```
if [[ "$1" -eq "3" ]] ; then
        wget -c $P_URL/$P_FILES &&  tar -jxvf $P_FILES && cd $P_FILES_DIR
&&./configure  --prefix=$P_PREFIX  --with-config-file-path=$P_PREFIX/etc  --
with-mysql=$M_PREFIX --with-apxs2=$H_PREFIX/bin/apxs
        if [ $? -eq 0 ] ; then
                make ZEND_EXTRA_LIBS='-liconv' && make install
                echo -e "\n\033[32m----------------------------------------------
-\033[0m"
                echo  -e  "\033[32mThe  $P_FILES_DIR  Server  Install
Success !\033[0m"
        else
                echo  -e  "\033[32mThe  $P_FILES_DIR  Make  or  Make  install
ERROR,Please Check....."
                exit 0
        fi
fi
if [[ "$1" -eq "4" ]] ; then
    sed  -i  '/DirectoryIndex/s/index.html/index.php  index.html/g'
$H_PREFIX/conf/httpd.conf
```

```
    $H_PREFIX/bin/apachectl restart
    echo        "AddType                                application/x-httpd-
php .php" >>$H_PREFIX/conf/httpd.conf
    IP=`ifconfig eth1|grep "Bcast"|awk '{print $2}'|cut -d: -f2`
    echo "You can access http://$IP/"
cat >$H_PREFIX/htdocs/index.php <<EOF
<?php
phpinfo() ;
?>
EOF
fi
```

16.10 For 循环语句实战

for 循环语句主要用于对某个数据域进行循环读取、对文件进行遍历，通常用于需要循环某个文件或者列表。其语法格式以 for...do 开头，done 结尾。语法格式如下：

```
For  var  in  （表达式）
do
    语句 1
done
```

For 循环语句 Shell 脚本编程案例如下：

（1） 循环打印 BAT 企业官网：

```
#!/bin/bash
#By author jfedu.net 2022
for website in www.baidu.com www.taobao.com www.qq.com
do
    echo $website
done
```

（2） 循环打印 1 至 100 数字，seq 表示列出数据范围：

```
#!/bin/bash
#By author jfedu.net 2022
for i in `seq 1 100`
do
    echo "NUM is $i"
done
```

（3） For 循环求 1-100 的总和：

```
#!/bin/bash
#By author jfedu.net 2022
#auto sum 1 100
```

```
j=0

for   ((i=1；i<=100；i++))

do

      j=`expr $i + $j`

done

echo $j
```

（4） 对系统日志文件进行分组打包：

```
#!/bin/bash

#By author jfedu.net 2022

for    i    in   `find /var/log   -name   "*.log" `

do

     tar   -czf   2022_log$i.tgz   $i

done
```

（5） For 循环批量远程主机文件传输：

```
#!/bin/bash

#auto scp files for client

#By author jfedu.net 2022

for i in `seq 100 200`

do
```

```
        scp -r /tmp/jfedu.txt root@192.168.1.$i:/data/webapps/www

done
```

（6） For 循环批量远程主机执行命令：

```
#!/bin/bash

#auto scp files for client

#By author jfedu.net 2022

for i in `seq 100 200`

do

          ssh -l   root 192.168.1.$i    'ls /tmp'

done
```

（7） For 循环打印 10 秒等待提示：

```
for ((j=0 ; j<=10 ; j++))

do

      echo   -ne   "\033[32m-\033[0m"

      sleep 1

done

echo
```

16.11 Shell 编程函数实战

Shell 允许将一组命令集或语句形成一个可用块，这些块称为 Shell 函数，Shell 函数的用于在于只需定义一次 ,后期随时使用即可 ,无需在 Shell 脚本中添加重复的语句块，其语法格式以 function name（ ）{开头，以}结尾。

Shell 编程函数默认不能将参数传入（ ）内部，Shell 函数参数传递在调用函数名称传递，例如 name args1 args2。

```
function name (){
        command1
        command2
        ........
}
name args1 args2
```

（1） 创建 Apache 软件安装函数，给函数 Apache_install 传递参数 1：

```
#!/bin/bash
#auto install LAMP
#By author jfedu.net 2022
#Httpd define path variable
H_FILES=httpd-2.2.31.tar.bz2
```

```
H_FILES_DIR=httpd-2.2.31

H_URL=http://mirrors.cnnic.cn/apache/httpd/

H_PREFIX=/usr/local/apache2/

function Apache_install（ ）

{

#Install httpd web server

if [[ "$1" -eq "1" ]] ; then

    wget -c $H_URL/$H_FILES &&  tar -jxvf $H_FILES && cd $H_FILES_DIR

&&./configure --prefix=$H_PREFIX

    if [ $? -eq 0 ] ; then

        make && make install

        echo  -e  "\n\033[32m----------------------------------------------

\033[0m"

        echo -e "\033[32mThe $H_FILES_DIR Server Install Success !\033[0m"

    else

        echo  -e  "\033[32mThe  $H_FILES_DIR  Make  or  Make  install

ERROR,Please Check....."

        exit 0

    fi
```

```
fi

}

Apache_install 1
```

（2） 创建 judge_ip 判断 IP 函数：

```
#!/bin/bash

#By author jfedu.net 2022

judge_ip(){

        read -p "Please enter ip Address,example 192.168.0.11 ip": IPADDR

        echo $IPADDR|grep -v "[Aa-Zz]"|grep --color -E "([0-9]{1,3}\.){3}[0-9]{1,3}"

}

judge_ip
```

16.12 Shell 编程四剑客之 Find

通过如上基础语法的学习，读者对 Shell 编程有了更近一步的理解，Shell 编程不再是简单命令的堆积，而是演变成了各种特殊的语句、各种语法、编程工具、各种命令的集合。

在 Shell 编程工具中，四剑客工具的使用更加的广泛，Shell 编程四剑客包括：find、sed、grep、awk，熟练掌握四剑客会对 Shell 编程能力极大的提升。

四剑客之 Find 工具实战，Find 工具主要用于操作系统文件、目录的查找，其语法

参数格式为：

```
find    path    -option    [    -print ]    [ -exec    -ok    command ]    { }   \ ;
```

其 option 常用参数详解如下：

```
-name     filename                  #查找名为 filename 的文件；
-type     b/d/c/p/l/f               #查是块设备、目录、字符设备、管道、符号链接、普
通文件；
-size       n[c]                    #查长度为 n 块[或 n 字节]的文件；
-perm                               #按执行权限来查找；
-user     username                  #按文件属主来查找；
-group    groupname                 #按组来查找；
-mtime     -n +n                    #按文件更改时间来查找文件，-n 指 n 天以内，+n
指 n 天以前；
-atime     -n +n                    #按文件访问时间来查找文件；
-ctime     -n +n                    #按文件创建时间来查找文件；
-mmin      -n +n                    #按文件更改时间来查找文件，-n 指 n 分钟以
内，+n 指 n 分钟以前；
-amin      -n +n                    #按文件访问时间来查找文件；
-cmin      -n +n                    #按文件创建时间来查找文件；
-nogroup                            #查无有效属组的文件；
```

```
-nouser                          #查无有效属主的文件；
-newer    f1 !f2                 #找文件，-n 指 n 天以内，+n 指 n 天以前；
-depth                           #使查找在进入子目录前先行查找完本目录；
-fstype                          #查更改时间比 f1 新但比 f2 旧的文件；
-mount                           #查文件时不跨越文件系统 mount 点；
-follow                          #如果遇到符号链接文件，就跟踪链接所指的文件;
-cpio                            #查位于某一类型文件系统中的文件；
-prune                           #忽略某个目录；
-maxdepth                        #查找目录级别深度。
```

（1） Find 工具-name 参数案列：

```
find    /data/    -name    "*.txt"          #查找/data/目录以.txt 结尾的文件；
find    /data/    -name    "[A-Z]*"         #查找/data/目录以大写字母开头的
文件；
find    /data/    -name    "test*"          #查找/data/目录以 test 开头的文件;
```

（2） Find 工具-type 参数案列：

```
find    /data/    -type d                  #查找/data/目录下的文件夹；
find    /data/    !    -type    d          #查找/data/目录下的非文件夹；
find    /data/    -type l                  #查找/data/目录下的链接文件。
find   /data/ -type d|xargs chmod 755 -R  #查目录类型并将权限设置为 755；
```

```
find   /data/ -type f|xargs chmod 644 -R   #查文件类型并将权限设置为 644；
```

（3） Find 工具-size 参数案列：

```
find    /data/     -size    +1M                #查文件大小大于 1Mb 的文件；

find    /data/     -size    10M                 #查文件大小为 10M 的文件；

find    /data/     -size    -1M                 #查文件大小小于 1Mb 的文件；
```

（4） Find 工具-perm 参数案列：

```
find    /data/     -perm    755              #查找/data/目录权限为 755 的文件或者
目录；

find    /data/     -perm    -007             #与-perm 777 相同，表示所有权限；

find    /data/     -perm    +644             #文件权限符号 644 以上；
```

（5） Find 工具-mtime 参数案列：

```
atime,access time     文件被读取或者执行的时间；

ctime,change time     文件状态改变时间；

mtime,modify time          文件内容被修改的时间；

find /data/ -mtime +30   -name   "*.log"     #查找 30 天以前的 log 文件；

find /data/ -mtime -30    -name   "*.txt"    #查找 30 天以内的 log 文件；

find /data/ -mtime 30     -name        "*.txt"    #查找第 30 天的 log 文件；

find /data/ -mmin   +30  -name        "*.log"     #查找 30 分钟以前修改的 log 文件;

find /data/ -amin   -30   -name        "*.txt"    #查找 30 分钟以内被访问的 log 文
```

```
件；
find /data/ -cmin   30     -name        "*.txt"    #查找第 30 分钟改变的 log 文件。
```

（6） Find 工具参数综合案列：

```
#查找/data 目录以.log 结尾，文件大于 10k 的文件，同时 cp 到/tmp 目录；
find /data/ -name "*.log"   –type f   -size +10k -exec cp {} /tmp/ \ ;
#查找/data 目录以.txt 结尾，文件大于 10k 的文件，权限为 644 并删除该文件；
find /data/ -name "*.log"   –type f   -size +10k   -m perm 644 -exec rm –rf {} \ ;
#查找/data 目录以.log 结尾，30 天以前的文件，大小大于 10M 并移动到/tmp 目录；
find /data/ -name "*.log"   –type f   -mtime +30 –size +10M -exec mv {} /tmp/
\ ;
```

16.13 Shell 编程四剑客之 GREP

全面搜索正则表达式（Global search regular expression(RE) ，GREP）是一种强大的文本搜索工具，它能使用正则表达式搜索文本，并把匹配的行打印出来。

Unix/Linux 的 grep 家族包括 grep、egrep 和 fgrep，其中 egrep 和 fgrep 的命令跟 grep 有细微的区别，egrep 是 grep 的扩展，支持更多的 re 元字符， fgrep 是 fixed grep 或 fast grep 简写，它们把所有的字母都看作单词，正则表达式中的元字符表示其自身的字面意义，不再有其他特殊的含义，一般使用比较少。

目前 Linux 操作系统默认使用 GNU 版本的 grep。它功能更强，可以通过-G、-

E、-F 命令行选项来使用 egrep 和 fgrep 的功能。其语法格式及常用参数详解如下：

grep -[acinv] 'word' Filename

Grep 常用参数详解如下：

-a	以文本文件方式搜索；
-c	计算找到的符合行的次数；
-i	忽略大小写；
-n	顺便输出行号；
-v	反向选择，即显示不包含匹配文本的所有行；
-h	查询多文件时不显示文件名；
-l	查询多文件时只输出包含匹配字符的文件名；
-s	不显示不存在或无匹配文本的错误信息；
-E	允许使用 egrep 扩展模式匹配。

学习 Grep 时，需要了解通配符、正则表达式两个概念，很多读者容易把彼此搞混淆，通配符主要用在 Linux 的 Shell 命令中，常用于文件或者文件名称的操作，而正则表达式用于文本内容中的字符串搜索和替换，常用在 AWK、GREP、SED、VIM 工具中对文本的操作。

通配符类型详解：

*	0 个或者多个字符、数字；
?	匹配任意一个字符；

#	表示注解；
|	管道符号；
；	多个命令连续执行；
&	后台运行指令；
!	逻辑运算非；
[ ]	内容范围，匹配括号中内容；
{ }	命令块，多个命令匹配。

正则表达式详解：

*	前一个字符匹配 0 次或多次；
.	匹配除了换行符以外任意一个字符；
.*	代表任意字符；
^	匹配行首，即以某个字符开头；
$	匹配行尾，即以某个字符结尾；
\(..\)	标记匹配字符；
[]	匹配中括号里的任意指定字符，但只匹配一个字符；
[^]	匹配除中括号以外的任意一个字符；
\	转义符，取消特殊含义；
\<	锚定单词的开始；
\>	锚定单词的结束；

{n}	匹配字符出现 n 次；
{n,}	匹配字符出现大于等于 n 次；
{n,m}	匹配字符至少出现 n 次，最多出现 m 次；
\w	匹配文字和数字字符；
\W	\w 的反置形式，匹配一个或多个非单词字符；
\b	单词锁定符；
\s	匹配任何空白字符；
\d	匹配一个数字字符，等价于[0-9]。

常用 GREP 工具企业演练案列：

grep	-c "test"	jfedu.txt	统计 test 字符总行数；
grep	-i "TEST"	jfedu.txt	不区分大小写查找 TEST 所有的行；
grep	-n "test"	jfedu.txt	打印 test 的行及行号；
grep	-v "test"	jfedu.txt	不打印 test 的行；
grep	"test[53]"	jfedu.txt	以字符 test 开头，接 5 或者 3 的行；
grep	"^[^test]"	jfedu.txt	显示输出行首不是 test 的行；
grep	"[Mm]ay"	jfedu.txt	匹配 M 或 m 开头的行；
grep	"K...D"	jfedu.txt	匹配 K，三个任意字符，紧接 D 的行；
grep	"[A-Z][9]D"	jfedu.txt	匹配大写字母，紧跟 9D 的字符行；
grep	"T\{2,\}"	jfedu.txt	打印字符 T 字符连续出现 2 次以上的行；

grep	"T\{4,6\}" jfedu.txt	打印字符 T 字符连续出现 4 次及 6 次的行；
grep	-n "^$" jfedu.txt	打印空行的所在的行号；
grep	-vE "#|^$" jfedu.txt	不匹配文件中的#和空行；
grep	--color -ra -E "db|config|sql" *	匹配包含 db 或者 config 或者 sql 的文件；
grep	--color -E "\<([0-9]{1,3}\.){3}([0-9]{1,3})\>" jfedu.txt	匹配 IPV4 地址。

16.14 Shell 数组编程

数组是相同数据类型的元素按一定顺序排列的集合，把有限个类型相同的变量用一个名字命名，然后用编号区分他们变量的集合，这个名称称之为数组名，编号成为下标。Linux Shell 编程中常用一维数组。

数组的设计其实了为了处理方便，把具有相同类型的若干变量按有序的形式组织起来的一种形式，以减少重复频繁的单独定义。如图 17-3 所示：

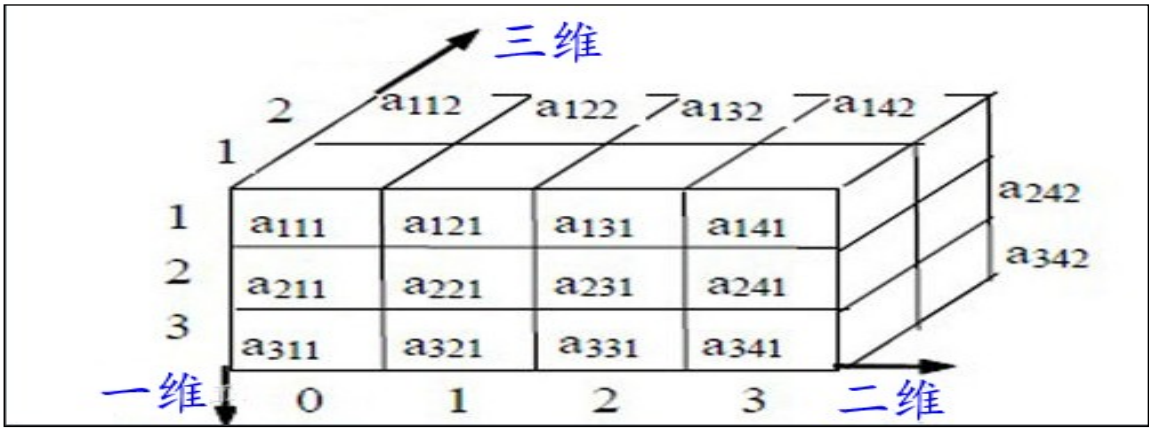


图 17-3 一维、二维、三维数组

定义数组一般以小括号的方式来定义，数组的值可以随机指定，如下为一维数组的定义、统计、引用和删除操作，:

（1） 一唯数组定义及创建：

```
JFTEST=(
        test1
        test2
        test3
)
LAMP=(httpd  php  php-devel php-mysql mysql mysql-server)
```

（2） 数组下标一般从 0 开始，如下为引用数组的方法：

```
echo    ${JFTEST[0]}              引用第一个数组变量，结果打印 test1；
echo    ${JFTEST[1]}          引用第二个数组变量；
echo    ${JFTEST[@]}          显示该数组所有参数；
echo    ${#JFTEST[@]}         显示该数组参数个数；
echo     ${#JFTEST[0]}         显示 test1 字符长度；
echo     ${JFTEST[@]:0}        打印数组所有的值；
echo     ${JFTEST[@]:1}        打印从第二个值开始的所有值；
echo     ${JFTEST[@]:0:2}      打印从第一个值与第二个值；
```

```
echo      ${JFTEST[@]:1:2}      打印从第二个值与第三个值。
```

（3） 数组替换操作：

```
JFTEST=( [0]=www1 [1]=www2 [2]=www3 )          数组赋值；

echo   ${JFTEST[@]/test/jfedu}                 将数组值 test 替换为 jfedu；

NEWJFTEST=`echo   ${JFTEST[@]/test/jfedu}`     将结果赋值新数组。
```

（4） 数组删除操作：

```
unset array[0]                  删除数组第一个值；

unset array[1]                  删除数组第二个值；

unset array                     删除整个数组。
```

（5） 数组 Shell 脚本企业案例一，网卡 bond 绑定脚本：

```
#!/bin/bash

#Auto Make KVM Virtualization

#Auto config bond scripts

#By author jfedu.net 2022

eth_bond()

{

NETWORK=(

   HWADDR=`ifconfig  eth0  |egrep  "HWaddr|Bcast"  |tr  "\n"  "  "|awk  '{print

$5,$7,$NF}'|sed -e 's/addr://g' -e 's/Mask://g'|awk '{print $1}'`
```

```
  IPADDR=`ifconfig eth0 |egrep "HWaddr|Bcast" |tr "\n" " "|awk '{print $5,$7,$NF}'|sed -e 's/addr://g' -e 's/Mask://g'|awk '{print $2}'`
  NETMASK=`ifconfig eth0 |egrep "HWaddr|Bcast" |tr "\n" " "|awk '{print $5,$7,$NF}'|sed -e 's/addr://g' -e 's/Mask://g'|awk '{print $3}'`
  GATEWAY=`route -n|grep "UG"|awk '{print $2}'`
)
cat >ifcfg-bond0<<EOF
DEVICE=bond0
BOOTPROTO=static
${NETWORK[1]}
${NETWORK[2]}
${NETWORK[3]}
ONBOOT=yes
TYPE=Ethernet
NM_CONTROLLED=no
EOF
```

（6） 数组 Shell 脚本企业案例二，定义 IPv4 值：

```
#!/bin/bash
#auto Change ip netmask gateway scripts
```

```
#By author jfedu.net 2022

ETHCONF=/etc/sysconfig/network-scripts/ifcfg-eth0

HOSTS=/etc/hosts

NETWORK=/etc/sysconfig/network

DIR=/data/backup/`date +%Y%m%d`

NETMASK=255.255.255.0

echo "----------------------------"

count_ip(){

        count=(`echo    $IPADDR|awk    -F.    '{print $1,$2,$3,$4}'`)

        IP1=${count[0]}

        IP2=${count[1]}

        IP3=${count[2]}

        IP4=${count[3]}

}
```

第17章　Shell 编程高级企业实战

企业生产环境中，服务器规模成百上千，如果依靠人工去维护和管理是非常吃力的，

基于 Shell 编程脚本管理和维护服务器变得简单、从容，而且对企业自动化运维之路的建设起到极大的推动作用。

本章向读者介绍企业生产环境 Shell 编程案例、自动化备份 MySQL 数据、服务器信息收集、防止恶意 IP 访问、LAMP+MySQL 主从实战、千台服务器 IP 修改、Nginx+Tomcat 高级自动化部署脚本、Nginx 虚拟主机配置、Docker 管理平台等。

17.1 Shell 编程实战系统备份脚本

日常企业运维中，需要备份 Linux 操作系统中重要的文件，例如/etc、/boot 分区、重要网站数据等，在备份数据时，由于数据量非常大，需要指定高效的备份方案，如下为常用的备份数据方案：

- 每周日进行完整备份，周一至周六使用增量备份；
- 每周六进行完整备份，周日至周五使用增量备份。

企业备份数据的工具主要有：tar、cp、rsync、scp、sersync、dd 等工具。如下为基于开源 tar 工具实现系统数据备份方案：

Tar 工具手动全备份网站，-g 参数指定新的快照文件：

```
tar -g /tmp/snapshot -czvf /tmp/2022_full_system_data.tar.gz /data/sh/
```

Tar 工具手动增量备份网站，-g 参数指定全备已生成的快照文件，后续增量备份基于上一个增量备份快照文件：

```
tar -g /tmp/snapshot -czvf /tmp/2014_add01_system_data.tar.gz
/data/sh/
```

Tar 工具全备、增量备份网站，Shell 脚本实现自动打包备份编写思路如下：

- 系统备份数据按每天存放；
- 创建完整备份函数块；
- 创建增量备份函数块；
- 根据星期数判断完整或增量；
- 将脚本加入 Crontab 实现自动备份；

Tar 工具全备、增量备份网站，Shell 脚本实现自动打包备份，代码如下：

```
#!/bin/bash
#Auto Backup Linux System Files
#By author jfedu.net 2022
#Define Path variables
SOURCE_DIR=(
    $*
)
TARGET_DIR=/data/backup/
YEAR=`date +%Y`
MONTH=`date +%m`
```

```
DAY=`date +%d`

WEEK=`date +%u`

A_NAME=`date +%H%M`

FILES=system_backup.tgz

CODE=$?

if

    [ -z "$*" ] ; then

    echo -e "\033[32mUsage:\nPlease Enter Your Backup Files or Directories\n--

----------------------------------------\n\nUsage: { $0 /boot /etc}\033[0m"

    exit

fi

#Determine Whether the Target Directory Exists

if

    [ ! -d $TARGET_DIR/$YEAR/$MONTH/$DAY ] ; then

    mkdir -p $TARGET_DIR/$YEAR/$MONTH/$DAY

    echo -e "\033[32mThe $TARGET_DIR Created Successfully !\033[0m"

fi

#EXEC Full_Backup Function Command

Full_Backup()
```

```
{
if
    [ "$WEEK" -eq "7" ] ; then
    rm -rf $TARGET_DIR/snapshot
    cd $TARGET_DIR/$YEAR/$MONTH/$DAY  ;tar -g $TARGET_DIR/snapshot -
czvf $FILES ${SOURCE_DIR[@]}
    [ "$CODE" == "0" ]&&echo -e  "----------------------------------------
\n\033[32mThese Full_Backup System Files Backup Successfully !\033[0m"
fi
}
#Perform incremental BACKUP Function Command
Add_Backup()
{
   if
        [ $WEEK -ne "7" ] ; then
        cd  $TARGET_DIR/$YEAR/$MONTH/$DAY  ;  tar  -g
$TARGET_DIR/snapshot -czvf $A_NAME$FILES ${SOURCE_DIR[@]}
        [ "$CODE" == "0" ]&&echo -e  "----------------------------------------
\n\033[32mThese  Add_Backup  System  Files
```

```
$TARGET_DIR/$YEAR/$MONTH/$DAY/${YEAR}_$A_NAME$FILES          Backup
Successfully !\033[0m"
    fi
}
sleep 3
Full_Backup；Add_Backup
```

Crontab 任务计划中添加如下语句，每天凌晨 1 点整执行备份脚本即可：

```
0   1   *   *   * /bin/sh /data/sh/auto_backup.sh /boot /etc/ >> /tmp/back.log
2>&1
```

17.2 Shell 编程实战收集服务器信息

在企业上产环境中，经常会对服务器资产进行统计存档，单台服务器可以手动去统计服务器的 CPU 型号、内存大小、硬盘容量、网卡流量等，如果服务器数量超过百台、千台，使用手工方式就变得非常吃力。

基于 Shell 脚本实现自动化服务器硬件信息的收集，并将收集的内容存放在数据库，能更快、更高效的实现对服务器资产信息的管理。Shell 脚本实现服务器信息自动收集，编写思路如下：

- 创建数据库和表存储服务器信息；
- 基于 Shell 四剑客 awk、find、sed、grep 获取服务器信息；

- 将获取的信息写成 SQL 语句；
- 定期对 SQL 数据进行备份；
- 将脚本加入 Crontab 实现自动备份；

创建数据库表，创建 SQL 语句如下：

```
CREATE TABLE `audit_system` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `ip_info` varchar(50) NOT NULL,
  `serv_info` varchar(50) NOT NULL,
  `cpu_info` varchar(50) NOT NULL,
  `disk_info` varchar(50) NOT NULL,
  `mem_info` varchar(50) NOT NULL,
  `load_info` varchar(50) NOT NULL,
  `mark_info` varchar(50) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `ip_info` (`ip_info`),
  UNIQUE KEY `ip_info_2` (`ip_info`)
  ) ;
```

Shell 脚本实现服务器信息自动收集，代码如下：

```
#!/bin/bash
```

```
#Auto get system info

#By author jfedu.net 2022

#Define Path variables

echo -e "\033[34m \033[1m"

cat <<EOF

++++++++++++++++++++++++++++++++++++++++++++++++++

++++++++Welcome to use system Collect+++++++++

++++++++++++++++++++++++++++++++++++++++++++++++++

EOF

ip_info=`ifconfig |grep "Bcast"|tail -1 |awk '{print $2}'|cut -d: -f 2`

cpu_info1=`cat /proc/cpuinfo |grep 'model name'|tail -1 |awk -F: '{print $2}'|sed

's/^ //g'|awk '{print $1,$3,$4,$NF}'`

cpu_info2=`cat /proc/cpuinfo |grep "physical id"|sort |uniq -c|wc -l`

serv_info=`hostname |tail -1`

disk_info=`fdisk -l|grep "Disk"|grep -v "identifier"|awk '{print $2,$3,$4}'|sed

's/,//g'`

mem_info=`free -m |grep "Mem"|awk '{print "Total",$1,$2"M"}'`

load_info=`uptime |awk '{print "Current Load: "$(NF-2)}'|sed 's/\,//g'`

mark_info='BeiJing_IDC'
```

```
echo -e "\033[32m-----------------------------------------\033[1m"
echo IPADDR:${ip_info}
echo HOSTNAME:$serv_info
echo CPU_INFO:${cpu_info1} X${cpu_info2}
echo DISK_INFO:$disk_info
echo MEM_INFO:$mem_info
echo LOAD_INFO:$load_info
echo -e "\033[32m-----------------------------------------\033[0m"
echo -e -n "\033[36mYou want to write the data to the databases? \033[1m"  ;
read ensure
if    [ "$ensure" == "yes" -o "$ensure" == "y" -o "$ensure" == "Y" ] ; then
     echo "-----------------------------------------"
     echo -e    '\033[31mmysql -uaudit -p123456 -D audit -e ''' "insert into
audit_system                    values('','${ip_info}','$serv_info','${cpu_info1}
X${cpu_info2}','$disk_info','$mem_info','$load_info','$mark_info')" ''' \033[0m '
     mysql -uroot -p123456 -D test -e "insert into audit_system
values('','${ip_info}','$serv_info','${cpu_info1}
X${cpu_info2}','$disk_info','$mem_info','$load_info','$mark_info')"
else
```

```
    echo "Please wait，exit....."
    exit
fi
```

手动读取数据库服务器信息命令：

```
mysql -uroot -p123 -e 'use wugk1 ；select * from audit_audit_system；'|sed 's/-//g'|grep -v "id"
```

17.3 Shell 编程实战拒绝恶意 IP 登录

企业服务器暴露在外网，每天会有大量的人使用各种用户名和密码尝试登陆服务器，如果让其一直尝试，难免会猜出密码，通过开发 Shell 脚本，可以自动将尝试登陆服务器错误密码次数的 IP 列表加入到防火墙配置中。

Shell 脚本实现服务器拒绝恶意 IP 登陆，编写思路如下：

- 登陆服务器日志/var/log/secure；
- 检查日志中认证失败的行并打印其 IP 地址；
- 将 IP 地址写入至防火墙；
- 禁止该 IP 访问服务器 SSH 22 端口；
- 将脚本加入 Crontab 实现自动禁止恶意 IP；

Shell 脚本实现服务器拒绝恶意 IP 登陆，代码如下：

```
#!/bin/bash
```

```
#Auto drop ssh failed IP address

#By author jfedu.net 2022

#Define Path variables

SEC_FILE=/var/log/secure

IP_ADDR=`awk '{print $0}' /var/log/secure|grep -i "fail"| egrep -o "([0-

9]{1,3}\.){3}[0-9]{1,3}" | sort -nr | uniq -c |awk '$1>=15 {print $2}'`

IPTABLE_CONF=/etc/sysconfig/iptables

echo

cat <<EOF

++++++++++++++welcome to use ssh login drop failed

ip++++++++++++++++++

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

++++++++++++++++++++

++++++++++++++++----------------------------------

++++++++++++++++++

EOF

echo

for ((j=0 ; j<=6 ; j++)) ; do echo -n "-" ; sleep 1 ; done

echo
```

```
for i in `echo $IP_ADDR`
do
   cat $IPTABLE_CONF |grep $i >/dev/null
if
   [ $? -ne 0 ] ; then
    sed -i "/lo/a -A INPUT -s $i -m state --state NEW -m tcp -p tcp --dport 22 -j
DROP" $IPTABLE_CONF
fi
done
NUM=`find /etc/sysconfig/    -name iptables -a -mmin -1|wc -l`
            if [ $NUM -eq 1 ] ; then
                        /etc/init.d/iptables restart
            fi
```

17.4 Shell 编程实战 LAMP 一键安装

LAMP 是目前互联网主流 WEB 网站架构，通过源码安装、维护和管理对于单台很轻松，如果服务器数量多，手工管理就非常困难，基于 Shell 脚本可以更快速的维护 LAMP 架构。

Shell 脚本实现服务器 LAMP 一键源码安装配置，编写思路如下：

- 脚本的功能，实现安装 LAMP 环境、论坛网站；
- Apache 安装配置、MYSQL、PHP 安装；
- 源码 LAMP 整合配置；
- 启动数据库，创建数据库并授权；
- 重启 LAMP 所有服务， 验证访问；

Shell 脚本实现服务器 LAMP 一键源码安装配置，代码如下：

```
#!/bin/bash

#Auto install LAMP

#By author jfedu.net 2022

#Define Path variables

#Httpd define path variable

H_FILES=httpd-2.2.32.tar.bz2

H_FILES_DIR=httpd-2.2.32

H_URL=http://mirrors.cnnic.cn/apache/httpd/

H_PREFIX=/usr/local/apache2/

#MySQL define path variable

M_FILES=mysql-5.5.20.tar.gz

M_FILES_DIR=mysql-5.5.20

M_URL=http://down1.chinaunix.net/distfiles/
```

```
M_PREFIX=/usr/local/mysql/

#PHP define path variable

P_FILES=php-5.3.28.tar.bz2

P_FILES_DIR=php-5.3.28

P_URL=http://mirrors.sohu.com/php/

P_PREFIX=/usr/local/php5/

function httpd_install(){

if [[ "$1" -eq "1" ]] ; then

    wget -c $H_URL/$H_FILES &&   tar -jxvf $H_FILES && cd $H_FILES_DIR

&&./configure --prefix=$H_PREFIX

    if [ $? -eq 0 ] ; then

        make && make install

    fi

fi

}

function mysql_install(){

if [[ "$1" -eq "2" ]] ; then

wget -c $M_URL/$M_FILES &&   tar -xzvf $M_FILES && cd $M_FILES_DIR

&&yum install cmake ncurses-devel -y  ; cmake . -
```

```
DCMAKE_INSTALL_PREFIX=$M_PREFIX \

-DMYSQL_UNIX_ADDR=/tmp/mysql.sock \

-DMYSQL_DATADIR=/data/mysql \

-DSYSCONFDIR=/etc \

-DMYSQL_USER=mysql \

-DMYSQL_TCP_PORT=3306 \

-DWITH_XTRADB_STORAGE_ENGINE=1 \

-DWITH_INNOBASE_STORAGE_ENGINE=1 \

-DWITH_PARTITION_STORAGE_ENGINE=1 \

-DWITH_BLACKHOLE_STORAGE_ENGINE=1 \

-DWITH_MYISAM_STORAGE_ENGINE=1 \

-DWITH_READLINE=1 \

-DENABLED_LOCAL_INFILE=1 \

-DWITH_EXTRA_CHARSETS=1 \

-DDEFAULT_CHARSET=utf8 \

-DDEFAULT_COLLATION=utf8_general_ci \

-DEXTRA_CHARSETS=all \

-DWITH_BIG_TABLES=1 \

-DWITH_DEBUG=0
```

```
if [ $? -eq 0 ] ; then
    make && make install
    echo -e "\n\033[32m-----------------------------------------------\033[0m"
                echo -e "\033[32mThe $M_FILES_DIR Server Install
Success !\033[0m"
        else
                echo -e "\033[32mThe $M_FILES_DIR Make or Make install
ERROR,Please Check....."
                exit 0
fi
/bin/cp support-files/my-small.cnf   /etc/my.cnf
/bin/cp support-files/mysql.server /etc/init.d/mysqld
chmod +x /etc/init.d/mysqld
chkconfig --add mysqld
chkconfig mysqld on
fi
}
function php_install(){
if [[ "$1" -eq "3" ]] ; then
```

```
        yum install   libxml2-devel   perl-devel   perl   libtool* -y
        wget -c $P_URL/$P_FILES &&   tar -jxvf $P_FILES && cd $P_FILES_DIR
&&./configure --prefix=$P_PREFIX --with-config-file-path=$P_PREFIX/etc --
with-mysql=$M_PREFIX --with-apxs2=$H_PREFIX/bin/apxs
        if [ $? -eq 0 ] ; then
                make ZEND_EXTRA_LIBS='-liconv' && make install
                echo -e "\n\033[32m-----------------------------------------
--\033[0m"
                echo -e "\033[32mThe $P_FILES_DIR Server Install
Success !\033[0m"
        else
                echo -e "\033[32mThe $P_FILES_DIR Make or Make install
ERROR,Please Check....."
                exit 0
        fi
fi
}
function lamp_config(){
if [[ "$1" -eq "4" ]] ; then
```

```
    sed -i '/DirectoryIndex/s/index.html/index.php index.html/g'
$H_PREFIX/conf/httpd.conf
    $H_PREFIX/bin/apachectl restart
    echo "AddType        application/x-httpd-
php .php" >>$H_PREFIX/conf/httpd.conf
    IP=`ifconfig eth0|grep "Bcast"|awk '{print $2}'|cut -d: -f2`
    echo "You can to access http://$IP/"

cat >$H_PREFIX/htdocs/index.php<<EOF
<?php
phpinfo() ;
?>
EOF
fi
}
PS3="Please enter you select install menu:"
select i in http mysql php config quit
do
```

```
case $i in
    http)
    httpd_install 1
    ;;
    mysql)
    mysql_install 2
    ;;
    php)
    php_install 3
    ;;
    config)
    lamp_config 4
    ;;
    quit)
    exit
esac
done
```

17.5 Shell 编程实战 MYSQL 主从复制

MYSQL 数据库服务器应用主要应用于与动态网站结合，存放网站必要的数据，例如订单、交易、员工表、薪资等记录，为了实现数据备份，需引入 MYSQL 主从架构，MYSQL 主从架构脚本可以实现自动化安装、配置和管理。

Shell 脚本实现服务器 MYSQL 一键 YUM 安装配置，编写思路如下：

MYSQL 主库的操作：

- 主库上安装 MYSQL,设置 server-id、bin-log；
- 授权复制同步的用户，对客户端授权；
- 确认 bin-log 文件名、position 位置点。

MYSQL 从库的操作：

- 从库上安装 MYSQL,设置 server-id；
- change master 指定主库和 bin-log 名和 position；
- start slave； 启动从库 IO 线程；
- show slave status\G 查看主从的状态。

Shell 脚本实现服务器 MYSQL 一键 YUM 安装配置，需要提前手动授权主库可以免密码登录从库服务器，代码如下：

```
#!/bin/bash

#Auto install Mysql AB Repliation
```

```
#By author jfedu.net 2022

#Define Path variables

MYSQL_SOFT="mysql mysql-server mysql-devel php-mysql mysql-libs"

NUM=`rpm -qa |grep -i mysql |wc -l`

INIT="/etc/init.d/mysqld"

CODE=$?

#Mysql To Install 2022

if [ $NUM -ne 0 -a -f $INIT ] ; then

    echo -e "\033[32mThis Server Mysql already Install.\033[0m"

    read -p "Please ensure yum remove Mysql Server,YES or NO": INPUT

    if [ $INPUT == "y" -o $INPUT == "yes" ] ; then

        yum remove $MYSQL_SOFT -y  ; rm -rf /var/lib/mysql /etc/my.cnf

        yum install $MYSQL_SOFT -y

    else

        echo

    fi

else

    yum remove $MYSQL_SOFT -y  ; rm -rf /var/lib/mysql /etc/my.cnf

    yum install $MYSQL_SOFT -y
```

```
    if [ $CODE -eq 0 ]；then
        echo -e "\033[32mThe Mysql Install Successfully.\033[0m"
    else
        echo -e "\033[32mThe Mysql Install Failed.\033[0m"
        exit 1
    fi
fi
my_config(){
cat >/etc/my.cnf<<EOF
[mysqld]
datadir=/var/lib/mysql
socket=/var/lib/mysql/mysql.sock
user=mysql
symbolic-links=0
log-bin=mysql-bin
server-id = 1
auto_increment_offset=1
auto_increment_increment=2
[mysqld_safe]
```

```
log-error=/var/log/mysqld.log

pid-file=/var/run/mysqld/mysqld.pid

EOF

}

my_config

/etc/init.d/mysqld restart

ps -ef |grep mysql

MYSQL_CONFIG(){

#Master Config Mysql

mysql -e "grant  replication  slave  on *.* to  'tongbu'@'%'  identified by

'123456' ; "

MASTER_FILE=`mysql -e "show master status ; "|tail -1|awk '{print $1}'`

MASTER_POS=`mysql -e "show master status ; "|tail -1|awk '{print $2}'`

MASTER_IPADDR=`ifconfig eth0|grep "Bcast"|awk '{print $2}'|cut -d: -f2`

read -p "Please Input Slave IPaddr: " SLAVE_IPADDR

#Slave Config Mysql

ssh -l root $SLAVE_IPADDR "yum remove $MYSQL_SOFT -y  ; rm -rf

/var/lib/mysql /etc/my.cnf  ; yum install $MYSQL_SOFT -y"

ssh -l root $SLAVE_IPADDR "$my_config"
```

```
#scp -r /etc/my.cnf root@192.168.111.129:/etc/

ssh -l root $SLAVE_IPADDR "sed -i 's#server-id = 1#server-id = 2#g'

/etc/my.cnf"

ssh -l root $SLAVE_IPADDR "sed -i '/log-bin=mysql-bin/d' /etc/my.cnf"

ssh -l root $SLAVE_IPADDR "/etc/init.d/mysqld restart"

ssh -l root $SLAVE_IPADDR "mysql -e \"change master to

master_host='$MASTER_IPADDR',master_user='tongbu',master_password='12

3456',master_log_file='$MASTER_FILE',master_log_pos=$MASTER_POS ; \""

ssh -l root $SLAVE_IPADDR "mysql -e \"slave start ; \""

ssh -l root $SLAVE_IPADDR "mysql -e \"show slave status\G ; \""

}


read -p "Please ensure your Server is Master and you will config mysql

Replication?yes or no": INPUT

if [ $INPUT == "y" -o $INPUT == "yes" ] ; then

    MYSQL_CONFIG

else

    exit 0

fi
```

17.6 Shell 编程实战修改 IP 及主机名

企业中服务器 IP 地址系统通过自动化工具安装完系统，IP 均是自动获取的，而服务器要求固定的静态 IP，百台服务器手工去配置静态 IP 是不可取的，可以基于 Shell 脚本自动修改 IP、主机名等信息。

Shell 脚本实现服务器 IP、主机名自动修改及配置，编写思路如下：

- 静态 IP 修改；
- 动态 IP 修改；
- 根据 IP-生成主机名并配置；
- 修改 DNS 域名解析；

Shell 脚本实现服务器 IP、主机名自动修改及配置，代码如下：

```
#!/bin/bash

#Auto Change ip netmask gateway scripts

#By author jfedu.net 2022

#Define Path variables

ETHCONF=/etc/sysconfig/network-scripts/ifcfg-eth0

HOSTS=/etc/hosts

NETWORK=/etc/sysconfig/network

DIR=/data/backup/`date +%Y%m%d`
```

```
NETMASK=255.255.255.0

echo "----------------------------"

judge_ip(){

    read -p "Please enter ip Address,example 192.168.0.11 ip": IPADDR

    echo $IPADDR|grep -v "[Aa-Zz]"|grep --color -E "([0-9]{1,3}\.){3}[0-9]{1,3}"

}

count_ip(){

    count=(`echo $IPADDR|awk -F. '{print $1,$2,$3,$4}'`)

    IP1=${count[0]}

    IP2=${count[1]}

    IP3=${count[2]}

    IP4=${count[3]}

}

ip_check()

{

judge_ip

while [ $? -ne 0 ]

do

    judge_ip
```

```
done

count_ip

while [ "$IP1" -lt 0 -o "$IP1" -ge 255 -o "$IP2" -ge 255 -o "$IP3" -ge 255 -o
"$IP4" -ge 255 ]

do

    judge_ip

    while [ $? -ne 0 ]

    do

        judge_ip

    done

    count_ip

done

}

change_ip()

{

if [ ! -d $DIR ] ; then

    mkdir -p $DIR

fi

echo "The Change ip address to Backup Interface eth0"
```

```
cp $ETHCONF    $DIR

grep "dhcp"    $ETHCONF

if [ $? -eq 0 ] ; then

      read -p "Please enter ip Address:" IPADDR

      sed -i 's/dhcp/static/g' $ETHCONF

      echo -e "IPADDR=$IPADDR\nNETMASK=$NETMASK\nGATEWAY=`echo

$IPADDR|awk -F. '{print $1"."$2"."$3}'`.2" >>$ETHCONF

      echo "The IP configuration success. !"

else

      echo -n    "Static IP has been configured,please confirm whether to

modify,yes or No":

      read i

fi

if    [ "$i" == "y" -o "$i" == "yes" ] ; then

      ip_check

      sed -i -e '/IPADDR/d' -e '/NETMASK/d' -e '/GATEWAY/d' $ETHCONF

      echo -e "IPADDR=$IPADDR\nNETMASK=$NETMASK\nGATEWAY=`echo

$IPADDR|awk -F. '{print $1"."$2"."$3}'`.2" >>$ETHCONF

      echo "The IP configuration success. !"
```

```
        echo
else
        echo "Static IP already exists,please exit."
        exit $?
fi
}
change_hosts()
{

if [ ! -d $DIR ] ; then
        mkdir -p $DIR
fi
cp $HOSTS $DIR
ip_check
host=` echo $IPADDR|sed 's/\./-/g'|awk '{print "BJ-IDC-"$0"-jfedu.net"}'`
cat $HOSTS |grep "$host"
if [ $? -ne 0 ] ; then
        echo "$IPADDR       $host" >> $HOSTS
        echo "The hosts modify success "
```

```
fi

grep "$host" $NETWORK

if [ $? -ne 0 ] ; then

    sed -i "s/^HOSTNAME/#HOSTNAME/g" $NETWORK

    echo "NETWORK=$host" >>$NETWORK

    hostname $host ; su

fi

}

PS3="Please Select configuration ip or configuration host:"

select i in   "modify_ip" "modify_hosts" "exit"

do

    case $i in

            modify_ip)

            change_ip

        ;;

            modify_hosts)

            change_hosts

        ;;

            exit)
```

```
                exit
            ;;
                *)
                echo -e "1) modify_ip\n2) modify_ip\n3)exit"
    esac

done
```

17.7 Shell 编程实战 Zabbix 安装配置

Zabbix 是一款分布式监控系统，基于 C/S 模式，需在服务器安装 Zabbix_server，在客户端安装 Zabbix_agent，通过 Shell 脚本可以更快速的实现该需求。

Shell 脚本实现 Zabbix 服务器端和客户端自动安装，编写思路如下：

- Zabbix 软件的版本源码安装、路径、--enable-server、--enable-agent；
- cp zabbix_agentd 启动进程-/etc/init.d/zabbix_agentd、给执行 x 权限；
- 配置 zabbix_agentd.conf 文件，指定 server IP 变量；
- 指定客户端的 Hostname 其实可以等于客户端 IP 地址；
- 启动 zabbix_agentd 服务，创建 zabbix user。

Shell 脚本实现 Zabbix 服务器端和客户端自动安装，代码如下：

```
#!/bin/bash

#Auto install zabbix server and client

#By author jfedu.net 2022

#Define Path variables

ZABBIX_SOFT="zabbix-3.2.6.tar.gz"

INSTALL_DIR="/usr/local/zabbix/"

SERVER_IP="192.168.111.128"

IP=`ifconfig|grep Bcast|awk '{print $2}'|sed 's/addr://g'`

SERVER_INSTALL(){

yum -y install curl curl-devel net-snmp net-snmp-devel perl-DBI

groupadd zabbix  ; useradd -g zabbix zabbix ; usermod -s /sbin/nologin

zabbix

tar -xzf $ZABBIX_SOFT ; cd `echo $ZABBIX_SOFT|sed 's/.tar.*//g'`

./configure   --prefix=/usr/local/zabbix --enable-server --enable-agent --with-

mysql --enable-ipv6 --with-net-snmp --with-libcurl &&make install

if [ $? -eq 0 ] ; then

      ln -s /usr/local/zabbix/sbin/zabbix_* /usr/local/sbin/

fi

cd -  ; cd zabbix-3.2.6
```

```
cp  misc/init.d/tru64/{zabbix_agentd,zabbix_server}  /etc/init.d/  ; chmod
o+x /etc/init.d/zabbix_*
mkdir -p /var/www/html/zabbix/ ; cp -a  frontends/php/*
/var/www/html/zabbix/
#config zabbix server
cat >$INSTALL_DIR/etc/zabbix_server.conf<<EOF
LogFile=/tmp/zabbix_server.log
DBHost=localhost
DBName=zabbix
DBUser=zabbix
DBPassword=123456
EOF
#config zabbix agentd
cat >$INSTALL_DIR/etc/zabbix_agentd.conf<<EOF
LogFile=/tmp/zabbix_agentd.log
Server=$SERVER_IP
ServerActive=$SERVER_IP
Hostname = $IP
EOF
```

```
#start zabbix agentd

/etc/init.d/zabbix_server restart

/etc/init.d/zabbix_agentd restart

/etc/init.d/iptables stop

setenforce 0

}

AGENT_INSTALL(){

yum -y install curl curl-devel net-snmp net-snmp-devel perl-DBI

groupadd zabbix  ; useradd -g zabbix zabbix ; usermod -s /sbin/nologin

zabbix



tar -xzf $ZABBIX_SOFT ; cd `echo $ZABBIX_SOFT|sed 's/.tar.*//g'`

./configure  --prefix=/usr/local/zabbix  --enable-agent&&make install

if [ $? -eq 0 ] ; then

    ln -s /usr/local/zabbix/sbin/zabbix_* /usr/local/sbin/

fi

cd -  ; cd zabbix-3.2.6

cp  misc/init.d/tru64/zabbix_agentd  /etc/init.d/zabbix_agentd  ; chmod o+x

/etc/init.d/zabbix_agentd
```

```
#config zabbix agentd

cat >$INSTALL_DIR/etc/zabbix_agentd.conf<<EOF

LogFile=/tmp/zabbix_agentd.log

Server=$SERVER_IP

ServerActive=$SERVER_IP

Hostname = $IP

EOF

#start zabbix agentd

/etc/init.d/zabbix_agentd restart

/etc/init.d/iptables stop

setenforce 0

}



read -p "Please confirm whether to install Zabbix Server,yes or no? " INPUT

if [ $INPUT == "yes" -o $INPUT == "y" ] ; then

    SERVER_INSTALL

else

    AGENT_INSTALL

fi
```

17.8 Shell 编程实战 Nginx 虚拟主机

Nginx WEB 服务器的最大特点在于 Nginx 常被用于负载均衡、反向代理，单台 Nginx 服务器配置多个虚拟主机，百台服务器配置 N 多虚拟主机，基于 Shell 脚本可以更加高效的配置虚拟主机及添加、管理。

Shell 脚本实现 Nginx 自动安装及虚拟主机的维护，编写思路如下：

- 脚本指定参数 v1.jfedu.net；
- 创建 v1.jfedu.net 同时创建目录/var/www/v1；
- 将 Nginx 虚拟主机配置定向到新的目录；
- 重复虚拟主机不再添加。

Shell 脚本实现 Nginx 自动安装及虚拟主机的维护，代码如下：

```
#!/bin/bash
#Auto config Nginx virtual Hosts
#By author jfedu.net 2022
#Define Path variables
NGINX_CONF="/usr/local/nginx/conf/"
NGINX_MAKE="--user=www --group=www --prefix=/usr/local/nginx --with-
http_stub_status_module --with-http_ssl_module"
NGINX_SBIN="/usr/local/nginx/sbin/nginx"
```

```
NGINX_INSTALL(){

#Install Nginx server

NGINX_FILE=nginx-1.12.0.tar.gz

NGINX_DIR=`echo $NGINX_FILE|sed 's/.tar*.*//g'`

if [ ! -e /usr/local/nginx/ -a ! -e /etc/nginx/ ] ; then

    pkill nginx

    wget -c http://nginx.org/download/$NGINX_FILE

    yum install pcre-devel pcre -y

    rm -rf $NGINX_DIR  ; tar xf $NGINX_FILE

    cd $NGINX_DIR ; useradd www ; ./configure $NGINX_MAKE

    make &&make install

    grep -vE "#|^$"

$NGINX_CONF/nginx.conf >$NGINX_CONF/nginx.conf.swp

    \mv   $NGINX_CONF/nginx.conf.swp $NGINX_CONF/nginx.conf

    for i in `seq 1 6` ; do sed -i '$d' $NGINX_CONF/nginx.conf ; done

    echo "}" >>$NGINX_CONF/nginx.conf

    cd ../

fi

}
```

```
NGINX_CONFIG(){

#config tomcat nginx vhosts

grep "include domains" $NGINX_CONF/nginx.conf >>/dev/null

if [ $? -ne 0 ] ; then

    #sed -i '$d' $NGINX_CONF/nginx.conf

    echo -e "\ninclude domains/* ; \n}" >>$NGINX_CONF/nginx.conf

    mkdir -p $NGINX_CONF/domains/

fi

VHOSTS=$1

ls $NGINX_CONF/domains/$VHOSTS>>/dev/null 2>&1

if [ $? -ne 0 ] ; then

    #cp -r xxx.jfedu.net $NGINX_CONF/domains/$VHOSTS

    #sed -i "s/xxx/$VHOSTS/g" $NGINX_CONF/domains/$VHOSTS

    cat>$NGINX_CONF/domains/$VHOSTS<<EOF

    #vhost server $VHOSTS

    server {

        listen          80 ;

        server_name   $VHOSTS ;

        location / {
```

```
            root     /data/www/$VHOSTS/ ;
            index    index.html index.htm ;
        }
    }
EOF
    mkdir -p /data/www/$VHOSTS/
    cat>/data/www/$VHOSTS/index.html<<EOF
    <html>
    <h1><center>The First Test Nginx page.</center></h1>
    <hr color="red">
    <h2><center>$VHOSTS</center></h2>
    </html>
EOF
    echo -e "\033[32mThe $VHOSTS Config success,You can to access
http://$VHOSTS/\033[0m"
    NUM=`ps -ef |grep nginx|grep -v grep|grep -v auto|wc -l`
    $NGINX_SBIN -t >>/dev/null 2>&1
    if [ $? -eq 0 -a $NUM -eq 0 ] ; then
        $NGINX_SBIN
```

```
    else
        $NGINX_SBIN -t >>/dev/null 2>&1
        if [ $? -eq 0 ] ; then
            $NGINX_SBIN -s reload
        fi
    fi
else
    echo -e "\033[32mThe $VHOSTS has been config,Please exit.\033[0m"
fi
}
if [ -z $1 ] ; then
    echo -e "\033[32m--------------------\033[0m"
    echo -e "\033[32mPlease enter sh $0 xx.jf.com.\033[0m"
    exit 0
fi
NGINX_INSTALL
NGINX_CONFIG $1
```

17.9 Shell 编程实战 Nginx、Tomcat 脚本

Tomcat 用于发布 JSP WEB 页面，根据企业实际需求，会在单台服务器配置 N 个 Tomcat 实例，同时手动将 Tomcat 创建后的实例加入至 Nginx 虚拟主机中，同时重启 Nginx，开发 Nginx、Tomcat 自动创建 Tomcat 实例及 Nginx 虚拟主机管理脚本能让大大的减轻人工的干预，实现快速的交付。

Shell 脚本实现 Nginx 自动安装、虚拟主机及自动将 Tomcat 加入至虚拟主机，编写思路如下：

- 手动拷贝 Tomcat 与脚本一致目录（可自动修改）；
- 手动修改 Tomcat 端口为 6001、7001、8001（可自动修改）；
- 脚本指定参数 v1.jfedu.net；
- 创建 v1.jfedu.net Tomcat 实例；
- 修改 Tomcat 实例端口，保证 Port 唯一；
- 将 Tomcat 实例加入 Nginx 虚拟主机；
- 重复创建 Tomcat 实例，端口自动增加，并加入原 Nginx 虚拟主机，实现负载均衡；

Shell 脚本实现 Nginx 自动安装、虚拟主机及自动将 Tomcat 加入至虚拟主机，代码如下：

```
#!/bin/bash
```

```
#Auto config Nginx and tomcat cluster

#By author jfedu.net 2022

#Define Path variables

NGINX_CONF="/usr/local/nginx/conf/"

install_nginx(){

    NGINX_FILE=nginx-1.10.2.tar.gz

    NGINX_DIR=`echo $NGINX_FILE|sed 's/.tar*.*//g'`

    wget -c http://nginx.org/download/$NGINX_FILE

    yum install pcre-devel pcre -y

    rm -rf $NGINX_DIR  ; tar xf $NGINX_FILE

    cd $NGINX_DIR ; useradd www ; ./configure --user=www --group=www -

-prefix=/usr/local/nginx2 --with-http_stub_status_module --with-

http_ssl_module

    make &&make install

    cd ../

}

install_tomcat(){

    JDK_FILE="jdk1.7.0_25.tar.gz"

    JDK_DIR=`echo $JDK_FILE|sed 's/.tar.*//g'`
```

```
    tar -xzf $JDK_FILE    ; mkdir -p /usr/java/   ; mv $JDK_DIR /usr/java/
    sed -i '/JAVA_HOME/d ; /JAVA_BIN/d ; /JAVA_OPTS/d' /etc/profile
    cat >> /etc/profile <<EOF
    export JAVA_HOME=/export/servers/$JAVA_DIR
    export JAVA_BIN=/export/servers/$JAVA_DIR/bin
    export PATH=\$JAVA_HOME/bin:\$PATH
    export CLASSPATH=.:\$JAVA_HOME/lib/dt.jar:\$JAVA_HOME/lib/tools.jar
    export JAVA_HOME JAVA_BIN PATH CLASSPATH
EOF
    source /etc/profile ; java -version
    #install tomcat start
    ls tomcat
}
config_tomcat_nginx(){
    #config tomcat nginx vhosts
    grep "include domains" $NGINX_CONF/nginx.conf >>/dev/null
    if [ $? -ne 0 ] ; then
        sed -i '$d' $NGINX_CONF/nginx.conf
        echo -e "\ninclude domains/* ; \n}" >>$NGINX_CONF/nginx.conf
```

```
        mkdir -p $NGINX_CONF/domains/
fi
VHOSTS=$1
NUM=`ls /usr/local/|grep -c tomcat`
if [ $NUM -eq 0 ] ; then
        cp -r tomcat /usr/local/tomcat_$VHOSTS
        cp -r xxx.jfedu.net $NGINX_CONF/domains/$VHOSTS
        #sed -i "s/VHOSTS/$VHOSTS/g" $NGINX_CONF/domains/$VHOSTS
        sed -i "s/xxx/$VHOSTS/g" $NGINX_CONF/domains/$VHOSTS
        exit 0
fi
#-------------------------------
#VHOSTS=$1
VHOSTS_NUM=`ls $NGINX_CONF/domains/|grep -c $VHOSTS`
SERVER_NUM=`grep -c "127" $NGINX_CONF/domains/$VHOSTS`
SERVER_NUM_1=`expr $SERVER_NUM + 1`
rm -rf /tmp/.port.txt
for i in `find /usr/local/ -maxdepth 1 -name "tomcat*"` ; do
        grep "port" $i/conf/server.xml |egrep -v "\--|8080|SSLEnabled"|awk
```

```
'{print $2}'|sed 's/port=//g；s/\"//g'|sort -nr >>/tmp/.port.txt
    done
    MAX_PORT=`cat /tmp/.port.txt|grep -v 8443|sort -nr|head -1`
    PORT_1=`expr $MAX_PORT - 2000 + 1`
    PORT_2=`expr $MAX_PORT - 1000 + 1`
    PORT_3=`expr $MAX_PORT + 1`
    if [ $VHOSTS_NUM -eq 1 ]；then
        read -p "The $VHOSTS is exists,You sure create mulit Tomcat for the
$VHOSTS? yes or no " INPUT
        if [ $INPUT == "YES" -o $INPUT == "Y" -o $INPUT == "yes" ]；then
            cp -r tomcat /usr/local/tomcat_${VHOSTS}_${SERVER_NUM_1}
            sed -i "s/6001/$PORT_1/g"
/usr/local/tomcat_${VHOSTS}_${SERVER_NUM_1}/conf/server.xml
            sed -i "s/7001/$PORT_2/g"
/usr/local/tomcat_${VHOSTS}_${SERVER_NUM_1}/conf/server.xml
            sed -i "s/8001/$PORT_3/g"
/usr/local/tomcat_${VHOSTS}_${SERVER_NUM_1}/conf/server.xml
            sed -i "/^upstream/a        server 127.0.0.1:${PORT_2} weight=1
max_fails=2 fail_timeout=30s；" $NGINX_CONF/domains/$VHOSTS
```

```
                exit 0
        fi
        exit
    fi
    cp -r tomcat /usr/local/tomcat_$VHOSTS
        cp -r xxx.jfedu.net $NGINX_CONF/domains/$VHOSTS
        sed -i "s/VHOSTS/$VHOSTS/g" $NGINX_CONF/domains/$VHOSTS
        sed -i "s/xxx/$VHOSTS/g" $NGINX_CONF/domains/$VHOSTS
    sed -i "s/7001/${PORT_2}/g" $NGINX_CONF/domains/$VHOSTS
    #######config tomcat
        sed -i "s/6001/$PORT_1/g"
/usr/local/tomcat_${VHOSTS}/conf/server.xml
        sed -i "s/7001/$PORT_2/g"
/usr/local/tomcat_${VHOSTS}/conf/server.xml
        sed -i "s/8001/$PORT_3/g"
/usr/local/tomcat_${VHOSTS}/conf/server.xml
}
if [ ! -d $NGINX_CONF -o ! -d   /usr/java/$JDK_DIR ] ; then
    install_nginx
```

```
    install_tomcat
fi
config_tomcat_nginx $1
```

17.10 Shell 编程实战 Docker 管理脚本

Docker 虚拟化是目前主流的虚拟化解决方案，越来越多的企业在使用 Docker 轻量级虚拟化，构建、维护和管理 Docker 虚拟化平台是运维人员非常重要的一个环节，开发 Docker Shell 脚本可以在命令行界面快速管理和维护 Docker。

Shell 脚本实现 Docker 自动安装、自动导入镜像、创建虚拟机、指定 IP 地址、将创建的 Docker 虚拟机加入 Excel 存档或者加入 MYSQL 数据库，编写思路如下：

- 基于 CentOS6.5+或者 7.x YUM 安装 Docker；
- Docker 脚本参数指定 CPU、内存、硬盘容量；
- Docker 自动检测局域网 IP 并赋予 Docker 虚拟机；
- Docker 基于 pipework 指定 IP；
- 将创建的 Docker 虚拟机加入至 CSV（Excel）或者 MYSQL 库；

Shell 脚本实现 Docker 自动安装、自动导入镜像、创建虚拟机、指定 IP 地址、将创建的 Docker 虚拟机加入 CSV（Excel）存档或者加入 MYSQL 数据库，代码如下：

```
#!/bin/bash
#Auto install docker and Create VM
```

```
#By author jfedu.net 2022

#Define Path variables

IPADDR=`ifconfig|grep -E "\<inet\>"|awk '{print $2}'|grep "192.168"|head -1`

GATEWAY=`route -n|grep "UG"|awk '{print $2}'|grep "192.168"|head -1`

IPADDR_NET=`ifconfig|grep -E "\<inet\>"|awk '{print $2}'|grep "192.168"|head

-1|awk -F. '{print $1"."$2"."$3"."}'`

LIST="/root/docker_vmlist.csv"

if [ ! -f /usr/sbin/ifconfig ] ; then

    yum install net-tools* -y

fi

for i in `seq 1 253` ; do ping -c 1 ${IPADDR_NET}${i}  ; [ $? -ne 0 ]&&

DOCKER_IPADDR="${IPADDR_NET}${i}" &&break ; done >>/dev/null 2>&1

echo "###################"

echo -e "Dynamic get docker IP,The Docker IP address\n\n$DOCKER_IPADDR"

NETWORK=(

    HWADDR=`ifconfig eth0|grep ether|awk '{print $2}'`

    IPADDR=`ifconfig eth0|grep -E "\<inet\>"|awk '{print $2}'`

    NETMASK=`ifconfig eth0|grep -E "\<inet\>"|awk '{print $4}'`

    GATEWAY=`route -n|grep "UG"|awk '{print $2}'`
```

```
)
if [ -z "$1" -o -z "$2" ] ; then
    echo -e "\033[32m--------------------------------\033[0m"
    echo -e "\033[32mPlease exec $0 CPU(C) MEM(G),example $0 4 8\033[0m"
    exit 0
fi
#CPU=`expr $2 - 1`
if [ ! -e /usr/bin/bc ] ; then
    yum install bc -y >>/dev/null 2>&1
fi
CPU_ALL=`cat /proc/cpuinfo |grep processor|wc -l`
if [ ! -f $LIST ] ; then
    CPU_COUNT=$1
    CPU_1="0"
    CPU1=`expr $CPU_1 + 0`
    CPU2=`expr $CPU1 + $CPU_COUNT - 1`
    if [ $CPU2 -gt $CPU_ALL ] ; then
        echo -e "\033[32mThe System CPU count is $CPU_ALL,not more than
it.\033[0m"
```

```
            exit
        fi
else
        CPU_COUNT=$1
        CPU_1=`cat $LIST|tail -1|awk -F"," '{print $4}'|awk -F"-" '{print $2}'`
        CPU1=`expr $CPU_1 + 1`
        CPU2=`expr $CPU1 + $CPU_COUNT - 1`
        if [ $CPU2 -gt $CPU_ALL ] ; then
            echo -e "\033[32mThe System CPU count is $CPU_ALL,not more than
it.\033[0m"
            exit
        fi
fi
MEM_F=`echo $2 \* 1024|bc`
MEM=`printf "%.0f\n" $MEM_F`
DISK=20
USER=$3
REMARK=$4
ping $DOCKER_IPADDR -c 1 >>/dev/null 2>&1
```

```
if [ $? -eq 0 ] ; then
    echo -e "\033[32m--------------------------------\033[0m"
    echo -e "\033[32mThe IP address to be used,Please change other
IP,exit.\033[0m"
    exit 0
fi
if [ ! -e /usr/bin/docker ] ; then
    yum install docker* device-mapper*   -y
    mkdir -p /export/docker/
    cd /var/lib/  ; rm -rf docker  ; ln -s /export/docker/ .
    mkdir -p /var/lib/docker/devicemapper/devicemapper
    dd if=/dev/zero of=/var/lib/docker/devicemapper/devicemapper/data
bs=1G count=0 seek=2000
    service docker start
    if [ $? -ne 0 ] ; then
        echo "Docker install error ,please check."
        exit
    fi
fi
```

```
cd   /etc/sysconfig/network-scripts/

     mkdir -p /data/backup/`date +%Y%m%d-%H%M`

     yes|cp ifcfg-eth* /data/backup/`date +%Y%m%d-%H%M`/

if

     [ -e /etc/sysconfig/network-scripts/ifcfg-br0 ] ; then

     echo

else

     cat >ifcfg-eth0 <<EOF

     DEVICE=eth0

     BOOTPROTO=none

     ${NETWORK[0]}

     NM_CONTROLLED=no

     ONBOOT=yes

     TYPE=Ethernet

     BRIDGE="br0"

     ${NETWORK[1]}

     ${NETWORK[2]}

     ${NETWORK[3]}

     USERCTL=no
```

```
EOF
        cat >ifcfg-br0 <<EOF
        DEVICE="br0"
        BOOTPROTO=none
        ${NETWORK[0]}
        IPV6INIT=no
        NM_CONTROLLED=no
        ONBOOT=yes
        TYPE="Bridge"
        ${NETWORK[1]}
        ${NETWORK[2]}
        ${NETWORK[3]}
        USERCTL=no
EOF
        /etc/init.d/network restart
fi
echo 'Your can restart Ethernet Service: /etc/init.d/network restart !'
echo '----------------------------------------------------------'
```

```
cd -

#######create docker container

service docker status >>/dev/null

if [ $? -ne 0 ] ; then

    service docker restart

fi

NAME="Docker_`echo $DOCKER_IPADDR|awk -F"." '{print $(NF-1)"_"$NF}'`"

IMAGES=`docker images|grep -v "REPOSITORY"|grep -v "none"|grep

"jfedu"|head -1|awk '{print $1}'`

if [ -z $IMAGES ] ; then

    echo "Plesae Download Docker Centos Images,you can to be use docker

search centos,and docker pull centos6.5-ssh,exit 0"

    if [ ! -f jfedu_centos68.tar ] ; then

        echo "Please upload jfedu_centos68.tar for docker server."

        exit

    fi

    cat jfedu_centos68.tar|docker import - jfedu_centos6.8

fi

IMAGES=`docker images|grep -v "REPOSITORY"|grep -v "none"|grep
```

```
"jfedu"|head -1|awk '{print $1}'`

CID=$(docker run -itd --privileged --cpuset-cpus=${CPU1}-${CPU2} -m

${MEM}m --net=none --name=$NAME $IMAGES /bin/bash)

echo $CID

docker ps -a |grep "$NAME"

pipework br0 $NAME    $DOCKER_IPADDR/24@$IPADDR

docker exec $NAME /etc/init.d/sshd start

if [ ! -e $LIST ] ; then

      echo "编号,容器 ID,容器名称,CPU,内存,硬盘,容器 IP,宿主机 IP,使用人,备注

" >$LIST

fi

####################

NUM=`cat $LIST |grep -v CPU|tail -1|awk -F, '{print $1}'`

if [[ $NUM -eq "" ]] ; then

            NUM="1"

else

            NUM=`expr $NUM + 1`

fi

###################
```

```
echo -e "\033[32mCreate virtual client Successfully.\n$NUM `echo $CID|cut -b
1-12`,$NAME,$CPU1-
$CPU2,${MEM}M,${DISK}G,$DOCKER_IPADDR,$IPADDR,$USER,$REMARK\033[
0m"
if [ -z $USER ]；then
    USER="NULL"
    REMARK="NULL"
fi
echo $NUM,`echo $CID|cut -b 1-12`,$NAME,$CPU1-
$CPU2,${MEM}M,${DISK}G,$DOCKER_IPADDR,$IPADDR,$USER,$REMARK >>$L
IST
rm -rf /root/docker_vmlist_*
iconv -c -f utf-8 -t gb2312 $LIST   -o /root/docker_vmlist_`date +%H%M`.csv
```

17.11 Shell 编程实战 Bind 管理脚本

Bind 主要应用于企业 DNS 构建平台，而 DNS 用于将域名与 IP 进行解析，用户在浏览器只需输入域名，即可访问服务器 IP 地址的虚拟主机网站。

Bind 难点在于创建各种记录，例如 A 记录、mail 记录、反向记录、资源记录，基于 Shell 脚本可以减轻人工的操作，节省大量的时间成本。

Shell 脚本实现 Bind 自动安装、初始化 Bind 环境、自动添加 A 记录、反向记录、批量添加 A 记录，编写思路如下：

- YUM 方式自动安装 Bind；
- 自动初始化 Bind 配置；
- 创建安装、初始化、添加记录函数；
- 自动添加单个 A 记录及批量添加 A 记录和反向记录；

Shell 脚本实现 Bind 自动安装、初始化 Bind 环境、自动添加 A 记录、反向记录、批量添加 A 记录，代码如下：

```
#!/bin/bash
#Auto install config bind server
#By author jfedu.net 2022
#Define Path variables
BND_ETC=/var/named/chroot/etc
BND_VAR=/var/named/chroot/var/named
BAK_DIR=/data/backup/dns_`date +%Y%m%d-%H%M`
##Backup named server
if
        [ ! -d    $BAK_DIR ] ; then
        echo "Please waiting    Backup Named Config ....."
```

```
        mkdir     -p    $BAK_DIR

        cp -a    /var/named/chroot/{etc,var}     $BAK_DIR

        cp -a    /etc/named.* $BAK_DIR

fi

##Define Shell Install Function

Install ()

{

   if

       [ ! -e /etc/init.d/named ] ; then

       yum install bind* -y

else

       echo ----------------------------------------------

       echo "The Named Server is exists ,Please exit ....."

       sleep 1

 fi

}

##Define Shell Init Function

Init_Config ()

{
```

```
        sed    -i -e 's/localhost ; /any ; /g' -e '/port/s/127.0.0.1/any/g'
/etc/named.conf
        echo ----------------------------------------------
        sleep 2
        echo "The named.conf config Init success !"
}
##Define Shell Add Name Function
Add_named ()
{
##DNS name
        read -p    "Please    Insert Into Your Add Name ,Example 51cto.com :"
NAME
        echo $NAME |grep -E "com|cn|net|org"
        while
         [ "$?" -ne 0 ]
           do
         read -p    "Please    reInsert Into Your Add Name ,Example 51cto.com :"
NAME
         echo $NAME |grep -E "com|cn|net|org"
```

```
      done
## IP address
         read -p   "Please   Insert Into Your Name Server IP ADDress:" IP
         echo $IP |egrep -o "([0-9]{1,3}\.){3}[0-9]{1,3}"
         while
         [ "$?" -ne "0" ]
          do
          read -p   "Please   reInsert Into Your Name Server IP ADDress:" IP
         echo $IP |egrep -o "([0-9]{1,3}\.){3}[0-9]{1,3}"
        done
         ARPA_IP=`echo $IP|awk -F. '{print $3"."$2"."$1}'`
         ARPA_IP1=`echo $IP|awk -F. '{print $4}'`
         cd   $BND_ETC
         grep   "$NAME" named.rfc1912.zones
if
           [ $? -eq 0 ] ; then
           echo "The $NAME IS exist named.rfc1912.zones conf ,please exit ..."
           exit
else
```

```
        read -p    "Please    Insert Into SLAVE Name Server IP ADDress:" SLAVE
        echo $SLAVE |egrep -o "([0-9]{1,3}\.){3}[0-9]{1,3}"
        while
        [ "$?" -ne "0" ]
        do
            read -p    "Please    Insert Into SLAVE Name Server IP ADDress:"
SLAVE
            echo $SLAVE |egrep -o "([0-9]{1,3}\.){3}[0-9]{1,3}"
        done
            grep    "rev" named.rfc1912.zones
       if
         [ $? -ne 0 ] ; then
            cat >>named.rfc1912.zones <<EOF
#`date +%Y-%m-%d` Add $NAME CONFIG
zone "$NAME" IN {
        type master ;
        file "$NAME.zone" ;
        allow-update { none ;  } ;
} ;
```

```
zone "$ARPA_IP.in-addr.arpa" IN {
            type master ;
            file "$ARPA_IP.rev" ;
            allow-update { none ;  } ;
} ;
EOF
        else
            cat >>named.rfc1912.zones <<EOF

#`date +%Y-%m-%d` Add $NAME CONFIG
zone "$NAME" IN {
            type master ;
            file "$NAME.zone" ;
            allow-update { none ;  } ;
} ;
EOF
     fi
fi
          [ $? -eq 0 ]&& echo "The $NAME config name.rfc1912.zones success !"
```

```
        sleep 3   ; echo "Please waiting config $NAME zone File ....."
        cd   $BND_VAR
        read -p "Please insert Name DNS A HOST ,EXample   www or mail :"
HOST
        read -p "Please insert Name DNS A NS IP ADDR ,EXample
192.168.111.130 :" IP_HOST
        echo $IP_HOST |egrep -o "([0-9]{1,3}\.){3}[0-9]{1,3}"
        ARPA_IP2=`echo $IP_HOST|awk -F. '{print $3"."$2"."$1}'`
        ARPA_IP3=`echo $IP_HOST|awk -F. '{print $4}'`
        while
        [ "$?" -ne "0" ]
do
        read -p "Please Reinsert Name DNS A IPADDRESS ,EXample
192.168.111.130 :" IP_HOST
        echo $IP_HOST |egrep -o "([0-9]{1,3}\.){3}[0-9]{1,3}"
done
        cat >$NAME.zone <<EOF
\$TTL      86400
@                   IN SOA    localhost.          root.localhost. (
```

```
                                        43                  ;   serial (d. adams)
                                        1H                  ;   refresh
                                        15M                 ;   retry
                                        1W                  ;   expiry
                                        1D )                ;   minimum


                IN    NS              $NAME.
EOF
        REV=`ls    *.rev`
        ls    *.rev >>/dev/null
if
        [ $? -ne 0 ] ; then
        cat >>$ARPA_IP.rev <<EOF
\$TTL      86400
@          IN         SOA        localhost.      root.localhost.    (
                                        1997022703    ;   Serial
                                        28800         ;   Refresh
                                        14400         ;   Retry
                                        3600000       ;   Expire
```

```
                                        86400 )       ;  Minimum


                IN   NS   $NAME.
EOF
        echo   "$HOST                IN   A
$IP_HOST" >>$NAME.zone
        echo   "$ARPA_IP3            IN   PTR
$HOST.$NAME." >>$ARPA_IP.rev
        [ $? -eq 0 ]&& echo -e "The $NAME config success:\n$HOST          IN
A               $IP_HOST\n$ARPA_IP3            IN   PTR
$HOST.$NAME."
else
        sed -i   "9a IN   NS   $NAME." $REV
        echo   "$HOST                IN   A
$IP_HOST" >>$NAME.zone
        echo   "$ARPA_IP3            IN   PTR
$HOST.$NAME." >>$REV
        [ $? -eq 0 ]&& echo -e "The $NAME config success1:\n$HOST
IN   A              $IP_HOST\n$ARPA_IP3            IN   PTR
```

```
$HOST.$NAME."

fi

}

##Define Shell List A Function

Add_A_List ()

{

if

        cd   $BND_VAR

        REV=`ls   *.rev`

        read -p   "Please   Insert Into Your Add Name ,Example 51cto.com :"

NAME

        [ ! -e "$NAME.zone" ] ; then

        echo "The $NAME.zone File is not exist ,Please ADD $NAME.zone File :"

        Add_named   ;

else

        read -p "Please Enter List Name A NS File ,Example /tmp/name_list.txt:

" FILE

    if

        [ -e $FILE ] ; then
```

```
        for i in    `cat $FILE|awk '{print $2}'|sed "s/$NAME//g"|sed 's/\.$//g'`
        #for i in    `cat $FILE|awk '{print $1}'|sed "s/$NAME//g"|sed 's/\.$//g'`
do
        j=`awk -v I="$i.$NAME" '{if(I==$2)print $1}' $FILE`
        echo ----------------------------------------------------------
        echo "The $NAME.zone File is exist ,Please Enter insert NAME HOST ...."
        sleep 1
        ARPA_IP=`echo $j|awk -F. '{print $3"."$2"."$1}'`
        ARPA_IP2=`echo $j|awk -F. '{print $4}'`
        echo    "$i                  IN    A               $j" >>$NAME.zone
        echo    "$ARPA_IP2         IN    PTR         $i.$NAME." >>$REV
        [ $? -eq 0 ]&& echo -e "The $NAME config success:\n$i         IN    A
$j\n$ARPA_IP2             IN    PTR             $i.$NAME."
done
      else
        echo "The $FILE List File IS Not Exist .......,Please exit ..."
      fi
fi
}
```

```
##Define Shell Select Menu

PS3="Please select Menu Name Config: "

select i in "自动安装 Bind 服务"  "自动初始化 Bind 配置" "添加解析域名"  "批量添
加 A 记录"

do

case  $i  in

        "自动安装 Bind 服务")

        Install

;;

        "自动初始化 Bind 配置")

        Init_Config

;;

        "添加解析域名")

        Add_named

;;

        "批量添加 A 记录")

        Add_A_List

;;

        * )
```

```
        echo ----------------------------------------------------
        sleep 1
        echo "Please exec: sh   $0   { Install(1)   or Init_Config(2) or
Add_named(3) or Add_config_A(4) }"
 ;;
esac
done
```

第18章 黑客攻防 Linux 服务器企业实战

从各大云服务器厂商购买的云主机 ,通常都有外网 IP 地址 ,用户名和密码 ,通过 CRT 或者 Xshell 可以远程登录服务器的 22 端口。

每天大量的黑客通过各种工具，扫描登录你的服务器的 22 端口，企图以用户名和密码一直循环登录你的服务器，如果服务器的密码复杂度不够的话，被黑客拿到 root 或者其他普通用户密码，整个服务器就裸奔在黑客的眼皮底下了。这样就变成了人为刀俎我为鱼肉！

基于 DenyHosts 工具可以阻止试图猜测 SSH 登录口令，该软件会分析 /var/log/secure 等日志文件 ,当发现同一 IP 在进行多次 SSH 密码尝试时就将客户的 IP 记录到/etc/hosts.deny 文件，从而可以实现禁止该 IP 访问服务器。

18.1 基于二进制方式安装 DenyHosts

yum install epel-release -y

yum install denyhosts* -y

18.2 DenyHosts 配置目录详解

DenyHosts 软件涉及的配置文件和目录非常繁多 ,如下为每个目录具体的功能详解:

该目录中主要存放计划任务，日志压缩、chkconfig 、service 启动文件；

```
/etc/cron.d/denyhosts

/etc/denyhosts.conf

/etc/logrotate.d/denyhosts

/etc/rc.d/init.d/denyhosts

/etc/sysconfig/denyhosts
```

如下目录主要存放 denyhosts 所拒绝及允许的一些主机信息：

```
/var/lib/denyhosts/allowed-hosts

/var/lib/denyhosts/allowed-warned-hosts

/var/lib/denyhosts/hosts

/var/lib/denyhosts/hosts-restricted

/var/lib/denyhosts/hosts-root

/var/lib/denyhosts/hosts-valid
```

```
/var/lib/denyhosts/offset

/var/lib/denyhosts/suspicious-logins

/var/lib/denyhosts/sync-hosts

/var/lib/denyhosts/users-hosts

/var/lib/denyhosts/users-invalid

/var/lib/denyhosts/users-valid

/var/log/denyhosts
```

18.3 DenyHosts 配置实战

egrep -vE "^$|^#" /etc/denyhosts.conf

```
#系统安全日志文件，主要获取 ssh 信息

SECURE_LOG = /var/log/secure

# 拒绝写入 IP 文件 hosts.deny

HOSTS_DENY = /etc/hosts.deny

#过多久后清除已经禁止的，其中 w 代表周，d 代表天，h 代表小时，s 代表秒，m 代表分钟

PURGE_DENY = 4w

# denyhosts 所要阻止的服务名称

BLOCK_SERVICE  = sshd

# 允许无效用户登录失败的次数
```

```
DENY_THRESHOLD_INVALID = 3

# 允许普通用户登录失败的次数

DENY_THRESHOLD_VALID = 3

# 允许 ROOT 用户登录失败的次数

DENY_THRESHOLD_ROOT = 3

# 设定 deny host 写入到该资料夹

DENY_THRESHOLD_RESTRICTED = 1

# 将 deny 的 host 或 ip 纪录到 Work_dir 中

WORK_DIR = /var/lib/denyhosts

SUSPICIOUS_LOGIN_REPORT_ALLOWED_HOSTS=YES

# 是否做域名反解

HOSTNAME_LOOKUP=YES

# 将 DenyHOts 启动的 pid 纪录到 LOCK_FILE 中，已确保服务正确启动，防止同时启动多个

服务

LOCK_FILE = /var/lock/subsys/denyhosts

############## 管理员 Mail 地址

ADMIN_EMAIL = root

SMTP_HOST = localhost

SMTP_PORT = 25
```

```
SMTP_FROM = DenyHosts <nobody@localhost>

SMTP_SUBJECT = DenyHosts Report from $[HOSTNAME]

# 有效用户登录失败计数归零的时间

AGE_RESET_VALID=5d

# ROOT 用户登录失败计数归零的时间 AGE_RESET_ROOT=25d

# 用户的失败登录计数重置为 0 的时间(/usr/share/denyhosts/restricted-usernames)

AGE_RESET_RESTRICTED=25d

# 无效用户登录失败计数归零的时间

AGE_RESET_INVALID=10d

DAEMON_LOG = /var/log/denyhosts

DAEMON_SLEEP = 30s

# 该项与 PURGE_DENY 设置成一样，也是清除 hosts.deniedssh 用户的时间

DAEMON_PURGE = 1h
```

18.4 启动 DenyHosts 服务

service denyhosts restart

```
[root@www-jfedu-net ~]# ll /var/lib/denyhosts/allowed-hosts
-rw-r--r-- 1 root root 39 Feb 16  2015 /var/lib/denyhosts/allowed-
[root@www-jfedu-net ~]# cat /var/lib/denyhosts/allowed-hosts
# We mustn't block localhost
127.0.0.1
[root@www-jfedu-net ~]# service denyhosts restart
Redirecting to /bin/systemctl restart denyhosts.service
[root@www-jfedu-net ~]# ps -ef|grep denyhosts
root      9272     1  0 13:25 ?        00:00:00 /usr/bin/python2 /
und --unlock --config=/etc/denyhosts.conf
root      9276 31516  0 13:25 pts/0    00:00:00 grep --color=auto
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
```

分别测试 invalid、valid、root 三类用户设置不同的 ssh 连接失败次数，验证 Denyhosts：

允许 invalid 用户只能失败 5 次、ROOT 用户失败 4 次、valid 用户失败 10 次

```
DENY_THRESHOLD_INVALID = 5

DENY_THRESHOLD_VALID = 10

DENY_THRESHOLD_ROOT = 4
```

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
root@47.98.151.187's password:
Permission denied, please try again.
root@47.98.151.187's password:

[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
Wssh_exchange_identification: Connection closed by remote
[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
ssh_exchange_identification: Connection closed by remote h
[root@www-jfedu-net ~]#
```

18.5 删除被 DenyHosts 禁止 IP

删除一个已经禁止的主机 IP，在/etc/hosts.deny 删除无效，还需进入

/var/lib/denyhosts，进入以下操作：

1) 停止 DenyHosts 服务，service denyhosts stop；

2) /etc/hosts.deny 中删除被禁止的主机 IP；

3) vi /var/lib/denyhosts，并且删除已被添加的主机信息；

```
/var/lib/denyhosts/hosts
/var/lib/denyhosts/hosts-restricted
/var/lib/denyhosts/hosts-root
/var/lib/denyhosts/hosts-valid
/var/lib/denyhosts/users-hosts
/var/lib/denyhosts/users-invalid
/var/lib/denyhosts/users-valid
```

4) vi /var/lib/denyhosts/allowed-hosts 加入允许的 IP 即可；

5) 启动 DenyHosts 服务： service denyhosts restart

也可以通过 SHELL 指令批量解封：

for i in `ls /var/lib/denyhosts/`;do sed -i '/139.199.228.59/d' /var/lib/denyhosts/$i;done

sed -i '/139.199.228.59/d' /etc/hosts.deny

echo "139.199.228.59" >>/var/lib/denyhosts/allowed-hosts

```
[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
Wssh_exchange_identification: Connection closed by remote host
[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
ssh_exchange_identification: Connection closed by remote host
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
^C
[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
^C
[root@www-jfedu-net ~]# ssh -l root 47.98.151.187
root@47.98.151.187's password:
Last login: Wed Apr 25 13:28:47 2018 from 139.224.227.121

Welcome to Jfedu Elastic Compute Service !

[root@www-jfedu-net ~]#
```

```
#!/bin/bash

#2022 年 7 月 15 日 21:38:32

#auto remove deny ip

#by author www.jfedu.net

########################

DENY_DIR="/var/lib/denyhosts"

DENY_IP=$1

if [ $# -eq 0 ];then

        echo -e "\033[32m----------------\033[0m"

        echo        -e        "\033[32mUsage:{/bin/bash        $0

1.1.1.1|192.168.1.100|help}\033[0m"

        exit 0

fi

if [ $1 == "help" ];then
```

```
        echo -e "\033[32m----------------\033[0m"
        echo -e "\033[32mUsage:{/bin/bash $0
1.1.1.1|192.168.1.100|help}\033[0m"
        exit 1
fi
for i in `ls $DENY_DIR/`
do
        sed -i "/$DENY_IP/d" $DENY_DIR/$i
done
sed -i "/$DENY_IP/d" /etc/hosts.deny
echo "$DENY_IP" >>$DENY_DIR/allowed-hosts
echo -e "\033[32m----------------\033[0m"
echo -e "\033[32mThe Deny IP removed Success.\033[0m"
service denyhosts restart
```

18.6 配置 Denyhosts 发送报警邮件

vi /etc/denyhosts.conf，配置 SMTP 相关如下代码：

```
ADMIN_EMAIL = wgkgood@163.com
SMTP_HOST = smtp.163.com
```

```
SMTP_PORT = 25

SMTP_FROM = wgkgood@163.com

SMTP_PASSWORD= jfedu6666
```

18.7 基于 SHELL 全自动脚本实现防黑客攻击

企业服务器暴露在外网，每天会有大量的人使用各种用户名和密码尝试登陆服务器，如果让其一直尝试，难免会猜出密码，通过开发 Shell 脚本，可以自动将尝试登陆服务器错误密码次数的 IP 列表加入到防火墙配置中。

Shell 脚本实现服务器拒绝恶意 IP 登陆，编写思路如下：

- 登陆服务器日志/var/log/secure；
- 检查日志中认证失败的行并打印其 IP 地址；
- 将 IP 地址写入至防火墙；
- 禁止该 IP 访问服务器 SSH 22 端口；
- 将脚本加入 Crontab 实现自动禁止恶意 IP；

Shell 脚本实现服务器拒绝恶意 IP 登陆，代码如下：

```
#!/bin/bash

#Auto drop ssh failed IP address

#By author jfedu.net 2022

#Define Path variables
```

```
SEC_FILE=/var/log/secure

IP_ADDR=`awk '{print $0}' /var/log/secure|grep -i "fail"| egrep -o "([0-

9]{1,3}\.){3}[0-9]{1,3}" | sort -nr | uniq -c |awk '$1>=15 {print $2}'`

IPTABLE_CONF=/etc/sysconfig/iptables

echo

cat <<EOF

++++++++++++++welcome to use ssh login drop failed

ip++++++++++++++++++

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

++++++++++++++++++++

++++++++++++++++----------------------------------

++++++++++++++++++

EOF

echo

for ((j=0 ; j<=6 ; j++)) ; do echo -n "-" ; sleep 1 ; done

echo

for i in `echo $IP_ADDR`

do

   cat $IPTABLE_CONF |grep $i >/dev/null
```

```
if
   [ $? -ne 0 ] ; then
    sed -i "/lo/a -A INPUT -s $i -m state --state NEW -m tcp -p tcp --dport 22 -j
DROP" $IPTABLE_CONF
fi
done
NUM=`find /etc/sysconfig/    -name iptables -a -mmin -1|wc -l`
          if [ $NUM -eq 1 ] ; then
                    /etc/init.d/iptables restart
          fi
```

第19章　企业邮件入门简介

邮件服务器是一种用来负责电子邮件收发管理的设备。自主构建邮件服务器比网络上的免费邮箱更安全和高效，因此邮件服务器是每个公司必备的服务软件之一。对于运维 SA 需要掌握服务器相关的知识以及维护方法。

邮件服务器构成了电子邮件系统的核心，每个收信人都有一个位于某个邮件服务器上的邮箱(mailbox)，一个邮件消息的典型旅程

是从发信人的用户代理开始，邮件发信人的邮件服务器，中转到收信人的邮件服务器，然后投递到收信人的邮箱中。

简单邮件传送协议(Simple Mail Transfer Protocol，SMTP)是因特网电子邮件系统首要的应用层协议。它使用由 TCP 提供的可靠的数据传输服务把邮件消息从发信人的邮件服务器传送到收信人的邮件服务器。

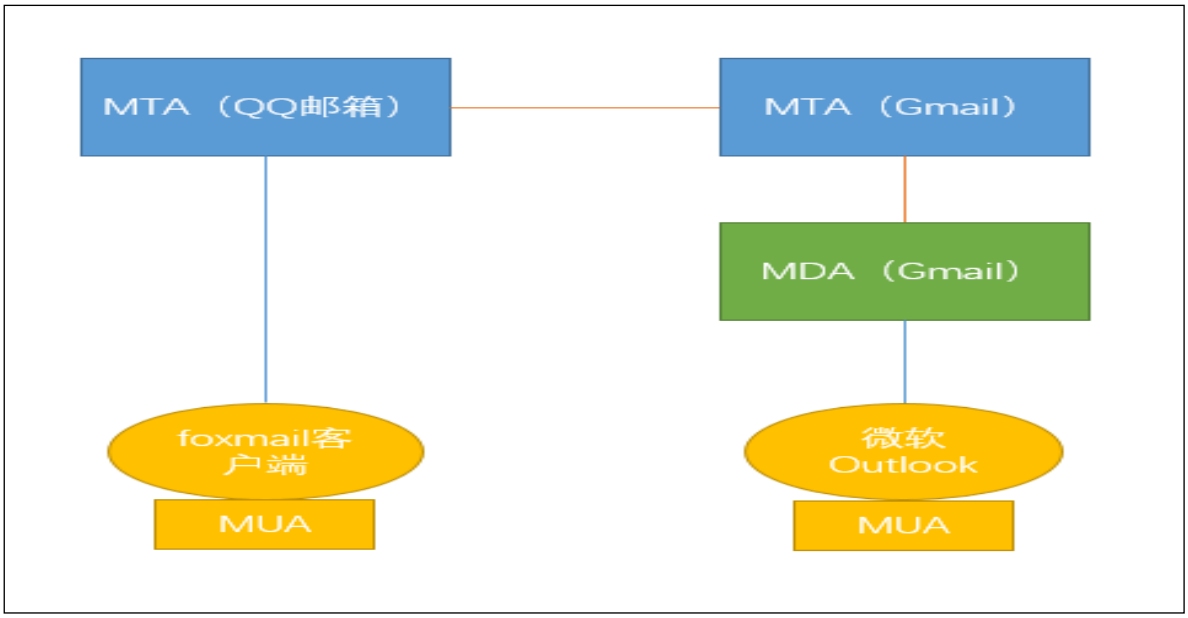
为使用户的系统域名能被正确解析为相应服务器地址，邮件服务器需要能够在互联网络上被识别和查找到。这样邮件系统才能实现邮件的投递和接收。因此邮件服务器需要对此做 DNS 设置，包括 MX 记录和 A 记录的设置。

整个邮件系统包括:服务器端和客户端，服务器需要基于 SMTP 协议，客户端基于 POP3、IMAP 等协议。SMTP 监听端口为 TCP 25 端口，POP3 监听端口为 110，IMAP 监听端口为 143。

例如当发送一封电子邮件信息时，信息会一台服务器、一台服务器地的被传递，直到发送到收件人的电子邮件服务器。

更准确地说，信息被发送到负责传输邮件的服务器（被称为邮件传输代理（Mail Transport Agent，MTA），经过若干 MTA 后，最终到达收件人的 MTA。在互联网上，MTA 之间使用 SMTP 协议进行通信，也称为 SMTP 服务器。

收件人的 MTA 会将电子邮件投递给邮件接收服务器（被称为邮件投递代理（Mail Delivery Agent，MDA），MDA 会保存邮件并等待用户收取。MDA 主要有两种协议：POP（Post Office Protocol）。



最简单比喻就是：MTA 类似邮局，而 MDA 类似信箱，MDA 存储邮件并等待收件人检查信箱。收件人不需要让发件人与收件人建立连接。为避免人们的邮件被其他人看到，MDA 要验证用户名和密码才能访问。收取邮件的工作由一个叫做邮件用户代理(Mail User Agent，MUA）的程序来完成。如果 MUA 是用户电脑或其他设备上的一个程序，则称它为邮件客户端（如 Mozilla Thunderbird，网易邮箱大师，foxmail，微软的 Outlook）。

POP3：协议允许电子邮件客户端下载服务器上的邮件，但是在客户端的操作（如移动邮件、标记已读等），不会反馈到服务器上，比如通过客户端收取了邮箱中的 3 封邮件并移动到其他文件夹，邮箱服务器上的这些邮件是没有同时被移动的，所以有很多同学反馈使用 foxmail 客户端配置 pop3 收取邮件的时候，有时候非常快，而使用 IMAP 方式，收取打开邮件非常慢。

IMAP：协议提供 webmail 与电子邮件客户端之间的双向通信，客户端的操作都会反馈到服务器上，对邮件进行的操作，服务器上的邮件也会做相应的动作。

目前主流免费邮箱服务商有：网易邮箱、QQ 邮箱、新浪、搜狐邮箱等等。邮箱的选择，如果是个人甚至中小型公司的话，可以直接使用网易或者 QQ 企业邮箱。因为维护一个邮件服务器也是一项非常庞大的工程。

Linux 平台开源免费的邮件服务器包括：Sendmail、Postfix、Qmail；而 Windows 平台主要为 Exchange 服务器（正版需要收费），如下章节主要实战基于 Linux 平台来构建企业独立的邮件服务器系统。

邮件相关名称概念理解，汇总如下：

MUA：用户代理端，即用户使用的写信、收信客户端软件;

MTA：邮件传送端，即常说的邮件服务器，用于转发、收取用户邮件。

MDA :邮件代理端，相当于 MUA 和 MTA 的中间人，可用于过滤垃圾邮件。

POP :邮局协议，用于 MUA 连接服务器收取用户邮件，通信端口 110。

IMAP：互联网应用协议，功能较 POP 多，通信端口 143。

SMTP：简单邮件传送协议，MUA 连接 MTA 或 MTA 连接 MTA 发送邮件使用此协议，通信端口 25。

19.1 Sendmail 安装配置

作为 Linux/UNIX 下的老牌邮件服务器，Sendmail 作为一种免费的邮件服务器软件，已被广泛的应用于各种服务器中，它在稳定性、可移植性、及确保没有 bug 等方面具有一定的特色，且可以在网络中搜索到大量的使用资料，是一款最经典的 Linux 系统下邮件服务器。

（1） Sendmail 环境版本

系统版本：CentOS 6.8 64 位

Sendmail 版本：Sendmail-8.14

Openwebmail 版本：openwebmail-2.53-6、openwebmail-data-2.53-6

（2） SendMail 安装

```
yum install sendmail* -y
```

```
[root@localhost ~]# yum install sendmail*

Loaded plugins: fastestmirror
Setting up Install Process
Determining fastest mirrors
base
extras
updates
Resolving Dependencies
--> Running transaction check
---> Package sendmail.x86_64 0:8.14.4-9.el6_8.1 will be instal
--> Processing Dependency: procmail for package: sendmail-8.14
--> Processing Dependency: libhesiod.so.0()(64bit) for package
---> Package sendmail-cf.noarch 0:8.14.4-9.el6_8.1 will be ins
---> Package sendmail-devel.x86_64 0:8.14.4-9.el6_8.1 will be
---> Package sendmail-doc.noarch 0:8.14.4-9.el6_8.1 will be in
---> Package sendmail-milter.x86_64 0:8.14.4-9.el6_8.1 will be
```

```
  Verifying  : sendmail-devel-8.14.4-9.el6_8.1.x86_64
  Verifying  : sendmail-8.14.4-9.el6_8.1.x86_64
  Verifying  : procmail-3.22-25.1.el6_5.1.x86_64
  Verifying  : sendmail-milter-8.14.4-9.el6_8.1.x86_64
  Verifying  : sendmail-doc-8.14.4-9.el6_8.1.noarch
  Verifying  : sendmail-cf-8.14.4-9.el6_8.1.noarch

Installed:
  sendmail.x86_64 0:8.14.4-9.el6_8.1                    sen
  sendmail-devel.x86_64 0:8.14.4-9.el6_8.1              sen
  sendmail-milter.x86_64 0:8.14.4-9.el6_8.1

Dependency Installed:
  hesiod.x86_64 0:3.1.0-19.el6                    procmail

Complete!
```

```
[root@localhost yum.repos.d]# yum install openwebmail
Loaded plugins: fastestmirror
Setting up Install Process
Loading mirror speeds from cached hostfile
openwebmail
openwebmail/primary_db
Resolving Dependencies
--> Running transaction check
---> Package openwebmail.noarch 0:2.53-6.el6 will be installe
--> Processing Dependency: openwebmail-data = 2.53-6.el6 for
--> Processing Dependency: perl-suidperl for package: openweb
--> Processing Dependency: perl-Text-Iconv for package: openw
--> Processing Dependency: perl-Compress-Zlib for package: op
--> Processing Dependency: perl-CGI for package: openwebmail-
--> Running transaction check
---> Package openwebmail-data.noarch 0:2.53-6.el6 will be ins
```

（3） SendMail 服务配置

配置 sendmail.cf 服务，通过 local-host-names 设置邮件服务器提供邮件服务的域名为 jfteach.com(jingfengjiaoyu.com)；

```
cp      /etc/mail/sendmail.mc /etc/mail/sendmail.mc.back
cp      /etc/mail/sendmail.cf /etc/mail/sendmail.cf.back
echo    "jfteach.com" >>/etc/mail/local-host-names
```

配置 Sendmail 监听服务器网卡地址为 0.0.0.0：

```
sed          -i          's/Addr=127.0.0.1/Addr=0.0.0.0/g'
/etc/mail/sendmail.mc
```

修改 vi /etc/mail/sendmail.mc 如下两行，开启 SMTP 所有的用户必须认证。

```
Dnl  TRUST_AUTH_MECH(`EXTERNAL  DIGEST-MD5  CRAM-
```

```
MD5 LOGIN PLAIN’)dnl
Dnl define(`confAUTH_MECHANISMS’,`EXTERNAL GSSAPI
DIGEST-MD5 CRAM-MD5 LOGIN PLAIN’)dnl
```

修改为如下：

```
TRUST_AUTH_MECH(`EXTERNAL DIGEST-MD5 CRAM-MD5
LOGIN PLAIN’)dnl
define(`confAUTH_MECHANISMS’ ,`EXTERNAL GSSAPI
DIGEST-MD5 CRAM-MD5 LOGIN PLAIN’)dnl
```

即去掉首行的 dnl 。这样 sendmail 配置完毕。

Sendmail.mc 文件修改完毕，使用 m4 命令生成 sendmail.cf 主配置文件：

```
m4 sendmail.mc >sendmail.cf
```

```
[root@localhost ~]# /etc/init.d/sendmail restart
Shutting down sm-client:                                   [
Shutting down sendmail:                                    [
                                                           [
Starting sendmail:                                         [
Starting sm-client:                                        [
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]# ps -ef |grep sendmail
root      8905     1  0 21:24 ?        00:00:00 sendmail: acc
smmsp     8914     1  0 21:24 ?        00:00:00 sendmail: Que
queue
root      8918  8704  0 21:24 pts/0    00:00:00 grep sendmail
[root@localhost ~]# netstat -tnl |grep 25
tcp        0      0 0.0.0.0:25                  0.0.0.0:*
[root@localhost ~]#
```

（4） 配置 SMTP 认证

Saslauthd 服务作用，提供 SMTP 用户验证，检查用户名和密码是否正确，基于系统 shadow 文件来实现验证配置。

```
service saslauthd restart
```

19.2 Dovecot 服务配置

Dovecot 是一个开源的 IMAP 和 POP3 邮件服务器，支持 Linux/Unix 系统，作为 IMAP 和 POP3 服务器，Dovecot 为邮件用户代理(MUA)提供了一种访问服务器上存储的邮件的方法。但是 Dovecot 并不负责从其他邮件服务器接收邮件。(类似 MDA)

Dovecot 只是将已存储在邮件服务器上的邮件通过 MUA 显示出来，IMAP 和 POP3 是用于连接 MUA 与邮件存储服务器的两种常见的协议。

```
yum install   dovecot*   -y
```

去掉/etc/dovecot.conf 如下行前面的#号即可：

```
protocols = imap pop3 lmtp
```

（1） 配置 Dovecot 禁止 SSL，设置邮箱：

```
vim /etc/dovecot/conf.d/10-ssl.conf

ssl = no

vim /etc/dovecot/conf.d/10-auth.conf

disable_plaintext_auth = no

vim /etc/dovecot/conf.d/10-mail.conf

mail_location = mbox:~/mail:INBOX=/var/mail/%u
```

（2） Sendmail 配置完毕

Sendmail 邮箱用户的建立，可以用客户端收发邮件即正常

```
groupadd    mailgroup
useradd    -g mailgroup -s /sbin/nologin    jfedu
echo 123456|passwd    --stdin jfedu
mkdir -p /home/jfedu/mail/.imap/INBOX
chown    -R    jfedu.jfedu    /home/jfedu/
```

```
service    sendmail    restart
service    dovecot    restart
service    saslauthd    restart
```



19.3 Sendmail 别名配置

Sendmail 服务器中可以使用 aliases 机制实现邮件别名和邮件群发功能，也可以创建用户组，将用户加入某个组中，实现组邮件的群发。

/etc/下存在 aliases 和 aliases.db 的两个文件，aliases 文件是文本文件，其内容是可阅读和编辑的，aliases.db 是数据库文件，是由 aliases 文件生成而来的。/etc/aliases 文件中添加如下代码：

```
jfedu:    jf1,jf2
```

给 jfedu@jfteach.com 发送邮件，则群发到 jf1@jfteach.com，jf2@jfteach.com，设置完成之后，通过 newaliases 指令生成新的 aliases.db 文件。

19.4 测试邮件收发

MAIL 命令发送邮件和收取邮件如下图（前提是能 ping 通 jfteach.com 域名，可以添加 hosts 或者映射 DNS）：

```
echo "This first test Mail"|mail -s "Test Mail Postfix" jfedu1@jfteach.com
```

查看并收取邮件：

```
[root@localhost ~]# echo "This first test Mail"|mail -s "T
[root@localhost ~]#
[root@localhost ~]# tail -fn 10 /var/spool/mail/jfedu1
Date: Wed, 28 Jun 2017 09:14:24 +0800
To: jfedu1@ifteach.com
Subject: Test Mail Postfix
User-Agent: Heirloom mailx 12.4 7/29/08
MIME-Version: 1.0
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

This first test Mail
```



基于 Foxmail 邮件收发



19.5 配置 Openwebmail

如上配置完 Sendmail 邮件服务器，可以通过 Outlook、Foxmail 收发邮件，如果需要 WEB 页面收发邮件，可以基于 Openwebmail 来实现。例如访问：http://mail.jfteach.com/，以用户名和密码登录，进行邮件的发送与收取，则需配置 openwebmail 软件，配置如下：

（1） Openwebmail 服务安装

```
cd /etc/yum.repos.d
wget                                                    -q
http://openwebmail.org/openwebmail/download/redhat/r
```

```
pm/release/openwebmail.repo

yum install openwebmail -y
```

（2） Openwebmail 配置：

```
yum install httpd httpd-devel -y

/var/www/cgi-bin/openwebmail/openwebmail-tool.pl --
init
```

```
localhost ~]# /var/www/cgi-bin/openwebmail/openwebmail-tool.pl --init

ng db /var/www/cgi-bin/openwebmail/etc/maps/b2g ...done.
ng db /var/www/cgi-bin/openwebmail/etc/maps/g2b ...done.
ng db /var/www/cgi-bin/openwebmail/etc/maps/lunar ...done.

ng UTF-8 locales...
nv ar_AE.CP1256 -> ar_AE.UTF-8
nv ar_AE.ISO8859-6 -> ar_AE.UTF-8
nv bg_BG.CP1251 -> bg_BG.UTF-8
nv ca_ES.ISO8859-1 -> ca_ES.UTF-8
nv cs_CZ.ISO8859-2 -> cs_CZ.UTF-8
nv da_DK.ISO8859-1 -> da_DK.UTF-8
```

```
Welcome to the OpenWebMail!

This program is going to send a short message back to th
so we could have the idea that who is installing and how
using this software, the content to be sent is:

OS: Linux 2.6.32-642.el6.x86_64 x86_64
Perl: 5.010001
WebMail: OpenWebMail 2.53 20080123

Send the site report?(Y/n) y
sending report...

Thank you.
```

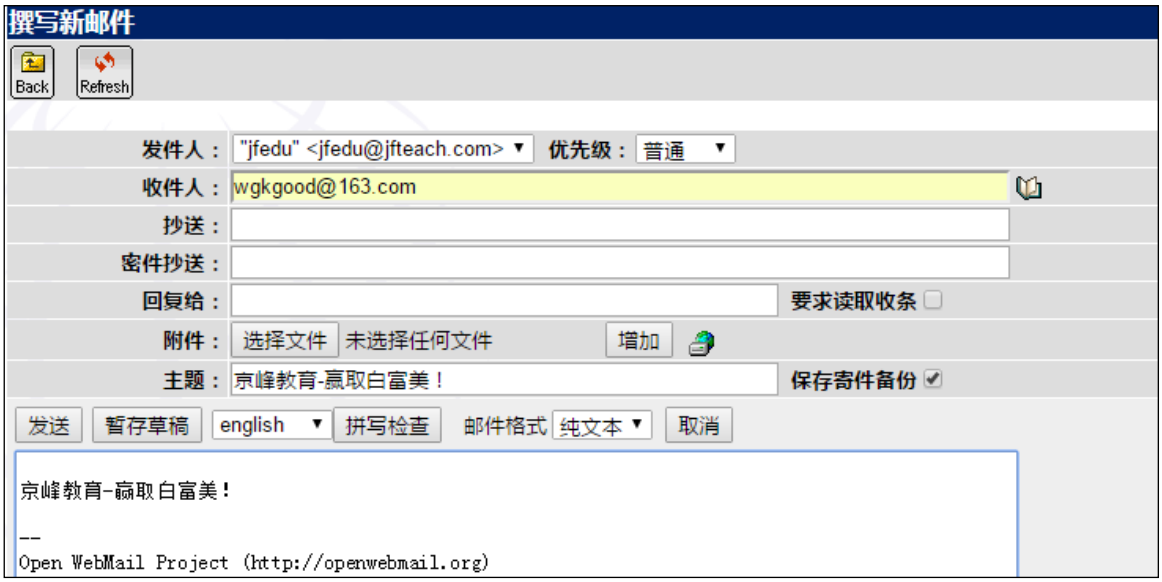
vi /var/www/cgi-bin/openwebmail/etc/openwebmail.conf

domainnames	jfteach.com
default_language	zh_CN.GB2312
default_iconset	Cool3D.Chinese.Simplified

/var/www/cgi-bin/openwebmail/openwebmail-tool.pl --init

通过浏览器访问 openwebmail，访问如图：



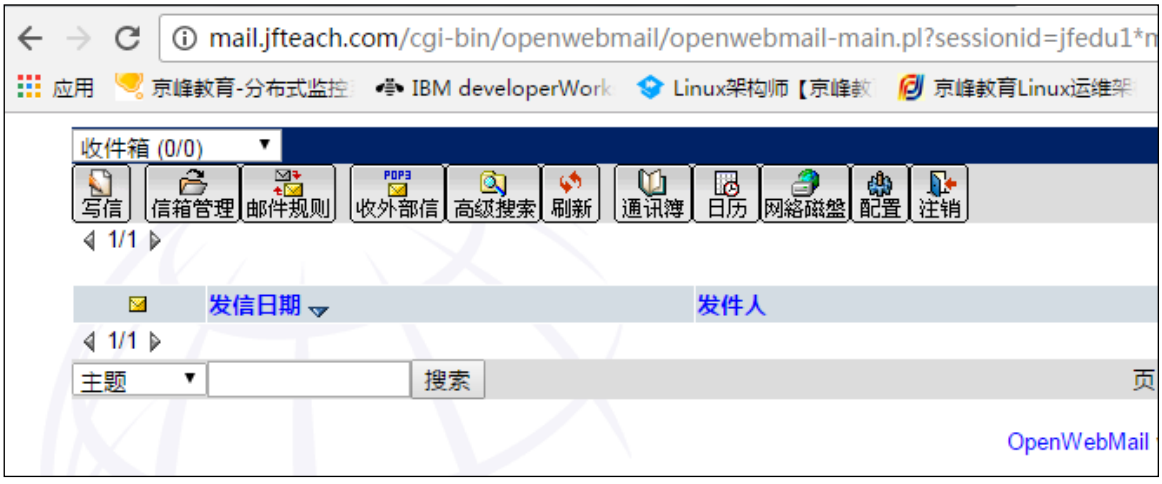




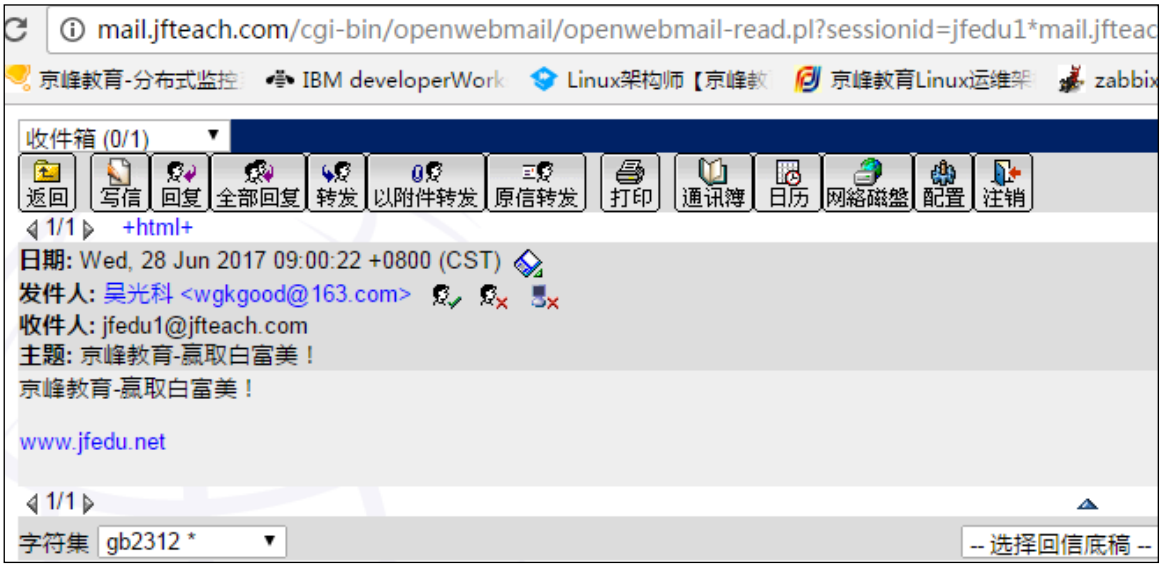
SendMail 外网域名配置，配置方法如下：


jfteach.com
解析设置
批量导入解析
网站监控
安全防护
全球负载均衡
解析量统计
CDN加速
解析设置
添加解析
批量导入解析
导出解析记录
新手引导设置
建议您在电脑上修改公共DNS，让解析设置实时生效。
下载DNS修改工具
什么是公共DNS
记录类型
主机记录
解析线路(运营商)
暂无任何记录


添加解析
批量导入解析
导出解析记录
新手引导设置
建议您在电脑上修改公共DNS，让解析设置实时生效。
下载DNS修改工具
什么是公共DNS，如何修改？
记录类型
主机记录
解析线路(运营商)
记录值
MX优先级
A @ 默认 113.209.20.234 --
MX mail 默认 jfteach.com. 9
A mail 默认 113.209.20.234 --
暂停
启用
删除


mail.jfteach.com/cgi-bin/openwebmail/openwebmail-main.pl?sessionid=jfedu1*n
应用
京峰教育-分布式监控
IBM developerWork
Linux架构师【京峰教
京峰教育Linux运维架
收件箱 (0/0)
写信
信箱管理
邮件规则
收外部信
高级搜索
刷新
通讯簿
日历
网络磁盘
配置
注销
1/1
发信日期
发件人
1/1
主题
搜索
页
OpenWebMail





同时配置 apache rewrite 规则，开启访问 mail.jfteach.com 直接访问 Openwebmail 页面，规则添加如下：

```
RewriteEngine On
ProxyPreserveHost On
RewriteRule ^/$ http://mail.jfteach.com/cgi-
```

bin/openwebmail/openwebmail.pl [P,L,NC]

```
#    CustomLog logs/dummy-host.example.com-access_log comm
#</VirtualHost>
RewriteEngine On
ProxyPreserveHost On
RewriteRule ^/$ http://mail.jfteach.com/cgi-bin/openwebma
~
"/etc/httpd/conf/httpd.conf" 1011L, 34519C written
[root@localhost ~]# /etc/init.d/httpd restart
Stopping httpd:
Starting httpd: httpd: Could not reliably determine the se
1 for ServerName

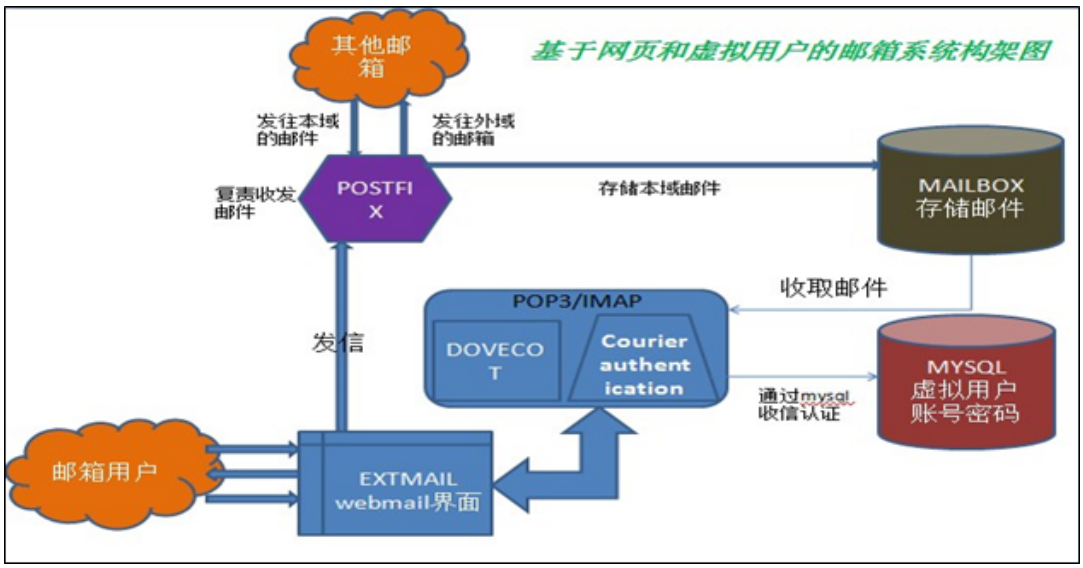
[root@localhost ~]#
[root@localhost ~]#
```



19.6 Postfix 入门简介

Postfix 是 Wietse Venema 在 IBM 的 GPL 协议之下开发的 MTA（邮件传输代理）软件。Postfix 是 Wietse Venema 想要为使用最广泛的 sendmail 提供替代品的一个尝试。

在 Internet 世界中，大部分的电子邮件都是通过 sendmail 来投递的，大约有 100 万用户使用 sendmail，每天投递上亿封邮件。这是一个让人吃惊的数字，Postfix 试图更快、更容易管理、更安全，同时还与 sendmail 保持足够的兼容性。



19.7 Postfix 服务安装

```
yum -y install postfix*
```

```
[root@localhost ~]# yum -y install postfix*
Loaded plugins: fastestmirror
Setting up Install Process
Loading mirror speeds from cached hostfile
Resolving Dependencies
--> Running transaction check
---> Package postfix.x86_64 2:2.6.6-6.el6_7.1 will be upd
---> Package postfix.x86_64 2:2.6.6-8.el6 will be an upda
---> Package postfix-perl-scripts.x86_64 2:2.6.6-8.el6 wi
--> Processing Dependency: perl(Date::Calc) for package:
--> Running transaction check
---> Package perl-Date-Calc.noarch 0:6.3-3.el6 will be in
--> Processing Dependency: perl(Bit::Vector) >= 7.1 for p
--> Processing Dependency: perl(Carp::Clan) for package:
--> Processing Dependency: perl(Bit::Vector) for package:
--> Running transaction check
```

19.8 Postfix 服务器配置

编辑 postfix 主配置文件，vi /etc/postfix/main.cf

以下配置案例域名为：jfteach.com，邮件服务器主机名是 mail.jfteach.com

（1）修改 myhostname

myhostname = mail.jfteach.com

myhostname 参数即是邮件服务器的主机名称。

（2）修改 mydomain

mydomain = jfteach.com

mydomain 参数设定网域名称(Domain Name)，主机名称通常都是在建立在网域名称之内，像是 www.google.com 则是网页服务，mail.google.com 则会是邮件主机服务，通常网域名称都会是主机

名称(Hostname)去掉第一个掉包含前面的文字部份，比如说 www.google.com 的 Domain 则是 google.com

（3）修改 myorigin

myorigin = $mydomain

myorigin 是我们的邮件地址在”@”后面的文字内容，例如 wugk@163.com，163.com 则为 Origin Name

（4）修改 smtp 监听端口

inet_interfaces = all

inet_interfaces 参数指定 postfix 系统监听的网络接口。Postfix 预设只会监听来自于本机端的所传出的封包，必须使用上列设定，才可以传收所有来自于网路端的所有封包。

（5）修改 inet_protocols

修改 Postfix 的通讯协定，目前网路的协定主流有 IPv4 与 IPv6，在大部份的情况下，我们都是利用 IPv4 在通讯的，如果你的 Mail Server 没有需要使用到 IPv6，可以做以下的设定修改

inet_protocols = ipv4

（6）修改 mydestination

mydestination = $myhostname, localhost.$mydomain, localhost, $mydomain

mydestination 参数设定能够接收信件的主机名称，Postfix 预设只能收到设定的 Hostname 与 Domain Name 以及本机端的信件，此步骤是再增加能收信件的网络名称。

（7）设定信任用户端

mynetworks = 127.0.0.0/8, 192.168.1.0/24, hash:/etc/postfix/access

mynetworks 参数设定信任的用户端，当要寄信时，会参考此值，若非信任的用户，则不会帮你的信件转到其它的 MTA 主机上。

（8）设定 relay_domain 转发邮件域名

规范可以 relay 的 MTA 主机位址(收发 Mail 的程序一般统称为邮件用户代理 MUA(Mail User Agent))，通常这个都直接设为 mydestination。

relay_domains = $mydestination

（9）设定邮件别名的路径

检查”alias_maps”设定是否为以下字串设定

alias_maps = hash:/etc/aliases

（10） 设定指定邮件别名表资料库路径

检查” alias_ database”设定是否为以下字串设定

alias_database = hash:/etc/aliases

（11） 执行以下命令

用于设定邮件主机使用权限与过滤机制及邮件别名

postmap hash:/etc/postfix/access

postalias hash:/etc/aliases

（12） 重启 postfix 服务

如上 Postfix 配置完毕，同样需要配置 Dovecot 及启动 saslauthd 服务，方可进行邮件的收发！

```
/etc/init.d/postfix restart
/etc/init.d/saslauthd restart
/etc/init.d/dovecot restart
```

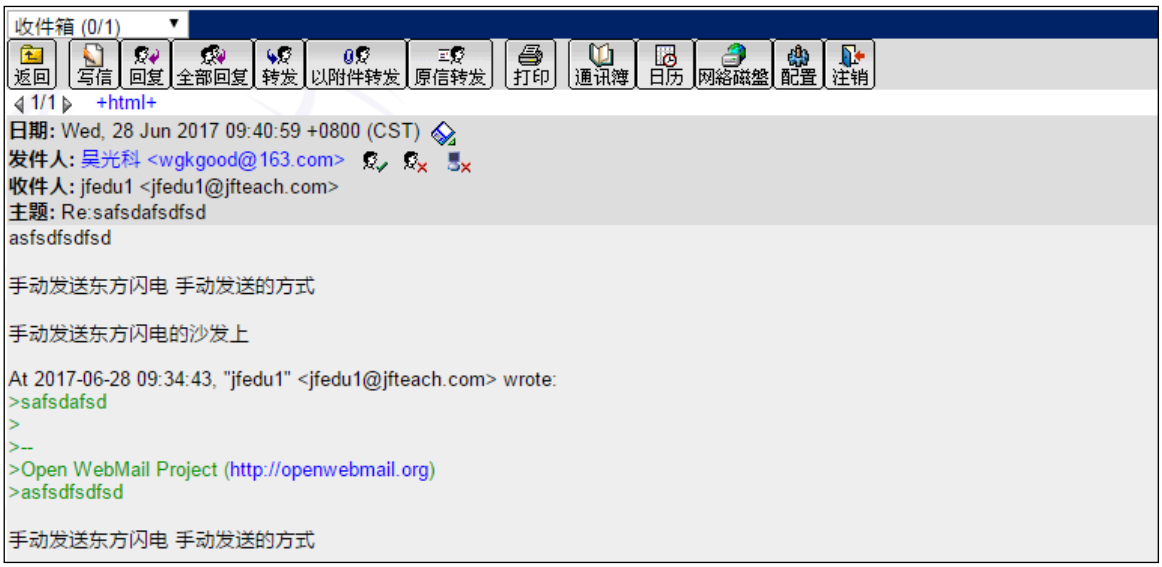
如下为完整 Postfix 邮件服务器 main.cf 配置代码：

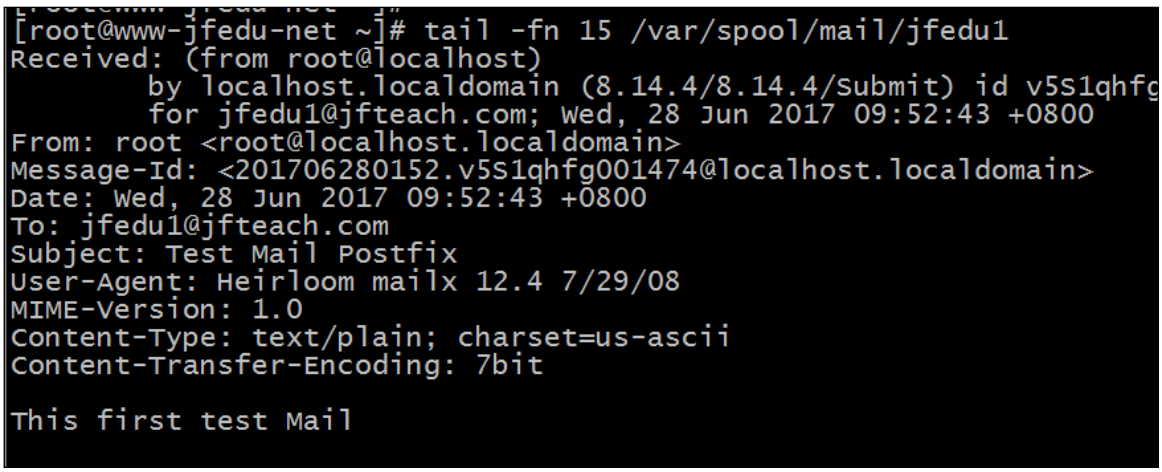
```
queue_directory = /var/spool/postfix
command_directory = /usr/sbin
daemon_directory = /usr/libexec/postfix
data_directory = /var/lib/postfix
mail_owner = postfix
myhostname = mail.jfteach.com
mydomain = jfteach.com
myorigin = $mydomain
```

```
inet_interfaces = all
inet_protocols = ipv4
mydestination = $myhostname, localhost.$mydomain,
localhost, $mydomain
unknown_local_recipient_reject_code = 550
mynetworks = 0.0.0.0/0
relay_domains = $mydestination
alias_maps = hash:/etc/aliases
alias_database = hash:/etc/aliases
debug_peer_level = 2
debugger_command =
    PATH=/bin:/usr/bin:/usr/local/bin:/usr/X11R6/bin
    ddd $daemon_directory/$process_name $process_id &
sleep 5
sendmail_path = /usr/sbin/sendmail.postfix
newaliases_path = /usr/bin/newaliases.postfix
mailq_path = /usr/bin/mailq.postfix
setgid_group = postdrop
html_directory = no
```

```
manpage_directory = /usr/share/man
sample_directory = /usr/share/doc/postfix-2.6.6/samples
readme_directory = /usr/share/doc/postfix-2.6.6/README_FILES
```

测试邮件收发，如下图所示：





19.9 Foxmail 本地邮箱配置

（1） Linux 系统创建用户 jfedu1、jfedu2，同时设置密码，命令如下：

```
useradd   jfedu1 ;echo 123456|passwd   --stdin jfedu1
useradd   jfedu2 ;echo 123456|passwd   --stdin jfedu2
mkdir -p   /home/jfedu1/mail/.imap/INBOX/
mkdir -p   /home/jfedu2/mail/.imap/INBOX/
chown   -R   jfedu1.jfedu1   /home/jfedu1/
chown   -R   jfedu2.jfedu2   /home/jfedu2/
```

（2） Dovecot Smtp 验证设置:

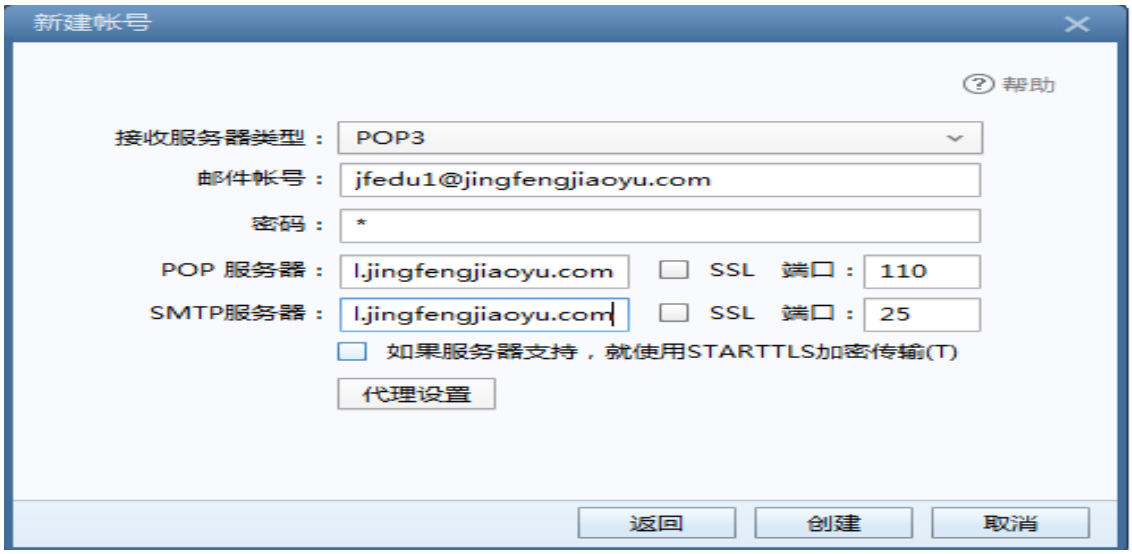
```
vim   /etc/dovecot/conf.d/10-auth.conf
disable_plaintext_auth = no
vim   /etc/dovecot/conf.d/10-mail.conf
mail_location = mbox:~/mail:INBOX=/var/mail/%u
vim   /etc/dovecot/conf.d/10-ssl.conf
ssl = no
```

（3） 重启 postfix 及 dovecot 服务：

```
/etc/init.d/postfix     restart
```

```
/etc/init.d/dovecot    restart
```

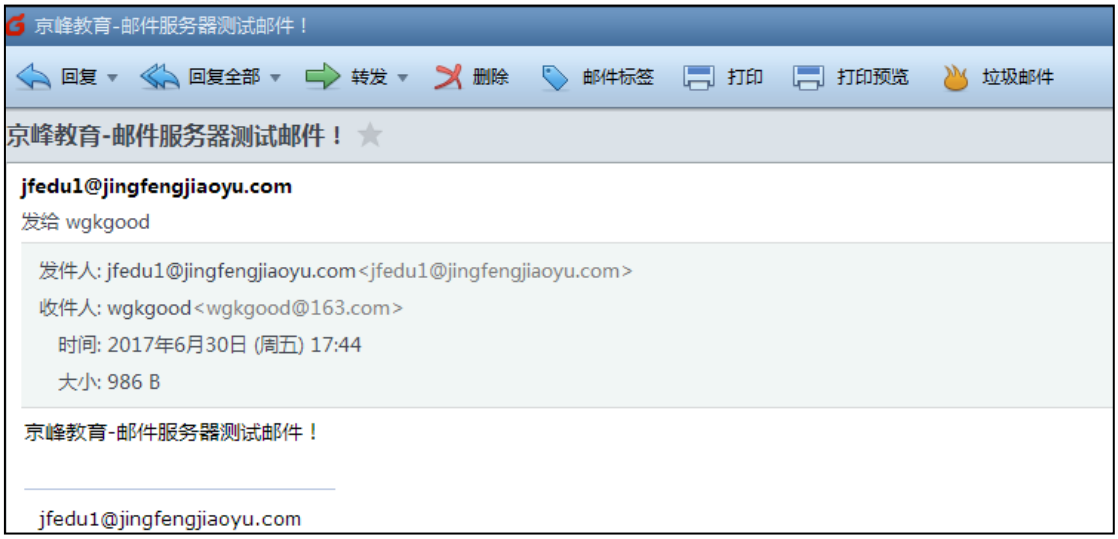
（4） Foxmail 客户端设置



POP 服务器及 SMTP 服务器：mail.jingfengjiaoyu.com



发送邮件给 wgkgood@163.com，如图所示：



登录 wgkgood@163.com 邮箱收取邮件，如图所示：



基于 wgkgood@163.com 发送邮件给 jfedu1@jingfengjiaoyu.com,收件如图所示：



19.10 Postfixadmin 配置

Postfix 一般管理均是基于命令行管理，对应运维人员来说比较麻烦，有没有 postfix 图形界面管理工具呢，而 Postfixadmin 就是为 Postfix 邮件服务器提供的图形界面管理工具，用它可以很方便的管理 postfix 服务器。

如下为 Postfixadmin 安装配置方法，由于 Postfixadmin 基于 PHP 语言编写，所以需要安装 LAMP（LNMP、LEMP）环境，同时最新版本 Postfixadmin 需要 PHP5.4 以上支持。

（1） 安装 LAMP 及 Postfixadmin 软件；

```
rpm -Uvh http://repo.webtatic.com/yum/el6/latest.rpm
yum remove php*
yum install php56w.x86_64 php56w-cli.x86_64 php56w-
```

```
common.x86_64 php56w-gd.x86_64 php56w-ldap.x86_64
php56w-mbstring.x86_64 php56w-mcrypt.x86_64 php56w-
mysql.x86_64 php56w-pdo.x86_64 -y
yum install httpd httpd-devel httpd-tools mysql
mysql-devel mysql-server -y
wget
https://jaist.dl.sourceforge.net/project/postfixadmin/postfi
xadmin/postfixadmin-3.1/postfixadmin-3.1.tar.gz
tar xzf postfixadmin-3.1.tar.gz -C /var/www/html/
cd /var/www/html/
mv postfixadmin-3.1/ postfixadmin
mkdir -p /var/www/html/postfixadmin/templates_c
chmod -R 777
/var/www/html/postfixadmin/templates_c
chown -R root.root /var/www/html/
```

（2）修改 Postfixadmin 配置

将/var/www/html/postfixadmin/config.inc.php 如下选项改成如下代码：

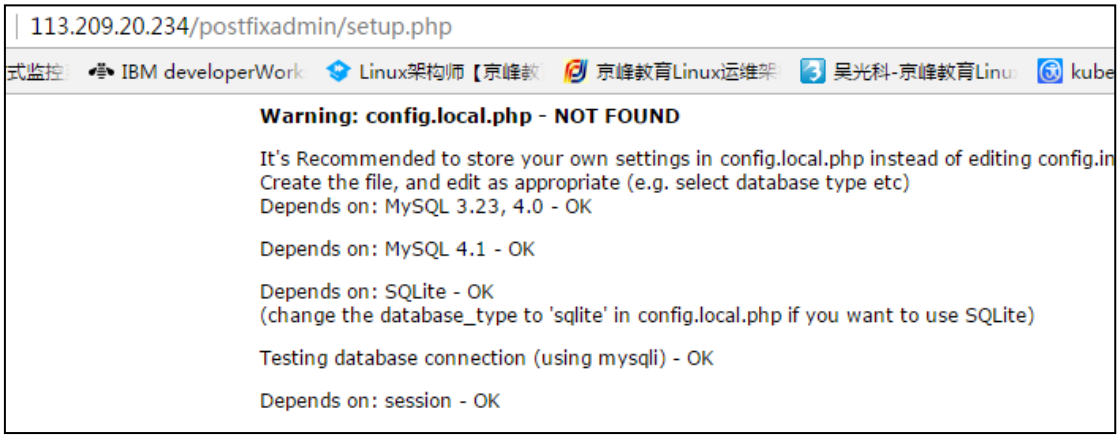
```
$CONF['configured'] = true;
```

（3） 创建 postfix 数据库

```
create database postfix charset=utf8;
grant all on *.* to postfix@'localhost' identified by "postfixadmin";
flush privileges;
```

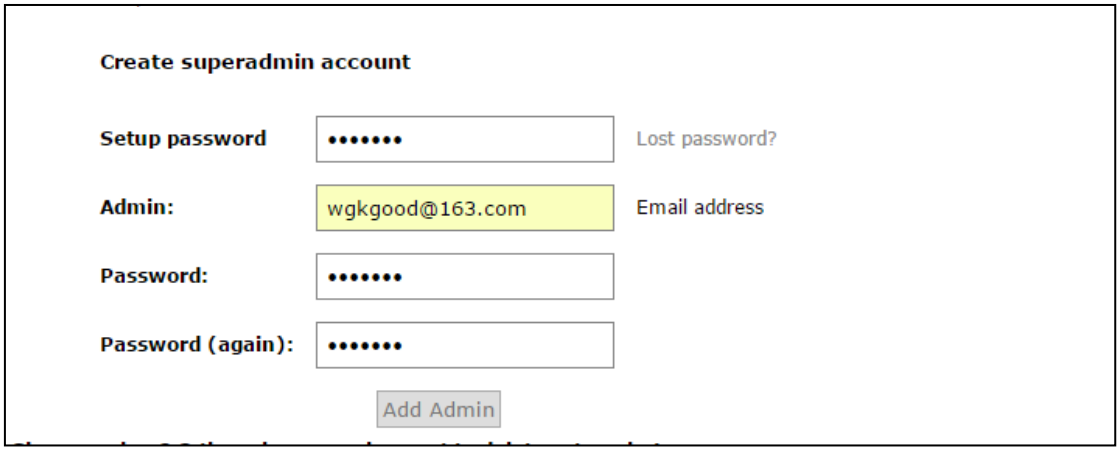
（4） 创建 postfixadmin 管理员：

访问 postfixadmin 网页：http://113.209.20.234/postfixadmin/setup.php



修改 config.inc.php 配置文件，添加密码：

```
$CONF['setup_password'] = 'c11c1f22771ecc03d4507f366aeead44:3300e64c1a1e11a446ca2e964dadd4468ec163d3';
```

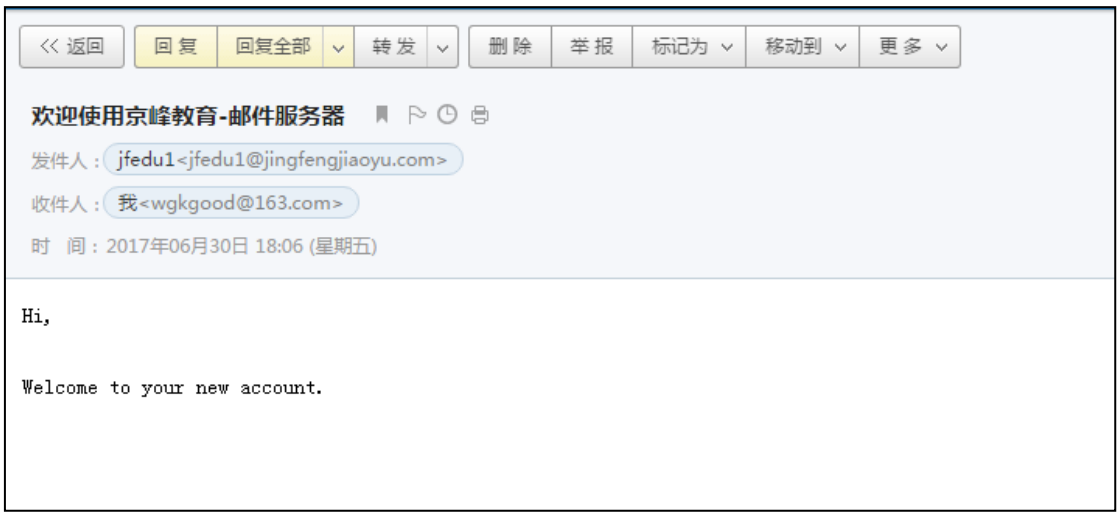

Create superadmin account
Setup password
Lost password?
Admin:
wgkgood@163.com
Email address
Password:
Password (again):
Add Admin


postfix.admin
Mail admins login here to administer your domain.
Login (email):
wgkgood@163.com
Password:
Language:
中文 - Chinese simplified (gb2312)
Login


Everything seems fine... attempting to create/update database structure
Database is up to date
新增管理员成功! (jfedu1@jingfengjiaoyu.com)
You are done with your basic setup.
You can now login to PostfixAdmin using the account you just created.
Create superadmin account
Setup password
Lost password?
管理员:
邮件地址
密码:


postfix.admin
管理员清单
域名清单
虚拟用户清单
Fetch Email
发送邮件
修改密码
查看日志
退出
概览 显示你的邮件别名和邮箱. 你可以在这儿进行编辑/删除操作.
新建别名 在您的域中新建一个别名.
新建邮箱 在您的域中新建一个邮箱.
发送邮件 发一封邮件到你新建的一个邮箱中.
修改密码 修改你的管理员密码.
查看日志 查看日志文件.
退出 退出系统


发送邮件.
发件人: jfedu1@jingfengjiaoyu.com
收件人: wgkgood@163.com
主题: 欢迎使用京峰教育-邮件服务器
内容: Hi,
Welcome to your new account.
发送


返回
回复
回复全部
转发
删除
举报
标记为
移动到
更多
欢迎使用京峰教育-邮件服务器
发件人：jfedu1<jfedu1@jingfengjiaoyu.com>
收件人：我<wgkgood@163.com>
时 间：2017年06月30日 18:06 (星期五)
Hi,
Welcome to your new account.

19.11 RoundCube GUI WEB 配置

（1） Roundcube Webmail 安装配置

```
wget
https://jaist.dl.sourceforge.net/project/roundcubemail/rou
ndcubemail/1.1.4/roundcubemail-1.1.4-complete.tar.gz
tar -xzf roundcubemail-1.1.4-complete.tar.gz
mv roundcubemail-1.1.4 /var/www/html/webmail/
yum install php56w-dom epel-release libmcrypt* php56w-
intl php56w-mbstring php56w-ldap php56w-mcrypt -y
sed -i 's#;date.timezone =#date.timezone = PRC#g'
/etc/php.ini
/etc/init.d/httpd restart
```

（2） 配置 WEB MAIL：

http://113.209.20.234/webmail/installer/index.php

老男孩 1 对 1 职业规划咨询+100G 运维入门到精通全套资料 加老师 V：Linux9090

Checking PHP extensions

The following modules/extensions are *required* to run Roundcube:
PCRE: **OK**
DOM: **OK**
Session: **OK**
XML: **OK**
JSON: **OK**
PDO: **OK**
The next couple of extensions are *optional* and recommended to get the best performance:
FileInfo: **OK**
Libiconv: **OK**
Multibyte: **OK**
OpenSSL: **OK**
Mcrypt: **OK**
Intl: **NOT AVAILABLE** (Could be loaded. Please add in php.ini; See http://www.php.net/manual/en/book.int
Exif: **OK**
LDAP: **NOT AVAILABLE** (See http://www.php.net/manual/en/book.ldap.php)

Checking available databases

Version: **OK** (PHP 5.6.30 detected)

Checking PHP extensions

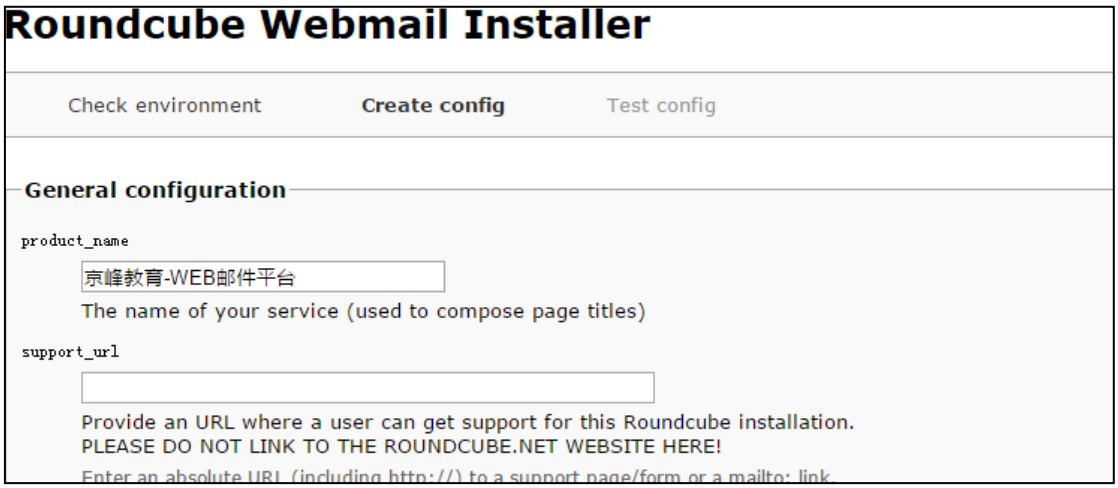
The following modules/extensions are *required* to run Roundcube:
PCRE: **OK**
DOM: **OK**
Session: **OK**
XML: **OK**
JSON: **OK**
PDO: **OK**
The next couple of extensions are *optional* and recommended to get the best performance:
FileInfo: **OK**
Libiconv: **OK**
Multibyte: **OK**
OpenSSL: **OK**
Mcrypt: **OK**
Intl: **OK**
Exif: **OK**
LDAP: **OK**

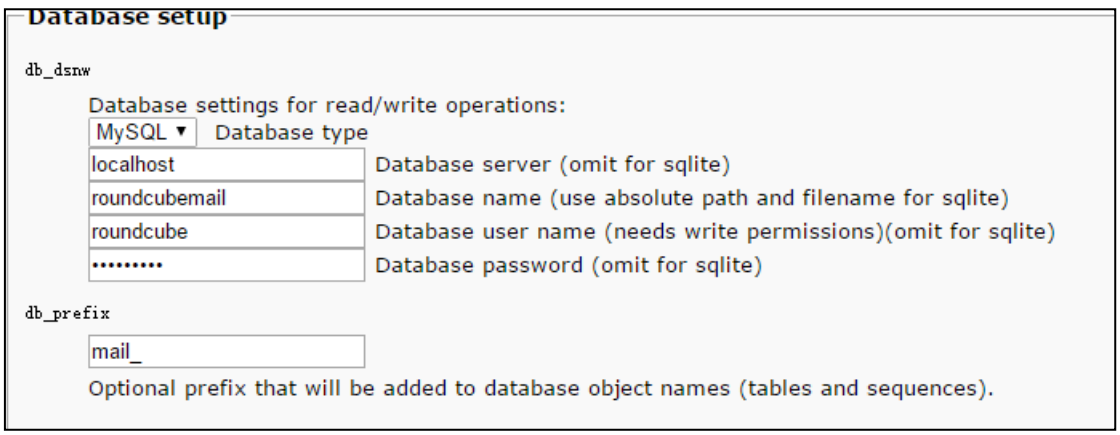
This also checks if the include path is set correctly.
PEAR: **OK**
Auth_SASL: **OK**
Net_SMTP: **OK**
Net_IDNA2: **OK**
Mail_mime: **OK**
Net_LDAP3: **OK**

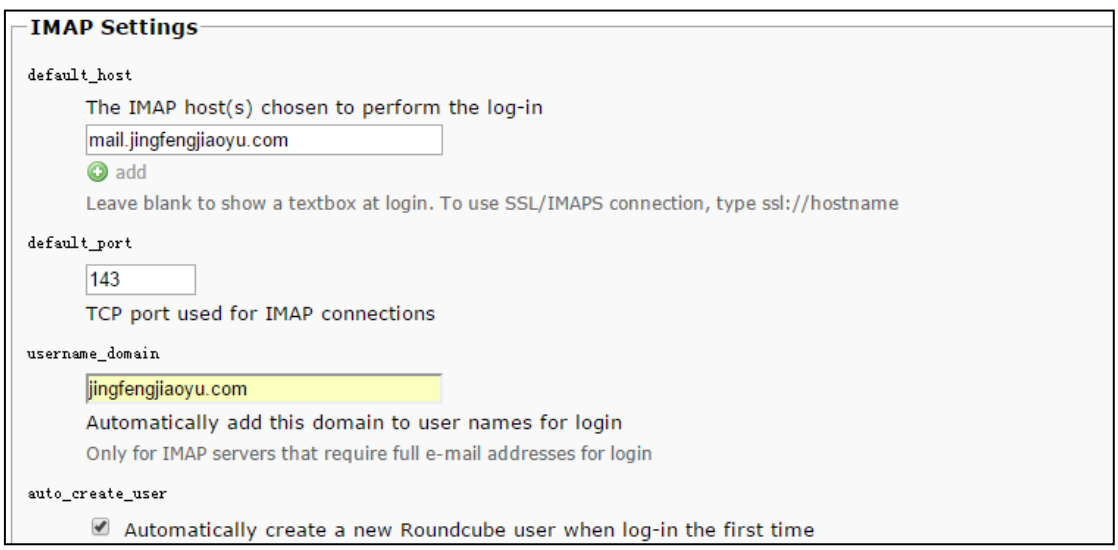
Checking php.ini/.htaccess settings

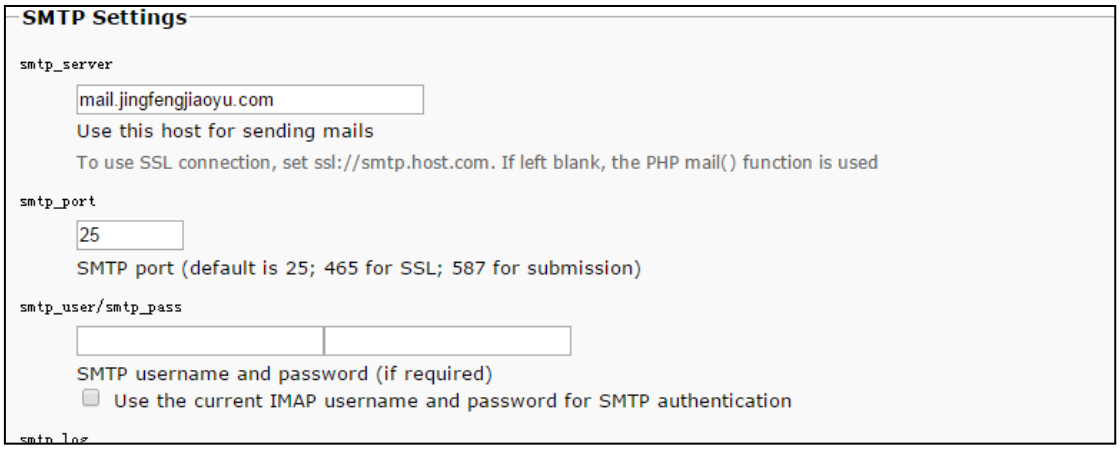
The following settings are *required* to run Roundcube:
file_uploads: **OK**
session.auto_start: **OK**
mbstring.func_overload: **OK**
suhosin.session.encrypt: **OK**
magic_quotes_runtime: **OK**
magic_quotes_sybase: **OK**
The following settings are *optional* and recommended:
allow_url_fopen: **OK**
date.timezone: **OK**
register_globals: **OK**

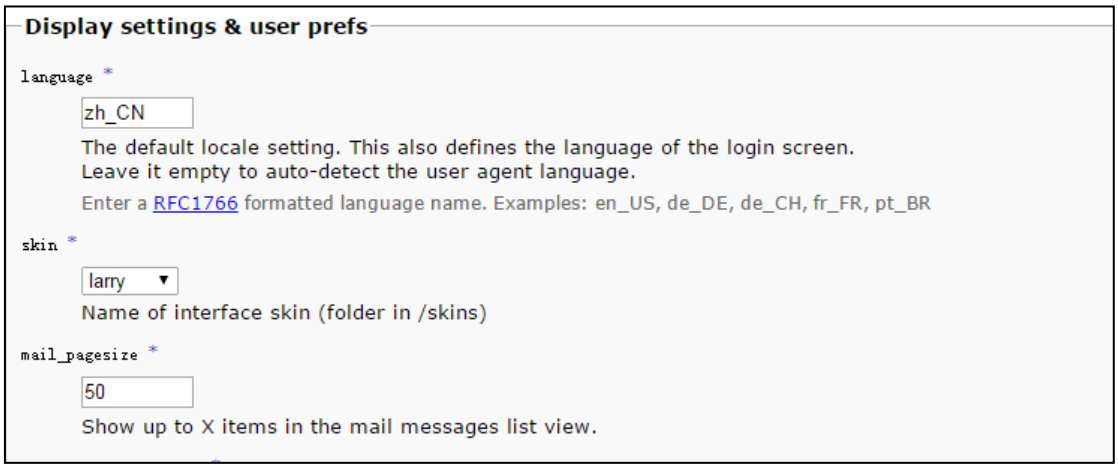
老男孩 1 对 1 职业规划咨询+100G 运维入门到精通全套资料 加老师 V：Linux9090





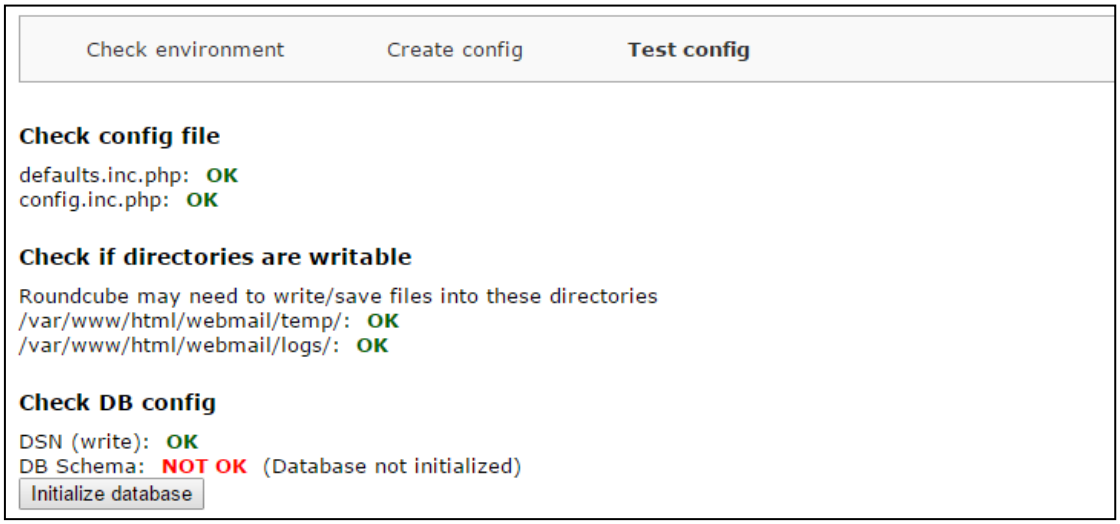


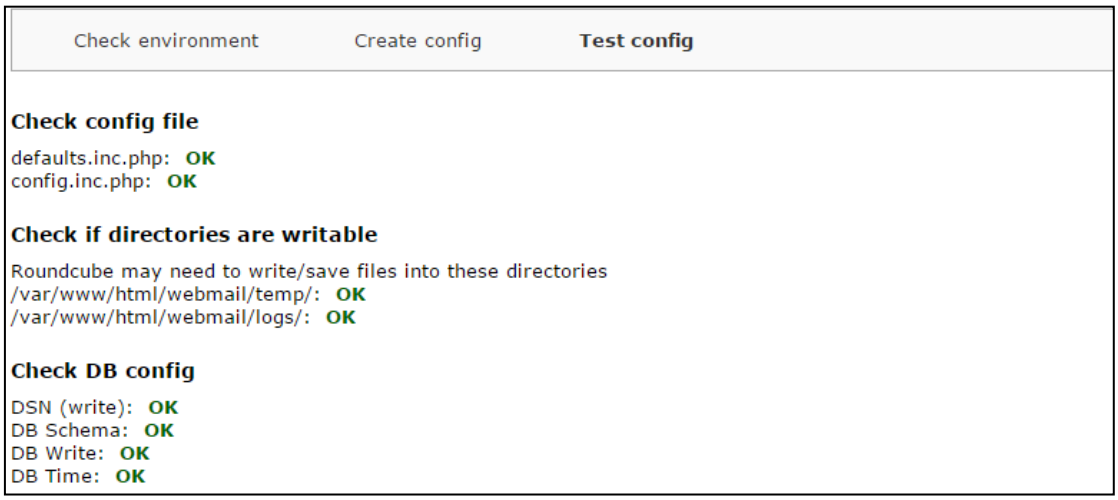


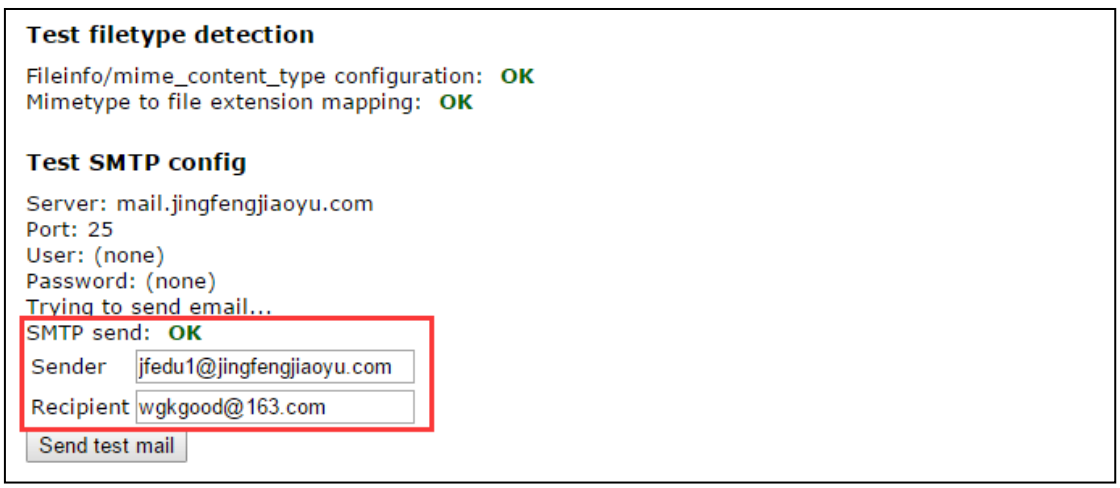


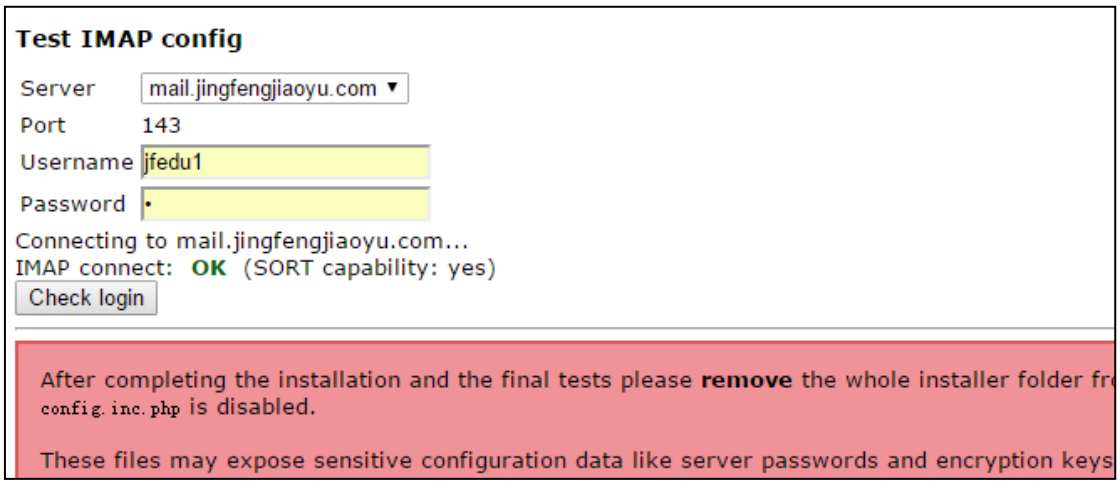
（3） 创建 WEB MAIL 数据库信息，命令如下：

```
create database roundcubemail charset=utf8;
grant all on *.* to roundcube@'localhost' identified by "roundcube";
flush privileges;
chmod 777 -R temp/ logs/
```


Check environment
Create config
Test config
Check config file
defaults.inc.php: OK
config.inc.php: OK
Check if directories are writable
Roundcube may need to write/save files into these directories
/var/www/html/webmail/temp/: OK
/var/www/html/webmail/logs/: OK
Check DB config
DSN (write): OK
DB Schema: NOT OK (Database not initialized)
Initialize database


Check environment
Create config
Test config
Check config file
defaults.inc.php: OK
config.inc.php: OK
Check if directories are writable
Roundcube may need to write/save files into these directories
/var/www/html/webmail/temp/: OK
/var/www/html/webmail/logs/: OK
Check DB config
DSN (write): OK
DB Schema: OK
DB Write: OK
DB Time: OK


Test filetype detection
Fileinfo/mime_content_type configuration: OK
Mimetype to file extension mapping: OK
Test SMTP config
Server: mail.jingfengjiaoyu.com
Port: 25
User: (none)
Password: (none)
Trying to send email...
SMTP send: OK
Sender jfedu1@jingfengjiaoyu.com
Recipient wgkgood@163.com
Send test mail



（4） 登录 WEB 控制台，登录失败：

http://113.209.20.234/webmail/ 输入 jfedu1 用户名和密码登录。



（5） 查看 Dovecot 后台日志，如下错误：

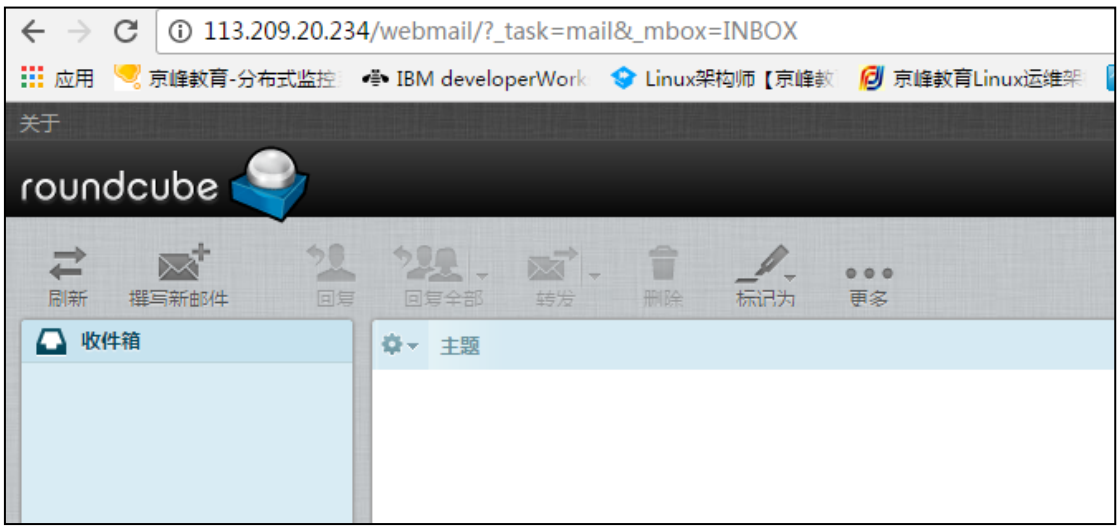
```
Jun 30 18:34:58 imap-login: Info: Disconnected (auth failed, 1 attempts): user=<jfedu1@jingfengjiaoyu.com>, method=PLAIN, rip=113.209.20.234, lip=192.168.0.3
```

Jun 30 18:35:17 imap-login: Info: Disconnected (auth failed, 1 attempts): user=<jfedu2@jingfengjiaoyu.com>, method=PLAIN, rip=113.209.20.234, lip=192.168.0.3

如上错误解决方法，在/etc/dovecot/dovecot.conf 文件中，添加如下代码即可：（表示去掉用户后面其他信息，保留用户名称，例如 jfedu1）

auth_username_format = %n





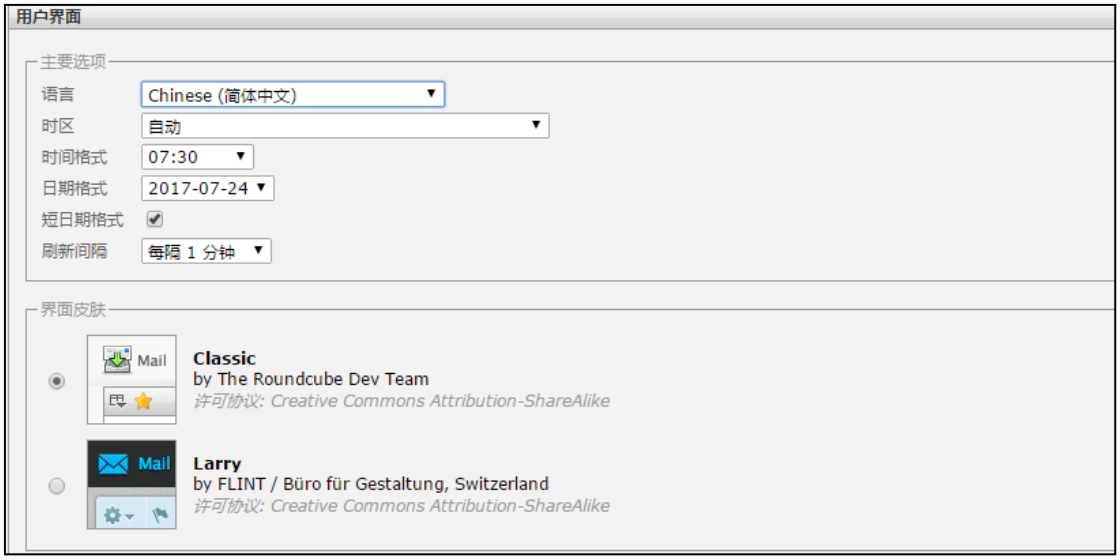
（6） 创建测试邮件，如图所示：

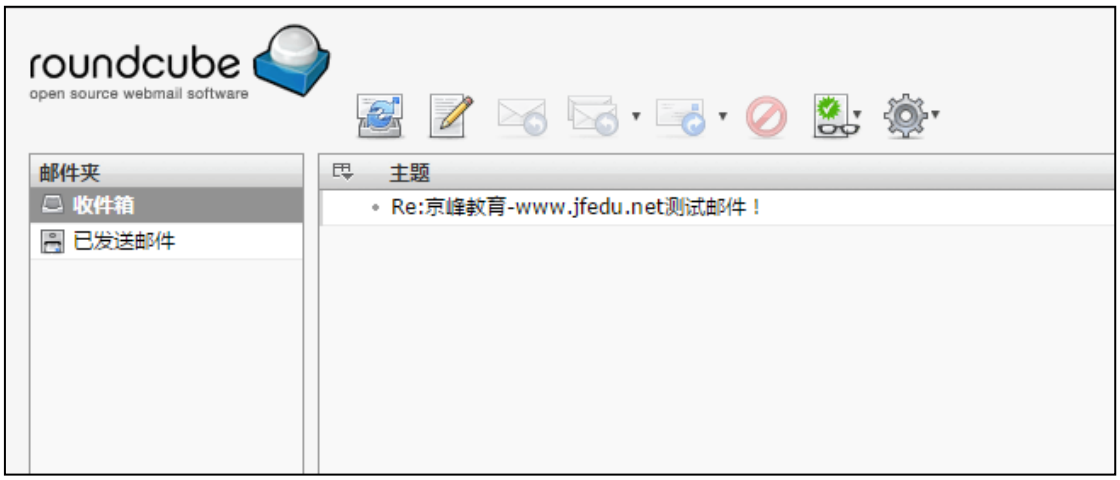


（7） 登录 163 邮箱收取邮件，如图所示：

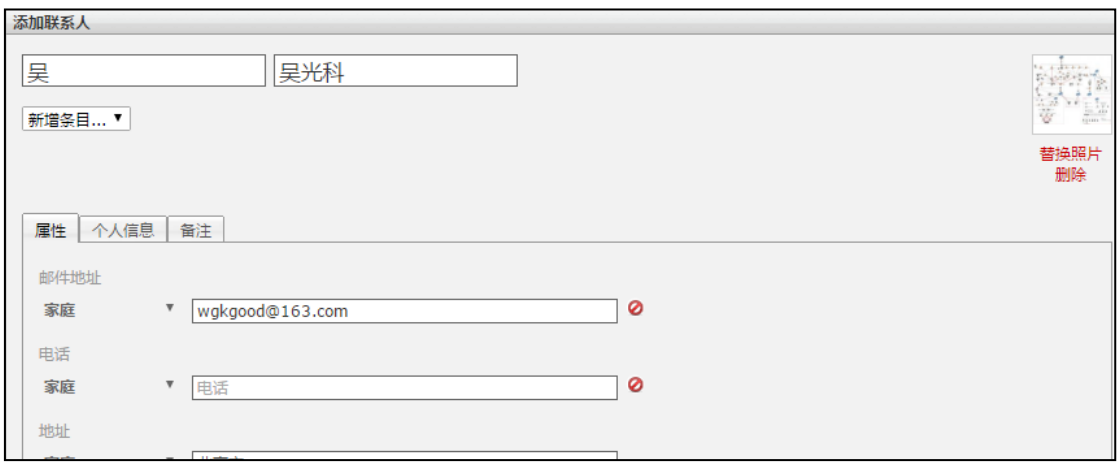


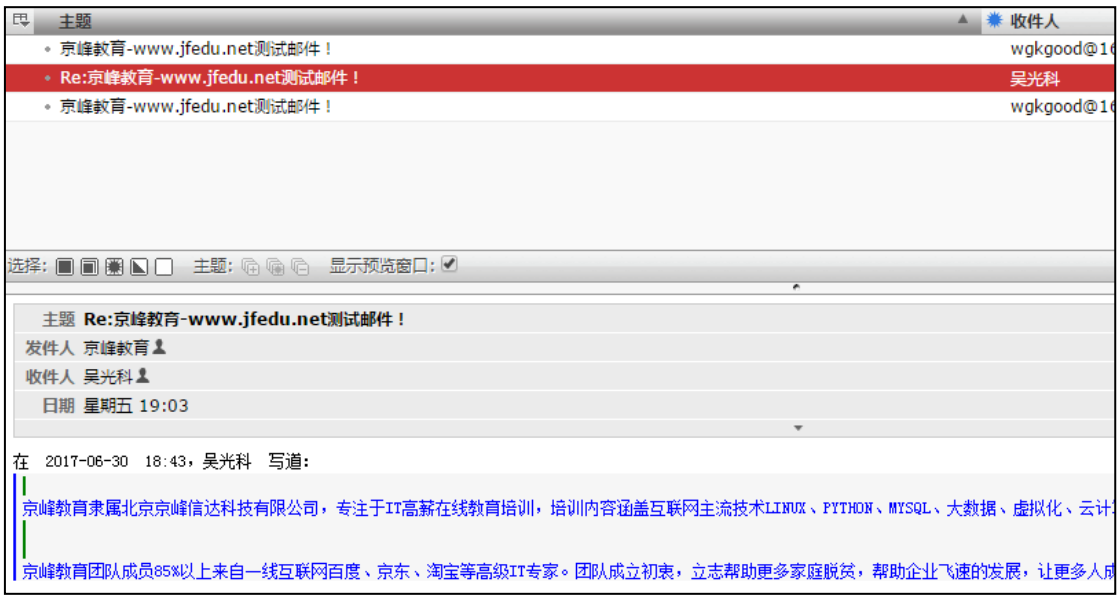
（8） 设置 Roundcube Webmail 界面风格：





（9） WEBMAIL 添加通讯录：





（10） 创建垃圾邮件及已删除邮件夹：

通过 RoundCube 默认删除邮件，会报错，报错信息如下：

UID COPY: Mailbox doesn't exist: Trash

基于 Dovecot 自动创建垃圾邮件及已删除邮件夹，代码如下：

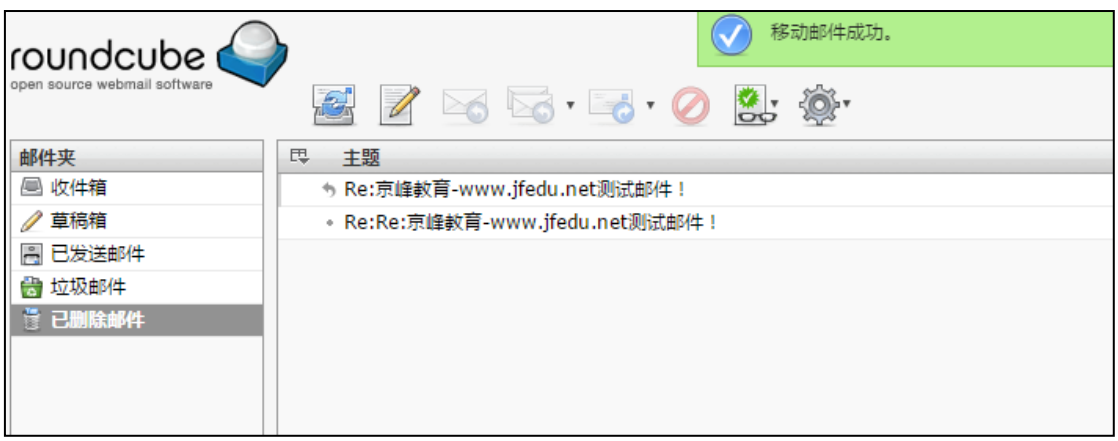
vim /etc/dovecot/conf.d/20-imap.conf

```
mail_plugins = $mail_plugins autocreate
plugin {
  autocreate = Trash
  autocreate2 = Junk
  autocreate3 = Drafts
  autocreate4 = Sent
  autosubscribe = Trash
```

```
    autosubscribe2 = Junk
    autosubscribe3 = Drafts
    autosubscribe4 = Sent
}
```

重启 dovecot 服务即可。





（11）删除 Roundcube 安装信息及设置禁止安装：

```
rm -rf /var/www/html/webmail/installer/
```

vim /var/www/html/webmail/config/config.inc.php 添加如下代码：

```
$config['enable_installer'] = false;
```

vim /etc/httpd/conf/httpd.conf 加入如下代码：

```
<Directory "/var/www/html/webmail/installer/">
    Options Indexes FollowSymLinks
    AllowOverride None
    Order allow,deny
    Deny  from  all
</Directory>
```

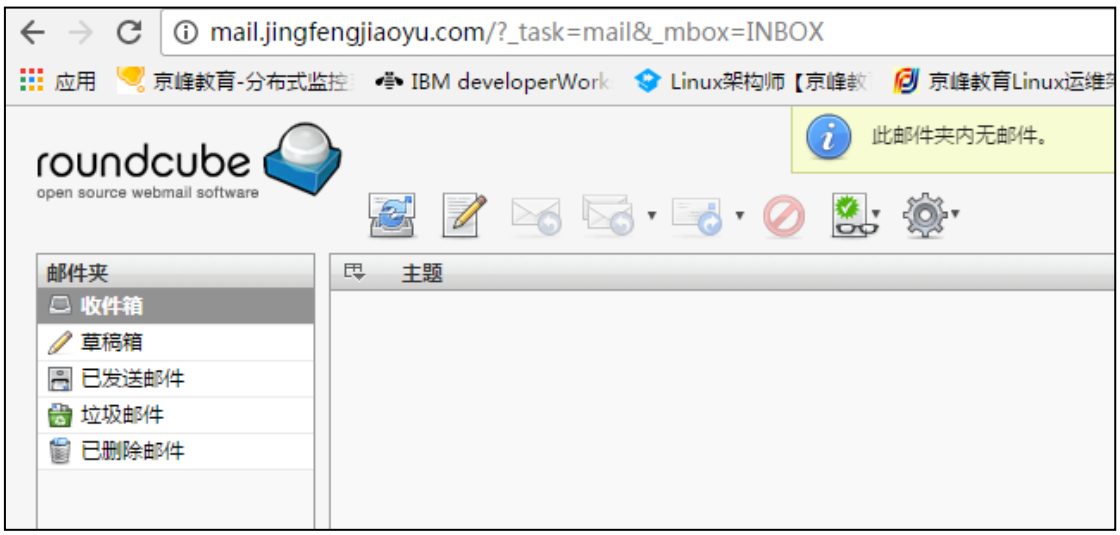


（12）实现域名访问邮件服务器

Apache 配置文件 httpd.conf 末尾加入如下代码 ,重启 apache 访问 webmail 如图所示：

```
<IfModule alias_module>
    Alias   /   /var/www/html/webmail/
</IfModule>
```





19.12 Postfix 虚拟用户配置

通过如上章节的配置 ,使用系统用户可以登录到 postfix 邮件服务器，并且进行邮件的发送和接收，为了管理的方便和系统安全，

一般通过 postfix 虚拟用户来管理邮件用户的。

Postfix 虚拟用户的原理：基于系统中创建的一个映射用户，该用户不能登录到系统，然后将其他虚拟用户全部映射到该系统用户所属目录。

对于操作系统来说所有的操作均是通过该系统用户进行的，但是对于 postfix 的邮件用户来说又是各自独立的。Postfix 虚拟用户与 Vsftpd 虚拟用户相似，所有的虚拟用户或者系统用户可以通过 Postfixadmin 来进行管理。

（1） 配置 Postfix SMTP 认证基于 MYSQL 数据库，读取 MYSQL 虚拟用户配置：

修改 Postfix 真实用户的 uid 和 gid，命令如下：

```
usermod   -u 1000 postfix
groupmod -g 1000 postfix
chown   -R   postfix.postfix /var/spool/mail/
```

（2） 创建 postfix 主配置文件 mail.cf 内容为如下：

```
queue_directory = /var/spool/postfix
command_directory = /usr/sbin
daemon_directory = /usr/libexec/postfix
data_directory = /var/lib/postfix
```

```
mail_owner = postfix
myhostname = mail.jingfengjiaoyu.com
mydomain = jingfengjiaoyu.com
myorigin = $mydomain
inet_interfaces = all
inet_protocols = ipv4
#mydestination = localhost
#mydestination =
$myhostname,localhost.$mydomain,localhost,$mydomain
unknown_local_recipient_reject_code = 550
#mynetworks = 192.168.0.0/24,127.0.0.0/8
mynetworks = 0.0.0.0/0
relay_domains = $mydestination
alias_maps = hash:/etc/aliases
alias_database = hash:/etc/aliases
debug_peer_level = 2
debugger_command =
    PATH=/bin:/usr/bin:/usr/local/bin:/usr/X11R6/bin
    ddd $daemon_directory/$process_name $process_id &
```

```
sleep 5
sendmail_path = /usr/sbin/sendmail.postfix
newaliases_path = /usr/bin/newaliases.postfix
mailq_path = /usr/bin/mailq.postfix
setgid_group = postdrop
html_directory = no
manpage_directory = /usr/share/man
sample_directory = /usr/share/doc/postfix-2.6.6/samples
readme_directory = /usr/share/doc/postfix-2.6.6/README_FILES
#Config Virtual Mailbox Settings 2022
virtual_minimum_uid = 100
virtual_mailbox_base = /var/spool/mail
virtual_mailbox_maps =
mysql:/etc/postfix/mysql_virtual_mailbox_maps.cf
virtual_mailbox_domains =
mysql:/etc/postfix/mysql_virtual_domains_maps.cf
virtual_alias_domains = $virtual_alias_maps
virtual_alias_maps =
```

```
mysql:/etc/postfix/mysql_virtual_alias_maps.cf
virtual_uid_maps = static:1000
virtual_gid_maps = static:1000
virtual_transport = virtual
maildrop_destination_recipient_limit = 1
maildrop_destination_concurrency_limit = 1
#Config  QUOTA  2022
message_size_limit = 52428800
mailbox_size_limit = 209715200
virtual_mailbox_limit = 209715200
virtual_create_maildirsize = yes
virtual_mailbox_extended = yes
virtual_mailbox_limit_maps                                    =
mysql:/etc/postfix/mysql_virtual_mailbox_limit_maps.cf
virtual_mailbox_limit_override = yes
virtual_maildir_limit_message = Sorry, the user's maildir has
overdrawn his diskspace quota, please try again later.
virtual_overquota_bounce = yes
#Config SASL 2022
```

```
broken_sasl_auth_clients = yes
smtpd_recipient_restrictions =
permit_mynetworks,permit_sasl_authenticated,reject_invali
d_hostname,reject_non_fqdn_hostname,reject_unknown_se
nder_domain,reject_non_fqdn_sender,reject_non_fqdn_reci
pient,reject_unknown_recipient_domain,reject_unauth_pipe
lining,reject_unauth_destination,permit
smtpd_sasl_auth_enable = yes
smtpd_sasl_type = dovecot
smtpd_sasl_path = /var/run/dovecot/auth-client
smtpd_sasl_local_domain = $myhostname
smtpd_sasl_security_options = noanonymous
smtpd_sasl_application_name = smtpd
smtpd_banner=$myhostname SMTP "Version not
Available"
```

（3） 创建 POSTFIX 读取 MYSQL 配置信息：

```
cat>/etc/postfix/mysql_virtual_alias_maps.cf<<EOF
user = postfix
password = postfix
```

```
hosts = localhost
dbname = postfix
table = alias
select_field = goto
where_field = address
EOF
```

```
cat>/etc/postfix/mysql_virtual_domains_maps.cf<<EOF
user = postfix
password = postfix
hosts = localhost
dbname = postfix
table = domain
select_field = description
where_field = domain
EOF
```

```
cat>/etc/postfix/mysql_virtual_mailbox_limit_maps.cf<<EO
F
user = postfix
password = postfix
```

```
hosts = localhost
dbname = postfix
table = mailbox
select_field = quota
where_field = username
EOF
```

```
cat>/etc/postfix/mysql_virtual_mailbox_maps.cf<<EOF
user = postfix
password = postfix
hosts = localhost
dbname = postfix
table = mailbox
select_field = maildir
where_field = username
EOF
```

mysql_virtual_mailbox_maps.cf：服务器邮箱文件的存储路径。

mysql_virtual_domains_maps.cf：邮件服务器上所有的虚拟域。

mysql_virtual_alias_maps.cf：邮件服务器上虚拟别名和实际邮

件地址间的对应关系。

mysql_virtual_mailbox_limit_maps.cf：服务器上邮箱的一些限制参数。

如上配置文件可以供 Postfix 读取数据库表。

默认是基于 hash:/etc/aliases 查询表，哈希文件，文件的路径是/etc/aliases。Postfix 在需要的时候读取这些配置信息，然后根据这些配置信息的指示，到另外的文件或者数据库中去读取实际的数据。

（4） 创建数据库并授权：

```
create database postfix charset=utf8;
grant all privileges on postfix.* to postfix@'localhost' identified by "postfix";
flush privileges;
```

（5） 修改 postfixadmin 配置文件数据库连接信息：

```
// Database Config
// mysql = MySQL 3.23 and 4.0, 4.1 or 5
// mysqli = MySQL 4.1+ or MariaDB
// pgsql = PostgreSQL
// sqlite = SQLite 3
$CONF['database_type'] = 'mysqli';
$CONF['database_host'] = 'localhost';
$CONF['database_user'] = 'postfix';
$CONF['database_password'] = 'postfix';
$CONF['database_name'] = 'postfix';
// If you need to specify a different port for a MYSQL dat
//   $CONF['database_host'] = '172.30.33.66:3308';
// If you need to specify a different port for POSTGRESQL
//   uncomment and change the following
// $CONF['database_port'] = '5432';
```

（6） 通过 Postfixamin 创建 4 个虚拟用户，如图所示：



（7） 配置 Dovecot 服务，修改 dovecot.conf 配置文件代码如下：

```
#/etc/dovecot/dovecot.conf
#CentOS: Linux 2.6.32
#2022 年 7 月 1 日 09:53:30
auth_mechanisms = PLAIN LOGIN
#auth_mechanisms = PLAIN LOGIN CRAM-MD5 DIGEST-MD5
disable_plaintext_auth = no
first_valid_uid = 1000
listen = *
mail_location = maildir:/var/spool/mail/%d/%n
managesieve_notify_capability = mailto
```

```
managesieve_sieve_capability = fileinto reject envelope encoded-character vacation subaddress comparator-i;ascii-numeric relational regex imap4flags copy include variables body enotify environment mailbox date
passdb {
  args = /etc/dovecot/mysql.conf
  driver = sql
}
protocols = imap pop3
service auth {
  unix_listener auth-client {
    group = postfix
    mode = 0660
    user = postfix
  }
}
ssl = no
userdb {
  args = /etc/dovecot/mysql.conf
```

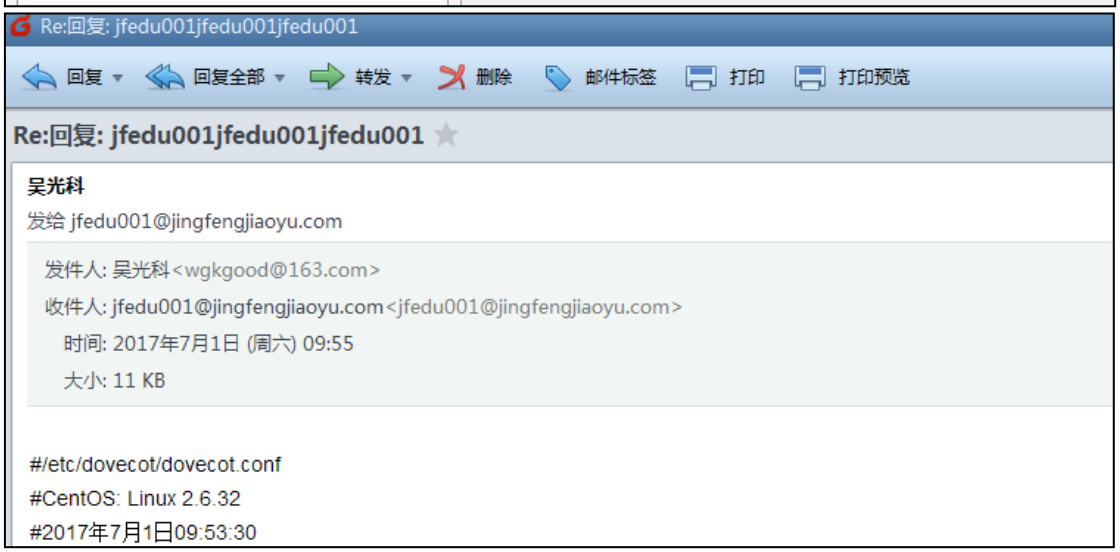
```
  driver = sql
}
```

（8） 创建 Dovecot 连接 MYSQL 认证，配置文件 /etc/dovecot/mysql.conf 代码如下：

```
driver = mysql
connect = host=/var/lib/mysql/mysql.sock dbname=postfix
user=postfix password=postfix
default_pass_scheme = MD5
password_query = SELECT password FROM mailbox WHERE
username = '%u'
user_query = SELECT maildir, 1000 AS uid, 1000 AS gid
FROM mailbox WHERE username = '%u'
```

（9） 使用虚拟用户登录 RoundCube，如图所示：





19.13 Postfix+Extmail 配置实战

通过上面的部署我们已经完成了一个基本的邮件服务器系统，它能够发送、接收邮件，能够对用户进行身份验证等工作。用户可以使用 Outlook、Foxmail、Roundcube 等的工具来发送和接收邮件。

使用 Roundcube WEB MAIL 页面功能相对比较少，可以使用 Extmail 实现 WEB 端邮件收取和发送。

Extmail 是一个以 perl 语言编写，面向大容量/ISP 级应用，免费的高性能 Webmail 软件，ExtMail 套件用于提供从浏览器中登录、使用邮件系统的 Web 操作界面，它以 GPL 版权释出，设计初衷是希望设计一个适应当前高速发展的 IT 应用环境，满足用户多变的需求，能快速进行开发、改进和升级，适应能力强的 webmail 系统。

对于国内的电子邮件系统来说，无论是从系统功能、易用性还是中文化等方面，ExtMail 平台都是一个相当不错的选择。Extmail 套件可以提供给普通邮件用户使用，而 Extman 套件可以提供给邮件系统的管理员使用。

Extmail 配置

官网下载：extmail-1.2.tar.gz WEB MAIL，解压安装：

```
tar   -zxf   extmail-1.2.tar.gz
mkdir   -p /var/www/extsuite
mv   extmail-1.2 /var/www/extsuite/extmail
cd   /var/www/extsuite/extmail/
cp   webmail.cf.default   webmail.cf
```

```
yum install perl-devel perl perl-Unix-Syslog -y
```

修改 webmail.cf 主配置文件

vim /var/www/extsuite/extmail/webmail.cf，配置代码如下：

```
SYS_CONFIG = /var/www/extsuite/extmail/

SYS_LANGDIR = /var/www/extsuite/extmail/lang

SYS_TEMPLDIR = /var/www/extsuite/extmail/html

SYS_HTTP_CACHE = 0

SYS_SMTP_HOST = mail.jingfengjiaoyu.com

SYS_SMTP_PORT = 25

SYS_SMTP_TIMEOUT = 5

SYS_SPAM_REPORT_ON = 0

SYS_SPAM_REPORT_TYPE = dspam

SYS_SHOW_WARN = 0

SYS_IP_SECURITY_ON = 1
```

```
SYS_PERMIT_NOQUOTA = 1

SYS_SESS_DIR = /tmp

SYS_UPLOAD_TMPDIR = /tmp

SYS_LOG_ON = 1

SYS_LOG_TYPE = syslog

SYS_LOG_FILE = /var/log/extmail.log

SYS_SESS_TIMEOUT = 0

SYS_SESS_COOKIE_ONLY = 1

SYS_USER_PSIZE = 10

SYS_USER_SCREEN = auto

SYS_USER_LANG = en_US

SYS_APP_TYPE = WebMail

SYS_USER_TEMPLATE = default
```

```
SYS_USER_CHARSET = utf-8

SYS_USER_TRYLOCAL = 1

SYS_USER_TIMEZONE = +0800

SYS_USER_CCSENT = 1

SYS_USER_SHOW_HTML = 1

SYS_USER_COMPOSE_HTML = 1

SYS_USER_CONV_LINK =1

SYS_USER_ADDR2ABOOK = 1

SYS_MESSAGE_SIZE_LIMIT = 5242880

SYS_MIN_PASS_LEN = 2

SYS_MFILTER_ON = 1

SYS_NETDISK_ON = 1

SYS_SHOW_SIGNUP = 1
```

```
SYS_DEBUG_ON = 1

SYS_AUTH_TYPE = mysql

SYS_MAILDIR_BASE = /var/spool/mail/

SYS_AUTH_SCHEMA = virtual

SYS_CRYPT_TYPE = md5crypt

SYS_MYSQL_USER = postfix

SYS_MYSQL_PASS = postfix

SYS_MYSQL_DB = postfix

SYS_MYSQL_HOST = localhost

SYS_MYSQL_SOCKET = /var/lib/mysql/mysql.sock

SYS_MYSQL_TABLE = mailbox

SYS_MYSQL_ATTR_USERNAME = username

SYS_MYSQL_ATTR_DOMAIN = domain
```

SYS_MYSQL_ATTR_PASSWD = password

SYS_MYSQL_ATTR_CLEARPW = clearpwd

SYS_MYSQL_ATTR_QUOTA = quota

SYS_MYSQL_ATTR_NDQUOTA = netdiskquota

SYS_MYSQL_ATTR_HOME = homedir

SYS_MYSQL_ATTR_MAILDIR = maildir

SYS_MYSQL_ATTR_DISABLEWEBMAIL = disablewebmail

SYS_MYSQL_ATTR_DISABLENETDISK = disablenetdisk

SYS_MYSQL_ATTR_DISABLEPWDCHANGE = disablepwdchange

SYS_MYSQL_ATTR_ACTIVE = active

SYS_MYSQL_ATTR_PWD_QUESTION = question

SYS_MYSQL_ATTR_PWD_ANSWER = answer

```
SYS_LDAP_BASE = o=extmailAccount,dc=example.com

SYS_LDAP_RDN = cn=Manager,dc=example.com

SYS_LDAP_PASS = secret

SYS_LDAP_HOST = localhost

SYS_LDAP_ATTR_USERNAME = mail

SYS_LDAP_ATTR_DOMAIN = virtualDomain

SYS_LDAP_ATTR_PASSWD = userPassword

SYS_LDAP_ATTR_CLEARPW = clearPassword

SYS_LDAP_ATTR_QUOTA = mailQuota

SYS_LDAP_ATTR_NDQUOTA = netdiskQuota

SYS_LDAP_ATTR_HOME = homeDirectory

SYS_LDAP_ATTR_MAILDIR = mailMessageStore

SYS_LDAP_ATTR_DISABLEWEBMAIL = disablewebmail
```

SYS_LDAP_ATTR_DISABLENETDISK = disablenetdisk

SYS_LDAP_ATTR_DISABLEPWDCHANGE =
disablePasswdChange

SYS_LDAP_ATTR_ACTIVE = active

SYS_LDAP_ATTR_PWD_QUESTION = question

SYS_LDAP_ATTR_PWD_ANSWER = answer

SYS_AUTHLIB_SOCKET = /var/spool/authdaemon/socket

SYS_G_ABOOK_TYPE = file

SYS_G_ABOOK_LDAP_HOST = localhost

SYS_G_ABOOK_LDAP_BASE =
ou=AddressBook,dc=example.com

SYS_G_ABOOK_LDAP_ROOTDN =
cn=Manager,dc=example.com

SYS_G_ABOOK_LDAP_ROOTPW = secret

SYS_G_ABOOK_LDAP_FILTER = objectClass=OfficePerson

SYS_G_ABOOK_FILE_PATH = /var/www/extsuite/extmail/globabook.cf

SYS_G_ABOOK_FILE_LOCK = 1

SYS_G_ABOOK_FILE_CONVERT = 0

SYS_G_ABOOK_FILE_CHARSET = utf-8

常用配置文件参数详解：

SYS_MESSAGE_SIZE_LIMIT = 5242880

用户可以发送的最大邮件

SYS_USER_LANG = en_US

语言选项，可改作：

SYS_USER_LANG = zh_CN

SYS_MAILDIR_BASE = /home/domains

此处即为您在前文所设置的用户邮件的存放目录，可改作：

SYS_MAILDIR_BASE = /var/spool/mail/

SYS_MYSQL_USER = postfix

SYS_MYSQL_PASS = db_pass

以上两句句用来设置连接数据库服务器所使用用户名、密码和邮件服务器用到的数据库，这里修改为：

SYS_MYSQL_USER = postfix

SYS_MYSQL_PASS = postfix

SYS_MYSQL_HOST = localhost

指明数据库服务器主机名，这里默认即可

SYS_MYSQL_TABLE = mailbox

SYS_MYSQL_ATTR_USERNAME = username

SYS_MYSQL_ATTR_DOMAIN = domain

SYS_MYSQL_ATTR_PASSWD = password

以上用来指定验正用户登录里所用到的表，以及用户名、域名和用户密码分别对应的表中列的名称；这里默认即可

Apache 配置文件 httpd.conf 设置方式一：

```
User postfix

Group postfix

<VirtualHost *:80>

ServerName          mail.jingfengjiaoyu.com

DocumentRoot       /var/www/extsuite/extmail/html/

ScriptAlias           /extmail/cgi
/var/www/extsuite/extmail/cgi

Alias                  /extmail /var/www/extsuite/extmail/html

</VirtualHost>
```

Apache 配置文件 httpd.conf 设置方式二：

由于 Extmail 要进行本地邮件的投递操作，须将运行 apache 服务器用户的身份修改为邮件投递代理的用户，本案例为 postfix；

开启 apache 服务器的 suexec 功能，使用如下方法实现虚拟主机运行身份的指定。

```
<VirtualHost *:80>
ServerName    mail.jingfengjiaoyu.com
DocumentRoot /var/www/extsuite/extmail/html/
ScriptAlias      /extmail/cgi    /var/www/extsuite/extmail/cgi
Alias            /extmail     /var/www/extsuite/extmail/html
SuexecUserGroup postfix postfix
</VirtualHost>
```

浏览器访问 Extmail，截图如下：







19.14 Postfix+Extman 配置实战

Extmail WebMail 推出之后，缺少一个与之配套的邮件虚拟域和账号管理的后台。虽然目前已经有不少类似的软件，其中也不乏非常优秀的产品，例如 postfixadmin 等，但是没有任何一款可以不经修改的为我所用，再加上编程语言和系统兼容性方面的考虑，

我们推出了 Extman 系统。顾名思义，Extman 就是 Extmail Management 的意思。有了他，Extmail Webmail 的用户可以方便的管理 Extmail 邮件系统。后来由于需要，又增加了图形化日志监控等等新的功能，从而形成了今天大家看到的 Extman。

Extman 是 Extmail 项目组在推出 Extmail WebMail 之后，推出的一个用来管理 Extmail 虚拟账号的管理软件。

使用 Perl 语言编写，目前支持 MySQL 和 OpenLDAP 作为账号信息存储源，新的存储源支持正在开发之中。

它可以使你通过浏览器轻松的管理您在 Extmail 系统中的虚拟域和账号信息。同时还加入了图形化日志监控工具，使邮局的管理更加方便。

```
tar xzf extman-1.1.tar.gz -C /var/www/extsuite/
cd    /var/www/extsuite/
mv    extman-1.1 extman
chown -R postfix.postfix /var/www/extsuite/extman/cgi/
cd    extman/
cp    webman.cf.default  webman.cf
mkdir  -p  /tmp/extman
chown postfix.postfix /tmp/extman
```

```
yum  install  perl-rrdtool*  -y
```

创建 extman 所需数据库：

```
cd  /var/www/extsuite/extman/docs
mysql  -uroot -p <extmail.sql
mysql  -uroot -p <init.sql
进入 mysql 命令行执行：
grant all on extmail.* to webman@'localhost' identified by
"webman";
flush  privileges;
```

vim /var/www/extsuite/extman/webmail.cf，配置代码如下：

```
SYS_CONFIG = /var/www/extsuite/extman/
SYS_LANGDIR = /var/www/extsuite/extman/lang
SYS_TEMPLDIR = /var/www/extsuite/extman/html
SYS_MAILDIR_BASE = /var/spool/mail
SYS_SHOW_WARN = 0
SYS_SESS_DIR = /tmp/extman/
SYS_CAPTCHA_ON = 1
SYS_CAPTCHA_KEY = r3s9b6a7
SYS_CAPTCHA_LEN = 6
```

```
SYS_PURGE_DATA = 0
SYS_PSIZE = 20
SYS_APP_TYPE = ExtMan
SYS_TEMPLATE_NAME = default
SYS_DEFAULT_EXPIRE = 1y
SYS_GROUPMAIL_SENDER = postmaster@extmail.org
SYS_DEFAULT_SERVICES =
webmail,smtpd,smtp,pop3,netdisk
SYS_ISP_MODE = no
SYS_DOMAIN_HASHDIR = yes
SYS_DOMAIN_HASHDIR_DEPTH = 2x2
SYS_USER_HASHDIR = yes
SYS_USER_HASHDIR_DEPTH = 2x2
SYS_MIN_UID = 500
SYS_MIN_GID = 100
SYS_DEFAULT_UID = 1000
SYS_DEFAULT_GID = 1000
SYS_QUOTA_MULTIPLIER = 1048576
SYS_QUOTA_TYPE = courier
```

```
SYS_DEFAULT_MAXQUOTA = 500

SYS_DEFAULT_MAXALIAS = 100

SYS_DEFAULT_MAXUSERS = 100

SYS_DEFAULT_MAXNDQUOTA = 500

SYS_USER_DEFAULT_QUOTA = 5

SYS_USER_DEFAULT_NDQUOTA = 5

SYS_USER_DEFAULT_EXPIRE = 1y

SYS_BACKEND_TYPE = mysql

SYS_CRYPT_TYPE = md5crypt

SYS_MYSQL_USER = webman

SYS_MYSQL_PASS = webman

SYS_MYSQL_DB = extmail

SYS_MYSQL_HOST = localhost

SYS_MYSQL_SOCKET = /var/lib/mysql/mysql.sock

SYS_MYSQL_TABLE = manager

SYS_MYSQL_ATTR_USERNAME = username

SYS_MYSQL_ATTR_PASSWD = password

SYS_LDAP_BASE = dc=extmail.org

SYS_LDAP_RDN = cn=Manager,dc=extmail.org
```

```
SYS_LDAP_PASS = secret
SYS_LDAP_HOST = localhost
SYS_LDAP_ATTR_USERNAME = mail
SYS_LDAP_ATTR_PASSWD = userPassword
SYS_RRD_DATADIR = /var/lib
SYS_RRD_TMPDIR = /tmp/viewlog
SYS_RRD_QUEUE_ON = yes
SYS_CMDSERVER_SOCK = /tmp/cmdserver.sock
SYS_CMDSERVER_MAXCONN = 5
SYS_CMDSERVER_PID = /var/run/cmdserver.pid
SYS_CMDSERVER_LOG = /var/log/cmdserver.log
SYS_CMDSERVER_AUTHCODE = your_auth_code_here
SYS_IGNORE_SERVER_LIST = web
```

修改 httpd.conf 配置文件代码如下：

```
User postfix
Group postfix
<VirtualHost *:80>
ServerName          mail.jingfengjiaoyu.com
DocumentRoot       /var/www/extsuite/extmail/html/
```

```
ScriptAlias /extmail/cgi /var/www/extsuite/extmail/cgi
Alias /extmail /var/www/extsuite/extmail/html
ScriptAlias /extman/cgi /var/www/extsuite/extman/cgi
Alias /extman /var/www/extsuite/extman/html
</VirtualHost>
```

使用默认用户名/密码登录

Extman 后台管理员：root@extmail.org

Extman 后台管理员初始密码:extmail*123*

Postmaster 用户密码：extmail

浏览器访问：http://mail.jingfengjiaoyu.com/extman/





19.15 MailGraph_ext 安装配置：

安装 MailGraph_ext 之前，必须要安装他所依赖的一些软件包，否则 MailGraph_ext 就不能正常工作。

*rrdtool 及 rrdtool 的 perl 包，地址：

http://people.ee.ethz.ch/~oetiker/webtools/rrdtool/

*File::Tail (还需要 Time::HiRes 模块)

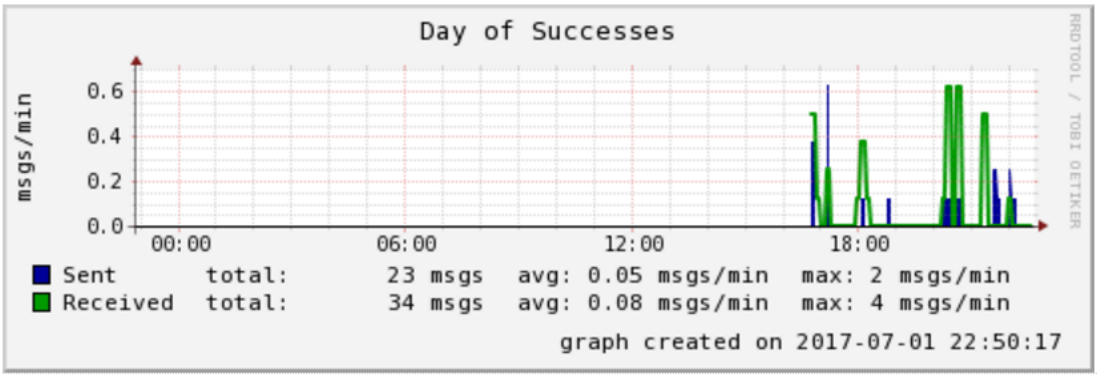
```
cp    -r    /var/www/extsuite/extman/addon/mailgraph_ext/
/usr/local/mailgraph_ext/
/usr/local/mailgraph_ext/mailgraph-init start
/var/www/extsuite/extman/daemon/cmdserver --daemon
```

将 MailGraph_ext 及 cmdserver 加入系统自启动

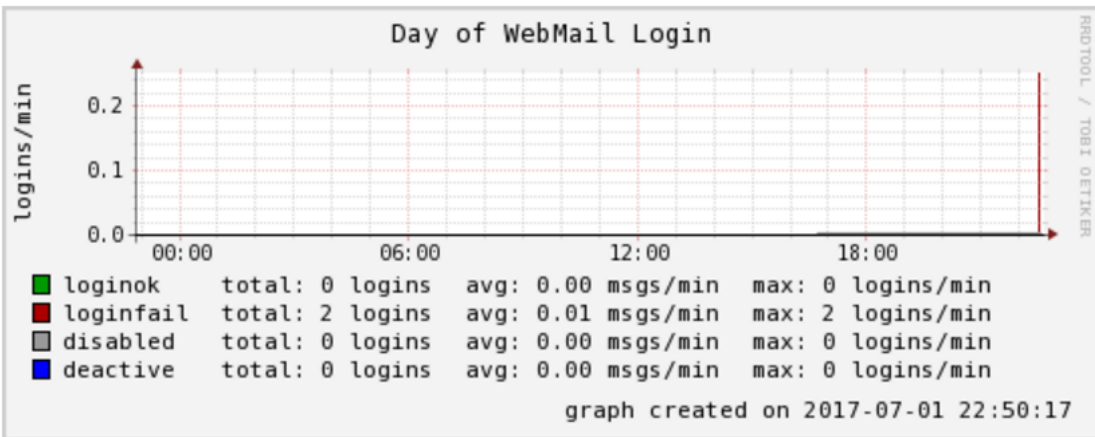
```
echo "/usr/local/mailgraph_ext/mailgraph-init start" >> /etc/rc.d/rc.local
echo "/var/www/extsuite/extman/daemon/cmdserver -v -d" >> /etc/rc.d/rc.local
```

>> [投递成功信件分析图] [失败投递信件分析图] [邮件投递流量分析图]
>> [**IMAP/POP3** 登陆日志图] [邮件队列分析图] [**WebMail** 登陆日志图]

投递成功信件分析图



WebMail 登陆日志图



19.16 基于 Postfix+Extman 虚拟用户注册





19.17 基于 Extman 自动注册并登陆

（1） 通过 extman 自动注册用户和设置密码之后，发现用户无法登陆 extmail WEB 平台，提示用户名和密码错误，如图所示所示：



问题分析步骤：

由于 extman 数据库连接的是 extmail 库，而虚拟用户读取的库 postfix，导致数据不同步。解决方法可以同步数据，或者合并库；

```
mysqldump -uroot -p postfix >postfix.sql
mysqldump -uroot -p extmail >extmail.sql
进入 mysql 命令行：
drop database extmail;
use postfix;
```

```
source /tmp/extmail.sql;
```

修改 webman.cf 配置文件数据库连接信息：

```
SYS_MYSQL_USER = postfix

SYS_MYSQL_PASS = postfix

SYS_MYSQL_DB = postfix
```

重新登录 Extmail 访问，使用用户名 jfedu888 登录如图所示：



重新注册虚拟用户 ,验证登录 ,注册用户 jfedu999 ,密码 jfedu999





（2） Postfixadmin 无法登陆，报错如下：

PHP Warning: Unknown: Failed to write session data (files). Please verify that the current setting of session.save_path is correct (/var/lib/php/session) in Unknown on line 0, referer: http://mail.jingfengjiaoyu.com/postfixadmin/login.php [Sat Jul 01 23:24:42 2022] [error] [client 36.102.227.109] PHP Warning: session_destroy(): Session object destruction

```
failed in /var/www/html/postfixadmin/common.php on line 30, referer: http://mail.jingfengjiaoyu.com/postfixadmin/main.php
```



Apache 配置文件添加如下代码：

```
User postfix
Group postfix
<VirtualHost *:80>
ServerName mail.jingfengjiaoyu.com
DocumentRoot
/var/www/extsuite/extmail/html/
ScriptAlias /extmail/cgi
/var/www/extsuite/extmail/cgi
```

```
Alias                                   /extmail
/var/www/extsuite/extmail/html
ScriptAlias                             /extman/cgi
/var/www/extsuite/extman/cgi
Alias                                   /extman
/var/www/extsuite/extman/html
Alias                                   /postfixadmin
/var/www/html/postfixadmin/
</VirtualHost>
```

重启 Apache 服务，访问如图所示：





无法登录 postfixadmin，查看 maillog 日志如图所示：

tail -fn 100 /var/log/httpd/error_log

```
[error] [client 36.102.227.109] PHP Warning:  Unknown: open(/var/lib/php/
4sqj85oj6, O_RDWR) failed: Permission denied (13) in Unknown on line 0,
iaoyu.com/postfixadmin/login.php
[error] [client 36.102.227.109] PHP Warning:  Unknown: Failed to write se
ify that the current setting of session.save_path is correct (/var/lib/p
 0, referer: http://mail.jingfengjiaoyu.com/postfixadmin/login.php
[error] [client 36.102.227.109] PHP Warning:  session_destroy(): Session
/var/www/html/postfixadmin/common.php on line 30, referer: http://mail.j
main.php
[error] [client 36.102.227.109] PHP Warning:  session_destroy(): Session
/var/www/html/postfixadmin/common.php on line 30, referer: http://mail.j
login.php
[error] [client 36.102.227.109] PHP Warning:  Unknown: open(/var/lib/php/
l12u1cbt3, O_RDWR) failed: Permission denied (13) in Unknown on line 0,
iaoyu.com/postfixadmin/login.php
```

解决方法：chmod 757　-R　/var/lib/php/session/



第20章　自动化运维发展前景

随着企业服务器数量越来越多，当到达几百台，上千台服务器之后，服务器日常管理也逐渐繁杂，每天如果通过人工去频繁的更新或者部署及管理这些服务器，势必会浪费大量的时间，而且有可能人为的操作也会造成某些疏忽而遗漏。那我们来看一下传统的运维以及今后运维的发展方向。

本章向读者介绍如何构建企业自动化运维之路、传统运维方案及自动化运维方案、如何建立高效的 IT 自动化运维平台以及自动化运维体系各种工具等。

20.1 传统运维方式简介

传统的 IT 运维仍然是等到 IT 故障出现后再由运维人员采取相应的补救措施。这种被动、孤立、半自动式的 IT 运维管理模式经常让 IT 部门疲惫不堪，主要表现在以下三个方

面：

（1） 运维人员被动、效率低

在 IT 运维过程中，只有当事件已经发生并已造成业务影响时才能发现和着手处理，这种被动“救火”不但使 IT 运维人员终日忙碌，也使 IT 运维本身质量很难提高，导致 IT 部门和业务部门对 IT 运维的服务满意度都不高。

（2） 缺乏一套高效的 IT 运维机制

许多企业在 IT 运维管理过程中缺少自动化的运维管理模式，也没有明确的角色定义和责任划分，使到问题出现后很难快速、准确地找到根本原因，无法及时地找到相应的人员进行修复和处理，或者是在问题找到后缺乏流程化的故障处理机制，而在处理问题时不但欠缺规范化的解决方案，也缺乏全面的跟踪记录。

（3） 缺乏高效的 IT 运维技术工具

随着信息化建设的深入，企业 IT 系统日趋复杂，林林总总的网络设备、服务器、中间件、业务系统等让 IT 运维人员难以从容应对，即使加班加点地维护、部署、管理也经常会因设备出现故障而导致业务的中断，严重影响企业的正常运转。

出现这些问题部分原因是企业缺乏事件监控和诊断工具等 IT 运维技术工具，因为在没有高效的技术工具的支持下故障事件很难得到主动、快速的处理。

20.2 自动化运维简介

IT 运维已经在风风雨雨中走过了十几个春秋，如今它正以一种全新的姿态摆在我们

面前 ,运维自动化-这是 IT 技术发展的必然结果 ,现在 IT 系统的复杂性已经客观上要求 IT 运维必须能够实现数字化、自动化维护。

运维自动化是指将 IT 运维中日常的、大量的重复性工作自动化，把过去的手工执行转为自动化操作。自动化是 IT 运维工作的升华，IT 运维自动化不单纯是一个维护过程，更是一个管理的提升过程，是 IT 运维的最高层次，也是未来的发展趋势。

20.3 运维自动化的具体内容

日常 IT 运维中大量的重复性工作（小到简单的日常检查、配置变更和软件安装，大到整个变更流程的组织调度）由过去的手工执行转为自动化操作，从而减少乃至消除运维中的延迟，实现“零延时”的 IT 运维。

简单的说，IT 运维自动化是指基于流程化的框架，将事件与 IT 流程相关联，一旦被监控系统发生性能超标或宕机，会触发相关事件以及事先定义好的流程，可自动启动故障响应和恢复机制。

20.4 建立高效的 IT 自动化运维管理

建立高效的 IT 自动化运维管理步骤主要包括以下几点：

（1） 建立自动化运维管理平台

IT 运维自动化管理建设的第一步是要先建立 IT 运维的自动化监控和管理平台。通过监控工具实现对用户操作规范的约束和对 IT 资源进行实时监控，包括服务器、数据库、中间件、存储备份、网络、安全、机房、业务应用和客户端等内容，通过自动监控管理平台

实现故障或问题综合处理和集中管理。

（2） 建立故障事件自动触发流程，提高故障处理效率

所有 IT 设备在遇到问题时要会自动报警，无论是系统自动报警还是使用人员报的故障，应以红色标识显示在运维屏幕上。然后 IT 运维人员只需要按照相关知识库的数据，一步一步操作就可以。

（3） 建立规范的事件跟踪流程，强化运维执行力度

需要建立故障和事件处理跟踪流程，利用表格工具等记录故障及其处理情况，以建立运维日志，并定期回顾从中辨识和发现问题的线索和根源。

（4） 设立 IT 运维关键流程，引入优先处理原则

设置自动化流程时还需要引入优先处理原则，例行的事按常规处理，特别事件要按优先级次序处理，也就是把事件细分为例行事件和例外关键事件。

20.5 IT 自动化运维工具

对于企业来说，要特别关注两类自动化工具：一是 IT 运维监控和诊断优化工具；二是运维流程自动化工具。这两类工具主要应用于：

（1） 监控自动化：是指对重要的 IT 设备实施主动式监控,如路由器、交换机、防火墙等。

（2） 配置变更检测自动化：是指 IT 设备配置参数一旦发生变化，将触发变更流程转给相关技术人员进行确认,通过自动检测协助 IT 运维人员发现和维护配置。

（3） 维护事件提醒自动化：是指通过对 IT 设备和应用活动的时时监控,当发生异常事件时系统自动启动报警和响应机制,第一事件通知相关责任人。

（4） 系统健康检测自动化：是指定期自动地对 IT 设备硬件和应用系统进行健康巡检,配合 IT 运维团队实施对系统的健康检查和监控。

（5） 维护报告生成自动化：是指定期自动的对系统做日志的收集分析,记录系统运行状况,并通过阶段性的监控、分析和总结,定时提供 IT 运维的可用性、性能、系统资源利用状况分析报告。

20.6 IT 自动化运维体系

一个完善的自动化运维体系包括系统预备、配置管理以及监控报警三种环节，每个环节实现的功能也各不相同，具体功能如下：

（1） 系统预备类：

- 自动化安装操作系统；
- 自动初始化系统；
- 自动安装各种软件包。

（2） 配置管理类：

- 自动化部署业务系统软件包并完成配置；
- 远程管理服务器；
- 配置文件、自动部署 Jenkins、网站代码变更回滚。

（3） 监控报警类：

- 服务器可用性、性能、安全监控；
- 向管理员发送报警信息。

根据提供的功能不同，自动化运维工具软件分为以下 3 类，如下 19-1 表所示：

编号	预备类工具	配置管理类	监控报警类
1	Kickstart	Puppet	Nagios
2	Cobbler	Saltstack	Cacti
3	OpenQRM	Func	Ganglia
4	Spacewalk	Ansible	Zabbix

表 19-1 自动化运维工具分类

第21章 Puppet 自动运维企业实战

Puppet 是目前互联网主流三大自动化运维工具（Puppet、Ansible、Saltstack）之一，Puppet 是一种 Linux、Unix 平台的集中配置管理系统，所谓配置管理系统，就是管理机器里面诸如文件、用户、进程、软件包等资源，其设计目标是简化对这些资源的管理以及妥善处理资源间的依赖关系。

本章向读者介绍 Puppet 工作原理、Puppet 安装配置、企业资源案例讲解、Puppet 高可用集群配置、Puppet 批量更新部署网站、Puppet+SVN 实现代码自动部署等。

21.1 Puppet 入门简介

Puppet 使用一种描述性语言来定义配置项，配置项中被称为”资源”，描述性语言可以声明你的配置的状态，比如声明一个软件包应该被安装或者一个服务应该被启动用。

Puppet 可以运行一台服务器端，每个客户端通过 ssl 证书连接服务器，得到本机器的配置列表，然后更加列表的来完成配置工作，所以如果硬件性能比较高，维护个管理上千上万台机器是非常轻松的，前提是客户端机器的配置、服务器路径、软件需要保持一致。

在企业级大规模的生成环境中，如果只有一台 puppet master 压力会非常大，因为 puppet 是用 ruby 语言编写，ruby 是解析型语言，每个客户端来访问都要解析一次，当客户端服务器很多，会操作服务器端压力很多，所以需要扩展成一个服务器集群组。

Puppet master 可以看作一个 web 服务器，实际上也是由 ruby 提供的 web 服务器模块来做的。因此可以利用 web 代理软件来配合 puppet master 做集群设置，一般使用 Nginx+Puppet Master 整合构建大型企业自动化运维管理工具 ,puppet 项目主要开发者是 Luke Kanies，目前为 puppet labs CEO，Puppet 遵循 GPLv2 版权协议。

从 1997 年开始 Kanies 参与 UNIX 的系统管理工作，Puppet 的开发源于这些经验。因为对已有的配置工具不甚满意，从 2001 年到 2005 年间，Kanies 开始在 Reductive 实验室从事工具的开发。很快 Reductive 实验室发布了他们新的旗舰产品。

Puppet 是开源的基于 Ruby 的系统配置管理工具，Puppet 工作流程为：puppet 是一个 C/S 结构 ,所有的 puppet 客户端同一个服务器端的 puppet 通讯 ,每个 puppet

客户端每半小时(可以设置)连接一次服务器端，下载最新的配置文件，并且严格按照配置文件来配置服务器，配置完成以后 puppet 客户端可以反馈给服务器端一个消息，如果报错会给服务器端反馈一个消息。

21.2 Puppet 工作原理

要熟练掌握 Puppet 在企业生产环境中的应用，需要深入理解 Puppet 服务端与客户端详细的工作流程及原理，如图 20-1（a）、20-1（b）所示，为 Puppet Master 与 Agent 完整工作流程图：

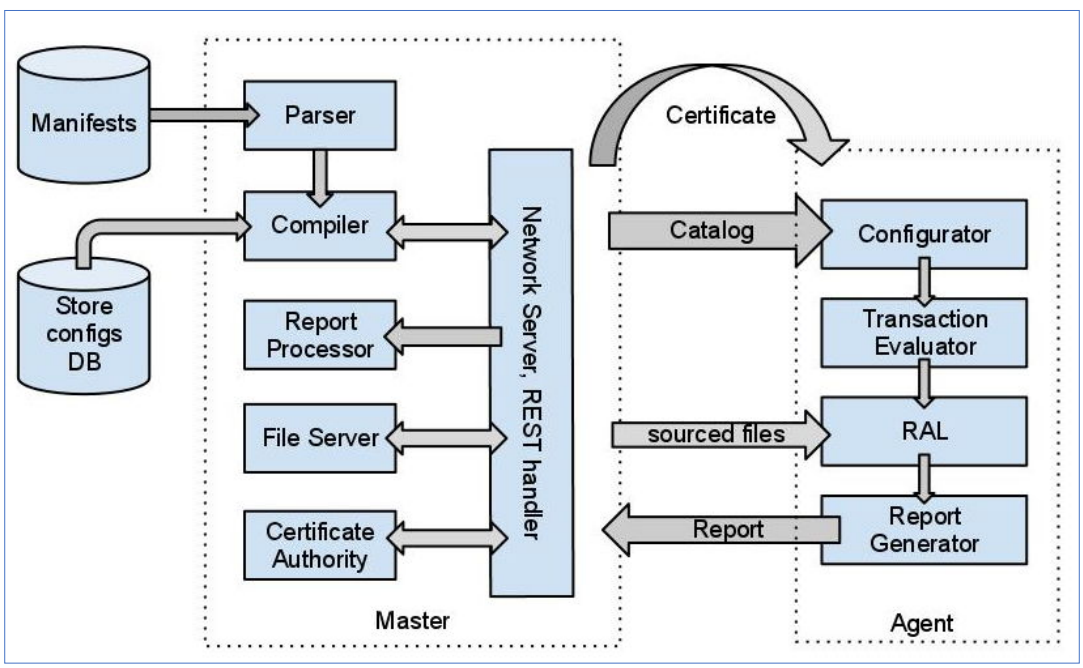


图 20-1（a） Puppet 工作原理图

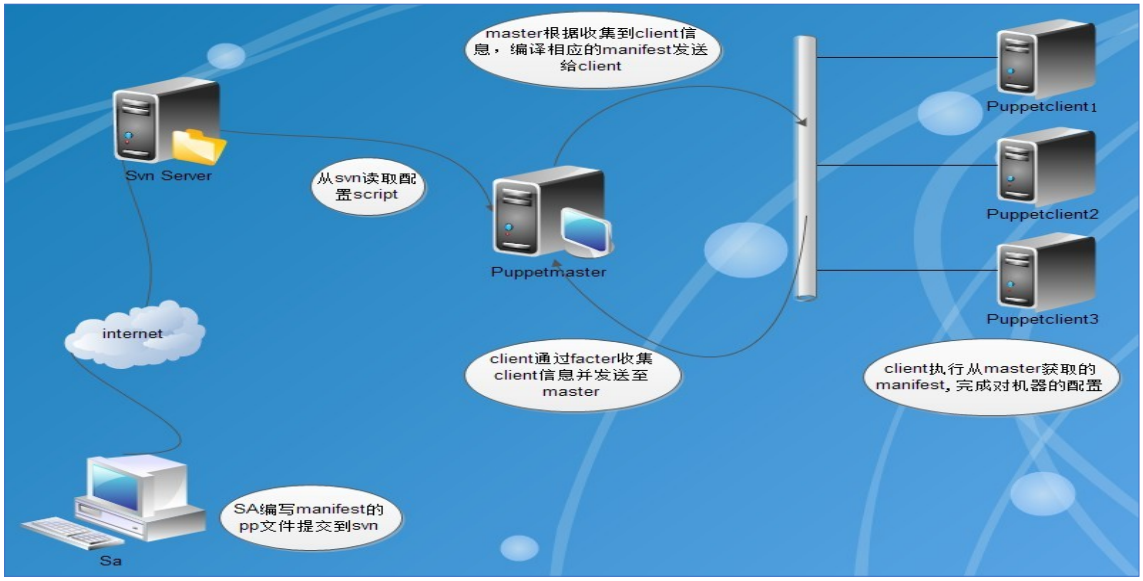


图 20-1（b） Puppet 工作原理图

Puppet 工作原理详解如下：

- 客户端 puppetd 调用本地 facter，facter 会探测出该主机的常用变量，例如主机名、内存大小、IP 地址等。然后 puppetd 把这些信息发送到 Puppet 服务端。
- Puppet 服务端检测到客户端的主机名，然后会检测 manifest 中对应的 node 配置，并对这段内容进行解析，facter 发送过来的信息可以作为变量进行处理。
- Puppet 服务端匹配 Puppet 客户端相关联的代码才进行解析 ,其它的代码不解析，解析分几个过程：语法检查、然后会生成一个中间的伪代码，然后再把伪代码发给 Puppet 客户端。
- Puppet 客户端接收到伪代码之后就会执行，执行完后会将执行的结果发送给 Puppet 服务端。
- Puppet 服务端再把客户端的执行结果写入日志。

21.3 Puppet 安装配置

由于 Puppet 为 C/S 模式，构建 Puppet 平台需安装 Puppet Sserver 端和 Client 端，安装之前准备好系统环境，如下：

```
操作系统版本：CentOS 6.5 x64

服务端 ip 192.168.149.128    hostname：192-168-149-128-jfedu.net

客户端 ip 192.168.149.130    hostname：192-168-149-130-jfedu.net
```

（1） Puppet 服务端安装

由于 Puppet 主要是基于 hostname 来检测的，所以 Puppet 服务器端需修改主机名称为：192-168-149-128-jfedu.net，并且在 hosts 文件添加主机名和本机 IP 的对应关系，如果本地局域网有 DNS 服务器，可以无需修改 hosts 文件，修改主机名及配置 hosts 代码如下：

```
hostname `ifconfig eth0 |grep Bcast|awk '{print $2}'|cut -d:    -f    2 |sed 's/\./\-/g'`-
jfedu.net
cat >>/etc/hosts<<EOF
192.168.149.128 192-168-149-128-jfedu.net
192.168.149.130 192-168-149-130-jfedu.net
EOF
```

Puppet 服务端除了需要安装 Puppet-server 外，还需要 Ruby 的支持，需要安装

Ruby 相关软件包 ,默认 YUM 安装 Puppet Server ,会自动下载并安装 Ruby 相关软件，

如图 20-2 所示：

```
rpm -Uvh http://yum.puppetlabs.com/el/6/products/x86_64/puppetlabs-release-6-
1.noarch.rpm
yum  install  puppet-server -y
/etc/init.d/puppetmaster  start
/etc/init.d/iptables  stop
sed -i  '/SELINUX/S/enforce/disabled/' /etc/selinux/config
setenforce  0
```

```
[root@192-168-149-128-jfedu ~]# rpm -Uvh http://yum.puppetlabs.com/
Retrieving http://yum.puppetlabs.com/el/6/products/x86_64/puppetlab
warning: /var/tmp/rpm-tmp.03j7NA: Header V4 RSA/SHA1 Signature, key
Preparing...                ########################################
   1:puppetlabs-release     ########################################
[root@192-168-149-128-jfedu ~]# yum  install  puppet-server -y
Loaded plugins: fastestmirror
Determining fastest mirrors
epel/metalink
 * base: mirrors.btte.net
 * epel: mirrors.tuna.tsinghua.edu.cn
 * extras: mirrors.tuna.tsinghua.edu.cn
 * updates: mirror.bit.edu.cn
base
epel
```

图 20-2 Puppet-server 服务端安装

（2） Puppet 客户端安装

Puppet 主要是基于 hostname 来检测的，所以 Puppet 客户端也需要修改主机名称为：192-168-149-130-jfedu.net，并且在 hosts 文件添加主机名和本机 IP 的对应关系 ,如果本地局域网有 DNS 服务器 ,可以无需修改 hosts 文件 ,修改主机名及配置 hosts

代码如下：

```
hostname `ifconfig eth0 |grep Bcast|awk '{print $2}'|cut -d:    -f    2 |sed 's/\./\-/g'`-
jfedu.net
cat >>/etc/hosts<<EOF
192.168.149.128 192-168-149-128-jfedu.net
192.168.149.130 192-168-149-130-jfedu.net
EOF
```

Puppet 客户端除了需要安装 puppet 外，还需要 Ruby 的支持，需要安装 Ruby 相关软件包，默认 YUM 安装 Puppet，会自动下载并安装 Ruby 相关软件，如图 20-3 所示：

```
rpm -Uvh http://yum.puppetlabs.com/el/6/products/x86_64/puppetlabs-release-6-
1.noarch.rpm
yum    install    puppet -y
/etc/init.d/puppetmaster    start
/etc/init.d/iptables          stop
sed -i    '/SELINUX/S/enforce/disabled/' /etc/selinux/config
setenforce    0
```

```
[root@192-168-149-130-jfedu ~]# rpm -Uvh http://yum.puppetlabs.com/e

yum  install  puppet -y
/etc/init.d/puppetmaster  start
/etc/init.d/iptables      stop
sed -i  '/SELINUX/S/enforce/disabled/' /etc/selinux/config
setenforce  0
Retrieving http://yum.puppetlabs.com/el/6/products/x86_64/puppetlabs

warning: /var/tmp/rpm-tmp.3G98fV: Header V4 RSA/SHA1 Signature, key
Preparing...                ########################################
   1:puppetlabs-release     ########################################
[root@192-168-149-130-jfedu ~]# yum  install  puppet -y
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
epel/metalink
```

图 20-3 Puppet 客户端服务安装

（3） Puppet 客户端申请证书

由于 Puppet 客户端与 Puppet 服务端是通过 SSL 隧道通信的，客户端安装完成后，首次使用需向服务器端申请 Puppet 通信证书， Puppet 客户端第一次连接服务器端会发起证书申请，在 Puppet 客户端执行命令如下，返回结果如图 20-4 所示：

```
puppet  agent  --server 192-168-149-128-jfedu.net  --test
```

```
-jfedu ~]#
-jfedu ~]# puppet  agent  --server 192-168-149-128-jfedu.net  --test
cate for ca
file loading from /etc/puppet/csr_attributes.yaml
SSL certificate request for 192-168-149-130-jfedu.net
uest fingerprint (SHA256): EC:89:AE:7C:42:50:5A:66:45:D1:0F:1D:4B:5F
E
cate for ca
te found and waitforcert is disabled
-jfedu ~]#
```

图 20-4 Puppet 客户端发起证书申请

（4） Puppet 服务端颁发证书

Puppet 客户端向服务器发起证书申请，服务器端必须审核证书，如果不审核，客户

端与服务器端无法进行后续正常通信，Puppet 服务端颁发证书命令代码如下，返回结果如图 20-5 所示：

puppet cert --list	查看申请证书的客户端主机名；
puppet cert -s 192-168-149-130-jfedu.net	颁发证书给客户端；
puppet cert -s	为特定的主机颁发证书；
puppet cert -s and -a	给所有的主机颁发证书；
puppet cert --list --all	查看已经颁发的所有证书。

```
[root@192-168-149-128-jfedu ~]# puppet  cert  --list
  "192-168-149-130-jfedu.net" (SHA256) EC:89:AE:7C:42:50:5A:66:45:D1:0F
7:D6:9A:10:8E
[root@192-168-149-128-jfedu ~]#
[root@192-168-149-128-jfedu ~]# puppet  cert  -s  192-168-149-130-jfedu
Notice: Signed certificate request for 192-168-149-130-jfedu.net
Notice: Removing file Puppet::SSL::CertificateRequest 192-168-149-130-j
68-149-130-jfedu.net.pem'
[root@192-168-149-128-jfedu ~]#
[root@192-168-149-128-jfedu ~]# puppet  cert  --list
[root@192-168-149-128-jfedu ~]#
[root@192-168-149-128-jfedu ~]# puppet  cert  --list --all
+ "192-168-149-128-jfedu.net" (SHA256) E5:E7:7B:2C:FE:EA:AC:04:3D:C5:21
C:23:FF:80:EE (alt names: "DNS:192-168-149-128-jfedu.net", "DNS:puppet"
+ "192-168-149-130-jfedu.net" (SHA256) C1:B5:7E:05:81:D0:BE:77:91:E3:62
0:1F:5B:DD:0C
```

图 20-5 Puppet 服务端颁发证书

21.4 Puppet 企业案例演示

Puppet 是基于 C/S 架构，服务器端保存着所有对客户端服务器的配置代码，在 puppet 服务端该配置文件叫 manifest ,客户端下载 manifest 之后 ,可以根据 manifest 对客户端进行配置，例如软件包管理、用户管理、文件管理、命令管理、脚本管理等，Puppet 主要基于各种资源或者模块来管理客户端。

默认 Puppet 服务器端 manifest 目录在/etc/puppet/manifests/下 ,只需要在该目

录下创建一个 site.pp 文件，然后写入相应的配置代码，Puppet 客户端跟 Puppet 服务端同步时，会检查客户端 node 配置文件，匹配之后会将该代码下载至客户端，对代码进行解析，然后在客户端执行。

如下为在 Puppet 客户端创建 test.txt 文件，并在该文件中写入测试内容，操作方法如下：

（1） Puppet 服务端创建 node 代码，创建或者编辑 vi /etc/puppet/manifests/site.pp 文件，在文件中加入如下代码：

```
node   default {
file   {
"/tmp/test.txt":
 content => "Hello World，jfedu.net   2022"；
    }
}
```

manifests site.pp 配置文件代码详解如下：

```
node   default       新建 node 节点，default 表示所有主机，可修改为特定主机名；
file                 基于 file 资源模块管理客户端文件或者目录操作；
"/tmp/test.txt":     需在客户端文件创建的文件名；
 content             客户端服务器文件内容。
```

（2） 客户端执行同步命令，获取 Puppet 服务端 node 配置，代码如下，如图 20-6

所示，执行报错：

```
puppet  agent  --server=192-168-149-128-jfedu.net  --test
```

```
[root@192-168-149-130-jfedu tmp]# puppet  agent  --server=192-168-1
Warning: Unable to fetch my node definition, but the agent run will
Warning: SSL_connect returned=1 errno=0 state=SSLv3 read server cer
 not yet valid for /CN=Puppet CA: 192-168-149-128-jfedu.net]
Info: Retrieving pluginfacts
Error: /File[/var/lib/puppet/facts.d]: Failed to generate additiona
d=1 errno=0 state=SSLv3 read server certificate B: certificate veri
t CA: 192-168-149-128-jfedu.net]
Error: /File[/var/lib/puppet/facts.d]: Could not evaluate: Could no
jfedu.net/pluginfacts: SSL_connect returned=1 errno=0 state=SSLv3 r
certificate is not yet valid for /CN=Puppet CA: 192-168-149-128-jfe
Info: Retrieving plugin
Error: /File[/var/lib/puppet/lib]: Failed to generate additional re
errno=0 state=SSLv3 read server certificate B: certificate verify f
: 192-168-149-128-jfedu.net]
Error: /File[/var/lib/puppet/lib]: Could not evaluate: Could not re
```

图 20-6 Puppet 客户端同步服务端配置

报错原因是因为服务器端与客户端时间不同步导致，需要同步时间，然后再次执行

puppet agent 命令，如图 20-7 所示：

```
ntpdate  pool.ntp.org

puppet  agent  --server=192-168-149-128-jfedu.net  --test
```

```
[root@192-168-149-130-jfedu tmp]# puppet  agent  --server=192-168-149-
Info: Caching certificate_revocation_list for ca
Warning: Unable to fetch my node definition, but the agent run will co
Warning: undefined method `include?' for nil:NilClass
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496805041'
Notice: /Stage[main]/Main/Node[default]/File[/tmp/test.txt]/ensure: de
38d9'
Info: Creating state file /var/lib/puppet/state/state.yaml
Notice: Finished catalog run in 0.04 seconds
[root@192-168-149-130-jfedu tmp]#
[root@192-168-149-130-jfedu tmp]# ls
test.txt
[root@192-168-149-130-jfedu tmp]# cat test.txt
Hello World, jfedu.net  2017[root@192-168-149-130-jfedu tmp]#
```

图 20-7 Puppet 客户端获取服务端 node 配置

Puppet 客户端执行同步效果，执行日志如下，会在/tmp/目录创建 test.txt 文件，

内容为：“Hello World ,jfedu.net”,即证明 Puppet 客户端成功获取服务端 node 配置。

```
Info: Caching certificate_revocation_list for ca

Warning: Unable to fetch my node definition, but the agent run will continue:

Warning: undefined method `include?' for nil:NilClass

Info: Retrieving pluginfacts

Info: Retrieving plugin

Info: Caching catalog for 192-168-149-130-jfedu.net

Info: Applying configuration version '1496805041'

Notice: /Stage[main]/Main/Node[default]/File[/tmp/test.txt]/ensure: defined

content as '{md5}d1c2906ad0b249a330e936e3bc1d38d9'

Info: Creating state file /var/lib/puppet/state/state.yaml

Notice: Finished catalog run in 0.04 seconds
```

21.5 Puppet 常见资源及模块

Puppet 主要基于各种资源模块管理客户端 ,目前企业主流 Puppet 管理客户端资源模块如下：

file	主要负责管理文件；
package	软件包的安装管理；
service	系统服务的管理；

cron	配置自动任务计划；
exec	远程执行运行命令。

通过命令 puppet describe -l 可以查看 puppet 支持的所有资源和模块，如图 20-8（a）、20-8（b）所示：

```
[root@192-168-149-128-jfedu ~]# puppet describe -l
These are the types known to puppet:
augeas           - Apply a change or an array of changes to the  ...
computer         - Computer object management using DirectorySer ...
cron             - Installs and manages cron jobs
exec             - Executes external commands
file             - Manages files, including their content, owner ...
filebucket       - A repository for storing and retrieving file  ...
group            - Manage groups
host             - Installs and manages host entries
interface        - This represents a router or switch interface
k5login          - Manage the `.k5login` file for a user
macauthorization - Manage the Mac OS X authorization database
mailalias        - .. no documentation ..
maillist         - Manage email lists
```

图 20-8（a） Puppet 支持的资源及模块

```
router           - .. no documentation ..
schedule         - Define schedules for Puppet
scheduled_task   - Installs and manages Windows Scheduled Tasks
selboolean       - Manages SELinux booleans on systems with SELi ...
selmodule        - Manages loading and unloading of SELinux poli ...
service          - Manage running services
ssh_authorized_key - Manages SSH authorized keys
sshkey           - Installs and manages ssh host keys
stage            - A resource type for creating new run stages
tidy             - Remove unwanted files based on specific crite ...
user             - Manage users
vlan             - .. no documentation ..
whit             - Whits are internal artifacts of Puppet's curr ...
yumrepo          - The client-side description of a yum reposito ...
zfs              - Manage zfs
zone             - Manages Solaris zones
```

图 20-8（b） Puppet 支持的资源及模块

通过命令 puppet describe -s file 可以查看 puppet file 资源所有的帮助信息，如图 20-9（a）、20-9（b）所示：

```
[root@192-168-149-128-jfedu ~]# puppet describe -s file

file
====
Manages files, including their content, ownership, and permissions.
The `file` type can manage normal files, directories, and symlinks;
type should be specified in the `ensure` attribute.
File contents can be managed directly with the `content` attribute,
downloaded from a remote source using the `source` attribute; the la
can also be used to recursively serve directories (when the `recurse
attribute is set to `true` or `local`). On Windows, note that file
contents are managed in binary mode; Puppet never automatically tran
line endings.
**Autorequires:** If Puppet is managing the user or group that owns
file, the file resource will autorequire them. If Puppet is managing
parent directories of a file, the file resource will autorequire the
```

图 20-9（a） Puppet file 资源模块详情

```
**Autorequires:** If Puppet is managing the user or group that owns a
file, the file resource will autorequire them. If Puppet is managing any
parent directories of a file, the file resource will autorequire them.


Parameters
----------
    backup, checksum, content, ctime, ensure, force, group, ignore, links
    mode, mtime, owner, path, purge, recurse, recurselimit, replace,
    selinux_ignore_defaults, selrange, selrole, seltype, seluser, show_di
    source, source_permissions, sourceselect, target, type, validate_cmd,
    validate_replacement

Providers
---------
    posix, windows
```

图 20-9（b） Puppet file 资源模块详情

21.6 Puppet file 资源案例

Puppet file 资源主要用于管理客户端文件，包括文件的内容、所有权和权限，其可管理的文件类型包括：普通文件、目录以及符号链接等。

类型应在“确保”属性中指定，如果是文件内容可以直接用'content 属性来管理，或者使用 source 属性从远程源下载；后者也可以用 recurse 服务目录（当 recurse 属性设置为“true”或“local”。Puppet file 资源支持参数如下：

ensure	默认为文件或目录；
backup	通过 filebucket 备份文件；
checksum	检查文件是否被修改的方法；
ctime	只读属性，文件的更新时间；
mtime	只读属性，文件的修改时间；
content	文件的内容，与 source 和 target 互斥 ；
force	强制执行删除文件、软链接机目录的操作；
owner	用户名或用户 ID；
group	指定文加年的用户组或组 id；
link	软链接；
mode	文件权限配置，通常采用数字符号；
path	文件路径；

Parameters

backup, checksum, content, ctime, ensure, force, group, ignore, links, mode, mtime, owner, path, purge, recurse, recurselimit, replace, selinux_ignore_defaults, selrange, selrole, seltype, seluser, show_diff, source, source_permissions, sourceselect, target, type, validate_cmd, validate_replacement

Providers：

```
    posix, windows
```

（1） 从 Puppet 服务器下载 nginx.conf 文件至客户端/tmp 目录，首先需要将 nginx.conf 文件 cp 至/etc/puppet/files 目录 然后在/etc/puppet/fileserver.conf 中添加如下三行代码，并重启 Puppetmaster 即可：

```
[files]
path    /etc/puppet/files/
allow *
```

创建 site.pp 文件，文件代码如下：

```
node    default {
file    {
            '/tmp/nginx.conf':
            mode => '644',
            owner => 'root',
            group => 'root',
            source => 'puppet://192-168-149-128-jfedu.net/files/nginx.conf',
    }
}
```

客户端同步配置，如图 20-10 所示：

```
[root@192-168-149-130-jfedu tmp]#
[root@192-168-149-130-jfedu tmp]# puppet  agent  --server=192-168-
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496972457'
Notice: /Stage[main]/Main/Node[default]/File[/tmp/nginx.conf]/ensu
md5}349368d93dc27680f546b5a4991aa2ff'
Notice: Finished catalog run in 0.22 seconds
[root@192-168-149-130-jfedu tmp]# ll
total 8
-rw-r--r-- 1 root root 2650 Jun  9 09:40 nginx.conf
-rw-r--r-- 1 root root   29 Jun  9 08:58 test.txt
[root@192-168-149-130-jfedu tmp]#
```

图 20-10 Puppet file 资源远程下载文件

（2） 从 Puppet 服务器下载 sysctl.conf，如果客户端该文件存在则备份为 sysctl.conf.bak，然后再覆盖原文件，site.pp 代码如下，如图 20-11 所示：

```
node   default {

file {

          "/etc/sysctl.conf":

          source => "puppet://192-168-149-128-jfedu.net/files/sysctl.conf",

          backup => ".bak_$uptime_seconds",

   }

}
```

```
[root@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496973190'
Notice: /Stage[main]/Main/Node[default]/File[/etc/sysctl.conf]/conte
--- /etc/sysctl.conf    2017-05-26 20:48:35.582406831 +0800
+++ /tmp/puppet-file20170609-4129-1mi9ifq-0     2017-06-09 09:53:10.
@@ -1,30 +1,33 @@
 net.ipv4.ip_forward = 0
-
-# Controls source route verification
 net.ipv4.conf.default.rp_filter = 1
-
-# Do not accept source routing
 net.ipv4.conf.default.accept_source_route = 0
```

图 20-11 Puppet file 资源备份文件

（3） 在 Agent 上创建/export/docker 的软连接为/var/lib/docker/，site.pp 代码如下，如图 20-12 所示：

```
node   default {

file {

          "/var/lib/docker":

          ensure => link,

          target => "/export/docker",

  }
```

```
@192-168-149-130-jfedu ~]#
@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-128-jfedu.
 Retrieving pluginfacts
 Retrieving plugin
 Caching catalog for 192-168-149-130-jfedu.net
 Applying configuration version '1496974040'
e: /Stage[main]/Main/Node[default]/File[/var/lib/docker]/ensure: created
e: Finished catalog run in 0.16 seconds
@192-168-149-130-jfedu ~]#
@192-168-149-130-jfedu ~]# ll /var/lib/|grep docker
wxrwx   1 root    root     14 Jun  9 10:07 docker -> /export/docker
@192-168-149-130-jfedu ~]#
@192-168-149-130-jfedu ~]#
```

图 20-12 Puppet file 资源备份文件

（4） 在 Agent 上创建目录/tmp/20501212，site.pp 代码如下，如图 20-13 所示：

```
node   default {

file {

          "/tmp/20501212":

          ensure => directory;

          }

}
```

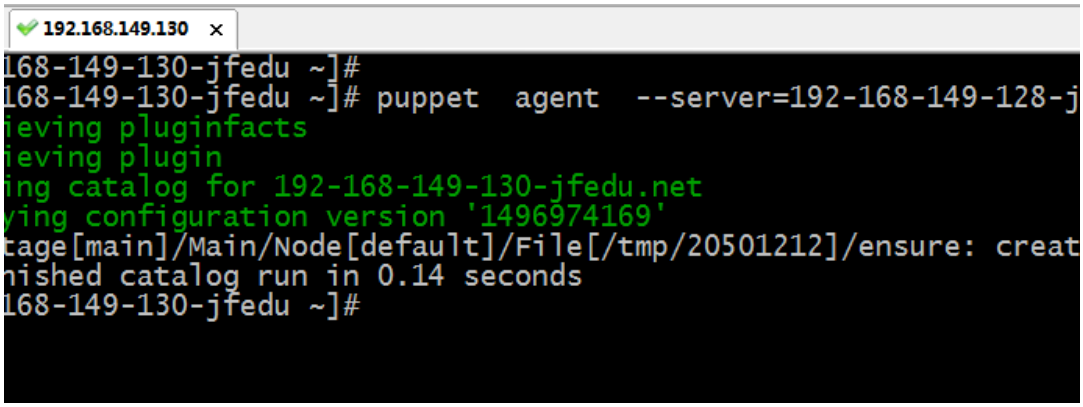


图 20-13 Puppet file 创建目录

21.7 Puppet package 资源案例

Puppet package 资源主要用于管理客户端服务器的软件包，yum 源为/etc/yum.repo.d/安装和升级操作，通过 puppet 基于 yum 自动安装软件包，所以需要先配置好 yum 源。

常见的操作可以对软件包进行安装、卸载以及升级操作。Puppet package 资源支持参数如下：

```
Parameters

    adminfile, allow_virtual, allowcdrom, category, configfiles,

    description, ensure, flavor, install_options, instance, name,

    package_settings, platform, responsefile, root, source, status,

    uninstall_options, vendor

Providers

    aix, appdmg, apple, apt, aptitude, aptrpm, blastwave, dpkg, fink,

    freebsd, gem, hpux, macports, msi, nim, openbsd, opkg, pacman, pip, pkg,

    pkgdmg, pkgin, pkgutil, portage, ports, portupgrade, rpm, rug, sun,

    sunfreeware, up2date, urpmi, windows, yum, zipper

ensure => {installed|absent|pureged|latest}

present                 检查软件是否存在，不存在则安装；

installed               表示安装软件；

absent                  删除（无依赖），当别的软件包依赖时，不可删除；

pureged                 删除所有配置文件和依赖包，有潜在风险，慎用；

latest                  升级到最新版本；

version                     指定安装具体的某个版本号。
```

（1） 客户端安装 ntpdate 及 screen 软件，代码如下，执行结果如图 20-14 所示：

```
node   default {
```

```
package {

 ["screen","ntp"]:

 ensure => "installed";

}
```

```
[root@192-168-149-130-jfedu ~]# ntpdate
-bash: /usr/sbin/ntpdate: No such file or directory
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-128-
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496975410'
Notice: /Stage[main]/Main/Node[default]/Package[screen]/ensure: created
Notice: /Stage[main]/Main/Node[default]/Package[ntpdate]/ensure: created
Notice: Finished catalog run in 4.32 seconds
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]# ntpdate
 9 Jun 10:30:55 ntpdate[6044]: no servers can be used, exiting
```

图 20-14 Puppet package 安装软件

（2） 客户端卸载 ntpdate 及 screen 软件，代码如下，执行结果如图 20-15 所示：

```
node   default {

package {

 ["screen","ntp"]:

 ensure => "absent";

}
```

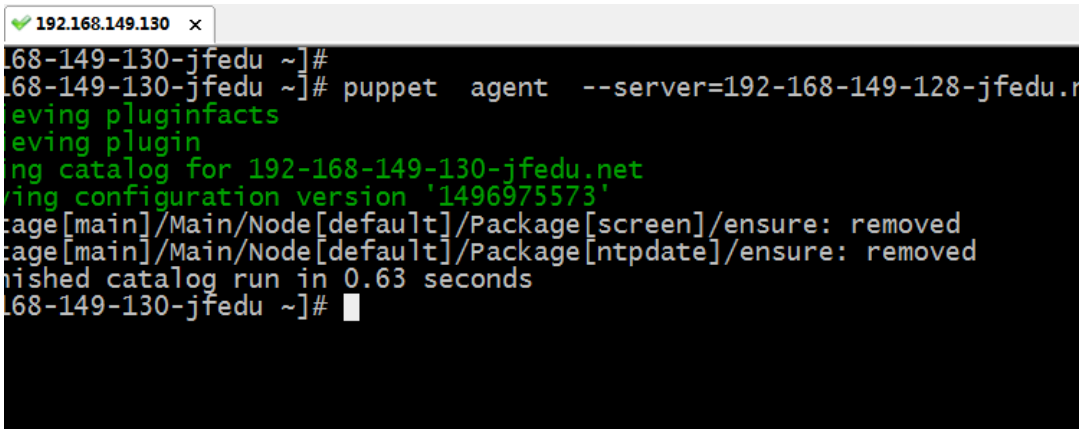


图 20-15 Puppet package 卸载软件

21.8 Puppet service 资源案例

Puppet service 资源主要用于启动、重启和关闭客户端的守护进程，同时可以监控进程的状态，还可以将守护进程加入到自启动中。Puppet service 资源支持参数如下：

Parameters binary, control, enable, ensure, flags, hasrestart, hasstatus, manifest, name, path, pattern, restart, start, status, stop	
Providers base, bsd, daemontools, debian, freebsd, gentoo, init, launchd, openbsd, openrc, openwrt, redhat, runit, service, smf, src, systemd, upstart, windows	
enable	指定服务在开机的时候是否启动，可以设置 true 和 false；

ensure	是否运行服务，running 表示运行，stopped 表示停止服务；
name	守护进程的名字；
path	启动脚本搜索路径；
provider	默认为 init；
hasrestart	管理脚本是否支持 restart 参数,如果不支持,就用 stop 和 start 实现 restart 效果；
hasstatus	管理脚本是否支持 status 参数，puppet 用 status 参数来判断服务是否已经在运行了,如果不支持 status 参数,puppet 利用查找运行进程列表里面是否有服务名来判断服务是否在运行。

（1） 启动 Agent httpd 服务，停止 nfs 服务，代码如下，结果如图 20-16（a）、20-16（b）所示：

```
node   default {
service {
          "httpd":
          ensure => running;
          "nfs":
          ensure => stopped;
  }
```

```
root      6399     2  0 11:10 ?        00:00:00 [nfsd]
root      6400     2  0 11:10 ?        00:00:00 [nfsd]
root      6459  1061  0 11:10 pts/0    00:00:00 grep -E httpd|nfs
[root@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-128-jfe
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496977872'
Notice: /Stage[main]/Main/Node[default]/Service[nfs]/ensure: ensure changed
topped'
Notice: /Stage[main]/Main/Node[default]/Service[httpd]/ensure: ensure chang
'running'
Info: /Stage[main]/Main/Node[default]/Service[httpd]: Unscheduling refresh
]
Notice: Finished catalog run in 1.37 seconds
```

图 20-16（a） Puppet service 重启服务

```
[root@192-168-149-130-jfedu ~]# ps -ef |grep http
root      6725     1  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6737  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6738  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6739  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6740  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6741  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6742  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6743  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
apache    6744  6725  0 11:11 ?        00:00:00 /usr/sbin/httpd
root      6756  1061  0 11:13 pts/0    00:00:00 grep http
[root@192-168-149-130-jfedu ~]# ps -ef |grep nfs
root      6761  1061  0 11:13 pts/0    00:00:00 grep nfs
[root@192-168-149-130-jfedu ~]#
```

图 20-16（b） Puppet service 重启服务

（2） 启动 Agent httpd 服务并且开启启动，停止 nfs 服务，开启不启动，代码如下，结果如图 20-17（a）、20-17（b）所示：

```
node   default {

service {

        "httpd":

        ensure => running,

        enable => true;

        "nfs":
```

```
        ensure => stopped,

        enable => false;

}
```

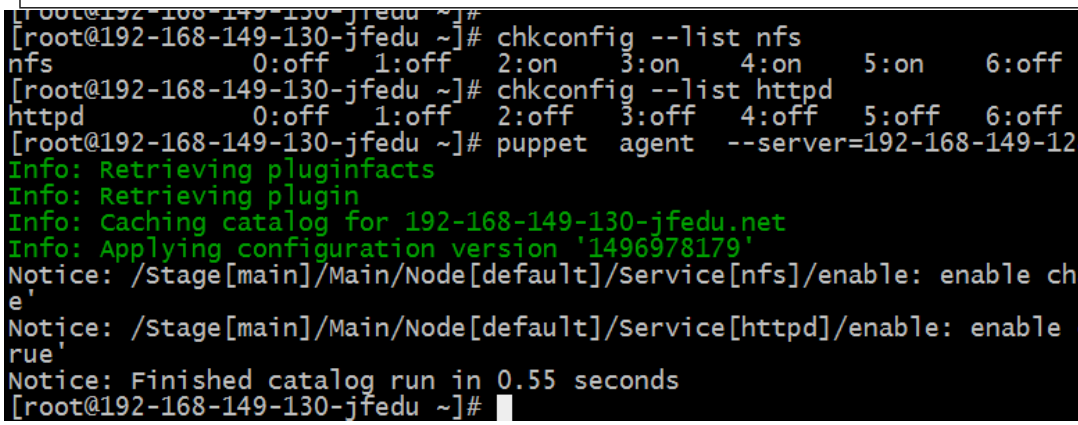


图 20-17（a） Puppet service 开启启动

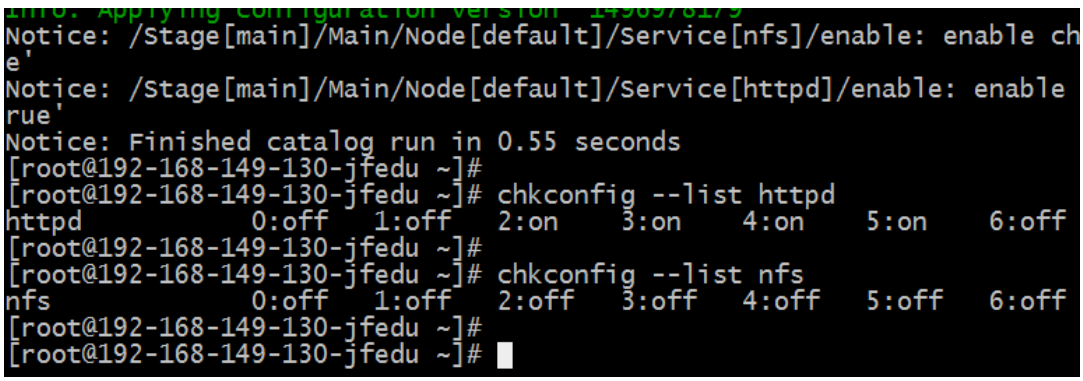


图 20-17（b） Puppet service 开启启动

21.9 Puppet exec 资源案例

Puppet exec 资源主要用于客户端远程执行命令或者软件安装等 ,相当于 shell 的调用，exec 是一次性执行资源，在不同类里面 exec 名字可以相同。Puppet exec 资源支持参数如下：

Parameters

command, creates, cwd, environment, group, logoutput, onlyif, path,

refresh, refreshonly, returns, timeout, tries, try_sleep, umask, unless,

user

Providers

posix, shell, windows

command	指定要执行的系统命令；
creates	指定命令所生成的文件；
cwd	指定命令执行目录，如果目录不存在，则命令执行失败；
group	执行命令运行的账户组；
logoutput	是否记录输出；
onlyif	exec 只会在 onlyif 设定的命令返回 0 时才执行；
path	命令执行的搜索路径；
refresh =>true|false	刷新命令执行状态；
refreshonly =>true|false	该属性可以使命令变成仅刷新触发的，
returns	指定返回的代码；
timeout	命令运行的最长时间；
tries	命令执行重试次数，默认为 1；
try_sleep	设置命令重试的间隔时间，单位为秒；

user	指定执行命令的账户；
provider	shell 和 windows；
environment	为命令设定额外的环境变量；要注意的是如果设定 PATH，PATH 的属性会被覆盖。

（1） Agent 服务器执行 tar 解压 nginx 软件包，代码如下，结果如图 20-18 所示：

```
node   default {
exec {
      'Agent tar xzf nginx-1.12.0.tar.gz':
      path => ["/usr/bin","/bin"],
      user => 'root',
      group => 'root',
      timeout => '10',
      command => 'tar -xzf /tmp/nginx-1.12.0.tar.gz',
}
}
```

```
[root@192-168-149-130-jfedu tmp]# ll /var/log/nginx.log
ls: cannot access /var/log/nginx.log: No such file or directory
[root@192-168-149-130-jfedu tmp]#
[root@192-168-149-130-jfedu tmp]# puppet  agent  --server=192-168-149-128-jf
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496979312'
Notice: /Stage[main]/Main/Node[default]/Exec[Agent tar xzf nginx-1.6.2.tar.g
cuted successfully
Notice: Finished catalog run in 0.65 seconds
[root@192-168-149-130-jfedu tmp]# ls
20170808  20501212  nginx-1.6.2  nginx-1.6.2.tar.gz  nginx.conf  test.txt
[root@192-168-149-130-jfedu tmp]#
```

图 20-18 Puppet exec 远程执行命令

（2） Agent 服务器远程执行 auto_install_nginx.sh 脚本 ,代码如下 结果如图 20-19 所示：

```
node   default {

file {

          "/tmp/auto_install_nginx.sh":

          source                                          =>"puppet://192-168-149-128-

jfedu.net/files/auto_install_nginx.sh",

          owner => "root",

          group => "root",

          mode => 755,

     }

exec {

          "/tmp/auto_install_nginx.sh":
```

```
        cwd => "/tmp",

        user => root,

        path => ["/usr/bin","/usr/sbin","/bin","/bin/sh"],

}
```

```
[root@192-168-149-130-jfedu tmp]#
[root@192-168-149-130-jfedu tmp]# ls /usr/local/nginx/
ls: cannot access /usr/local/nginx/: No such file or directory
[root@192-168-149-130-jfedu tmp]#
[root@192-168-149-130-jfedu tmp]# puppet  agent  --server=192-168-1
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496980772'
Notice: /Stage[main]/Main/Node[default]/Exec[/tmp/auto_install_ngin
Notice: Finished catalog run in 54.97 seconds
[root@192-168-149-130-jfedu tmp]# ls /usr/local/nginx/
conf  html  logs  sbin
[root@192-168-149-130-jfedu tmp]#
```

图 20-19　Puppet exec 执行 Nginx 安装脚本

（3）　Agent 服务器更新 sysctl.conf，如果该文件发生改变，则执行命令 sysctl -p，代码如下，结果如图 20-20（a）、20-20（b）所示：

```
node   default {

file {

        "/etc/sysctl.conf":

        source =>"puppet://192-168-149-128-jfedu.net/files/sysctl.conf",

        owner => "root",

        group => "root",

        mode => 644,
```

```
    }

exec {

        "sysctl refresh kernel config":

        path => ["/usr/bin", "/usr/sbin", "/bin", "/sbin"],

        command   => "/sbin/sysctl -p",

        subscribe => File["/etc/sysctl.conf"],

        refreshonly => true

    }

}
```

```
[root@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-128
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496979816'
Notice: /Stage[main]/Main/Node[default]/Exec[Agent tar xzf nginx-1.6.2.
Notice: /Stage[main]/Main/Node[default]/File[/etc/sysctl.conf]/content:
--- /etc/sysctl.conf    2017-06-09 11:44:13.357579239 +0800
+++ /tmp/puppet-file20170609-7827-kmfu63-0      2017-06-09 11:44:19.732
@@ -1,3 +1,4 @@
+net.ipv4.ip_forward = 0
 net.ipv4.conf.default.rp_filter = 1
 net.ipv4.conf.default.accept_source_route = 0
 kernel.sysrq = 0
```

图 20-20（a） Puppet exec 更新执行触发命令

```
Info: Computing checksum on file /etc/sysctl.conf
Info: FileBucket got a duplicate file {md5}f6839b49be2828e7416c50f3e5aa
Info: /Stage[main]/Main/Node[default]/File[/etc/sysctl.conf]: Filebucket
2828e7416c50f3e5aa67c6
Notice: /Stage[main]/Main/Node[default]/File[/etc/sysctl.conf]/content:
a67c6' to '{md5}2a159a404fb0e33557d83bae50c467f0'
Info: /Stage[main]/Main/Node[default]/File[/etc/sysctl.conf]: Scheduling
Notice: /Stage[main]/Main/Node[default]/Exec[sysctl refresh kernel conf
Notice: Finished catalog run in 0.87 seconds
[root@192-168-149-130-jfedu ~]#
```

图 20-20（b） Puppet exec 更新执行触发命令

21.10 Puppet cron 资源案例

Puppet cron 资源主要用于安装和管理 crontab 计划任务，每一个 cron 资源需要一个 command 属性和 user 属性以及至少一个周期属性（hour，minute，month，monthday，weekday）。

Crontab 计划任务的名字不是计划任务的一部分，它是 puppet 用来存储和检索该资源。假如你指定了一个除了名字其他的都和一个已经存在的计划任务相同，那么这两个计划任务被认为是等效的，并且新名字将会永久地与该计划任务相关联。Puppet cron 资源支持参数如下：

Parameters	
command, ensure, environment, hour, minute, month, monthday, name, special, target, user, weekday	
Providers	
crontab	
user	加某个用户的 crontab 任务，默认是运行 puppet 的用户；
command	要执行的命令或脚本路径，可不写，默认是 title 名称；
ensure	定该资源是否启用，可设置成 true 或 false；
environment	crontab 环境里面指定环境变量；
hour	设置 crontab 的小时，可设置成 0-23；

minute	指定 crontab 的分钟，可设置成 0-59；
month	指定 crontab 运行的月份，1-12；
monthday	指定月的天数 1-31；
name	crontab 的名字，区分不同的 crontab；
provider	可用的 provider 有 crontab 默认的 crontab 程序；
target	crontab 作业存放的位置；
weekday	行 crontab 的星期数，0-7，周日是为 0。

（1） Agent 服务器添加 ntpdate 时间同步任务，代码如下，结果如图 20-21 所示：

```
node   default {
cron{
          "ntpdate":
          command => "/usr/sbin/ntpdate   pool.ntp.org",
          user => root,
          hour => 0,
          minute => 0,
  }
}
```

```
[root@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-1
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496985201'
Notice: /Stage[main]/Main/Node[default]/Cron[ntpdate]/ensure: created
Notice: Finished catalog run in 0.46 seconds
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]# crontab  -l
# HEADER: This file was autogenerated at Fri Jun 09 13:13:22 +0800 20
# HEADER: While it can still be managed manually, it is definitely no
# HEADER: Note particularly that the comments starting with 'Puppet N
# HEADER: not be deleted, as doing so could cause duplicate cron jobs
# Puppet Name: ntpdate
0 0 * * * /usr/sbin/ntpdate pool.ntp.org
[root@192-168-149-130-jfedu ~]#
```

图 20-21 Puppet cron 创建任务计划

（2） Agent 服务器删除 ntpdate 时间同步任务，代码如下，结果如图 20-22 所示：

```
node   default {

cron{

          "ntpdate":

          command => "/usr/sbin/ntpdate   pool.ntp.org",

          user => root,

          hour => 0,

          minute => 0,

          ensure => absent,

  }

}
```

```
# HEADER: not be deleted, as doing so could cause duplicate cron jobs.
# Puppet Name: ntpdate
0 0 * * * /usr/sbin/ntpdate pool.ntp.org
[root@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-128-j
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1496988132'
Notice: /Stage[main]/Main/Node[default]/Cron[ntpdate]/ensure: removed
Notice: Finished catalog run in 0.53 seconds
[root@192-168-149-130-jfedu ~]# crontab  -l
# HEADER: This file was autogenerated at Fri Jun 09 14:02:13 +0800 2017 b
# HEADER: While it can still be managed manually, it is definitely not re
# HEADER: Note particularly that the comments starting with 'Puppet Name'
# HEADER: not be deleted, as doing so could cause duplicate cron jobs.

[root@192-168-149-130-jfedu ~]#
```

图 20-22 Puppet cron 删除任务计划

21.11 Puppet 日常管理与配置

Puppet 平台构建完毕，能够简单实用 Puppet 去管理客户端，对文件、服务、脚本、各种配置的变更，如果要管理批量服务器，还需要进行一些步骤的配置。

21.11.1 Puppet 自动认证

企业新服务器通过 kickstart 自动安装 Linux 操作系统，安装完毕，可以自动安装 puppet 相关软件包，Puppet 客户端安装完毕，需向 Puppet 服务端请求证书，然后 Puppet 服务端颁发证书给客户端，默认需要手动颁发，可以通过配置让 Puppet 服务端自动颁发证书。

自动颁发证书前提是服务端与客户端能 ping 通彼此的主机名方可，配置自动颁发证书需 Puppet 服务器端的 puppet.conf 配置文件 main 段加入如下代码，如图 20-23 所示：

```
[main]
```

```
autosign = true
```

```
[root@192-168-149-128-jfedu ~]# vim /etc/puppet/puppet.conf
########
[main]
    # The Puppet log directory.
    # The default value is '$vardir/log'.
    logdir = /var/log/puppet

    # Where Puppet PID files are kept.
    # The default value is '$vardir/run'.
    rundir = /var/run/puppet

    autosign = true

    # Where SSL certificates are kept.
    # The default value is '$confdir/ssl'.
```

图 20-23 Puppet 服务端添加自动颁发证书

重启 puppetmaster 服务，并且删除 192.168.149.130 证书：

```
/etc/init.d/puppetmaster  restart

puppet  cert  --clean  192-168-149-130-jfedu.net
```

删除 Puppet 客户端 SSL 文件，重新生成 SSL 文件，执行如下命令自动申请证书：

```
rm  -rf  /var/lib/puppet/ssl/

puppet  agent  --server=192-168-149-128-jfedu.net  --test
```

Puppet 服务端会自动认证 ,即服务器端不必手动颁发证书 ,减轻人工的干预和操作，

如图 20-24 所示：

```
[root@192-168-149-130-jfedu ~]# puppet  agent  --server=192-168-149-128-jfe
Info: Creating a new SSL key for 192-168-149-130-jfedu.net
Info: Caching certificate for ca
Info: csr_attributes file loading from /etc/puppet/csr_attributes.yaml
Info: Creating a new SSL certificate request for 192-168-149-130-jfedu.net
Info: Certificate Request fingerprint (SHA256): 65:F6:C0:E9:26:DF:91:5C:33
2:D5:FD:19:10:D9:EC:25
Info: Caching certificate for 192-168-149-130-jfedu.net
Info: Caching certificate_revocation_list for ca
Info: Caching certificate for ca
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Caching catalog for 192-168-149-130-jfedu.net
Info: Applying configuration version '1497064668'
Notice: Finished catalog run in 0.08 seconds
[root@192-168-149-130-jfedu ~]#
```

图 20-24　Puppet 客户端自动获取证书

21.11.2 Puppet 客户端自动同步

Puppet 客户端安装完 ,并且认证完之后 ,如果在 Puppet 服务端配置了 node 信息，客户端启动服务，默认 30 分钟自动与服务端同步信息，如何修改同步的时间频率呢，修改 puppet 客户端配置信息即可：

Puppet 客户端配置 puppet 相关参数和同步时间 ,修改/etc/sysconfig/puppet 配置文件，最终代码如下：

```
# The puppetmaster server

PUPPET_SERVER=192-168-149-128-jfedu.net

# If you wish to specify the port to connect to do so here

PUPPET_PORT=8140

# Where to log to. Specify syslog to send log messages to the system log.

PUPPET_LOG=/var/log/puppet/puppet.log

# You may specify other parameters to the puppet client here
```

```
PUPPET_EXTRA_OPTS=--waitforcert=500
```

/etc/sysconfig/puppet 配置文件参数详解：

```
PUPPET_SERVER=192-168-149-128-jfedu.net     指定 Puppet master 主机名；

PUPPET_PORT=8140                            指定 puppet master 端口；

PUPPET_LOG=/var/log/puppet/puppet.log       puppet 客户端日志路径；

PUPPET_EXTRA_OPTS=--waitforcert=500     获取 Puppet master 证书返回等待时间。
```

重启 puppet 客户端服务，户端会半个小时跟服务器同步一次配置信息：

```
/etc/init.d/puppet    restart
```

可以修改与服务器端同步配置信息的时间 ,修改 vi /etc/puppet/puppet.conf 文件，在[agent]段加入如下语句，表示 60 秒于 puppet master 同步一次配置信息，重启 puppet，同步结果如图 20-25 所示：

```
[agent]

runinterval = 60
```

```
[root@192-168-149-130-jfedu tmp]# /etc/init.d/puppet restart
Stopping puppet agent:                                     [FA
Starting puppet agent:                                     [
[root@192-168-149-130-jfedu tmp]#
[root@192-168-149-130-jfedu tmp]# tail -fn 100 /var/log/puppet
Sat Jun 10 11:36:01 +0800 2017 Puppet (notice): Reopening log
Sat Jun 10 11:36:01 +0800 2017 Puppet (notice): Starting Puppe
Sat Jun 10 11:36:02 +0800 2017 Puppet (notice): Finished catal
--------------------
Sat Jun 10 11:37:02 +0800 2017 /Stage[main]/Main/Node[default]
'{md5}349368d93dc27680f546b5a4991aa2ff'
Sat Jun 10 11:37:02 +0800 2017 Puppet (notice): Finished catal
--------------------
Sat Jun 10 11:38:02 +0800 2017 /Stage[main]/Main/Node[default]
ed '0755' to '0644'
Sat Jun 10 11:38:02 +0800 2017 Puppet (notice): Finished catal
```

图 20-25 Puppet 客户端自动同步服务端配置

21.11.3 Puppet 服务端主动推送

如上 puppet 客户端配置每 60 秒与服务端同步配置信息 ,如果服务器端更新了配置信息 ,想立刻让客户端同步 ,如何通知客户端来获取最新的配置信息呢 ,可以使用 Puppet master 主动推送的方式，让客户端更新服务端最新的配置信息。

Puppet 服务器端使用 puppet run 命令可以给客户端发送一段信号，告诉客户端立刻跟服务器同步配置信息，配置方法如下：

修改 Puppet 客户端配置文件/etc/puppet/puppet.conf，在 agent 段加入如下代码：

```
[agent]

listen = true
```

修改 Puppet 客户端配置文件/etc/sysconfig/puppet 指定 puppet master 端主机名：

```
PUPPET_SERVER=192-168-149-128-jfedu.net
```

创建 Puppet 客户端配置文件 namespaceauth.conf，写入如下代码：

```
[puppetrunner]

allow *
```

修改 Puppet 客户端配置文件 auth.conf，在 path /前添加如下代码：

```
path    /run

method    save

allow    *
```

重启 Puppet 客户端： /etc/init.d/puppet restart

Puppet 服务端执行如下命令 ,通知客户端来同步配置 ,也可以批量通知其他客户端，只需将客户端的主机名写入 host.txt 文件，如图 20-26 所示：

```
puppet    kick    -d    192-168-149-130-jfedu.net

#puppet    kick    -d    'cat host.txt'
```

```
Debug: /File[/var/lib/puppet/ssl/private_keys]: Autorequiring File[/va
Debug: /File[/var/lib/puppet/state]: Autorequiring File[/var/lib/puppe
Debug: /File[/var/lib/puppet/ssl/certs]: Autorequiring File[/var/lib/p
Debug: /File[/var/lib/puppet/preview]: Autorequiring File[/var/lib/pup
Debug: /File[/var/lib/puppet/ssl]: Autorequiring File[/var/lib/puppet]
Debug: /File[/var/lib/puppet/ssl/certs/192-168-149-128-jfedu.net.pem]:
Debug: /File[/var/lib/puppet/lib]: Autorequiring File[/var/lib/puppet]
Debug: Finishing transaction 70120175045640
Debug: Creating new connection for https://192-168-149-130-jfedu.net:8
Getting status
status is success
192-168-149-130-jfedu.net finished with exit code 0
Finished
```

图 20-26 Puppet 主动通知客户端同步配置

21.12 Puppet 批量部署案例

随着 IT 行业的迅猛发展，传统的运维方式靠大量人力比较吃力，近几年自动化运维管理快速的发展 ,得到了很多 IT 运维人员的青睐 ,一个完整的自动化运维包括系统安装、配置管理、服务监控三个方面。如下为 Puppet 生成环境中应用案例。

某互联网公司新到 100 台硬件服务器，要求统一安装 Linux 系统，并部署上线以及

后期的管理配置。对于 Linux 系统安装，需采用批量安装，批量安装系统主流工具为 Kickstart 和 Cobbler，任选其一即可。

如果采用自动安装的话，我们可以自动初始化系统、内核简单优化、及常见服务、软件客户端等安装。当然 puppet 客户端也可以放在 kickstart 中安装并配置完毕。

当 Linux 操作系统安装完成后，需要对服务器进行相应的配置，方可在应对高并发网站，例如修改动态 IP 为静态 IP、安装及创建 Crontab 任务计划、同步操作系统时间、安装 Zabbix 客户端软件、优化内核参数等，可以基于 Puppet 统一调整：

21.12.1 Puppet 批量修改静态 IP 案例

现需要修改 100 台 Linux 服务器原 Dhcp 动态获取的 IP 为 Static IP 地址，首先需有修改 IP 脚本，将该脚本推送到客户端，然后执行脚本并重启网卡即可，步骤如下：

（1） 修改 IP 为静态 IP 的 Shell 脚本代码如下：

```
#!/bin/bash

#auto Change ip netmask gateway scripts

#By author jfedu.net 2022

#Define Path variables

ETHCONF=/etc/sysconfig/network-scripts/ifcfg-eth0

DIR=/data/backup/`date +%Y%m%d`

IPADDR=`ifconfig|grep inet|grep 192|head -1|cut -d: -f2|awk '{print $1}'`

NETMASK=255.255.255.0
```

```
grep dhcp    $ETHCONF

if [ $? -eq 0 ];then

            sed -i 's/dhcp/static/g' $ETHCONF

            echo   -e   "IPADDR=$IPADDR\nNETMASK=$NETMASK\nGATEWAY=`echo

$IPADDR|awk -F. '{print $1"."$2"."$3}'`.2" >>$ETHCONF

            echo "The IP configuration success. !"

            service network restart

fi
```

（2） Puppet master 执行 kick 推送配置至 Agent 服务器远程，Puppet 客户端修改 IP 脚本代码如下，结果如图 20-27（a）、20-27（b）所示：

```
node    default {

file {

            "/tmp/auto_change_ip.sh":

            source =>"puppet://192-168-149-128-jfedu.net/files/auto_change_ip.sh",

            owner => "root",

            group => "root",

            mode => 755,

      }

exec {
```

```
        "/tmp/auto_change_ip.sh":

        cwd => "/tmp/",

        user => root,

        path => ["/usr/bin","/usr/sbin","/bin","/bin/sh"],

}

}
```

```
Debug: /File[/var/lib/puppet/state]: Autorequiring File[/var/lib/pupp
Debug: /File[/var/lib/puppet/ssl/certs]: Autorequiring File[/var/lib/
Debug: /File[/var/lib/puppet/preview]: Autorequiring File[/var/lib/pu
Debug: /File[/var/lib/puppet/ssl]: Autorequiring File[/var/lib/puppet
Debug: /File[/var/lib/puppet/ssl/certs/192-168-149-128-jfedu.net.pem]
Debug: /File[/var/lib/puppet/lib]: Autorequiring File[/var/lib/puppet
Debug: /File[/var/lib/puppet/ssl/private]: Autorequiring File[/var/li
Debug: Finishing transaction 70000861822320
Debug: Creating new connection for https://192-168-149-130-jfedu.net:
Getting status
status is success
192-168-149-130-jfedu.net finished with exit code 0
Finished
[root@192-168-149-128-jfedu ~]#
```

图 20-27（a） Puppet 主动通知客户端同步配置

```
[root@192-168-149-130-jfedu tmp]# cat /etc/sysconfig/network-script
DEVICE=eth0
TYPE=Ethernet
ONBOOT=yes
NM_CONTROLLED=yes
BOOTPROTO=dhcp
[root@192-168-149-130-jfedu tmp]# cat /etc/sysconfig/network-script
DEVICE=eth0
TYPE=Ethernet
ONBOOT=yes
NM_CONTROLLED=yes
BOOTPROTO=static
IPADDR=192.168.149.130
NETMASK=255.255.255.0
GATEWAY=192.168.149.2
[root@192-168-149-130-jfedu tmp]#
```

图 20-26 Puppet 客户端 IP 自动配置为 Static 方式

21.12.2 Puppet 批量配置 NTP 同步服务器

在 100 台 Linux 服务器上配置 crontab 任务，修改 ntpdate 与 ntp 服务端同步时间，操作步骤如下：

（1） Puppet master 上创建客户端 node 配置，可以编写 NTP 模块，使用 class 可以定义模块分组，对不同业务进行分组管理，/etc/puppet/modules/ntp/manifests/init.pp 配置文件代码如下，将原 ntpdate 同步时间从 0:0 分改成每 5 分钟同步一次时间，并且修改原 pool.ntp.org 服务器为本地局域网 NTP 时间服务器的 IP 地址：

```
class   ntp {
Exec { path =>"/bin:/sbin:/bin/sh:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin"}
exec {
"auto change crontab ntp config":
command  =>"sed  -i  -e  '/ntpdate/s/0/*\/5  /2'     -e  's/pool.ntp.org/10.1.1.21/'
/var/spool/cron/root",
 }
}
```

（2） 在/etc/puppet/manifests 目录创建两个文件，分别为 modules.pp 和 nodes.pp，模块入口文件以及 node 配置段。

modules.pp 配置文件内容如下：

```
import   "ntp"
```

nodes.pp 配置文件内容如下：

```
node   default {

include   ntp

}
```

（3） 在 site.pp 中加载导入 modules.pp 和 nodes.pp 名称，site.pp 代码如下：

```
import   "modules.pp"

import   "nodes.pp"
```

（4） Puppet master 执行 kick 推送配置至 Agent 服务器远程，Puppet 客户端最终结果如图 20-28（a）、20-28（b）所示：

```
[root@192-168-149-128-jfedu manifests]# ls
modules.pp  nodes.pp  site.pp
[root@192-168-149-128-jfedu manifests]#
[root@192-168-149-128-jfedu manifests]# cat *
import  "ntp"
node  default {
        include  ntp
}
import  "modules.pp"
import  "nodes.pp"
[root@192-168-149-128-jfedu manifests]# cat /etc/puppet/modules/ntp/mani
class  ntp {
Exec { path =>"/bin:/sbin:/bin/sh:/usr/bin:/usr/sbin:/usr/local/bin:/usr

exec {
        "auto change crontab ntp config":
        command =>"sed -i -e '/ntpdate/s/0/*\/5 /2' -e 's/pool.ntp.org/1
```

图 20-28（a） Puppet 服务端 class 模块配置

```
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]# crontab -l
0 0   * * *   /usr/sbin/ntpdate   pool.ntp.org >/tmp/ntp.log 2>&1
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]# crontab -l
0 */5    * * *   /usr/sbin/ntpdate   10.1.1.21 >/tmp/ntp.log 2>&1
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]# crontab -l
0 */5    * * *   /usr/sbin/ntpdate   10.1.1.21 >/tmp/ntp.log 2>&1
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]#
[root@192-168-149-130-jfedu ~]#
```

图 20-28（b） Puppet 主动通知客户端修改 NTP 同步配置

当服务器分组之后，为了更好的管理和配置，可以使用正则表达式来进行定义 node，在定义一个 node 节点时，要指定节点的名字，并使用单引号将名字引起来，然后在大括号中指定需要应用的配置。

客户端节点名字可以是主机名也可以是客户端的正式域名，目前 Puppet 版本还不能使用通配符来指定节点，例如不能用*.jfedu.net，可以使用正则表达式，如下代码：

```
node /^Beijing-IDC-web0\d+\-jfedu\.net {

   include   ntp

}
```

如上规则会匹配所有在 jfedu.net 域并且主机名以 Beijing-IDC 开头，紧跟 web01、web02、web03...web100...等节点，由此可以进行批量服务器的分组管理。

21.12.3 Puppet 自动部署及同步网站

企业生产环境 100 台服务器，所有服务器要求要求数据一致，可以采用 rsync 同步，配置 rsync 服务器端，客户端执行脚本命令即可，同样可以使用 Puppet+Shell 脚本来

同步，这样比较快捷，也可以使用 Puppet rsync 模块。

（1） Puppet 服务器端配置，/etc/puppet/modules/www/manifests/init.pp 代码如下：

```
class www {
Exec { path =>"/bin:/sbin:/bin/sh:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin"}
file {
"/data/sh/rsync_www_client.sh":
source =>"puppet://192-9-117-162-tdt.com/files/www/rsync_www_client.sh",
owner =>"root",
group =>"root",
mode =>"755",
}
file {
"/etc/rsync.pas":
source =>"puppet://192-9-117-162-tdt.com/files/www/rsync.pas",
owner =>"root",
group =>"root",
mode =>"600",
}
```

```
exec {

"auto  backup  www  data":

command  =>"mkdir  -p  /data/backup/`date  +%Y%m%d`  ;  mv  /data/index

/data/backup/www/`date +%Y%m%d`  ;  /bin/sh /data/sh/rsync_www_client.sh ",

user =>"root",

subscribe =>File["/data/sh/rsync_www_client.sh"],

refreshonly =>"true",

 }

}
```

（2） 在/etc/puppet/manifests 目录创建两个文件，分别为 modules.pp 和 nodes.pp，模块入口文件以及 node 配置段。

modules.pp 配置文件内容如下：

```
import  "www"
```

nodes.pp 配置文件内容如下：

```
node /^Beijing-IDC-web0\d+\-jfedu\.net {

include  www

}
```

（3） 在 site.pp 中加载导入 modules.pp 和 nodes.pp 名称，site.pp 代码如下：

```
import  "modules.pp"
```

```
import   "nodes.pp"
```

Puppet master 端批量执行通知客户端来同步配置，命令如下：

```
puppet kick   -d   --host    `cat   hosts.txt`
```

（4） cat hosts.txt 内容为需要同步的客户端的主机名：

```
Beijing-IDC-web01-jfedu.net

Beijing-IDC-web02-jfedu.net

Beijing-IDC-web03-jfedu.net

Beijing-IDC-web04-jfedu.net
```

第22章 企业级 Docker 虚拟化平台实战

22.1 虚拟化技术概述及简介

IT 行业发展到今天，已经从传统技术、传统运维发展到当下的主流技术、自动化运维，未来掌握核心技术是赶上时代变化，拉开跟别人差距的最关键因素。当下主流的 IT 三大技术：虚拟化、云计算、大数据之一的虚拟化技术。虚拟化技术也越来越广泛的应用在企业中，例如百度、阿里巴巴、腾讯、京东、Google 等。

通俗的说，虚拟化就是把物理资源转变为逻辑上可以管理的资源，以打破物理结构间的壁垒，计算元件运行在虚拟的基础上而不是真实的基础上，可以扩大硬件的容量，简化软件的重新配置过程。

虚拟化技术允许一个平台同时运行多个操作系统，并且应用程序都可以在相互独立的空间内运行而互不影响，从而显著提高计算机的工作效率，是一个为了简化管理，优化资源的解决方案。

虚拟化解决方案的底部是要进行虚拟化的物理机器，这台机器可能直接支持虚拟化，也可能不会直接支持虚拟化，那么就需要系统管理程序层的支持。虚拟机管理程序(Virtual machine monitor)，或称为 VMM，可以看作是平台硬件和操作系统的抽象化，VMM 本质是我们常说的虚拟化技术软件。

22.2 互联网虚拟化技术种类

- KVM
- ESXI
- XEN
- Docker
- Hyper-V
- Open-vz

22.3 KVM 虚拟化概念

KVM 虚拟化全称为 kernel-based Virtual Machine，是一个开源的系统虚拟化模块，基于内核的虚拟机（KVM）是针对包含虚拟化扩展（Intel VT 或 AMD-V）的 x86 硬件上的 Linux 的完全原生的虚拟化解决方案。

KVM 是目前早期互联网广泛使用的虚拟化软件之一，最早由以色列的公司开发，现在已经被 RedHat 公司斥资 1.07 亿美元收购了 KVM 虚拟化管理程序厂商 Qumranet，KVM 虚拟化技术严格来讲，不是一个软件，而是 Linux 内核里面的一种加速虚拟机的功能扩展。

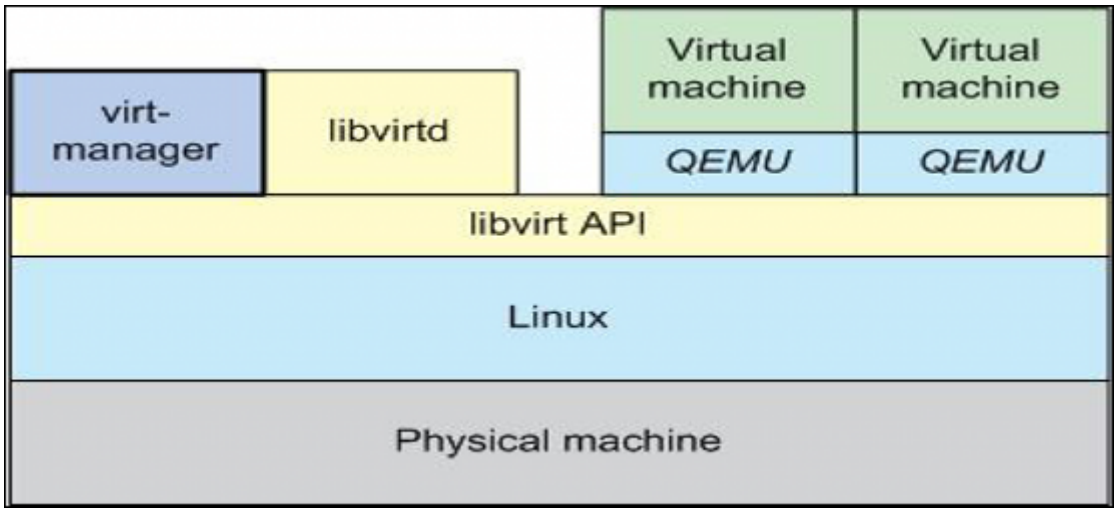
自 Linux 2.6.20 之后集成在 Linux 的各个主要发行版本中。KVM 的虚拟化需要硬件支持（如 Intel VT 技术或者 AMD V 技术)属于完全虚拟化。了解 KVM 之前，我们就需要了解 KVM 和 QEMU、Libvirt 有什么关系呢？

KVM 是一款支持虚拟机的技术，是 Linux 内核中的一个功能模块。它在 Linux2.6.20 之后的任何 Linux 分支中都被支持。它还有一个条件，对硬件要求的条件，必须达到一定标准的硬件架构。

Qemu 是什么呢。其实它也是一款虚拟化技术，就算不使用 kvm，单纯的 qemu 也可以完全实现一个虚拟机。

那为何还会有 Qemu-kvm 这个名词呢？虽然 kvm 的技术已经相当成熟而且可以对很多东西进行隔离，但是在某些方面还是无法虚拟出真实的机器。比如对网卡的虚拟，这个时候就需要另外的技术来做补充，而 qemu-kvm 则是这样一种技术。它补充了 kvm 技术的不足，而且在性能上对 kvm 进行了优化。

Libvirt 又是什么呢。它是一系列提供出来的库函数，用以其他技术调用，来管理机器上的虚拟机。包括各种虚拟机技术，kvm、xen 与 lxc 等，不同虚拟机技术就可以使用不同驱动，都可以调用 libvirt 提供的 api 对虚拟机进行管理。我们创建的各种虚拟机都是基于 libvirt 库及相关命令去管理的。



22.4 ESXI 虚拟化技术概念

Vmware 服务器虚拟化第一个产品命名为：ESX，后来 Vmware 在第 4 版本的时候推出了 ESXI，ESXI 和 ESX 的版本最大的技术区别是内核的变化。

从第 4 版本开始 VMware 把 ESX 及 ESXi 产品统称为 vSphere，但是 VMware 从 5 版本开始以后取消了原来的 ESX 版本，所以现在 VMware 虚拟化产品 vSphere 其实本质是 ESXI，只是两种叫法而已。官方称为 vSphere 虚拟化技术，个人也可以称为 ESXI 虚拟化技术。

VMware vSphere 是业界领先且最可靠的虚拟化平台。vSphere 将应用程序和操作系统从底层硬件分离出来，从而简化了 IT 操作。您现有的应用程序可以看到专有资源，而您的服务器则可以作为资源池进行管理。因此，您的业务将在简化但恢复能力极强的 IT 环境中运行。

VMware、vSphere、Essentials 和 Essentials Plus 套件专为工作负载不足 20 台服务器的 IT 环境而设计，只需极少的投资即可通过经济高效的服务器整合和业务连续性为小型企业提供企业级 IT 管理。结合使用 vSphere Essentials Plus 与 vSphere Storage Appliance 软件，无需共享存储硬件即可实现业务连续性。

VMware ESXI 虚拟化特点如下：

- 确保业务连续性和始终可用的 IT；
- 降低 IT 硬件和运营成本；
- 提高应用程序质量；
- 增强安全性和数据保护能力。

22.5 XEN 虚拟化技术概念

XEN 是一个基于 X86 架构、发展最快、性能最稳定、占用资源最少的开源虚拟化技术。Xen 可以在一套物理硬件上安全的执行多个虚拟机，与 Linux 是一个完美的开源组合，Novell SUSE Linux Enterprise Server 最先采用了 XEN 虚拟技术。它特别适用于服务器应用整合，可有效节省运营成本，提高设备利用率，最大化利用数据中心的 IT 基础架构。

实际上 XEN 出现的时间要早于 KVM 虚拟化，它是由剑桥大学开发的，严格来讲，XEN 是一个开源的虚拟机监视器，属于半虚拟化技术，其架构决定了它注定不是真正的虚拟机，只是自己运行了一个内核的例子。

XEN 虚拟化，同时区分 Xen+pv+和 Xen+hvm，其中 pv 只支持 Linux，而 hvm 则支持 Windows 系统。除此之外，XEN 还拥有更好的可用资源、平台支持、可管理性、实施、支持动态迁移和性能基准等优势。

22.6 Docker 虚拟化技术概念

Docker 是一款轻量级、高性能的虚拟化技术，是目前互联网使用最多的虚拟化技术，Docker 虚拟化技术的本质类似集装箱机制，最早集装箱没有出现的时候，码头上

有许多搬运的工人在搬运货物，集装箱出现以后，码头上看到更多的不是工人，而且集装箱的搬运模式更加单一，更加高效，还有其他的好处。

将货物多打包在集装箱里面，可以防止货物之间相互影响。并且到了另外一个码头需要转运的话，有了在集装箱以后，直接把它运送到另一个码头即可，完全可以保证里面的货物是整体的搬迁，并且不会损坏货物本身。

Docker 技术机制跟集装箱类似，Docker 虚拟化 是一个开源的应用容器引擎，让开发者可以打包他们的应用以及依赖包到一个可移植的容器中，然后发布到任何流行的 Linux 机器上，也可以实现虚拟化。

Docker 容器是完全使用沙箱机制，相互之间不会有任何接口，几乎没有性能开销，可以很容易地在机器和数据中心中运行。最重要的是，他们不依赖于任何语言、框架或包括系统。

Docker 应该是近年最火爆的技术之一，如果没有听说过，那么你就 out 了，2019 年将开启新的跨越。

Docker 自开源后受到广泛的关注和讨论，以至于 dotCloud 公司后来都改名为 Docker Inc。

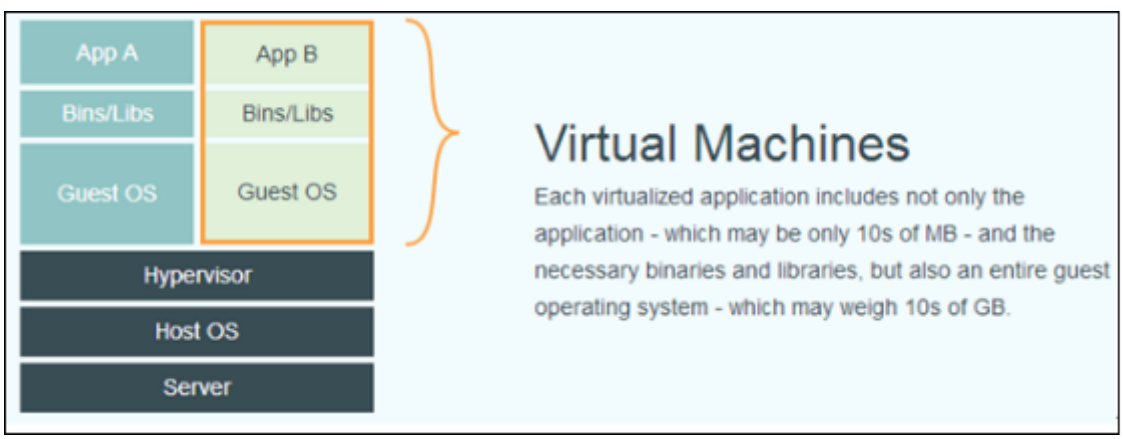
Redhat 已经在其 RHEL6.5 中集中支持 Docker；

Google 也在其 PaaS 产品中广泛应用，Docker 项目的目标是实现轻量级的操作系统虚拟化解决方案。

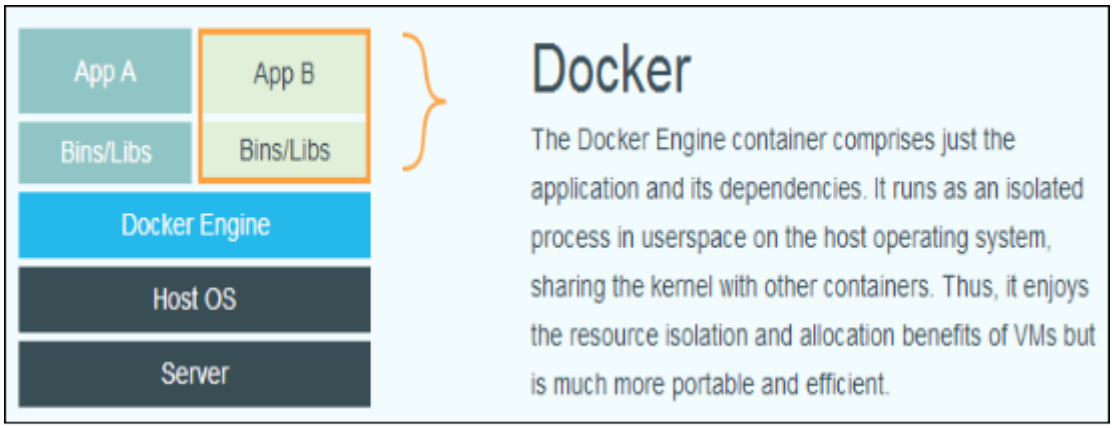
Docker 的基础是 Linux 容器（LXC）等技术。在 LXC 的基础上 Docker 进行了进一步的封装，让用户不需要去关心容器的管理，使得操作更为简便。用户操作 Docker 的容器就像操作一个快速轻量级的虚拟机一样简单。

下面对比了 Docker 和传统虚拟化（KVM、XEN、Hyper-V、ESXI）结构层级方式的不同之处，

图为传统虚拟化结构图：



图为 Docker 虚拟化结构图：



- Docker 虚拟化技术概念&总结：

Docker 虚拟化技术是在硬件的基础上，基于现有的操作系统层面上实现虚拟化，直接复用本地主机的操作系统，直接虚拟生成 Docker 容器，而 Docker 容器上部署相关的 APP 应用（Apache、MYSQL、PHP、JAVA）。

- 传统虚拟化技术概念&总结：

KVM、XEN、ESXI 传统虚拟化（完全、半虚拟化）是在硬件的基础上，基于现有的操作系统层面上实现虚拟化，但是不能复用本地主机的操作系统，而是必须虚拟出自己的 Guest OS 系统，然后在 Guest OS 系统上部署相关的 APP 应用（Apache、MYSQL、PHP、JAVA）。

Docker 虚拟化跟传统 VM 比较具有如下优点：

- 操作启动快：

运行时的性能可以获取极大提升，管理操作（启动，停止，开始，重启等等）都是以秒或毫秒为单位的。

- 轻量级虚拟化：

你会拥有足够的“操作系统”，仅需添加或减小镜像即可。在一台服务器上可以布署 100~1000 个 Containers 容器。但是传统虚拟化，你虚拟 10-20 个虚拟机就不错了。

- 开源免费：

开源的，免费的，低成本的。由现代 Linux 内核支持并驱动。注* 轻量的 Container 必定可以在一个物理机上开启更多“容器”，注定比 VMs 要便宜。

- 前景及云支持；

正在越来越受欢迎，包括各大主流公司都在推动 docker 的快速发展，性能有很大的优势。随着 Go 语言越来越被人熟知，Docker 的使用也越来越广泛。

22.7 虚拟化技术实现方式

- 完全拟化技术

通过软件实现对操作系统的资源再分配，比较成熟，完全虚拟化代表技术：

KVM、ESXI、Hyper-V；

- 半虚拟化技术

通过代码修改已有的系统，形成一种新的可虚拟化的系统，调用硬件资源去安

装多个系统，整体速度上相对高一点，半虚拟化代表技术：Xen；

- 轻量级虚拟化：

介于完全虚拟化、半虚拟化之间，轻量级虚拟化代表技术：Docker。

22.8 Docker 虚拟化特点

跟传统 VM 比较具有如下优点：

- 操作启动快

运行时的性能可以获取极大提升，管理操作（启动，停止，开始，重启等等） 都是以秒

或毫秒为单位的。

- 轻量级虚拟化

你会拥有足够的“操作系统”，仅需添加或减小镜像即可。在一台服务器上可以布署

100~1000 个 Containers 容器。但是传统虚拟化，你虚拟 10-20 个虚拟机就不错了。

- 开源免费

开源的，免费的，低成本的。由现代 Linux 内核支持并驱动。注* 轻量的 Container 必定可以在一个物理机上开启更多“容器”，注定比 VMs 要便宜。

- 前景及云支持

正在越来越受欢迎 ,包括各大主流公司都在推动 docker 的快速发展 ,性能有很大的优势。

- 跟传统 VM 比较具有如下缺点：

目前知道的人比较少；

相关的中文技术资料欠缺；

Go 语言还未完全成熟。

22.9 为什么使用 Docker

Docker 在如下几个方面具有较大的优势：

- 更快速的交付和部署

Docker 在整个开发周期都可以完美的辅助你实现快速交付。Docker 允许开发者在装有应用和服务本地容器做开发。可以直接集成到可持续开发流程中。

开发者可以使用一个标准的镜像来构建一套开发容器，开发完成之后，运维人员可以直接使用这个容器来部署代码。 Docker 可以快速创建容器，快速迭代应用程序，并让整个过程全程可见，使团队中的其他成员更容易理解应用程序是如何创建和工作的。Docker 容器很轻很快！容器的启动时间是秒级的，大量地节约开发、测试、部署的时间。

- 高效的部署和扩容

Docker 容器几乎可以在任意的平台上运行，包括物理机、虚拟机、公有云、私有云、个人电脑、服务器等。 这种兼容性可以让用户把一个应用程序从一个平台直接迁移到另外一个。

Docker 的兼容性和轻量特性可以很轻松的实现负载的动态管理。你可以快速扩容或方便的下线的你的应用和服务，这种速度趋近实时。

- 更高的资源利用率

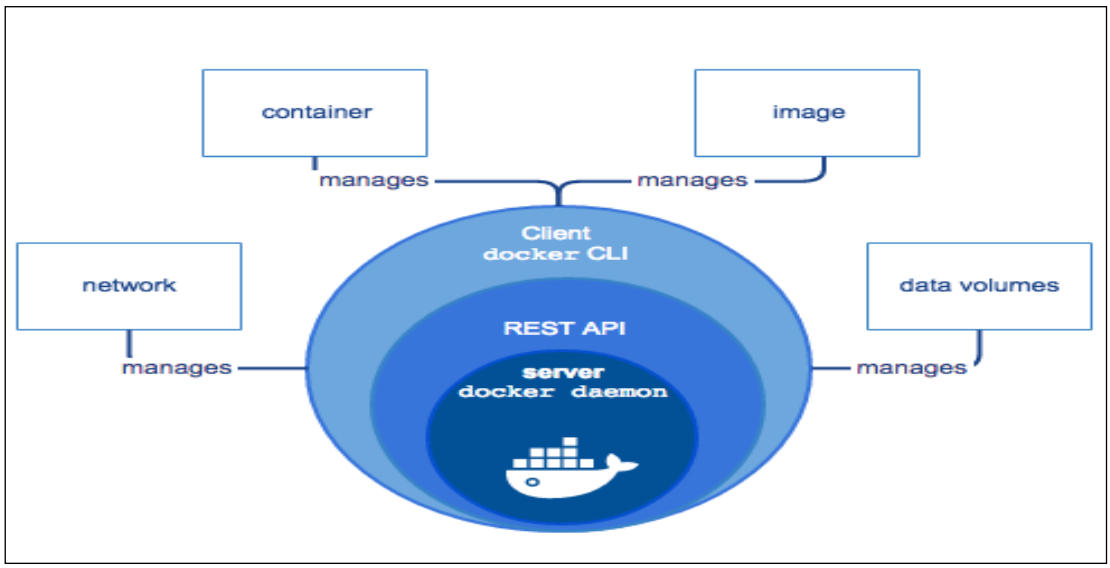
Docker 对系统资源的利用率很高，一台主机上可以同时运行数千个 Docker 容器。容器除了运行其中应用外，基本不消耗额外的系统资源，使得应用的性能很高，同时系统的开销尽量小。传统虚拟机方式运行 10 个不同的应用就要起 10 个虚拟机，而 Docker 只需要启动 10 个隔离的应用即可。

- 更简单的管理

使用 Docker，只需要小小的修改，就可以替代以往大量的更新工作。所有的修改都以增量的方式被分发和更新，从而实现自动化并且高效的管理。

22.10 Docker 引擎架构

docker 引擎是一个 C/S 结构的应用，组件如图所示：

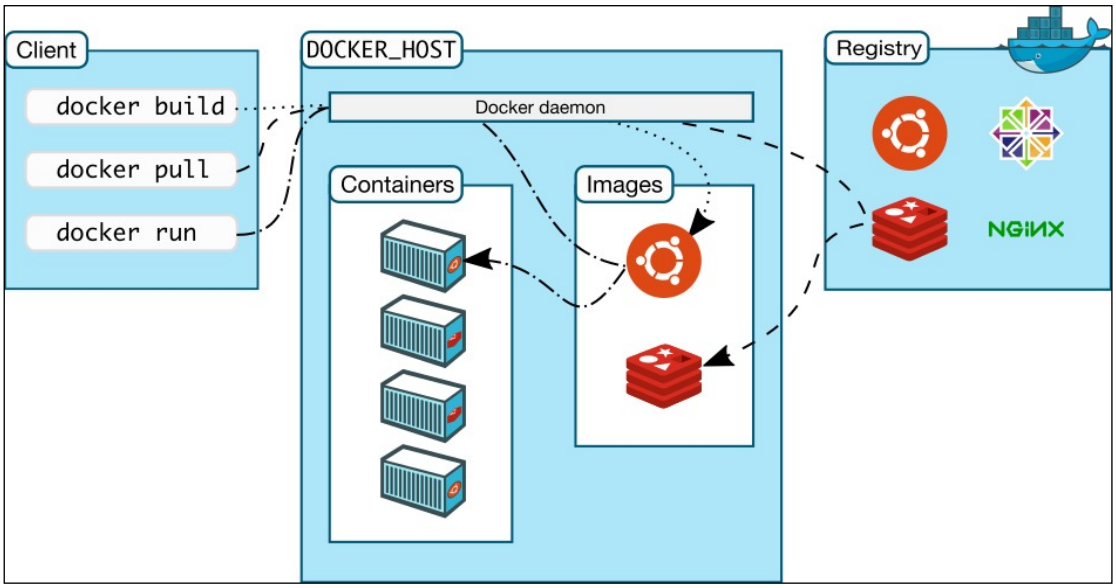


Server 是一个常驻进程；

REST API 实现了 client 和 server 间的交互协议；

CLI 实现容器和镜像的管理，为用户提供统一的操作界面。

Docker 使用 C/S 架构，Client 通过接口与 Server 进程通信实现容器的构建，运行和发布。client 和 server 可以运行在同一台集群，也可以通过跨主机实现远程通信，架构如图所示：



22.11 Docker 镜像&容器&仓库

熟悉了 Docker 虚拟化简介、组件和工作原理之后，还需要掌握 Docker 虚拟化镜像原理&引擎架构等知识。

Docker 虚拟化技术有三个基础概念：Docker 镜像、Docker 容器、Docker 仓库，三个概念详解如下：

1）Docker 镜像

Docker 虚拟化最基础的组件为镜像，镜像跟我们常见的 Linux ISO 镜像类似，但是 Docker 镜像是分层结构的，是由多个层级组成，每个层级分别存储各种软件实现某个功能，Docker 镜像是静止的、只读的，不能对镜像进行写操作。

2）Docker 容器：

Docker 容器是 Docker 虚拟化的产物，也是最早在生产环境使用的对象，Docker 容器的底层是 Docker 镜像，是基于镜像运行，并且在镜像最上层添加一层容器层之后的实体，容器层是可读、可写的，容器层如果需用到镜像层中的数据，可以通过 JSON 文件读取镜像层中的软件和数据，对整个容器进行修改，只能作用于容器层，不能直接对镜像层进行写操作。

3）Docker 仓库：

Docker 仓库是用于存放 Docker 镜像的地方，Docker 仓库分为两类，分别是：公共仓库（Public）和私有仓库（Private），国内和国外有很多默认的公共仓库，对外开放、免费或者付费使用，企业测试环境和生产环境推荐自建私有仓库，私有仓库的特点：安全、可靠、稳定、高效，能够更加自身的业务体系进行灵活升级和管理。

22.12 CentOS6.x（6.5+）Linux Docker 平台实战

掌握了 Docker 虚拟化概念和原理之后，最重要的就是要在生产环节中落地 Docker，Docker 虚拟化平台最早期只支持 Linux 操作系统，现在最新版 Windows 操作系统也慢慢支持 Docker 虚拟化。

本章节将选择不同的发行版本来构建 Docker 虚拟化平台，Linux 操作系统主流发行版本包括：Red Hat Linux、CentOS、Ubuntu、SUSE Linux、Fedora Linux 等，如下简要介绍即将部署 Docker 虚拟化平台的两个系统：CentOS 和 Ubuntu。

Docker 官方要求 Linux 内核版本在 3.8+以上，生产环境中尽量推荐使用 3.10+的 Linux 内核版本，Docker 从 1.13 版本起，采用时间线的方式作为版本号，Docker 版本现在基于 YY.MM，分为社区版（Community Edition）和企业版（ Enterprise Edition），社区版是免费提供给个人开发者和小型团体使用的，而企业版会提供额外的收费服务。

社区版按照 Stable 和 Edge 两种方式发布，每个季度更新 Stable 版本，如 17.06，17.09，每个月份更新 Edge 版本，如 17.09，17.10。

虚拟化&Docker 虚拟化技术本质的用途：

为了最大化的利用高配物理机的资源，提高硬件设备服务器的资源利用率，淘汰一些老、旧服务器，对老、旧服务器进行资源的重组、重用，满足企业飞快的发展。

虚拟化落地实施硬件设备选配上，尽量使用高配物理机资源，参考配置如下：

- 服务器品牌：Dell R730、R820；
- CPU 配置：Intel 至强 E5-2600 系列；

- MEM 配置：ECC DDR3 256G；
- DISK 配置：SAS 12TB（最大支持 24T）；
- NET 配置：Intel 四端口千兆网卡/双端口万兆网卡；

1）安装步骤和命令如下：

```
#安装 Epel 扩展源；

yum install epel-release -y

#安装 Docker-io 版本；

yum install docker-io lxc cgroup device-mapper* -y

#查 Docker 版本是否安装；

rpm -qa|grep -E "docker|lxc|device-mapper"

#启动 Docker 引擎服务；

service docker restart

#查看 Docker 服务进程；

ps -ef|grep docker
```

2）安装完成，如图所示：

```
[root@localhost ~]# yum install epel-release -y
#安装Docker-io版本;
yum install docker-io lxc cgroup device-mapper* -y
#查Docker版本是否安装;
rpm -qa|grep -E "docker|lxc|device-mapper"
#启动Docker引擎服务;
已加载插件：security
设置安装进程
Repository base is listed more than once in the configura
Repository updates is listed more than once in the config
Repository extras is listed more than once in the configu
Repository centosplus is listed more than once in the con
Repository contrib is listed more than once in the config
```

3）查看启动进程如图所示：

```
s-1.0.11-1.el6.x86_64
mapper-multipath-libs-0.4.9-106.el6_10.1.x86_64
mapper-libs-1.02.117-12.el6_9.1.x86_64
io-1.7.1-2.el6.x86_64
ocalhost ~]# #启动Docker引擎服务;
ocalhost ~]# service docker restart
cker:                                                      [失败]
g cgconfig service:                                        [确定]
g docker:                                                  [确定]
ocalhost ~]# #查看Docker服务进程;
ocalhost ~]# ps -ef|grep docker
   2958     1  1 19:41 pts/0    00:00:00 /usr/bin/docker -d
   3014  2830  0 19:41 pts/0    00:00:00 grep docker
```

4）查看 docker info 信息，如图所示：

```
[root@localhost ~]# docker info|more
Containers: 0
Images: 0
Storage Driver: devicemapper
 Pool Name: docker-252:1-860585-pool
 Pool Blocksize: 65.54 kB
 Backing Filesystem: extfs
 Data file: /dev/loop0
 Metadata file: /dev/loop1
 Data Space Used: 305.7 MB
 Data Space Total: 107.4 GB
 Data Space Available: 4.498 GB
 Metadata Space Used: 729.1 kB
```

5）从 Docker 仓库下载 Nginx 镜像：

```
[root@localhost ~]# docker pull nginx
latest: Pulling from nginx

0454203f6769: Downloading 2.964 MB/22.49 MB
97569d305060: Download complete
28a935a87300: Download complete
22883c0483ff: Download complete
81b77373eb80: Download complete
a0864a16d04f: Downloading 3.448 MB/22.2 MB
3dd89047b103: Download complete
004c60e28b8f: Download complete
020c4ca443ac: Download complete
23ca8637d096: Download complete
```

22.13 CentOS7.x（7.0+）Linux Docker 平台实战

1）安装步骤和命令如下：

```
#安装国内阿里源；

wget   -P /etc/yum.repos.d/ http://mirrors.aliyun.com/docker-

ce/linux/centos/docker-ce.repo

#安装 Docker-CE 版本；

yum install docker-ce* -y

#查 Docker 版本是否安装；

rpm -qa|grep -E "docker"

#启动 Docker 引擎服务；
```

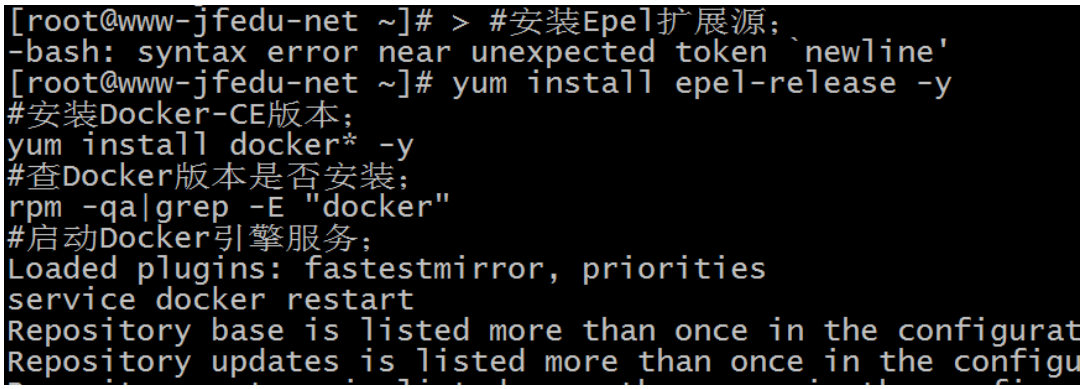
```
service docker restart

systemctl restart docker.service

#查看 Docker 服务进程；

ps -ef|grep docker
```

2）安装完成，如图所示：



3）查看启动进程如图所示：



4）查看 docker info 信息，如图所示：

```
[root@www-jfedu-net ~]# docker info|more
  WARNING: You're not using the default seccomp profile
Containers: 0
 Running: 0
 Paused: 0
 Stopped: 0
Images: 0
Server Version: 1.13.1
Storage Driver: overlay2
 Backing Filesystem: extfs
 Supports d_type: true
 Native Overlay Diff: true
```

5）从 Docker 仓库下载 Nginx 镜像：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker pull docker.io/nginx
Using default tag: latest
Trying to pull repository docker.io/library/nginx ...
latest: Pulling from docker.io/library/nginx
a5a6f2f73cd8: Extracting 6.652 MB/22.49 MB
a5a6f2f73cd8: Pull complete
67da5fbcb7a0: Pull complete
e82455fa5628: Pull complete
Digest: sha256:31b8e90a349d1fce7621f5a5a08e4fc519b634f7
Status: Downloaded newer image for docker.io/nginx:late
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
```

22.14 Ubuntu（16.04+）Linux Docker 平台实战

1）安装步骤和命令如下：

```
#更新 apt 源；

apt-get update
```

```
root@www-jfedu-net:~#
root@www-jfedu-net:~# apt-get update
Hit:1 http://repo.bcm.baidubce.com/deb ./all InRelease
Hit:2 http://mirrors.baidubce.com/ubuntu xenial InRelease
Hit:3 http://mirrors.baidubce.com/ubuntu xenial-updates I
Hit:4 http://mirrors.baidubce.com/ubuntu xenial-security
Hit:5 https://mirrors.ustc.edu.cn/docker-ce/linux/ubuntu
Hit:6 https://download.docker.com/linux/ubuntu xenial InR
Reading package lists... Done
root@www-jfedu-net:~#
```

#APT 源使用 HTTPS 以确保软件下载过程中不被篡改。需要添加使用 HTTPS

传输的软件包以及 CA 证书。

```
apt-get install \

apt-transport-https \

ca-certificates \

curl \

software-properties-common
```

#默认 APT 访问国外源，网络非常慢，此处建议使用国内源，需要添加软件源

的 GPG 密钥；

```
curl -fsSL https://mirrors.ustc.edu.cn/docker-ce/linux/ubuntu/gpg |

sudo apt-key add -
```

```
#curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-

key add -
```

#向 source.list 中添加 Docker 软件源；

```
add-apt-repository \

"deb [arch=amd64] https://mirrors.ustc.edu.cn/docker-ce/linux/ubuntu

$(lsb_release -cs) stable"

#官方源

#add-apt-repository "deb [arch=amd64]

https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
```

#更新 apt 软件包缓存；

apt-get update

#基于 apt-get 安装 docker-ce 社区版本；

apt-get install docker-ce

```
root@www-jfedu-net:~# apt-get install docker-ce
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  aufs-tools cgroupfs-mount containerd.io docker-ce-cli libse
Suggested packages:
  mountall
The following NEW packages will be installed:
  aufs-tools cgroupfs-mount containerd.io docker-ce docker-ce
The following packages will be upgraded:
  libseccomp2
1 upgraded, 6 newly installed, 0 to remove and 198 not upgrad
```

#查 Docker 版本是否安装；

dpkg -s docker-ce

```
root@www-jfedu-net:~# dpkg -s docker-ce
Package: docker-ce
Status: install ok installed
Priority: optional
Section: admin
Installed-Size: 83340
Maintainer: Docker <support@docker.com>
Architecture: amd64
Version: 5:18.09.0~3-0~ubuntu-xenial
Replaces: docker-engine
Depends: docker-ce-cli, containerd.io, iptables, libseccomp2 (
Recommends: aufs-tools, ca-certificates, cgroupfs-mount | cgro
Conflicts: docker (<< 1.5~), docker-engine, docker-engine-cs,
```

#启动 Docker 引擎服务；

service docker restart

systemctl restart docker

#查看 Docker 服务进程；

ps -ef|grep docker

2）安装完成，如图所示：

```
Processing triggers for man-db (2.7.5-1) ...
Processing triggers for ureadahead (0.100.0-19) ...
Processing triggers for systemd (229-4ubuntu13) ...
Setting up pigz (2.3.1-2) ...
Setting up aufs-tools (1:3.2+20130722-1.1ubuntu1) ...
Setting up cgroupfs-mount (1.2) ...
Setting up containerd.io (1.2.0-1) ...
Setting up docker-ce-cli (5:18.09.0~3-0~ubuntu-xenial) .
Setting up docker-ce (5:18.09.0~3-0~ubuntu-xenial) ...
update-alternatives: using /usr/bin/dockerd-ce to provid
Processing triggers for libc-bin (2.23-0ubuntu5) ...
Processing triggers for systemd (229-4ubuntu13) ...
Processing triggers for ureadahead (0.100.0-19) ...
```

3）查看启动进程如图所示：

```
root@www-jfedu-net:~# ps -ef|grep docker
root      25420      1  2 22:27 ?        00:00:00 /usr/bin/doc
root      25539  20763  0 22:28 pts/0    00:00:00 grep --color
root@www-jfedu-net:~#
root@www-jfedu-net:~# docker info|more
Containers: 0
 Running: 0
 Paused: 0
 Stopped: 0
Images: 0
Server Version: 18.09.0
Storage Driver: overlay2
 Backing Filesystem: extfs
```

4）查看 docker info 信息，如图所示：

```
Storage Driver: overlay2
 Backing Filesystem: extfs
 Supports d_type: true
 Native Overlay Diff: true
Logging Driver: json-file
Cgroup Driver: cgroupfs
Plugins:
 Volume: local
 Network: bridge host macvlan null overlay
 Log: awslogs fluentd gcplogs gelf journald json-file loca
Swarm: inactive
Runtimes: runc
Default Runtime: runc
Init Binary: docker-init
```

5）从 Docker 仓库下载 Nginx 镜像：

```
root@www-jfedu-net:~# docker pull docker.io/nginx
Using default tag: latest
latest: Pulling from library/nginx
a5a6f2f73cd8: Pull complete
67da5fbcb7a0: Pull complete
e82455fa5628: Pull complete
Digest: sha256:31b8e90a349d1fce7621f5a5a08e4fc519b634f
Status: Downloaded newer image for nginx:latest
root@www-jfedu-net:~# docker images
REPOSITORY          TAG                 IMAGE ID
nginx               latest              e81eb098537d
root@www-jfedu-net:~#
```

22.15 Docker 仓库源更新实战

Docker 默认连接的国外官方镜像，通常根据网络情况不同，访问时快时慢，大多时候获取速度非常慢，为了提示效率可以自建仓库或者先修改为国内仓库源，提升拉取镜像的速度。

Docker 可以配置的国内镜像有很多可供选择，例如：Docker 中国区官方镜像、阿

里云、网易蜂巢、DaoCloud 等，这些都是国内比较快的镜像仓库。

从国外官网下载 Docker Tomcat 镜像，访问速度慢，如图所示：

```
[root@www-jfedu-net ~]# docker pull tomcat
Using default tag: latest
Trying to pull repository docker.io/library/tomcat ...
latest: Pulling from docker.io/library/tomcat
cc1a78bfd46b: Downloading 457.9 kB/45.32 MB
6861473222a6: Pulling fs layer
7e0b9c3b5ae0: Downloading 1.747 MB/4.336 MB
ae14ee39877a: Waiting
8085c1b536f0: Waiting
6e1431e84c0c: Waiting
ca0e3df5a1fd: Waiting
d2cb611ced6c: Waiting
268dc3e43e66: Waiting
79a7e8d254c7: Waiting
5c848af92738: Waiting
789b92e37607: Waiting
```

Docker 镜像修改方法，vim /etc/docker/daemon.json，执行如下命令即可：

```
cat>/etc/docker/daemon.json<<EOF

{

"registry-mirrors":["https://registry.docker-cn.com"]

}

EOF

service docker restart
```

重启 Docker 服务即可，修改仓库地址为国内仓库后，获取镜像速度非常快，如图所示：

```
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# service docker restart
Redirecting to /bin/systemctl restart docker.service

[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# systemctl restart docker.service
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker pull tomcat
Using default tag: latest
Trying to pull repository docker.io/library/tomcat ...
latest: Pulling from docker.io/library/tomcat
cc1a78bfd46b: Downloading  37.2 MB/45.32 MB
6861473222a6: Downloading 7.439 MB/10.77 MB
7e0b9c3b5ae0: Download complete
ae14ee39877a: Download complete
8085c1b536f0: Download complete
6e1431e84c0c: Download complete
ca0e3df5a1fd: Downloading  10.5 MB/122.1 MB
d2cb611ced6c: Waiting
```

22.16 Docker 典型命令演练

Docker 虚拟化平台部署完成，默认没有图形界面管理，作为运维人员、测试人员、开发人员来讲，需要通过 Docker-Client 命令行操作，如下为 Docker 平台下 30+操作指令，熟练指令的操作能够帮助我们对 Docker 进行高效的管理和维护，从而提高自己的技能。

22.16.1 docker search 命令实战

#从 Docker 仓库中搜索 Nginx 镜像；

docker search nginx

#从 Docker 仓库中搜索 Tomcat 镜像；

docker search tomcat

22.16.2 docker pull 命令实战

#从 Docker 仓库下载 nginx 镜像；

docker pull docker.io/nginx

#从 Docker 仓库下载 tomcat 镜像；

docker pull docker.io/tomcat

```
[root@www-jfedu-net ~]# docker search nginx
INDEX       NAME
                    STARS     OFFICIAL   AUTOMATED
docker.io   docker.io/nginx
                    10210     [OK]
docker.io   docker.io/jwilder/nginx-proxy
y for docker c...   1459                 [OK]
docker.io   docker.io/richarvey/nginx-php-fpm
P-FPM capable ...   647                  [OK]
docker.io   docker.io/jrcs/letsencrypt-nginx-proxy-companic
 with nginx as...   436                  [OK]
docker.io   docker.io/kong
PI Management ...   245        [OK]
```

22.16.3 docker images 命令实战

#查看已下载的本地 Docker 镜像列表

docker images

```
258c74eb25ab: Pull complete
6eb86d11d371: Pull complete
adb5a4a6adf5: Pull complete
4685b434f297: Pull complete
Digest: sha256:94935124bfe3398d7d3264d010f5169f246128676
Status: Downloaded newer image for docker.io/tomcat:late
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker images
REPOSITORY          TAG                 IMAGE ID
docker.io/tomcat    latest              78b258e36eed
docker.io/nginx     latest              e81eb098537d
[root@www-jfedu-net ~]#
```

22.16.4 docker run 命令实战

#基于 docker run 启动 nginx 镜像，并且启动到/bin/bash 解释器；

docker run -itd docker.io/nginx /bin/bash

-i 表示：interactive 交互；

-t 表示：tty 终端；

d 表示：daemon 后台启动；

#基于 docker run 启动 nginx 镜像，启动到/bin/bash 解释器，同时映射本地

80 端口至容器 80 端口；

docker run -p 80:80 -itd docker.io/nginx /bin/bash

#-p 端口映射，第一个 80 宿主机监听端口，第二个 80 端口为容器监听；

#浏览器输入宿主机 IP+80 端口，即可访问容器中的 80 端口所在服务；

```
873ea9c87d5df1bb75b650926cfbcc4c53f675dfabb158830af0b115f99
ewer image for docker.io/nginx:latest
~]#
~]# clear
~]#
~]#
~]# docker run -itd  docker.io/nginx  /bin/bash
5ac2a3e75066decb7f1f8a197271ddade95660830f03
~]#
~]# docker ps
IMAGE               COMMAND             CREATED             STA
docker.io/nginx     "/bin/bash"         43 seconds ago      Up
scending_kare
```

22.16.5 docker ps 命令实战

#查看当前正在运行中的容器；

docker ps

#查看当前 Linux 系统所有容器，包括运行和已经停止、其他容器（所有容器）

docker ps -a

```
~]# docker ps
IMAGE               COMMAND             CREATED             STATUS
docker.io/nginx     "/bin/bash"         11 seconds ago      Up 10 secc
ical_wright
~]#
~]# docker ps -a
IMAGE               COMMAND             CREATED             STATUS
  NAMES
docker.io/nginx     "/bin/bash"         14 seconds ago      Up 12 secc
  quizzical_wright
docker.io/nginx     "/bin/bash"         5 minutes ago       Exited (0)
  condescending_kare
~]#
```

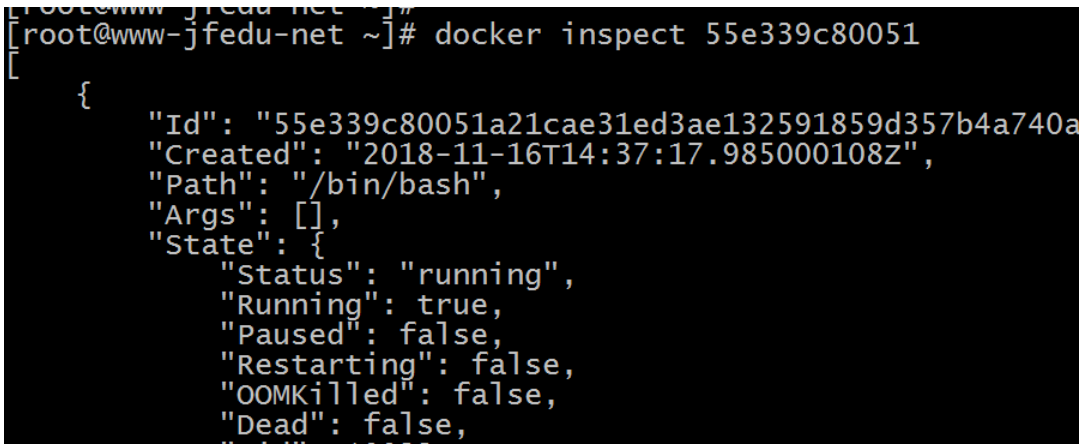
22.16.6 docker inspect 命令实战

#查看容器详细信息；

docker inspect 55e339c80051

#查看容器详细信息，并且从信息中过滤 IP 地址；

docker inspect 55e339c80051|grep -i ipaddr



22.16.7 docker exec 命令实战

#在 Docker 中容器运行指令:df -h；

docker exec 55e339c80051 df -h

#在 Docker 中容器/tmp 目录创建 jfedu.txt 文件；

docker exec 55e339c80051 touch /tmp/test.txt

#进入 Docker 容器/bin/bash 终端，然后执行 df -h 指令；

docker exec -it 55e339c80051 /bin/bash

df -h

```
        zabbix_agentd.pid
        说明.htm
w-jfedu-net ~]# clear
w-jfedu-net ~]#
w-jfedu-net ~]# docker exec 55e339c80051 touch /tmp/test.txt
w-jfedu-net ~]#
w-jfedu-net ~]#
w-jfedu-net ~]# docker exec 55e339c80051 ls /tmp/

w-jfedu-net ~]#
w-jfedu-net ~]# docker exec 55e339c80051 ls -l /tmp/test.txt
-- 1 root root 0 Nov 17 12:57 /tmp/test.txt
w-jfedu-net ~]#
w-jfedu-net ~]#
```

22.16.8 docker stop|start 命令实战

#基于 stop 停止正在运行中的容器；

docker stop 55e339c80051

#基于 start 启动已经停止的容器；

docker start 55e339c80051

```
[root@www-jfedu-net ~]# docker stop 55e339c80051
55e339c80051
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND
  PORTS               NAMES
55e339c80051        docker.io/nginx     "/bin/bash"
                      quizzical_wright
dc455c12ca7d        docker.io/nginx     "/bin/bash"
                      condescending_kare
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker start 55e339c80051
55e339c80051
```

22.16.9 docker kill 命令实战

#Kill 掉正在运行中某个容器；

docker kill 55e339c80051

```
[root@www-jfedu-net ~]# docker ps
CONTAINER ID        IMAGE               COMMAND
              NAMES
55e339c80051        docker.io/nginx     "/bin/bash"
p             quizzical_wright
[root@www-jfedu-net ~]# docker kill 55e339c80051
55e339c80051
[root@www-jfedu-net ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND
    PORTS               NAMES
55e339c80051        docker.io/nginx     "/bin/bash"
o                       quizzical_wright
dc455c12ca7d        docker.io/nginx     "/bin/bash"
```

22.16.10 docker rm 命令实战

#删除某个指定的已经停止 Docker 容器；

docker rm dc455c12ca7d

#强制删除某个指定的正在运行中的 Docker 容器；

docker rm -f 55e339c80051

```
[root@www-jfedu-net ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND
 PORTS               NAMES
55e339c80051        docker.io/nginx     "/bin/bash"
 80/tcp              quizzical_wright
dc455c12ca7d        docker.io/nginx     "/bin/bash"
                     condescending_kare
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker rm dc455c12ca7d
dc455c12ca7d
[root@www-jfedu-net ~]# docker rm -f 55e339c80051
55e339c80051
```

22.16.11 docker rmi 命令实战

#从 Docker images 列表中删除某个镜像；

docker rmi 78b258e36eed

#从 Docker images 列表中删除多个镜像；

docker rmi e81eb098537d 415381a6cb81

```
[root@www-jfedu-net ~]# docker images
REPOSITORY          TAG                 IMAGE ID
docker.io/nginx     latest              e81eb098537d
docker.io/redis     latest              415381a6cb81
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker rmi e81eb098537d 41538
Untagged: docker.io/nginx:latest
Untagged: docker.io/nginx@sha256:98b06873ea9c87d5df1b
Deleted: sha256:e81eb098537d6c4a75438eacc6a2ed94af74c
Deleted: sha256:7055505a92c39c6f943403d54a1cda020bfeb
Deleted: sha256:378a8fcc106dc4d3a9f2dc0b642b164e25de3
Deleted: sha256:ef68f6734aa485edf13a8509fe60e4272428d
```

22.17 CentOS6.x Docker 桥接网络实战

基于 CentOS6.x 构建 Docker 桥接网络，案例方法如下：

Docker0 桥接网络可以自定义，如下为自定义桥接网络的设置：

/etc/init.d/docker stop

关掉 docker0

ifconfig docker0 down

删除 docker

brctl delbr docker0

增加网桥 br0

yum install bridge-utils

CentOS6.5 下的配置：

vim /etc/sysconfig/docker

other_args="-b=br0"

192.168.1.6

```
# /etc/sysconfig/docker
#
# Other arguments to pass to the docker daemon process
# These will be parsed by the sysv initscript and appended
# to the arguments list passed to docker -d

other_args="-b=br0"
DOCKER_CERT_PATH=/etc/docker

# Location used for temporary files, such as those created by
# # docker load and build operations. Default is /var/lib/docker/tmp
# # Can be overriden by setting the following environment variable.
# # DOCKER_TMPDIR=/var/tmp
```

除此之外，配置 bridge 桥接网络可以直接设置网卡配置文件：

/etc/sysconfig/network-scripts/下，修改 ifcfg-eth0 网卡配置，同时增加 ifcfg-br0

桥接网卡配置，操作流程如下：

vi ifcfg-eth0 内容修改为如下：

```
DEVICE=eth0

BOOTPROTO=none

NM_CONTROLLED=no

ONBOOT=yes

TYPE=Ethernet

BRIDGE="br0"

IPADDR=192.168.43.81

NETMASK=255.255.255.0

GATEWAY=192.168.43.1

USERCTL=no
```

vi ifcfg-br0 内容如下：

```
DEVICE="br0"

BOOTPROTO=none

IPV6INIT=no

NM_CONTROLLED=no

ONBOOT=yes

TYPE="Bridge"

IPADDR=192.168.43.81
```

NETMASK=255.255.255.0

GATEWAY=192.168.43.1

USERCTL=no

```
[root@localhost network-scripts]# cat ifcfg-eth0
DEVICE=eth0
BOOTPROTO=none
ONBOOT=yes
TYPE=Ethernet
BOOTPROTO=static
BRIDGE=br0
[root@localhost network-scripts]# cat ifcfg-br0
DEVICE="br0"
ONBOOT=yes
NM_CONTROLLED=no
BOOTPROTO=dhcp
TYPE=Bridge
[root@localhost network-scripts]#
```

启动 docker 服务

/etc/init.d/docker start

查看服务器网卡信息如下：

```
192.168.1.6 ×
br0       Link encap:Ethernet  HWaddr 00:0C:29:67:FA:D3
          inet addr:192.168.1.6  Bcast:192.168.1.255  Mask:255.255.
          inet6 addr: fe80::20c:29ff:fe67:fad3/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:165 errors:0 dropped:0 overruns:0 frame:0
          TX packets:49 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:41513 (40.5 KiB)  TX bytes:6286 (6.1 KiB)

eth0      Link encap:Ethernet  HWaddr 00:0C:29:67:FA:D3
          inet6 addr: fe80::20c:29ff:fe67:fad3/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:7943 errors:0 dropped:0 overruns:0 frame:0
          TX packets:4158 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1168582 (1.1 MiB)  TX bytes:663488 (647.9 KiB)
          Interrupt:19 Base address:0x2000
```

启动一个新的 docker 客户端，查看客户端 ip 如下：

进入已经启动过的容器命令：docker attach 容器 ID 即可。

```
[root@localhost ~]# docker ps -l
CONTAINER ID        IMAGE                         COMMAND             CREATED
        STATUS                      PORTS               NAMES
a4cae461d340        jdeathe/centos-ssh:latest   "/bin/bash"         5 hours ago
        Exited (-127) 32 minutes ago   22/tcp              thirsty_payne
[root@localhost ~]# docker start a4cae461d340
a4cae461d340
[root@localhost ~]#
[root@localhost ~]# docker attach a4cae461d340

;@a4cae461d340:/[root@a4cae461d340 /]#
;@a4cae461d340:/[root@a4cae461d340 /]# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:42:C0:A8:01:02
          inet addr:192.168.1.2  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::42:c0ff:fea8:102/64 Scope:Link
          UP BROADCAST RUNNING  MTU:1500  Metric:1
          RX packets:81 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:8045 (7.8 KiB)  TX bytes:468 (468.0 b)

;@a4cae461d340:/[root@a4cae461d340 /]# _
```

从外网下载 nginx 包：

```
;@a4cae461d340:/[root@a4cae461d340 /]# wget http://nginx.org/download/nginx-1.6.
3.tar.gz
--2015-04-07 22:11:08--  http://nginx.org/download/nginx-1.6.3.tar.gz
Resolving nginx.org... 206.251.255.63
Connecting to nginx.org|206.251.255.63|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: http://124.202.164.5/files/12230000062C0589/nginx.org/download/nginx-1
.6.3.tar.gz [following]
--2015-04-07 22:11:08--  http://124.202.164.5/files/12230000062C0589/nginx.org/d
ownload/nginx-1.6.3.tar.gz
Connecting to 124.202.164.5:80... connected.
HTTP request sent, awaiting response...
200 OK
Length: 805253 (786K) [application/octet-stream]
Saving to: `nginx-1.6.3.tar.gz'

100%[======================================>] 805,253      456K/s   in 1.7s

2015-04-07 22:11:21 (456 KB/s) - `nginx-1.6.3.tar.gz' saved [805253/805253]

;@a4cae461d340:/[root@a4cae461d340 /]#
;@a4cae461d340:/[root@a4cae461d340 /]# _
```

22.18 CentOS7.x Docker 桥接网络实战

基于 CentOS7.x 构建 Docker 桥接网络，案例方法如下：

配置 bridge 桥接网络可以直接设置网卡配置文件：

/etc/sysconfig/network-scripts/下，修改 ifcfg-ens33 网卡配置，同时增加 ifcfg-br0

桥接网卡配置，操作流程如下：

vi ifcfg-ens33 内容修改为如下：

```
DEVICE=ens33

BOOTPROTO=none

NM_CONTROLLED=no

ONBOOT=yes

TYPE=Ethernet

BRIDGE="br0"

IPADDR=192.168.43.81

NETMASK=255.255.255.0

GATEWAY=192.168.43.1

USERCTL=no
```

vi ifcfg-br0 内容如下：

```
DEVICE="br0"

BOOTPROTO=none

IPV6INIT=no

NM_CONTROLLED=no
```

```
ONBOOT=yes

TYPE="Bridge"

IPADDR=192.168.43.81

NETMASK=255.255.255.0

GATEWAY=192.168.43.1

USERCTL=no
```

启动 docker 服务，即可；

service docker start

Docker 默认提供了一个隔离的内网环境，启动时会建立一个 docker0 的虚拟网卡，每个容器都是连接到 docker0 网卡上的。而 docker0 的 ip 段为 172.17.0.1，如果想让容器与宿主机同一网段的其他机器访问 ,就必须在启动 docker 的时候将某个端口映射到宿主机的端口。

KVM 的桥接网络非常方便，其实 docker 也比较方便，至少不是自带的桥接而已，上次课程我们讲解了 docker 容器在 centos6.5 下的实现方法 ,今天我们来讲解 centos7 下如果快速实现 docker 容器桥接网络，并为容器分配外网 IP。如下为通过 pipework 工具配置容器 IP 方法：

```
安装 pipework

git clone https://github.com/jpetazzo/pipework

cp ~/pipework/pipework /usr/local/bin/
```

启动容器并设置网络

docker run -itd --net=none --name=lamp2 centos7 /bin/bash

pipework br0 lamp2 192.168.1.11/24@192.168.1.88

进入容器查看 ip

docker exe lamp2 ifconfig

```
bash-4.1# ifconfig
eth1      Link encap:Ethernet  HWaddr FA:B9:11:74:4D:07
          inet addr:192.168.1.11  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::f8b9:11ff:fe74:4d07/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1007 errors:0 dropped:0 overruns:0 frame:0
          TX packets:43 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:192809 (188.2 KiB)  TX bytes:3574 (3.4 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 b)  TX bytes:0 (0.0 b)
```

22.19 基于 Docker 构建 JAVA 服务器实战

Tomcat 是由 Apache 软件基金会下属的 Jakarta 项目开发的一个 Servlet 容器，按照 Sun Microsystems 提供的技术规范，实现了对 Servlet 和 JavaServer Page（JSP）的支持，Tomcat 本身也是一个 HTTP 服务器，可以单独使用，apache 是一个以 C 语言编写的 HTTP 服务器。Tomcat 主要用来解析 JSP 语言。目前最新版本为 8.0。

Tomcat 安装配置

安装 tomcat 之前需要安装 jdk (Java Development Kit) 是 Java 语言的软件开发

工具包(SDK))，这里选择 jdk-6u18-linux-x64-rpm.bin，bin 文件安装跟 sh 文件方法一样，sh ./jdk-6u18-linux-x64-rpm.bin，回车即可 默认安装到/usr/java/jdk1.6.0_18 目录下。

配置 java 环境变量，vi /etc/profile 添加如下代码：

```
export JAVA_HOME=/usr/java/jdk1.6.0_18
export CLASSPATH=$CLASSPATH:$JAVA_HOME/lib:$JAVA_HOME/jre/lib
export PATH=$JAVA_HOME/bin:$JAVA_HOME/jre/bin:$PATH:$HOMR/bin
```

使环境变量立刻生效；

source /etc/profile

查看 java 版本，显示版本为 1.6.0_18，证明安装成功；

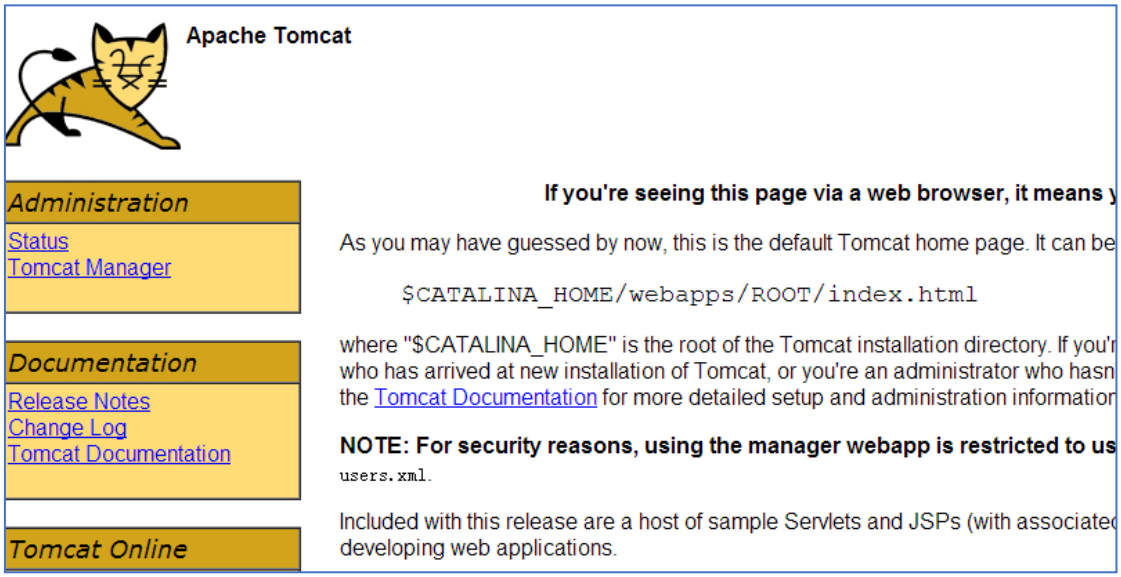
java -version

官网下载 tomcat 相应版本：apache-tomcat-6.0.30.tar.gz，执行如下命令部署 Tomcat：

```
tar -xzf apache-tomcat-6.0.30.tar.gz
mv apache-tomcat-6.0.30 /usr/local/tomcat
/usr/local/tomcat/bin/startup.sh
```

查看 ps -ef |grep tomcat 进程及端口是否存在；

通过页面访问 Tomcat 默认测试页面如下：



如何创建自定义的网站页面呢，定义自己的发布目录，方法如下：

server.xml 配置文件末尾加入代码：

```
<Context path="/"    docBase="/data/webapps/www"     reloadable="true"/>
```

```
        that are performed against this UserDatabase are immediately
        available for use by the Realm.  -->
    <Realm className="org.apache.catalina.realm.UserDatabaseRealm"
           resourceName="UserDatabase"/>

    <!-- Define the default virtual host
         Note: XML Schema validation will not work with Xerces 2.2.
     -->
    <Host name="localhost"  appBase="webapps"
          unpackWARs="true" autoDeploy="true"
          xmlValidation="false" xmlNamespaceAware="false">

      <!-- SingleSignOn valve, share authentication between web applications
           Documentation at: /docs/config/valve.html -->
      <!--
      <Valve className="org.apache.catalina.authenticator.SingleSignOn" />
      -->

      <!-- Access log processes all example.
           Documentation at: /docs/config/valve.html -->
      <!--
      <Valve className="org.apache.catalina.valves.AccessLogValve" directory="logs"
             prefix="localhost_access_log." suffix=".txt" pattern="common" resolveHosts="false"/>
      -->
      <Context path="/" docBase="/data/webapps/www"  reloadable="true"/>

    </Host>
  </Engine>
```

/data/webapps/www 创建自己的 jsp 代码，重启 tomcat 即可访问。

22.20 基于 Docker WEB 管理 Docker 容器

通常来讲，开发人员和管理人员默认通过命令行来创建及运行 Docker 容器，但 Docker 的 Remote API 让他们可以通过充分利用 REST（代表性状态传输协议）的 API，运行相同的命令。

DockerUI 也是基于 API 方式管理宿主机的 Docker 引擎。Docker UI Web 前端程序让你可以处理通常通过 Web 浏览器的命令行来管理的许多任务。

主机上的所有容器都可以通过仅仅一条连接来处理，该项目几乎没有任何依赖关系。该软件目前仍在大力开发之中，但是它采用麻省理工学院（MIT）许可证，所以可以免费地重复使用。

Docker UI 不包含任何内置的身份验证或安全机制，所以务必将任何公之于众的 DockerUI 连接放在用密码来保护的系统后面。

1）下载 Docker UI 镜像；

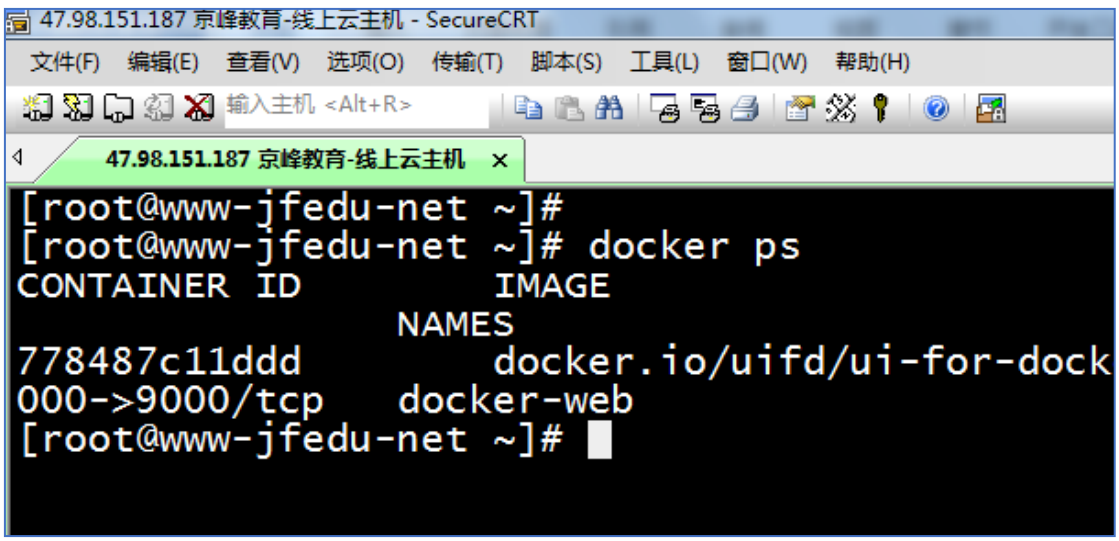
只需要在宿主机 pull 相关的镜像即可，指令如下：

```
docker pull uifd/ui-for-docker
docker images
```

```
-net ~]#
-net ~]# docker pull uifd/ui-for-docker
ag: latest
repository docker.io/uifd/ui-for-docker ...
 from docker.io/uifd/ui-for-docker
fe371ff5a69549269b24073a5ab1244dd4c0b834cbadf24487057
s up to date for docker.io/uifd/ui-for-docker:latest
-net ~]# docker images
                TAG                 IMAGE ID
t               latest              690cb3b9c7d1
                latest              c82521676580
ui-for-docker   latest              965940f98fa5
-net ~]#
```

2）启动 docker-UI 服务，并且映射 9090 至容器 9090；

docker run -it -d --name docker-web -p 9000:9000 -v /var/run/docker.sock:/var/run/docker.sock docker.io/uifd/ui-for-docker



3）如果启动 Docker 端口映射，报错信息如下：

docker0: iptables: No chain/target/match by that name.

解决方法：

如上报错信息是因为本地 iptables 规则策略没有匹配的链表，解决方案如下：

pkill docker

iptables -t nat -F

ifconfig docker0 down

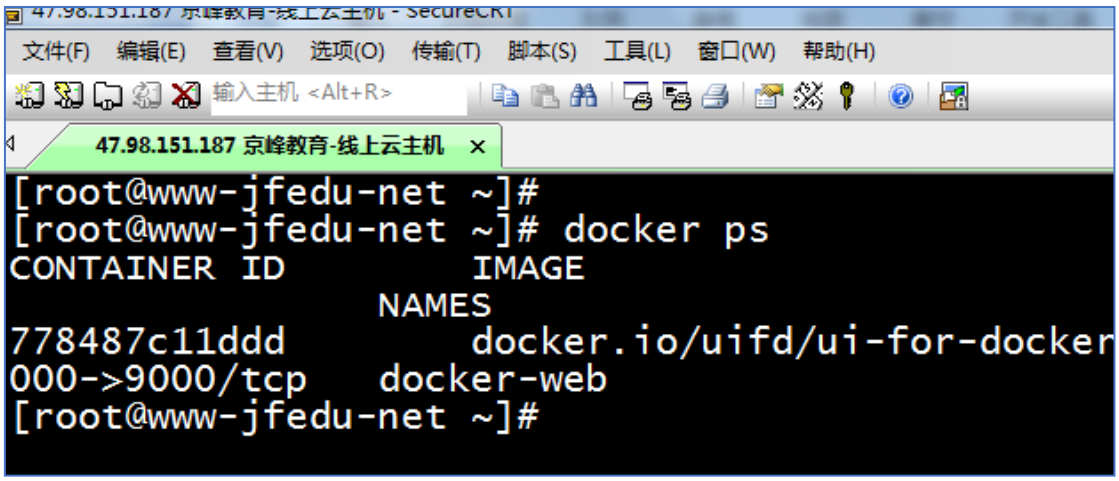
yum install -y bridge* -y

brctl delbr docker0

service docker restart

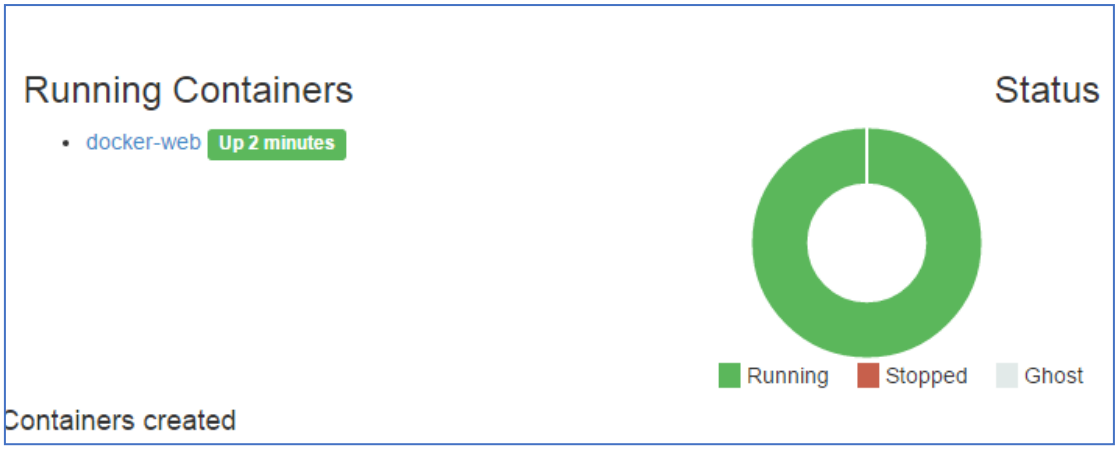
```
[root@www-jfedu-net ~]# docker ps -qa
e11075abd53a
[root@www-jfedu-net ~]# docker ps -aq
e11075abd53a
[root@www-jfedu-net ~]# docker ps -aq|xargs docker rm -
e11075abd53a
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]# docker run -it -d --name docker
cker.io/uifd/ui-for-docker
b510c42470c1120368100c80b5d5a375df74b8d9ad027305a5542b2
[root@www-jfedu-net ~]#
[root@www-jfedu-net ~]#
```

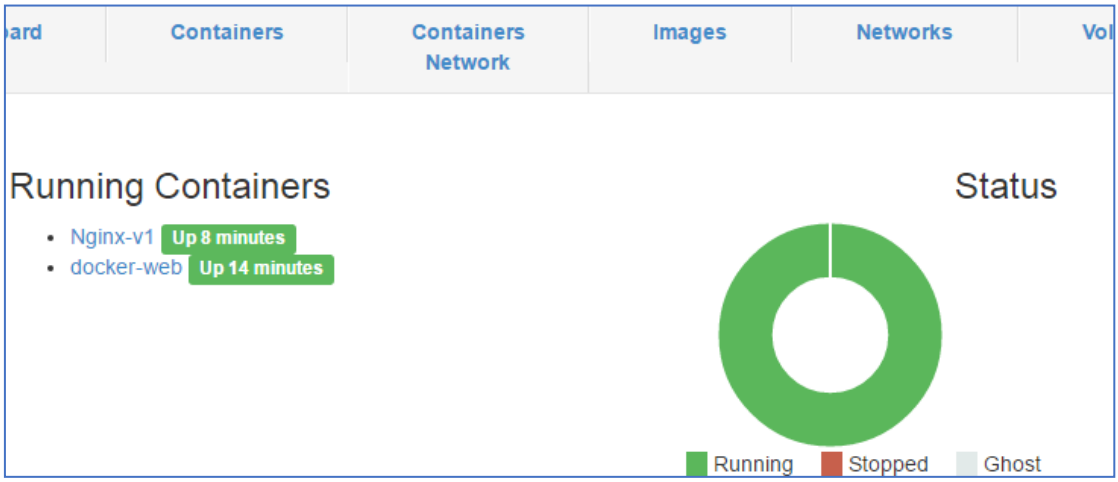
4）通过 docker ps 查看 Docker UI 状态；



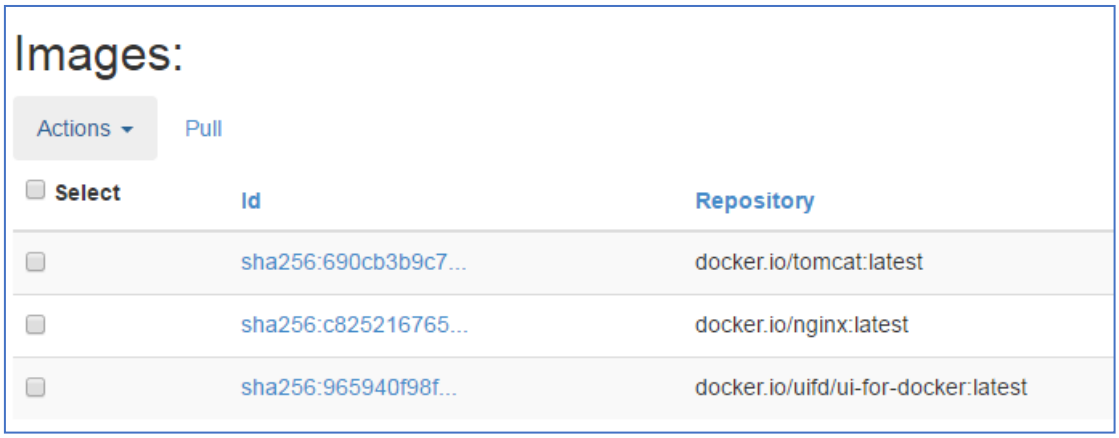
5）通过浏览器登录 WEB 9000 端口访问如图所示：



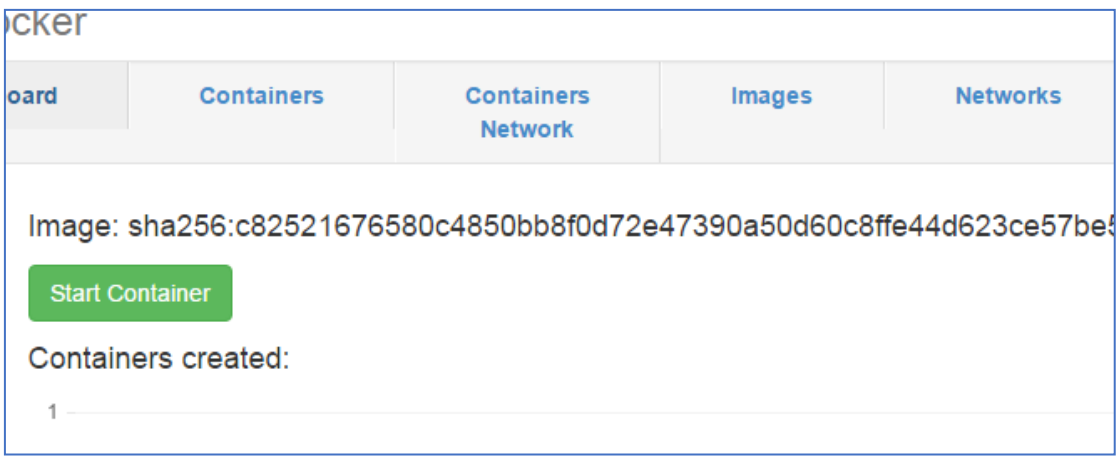


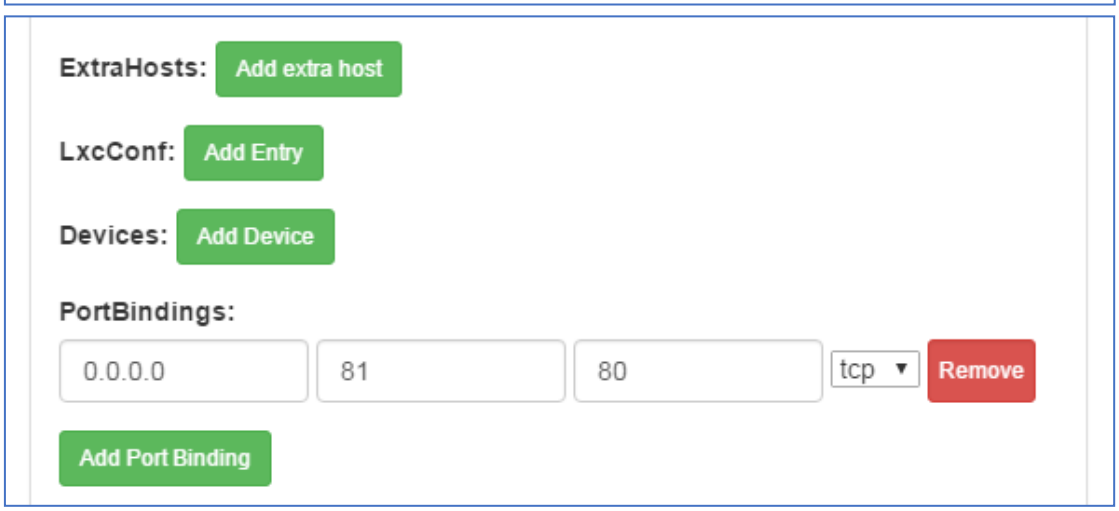


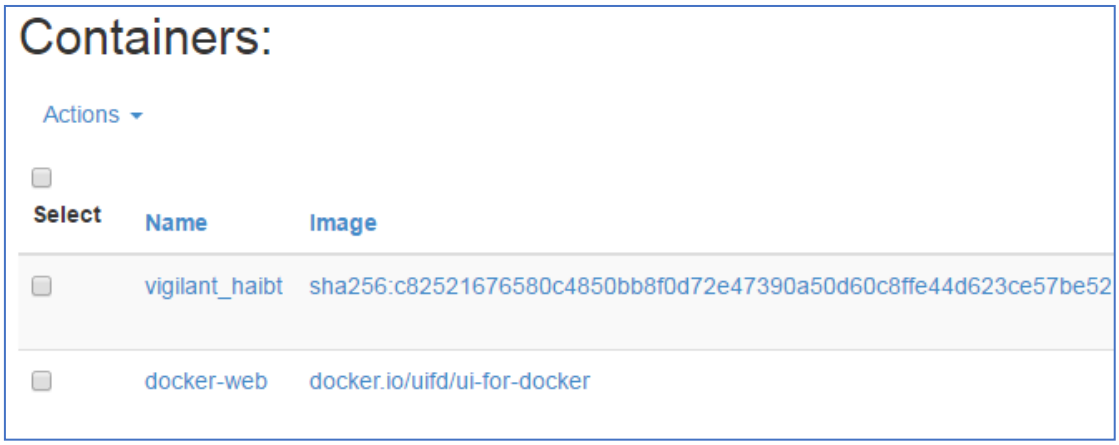
6）选择 WEB 界面 images 镜像列表，如图所示：



7）基于镜像启动 Docker 容器虚拟机，并且实现端口映射，如图所示：



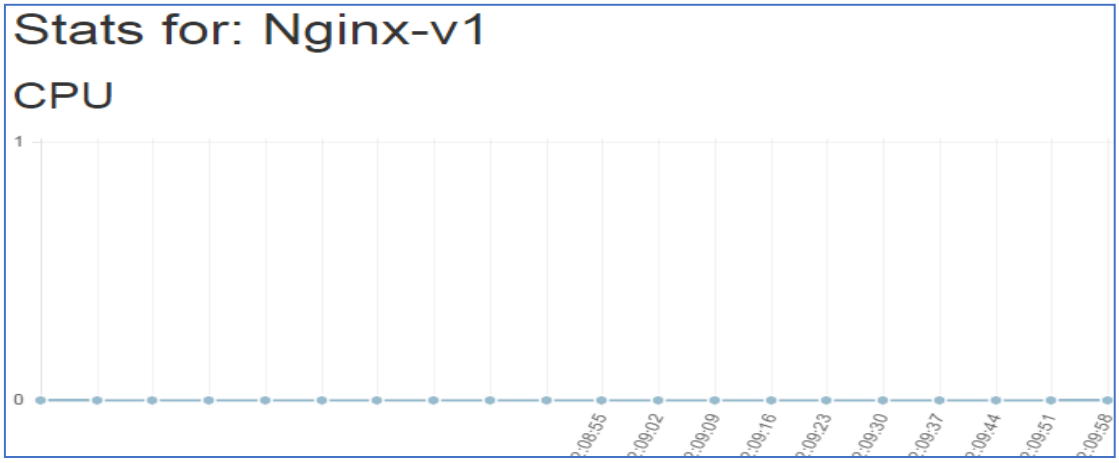


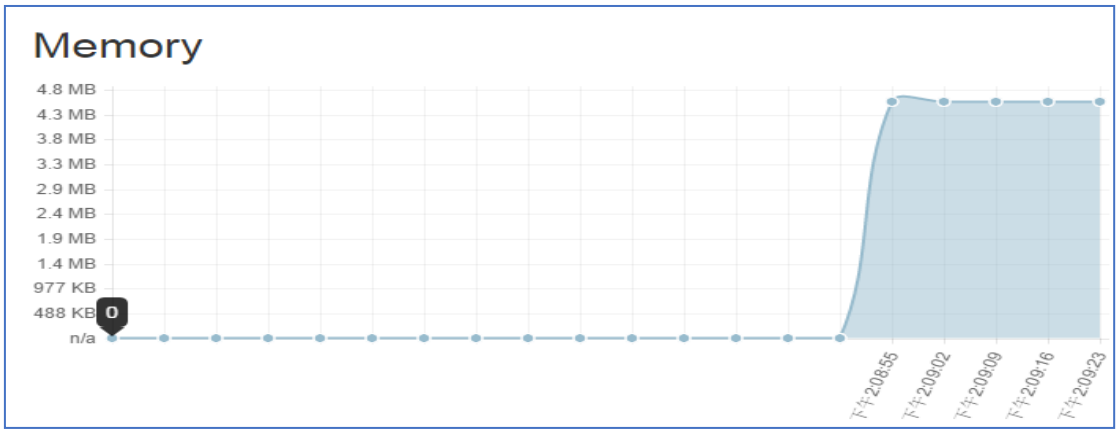


8）创建 Docker 容器之后，通过浏览器实现访问 81 端口，如图所示：



9）Docker 容器资源 WEB 监控如图所示：





22.21 Docker Tomcat 性能优化

线上环境使用默认 tomcat 配置文件，性能很一般，为了满足大量用户的访问，需要对 tomcat 进行参数性能优化，具体优化的地方如下：

- Linux 内核的优化；
- 服务器资源配置的优化；
- Tomcat 参数优化；
- 配置负载集群优化；

Tomcat 参数的优化：Server.xml 文件,关闭 DNS 查询、配置最大并发等参数。

maxThreads：tomcat 起动的最大线程数，即同时处理的任务个数，默认值为 200

acceptCount：当 tomcat 起动的线程数达到最大时，接受排队的请求个数，默认值为 100

当然这些值都不是越大越好，需要根据实际情况来设定。可以基于测试的基础上来不断的调优分析。Server.xml 里面优化如下：

<Connector port="8080"

protocol="org.apache.coyote.http11.Http11NioProtocol"

connectionTimeout="20000"

redirectPort="8443"

maxThreads="5000"

minSpareThreads="20"

acceptCount="1000"

disableUploadTimeout="true"

enableLookups="false"

URIEncoding="UTF-8" />

Catalina.sh JVM 参数优化，添加如下内容：

CATALINA_OPTS="$CATALINA_OPTS –Xms4000M –Xmx4000M –Xmn1000M -XX:SurvivorRatio=4 -XX:+UseConcMarkSweepGC -XX:CMSInitiatingOccupancyFraction=82 -DLOCALE=UTF-16LE -DRAMDISK=/ -DUSE_RAM_DISK=ture -DRAM_DISK=true"

配置多个 tomcat 实例，方法也很简单，只需要在服务器上 cp 多个 tomcat，然后修改三个端口和发布目录即可，然后分别启动即可。

为了提升整个网站的性能，还需要在 tomcat 前面架设 nginx web 反向代理服务器，用以提高用户高速访问。

```
[root@750ef5f019d5 ~]#
[root@750ef5f019d5 ~]# netstat -tnl
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address               Foreign Address
tcp        0      0 0.0.0.0:22                  0.0.0.0:*
tcp        0      0 ::ffff:127.0.0.1:8005       :::*
tcp        0      0 ::ffff:127.0.0.1:8006       :::*
tcp        0      0 :::8009                     :::*
tcp        0      0 :::8010                     :::*
tcp        0      0 :::8080                     :::*
tcp        0      0 :::80                       :::*
tcp        0      0 :::8081                     :::*
tcp        0      0 :::22                       :::*
[root@750ef5f019d5 ~]# ps -ef |grep java
root      4969     1 16 14:57 pts/1    00:00:04 /usr/java/jdk1.8.0_45/
ing.config.file=/usr/local/tomcat1/conf/logging.properties -Djava.util
he.juli.ClassLoaderLogManager -Xms512M -Xmx512M -Xmn100M -XX:SurvivorR
eepGC -XX:CMSInitiatingOccupancyFraction=82 -DLOCALE=UTF-16LE -DRAMDIS
DRAM_DISK=true -Djava.endorsed.dirs=/usr/local/tomcat1/endorsed -class
in/bootstrap.jar -Dcatalina.base=/usr/local/tomcat1 -Dcatalina.home=/u
o.tmpdir=/usr/local/tomcat1/temp org.apache.catalina.startup.Bootstrap
```

22.22 Docker 生产环境资源监控方案&监控实战

Docker 虚拟化平台自带了 Docker 容器（虚拟机）资源监控的功能，通过对 docker 容器的资源监控，我们可以随时掌握容器进行相关的资源性能，对容器性能进行更好的评估。通常而言，监控容器资源的指标，主要包括以下方面：

- 容器的 CPU 情况和使用量；
- 容器的内存情况和使用量；
- 容器的本地镜像情况；
- 主机的容器运行情况；

例如：使用 docker ps -a ,docker images 命令查看容器的本地镜像、容器运行的情况，使用 docker stats 命令可以监控相关容器实例情况。

22.22.1 docker stats 监控工具

1）查看本地镜像和容器列表，命令如下：

docker images

docker ps

```
[root@www-jfedu-net-186 ~]# docker images
REPOSITORY          TAG                 IMAGE ID
tomcat              latest              16858838
nginx               latest              f09fe80e
[root@www-jfedu-net-186 ~]#
[root@www-jfedu-net-186 ~]# docker ps
CONTAINER ID        IMAGE               COMMAND
     PORTS                    NAMES
1d1c1a547f98        nginx               "nginx -
     80/tcp                   awesome_rubin
fc4534a5ee3d        nginx               "nginx -
     80/tcp                   vibrant_wilson
ca993152f0a8        nginx               "nginx -
```

2）通过 docker stats 查看所有运行的容器的资源，如图所示：

docker stats



3）通过 docker stats 容器 id，查看指定容器的资源，如图所示：

docker stats 1d1c1a547f98



4）通过 docker stats 容器 id，--nostream 非流式查看容器资源：

docker stats 1d1c1a547f98 --no-stream

获取容器 CPU 的信息；

docker stats 1d1c1a547f98 --no-stream|awk 'NR>1 {print $1,"CPU:"$3}'

获取容器 MEM 的信息；

docker stats 1d1c1a547f98 --no-stream|awk 'NR>1 {print $1,"MEM:"$4}'

获取容器 IO 读写的信息；

docker stats 1d1c1a547f98 --no-stream|awk 'NR>1 {print $1,"IO:"$(NF-1)}'

如果想看到更为详细的容器属性，还可以通过 netcat，使用 Docker 远程 API 来查看。发送一个 HTTP GET 请求/containers/[CONTAINER_NAME]，其中 CONTAINER_NAME 是你想要统计的容器名称。你可以从这里看到一个容器 stats 请求的完整响应信息。在上述的例子中你会得到 CPU、IO 以及内存的详细信息。

当然这种方式并不会令人接受，大家还是希望能够看到非常直观，详细的可视化界面。

22.22.2 CAdvisor 监控工具

CAdvisor 是 google 开发的容器监控工具，CAdvisor 是一个易于设置并且非常有用的工具，我们不用 SSH 到服务器才能查看资源消耗，而且它还给我们生成了图表。此外当集群需要额外的资源时，压力表（pressure gauges ）提供了快速预览。而且，CAdvisor 是免费的，并且还开源。

它的资源消耗也比较低。但是它有它的局限性，它只能监控一个 Docker 主机。因此如果你是多节点的话，那就比较麻烦，我们需要在所有的主机上都安装一个 CAdvisor。值得注意的是，如果你使用的是 Kubernetes，你可以使用 heapster 来监

控多节点集群。在图表中的数据仅仅是时长一分钟的移动窗口，并没有方法来查看长期趋势。如果资源使用率在危险水平，它却没有生成警告的机制。

如果在 Docker 节点的资源消耗方面，你没有任何可视化界面，那么 CAdvisor 是一个不错的开端来带你步入容器监控，然而如果你打算在你的容器中运行任何关键任务，那你就需要一个更强大的工具或者方法。

22.22.3 CAdvisor 部署配置

1）下载 cadvisor 镜像，命令如下：

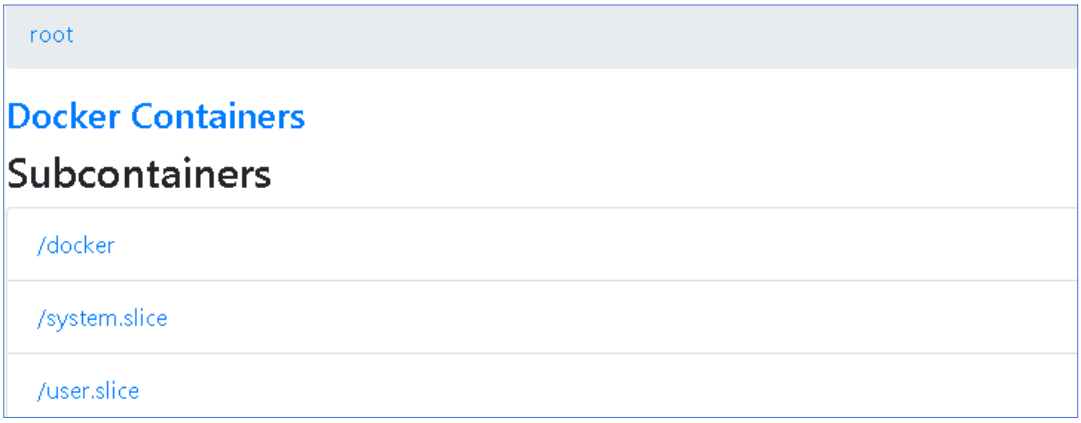
docker pull google/cadvisor

2）基于镜像启动 cadvisor 容器，命令如下：

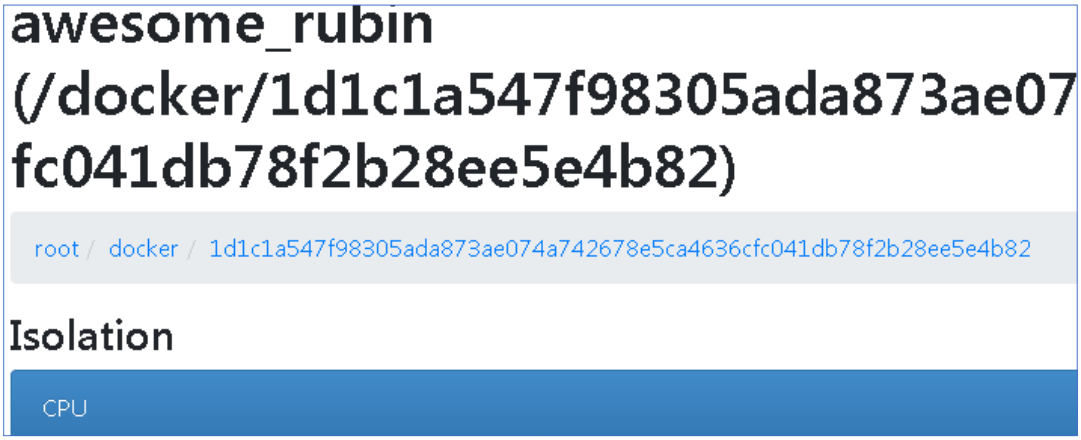
docker run -v /var/run:/var/run:rw -v /sys:/sys:ro -v /var/lib/docker:/var/lib/docker:ro -p 8080:8080 -d --name cadvisor google/cadvisor

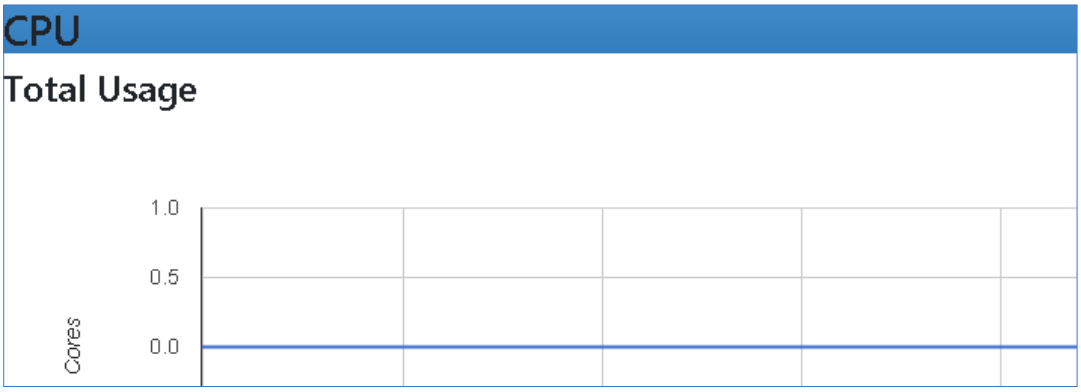
3）通过浏览器访问 cadviosr 8080 端口，如图所示：

http://106.12.133.186:8080/containers/



4）查看某个容器的详细资源，如图所示：





22.22.4 构建 Cadvisor+InfluxDB+Grafana 平台

1）CAdvisor 简介

CAdvisor 提供的操作界面略显简陋，而且需要在不同页面之间跳转，模拟只能监控一台服务器，这不免会让人质疑它的实用性。但 cAdvisor 的一个亮点是它可以将监控到的数据导出给第三方工具，由这些工具进一步加工处理。我们可以把 cAdvisor 定位为一个监控数据收集器，收集和导出数据是它的强项，而非展示数据。

2）InfluxDB 简介

InfluxDB 是用 Go 语言编写的一个开源分布式时序、事件和指标数据库，无需外部依赖。类似的数据库有 Elasticsearch、Graphite 等。

3）Grafana

Grafana 是一个可视化面板（Dashboard），有着非常漂亮的图表和布局展示，功能齐全的度量仪表盘和图形编辑器，支持 Graphite、zabbix、InfluxDB、Prometheus 和 OpenTSDB 作为数据源。Grafana 主要特性：灵活丰富的图形化选项；可以混合多种风格；支持白天和夜间模式，多个数据源。

4）Cadvisor+InfluxDB+Grafana 部署

Cadvisor+InfluxDB+Grafana 单独部署方式比较繁琐，此处采用 Docker-compose 方式部署，首先编写 compose.yml 文件，然后启动 compose 相关容器服务即可。

5）Docker-compose.yml 文件代码如下：

```
version: '3'services:

    influxdb:
        image: tutum/influxdb:0.9
        container_name: influxdb
        restart: always
        environment:
            - PRE_CREATE_DB=cadvisor
        ports:
            - "8083:8083"
            - "8086:8086"
        expose:
            - "8090"
            - "8099"
        volumes:
            - influxdbData:/data
    cadvisor:
        image: google/cadvisor
```

```
        container_name: cadvisor
        links:
            - influxdb:influxsrv
        command: -storage_driver=influxdb -storage_driver_db=cadvisor -
storage_driver_host=influxsrv:8086
        restart: always
        ports:
            - "8080:8080"
        volumes:
            - /:/rootfs:ro
            - /var/run:/var/run:rw
            - /sys:/sys:ro
            - /var/lib/docker/:/var/lib/docker:ro
    grafana:
        image: grafana/grafana
        container_name: grafana
        restart: always
        links:
            - influxdb:influxsrv
```

```
    ports:
      - "3000:3000"
    environment:
      - HTTP_USER=admin
      - HTTP_PASS=admin
      - INFLUXDB_HOST=influxsrv
      - INFLUXDB_PORT=8086
      - INFLUXDB_NAME=cadvisor
      - INFLUXDB_USER=root
      - INFLUXDB_PASS=root

volumes:
  influxdbData:
```

启动 Docker-compose，命令如下：

docker-compose up -d

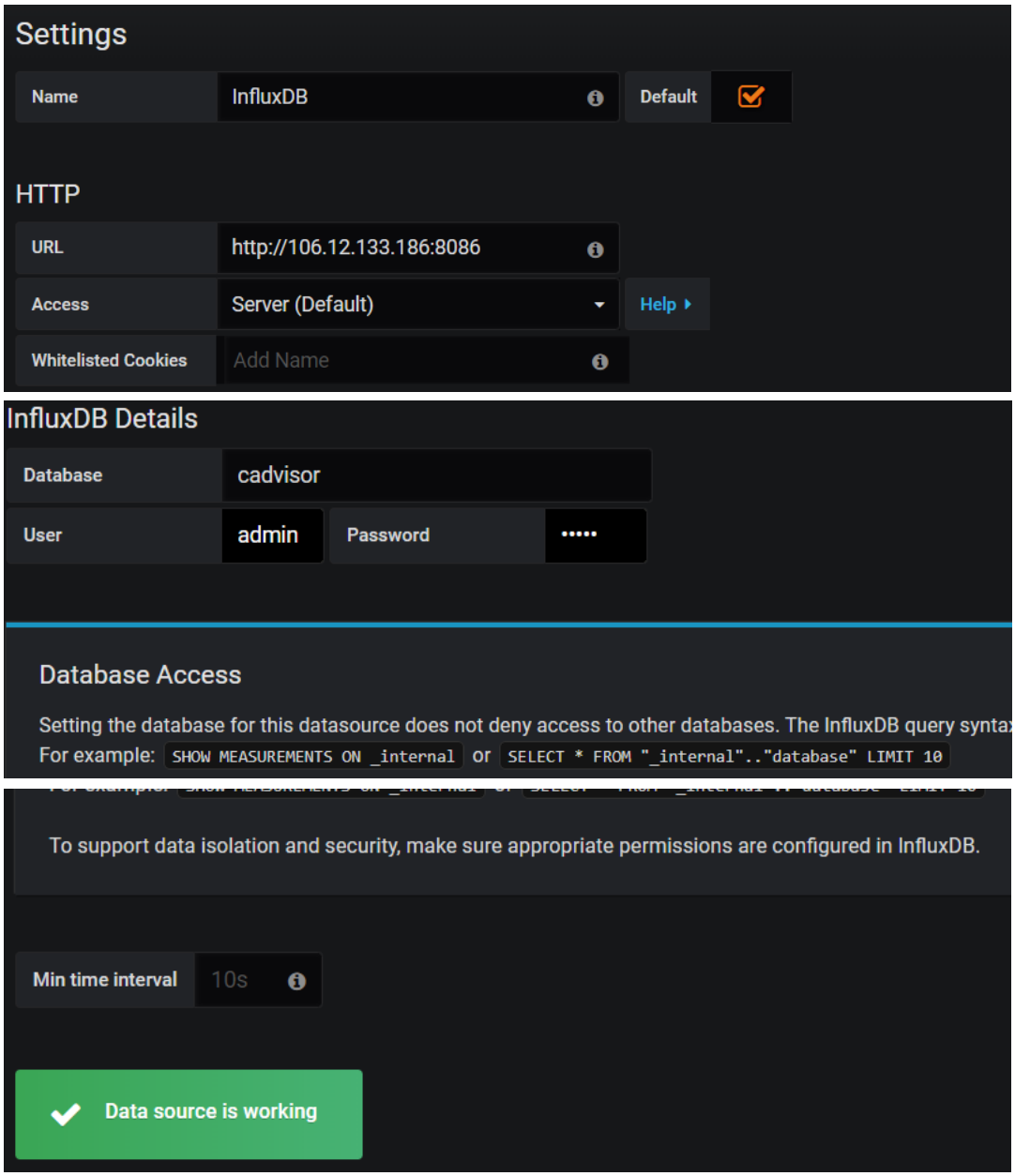
```
[root@www-jfedu-net sh]# docker-compose up -d
Creating network "sh_default" with the default driver
Pulling influxdb (tutum/influxdb:0.9)...
0.9: Pulling from tutum/influxdb
a3ed95caeb02: Pull complete
23efb549476f: Pull complete
aa2f8df21433: Pull complete
  072d3c9b41: Pull complete
▽9f371853f28: Pull complete
a248b0871c3c: Pull complete
749db6d368d0: Pull complete
09a7be9c234a: Pull complete
ce08d78abe37: Pull complete
```

7）Docker-compose 默认会启动三个类别容器，分别为：grafana、cadvisor、influxdb，对外访问 IP+端口如下：

- Grafana：http://106.12.133.186:3000/
- Cadvisor：http://106.12.133.186:8080/
- Influxdb：http://106.12.133.186:8086/

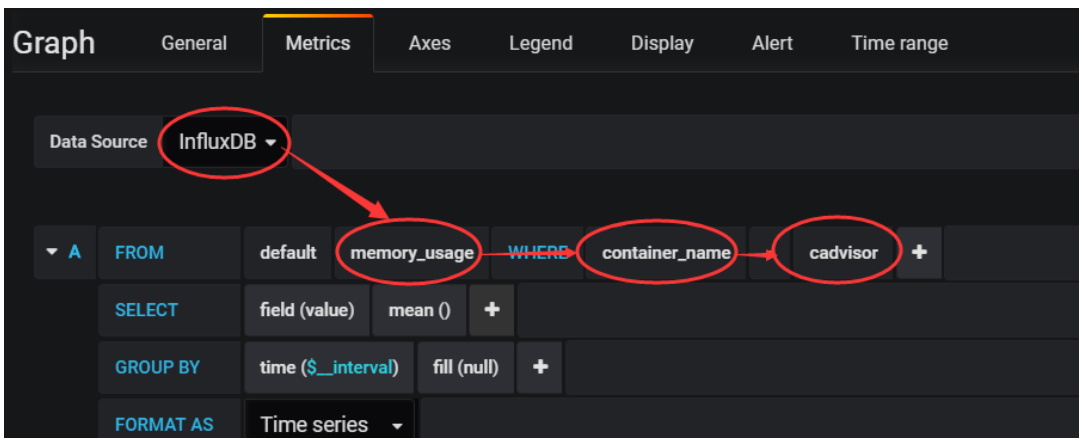
8）通过浏览器访问 Grafana WEB 界面，默认用户名和密码：admin/admin，然后选择 add-database source，填写 Influxdb 数据库的 IP 和端口，数据库名：cadvisor，用户名和密码：admin/admin，如图所示：

http://106.12.133.186:3000/login



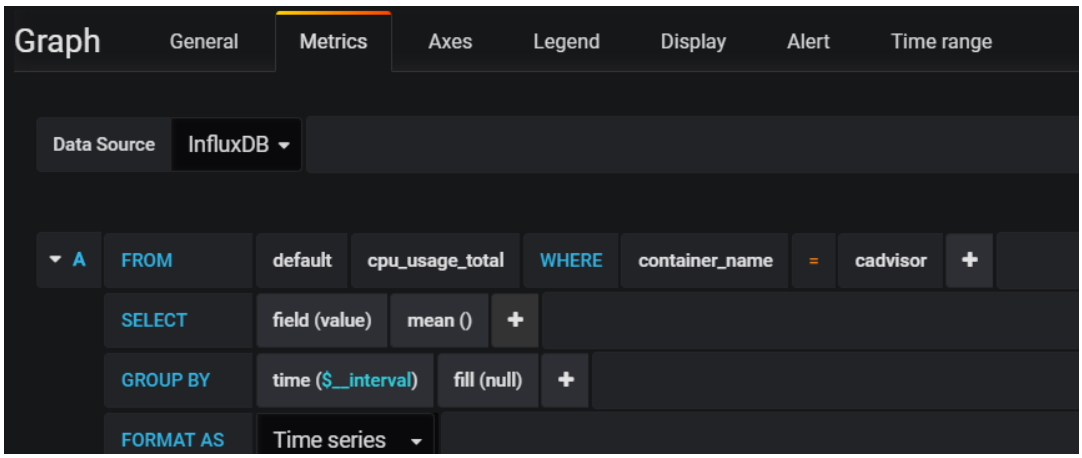
9）创建 Grafana 图像，设置监控项目，例如添加 MEM 内存使用监控，操作方法如下：





10）创建 Grafana 图像，设置监控项目，例如添加 CPU 使用监控，操作方法如下：





22.23 Docker 构建 Nginx 服务器实战

Nginx [engine x]是 Igor Sysoev 编写的一个 HTTP 和反向代理服务器 ,另外它也可以作为邮件代理服务器。 它已经在众多流量很大的俄罗斯网站上使用了很长时间，这些网站包括 Yandex、Mail.Ru、VKontakte，以及 Rambler。

据 Netcraft 统计 ,在 2012 年 8 月份 ,世界上最繁忙的网站中有 11.48%使用 Nginx 作为其服务器或者代理服务器。目前互联网主流公司 360、百度、新浪、腾讯、阿里等都在使用 nginx 作为自己的 WEB 服务器。

Nginx 由内核和模块组成，其中，内核的设计非常微小和简洁，完成的工作也非常

简单，仅仅通过查找配置文件将客户端请求映射到一个 location block（location 是 Nginx 配置中的一个指令，用于 URL 匹配），而在这个 location 中所配置的每个指令将会启动不同的模块去完成相应的工作。

Nginx 相对于 Apache 优点：

- 高并发响应性能非常好，官方 Nginx 处理静态文件并发 5w/s
- 反向代理性能非常好。（可用于负载均衡）
- 内存和 cpu 占用率低。（为 Apache 的 1/5-1/10）
- 功能较 Apache 少（常用功能均有）
- 对 php 可使用 cgi 方式和 fastcgi 方式。

Nginx WEB 安装

首先需要安装 pcre 库，然后再安装 Nginx：

#安装 pcre 支持 rewrite 库,也可以安装源码，注*安装源码时，指定 pcre 路径为解压源码的路径，而不是编译后的路径，否则会报错

（make[1]: *** [/usr/local/pcre/Makefile] Error 127 错误）

yum install pcre-devel pcre -y

#下载 Nginx 源码包

cd /usr/src ;wget -c http://nginx.org/download/nginx-1.4.2.tar.gz

#解压 Nginx 源码包

tar -xzf nginx-1.4.2.tar.gz

#进入解压目录，然后 sed 修改 Nginx 版本信息为 WS

cd nginx-1.4.2 ; sed -i -e 's/1.4.2//g' -e 's/nginx\//WS/g' -e

's/"NGINX"/"WS"/g' src/core/nginx.h

#预编译 Nginx

useradd www ;./configure --user=www --group=www --prefix=/usr/local/nginx

--with-

http_stub_status_module --with-http_ssl_module

#.configure 预编译成功后，执行 make 命令进行编译

make

#make 执行成功后，执行 make install 正式安装

make install

#自此 Nginx 安装完毕

/usr/local/nginx/sbin/nginx -t 检查 nginx 配置文件是否正确，返回 OK 即正确。

[root@localhost ~]# /usr/local/nginx/sbin/nginx -t

nginx: the configuration file /usr/local/nginx/conf/nginx.conf syntax is ok

nginx: configuration file /usr/local/nginx/conf/nginx.conf test is successful

[root@localhost ~]#

然后启动 nginx，/usr/local/nginx/sbin/nginx 回车即可。查看进程是否已启动：

[root@localhost ~]# ps -ef |grep nginx

nobody 5381 30285 0 May16 ? 00:04:31 nginx: worker process

root 30285 1 0 2014 ? 00:00:00 nginx: master process /usr/local/nginx/sbin/nginx

root 32260 32220 0 12:34 pts/0 00:00:00 grep nginx

[root@localhost ~]#

然后我们可以通过 docker 的 ip 去访问即可。

22.24 Docker 构建 LNAMP 架构实战

LNAMP(Linux+Nginx+Apache+Mysql+PHP)架构受到很多 IT 企业的青睐,取代了原来认为很好的 LNMP(Linux+Nginx+Mysql+PHP)架构,那我们说 LNAMP 到底有什么优点呢,还得从 Nginx 和 apache 的优缺点说起。

Nginx 处理静态文件能力很强,Apache 处理动态文件很强而且很稳定,把二者综合在一块,性能提升很多倍。可能很多 Linux SA 在从事 LNMP 运维中，会发现 PHP（fastcgi）模式会出现一些 502 错误的现象，这是因为 Nginx+PHP（fastcgi）组合不稳定的原因造成的。

YUM 安装 LAMP 方式：

yum install httpd httpd-tools mysql mysql-server mysql-devel php php-devel php-mysql -y

源码安装 LNAMP 之 Nginx

yum install prce-devel -y ;cd /usr/src ;wget http://nginx.org/download/nginx-1.6.0.tar.gz ;cd nginx-1.6.0 ;./configure -prefix=/usr/local/nginx && make &&make install

源码安装 LNAMP 之 Apache

yum install apr-devel apr-util-devel –y ;

cd /usr/src ; wget http://mirror.bit.edu.cn/apache/httpd/httpd-2.2.27.tar.gz ;tar xzf httpd-2.2.27.tar.gz ;cd httpd-2.2.27 ;./configure --prefix=/usr/local/apache --enable-so --enable-rewrite &&make &&make install

源码安装 LNAMP 之 MySQL

cd /usr/src ;wget http://downloads.mysql.com/archives/mysql-5.1/mysql-5.1.63.tar.gz ;tar xzf mysql-5.1.63.tar.gz ;cd mysql-5.1.63 ;./configure --prefix=/usr/local/mysql --enable-assembler &&make &&make install

```
make[3]: Nothing to be done for `install-data-am'.
make[3]: Leaving directory `/usr/src/mysql-5.1.63/server-tools'
make[2]: Leaving directory `/usr/src/mysql-5.1.63/server-tools'
Making install in instance-manager
make[2]: Entering directory `/usr/src/mysql-5.1.63/server-tools/instance-manager'
make[3]: Entering directory `/usr/src/mysql-5.1.63/server-tools/instance-manager'
test -z "/usr/local/mysql/libexec" || /bin/mkdir -p "/usr/local/mysql/libexec"
  /bin/sh ../../libtool --preserve-dup-deps   --mode=install /usr/bin/install -c 'mysqlmanager' '/usr/local/m
r'
libtool: install: /usr/bin/install -c mysqlmanager /usr/local/mysql/libexec/mysqlmanager
make[3]: Nothing to be done for `install-data-am'.
make[3]: Leaving directory `/usr/src/mysql-5.1.63/server-tools/instance-manager'
make[2]: Leaving directory `/usr/src/mysql-5.1.63/server-tools/instance-manager'
make[1]: Leaving directory `/usr/src/mysql-5.1.63/server-tools'
Making install in win
make[1]: Entering directory `/usr/src/mysql-5.1.63/win'
make[2]: Entering directory `/usr/src/mysql-5.1.63/win'
make[2]: Nothing to be done for `install-exec-am'.
make[2]: Nothing to be done for `install-data-am'.
make[2]: Leaving directory `/usr/src/mysql-5.1.63/win'
make[1]: Leaving directory `/usr/src/mysql-5.1.63/win'
[root@node2 mysql-5.1.63]#
[root@node2 mysql-5.1.63]# ./configure  --prefix=/usr/local/mysql --enable-assembler &&make &&make install
```

配置 Mysql 服务为系统服务：

cp /usr/local/mysql/share/mysql/my-medium.cnf /etc/my.cnf

cp /usr/local/mysql/share/mysql/mysql.server /etc/rc.d/init.d/mysqld

chkconfig --add mysqld

chkconfig --level 345 mysqld on

cd /usr/local/mysql

useradd mysql

chown -R mysql.mysql /usr/local/mysql

/usr/local/mysql/bin/mysql_install_db --user=mysql

chown -R mysql var

/usr/local/mysql/bin/mysqld_safe --user=mysql &

源码安装 LNAMP 之 PHP

cd /usr/src ;wget http://mirrors.sohu.com/php/php-5.3.28.tar.bz2 ;tar jxf php-5.3.28.tar.bz2 ;cd php-5.3.28 ;./configure --prefix=/usr/local/php5 --with-config-file-path=/usr/local/php/etc --with-apxs2=/usr/local/apache/bin/apxs --with-mysql=/usr/local/mysql/

```
Installing PHP CLI man page:      /usr/local/php5/man/man1/
Installing build environment:     /usr/local/php5/lib/php/build/
Installing header files:          /usr/local/php5/include/php/
Installing helper programs:       /usr/local/php5/bin/
  program: phpize
  program: php-config
Installing man pages:             /usr/local/php5/man/man1/
  page: phpize.1
  page: php-config.1
Installing PEAR environment:      /usr/local/php5/lib/php/
[PEAR] Archive_Tar    - installed: 1.3.11
[PEAR] Console_Getopt - installed: 1.3.1
warning: pear/PEAR requires package "pear/Structures_Graph" (recommended version 1.0.4)
warning: pear/PEAR requires package "pear/XML_Util" (recommended version 1.2.1)
[PEAR] PEAR           - installed: 1.9.4
Wrote PEAR system config file at: /usr/local/php5/etc/pear.conf
You may want to add: /usr/local/php5/lib/php to your php.ini include_path
[PEAR] Structures_Graph- installed: 1.0.4
[PEAR] XML_Util       - installed: 1.2.1
/usr/src/php-5.3.28/build/shtool install -c ext/phar/phar.phar /usr/local/php5/bin
ln -s -f /usr/local/php5/bin/phar.phar /usr/local/php5/bin/phar
Installing PDO headers:          /usr/local/php5/include/php/ext/pdo/
[root@node2 php-5.3.28]#
```

源码安装 Apache+PHP 整合

整合 apache+php 环境，修改 httpd.conf 配置文件，然后加入如下语句：

LoadModule　　php5_module modules/libphp5.so （默认已存在）

AddType　　application/x-httpd-php .php

DirectoryIndex index.php index.html (把 index.php 加入 index.html 之前)

然后在/usr/local/apache/htdocs 目录下创建 index.php 测试页面，执行如下命令：

cat >>/usr/local/apache/htdocs/index.php <<EOF

<?php

phpinfo();

?>

EOF

重新启动 apache 服务，通过 IP 访问界面如下图，即代表 LAMP 环境搭建成功。



System	Linux node2 2.6.18-308.el5 #1 SMP Tue Feb 21 20:06:06 EST 2012 x86_64
Build Date	May 26 2014 14:34:36
Configure Command	'./configure' '--prefix=/usr/local/php5' '--with-config-file-path=/usr/local/php/etc' '--with-apxs2=/usr/local/apache/bin/apxs' '--with-mysql=/usr/local/mysql/'
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/php/etc
Loaded Configuration File	(none)
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20090626
PHP Extension	20090626
Zend Extension	220090626

源码安装 DISCUZ 论坛

下载 discuz 源码包文件，然后解压：

cd /usr/src ;wget http://download.comsenz.com/DiscuzX/3.1/Discuz_X3.1_SC_UTF8.zip

解压 discuz 程序包：unzip Discuz_X3.1_SC_UTF8.zip -d /usr/local/apache/htdocs/

重命名程序文件：cd /usr/local/apache/htdocs/ ;mv upload/* .

赋予 discuz 目录完全访问权限：cd /usr/local/apache/htdocs/ ;chmod 777 -R data/ uc_server/ config/ uc_client/

然后访问 IP 安装 discuz 论坛，如下图，选择“我同意”



进入如下界面，数据库安装，如果不存在则需要新建数据库并授权。



数据库创建及授权命令如下：

create database discuz charset=utf8;

grant all on discuz.* to root@'localhost' identified by "123456";

```
[root@node2 ~]# /usr/local/mysql/bin/mysql -uroot -p123456
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 5.1.63-log Source distribution

Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database discuz charset=utf8;
Query OK, 1 row affected (0.00 sec)

mysql> grant all on discuz.* to root@'localhost' identified by "123456";
Query OK, 0 rows affected (0.00 sec)

mysql>
```

点击下一步，直至安装完成，进入等待已久的论坛画面：



自此 LAMP 环境整合并搭建成功，那如何使用 Nginx 来整合 LAMP 呢？

22.25 Docker 自动化部署二

生产环境中，由于 Docker 指令过多，操作比较繁琐，可以编写 SHELL 自动安装并配置 Docker 虚拟化及桥接网络，同时使用 pipework 脚本来配置容器 IP，能够实现容器的批量管理，此脚本适用于 CentOS7.x 系统；

CentOS7.x 系列 Docker 批量管理脚本代码如下：

```
#!/bin/bash
#auto install docker and Create VM
#by jfedu.net 2022
#Define PATH Varablies
IPADDR=`ifconfig|grep -E "\<inet\>"|awk '{print $2}'|grep "192.168"|head -1`
GATEWAY=`route -n|grep "UG"|awk '{print $2}'|grep "192.168"|head -1`
IPADDR_NET=`ifconfig|grep -E "\<inet\>"|awk '{print $2}'|grep "192.168"|head -1|awk -F. '{print $1"."$2"."$3"."}'`
LIST="/root/docker_vmlist.csv"
if [ ! -f /usr/sbin/ifconfig ];then
    yum install net-tools* -y
fi
for i in `seq 1 253`;do ping -c 1 ${IPADDR_NET}${i} ;[ $? -ne 0 ]&&
```

```
DOCKER_IPADDR="${IPADDR_NET}${i}" &&break;done >>/dev/null 2>&1

echo "###################"

echo -e "Dynamic get docker IP,The Docker IP address\n\n$DOCKER_IPADDR"

NETWORK=(

    HWADDR=`ifconfig eth0|grep ether|awk '{print $2}'`

    IPADDR=`ifconfig eth0|grep -E "\<inet\>"|awk '{print $2}'`

    NETMASK=`ifconfig eth0|grep -E "\<inet\>"|awk '{print $4}'`

    GATEWAY=`route -n|grep "UG"|awk '{print $2}'`

)

if [ -z "$1" -o -z "$2" ];then



    echo -e "\033[32m--------------------------------\033[0m"

    echo -e "\033[32mPlease exec $0 CPU(C) MEM(G),example $0 4 8\033[0m"

    exit 0

fi

#CPU=`expr $2 - 1`

if [ ! -e /usr/bin/bc ];then

    yum install bc -y >>/dev/null 2>&1

fi
```

```
CPU_ALL=`cat /proc/cpuinfo |grep processor|wc -l`
if [ ! -f $LIST ];then
    CPU_COUNT=$1
    CPU_1="0"
    CPU1=`expr $CPU_1 + 0`
    CPU2=`expr $CPU1 + $CPU_COUNT - 1`
    if [ $CPU2 -gt $CPU_ALL ];then
        echo -e "\033[32mThe System CPU count is $CPU_ALL,not more than it.\033[0m"
        exit
    fi
else
    CPU_COUNT=$1
    CPU_1=`cat $LIST|tail -1|awk -F"," '{print $4}'|awk -F"-" '{print $2}'`
    CPU1=`expr $CPU_1 + 1`
    CPU2=`expr $CPU1 + $CPU_COUNT - 1`
    if [ $CPU2 -gt $CPU_ALL ];then
        echo -e "\033[32mThe System CPU count is $CPU_ALL,not more than it.\033[0m"
```

```
            exit
        fi
fi
MEM_F=`echo $2 \* 1024|bc`
MEM=`printf "%.0f\n" $MEM_F`
DISK=20
USER=$3
REMARK=$4
ping $DOCKER_IPADDR -c 1 >>/dev/null 2>&1


if [ $? -eq 0 ];then


        echo -e "\033[32m--------------------------------\033[0m"
        echo -e "\033[32mThe IP address to be used,Please change other
IP,exit.\033[0m"
        exit 0
fi


if [ ! -e /usr/bin/docker ];then
```

```
    yum install docker* device-mapper* lxc   -y
    mkdir -p /export/docker/
    cd /var/lib/ ;rm -rf docker ;ln -s /export/docker/ .
    mkdir -p /var/lib/docker/devicemapper/devicemapper
    dd if=/dev/zero of=/var/lib/docker/devicemapper/devicemapper/data bs=1G
count=0 seek=2000
    service docker start
    if [ $? -ne 0 ];then
        echo "Docker install error ,please check."
        exit
    fi
fi


cd  /etc/sysconfig/network-scripts/
    mkdir -p /data/backup/`date +%Y%m%d-%H%M`
    yes|cp ifcfg-eth* /data/backup/`date +%Y%m%d-%H%M`/
if
    [ -e /etc/sysconfig/network-scripts/ifcfg-br0 ];then
    echo
```

```
else
    cat >ifcfg-eth0<<EOF
    DEVICE=eth0
    BOOTPROTO=none
    ${NETWORK[0]}
    NM_CONTROLLED=no
    ONBOOT=yes
    TYPE=Ethernet
    BRIDGE="br0"
    ${NETWORK[1]}
    ${NETWORK[2]}
    ${NETWORK[3]}
    USERCTL=no
EOF
    cat >ifcfg-br0 <<EOF
    DEVICE="br0"
    BOOTPROTO=none
    ${NETWORK[0]}
    IPV6INIT=no
```

```
    NM_CONTROLLED=no
    ONBOOT=yes
    TYPE="Bridge"
    ${NETWORK[1]}
    ${NETWORK[2]}
    ${NETWORK[3]}
    USERCTL=no
EOF
    /etc/init.d/network restart

fi
echo 'Your can restart Ethernet Service: /etc/init.d/network restart !'
echo '----------------------------------------------------------'

cd -
#######create docker container
service docker status >>/dev/null
if [ $? -ne 0 ];then
    service docker restart
```

```
fi


NAME="Docker_`echo $DOCKER_IPADDR|awk -F"." '{print $(NF-1)"_"$NF}'`"
IMAGES=`docker images|grep -v "REPOSITORY"|grep -v "none"|grep "centos"|head
-1|awk '{print $1}'`
if [ -z $IMAGES ];then
    echo "Plesae Download Docker Centos Images,you can to be use docker search
centos,and docker pull centos6.5-ssh,exit 0"
    if [ ! -f jfedu_centos68.tar ];then
        echo "Please upload jfedu_centos68.tar for docker server."
        exit
    fi
    cat jfedu_centos68.tar|docker import - jfedu_centos6.8
fi
IMAGES=`docker images|grep -v "REPOSITORY"|grep -v "none"|grep "centos"|head
-1|awk '{print $1}'`
CID=$(docker run -itd --privileged --cpuset-cpus=${CPU1}-${CPU2} -m ${MEM}m -
-net=none --name=$NAME $IMAGES /bin/bash)
echo $CID
```

```
docker ps -a |grep "$NAME"

pipework br0 $NAME    $DOCKER_IPADDR/24@$IPADDR

docker exec $NAME /etc/init.d/sshd start

if [ ! -e $LIST ];then

      echo "编号,容器 ID,容器名称,CPU,内存,硬盘,容器 IP,宿主机 IP,使用人,备注" >$LIST

fi

####################

NUM=`cat $LIST |grep -v CPU|tail -1|awk -F, '{print $1}'`

if [[ $NUM -eq "" ]];then

            NUM="1"

else

            NUM=`expr $NUM + 1`

fi

###################

echo -e "\033[32mCreate virtual client Successfully.\n$NUM `echo $CID|cut -b 1-

12`,$NAME,$CPU1-

$CPU2,${MEM}M,${DISK}G,$DOCKER_IPADDR,$IPADDR,$USER,$REMARK\033[0m"

if [ -z $USER ];then

      USER="NULL"
```

```
    REMARK="NULL"
fi
echo $NUM,`echo $CID|cut -b 1-12`,$NAME,$CPU1-$CPU2,${MEM}M,${DISK}G,$DOCKER_IPADDR,$IPADDR,$USER,$REMARK >>$LIST
rm -rf /root/docker_vmlist_*
iconv -c -f utf-8 -t gb2312 $LIST  -o /root/docker_vmlist_`date +%H%M`.csv
```